



**Desenvolva aplicações web
profissionais com uso dos poderosos
recursos de estilização das CSS3**

CSS3

**DESENVOLVA APLICAÇÕES WEB
PROFISSIONAIS COM USO DOS PODEROSOS
RECURSOS DE ESTILIZAÇÃO DAS CSS3**

Maurício Samy Silva

Novatec

São Paulo | 2019

Copyright © 2012 da Novatec Editora Ltda.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998.

É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates

Revisão gramatical: Marta Almeida de Sá Editoração eletrônica: Carolina Kuwabata Capa: Victor Bittow/Carolina Kuwabata Ilustração da capa: Roberto Coelho

ISBN: 978-85-7522-797-8

Histórico de edições impressas:

Novembro/2014 Terceira reimpressão

Outubro/2013 Segunda reimpressão

Setembro/2012 Primeira reimpressão Dezembro/2011 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil Tel.: +55 11 2959-6529

E-mail: novatec@novatec.com.br

Site: novatec.com.br

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

Dedico este livro a minha neta Natália e aos seus pais Daniela e Rodrigo.

Sumário

[Agradecimentos](#)

[Isenção de responsabilidade](#)

[Sobre este livro](#)

[Sobre o autor](#)

[Introdução](#)

[Para quem foi escrito este livro](#)

[Alerta!](#)

[Convenções tipográficas](#)

[Site do livro](#)

[Capítulo 1 ■ Fundamentos CSS](#)

[1.1 Definições e conceitos CSS](#)

[1.1.1 Definição](#)

[1.1.2 Finalidade](#)

[1.1.3 Histórico](#)

[1.1.4 Regra CSS](#)

[1.1.5 Tipos de componentes da regra CSS](#)

[1.1.5.5 Valores definidos em outra especificação](#)

[1.1.6 Tipos de valores CSS](#)

[1.2 Vinculando folhas de estilo a documentos](#)

[1.2.1 Estilos inline](#)

[1.2.2 Estilos incorporados](#)

[1.2.3 Estilos externos](#)

[1.3 Módulos CSS3](#)

[1.4 Status dos módulos](#)

[Capítulo 2 ■ Módulo CSS Basic Box Model](#)

[2.1 Box Model](#)

[2.1.1 Bloco de conteúdo](#)

[2.1.2 Box nível de bloco anônimo](#)

[2.1.3 Box inline anônimo](#)

[2.1.4 Diagrama do Box Model](#)

[Capítulo 3 ■ Posicionamento CSS](#)

[3.1 Esquemas de posicionamento CSS2.1](#)

[3.1.1 Esquema normal](#)

[3.1.2 Esquema float](#)

[3.1.3 Esquema absoluto](#)

[3.2 Posicionamento com z-index](#)

[3.2.1 Contexto para empilhamento](#)

[Capítulo 4 ■ Seletores, cascata e herança](#)

[4.1 Introdução](#)

[4.2 Definições](#)

[4.2.1 Seletor](#)

[4.2.2 Árvore do documento](#)

[4.2.3 Elemento-filho e elemento-pai](#)

[4.2.4 Elemento descendente e elemento ancestral](#)

[4.2.5 Elemento-irmão](#)

[4.2.6 Elemento precedente](#)

[4.2.7 Elemento subsequente](#)

[4.2.8 Sintaxe para seletor](#)

[4.2.9 Seletor universal](#)

[4.2.10 Seletor tipo](#)

[4.2.11 Seletor de atributo](#)

[4.2.12 Seletor classe](#)

[4.2.13 Seletor ID](#)

[4.2.14 Seletor pseudoclasse](#)

[4.2.15 Seletor pseudoelemento](#)

[4.2.16 Seletor simples](#)

[4.2.17 Grupamento de seletores](#)

[4.2.18 Caracteres de combinação](#)

[4.2.19 Sintaxe geral de um seletor](#)

[4.2.20 Seletores por namespace](#)

[4.2.21 Especificidade](#)

[4.2.22 Herança CSS](#)

[4.2.23 Efeito cascata](#)

[4.2.24 Ordem de cascata](#)

[4.2.25 Diretiva !important](#)

[Capítulo 5 ■ Background](#)

[5.1 Introdução](#)

[5.2 Propriedades background](#)

[5.2.1 background-color](#)

[5.2.2 background-image](#)

[5.2.3 background-repeat](#)

[5.2.4 background-attachment](#)

[5.2.5 background-position](#)

[5.2.6 background-clip](#)

[5.2.7 background-origin](#)

[5.2.8 background-size](#)

[5.2.9 Propriedade background](#)

[5.3 Múltiplos backgrounds](#)

[Capítulo 6 ■ Bordas e sombras](#)

[6.1 Introdução](#)

[6.2 Propriedade border](#)

[6.2.1 border-width](#)

[6.2.2 border-style](#)

[6.2.3 border-color](#)

[6.2.4 border](#)

[6.2.5 border-radius](#)

[6.2.6 border-image](#)

[6.3 Box Shadow](#)

[6.4 Text Shadow](#)

[Capítulo 7 ■ Cores e opacidade](#)

[7.1 Introdução](#)

[7.2 Propriedade color](#)

[7.2.1 hexadecimal e RGB](#)

[7.2.2 transparent](#)

[7.2.3 Palavra-chave](#)

[7.2.4 currentColor](#)

[7.2.5 HSL](#)

[7.2.6 HSLA](#)

[7.2.7 RGBA](#)

[7.2.8 Cores do SO](#)

[7.3 Propriedade opacity](#)

[Capítulo 8 ■ Fontes](#)

[8.1 Introdução](#)

[8.1.1 Tipografia](#)

[8.1.2 Fontes seguras](#)

[8.2 Propriedade font-family](#)

[8.2.1 Família serif](#)

[8.2.2 Família sans-serif](#)

[8.2.3 Família cursive](#)

[8.2.4 Família fantasy](#)

[8.2.5 Família monospace](#)

[8.3 Propriedade font-style](#)

[8.4 Propriedade font-variant](#)

[8.5 Propriedade font-weight](#)

[8.6 Propriedade font-size](#)

[8.7 Propriedade font-stretch](#)

[8.8 Propriedade font-size-adjust](#)

[8.9 Propriedade font](#)

[8.10 @font-face](#)

[8.10.1 Obtendo fontes](#)

[8.10.2 API fontes do Google](#)

[Capítulo 9 ■ Layout multicolunas](#)

[9.1 Introdução](#)

[9.2 Modelo multicolunas](#)

[9.3 Propriedades CSS](#)

[9.3.1 column-count](#)

[9.3.2 column-width](#)

[9.3.3 columns](#)

[9.3.4 column-gap](#)

[9.3.5 column-rule](#)

[9.3.6 column-span](#)

[9.3.7 column-fill](#)

[9.3.8 break-*](#)

[9.3.9 Overflow](#)

[Capítulo 10 ■ Media Queries](#)

[10.1 Introdução](#)

[10.2 Anatomia da media query](#)

[10.3 Funcionalidades de mídia](#)

[10.4 Exemplo](#)

[10.4.1 Primeiro passo](#)

[10.4.2 Segundo passo](#)

[10.4.3 Terceiro passo](#)

[10.4.4 Breakpoint](#)

[10.5 Suporte](#)

Capítulo 11 ■ Gradientes

[11.1 Introdução](#)

[11.2 Gradiente linear](#)

[11.2.1 Parâmetros da função gradiente](#)

[11.3 Gradiente radial](#)

[11.5 Repetição de gradientes](#)

Capítulo 12 ■ Transformações 2D

[12.1 Introdução](#)

[12.2 Sistema de coordenadas](#)

[12.3 Transformações](#)

[12.3.1 rotate\(\)](#)

[12.3.2 translateX\(\)](#)

[12.3.3 translateY\(\)](#)

[12.3.4 translate\(\)](#)

[12.3.5 scaleX\(\)](#)

[12.3.6 scaleY\(\)](#)

[12.3.7 scale\(\)](#)

[12.3.8 skewX\(\)](#)

[12.3.9 skewY\(\)](#)

[12.3.10 skew\(\)](#)

[12.3.11 matrix\(\)](#)

[12.4 Múltiplas transformações](#)

[12.5 Mudança da origem da transformação](#)

[12.6 Exemplo](#)

Capítulo 13 ■ Transformações 3D

[13.1 Introdução](#)

[13.2 Propriedades](#)

[13.2.1 transform-style](#)

[13.2.2 perspective](#)

[13.2.3 perspective-origin](#)

[13.2.4 backface-visibility](#)

[13.3 Funções](#)

[13.3.1 perspective\(n\)](#)

[13.3.2 rotate\(\)](#)

[13.3.3 translate\(\)](#)

[13.3.4 scale\(\)](#)

[13.3.5 skew\(\)](#)

[13.3.6 matrix\(\)](#)

Capítulo 14 ■ Transições CSS

[14.1 Introdução](#)

[14.2 Propriedades](#)

[14.2.1 transition-property](#)

[14.2.2 transition-duration](#)

[14.2.3 transition-timing-function](#)

[14.2.4 transition-delay](#)

[14.2.5 transition](#)

[14.3 Exemplo](#)

[Capítulo 15 ■ Animações CSS](#)

[15.1 Introdução](#)

[15.2 Propriedades](#)

[15.2.1 animation-name](#)

[15.2.2 animation-duration](#)

[15.2.3 animation-timing-function](#)

[15.2.4 animation-iteration-count](#)

[15.2.5 animation-direction](#)

[15.2.6 animation-play-state](#)

[15.2.7 animation-delay](#)

[15.2.8 animation](#)

[15.3 Exemplo](#)

[Capítulo 16 ■ Template Layout](#)

[16.1 Introdução](#)

[16.2 Visão geral](#)

[16.2.1 Plugin CSS Template Layout](#)

[16.2.2 Regra CSS para descrição do layout](#)

[16.3 Propriedades](#)

[16.3.1 display](#)

[16.3.2 position](#)

[16.3.3 ::slot\(\)](#)

[16.4 Exemplos](#)

[16.4.1 Exemplo 1 – Formulários](#)

[16.4.2 Exemplo 2 – Templates aninhados](#)

[16.4.3 Exemplo 3 – flutuando boxes](#)

[Capítulo 17 ■ Box Layout flexível](#)

[17.1 Introdução](#)

[17.2 Propriedades](#)

[17.2.1 display](#)

[17.2.2 flex-direction](#)

[17.2.3 flex-wrap](#)

[17.2.4 flex-flow](#)

[17.2.5 order](#)

[17.2.6 flex](#)

[17.2.7 justify-content](#)

[17.2.8 align-items](#)

[17.2.9 align-self](#)

[17.2.10 align-content](#)

[Apêndice A ■ Propriedades CSS](#)

[A.1 Propriedades das CSS3](#)

[A.1.1 Convenções](#)

[Apêndice B ■ Elementos da HTML5](#)

[Apêndice C ■ Atributos da HTML5](#)

[Apêndice D ■ Cores CSS](#)

[Referências bibliográficas](#)

Agradecimentos

Agradeço a Deus por ter me dado forças, disposição e motivação para escrever este livro.

Sou grato aos profissionais da Novatec Editora, em particular ao editor Rubens Prates, que, ao longo de todo o processo de criação, esteve presente, guiando-me com suas dicas e informações sobre as particularidades e implicações editoriais próprias à fase de criação de um livro.

Meu maior agradecimento é a você, leitor, por interessar-se em aprender CSS3 e honrar-me com a leitura deste livro.

Isenção de responsabilidade

Todos os esforços foram feitos para assegurar o fornecimento de informações as mais precisas, completas e exatas possíveis neste livro. Contudo, tais informações são fornecidas “como estão” e sem nenhuma garantia, seja expressa, seja implícita. O autor, a editora, os distribuidores e qualquer entidade envolvida direta ou indiretamente na sua comercialização não assumirão responsabilidade alguma por eventual prejuízo ou dano, direto ou indireto, consequente aos dados contidos neste livro.

Sobre este livro

Os diversos módulos para as especificações CSS3 encontram-se em diferentes estágios de desenvolvimento. Alguns módulos já são estáveis, enquanto outros nem sequer saíram da fase inicial de Rascunho de Trabalho no W3C. Outros tantos se distribuem entre esses dois status. Algumas funcionalidades já são suportadas nativamente pelos navegadores, outras com prefixos proprietários e outras ainda sem nenhum suporte.

Este livro foi escrito com a finalidade de apresentar, estudar e exemplificar as funcionalidades previstas nas especificações das CSS3 que já podem ser testadas com ou sem uso de prefixos proprietários. Não são contempladas as funcionalidades sem nenhum suporte.

Simplesmente apresentar, estudar e exemplificar as funcionalidades pressupõe que o leitor já tenha um sólido conhecimento da sintaxe CSS e de suas propriedades CSS2.1, além de conhecimento da construção de layouts e posicionamento CSS. Se assim fosse, o livro não teria nenhum valor para iniciantes ou mesmo leitores com conhecimentos intermediários das técnicas CSS.

Por essa razão, os primeiros capítulos deste livro são dedicados à apresentação, ao estudo e à exemplificação dos fundamentos das CSS, sua sintaxe, suas propriedades e seus esquemas de posicionamento, de acordo com os conceitos das CSS2.1. Tais capítulos visam a fornecer uma base mínima aos leitores não familiarizados com as CSS, para que possam tirar máximo proveito do livro.

Sobre o autor

Maurício Samy Silva é graduado em engenharia civil pelo Instituto Militar de Engenharia (IME). Fundador e ex-diretor-presidente da Planep Engenharia, exerceu o magistério paralelamente à engenharia, tendo sido, ao longo de 25 anos, professor de geometria descritiva e matemática. Seu primeiro contato com programação para computadores deu-se ainda na graduação, ao aprender a linguagem Fortran. Àquela época, era comum passar noites trabalhando em uma máquina perfuradora de cartões que seriam processados em um então poderoso IBM/360. Em 1982, adquiriu seu primeiro computador pessoal, um TK-80 com processador Z-80A de 3,25 MHz, teclado de membrana com 40 teclas e memória RAM de 1 kB expansível até 16 kB. Trabalhou com os modelos TK-82, TK-85, TK-2000, DGT-100 e assim por diante, até os dias atuais.

Sua experiência com desenvolvimento de sites teve início em 1999, com o FrontPage, considerado uma ferramenta sensacional para a construção de sites. Em 2002, por acaso, teve seu primeiro contato com o site do W3C e, como ficou vivamente impressionado com a proposta desse consórcio, começou a pesquisar e a estudar as Web Standards, tendo como fonte de consulta o material publicado na internet em língua inglesa. Ao contrário do que ocorre nos dias atuais, em que vários desenvolvedores escrevem em blogs pessoais sobre Web Standards, a literatura a respeito do assunto em português era, então, praticamente nula, mas, mesmo assim, quando encontrada, limitava-se ao básico do básico.

No segundo semestre de 2003, estimulado pela completa falta de material de consulta gratuita na internet e impulsionado por sua veia de educador, desenvolvida durante anos em sala de aula, decidiu lançar o site CSS para web design, o qual é nacionalmente conhecido como o site Maujor, hospedado em <http://www.maujor.com/>. A proposta inicial do site era divulgar a Cascading Style Sheet (CSS), ou folha de estilo em cascata, com base no compartilhamento de suas experiências com essa técnica. Com a pronta aceitação e o sucesso crescente do site, o objetivo inicial tornou-se mais ambicioso e passou a ser a divulgação das Web Standards.

No início de 2006, com a popularização e a adesão maciça aos blogs, criou o Blog do Maujor, hospedado em <http://www.maujor.com/blog/>, com propósito semelhante ao do site, mas com uma dinâmica mais ativa e a efetiva participação de seus leitores.

Tanto o site como o blog constituem-se em referência nacional para Web Standards e, sem dúvida, foram e continuam sendo grandes responsáveis pela

divulgação e popularização das Web Standards, assim como, em particular, das CSS no Brasil.

Maujor, como é conhecido o autor, é ainda um ativo frequentador de fóruns, escreve para vários portais brasileiros voltados a desenvolvedores web, é autor de artigos em inglês e de trabalhos relacionados às CSS publicados em sites internacionais sobre Web Standards. Ocupa o segundo lugar na lista de tradutores mundiais de documentos do W3C por quantidade de documentos versados. Publicou em seu site mais de 40 traduções de artigos sobre Web Standards escritos por consagrados autores internacionais. Traduziu para o português as ferramentas para testes de acessibilidade em sites, denominadas analisador de contraste de cores e barra de ferramentas para acessibilidade na web, ambas em parceria com o WAT-C, um consórcio australiano voltado ao desenvolvimento de ferramentas destinadas a testes de acessibilidade.

Maujor é autor dos seguintes livros: *Construindo sites com CSS e (X)HTML*; *Criando sites com HTML*; *jQuery*; *AJAX com jQuery*; *JavaScript*; *HTML5*; *jQuery Mobile*; *jQuery UI*; *Fundamentos da SVG*; *Web Design Responsivo* e *Foundation for Sites*; todos publicados pela Novatec Editora..

Introdução

Este livro tem o objetivo de fornecer aos profissionais envolvidos no desenvolvimento para a web uma visão detalhada dos módulos das especificações para as CSS3, cujas funcionalidades estão implementadas nos navegadores modernos, ainda que com uso de prefixos proprietários. Trata-se de um livro pioneiro em língua portuguesa que mostra e exemplifica muitas das funcionalidades das CSS3.

O livro é composto de 17 capítulos e quatro apêndices. Os primeiros capítulos apresentam os fundamentos, a sintaxe geral e as técnicas das CSS. Os demais capítulos tratam das funcionalidades das CSS3 capazes de ser testadas em pelo menos um dos navegadores modernos. A teoria é complementada com amplo apoio de imagens e capturas de tela ilustrando os exemplos e exercícios desenvolvidos. No site de apoio ao livro, em <http://livrocss3.com.br>, o leitor encontra todos os códigos mostrados no livro.

Desejo a todos os meus leitores uma ótima leitura e faço votos de que o livro seja muito útil para vocês.

Para quem foi escrito este livro

Este livro foi escrito para aquelas pessoas envolvidas com a criação de websites, tanto na área de design quanto na de programação, que desejam adiantar-se nos estudos de várias das fantásticas funcionalidades das CSS3, embora muitas delas ainda não possam ser usadas em fase de produção.

Os conceitos e o entendimento das técnicas e dos mecanismos das CSS3 em um futuro próximo serão indispensáveis como ferramentas para a criação da arte para sites. Profissionais da área, ao iniciarem seus experimentos com as técnicas aqui descritas, estarão atualizando seus conhecimentos com as mais modernas e revolucionárias técnicas de desenvolver CSS. Com CSS, não há limites para sua imaginação criadora.

Autores HTML com larga experiência em desenvolvimento de layouts baseados nas técnicas preconizadas pelas versões anteriores das CSS precisam rever desde já seus conceitos de criação e ficar em condições de aplicar as funcionalidades das CSS3 muito em breve. Este livro fornece a esses autores uma visão diferente das CSS, visão que complementa e estende os mecanismos atuais de estilização.

Este livro é recomendado àqueles que já possuem um conhecimento básico da sintaxe CSS2. Não se intimidem com qualquer conceito ou terminologia que lhes seja completamente desconhecida nos primeiros capítulos. Com a sequência de leitura, as dúvidas tendem a desaparecer naturalmente. Aprender o certo para depois conhecer o errado, tendo condições de corrigir, é a via mais apropriada para o aprendizado – e essa é a via que este livro traça para aprendizes.

Alerta!

Nenhum dos módulos das CSS3 encontra-se totalmente implementado nos navegadores atuais. Os mais populares navegadores atuais, à época em que este livro foi reimpresso, são: Internet Explorer 10, Firefox 23.0, Opera 16, Safari 6 e Chrome 29.

Muitos dos exemplos mostrados neste livro foram desenvolvidos com uso de funcionalidades que requerem prefixos proprietários, tais como `-moz-`, `-webkit-`, `-o-` e `-ms-` para Mozilla, WebKit, Opera e Microsoft. Por outro lado, muitas funcionalidades das CSS3 já são suportadas nativamente por um ou mais de um navegador atual.

Certamente, quando você estiver lendo este livro, alguns ou todos os navegadores listados já terão sido atualizados para versões mais avançadas. Contudo, durante algum tempo ainda, poucas das novas funcionalidades serão suportadas, quer nativamente, quer com uso de prefixos proprietários.

Assim, alerto os meus leitores para a necessidade imperativa de, após estudar e entender os exemplos do livro e tentar realizar uma experiência prática com qualquer funcionalidade, consultar um dos inúmeros sites que apresentam tabelas ou listas de suporte das funcionalidades CSS3 por navegadores não só atuais, como também por versões passadas e até futuras. Para isso, recomendo os sites <http://findmebyip.com> e <http://caniuse.com>.

Convenções tipográficas

Com a finalidade de destacar diferentes tipos de informação neste livro, foram adotadas algumas convenções tipográficas, mostradas a seguir: Dica

Texto contendo uma dica sobre o assunto tratado:



Escrever em linhas diferentes cada um dos valores para a propriedade **display** do Módulo Template Layout CSS facilita a leitura e a manutenção da regra CSS que descreve o layout.

Alerta

Texto contendo um lembrete sobre procedimento extra com relação ao assunto tratado:



Os prefixos proprietários para essa propriedade e seus valores seguem a sintaxe da especificação em sua versão anterior, na qual a atual propriedade **flex-direction** era definida com combinações de duas propriedades nomeadas **box-orient** e **box-direction**, com valores vertical ou horizontal e **normal** e **reverse**, respectivamente.

Terminologia

Texto estabelecendo a adoção de grafia padrão em todo o livro para termos ou frases com mais de uma terminologia, uma tradução ou um significado:



Deprecated é termo em inglês para definir os elementos ou atributos que serão excluídos das próximas versões do (X)HTML. Você encontrará este termo traduzido por ultrapassado, obsoleto, desatualizado ou em desuso. Neste livro, adotaremos o termo *em desuso*.

Chamada

Uma chamada para uma seção do livro na qual o assunto em questão está explicado com detalhes. No exemplo a seguir, medidas CSS de ângulos foram tratadas no capítulo 1, seção 1.1.5.4.

O parâmetro **a** para essa função é uma medida CSS para ângulo. Medidas CSS para ângulos são: **deg**, **grad**, **rad** e **turn**. Ver [1.1.5.4].

Códigos

Nos códigos que exemplificam a teoria, foram transcritos somente os trechos que interessam ao assunto tratado. Para as folhas de estilo, omitimos as regras que não dizem respeito ao exemplo. Assim, considere que, para cada bloco de código, há uma marcação padrão com um **DOCTYPE** válido, conforme mostrado a seguir:

```
<!DOCTYPE html>
```

```
<lang="pt-br">
```

```
<head>
```

```
<title>Título da página</title>
```

```
<meta charset="utf-8" />
```

```
<style>
```

```
...Entra folha de estilo para os exemplos...
```

```
</style>
```

```
</head>
```

```
<body>
```

```
...Entra o bloco de código HTML para os exemplos...
```

```
</body>
```

```
</html>
```

- **Blocos de código são marcados com fonte monoespaçada:** `body {
margin: 0;
padding: 0;`

```
font-family: Arial, Sans-Serif;
```

```
}
```

- **Trechos de código que merecem destaque são marcados em negrito:** `body {`

```
margin: 0;
```

```
padding: 0;
```

```
font-family: Arial, Sans-Serif;
```

```
background: #fff; }
```

- **Supressão de códigos é marcada com linha de sobrescrito:** `body {`

```
margin: 0;
```

```
padding: 0; padding: 20px;
```

```
font-family: Arial, Sans-Serif;
```

```
background: #fff;
```

```
}
```

- **Códigos, elementos HTML e sintaxe CSS contidos em textos são marcados com fonte monoespçada:**

Essa propriedade destina-se a definir a direção em que os elementos-filhos serão dispostos no elemento container. Os valores possíveis são: `row`, `column`, `row-reverse` e `column-reverse`.

Códigos em sequência

Nos exemplos que mostram estilização passo a passo, à medida que se progride na explicação e novos estilos são introduzidos, são apresentados somente os fragmentos de código HTML ou CSS acrescentados em relação ao passo anterior. No site do livro você poderá fazer o download dos arquivos contendo seus códigos completos.

Arquivos para download

As páginas contendo a marcação HTML, assim como as regras de estilo criadas para ilustrar com figuras e exemplos as técnicas ensinadas, estão disponíveis para download e/ou consulta online no site do livro, em <http://www.livrocss3.com.br>. Os documentos estão separados em pastas por capítulos e foram nomeados com sintaxe própria, lembrando o assunto em estudo ou a figura no livro, conforme o exemplo a seguir: *propriedade-flex-direction_row-reverse.html figura-6.5.html* O primeiro é um arquivo (*propriedade-flex-direction_row-reverse.html*) que demonstra a propriedade para definição da direção horizontal e o sentido inverso para disposição dos boxes, ou seja, segundo o *main axis* e da esquerda para a direita. O segundo é um arquivo (*figura-6.5.html*) que renderiza, conforme mostrado na figura 6.5.

Alguns exemplos constantes dos arquivos têm uma versão adicional estudada no

livro, mas sem uma figura ilustrativa correspondente.

Destaques em geral

Palavras ou termos cujo significado deva ser destacado são grafados em *itálico*.

Site do livro

O site de suporte a este livro está localizado em <http://www.livrocss3.com.br>.

No site foram incluídas as facilidades relacionadas a seguir: • **Arquivo de códigos:** os códigos dos exemplos mostrados no livro estão disponíveis para download.

- **Errata:** desenvolvemos um exaustivo trabalho de revisão tipográfica. Contudo, a prática mostra que nenhum livro está isento de erros – e com este não há de ser diferente. Uma página do site dedicada à errata está disponível para consulta.
- **Feedback:** incentivamos nossos leitores a emitirem sua opinião sobre qualquer aspecto do livro. Será de grande valia para nós a informação sobre qualquer erro detectado para que possamos aperfeiçoar futuras edições e atualizar a errata. Você pode se comunicar com a editora pelo e-mail novatec@novatec.com.br ou diretamente com o autor pelo e-mail maujorcass@maujor.com.

CAPÍTULO 1

Fundamentos CSS

Neste capítulo, teremos uma introdução às CSS. Veremos sua definição e um breve histórico de seu aparecimento. Em seguida, passaremos ao estudo da sintaxe, dissecando a regra CSS. Veremos a conceituação de unidades de medida CSS e os métodos para vincular folhas de estilo a documentos HTML. Ao final do capítulo, o leitor terá uma visão geral da evolução das CSS desde a sua criação, reconhecerá a terminologia dos fragmentos que compõem uma folha de estilo, terá uma noção sólida das unidades de medida CSS e saberá como linkar folhas de estilo a documentos HTML.

1.1 Definições e conceitos CSS

1.1.1 Definição

CSS é a abreviação para o termo em inglês *Cascading Style Sheet*, traduzido para o português como folhas de estilo em cascata. Neste livro, adotaremos CSS como abreviação e folhas de estilo em cascata para o termo por extenso.

A definição mais precisa e simples para folha de estilo encontra-se na homepage das CSS no site do W3C e diz: “Folha de estilo em cascata é um mecanismo simples para adicionar estilos (por exemplo: fontes, cores, espaçamentos) aos documentos web.”

1.1.2 Finalidade

As CSS têm por finalidade devolver à marcação HTML/XML o propósito inicial da linguagem. A HTML foi criada para ser uma linguagem exclusivamente de marcação e estruturação de conteúdos. Isso significa que, segundo seus idealizadores, não cabe à HTML fornecer informações ao agente do usuário sobre a apresentação dos elementos. Por exemplo: cores de fontes, tamanhos de textos, posicionamentos e todo o aspecto visual de um documento não devem ser funções da HTML. Cabem às CSS todas as funções de apresentação de um documento, e essa é sua finalidade maior. Daí a já consagrada frase que resume a dobradinha CSS + HTML: “HTML para estruturar e CSS para apresentar.”



O termo *agente de usuário* é empregado para se fazer referência a qualquer dispositivo capaz de interpretar um documento escrito em linguagem de marcação. O exemplo mais comum e conhecido de agente de usuário é o navegador. Leitores de tela, robôs de indexação e busca, dispositivos móveis como tablets e smartphones, com seus navegadores, são também alguns exemplos de agentes de usuário.

1.1.3 Histórico

Tim Berners-Lee, ao desenvolver o navegador Nexus, que serviu para implementar suas invenções, escreveu também, ainda que de forma bastante limitada, algumas funcionalidades intrínsecas que controlavam a apresentação dos documentos. Navegadores que se seguiram, nos anos de 1992 e 1993, também vinham com funcionalidades de estilização padrão, tais como no modelo desenvolvido por Tim. No navegador Mosaic, lançado em 1993 e que popularizou a web, também foram previstas funcionalidades mínimas para aplicar estilos. Na verdade, apenas o controle de algumas fontes e de cores era possível. As funcionalidades intrínsecas aqui referidas são aquelas que hoje em dia conhecemos como folhas de estilo padrão do navegador. Um conjunto de regras de estilo forma uma folha de estilo que o navegador aplica aos documentos por padrão nos casos em que o autor do documento ou o usuário não tenham definido nenhuma regra de estilo. Adiante, veremos detalhes da folha de estilo.

Em setembro de 1994, surgiu a primeira proposta para implementação das CSS. Até então, o próprio Tim considerava que a estilização era uma questão a ser resolvida pelo navegador, razão pela qual não se preocupou em publicar a sintaxe usada para criar a folha de estilo padrão do seu navegador.

No mês de dezembro de 1996, as CSS1 foram lançadas como uma Recomendação oficial do W3C.

1.1.4 Regra CSS

Regra CSS é a unidade básica de uma folha de estilo. Nessa definição, o termo unidade básica significa a menor porção de código capaz de produzir efeito de estilização. Uma regra CSS é composta de duas partes: o *seletor* e a *declaração*. A declaração compreende uma *propriedade* e um *valor*. A sintaxe para se escrever uma regra CSS é mostrada na figura 1.1.



Figura 1.1 – Sintaxe da regra CSS.

Definição dos componentes de uma regra CSS:

- **Seletor:** é o alvo da regra CSS.
- **Declaração:** determina os parâmetros de estilização. Compreende a propriedade e o valor.

- **Propriedade:** define qual será a característica do seletor a ser estilizada.
- **Valor:** é a quantificação ou a qualificação da propriedade.



A terminologia mostrada é a adotada pelo W3C e recomendo que você a adote também. Evite usar expressões em desacordo com a sintaxe oficial, tais como “atributo CSS” no lugar de *propriedade* CSS, “comando CSS” no lugar de *declaração* CSS e outras.

Uma regra CSS poderá conter várias declarações separadas por ponto e vírgula. Observe o exemplo a seguir: p {

```
color: red;
background-color: black;
font-size: 12px;
```

```
}
```

Nesse exemplo identificamos os seguintes componentes da regra CSS:

- **Seletor:** é o elemento **p**. Seletores para elementos da marcação são denominados seletores tipos.
- **Declarações:** são três – **color: red; background-color: black; font-size: 12px;**.
- **Propriedades:** são três – **color**, que define a cor dos textos contidos em **p**, **background-color**, que define a cor de fundo do elemento **p** e **font-size**, que define o tamanho da fonte para o elemento **p**.



Para ver a relação completa das propriedades CSS3, seus valores possíveis, valores iniciais e sua herança, consulte o apêndice A.

Note que, no exemplo anterior, colocamos uma declaração em cada linha. Fizemos isso apenas para facilitar a leitura do código, pois poderíamos ter colocado todas as declarações na mesma linha, sendo obrigatório apenas o uso de ponto e vírgula para separá-las. O uso de espaços em branco, entre os componentes de uma regra CSS, é facultativo e visa apenas a facilitar a leitura do código. A regra mostrada seguir, em uma linha, tem o mesmo efeito.

```
p { color:red; background-color:black; font-size:12px; } /* Menos legível, não é? */
```

O ponto e vírgula colocado na última declaração de uma regra de estilo, ou colocado na declaração única de uma regra, é facultativo, porém é de boa prática usá-lo, como fizemos no exemplo mostrado, pois, se no futuro tivermos que acrescentar mais declarações à regra, estamos livres da obrigação de colocar o ponto e vírgula e, melhor ainda, não correremos o risco de interromper o funcionamento da regra CSS por termos nos esquecido dele.

Quando o valor de uma propriedade for uma palavra composta, separada por espaços, deve-se usar sinais de aspas duplas (" ") ou, alternativamente, de aspas

```
simples (' '), conforme mostrado a seguir: p {  
  font-family: "Times New Roman";  
}
```

ou

```
p {  
  font-family: 'Times New Roman';  
}
```

Não se usam aspas em palavras compostas separadas por hífen:

```
p {  
  font-family: Sans-serif;  
}
```

A sintaxe da regra CSS *não é* sensível ao tamanho de caixa da fonte (você pode usar letras minúsculas ou maiúsculas, indiferentemente) e múltiplos espaços são tratados como espaço simples. Usar ou não espaços entre os componentes da regra CSS fica a critério do desenvolvedor. Todas as regras CSS mostradas a seguir são válidas: h1 { border: 1px solid red; }

```
h1 {border:1px solid red;}  
h1{ border: 1px solid red;}  
H1 { border: 1px solid RED;}  
h1{ BORDER: 1px solid red; }
```

Tratando-se de linguagem de programação, sempre que houver mais de uma forma válida de escrever o código, o desenvolvedor deve escolher uma delas e adotá-la como seu padrão pessoal. Isso torna o código consistente e facilita a manutenção. Com as folhas de estilo aplica-se essa prática, e você pode escolher qualquer uma das formas mostradas anteriormente ou, mesmo, outras variações para escrever suas folhas de estilo. Contudo, as duas formas de uso mais difundidas são as mostradas no primeiro e no segundo itens do exemplo anterior. A primeira adota um espaço em branco junto ao sinal de chaves ({ }). A segunda, um espaço somente para separar o seletor da declaração. A forma estendida de declarar as propriedades em linhas distintas é escrita conforme mostrado a seguir, sendo a endentação a critério do desenvolvedor.

```
h1 {  
  border: 1px solid red;  
}
```

Um componente facultativo, mas de grande utilidade na escrita de folhas de estilo, é o sinal para inserir comentários. À semelhança de qualquer linguagem de programação, existe um sinal próprio para marcar comentários no código de estilos, conforme mostrado nos exemplos a seguir: Comentário em uma linha:

```
/* Este é um comentário em uma linha */
```

Bloco de comentário:

```
/* Este é um bloco de comentário em linhas  
diferentes contendo muitas informações  
sobre um trecho da folha de estilo */
```

Você começa um comentário com o sinal `/*` e termina com o sinal `*/`.

Um conjunto de regras CSS é denominado *folha de estilo*. O conjunto das regras pode ser escrito no próprio documento ao qual as regras serão aplicadas ou em um arquivo externo gravado com a extensão padrão `.css`. Por exemplo: *principal.css*.

1.1.5 Tipos de componentes da regra CSS

Para aplicar uma regra CSS, o agente de usuário (por exemplo: o navegador) identifica o valor da propriedade e renderiza o elemento, casado pelo seletor, com a estilização de acordo com aquele valor. Observe as regras CSS que se seguem: `p { font-family: Arial, Sans-serif; }` `/* estiliza p com fonte da família sem serifas */`

```
p { width: 400px; } /* estiliza p com largura 400px */
```

```
p { font-size: 120%; } /* estiliza p com tamanho de fonte 1.2 vezes o  
valor de referência */
```

```
p { background-color: red; } /* estiliza p com fundo na cor vermelha */
```

```
p { height: 2em; } /* estiliza p com altura 2 vezes o valor de referência  
*/
```

Observe que alguns valores são absolutos (é aquele valor e pronto!) e outros relativos (dependem de um valor de referência), tais como as medidas CSS em porcentagem e `em`.

Os componentes de uma regra CSS foram agrupados, pela especificação do W3C, segundo seis tipos, conforme descrito nos subitens a seguir.

1.1.5.1 Componentes textuais

São os componentes da regra CSS do tipo palavra-chave quando expresso por uma string predefinida nas especificações. O exemplo típico para esse caso é quando usamos palavra-chave para definir valores para cores, conforme mostrado no exemplo a seguir: `p {`

```
color: red;
```

```
background-color: aqua; border-color: teal; }
```

Os valores `red`, `aqua` e `teal` são palavras-chave para cores, previstas nas

especificações para as CSS3. Consulte a lista completa das palavras-chave para cores no apêndice D.

Outros exemplos de componentes textuais para expressar valores CSS são:

Palavra-chave	Utilizada
<code>inherit</code>	para definir uma propriedade que deverá ser herdada.
<code>collapse</code>	para definir bordas de células de tabelas unidas.
<code>italic</code>	para definir fonte em itálico.
<code>uppercase</code>	para definir texto em caixa-alta.

Strings

Componentes CSS expressos com strings devem ser grafados com aspas simples (‘) ou duplas (“). Uma não pode ocorrer dentro de outra, a menos que seja escapada com uma barra invertida (\).

Uma string não pode conter uma quebra de linha, a menos que se use o caractere \A, que representa uma nova linha em CSS.

Para fins de legibilidade, é possível quebrar uma string em substrings com uso do caractere barra invertida (\).

Observe os exemplos a seguir, que esclarecem essas sintaxes:

```
"Esta é uma 'string'." /* aspas simples dentro de aspas duplas */
'Esta é uma "string".' /* aspas duplas dentro de aspas simples */
"Esta é uma \"string\"." /* aspas duplas escapadas dentro de aspas duplas */
'Esta é uma \'string\'' /* aspas simples escapadas dentro de aspas simples */
```

```
"Esta string está na primeira linha. \A E esta na segunda"
```

```
"Esta é uma string longa\
que foi quebrada para\
fins de legibilidade."
```

1.1.5.2 Componentes numéricos

Um componente da regra CSS é do tipo numérico quando expresso por um número inteiro, por um número real ou em porcentagem. A especificação adota a sintaxe `<integer>` para designar números inteiros, `<number>` para designar números reais e `<percentage>` para designar valores em porcentagem. Números inteiros são aqueles

formados por um ou mais algarismos de 0 a 9, e números reais são formados por um ou mais algarismos de 0 a 9, seguidos de uma vírgula e seguidos de um ou mais algarismos de 0 a 9. Todo número inteiro é também um número real.

Em CSS, números podem ser precedidos dos sinais “+” e “-” para indicar o sinal do número. Esses dois caracteres, quando usados, passam a fazer parte do componente CSS.

A restrição não se aplica ao número zero, que é um número não negativo.

1.1.5.3 Unidades de medida

Os valores CSS, quando expressos com números seguidos por uma unidade de medida, são classificados em cinco categorias, conforme descritas nos subtítulos a seguir.

Comprimento

Comprimento se refere às medidas horizontal e vertical. A sintaxe prevista nas especificações para designar essa categoria é `<length>`. Um valor CSS que usa uma medida de comprimento é formado por um número seguido da abrevitura para uma unidade de medida. Por exemplo: `14px`, `12em`, `18pt`. Se o número é zero, a unidade de medida é opcional. Recomendo não usar unidade de medida nesses casos, pois não vejo sentido em `0px`, `0em`, `0cm`, pois zero é zero, independentemente de unidades. As exceções a essa grafia ocorrem na sintaxe de definição da posição do primeiro frame de uma animação CSS, pois nesse caso, a definição do frame deve ser `0%`, sendo obrigatório constar o sinal de porcentagem depois do valor zero (ver capítulo 15) e na definição de ângulos de transformações CSS quando a unidade deve ser indicada como em `0deg`, `0rad` *etc.* (ver capítulo 12).

Medida relativa

Unidade de medida relativa é aquela cujo valor é determinado em função de outro valor para uma propriedade que lhe serve de referência. Definir medidas relativas em uma folha de estilo facilita o escalonamento e possibilita servi-la para diferentes tipos de mídias, por exemplo: para uma tela de computador ou para uma mídia de impressão.

As unidades de medidas relativas nas CSS3 são mostradas no quadro a seguir:

Unidade	Relativa
em	à font-size do elemento (ou do elemento-pai).
ex	ao valor x-height da fonte do elemento.

px	ao dispositivo de renderização.
gd	ao grid definido pelo “layout-grid”, descrito no Módulo Texto da CSS3.
rem	à font-size do elemento raiz.
vw	à largura da viewport.
vh	à altura da viewport.
vmin	à menor dimensão da viewport.
vmax	à maior dimensão da viewport.
ch	à largura do número “0”, renderizado de acordo com font-size. Se não existir “0” na fonte especificada, a largura média dos caracteres.

Medida absoluta

Unidade de medida absoluta é aquela cujo valor é determinado e fixo. Essas unidades são úteis para uso quando se conhece as dimensões físicas da mídia para a qual a folha de estilo será servida. As unidades de medidas absolutas nas CSS3 são mostradas no quadro a seguir:

Unidade	Descrição
in	polegada; 1 polegada = 2,54 cm
cm	centímetro
mm	milímetro
px	pixel; 1 pixel = 1/96 polegada
pt	ponto; 1 ponto = 1/72 polegada
pc	pica; 1 pica = 12 pontos

Porcentagem

O formato para definir um valor CSS em porcentagem é um número imediatamente seguido pelo sinal %. A sintaxe prevista nas especificações para designar essa categoria é **<percentage>**. Porcentagens são valores dependentes de outro valor, por exemplo: de um valor do tipo **<length>**.

As propriedades CSS que admitem valores em porcentagem também definem qual o valor de referência a considerar para cálculo da porcentagem. O valor de referência pode ser o valor de outra propriedade do mesmo elemento ao qual a porcentagem

foi aplicada, o de um elemento ancestral ou o valor de um contexto de formatação, como a largura de um bloco de conteúdo.

1.1.5.4 Outras unidades

Ângulo

O formato para definir um valor CSS em medida angular é um número imediatamente seguido por uma unidade de medida angular. A sintaxe prevista nas especificações para designar essa categoria é `<angle>`. As unidades de medida angular em CSS são:

Unidade	Descrição
deg	Graus
grad	Grados
rad	Radianos
turn	Volta

Valores expressos em medidas angulares são usados, por exemplo, para definir propriedades destinadas à mídia speech (mídia falada, tal como leitores de tela) ou às transformações CSS3.

Hora

O formato para definir um valor CSS em medida de hora é um número imediatamente seguido por uma unidade identificadora de tempo em segundos. A sintaxe prevista nas especificações para designar essa categoria é `<time>`. As unidades de medida de tempo em CSS são:

Unidade	Descrição
ms	Milissegundo
s	Segundo

Valores expressos em medidas de tempo são usados, por exemplo, para definir propriedades destinadas à mídia speech (mídia falada, tal como leitores de tela) ou duração de animações e transições CSS3.

Frequência

O formato para definir um valor CSS em medida de frequência é um número imediatamente seguido por uma unidade identificadora de frequência em hertz. A

sintaxe prevista nas especificações para designar essa categoria é **<frequency>**. As unidades de medida de frequência em CSS são:

Unidade	Descrição
Hz	Hertz
kHz	Quilohertz

Valores expressos em medidas de frequência são usados, por exemplo, para definir propriedades destinadas a mídia speech (mídia falada, tal como leitores de tela).

Resolução

O formato para definir um valor CSS em medida de resolução é um número imediatamente seguido por uma unidade identificadora de resolução. A sintaxe prevista nas especificações para designar essa categoria é **<resolution>**. As unidades de medida de resolução em CSS são:

Unidade	Descrição
dpi	dots (pontos) por polegada
dpcm	dots (pontos) por centímetro
dppx	dots (pontos) por pixel

1.1.5.5 Valores definidos em outra especificação

Fazem parte deste tipo os componentes CSS para cores, imagens e posicionamento. As sintaxes previstas nas especificações para designar essas categorias são **<color>**, **<image>** e **<position>**.

1.1.5.6 Notação funcional

Valores CSS podem ser expressos por uma função, e nesses casos são classificados como valores em notação funcional. Em CSS3, valores funcionais são usados para definir cores, atributos e URIs.

A sintaxe para escrita de um valor funcional é: nome da função seguido de uma lista de argumentos entre parênteses. Considere os exemplos mostrados a seguir: p { background-color: rgb(255, 0, 0); }

```
img { margin-top: attr(height, px); }
```

```
div { background-image: url(http://maujob.com/avatar.gif); }
```

Nos três exemplos mostrados, os valores das propriedades CSS, em destaque no código, são do tipo valor funcional e as respectivas funções CSS **rgb**, **attr** e **url**

retornam um valor a ser aplicado nas propriedades definidas para os seletores `p`, `img` e `div`, respectivamente.

1.1.6 Tipos de valores CSS

Vimos no item anterior [1.1.5] os tipos para definição de componentes CSS. É importante também conhecer o conceito de valor CSS para efeito de aplicação da regra CSS no elemento. Observe o exemplo a seguir: `p { font-size: 120%; }`

O valor `120%` para a propriedade `font-size` dessa regra CSS enquadra-se no agrupamento de valores denominado “Número com unidade de medida”, conforme vimos em [1.1.5.4]. Contudo, como o navegador aplica um tamanho de fonte igual a `120%`? Qual o valor em pixels? `120%` do quê?

Para aplicar valores CSS o navegador precisa, em certos casos, efetuar cálculos e, em outros, “tirar algumas conclusões” para chegar ao valor a aplicar. Ao longo do processo de investigação o navegador passa por etapas, e em cada etapa chega a um tipo de valor. Veremos a seguir esses tipos de valores.

Observe as regras CSS que se seguem, já mostradas anteriormente:

```
p { font-family: Arial, Sans-serif; } /* estiliza p com fonte na família
especificada (valor) */
p { width: 400px; } /* estiliza p com largura 400px */
p { font-size: 120%; } /* estiliza p com tamanho de fonte 1.2 vezes o
valor de referência */
p { background-color: red; } /* estiliza p com fundo na cor vermelha */
p { height: 2em; } /* estiliza p com altura 2 vezes o valor de
referência */
```

Observe que alguns valores são absolutos (é aquele valor e pronto!) e outros relativos (dependem de outros valores), tais como as medidas CSS em porcentagem e `em`. Para aplicar valores CSS, os mecanismos das CSS consideram cinco tipos de valores, que veremos a seguir.

1.1.6.1 Valor inicial

A todas as propriedades CSS é atribuído, por padrão, um valor denominado valor inicial.

O valor inicial de cada uma das propriedades CSS é definido por um mecanismo nativo do agente de usuário (por exemplo: navegador). Infelizmente, não há padronização para o valor inicial das propriedades CSS e cada navegador implementa essa funcionalidade à sua maneira. Esse comportamento pode trazer inconsistência de renderização em diferentes navegadores.

Felizmente, todos os navegadores adotam o mesmo valor para muitas das

propriedades CSS. As inconsistências, em sua maioria, estão relacionadas à definição de valores iniciais para **margin** e **padding**. Observe a seguir o valor inicial de algumas propriedades CSS: `border: none;`

```
color: black;
background: transparent;
font-family: serif;
font-size: 16px;
```

Todas as propriedades CSS admitem a palavra-chave **initial** para forçar a adoção do valor inicial da propriedade. Observe o exemplo a seguir, baseado no exemplo mostrado anteriormente: ► **HTML**

```
<div>
  <p>Parágrafo com <em>palavra</em> marcada com ênfase</p> </div>
```

► **CSS**

```
div {
  color: red;
  border: 1px solid blue;

}
```

```
p { color: initial; }
```

Nesse caso, forçamos o elemento **p** a ser estilizado com a cor inicial preta, anulando o efeito herança.

Nota: O valor **initial** não existia nas versões anteriores das CSS. Os navegadores Chrome e Safari já suportam nativamente esse valor e o navegador Firefox 4 implementa-o com uso do prefixo **-moz-**, como mostrado a seguir: `p { color: -moz-initial; }`

Em abril de 2007, Eric Meyer publicou em seu blog uma matéria comentando as inconsistências de renderização entre navegadores em razão da não padronização de valores iniciais, para as diferentes propriedades CSS. Propôs então uma solução que consistia em criar uma folha de estilos padrão que, em resumo, se destinava a padronizar os valores iniciais. Eric expôs ainda razões para preservar determinados valores iniciais e fez algumas considerações a mais, mas não é do escopo deste livro detalhar a matéria. Se você estiver interessado em aprofundar-se no assunto, a matéria encontra-se em <http://kwz.me/rb>.

Como consequência, Eric criou uma folha de estilos que vem sendo atualizada regularmente e hoje se encontra em sua versão 2.0, lançada em 26 de janeiro de 2011. A folha de estilos, mundialmente conhecida e usada, foi denominada “CSS Reset” e lançada para uso livre e gratuito sob o rótulo de domínio público. Inúmeros frameworks, ferramentas de desenvolvimento e desenvolvedores em geral usam a

folha de estilos criada por Eric como ponto de partida para os valores iniciais das propriedades CSS. Para obter uma cópia atualizada da “CSS Reset”, visite <http://kwz.me/rt>.

É comum encontrarmos em fóruns e matérias publicadas em blogs a indicação do uso de uma regra CSS para zerar os valores de `margin` e `padding` de todos os elementos da marcação com uso do seletor universal, como mostrado a seguir: `* {`

`margin: 0;`

`padding: 0;`

`}`

Eric adverte que, embora esse seja um bom ponto de partida, é preciso estar ciente de que todos os elementos da marcação terão aquelas duas propriedades zeradas e isso nem sempre ocorre na prática, pois para elementos de formulário, por exemplo, dependendo do navegador, a regra será ignorada ou ainda poderá haver alteração na apresentação do elemento, perdendo-se nesses casos a consistência.

1.1.6.2 Valor especificado

Tão logo um documento seja parseado e a árvore do documento tenha sido construída, o agente de usuário deverá atribuir a cada propriedade CSS um valor. A atribuição do valor segue um mecanismo que obedece a três ordens de precedência, como mostrado a seguir: • se o efeito cascata resulta em um valor, adote esse valor;

- não havendo um valor de cascata e se o valor da propriedade for herdável, adote o valor herdado; • não ocorrendo as hipóteses anteriores, adote o valor inicial da propriedade.

O valor resultante da aplicação desse mecanismo é denominado valor especificado.

1.1.6.3 Valor computado

O valor especificado pode ser absoluto (por exemplo: 2 mm, red etc.) ou relativo (por exemplo: 80%, 2 em etc.). Se o valor especificado for absoluto, então o valor computado é igual a ele e nenhum cálculo é necessário para se chegar ao valor computado.

Caso o valor especificado seja expresso em medida relativa, é necessário fazer um cálculo para chegar ao valor computado.

Por exemplo: porcentagens devem ser tomadas em relação a um valor de referência, que é definido por padrão para cada propriedade; valores relativos para unidades de medida, tais como `px`, `em` e `ex` devem ser tornados absolutos, multiplicando-se por tamanhos de fonte ou de pixel apropriados; valores `auto`

devem ser calculados por fórmulas definidas para cada uma das propriedades e assim por diante.

1.1.6.4 Valor usado

É o valor efetivamente usado para renderizar o documento. Valores computados podem ser processados antes mesmo de o documento ter sido formatado, contudo alguns valores só podem ser computados com o conhecimento do layout do documento. Por exemplo: um valor definido em porcentagem para a largura de um box somente poderá ser computado após o conhecimento da largura do box que o contém.

Denominamos de valor usado aquele resultante do valor computado depois de resolvidas todas as pendências de layout.

1.1.6.5 Valor atual

A princípio, o valor usado é aquele adotado para renderizar o documento; contudo, pode ocorrer o caso em que o agente de usuário não seja capaz de renderizar o valor usado. Por exemplo: chega-se a um valor usado para a espessura da borda de um elemento igual a **4.2px**. Nesse caso, o valor atual deverá ser o resultado do arredondamento do valor usado, pois não há como representar frações de pixel.

1.2 Vinculando folhas de estilo a documentos

Depois que você acabou de escrever sua folha de estilo, precisa informar ao documento onde ele deve buscá-la. Ou seja, você precisa de um método capaz de vincular a folha de estilo ao documento ao qual ela será aplicada.

As folhas de estilo podem ser escritas no próprio documento HTML ao qual serão aplicadas ou ser arquivos externos independentes, gravados com a extensão *.css*, como, por exemplo, um arquivo chamado de *main.css*, e linkados ao documento.

1.2.1 Estilos inline

O método direto e simples de aplicar estilos a um elemento da marcação é com o emprego do atributo `style` da HTML. Você escreve as regras de estilo diretamente dentro da tag de abertura do elemento a estilizar, conforme mostra o exemplo a seguir: `<p style="width: 200px; color: white; background: red; font-size: 1.8em;">`
`<!-- Parágrafo com aplicação de estilos inline -->`
`</p>`

Esse método dificulta a manutenção e retira um dos maiores poderes da folha de estilo, que é o controle centralizado da apresentação. Toda vez que for preciso alterar a apresentação, será necessário percorrer todo o código de marcação do documento

ou centenas de documentos, se o site for grande, à procura das regras de estilo inline.

1.2.2 Estilos incorporados

Outro método de escrever a folha de estilos no próprio documento HTML é com o emprego do elemento `style`. Você escreve as regras de estilo dentro das tags `<style>` `</style>`, como mostrado no exemplo a seguir: `<head>`

```
<style type="text/css" media="all"> body {  
margin: 0;  
padding: 0;  
font-size: 80%;  
color: black;  
background: white;  
  
}
```

```
</style>  
</head>
```

A vantagem desse método sobre o anterior é que, agora, localizamos com mais facilidade a folha de estilo, mas há a desvantagem de colocar a folha de estilos dentro do próprio documento. Não seria sensato vincular uma mesma folha de estilo a vários documentos empregando esse método. Toda vez que for preciso alterar a apresentação, será necessário abrir o documento ou centenas de documentos, se o site for grande, e fazer a mesma alteração de estilo em todos eles. Uma boa escolha para uso desse método seria para o caso de aplicação de estilos específicos a um documento do site.

O elemento `style` deve estar contido na seção `head` do documento. Note que em marcação HTML5 o uso de atributos no elemento `style` é facultativo. O atributo `type` informa qual tipo de dado está sendo enviado e o atributo `media` informa a qual tipo de mídia devem ser aplicados os estilos. Os valores do atributo `media` e a mídia a que se destinam são listados a seguir:

Valor	Mídia	Nota
screen	Telas de monitores	
tty	Teletipo e similares	
tv	Dispositivos tipo televisão	
projection	Projetores	
handheld	Dispositivos portáteis	

Valor	Mídia	Nota
<code>print</code>	Impressoras e visualização no modo impressão	
<code>braille</code>	Dispositivos táteis	
<code>aural</code>	Sintetizadores de voz	em desuso pela CSS3
<code>all</code>	Todos os tipos de mídia	
<code>speech</code>	Sintetizadores de voz	criada pela CSS3
<code>embossed</code>	Impressoras braille	criada pela CSS3

1.2.3 Estilos externos

Folha de estilo externa é aquela que não foi escrita no documento HTML. Trata-se de um arquivo de texto contendo as regras de estilo e os comentários CSS. Um arquivo de folha de estilo deve ser gravado com a extensão `.css` e pode ser vinculado a um documento HTML de duas maneiras distintas, conforme explicado em seguida.

1.2.3.1 Folhas de estilo linkadas

Você vincula uma folha de estilo externa a um documento empregando o elemento `link`. O elemento `link` deve estar contido na seção `head` do documento e tem por finalidade associar outros documentos ao documento no qual ele está contido. Veja, a seguir, o uso de `link` para associar (ou vincular) uma folha de estilo ao documento: `<head>`

```
...
<link rel="stylesheet" type="text/css" href="estilos.css" media="all" />
...
</head>
```

O atributo `href` aponta para o endereço no qual se encontra o arquivo da folha de estilo.

1.2.3.2 Folhas de estilo importadas

Nesse método, você vincula uma folha de estilo externa a um documento usando a diretiva `@import` dentro do elemento `style`. A sintaxe para esse tipo de vinculação é mostrada a seguir: `<head>`

```
...
<style type="text/css">
@import url("estilos.css") screen, projection;
</style>
```



```
...
</head>
```

Alternativamente, você pode usar uma forma abreviada da sintaxe, como se mostra a seguir: `<style type="text/css">`

```
@import "estilos.css" screen, projection;
</style>
```

As duas sintaxes apresentadas (com ou sem a notação `url (...)` na diretiva) são equivalentes. O tipo de mídia é definido em uma lista separada por vírgula na própria diretiva. Na ausência do tipo de mídia, os estilos serão aplicados para todas as mídias, ou seja, o efeito é o mesmo que o de declarar mídia `all`. Declarando ou não a mídia, deve-se terminar a diretiva com um ponto e vírgula.

Podemos também vincular uma folha de estilo externa a outra folha de estilo usando a diretiva `@import` dentro da folha de estilo. A sintaxe para esse tipo de vinculação é idêntica à mostrada anteriormente.

O exemplo a seguir mostra uma folha de estilo externa na qual importamos uma folha de estilo denominada *main.css*: **`@import "main.css"`**

```
body {
margin: 0;
font: 62.5% Arial, Sans-serif;

}
```

... mais regras de estilo ...

É válido importar mais de uma folha de estilo.

A declaração de importação de uma folha de estilo dentro de outra deve ocupar a primeira linha da folha, exceto no caso em que se use uma declaração de codificação de caracteres da folha de estilo. A diretiva `@charset` destina-se a declarar a codificação de caracteres de uma folha de estilos e deve ocupar a primeira linha na folha de estilo. Observe o exemplo a seguir, que demonstra as duas diretivas inseridas em uma folha de estilo: **`@charset "utf-8"`**

```
@import "main.css"
body {
margin: 0;
font: 62.5% Arial, Sans-serif;

}
```

... mais regras de estilo ...

A diretiva `@import` deve preceder todas as demais regras de estilo para o documento. Havendo necessidade de vincular outras folhas de estilo ao documento,

elas deverão ser declaradas após a diretiva.

O método de vincular folhas de estilo externas permite que se apliquem regras de estilo comuns a todos os documentos de um site. A grande vantagem do método é que o autor controla a apresentação do site em um arquivo central. A alteração de uma cor ou do tamanho de fonte na folha de estilo imediatamente se reflete no site inteiro, quer ele tenha 10 ou 10.000 páginas.

1.3 Módulos CSS3

A primeira versão das especificações do W3C para as Folhas de Estilos em Cascata foi para as CSS nível 1, lançada em 17 de dezembro de 1996, em documento único, que está dividido em nove seções e cinco apêndices. Em 12 de maio de 1998, foi lançada a versão para as CSS nível 2, em documento único, contendo 19 seções e oito apêndices. Em 2 de agosto de 2002, foi proposta a revisão da especificação, dando origem à versão das CSS nível 2.1, em documento único, contendo 18 seções e sete apêndices. A versão para as CSS nível 2.1 passou para o status de Recomendação do W3C em 7 de junho de 2011.

O W3C prefere o termo nível em vez de versão, para designar a progressão dos diferentes documentos que especificam as CSS, alegando ser aquele termo mais apropriado. Argumenta informando que cada novo nível é projetado com base no nível anterior, refinando definições e acrescentando novas funcionalidades às já existentes. O conjunto de funcionalidades de níveis anteriores é, sempre, um subconjunto das funcionalidades da versão atual, portanto por ela plenamente suportado. Dessa forma, um agente de usuário que suporte as funcionalidades das CSS de nível atual suporta também todas as funcionalidades das CSS de níveis anteriores.



Neste livro, deixaremos de lado essa consideração semântica a respeito de versões e níveis e adotaremos, por simplificação, o termo versão para designar os diferentes níveis de desenvolvimento das CSS. Assim, teremos CSS1, CSS2, CSS2.1 e CSS3.

O modelo de desenvolvimento das especificações para as CSS3 alterou o modelo adotado para as versões anteriores. Enquanto estas foram desenvolvidas em um documento único, as CSS3 estão sendo desenvolvidas em módulos.

Dividiu-se a especificação existente em assuntos e criou-se para cada assunto uma especificação independente. Por exemplo: as especificações para seletores CSS3 constituem um módulo, as bordas e fundos (background), outro módulo, as cores, outro, e assim por diante.

Cada módulo é desenvolvido de forma independente e segue um cronograma próprio. Hoje, temos módulos cujas especificações estão em fase de Candidata a

Recomendação, outros em fase de Rascunho de Trabalho e outros ainda cujas especificações nem começaram a ser desenvolvidas.

A grande vantagem da modularização está no fato de que o desenvolvimento das funcionalidades de um módulo não mais depende do andamento dos outros módulos, como aconteceu com o desenvolvimento das versões anteriores das CSS. Isso possibilita que fabricantes comecem a implementar, desde já, funcionalidades previstas em módulos mais adiantados no seu desenvolvimento. Muitas funcionalidades das CSS3 já vêm sendo largamente empregadas.

Nas tabelas mostradas a seguir, estão relacionados alguns dos módulos previstos para as CSS3 e o respectivo status de desenvolvimento.

1.4 Status dos módulos

O desenvolvimento das especificações no W3C segue etapas que são denominadas de status. As etapas ou os status das especificações são mostrados a seguir:

Etapas	Descrição
WD	Working Draft ou Rascunho de Trabalho. Essa etapa começa com a publicação de um rascunho das especificações para conhecimento e discussão públicos.
LC	Last Call ou Última Chamada. Essa etapa começa depois de supostamente concluídos os estudos e a discussão pública. É uma última chamada às considerações públicas.
CR	Candidate Recommendation ou Candidata a Recomendação. Essa fase começa quando o W3C acredita que foram resolvidas todas as pendências técnicas e a fase de implementação já pode começar.
PR	Proposed Recommendation ou Proposta de Recomendação. Essa fase começa quando as implementações já demonstraram a maturidade técnica da especificação. A especificação é remetida para o Comitê Consultivo para endosso pelo W3C.
REC	W3C Recommendation ou Recomendação do W3C. A especificação está terminada e foi aprovada pelos membros e pelo diretor do W3C.

A sequência de progresso pelas etapas não é irreversível, ou seja, uma especificação pode ter atingido um status de desenvolvimento e por razões diversas regredir para status anteriores.

A título meramente informativo, as tabelas a seguir mostram o status de alguns dos módulos das CSS3 e suas classificações para efeito de desenvolvimento segundo o W3C.

Para uma visão atualizada das classificações e do status dos módulos consulte: <http://www.w3.org/Style/CSS/current-work> As tabelas a seguir não abrangem todos os

módulos das CSS3, elas são meramente ilustrativas, e os nomes dos módulos foram mantidos em inglês.

Concluído	Status
Selectors	Recomendação do W3C
CSS Colors	Recomendação do W3C
CSS Namespaces	Recomendação do W3C
Media Queries	Recomendação do W3C

Estável	Status
CSS Style Attributes	Candidata a Recomendação

Em testes	Status
CSS Values and Units	Candidata a Recomendação
CSS Text	Candidata a Recomendação
CSS Image Values	Candidata a Recomendação

Em refinamento	Status
CSS Animations	Rascunho de Trabalho
CSS Transforms	Rascunho de Trabalho
CSS Transitions	Rascunho de Trabalho
CSS Text	Rascunho de Trabalho

Em revisão	Status
CSS Page Media	Rascunho de Trabalho
CSS Basic User Interface	Candidata a Recomendação
CSSOM View	Rascunho de Trabalho

Em estudos	Status
CSS Exclusions and Shape	Rascunho de Trabalho
Filter Effects	Rascunho de Trabalho
CSS Line Grid	Rascunho de Trabalho

CSS Masking	Rascunho de Trabalho
Sendo reescrita	Status
CSS Basic Box Model	Rascunho de Trabalho
CSS Generated Content	Rascunho de Trabalho
CSS Ruby	Rascunho de Trabalho
CSS Syntax	Rascunho de Trabalho

Abandonada	Status
Behavioral Extensions to CSS	Rascunho de Trabalho
CSS Hyperlink Presentation	Rascunho de Trabalho
CSS Grid Positioning	Rascunho de Trabalho

CAPÍTULO 2

Módulo CSS Basic Box Model

Neste capítulo, estudaremos o modelo CSS para boxes. Esse modelo determina as características dimensionais, visuais e de posicionamento que regem o layout dos elementos de marcação. Boxes, com o mesmo sentido da palavra em inglês, são verdadeiras “caixas”, que representam os elementos da marcação, seus conteúdos, espaçamentos internos e externos. A exata compreensão do comportamento e da constituição dos boxes CSS é essencial para o entendimento do funcionamento das CSS.

2.1 Box Model

As CSS admitem que o layout de um documento – ou seja, sua formatação visual – seja criado a partir de uma árvore de elementos de marcação. Cada elemento contém uma lista de zero ou mais elementos-filhos e, opcionalmente, uma string de texto antes da lista, uma string de texto depois da lista e strings de texto entre os itens da lista. O único elemento que não possui um elemento-pai é o elemento raiz do documento.

Para formatar visualmente o layout de um documento, as CSS descrevem como cada elemento e cada string de texto serão dimensionados e posicionados, transformando a árvore do documento em uma árvore de boxes, cujas dimensões, cujo posicionamento e relacionamento com boxes vizinhos dependem das propriedades de cada um dos boxes.

Há três tipos básicos de boxes, conforme descritos a seguir:

- **block-level boxes** – boxes nível de bloco. São os boxes gerados por elementos, tais como por um parágrafo;
- **line-boxes** – boxes em linha. São boxes, tais como aqueles gerados por linhas de texto;
- **inline-level boxes** – boxes nível inline. São boxes, tais como aqueles gerados pelas palavras em uma linha de texto.

2.1.1 Bloco de conteúdo

Bloco de conteúdo ou container é um box retangular que contém um ou mais boxes, que na árvore do documento são seus elementos descendentes. O conceito de bloco de conteúdo ou container é importante para o entendimento não só da herança CSS como também dos mecanismos de cálculo de dimensões relativas e de posicionamento dos seus boxes descendentes. Um box descendente não precisa, necessariamente, estar confinado dentro dos limites do seu bloco de conteúdo, podendo, portanto, ultrapassar aqueles limites. Esse comportamento é denominado

overflow.

2.1.2 Box nível de bloco anônimo

O conceito de box nível de bloco anônimo é mais bem entendido com uso de um exemplo. Considere a marcação a seguir: `<div>`

```
Texto inserido diretamente no elemento div  
<p>Texto de um parágrafo contido no div</p>  
Mais texto inserido diretamente no div  
</div>
```

A renderização da marcação e os boxes do modelo CSS são mostrados na figura 2.1.

Observe que a renderização das três linhas de texto se deu como se cada uma delas fosse um box independente. Isso porque, segundo o Box Model CSS, quando um container possui um elemento nível de bloco, todos os conteúdos inseridos diretamente no container se comportam como elementos nível de bloco e são chamados de boxes anônimos.

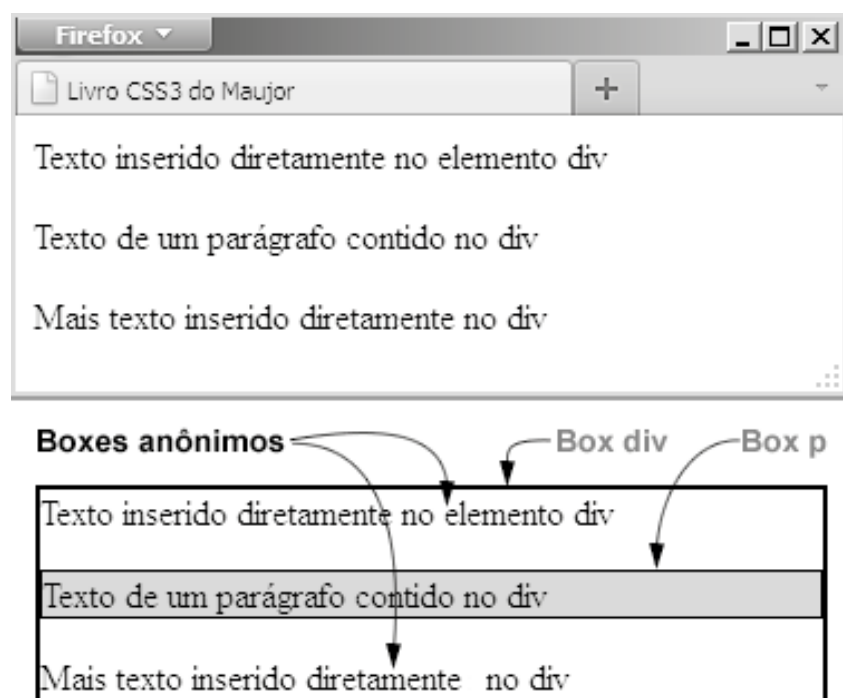


Figura 2.1 – Boxes nível de bloco anônimos.

No caso da figura mostrada, tudo se passa como se o container tivesse como elementos-filhos três boxes nível de bloco independentes.

Convém ressaltar que não há como casar blocos anônimos com seletores CSS. As regras de estilo aplicáveis aos blocos anônimos são aquelas herdadas dos seus elementos ancestrais. Por exemplo: se definirmos uma cor para o elemento `div`, os dois blocos anônimos herdarão a cor definida; contudo, não se pode definir uma cor para um dos blocos anônimos e outra cor para o outro bloco.

2.1.3 Box inline anônimo

O conceito de box inline anônimo é mais bem entendido com uso de um exemplo. Considere a marcação a seguir: `<div>`

```
Texto no elemento div
<b>palavras em negrito</b>
mais texto no div
</div>
```

A renderização da marcação e os boxes do modelo CSS são mostrados na figura 2.2.

Observe que a renderização das três linhas de texto se deu em uma linha única. Isso porque, segundo o Box Model CSS, quando um container possui um elemento inline, todos os conteúdos inseridos diretamente no container se comportam como elementos inline e são chamados de boxes anônimos.

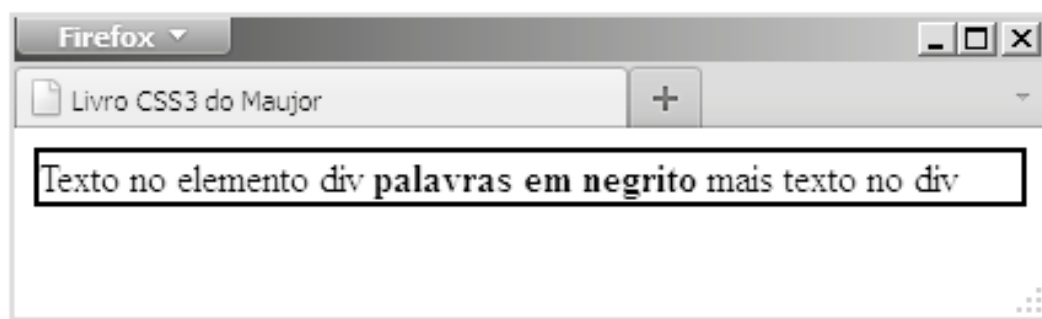


Figura 2.2 – Boxes inline anônimos.

Convém ressaltar que não há como casar blocos anônimos com seletores CSS. As regras de estilo aplicáveis aos blocos anônimos são aquelas herdadas dos seus elementos ancestrais. Por exemplo: se definirmos uma cor para o elemento `div`, os dois blocos anônimos herdarão a cor definida; contudo, não se pode definir uma cor para um dos blocos anônimos e outra cor para o outro bloco.

2.1.4 Diagrama do Box Model

Cada um dos elementos da marcação HTML possui propriedades CSS cujos valores podem ou não estar definidos por padrão pelos mecanismos nativos do navegador e/ou serem definidos em regras CSS explicitamente declaradas pelo desenvolvedor. Para ilustrar esse conceito, vamos tomar como exemplo o elemento HTML `h1` – cabeçalho de nível 1. Observe a marcação a seguir: ► **HTML**

```
...
<body>
<h1>Box Model das CSS</h1>
</body>
...
```

A renderização dessa marcação, em diferentes navegadores, é como mostrada na

figura 2.3.

Note que para todos os navegadores a fonte padrão de renderização foi a fonte genérica *Serif* e o tamanho da fonte é o mesmo. Na figura, destacamos no navegador Safari a largura para as margens superior e esquerda, que também estão presentes nos demais navegadores.

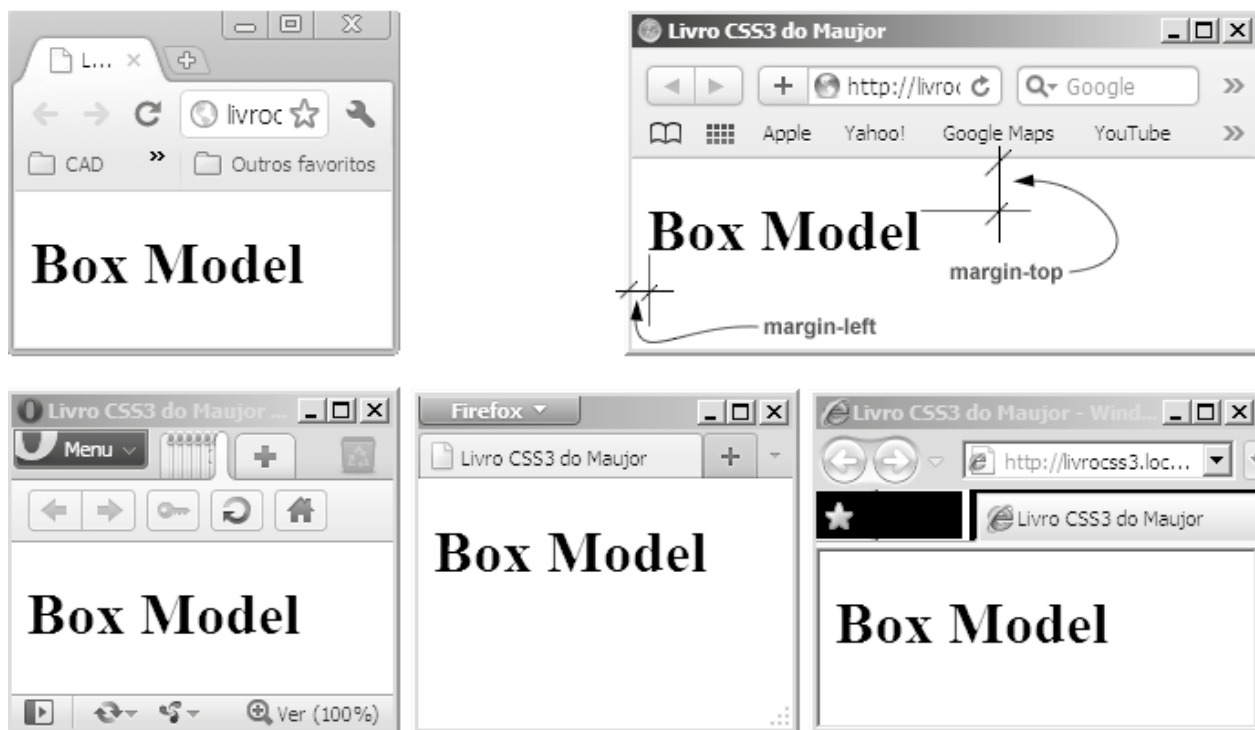


Figura 2.3 – Propriedades CSS padrão.

Todos os elementos de marcação possuem o chamado *valor inicial* para as propriedades CSS (ver apêndice A – Propriedades CSS). No nosso exemplo, podemos observar os valores iniciais para as propriedades CSS **font-family**, **font-size** e **margin-top** do elemento **h1**, bem como o valor de **margin-left** para o elemento **body**.

Aparentemente, essas propriedades têm o mesmo valor inicial para todos os navegadores mostrados, mas isso não acontece para todas as propriedades CSS e para todos os elementos de marcação. Os valores iniciais podem variar de acordo com o navegador e isso pode acarretar quebras de layout. Quando verificar seus projetos CSS em diferentes navegadores, não se esqueça de levar esse comportamento em consideração.

Eric Meyer criou e disponibilizou, para uso público livre, uma folha de estilos denominada *CSS Reset*, cuja finalidade é zerar o valor de todas as propriedades CSS para os elementos de marcação, tornando a renderização deles rigorosamente igual em qualquer navegador. Usar tal folha de estilo permite que você redefina todas as propriedades CSS para seu projeto. Isso pode ser vantajoso ou não, depende de cada caso, e somente a você cabe decidir sobre a conveniência de seu uso. Para consultar a

folha de estilos, busque no Google “CSS Reset”.

Já dissemos que o Box Model constrói o layout de um documento transformando cada elemento da árvore do documento em um box cujas dimensões e cujos comportamentos são definidos pelos valores das propriedades CSS.

As propriedades CSS que determinam as dimensões dos boxes são **margin**, **border**, **padding**, **width** e **height**; e as palavras para referência aos quatro lados dos boxes são **top**, **right**, **bottom**, **left**, tudo conforme mostrado no diagrama clássico de representação do Box Model reproduzido na figura 2.4.

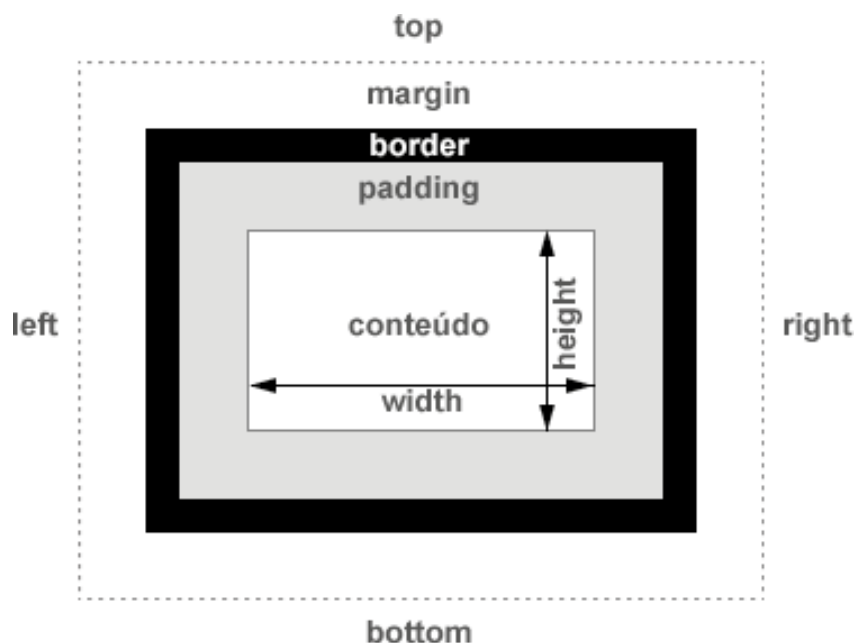


Figura 2.4 – Áreas no Box Model CSS.

No diagrama mostrado, destacamos quatro áreas em volta de cada um dos quatro lados do box, cada uma das áreas definida por uma das propriedades CSS citadas.



Neste livro, traduziremos a palavra **padding** para enchimento. Assim, as frases “o padding do elemento é 20px” e “o enchimento do elemento é 20px” têm o mesmo significado.

A área das margens é sempre transparente.

A borda tem sua cor conforme definida em regra CSS para a propriedade **border**. Se for definida uma regra CSS para a borda e omitida na regra a cor da borda, ela terá a mesma cor definida para o conteúdo do box.

Considere a marcação HTML mostrada anteriormente e transcrita a seguir: ► **HTML**

```
...
<body>
<h1>Box Model das CSS</h1>
</body>
...
```

Considere ainda a seguinte regra de estilo a ser aplicada ao elemento **h1** da

marcação mostrada: ► **CSS**

```
h1 {  
margin: 45px 40px 10px 50px;  
border-top: 5px solid #000;  
border-right: 10px solid #000;  
border-bottom: 8px solid #ccc;  
border-left: 20px solid #ccc;  
padding: 15px 30px 10px 40px;  
}
```

Na figura 2.5, mostramos a renderização da marcação e o diagrama do Box Model correspondente. Note que não definimos as propriedades `width` e `height` e elas assumem seus valores iniciais.

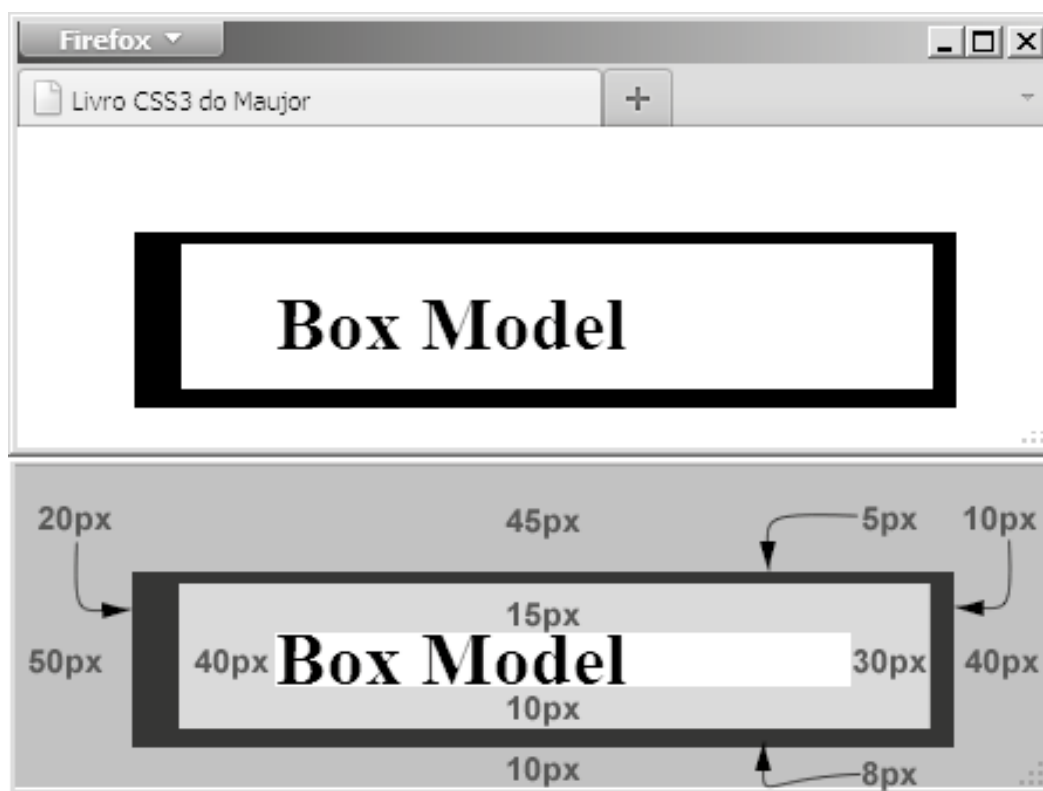


Figura 2.5 – Exemplo de Box Model CSS.

2.1.4.1 Propriedades CSS do Box Model

Mostramos as propriedades CSS que determinam a criação do box de um elemento da marcação. Vejamos, a seguir, as diretrizes e os detalhes para criar regras de estilo para essas propriedades.

margin

Essa propriedade define a espessura das margens de um box. Podemos definir a espessura com uso de declarações individuais ou com uso de uma declaração

abreviada. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar margens individualmente: ► **CSS**

```
h1 {  
  margin-top: 45px;  
  margin-right: 40px;  
  margin-bottom: 10px;  
  margin-left: 50px;  
}
```

e a seguir a sintaxe da regra CSS para declarar as mesmas margens de uma forma abreviada: ► **CSS**

```
h1 {  
  margin: 45px 40px 10px 50px;  
}
```

As duas sintaxes mostradas são equivalentes e produzem o mesmo resultado.



Várias propriedades CSS admitem uma forma abreviada de declaração de valores. Por padrão, quando usamos uma declaração abreviada para qualquer propriedade CSS que a suporte, os quatro valores obedecem à seguinte ordem: **top**, **right**, **bottom** e **left**.

Declarações de valores CSS na sua forma abreviada podem ser feitas com uma lista de um, dois, três ou quatro valores separados por espaço em branco, e seus significados são conforme mostrado a seguir: • **um valor**: o valor único é válido para os quatro lados do box, **top**, **right**, **bottom** e **left**; • **dois valores**: o primeiro é válido para os lados **top** e **bottom**, e o segundo, para **right** e **left**; • **três valores**: o primeiro é válido para o lado **top**, o segundo, para os lados **right** e **left**, e o terceiro, para o lado **bottom**; • **quatro valores**: os valores são válidos, respectivamente, para os lados **top**, **right**, **bottom** e **left** do box.

padding

Essa propriedade define a espessura do enchimento de um box. Chamamos de enchimento (ou **padding**) a espessura da área que separa a borda do conteúdo. Podemos definir o enchimento com uso de declarações individuais ou com uso de uma declaração abreviada. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar **padding** individualmente: ► **CSS**

```
h1 {  
  padding-top: 15px;  
  padding-right: 30px;  
  padding-bottom: 10px;
```

```
padding-left: 40px;
```

```
}
```

e, a seguir, a sintaxe da regra CSS para declarar **padding** de uma forma abreviada: •
CSS

```
h1 {  
padding: 15px 30px 10px 40px;
```

```
}
```

As duas sintaxes mostradas são equivalentes e produzem o mesmo resultado.

border

Essa propriedade define a espessura, o estilo e a cor da borda de um box. Podemos definir cada uma dessas três características da borda individualmente e para cada lado do box, com uso de declarações individuais, ou usar uma declaração abreviada. Essa propriedade admite as seguintes declarações individuais.

border-width

Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a espessura da borda individualmente: ► **CSS**

```
h1 {  
border-top-width: 5px;  
border-right-width: 10px;  
border-bottom-width: 8px;  
border-left-width: 20px;
```

```
}
```

e, a seguir, a sintaxe da regra CSS para declarar a espessura da borda de uma forma abreviada: ► **CSS**

```
h1 {  
border-width: 5px 10px 8px 20px;
```

```
}
```

As duas sintaxes mostradas são equivalentes e produzem o mesmo resultado.

Como explicado anteriormente, o uso da declaração abreviada nos permite declarar um, dois, três ou quatro valores.

border-style

Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar o estilo da borda individualmente: ► **CSS**

```
h1 {  
border-top-style: solid;  
border-right-style: dashed;  
border-bottom-style: dotted;  
border-left-style: ridge;  
  
}
```

e, a seguir, a sintaxe da regra CSS para declarar o estilo da borda de uma forma abreviada: ► **CSS**

```
h1 {  
border-style: solid dashed dotted ridge;  
  
}
```

As duas sintaxes mostradas são equivalentes e produzem o mesmo resultado.

Como explicado, quando mostramos a propriedade aqui também podemos declarar um, dois, três ou quatro valores.

Os estilos de bordas previstos nas especificações para as CSS3 são:

Estilo	Descrição
none	Ausência de borda. Espessura e cor, se declaradas, são ignoradas. Equivale a bordar com espessura zero, exceto se a borda é uma imagem (ver capítulo sobre bordas).
hidden	O mesmo que none , porém com comportamento diferente para bordas de tabela sobrepostas.
dotted	Borda pontilhada.
dashed	Borda tracejada.
solid	Borda sólida contínua.
double	Borda em linha dupla. A soma da espessura de cada linha com a do espaço que as separa é igual à espessura total declarada para a borda (border-width).
groove	Borda entalhada.
ridge	Borda em relevo.
inset	Borda em baixo-relevo.
outset	Borda em alto-relevo.

Por padrão, as bordas são renderizadas sobre o fundo do box e atrás do seu conteúdo quando há sobreposição do conteúdo à borda.

border-color

Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a cor da borda individualmente: ► **CSS**

```
h1 {  
  border-top-color: #f00; /* cor vermelha */  
  border-right-color: #0f0; /* cor verde */  
  border-bottom-color: #00f; /* cor azul */  
  border-left-color: #000; /* cor preta */  
}
```

e, a seguir, a sintaxe da regra CSS para declarar a cor da borda de uma forma abreviada: ► **CSS**

```
h1 {  
  border-color: #f00 #0f0 #00f #000;  
}
```

As duas sintaxes mostradas são equivalentes e produzem o mesmo resultado.

Como explicado anteriormente, o uso da declaração abreviada nos permite declarar um, dois, três ou quatro valores.

border

Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar as três propriedades da borda individualmente: ► **CSS**

```
h1 {  
  border-top: 2px solid #f00;  
  border-right: 3px inset #0f0;  
  border-bottom: 5px ridge #00f;  
  border-left: 1px outset #000;  
}
```

e, a seguir, a sintaxe da regra CSS para declarar as três características da borda de uma forma abreviada: ► **CSS**

```
h1 {  
  border: 1px dotted #000;  
}
```

Nesse caso, as duas sintaxes mostradas produzem resultados diferentes. A primeira declara as três características de cada borda de forma diferenciada e a segunda declara três características idênticas para as quatro bordas.

Observe as regras para estilização de margens mostradas a seguir: ► **CSS**

```
h1 {  
border-top-width: 1px;  
border-right-width: 2px;  
border-bottom-width: 3px;  
border-left-width: 4px;  
border-style: solid;  
border-color: #000;  
}
```

► **CSS**

```
h1 {  
border: solid #000;  
border-width: 1px 2px 3px 4px;  
}
```

As duas regras de estilo, mostradas anteriormente, produzem o mesmo efeito. Assim, sempre que possível, abrevie declarações de estilo que suportam a sintaxe abreviada.

width

Essa propriedade define a largura de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a largura. No Box Model, essa propriedade define a largura da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► **CSS**

```
div {  
width: 500px;  
}
```

height

Essa propriedade define a altura de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a altura. No Box Model, essa propriedade define a altura da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► CSS

```
div {  
  height: 300px;  
  
}
```

max-width

Essa propriedade define a largura máxima de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a largura máxima. No Box Model, essa propriedade define a largura máxima da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► CSS

```
div {  
  max-width: 960px;  
  
}
```

max-height

Essa propriedade define a altura máxima de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a altura máxima. No Box Model, essa propriedade define a altura máxima da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► CSS

```
div {  
  max-height: 768px;  
  
}
```

min-width

Essa propriedade define a largura mínima de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a largura mínima. No Box Model, essa propriedade define a largura mínima da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► CSS

```
div {  
  min-width: 600px;  
  
}
```

min-height

Essa propriedade define a altura mínima de um box. Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar a altura mínima. No Box Model, essa propriedade define a altura mínima da área de conteúdo do box, isto é, não inclui as espessuras de **margin**, **border** e **padding**.

► CSS

```
div {  
  min-height: 450px;
```

```
}
```



Ressaltamos anteriormente que os valores das propriedades **width** e **height** de um box não incluem as espessuras de **margin**, **border** e **padding**. Essa forma de computar as dimensões do box era imutável até as CSS2.1.

O módulo das CSS3 denominado Basic User Interface Level 3 (CSS3 UI) criou uma propriedade denominada **box-sizing**, que admite os valores **content-box**, **border-box** e **padding-box** que computam as dimensões do box como sendo, respectivamente, somente a área de conteúdo, a área de conteúdo mais **padding** e mais bordas e a área de conteúdo mais **padding**.

O valor padrão é **content-box**, que é, conforme o Box Model das CSS2.1, o valor **border-box** quando declarado para um box faz com que as espessuras de **border** e **padding** sejam computadas nas dimensões do box é suportado por todos os navegadores modernos.

O valor **padding-box** que está marcado na especificação como uma funcionalidade sob risco, ou seja, ainda não foi decidido se será ou não implementado, não é suportado por nenhum navegador à época em que esta edição do livro foi escrita.

display

Essa propriedade define o tipo de box a ser gerado para um elemento. Os valores possíveis para essa propriedade são os mostrados a seguir:

Valor	Descrição
inline	Define boxes inline.
block	Define boxes nível de bloco.
inline-block	Define boxes nível de bloco dispostos no fluxo como boxes inline.
list-item	Define boxes tipo item de lista (elemento li).

run-in	Define boxes de acordo com o contexto. Se o box que se segue for nível de bloco run-in , define um box inline, caso contrário, define um box nível de bloco.
compact	Define boxes de acordo com o contexto. Se o box que se segue for nível de bloco compact , define um box nível de bloco disposto no fluxo como box inline, caso contrário, define um box nível de bloco.
none	Define que o box não será criado para o elemento.

Os valores **table**, **inline-table**, **table-row-group**, **table-header-group**, **table-footer-group**, **table-row**, **table-column-group**, **table-column**, **table-cell**, **table-caption** são definidos no módulo para Tabelas das CSS3. Os valores **ruby**, **ruby-base**, **ruby-text**, **ruby-base-group**, **ruby-text-group** são definidos no módulo Ruby das CSS3. Os módulos para Layout das CSS3 também definem valores para essa propriedade.

Observe, no exemplo a seguir, a sintaxe da regra CSS para declarar essa propriedade.

► **CSS**

```
p {
  display: run-in;

}
```

CAPÍTULO 3

Posicionamento CSS

Neste capítulo, estudaremos os esquemas de posicionamento CSS. Veremos como posicionar elementos em uma página com uso das propriedades `position`, `float` e `z-index`.

Estabeleceremos as regras CSS de posicionamentos *fixo*, *absoluto* e *relativo* como forma de deslocar boxes de sua posição padrão no fluxo do documento. Veremos também a técnica de emprego das propriedades `float` e `clear` para controlar posicionamento. Terminaremos o capítulo explorando os conceitos e o emprego da propriedade `z-index` para controlar o empilhamento de boxes que se sobrepõem.

3.1 Esquemas de posicionamento CSS2.1

As propriedades CSS `position` e `float` determinam como um box será posicionado no modelo visual de formatação de um documento. As Recomendações do W3C para as CSS2.1 preveem a existência de três esquemas de posicionamento: *normal*, *float* e *absoluto*.

Cada esquema é regido por regras próprias, mas com folhas de estilo podemos definir diferentes esquemas de posicionamento para os boxes de um documento.

Elementos nível de bloco formam boxes – ver [2.1] – que denominaremos simplesmente de bloco. Elementos inline formam boxes inline. Assim, para melhor entendimento dos conceitos contidos neste capítulo, considere que há dois tipos de boxes: os blocos que são formados pelos elementos nível de bloco, tais como `div`, `h1-h6`, `p`, `blockquote` etc.; e os boxes inline, tais como `strong`, `b`, `em`, `img` etc.

As especificações para as CSS3 criaram outros esquemas de posicionamento que visam a facilitar a construção de layouts sem as restrições apresentadas pelos esquemas de posicionamento das CSS2.1 que estudaremos neste capítulo.

Os novos esquemas de posicionamento das CSS3, que já podem ser testados, serão estudados com detalhes nos capítulos 16 e 17.

3.1.1 Esquema normal

Esquema normal ou fluxo normal é o esquema padrão de posicionamento dos boxes. Segundo as regras desse esquema, elementos nível de bloco são colocados em sequência vertical, dentro do elemento que os contém e de cima para baixo, um após outro na ordem em que aparecem no fluxo do documento, isto é, na marcação HTML.

3.1.1.1 Elementos nível de bloco

A distância vertical entre cada elemento nível de bloco adjacente é determinada pela propriedade `margin`. Não tendo sido definida, por regras de estilo, uma margem vertical para o bloco, será tomada a margem inicial padrão, própria de cada elemento que forma o bloco. Margens verticais entre blocos adjacentes sempre se sobrepõem (collapse), ou seja, em lugar de somar a margem inferior de um bloco com a margem superior do bloco que lhe segue para obter a margem final, esta será tomada como sendo a maior entre as duas, conforme mostrado no esquema da figura 3.1. No esquema, mostramos dois boxes que se seguem na marcação HTML.

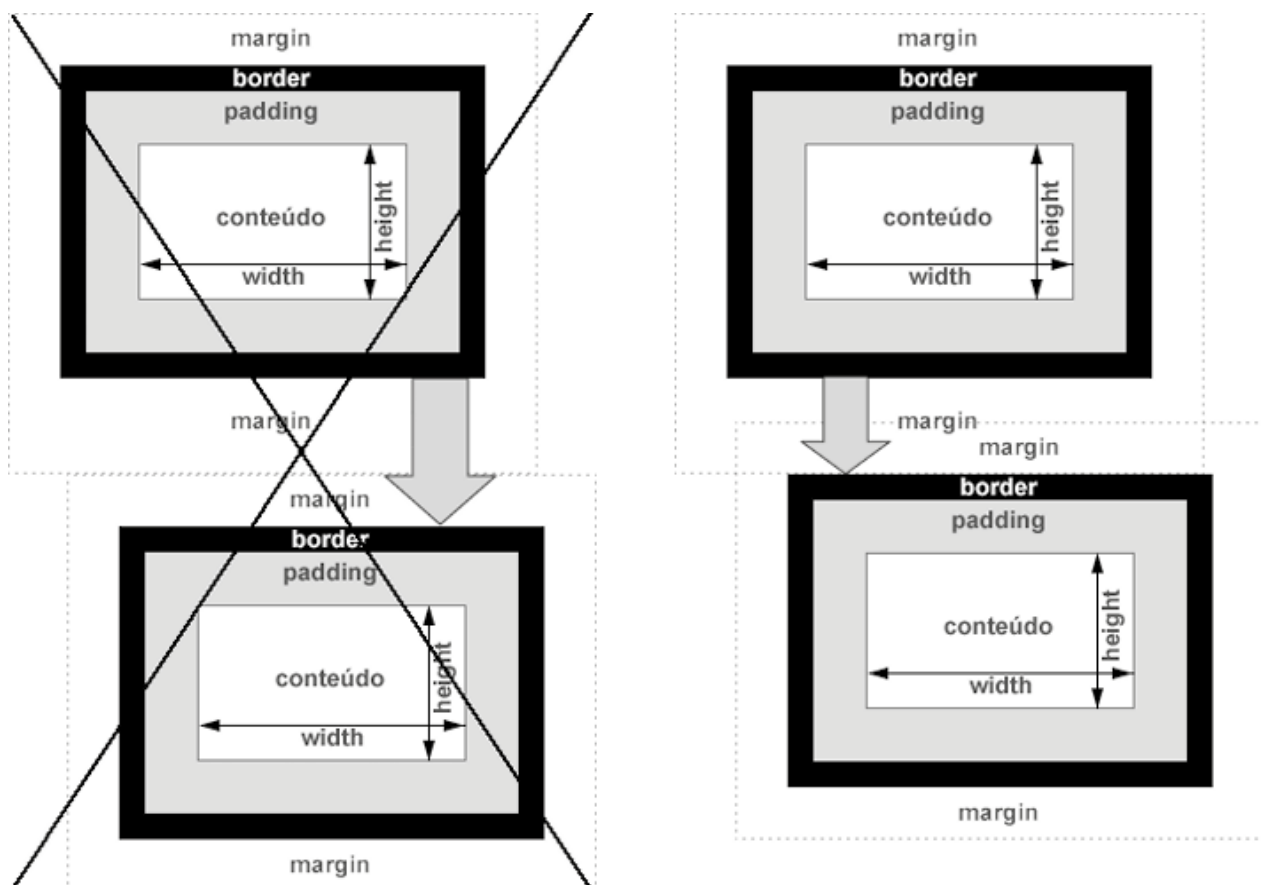


Figura 3.1 – Sobreposição de margens.

3.1.1.2 Elementos inline

Para elementos inline, a formatação se dá em linha (na horizontal) e dentro do bloco que contém o box inline. A largura de um box inline é limitada pela propriedade `width` do seu bloco container. Excedida aquela largura, o conteúdo do box inline passa para a linha seguinte.

Observe a marcação a seguir:

```
<p>Este texto contém <em>quatro</em> elementos <b>inline</b></p>
```

Trata-se de um parágrafo `p` contendo um elemento `em` e um elemento `b`. Podemos identificar quatro boxes inline contidos no parágrafo. Os conteúdos dos quatro boxes e seus tipos – ver [2.1] – são:

- Este texto contém (bloco

anônimo)

- quatro (bloco inline)
- elementos (bloco anônimo)
- inline (bloco inline)

O texto contido no parágrafo será formatado de acordo com a largura definida para o parágrafo. Nas renderizações mostradas na figura 3.2, foram definidas quatro larguras diferentes para o parágrafo, causando diferentes formatações visuais.

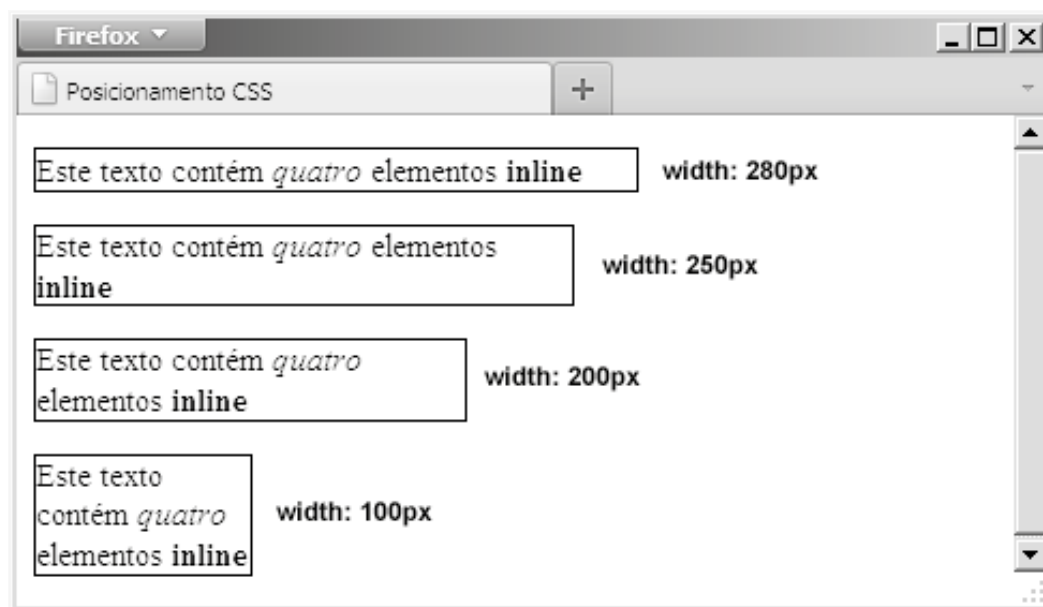


Figura 3.2 – Boxes inline.

3.1.1.3 Posicionamento relativo

A declaração CSS `position: relative` sozinha não causa nenhum efeito no posicionamento do bloco. Quando usada em conjunto com as propriedades `top`, `right`, `bottom` e `left`, movimentam o bloco da sua posição inicial de uma distância definida pelos valores declarados nessas propriedades. Medidas de comprimento CSS são empregadas para definir as coordenadas do deslocamento.

As propriedades `left` e `right` definem o quanto o bloco deve ser deslocado para a direita ou esquerda, respectivamente, e as propriedades `top` e `bottom` definem o deslocamento para baixo ou para cima, respectivamente.

Na figura 3.3, é mostrado o deslocamento de um bloco para o qual foi declarada a seguinte regra CSS de posicionamento relativo: `div.dois {`

```
position: relative;
left: 180px;
top: 30px;
```

```
}
```

Note que o bloco deslocado com a declaração `position: relative` sai da posição padrão, mas o espaço que ele ocupava naquela posição é preservado.

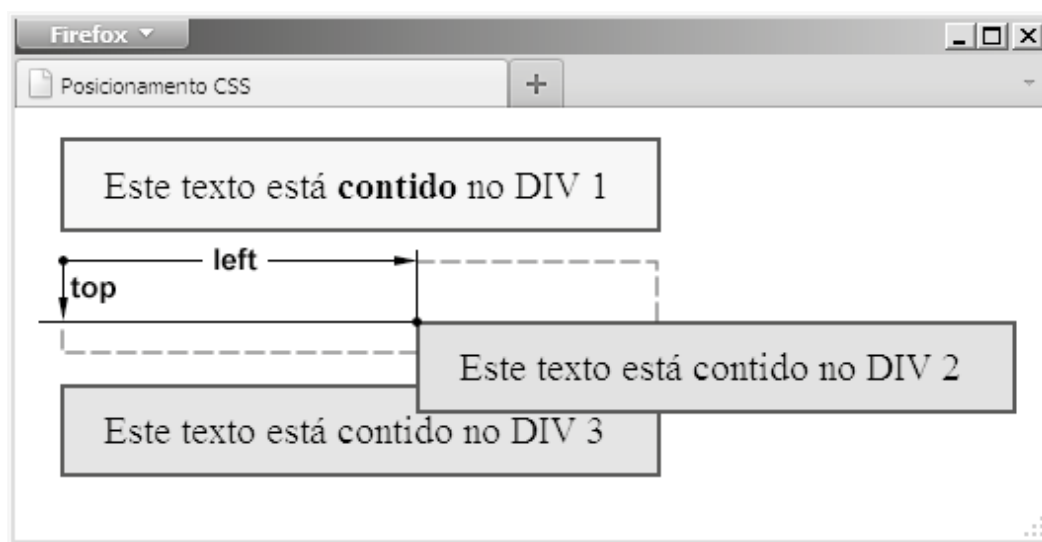


Figura 3.3 – Posicionamento relativo.

Elementos inline também podem ser deslocados da sua posição padrão com uso dessa declaração. Seguem-se a marcação e a regra de estilo que demonstram esse deslocamento: ► **HTML**

```
<p>Neste texto <strong>estas palavras</strong> foram deslocadas de sua posição.</p> ► CSS
```

```
strong {  
position: relative;  
left: 60px;  
top: 40px;
```

```
}
```

Na figura 3.4, é mostrado o efeito do deslocamento de um elemento inline conforme os códigos anteriores.

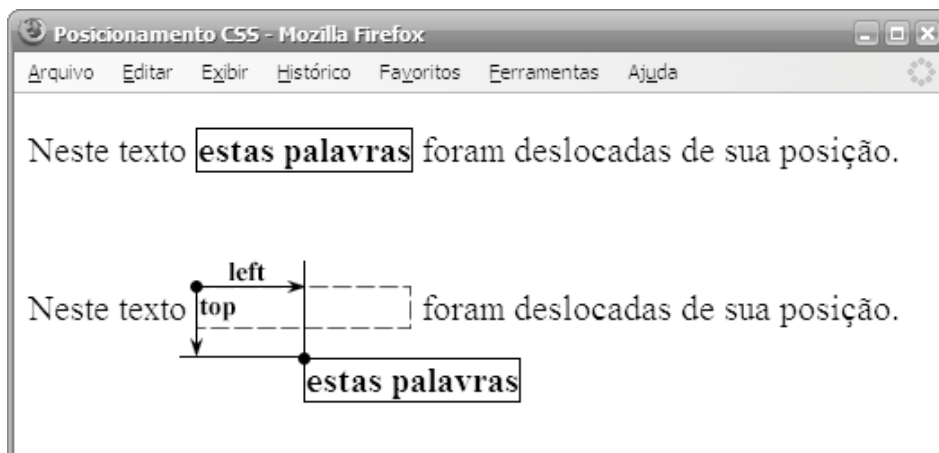


Figura 3.4 – Posicionamento relativo inline.

3.1.2 Esquema float

O esquema *float* de posicionamento é definido pela propriedade CSS `float` e seus valores `left`, `right`, `none` e `inherit` (herdado). Outra propriedade usada nesse esquema é a propriedade `clear` e seus valores `none`, `left`, `right`, `both` e `inherit` (herdado). Segundo as regras desse esquema, o box é retirado de sua posição no fluxo do documento e flutuado para a direita ou para a esquerda. O espaço original ocupado pelo box não será deixado livre, sendo preenchido pelo elemento que lhe segue no fluxo do documento.

3.1.2.1 Flutuando elementos nível de bloco

Considere vários blocos dentro de um mesmo container. O primeiro bloco flutuado desloca-se lateralmente até tocar as bordas lateral e superior do elemento que o contém. O segundo desloca-se até tocar a borda do primeiro e assim sucessivamente enquanto houver espaço horizontal no elemento container. Faltando espaço, o próximo bloco ocupará uma linha abaixo e assim por diante. Na figura 3.5, é mostrada uma ilustração da regra para flutuar blocos. À esquerda, o diagrama com seis blocos na posição padrão, e à direita, o resultado de declarar `float: left` para todos os blocos.

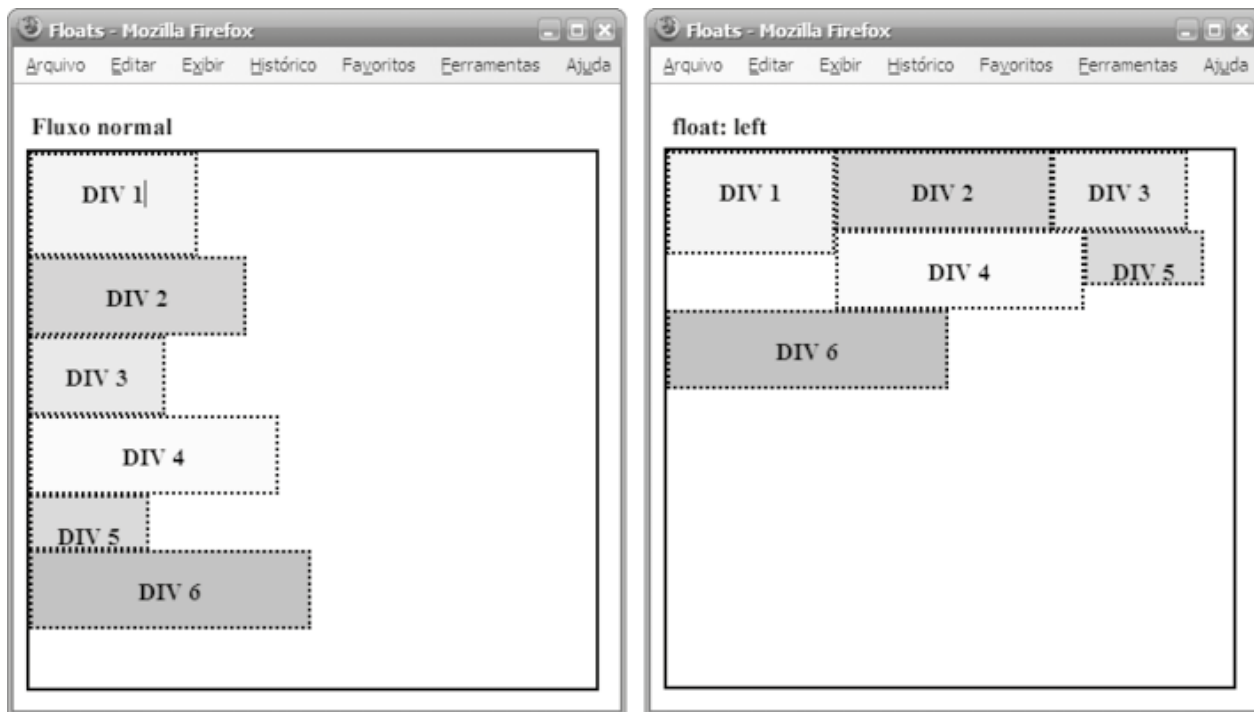


Figura 3.5 – Posicionamento *float*.

Uma aplicação prática do comportamento de uma sequência de blocos flutuados à esquerda, como mostrado anteriormente, é na construção de uma barra de navegação. Mecanismos de navegação em geral são marcados com o elemento HTML `lista`, nas quais cada item é um link, como mostrado a seguir: ► **HTML**


```
<ul>
<li><a href="home.html">Home</a></li>
<li><a href="quemsomos.html">Quem Somos</a></li> <li><a
href="portfolio.html">Portfólio</a></li> <li><a
href="contato.html">Contato</a></li>
</ul>
```

Em uma lista não ordenada, como a mostrada anteriormente, o elemento `ul` é o container para os elementos `li`.

Os elementos `li` são boxes nível de bloco e, como tal, são renderizados um em cada linha, formando uma barra de navegação vertical.

Se definirmos uma largura para o elemento `ul` suficiente para acomodar a soma das larguras dos elementos `li` e flutuarmos à esquerda esses elementos, iremos obter uma barra de navegação horizontal.

Na figura 3.6, são mostrados três DIVs no fluxo normal do documento e o comportamento do modelo de formatação *float* quando o DIV 1 é flutuado à direita. Note que, ao ser flutuado, o DIV 1 encosta na borda direita do elemento container. Os DIVs 2 e 3 sobem e ocupam o lugar deixado vago pelo DIV 1.

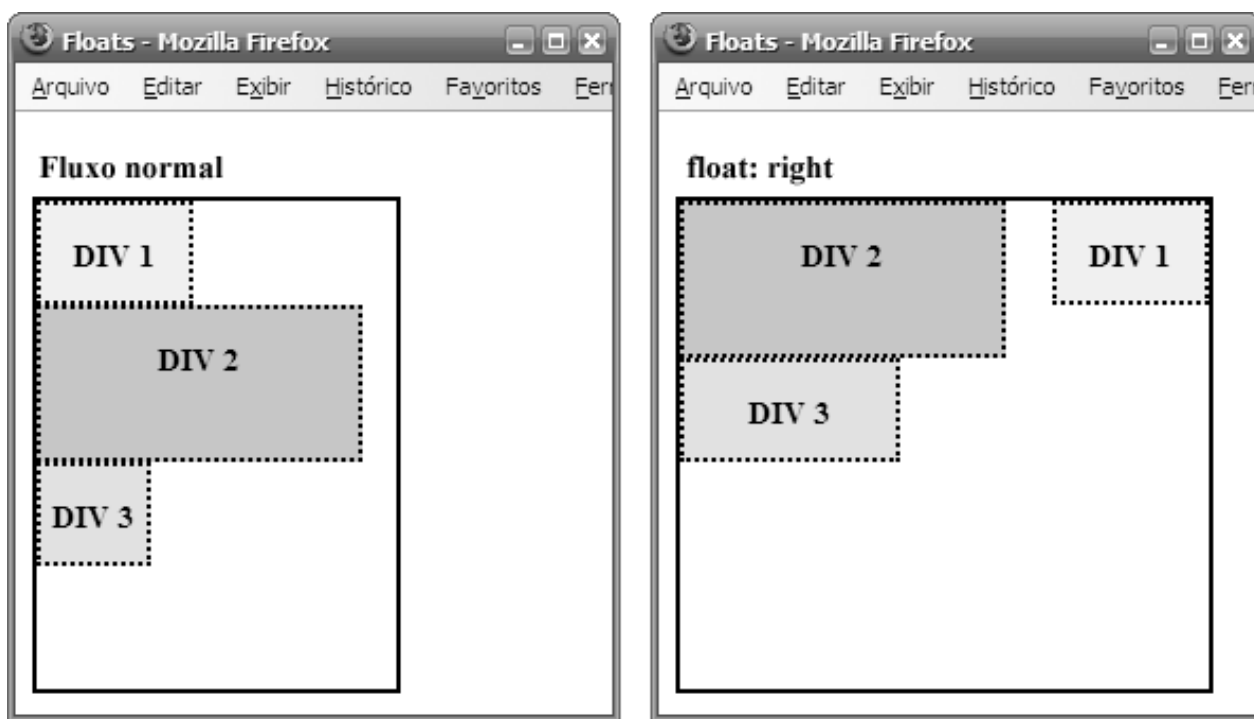


Figura 3.6 – Flutuando à direita.

Na figura 3.7, é mostrado o comportamento do modelo de formatação *float* quando o DIV 1 é flutuado à esquerda. Note que, ao ser flutuado, o DIV 1 encosta na borda esquerda do elemento container e é retirado do fluxo normal do documento. Em consequência, os DIVs 2 e 3 sobem e ocupam o lugar deixado vago pelo DIV 1.

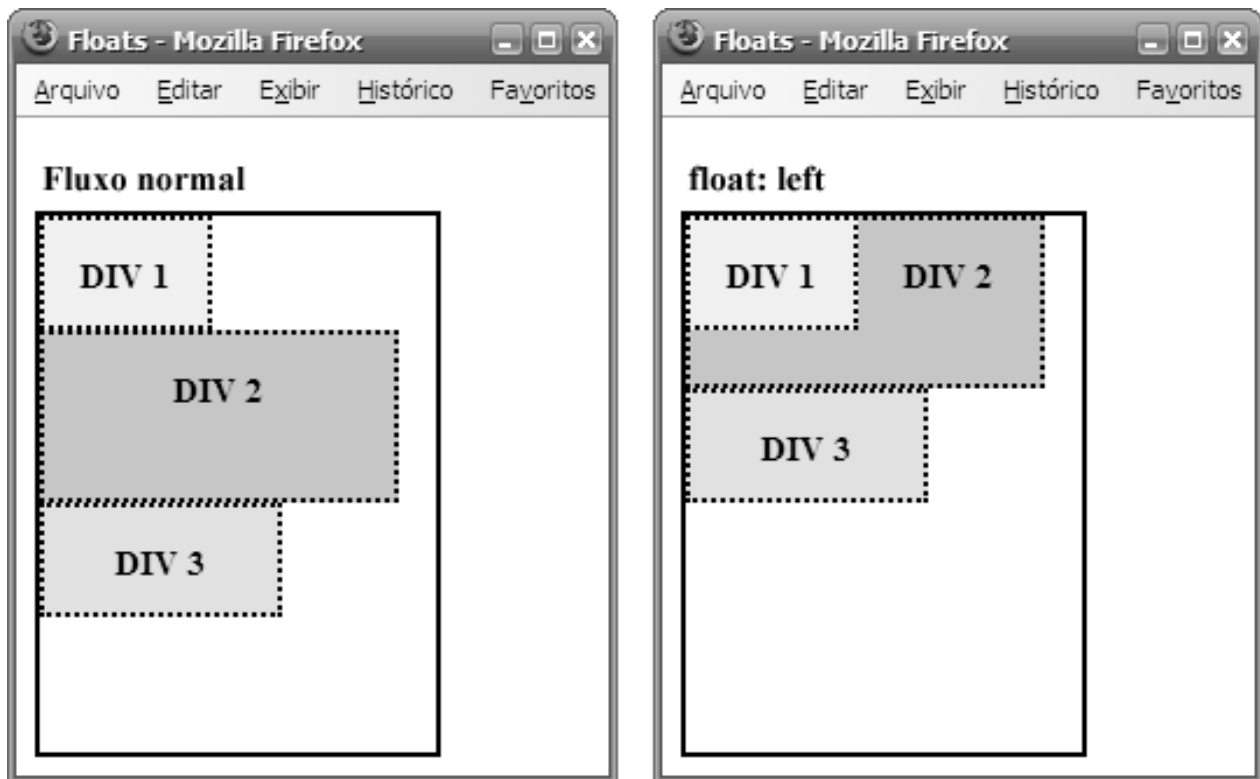


Figura 3.7 – Flutuando à esquerda.

3.1.2.2 Flutuando elementos inline

Vamos tomar como exemplo o caso de uma imagem dentro de um parágrafo. O elemento container é o parágrafo e o elemento inline a flutuar é a imagem. Considere dois parágrafos no fluxo do documento e a imagem no primeiro parágrafo conforme a marcação mostrada a seguir: ► **HTML**

```
<p>Lorem ipsum dolor. ...<p>  
<p>Suspendisse potenti. ...</p>
```

Na figura 3.8, é mostrada a renderização da marcação.

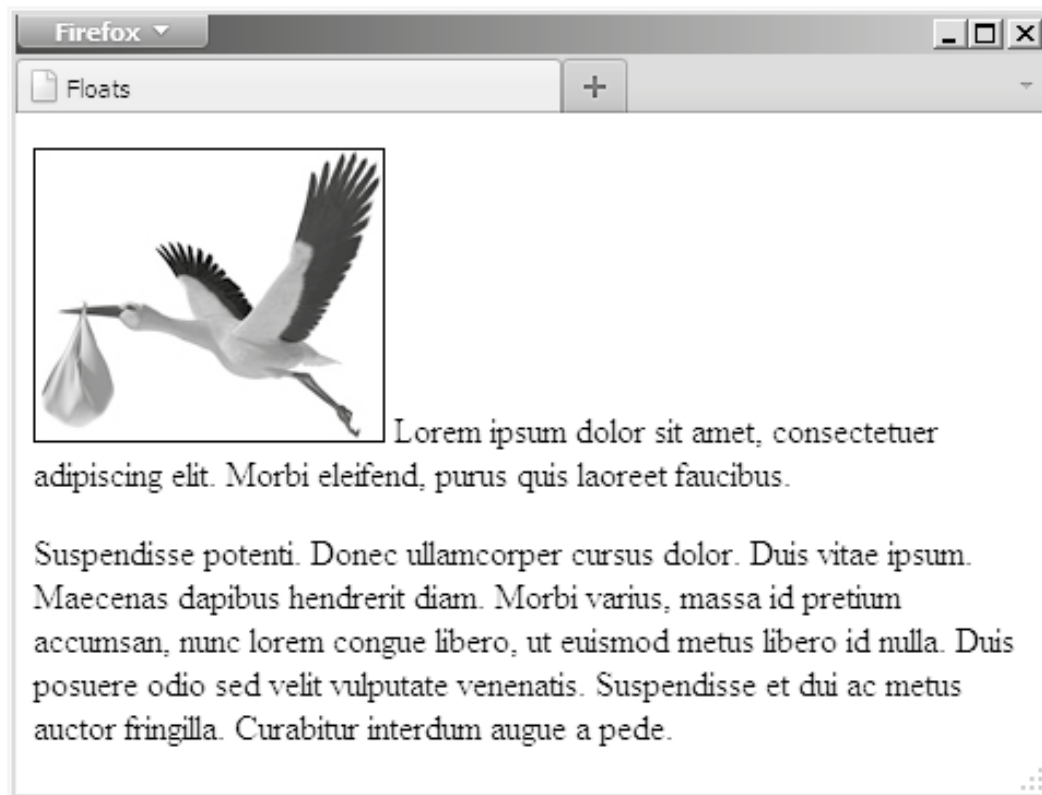


Figura 3.8 – Imagem inline em parágrafo.

Para flutuar a imagem à esquerda ou à direita, aplicamos a seguinte regra CSS:

```
img { float: left; }
```

ou

```
img { float: right; }
```

Obtendo o efeito mostrado na figura 3.9:

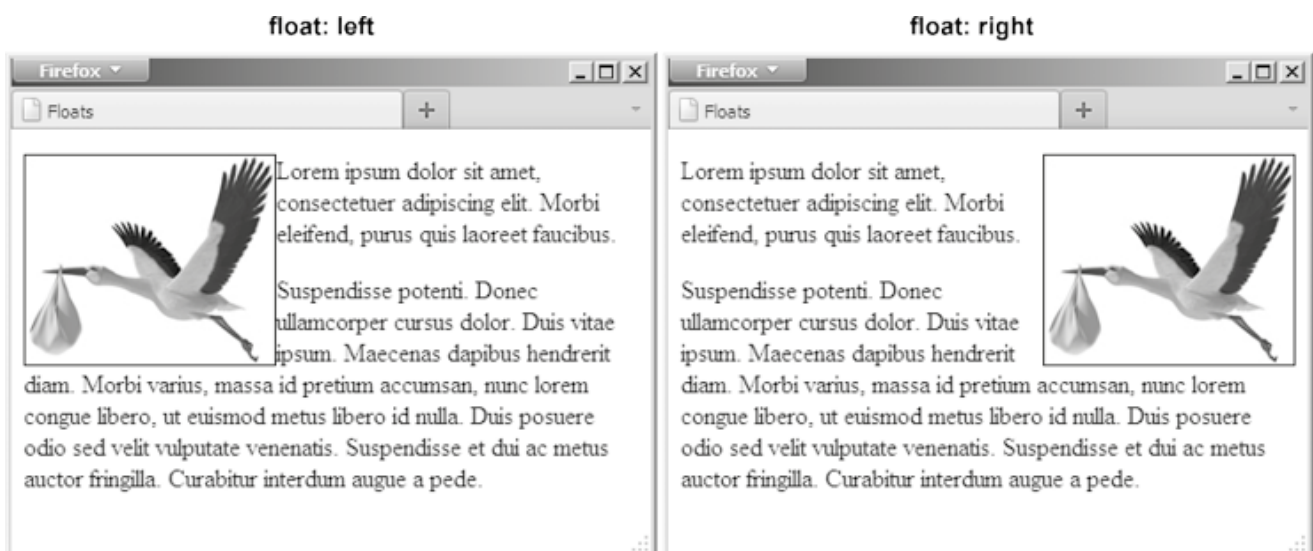


Figura 3.9 – Imagem float.

Observe que, para elementos inline, o comportamento é diferente do comportamento de blocos flutuados. A imagem flutuada foi deslocada para uma posição até tocar a borda do seu elemento container, no exemplo um parágrafo, e

tanto o texto, conteúdo do container, como o elemento bloco que se segue no fluxo do documento, o segundo parágrafo, ocuparam uma posição em volta da imagem. Na prática, esse comportamento é usado para alinhar imagens à esquerda ou à direita de um texto.

Outra aplicação prática consiste em destacar um pequeno texto relacionado ou retirado de um texto principal. Esse efeito é muito usado em mídia impressa e é conhecido como *pullquote*. O texto a destacar normalmente é uma cópia de uma citação contida no texto principal e que resume ou motiva sua leitura, ou uma citação relacionada ao texto principal, ou ainda um texto externo relacionado.

Veja um exemplo:

► HTML

```
<p>Lorem ipsum ..elit. <strong class="pullquote">Ut sollicitudin...  
</strong> Suspendisse potenti. Donec ...consectetuer.</p>
```

Marcamos o texto que servirá de *pullquote* com o elemento **strong** e a ele atribuímos uma classe. Poderíamos ter usado outro elemento inline, como **b**, **em**, **span**, **i** etc., mas preferimos **strong**, pois esse elemento é usado para dar forte ênfase a um texto e é exatamente esse o propósito de um *pullquote*.

Note, na marcação proposta, que temos um elemento inline – **strong** – dentro de um elemento container nível de bloco – **p** – parágrafo. É uma situação exatamente igual àquela mostrada anteriormente para imagens dentro de parágrafo. Basta flutuarmos o *pullquote* para obter o efeito desejado, conforme mostrado nas seguintes regras CSS: ► **CSS**

```
p {  
  text-align: justify;  
  
  }  
  
.pullquote {  
  width: 40%;  
  float: right;  
  margin: 10px 0 10px 10px;  
  padding: 5px 0 5px 5px;  
  border: solid #666;  
  border-width: 5px 0;  
  text-align: left;  
  
  }
```

Note que adicionamos margens, enchimento e bordas superior e inferior para melhorar o visual do nosso *pullquote*. Claro, você é livre para estilizar como bem

entender seus *pullquotes*.

Na figura 3.10, é mostrada a renderização do código e da estilização estudados.

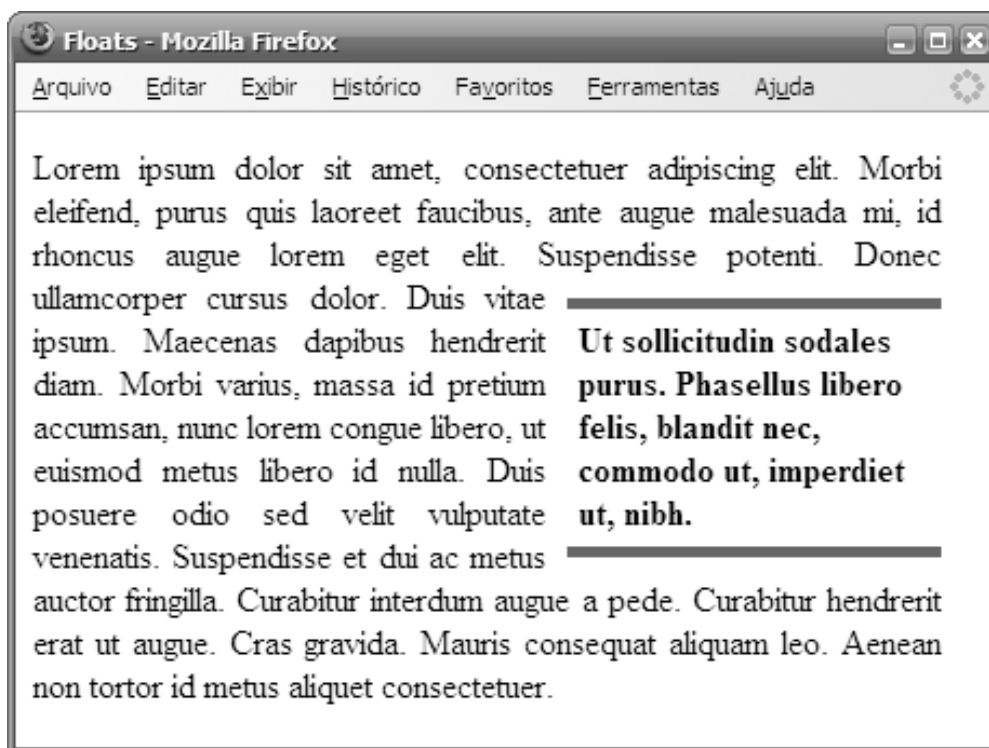


Figura 3.10 – Pullquote.

Vamos examinar o caso em que a altura do elemento inline flutuado é maior do que a altura do seu elemento container. Considere a seguinte marcação: ► **HTML**

```
<p>Lorem ...pouco texto... elit.  
<em class="flutuado">Ut ...muito texto... diam.</em>  
Aenean non tortor id metus aliquet consectetur.</p>  
<p>Curabitur ...texto... ligula.</p>
```

São dois parágrafos, sendo que, no primeiro, o elemento `em` que vai ser flutuado contém uma quantidade de texto tal que o bloco flutuado terá altura maior que o próprio parágrafo.

Vamos aplicar as seguintes regras CSS:

► **CSS**

```
p {  
width: 500px;  
border: 2px dotted #333;
```

```
}
```

```
.flutuado {  
width: 40%;  
float: right;
```

```
border: 1px solid #333;
```

```
}
```

Definimos uma largura máxima para os parágrafos somente para efeitos visuais. É como se uma coluna de texto no site tivesse aquela largura. Colocamos ainda uma borda pontilhada nos dois parágrafos e largura e borda sólida para o elemento flutuado. Na figura 3.11, é mostrado como os elementos envolvidos nesse contexto se comportam.

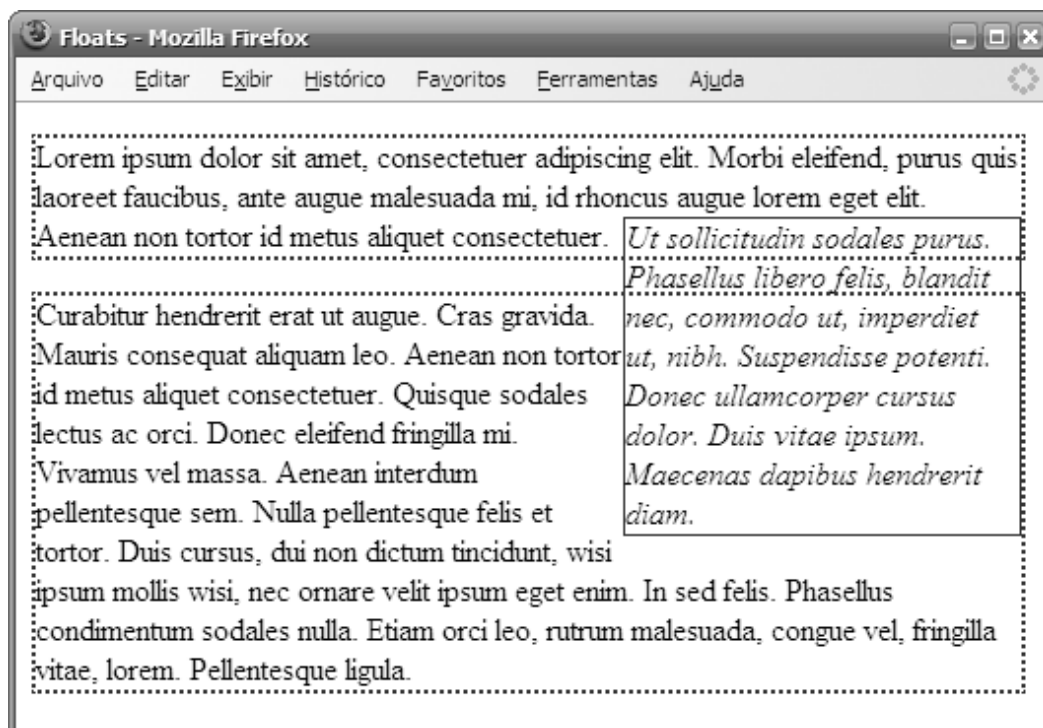


Figura 3.11 – Sobreposição de float.

Observe que o elemento flutuado ultrapassou o limite inferior do parágrafo, ou seja, pulou para fora do seu container, invadiu o elemento seguinte (o segundo parágrafo) e forçou este a encolher para se distribuir em volta dele. Esse é o comportamento previsto nas Recomendações para CSS do W3C e navegadores standards assim se comportam.

Para forçar o segundo parágrafo a se posicionar abaixo do elemento flutuado, usamos a propriedade CSS `clear`.

Adicionamos uma classe ao segundo parágrafo e acrescentamos uma regra CSS, como mostrado nos seguintes códigos: `<p class="clear">Curabitur ...texto... ligula.</p>` e nas CSS:

```
p.clear { clear: right; }
```

Na figura 3.12, é mostrado o efeito da aplicação da propriedade `clear` no segundo parágrafo.

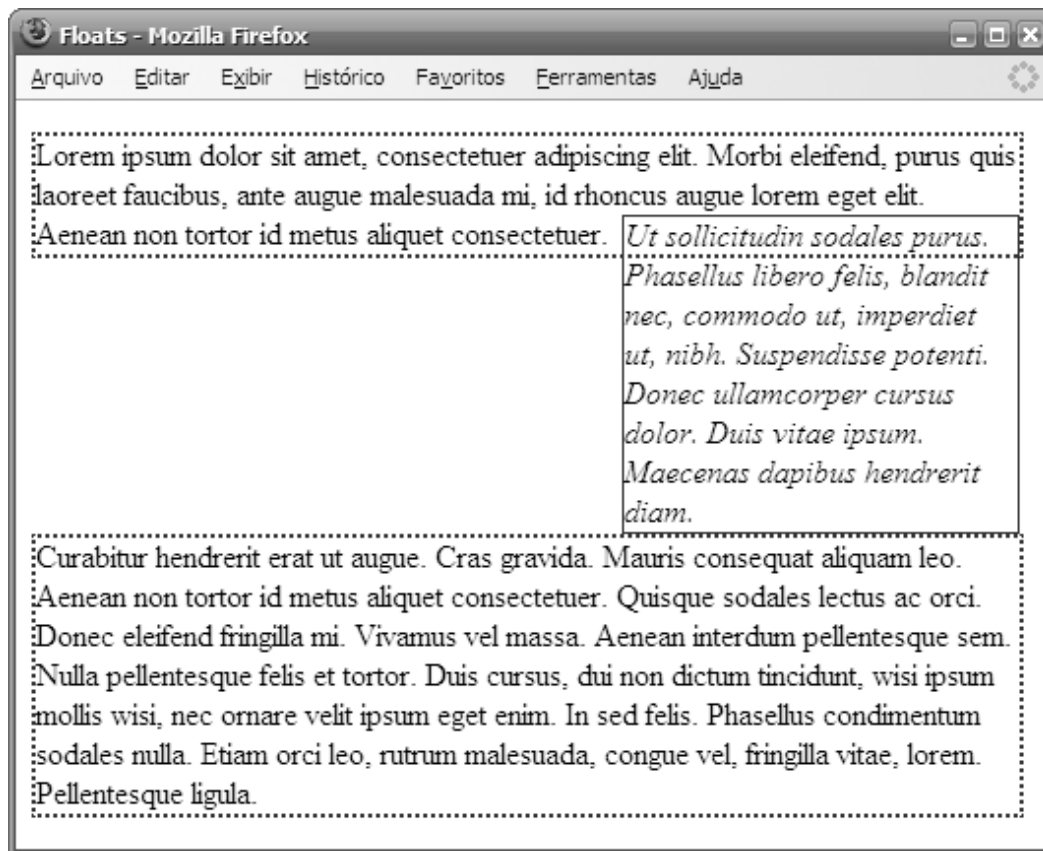


Figura 3.12 – Declaração clear: right.

Note que o elemento flutuado continua fora dos limites do seu container.

Na maioria dos casos de construção de layout com emprego de floats, nos deparamos com situação semelhante a essa, na qual seria muito interessante que o container de elementos flutuados se expandisse para contê-los. Talvez para colocar uma borda em volta do layout, ou uma imagem de fundo para simular uma coluna de cor diferente e uma série de outras situações nas quais precisaríamos de um container que realmente envolvesse os elementos flutuados.

As CSS nos oferecem alguns métodos para expandir o container dos floats. O mais simples consiste em flutuar o próprio container, contudo, dependendo do contexto, flutuar o container pode quebrar um layout. Em marcações simples, essa é uma solução a ser considerada, e em qualquer caso deve ser a primeira a ser testada. No nosso exemplo ela funciona perfeitamente, bastando definir uma classe para o primeiro parágrafo, o container, e flutuá-lo à esquerda ou à direita, indiferentemente. Veja os acréscimos na marcação e na folha de estilo: ► **HTML**

```
<p class="container">Lorem ...pouco texto... elit.  
<em class="flutuado">Ut ...muito texto... diam.</em>  
Aenean non tortor id metus aliquet consectetur.</p>
```

► CSS

```
p.container { float: left; }
```

Outro método consiste em definir a propriedade **overflow** com o valor **hidden** ou **auto** para o container. Esse método é o mais simples e imediato, contudo é preciso analisar cada caso para verificar se ocorre algum efeito indesejado no layout em consequência da declaração da propriedade **overflow**. Para aplicar esse método basta acrescentar a seguinte regra CSS: `p.container { overflow: hidden; }`

ou

```
p.container { overflow: auto; }
```

Outro método para expandir o container é conhecido como método sem uso de marcação extra, originalmente inventado por Tony Aslett, criador do fórum CSSCreator, em <http://www.csscreator.com>.

Declare na folha de estilo as seguintes regras CSS:

► CSS

```
.container: after {  
  content: ".";  
  display: block;  
  height: 0;  
  clear: both;  
  visibility: hidden;  
  }  
  
.container { display: inline-block; }  
/* Hides from IE-mac */  
* html .container { height: 1%; }  
.container { display: block; }  
/* End hide from IE-mac */
```

O método moderno, à época em que foi escrita esta edição do livro, é conhecido como *micro clearfix* e consiste em se declarar a classe **clearfix** para o container dos elementos flutuados.

Declare na folha de estilo as seguintes regras CSS:

► CSS

```
.clearfix:before,  
.clearfix:after {  
  display: table;  
  content: " ";  
  }  
  
.clearfix:after {
```



```
clear: both;
```

```
}
```

Na figura 3.13, é mostrada a expansão do container resultante da aplicação de qualquer um dos quatro métodos apresentados.

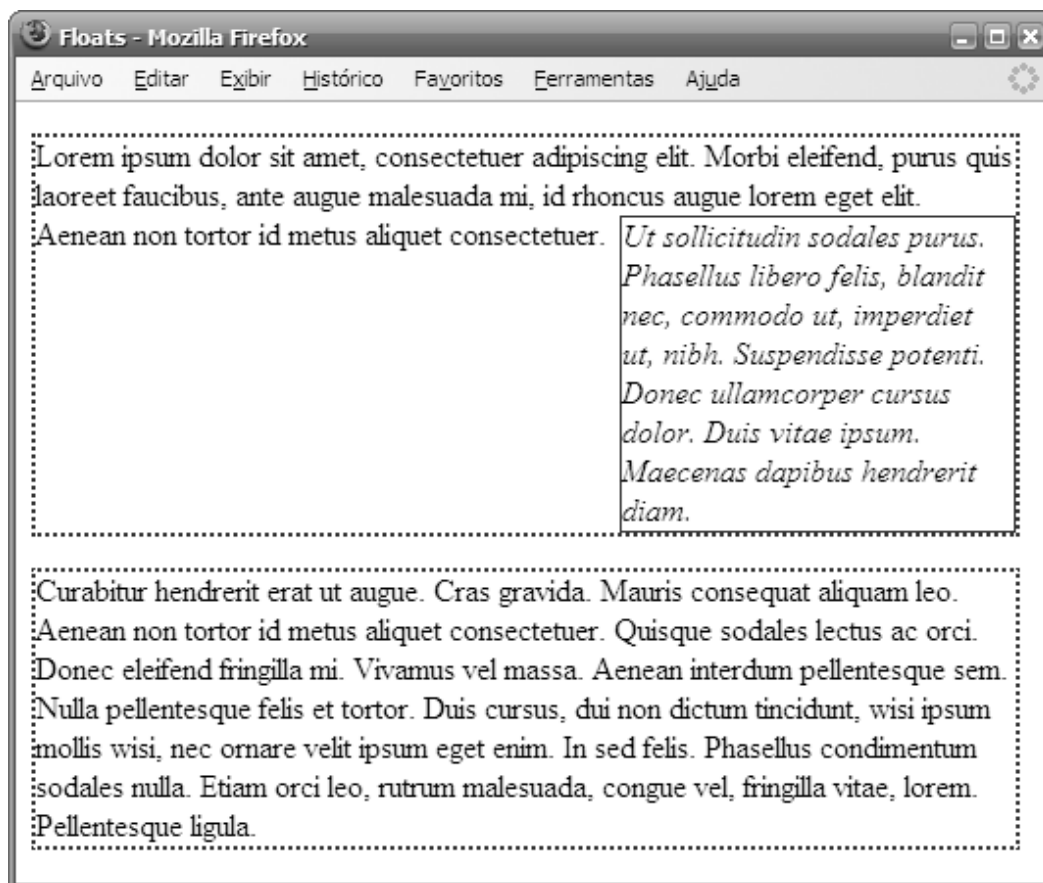


Figura 3.13 – Expansão do container.

Todo box flutuado comporta-se como um elemento nível de bloco, assim, defina sempre uma largura (usando a propriedade CSS `width`) para boxes flutuados; dispensável para imagens que possuem uma largura implícita.

3.1.3 Esquema absoluto

O esquema de posicionamento absoluto é regido pela propriedade CSS `position` e seus valores `absolute` e `fixed`. Ao contrário do posicionamento com o valor `relative`, conforme estudado em [3.1.1.3], esse posicionamento retira o elemento posicionado do fluxo do documento, fazendo com que o elemento que se segue ocupe seu lugar.

3.1.3.1 Posicionamento absoluto

A declaração CSS `position: absolute` retira o box do seu posicionamento na marcação e faz com que o box seguinte na marcação HTML se desloque para a

posição que ele ocupa. O deslocamento depende do que chamamos de contexto de posicionamento.

Um contexto de posicionamento estabelece um ponto origem para contagem das coordenadas de posicionamento, bem como a área total para posicionamento. As coordenadas de posicionamento são definidas pelas propriedades **top**, **right**, **bottom** e **left**. As propriedades **left** e **right** definem o quanto o bloco deve ser deslocado para a direita ou esquerda, respectivamente, em relação ao canto superior esquerdo da área de posicionamento; e as propriedades **top** e **bottom** definem o deslocamento para baixo e para cima, respectivamente, em relação ao canto inferior direito da área de posicionamento.

Em [3.1.1.3], estudamos o posicionamento relativo e vimos que o deslocamento do box tinha como origem o canto superior esquerdo do próprio box. Nesse tipo de posicionamento, a origem não varia, o contexto de posicionamento é sempre o mesmo. Ao contrário, no posicionamento absoluto o contexto de posicionamento de um box é determinado pelo posicionamento dos seus boxes ancestrais.

Se nenhum box ancestral for posicionado com uso da declaração CSS **position**, a origem do sistema de coordenadas é o canto superior esquerdo da tela (viewport).

Para exemplificar os diferentes casos de posicionamento absoluto, usaremos a seguinte marcação HTML: ► **HTML**

```
<div id="primeira">
  <p class="um">PARÁGRAFO UM<br />
  ...texto...texto...texto</p>
  <p class="dois">PARÁGRAFO DOIS<br />
  ...texto...texto...texto</p>
</div>

<div id="segunda">
  <p class="tres">PARÁGRAFO TRÊS<br />
  ...texto...texto...texto</p>
  <p class="quatro">PARÁGRAFO QUATRO<br />
  ...texto...texto...texto</p>
  <p class="cinco">PARÁGRAFO CINCO<br />
  ...texto...texto...texto</p>
</div>
```

O código cria dois boxes **div**, containers, o primeiro com dois parágrafos e o segundo com três parágrafos. Na folha de estilo, definimos bordas para os elementos da marcação e uma cor de fundo para fins de clareza nas figuras, mostrando os posicionamentos. A folha de estilo inicial é: ► **CSS**

```
div {
  font-weight: bold;
```

```
width: 300px;
border: 2px solid #333;
padding: 5px 20px;
margin-top: 10px;
```

```
}
```

```
p { width: 200px; border: 2px dotted #333; }
p.um { background: #cff; }
p.dois { background: #6f9; }
p.tres { background: #fcf; }
p.quatro { background: #ffc; }
p.cinco { background: #ccc; }
```

Na figura 3.14, é mostrado o efeito de declarar **position: absolute** para o primeiro parágrafo contido no segundo **div**, o parágrafo três da marcação.

Note que a simples declaração de **position: absolute**, sem definir as coordenadas do deslocamento, fez com que o parágrafo três permanecesse no mesmo lugar, liberando o espaço que ocupava e causando a subida dos parágrafos quatro e cinco que lhe seguem no fluxo do documento. No contexto do documento, o parágrafo três obscureceu a parte do parágrafo quatro que ficou atrás dele no eixo z.

Um box deslocado pode ou não ser obscurecido pelo box com o qual compartilhe a mesma posição. Na seção 3.2, estudaremos a propriedade CSS **z-index**, que rege o posicionamento de boxes sobrepostos.

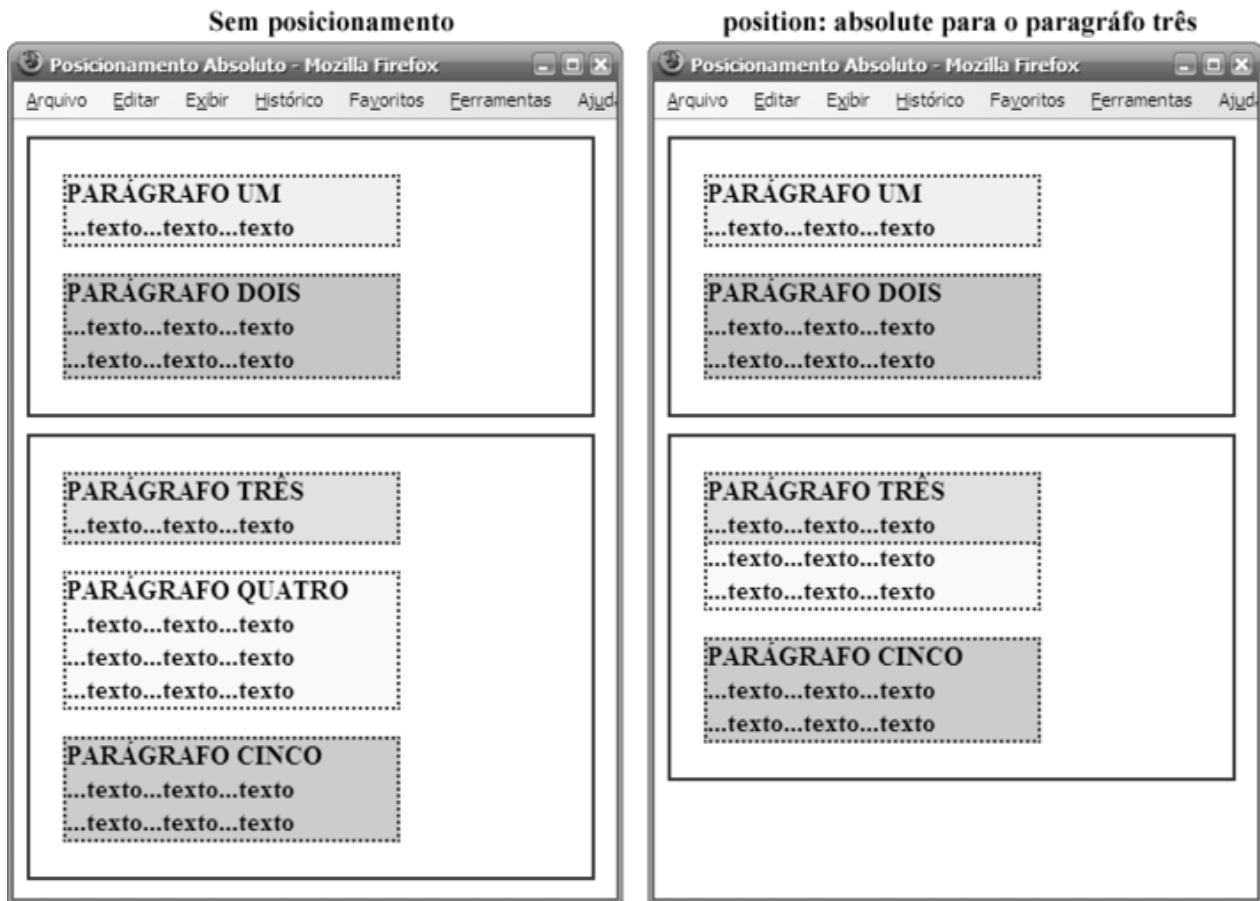


Figura 3.14 – Posicionamento absoluto.

Vamos definir coordenadas para o posicionamento do parágrafo três acrescentando as seguintes declarações na folha de estilo: ► **CSS**

```
p.tres {  
background: #fcf;  
position: absolute;  
left: 180px; top: 125px; }
```

A origem padrão das coordenadas para o posicionamento absoluto é o canto superior esquerdo da área de renderização na janela do navegador (viewport). Contudo, essa origem pode mudar, dependendo do contexto de posicionamento. A regra CSS mostrada anteriormente para o parágrafo três está definida no contexto de posicionamento padrão, e o resultado do deslocamento do parágrafo, depois de aplicada a nova folha de estilo, é mostrado na figura 3.15:

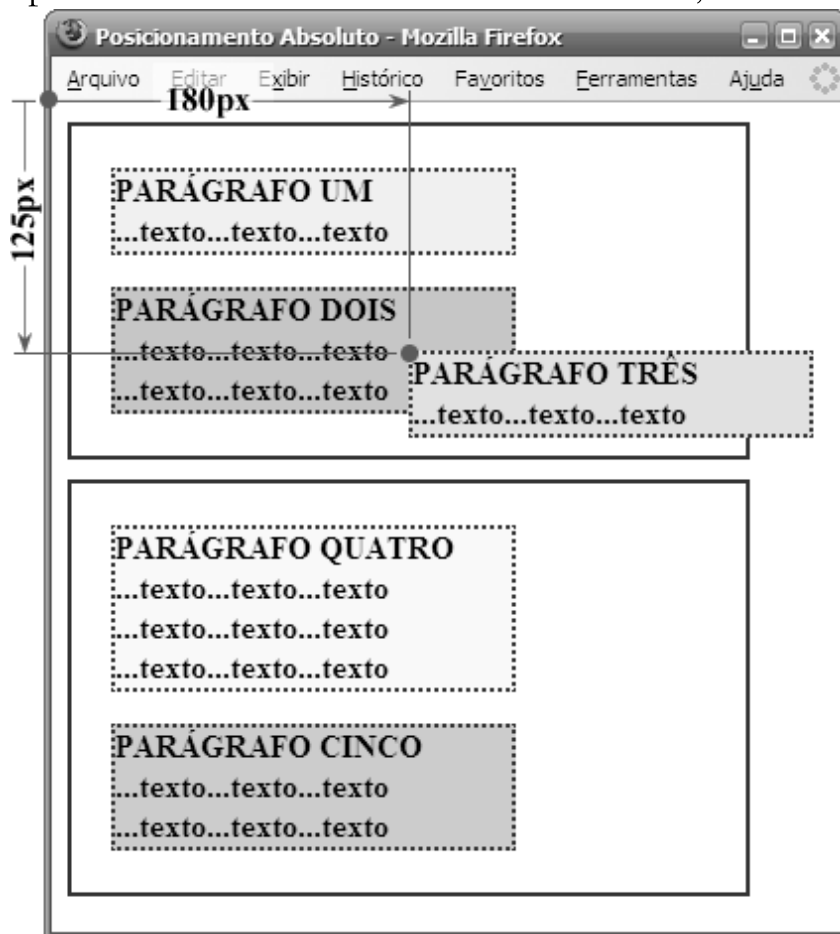


Figura 3.15 – Posicionamento absoluto padrão.

É o contexto de posicionamento que determina a origem do sistema de coordenadas para posicionamento absoluto. A regra que rege o posicionamento estabelece que elementos posicionados com a declaração **position: absolute** serão deslocados tomando como base para determinação das coordenadas o ancestral mais próximo posicionado, ou seja, para o qual tenham sido declarados os valores **relative**, **absolute** ou **fixed** para a propriedade **position**. Não havendo ancestral posicionado, o contexto de posicionamento será a janela do navegador.

Para esclarecer o conceito de contexto de posicionamento, vamos supor o mesmo cenário do exemplo anterior e adicionalmente posicionar o **div** que contém o parágrafo três. Vamos declarar **position: relative** para aquele **div** sem declarar coordenadas, mantendo-a em seu lugar no fluxo do documento. Assim fazendo, estaremos estabelecendo um novo contexto de posicionamento para o parágrafo três, pois agora o elemento-pai do parágrafo foi posicionado e a origem do sistema

de coordenadas será tomada a partir dele. A regra CSS a acrescentar na folha de estilo é a seguinte: `div#segunda { position: relative; }`

Na figura 3.16, é mostrado o efeito de se mudar o contexto de posicionamento.

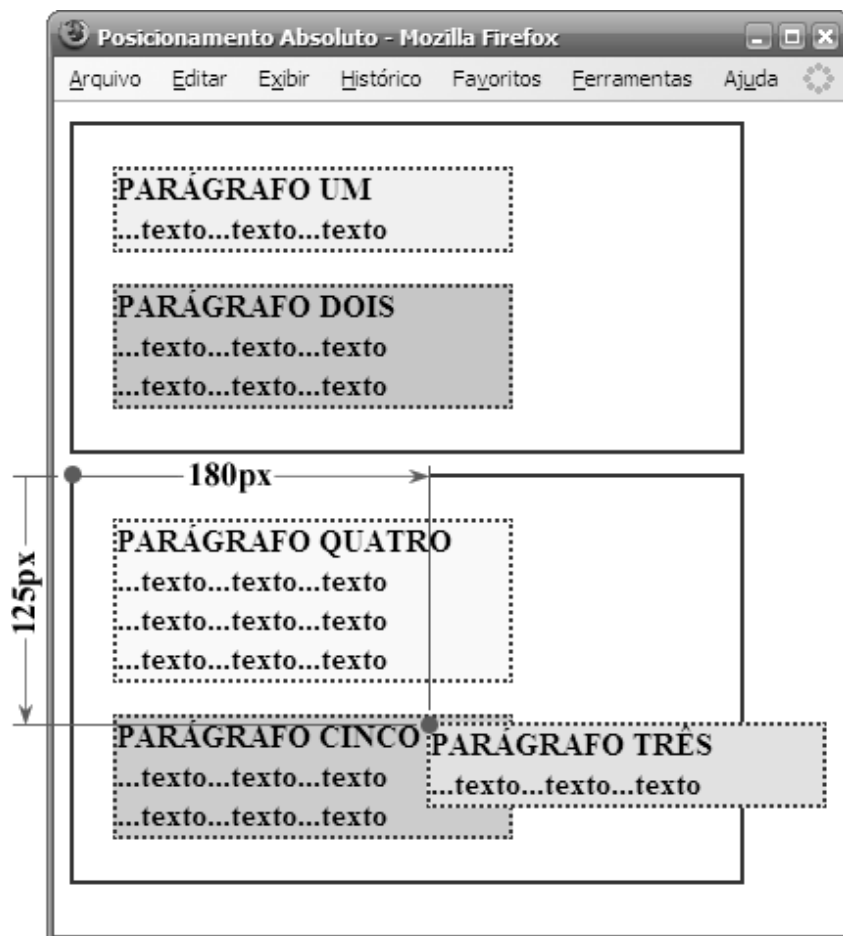


Figura 3.16 – Posicionamento em contexto.

Elementos posicionados de forma absoluta podem ser colocados em qualquer lugar, inclusive fora dos limites de seu elemento-pai. Podemos também concluir que, quando posicionamos um elemento de forma absoluta, ele cria um contexto de posicionamento para os seus elementos-filhos.

Boxes inline, quando posicionados de forma absoluta, se comportam como elementos nível de bloco.

3.1.3.2 Posicionamento estático

Os outros valores possíveis para a propriedade CSS `position` são: `static`, `fixed` e `inherit` (herdado). O valor `static` é o valor inicial ou padrão de posicionamento. Os boxes permanecem como no esquema fluxo normal. O uso dessa declaração é feito com a finalidade de sobrescrever um posicionamento declarado anteriormente. As propriedades de posicionamento `top`, `right`, `bottom` e `left` não se aplicam para esse posicionamento.

3.1.3.3 Posicionamento fixo

O posicionamento com a declaração **position: fixed** pode ser considerado um caso especial de posicionamento absoluto. A diferença é que nesse tipo de posicionamento o box posicionado permanece fixo em relação a uma referência. O contexto de posicionamento é sempre a área de renderização na janela do navegador (viewport). Em mídia visual, o box fica fixo em relação à viewport e não se movimenta quando há rolagem da página. Em mídia impressa, o box será impresso em cada uma das folhas.

3.2 Posicionamento com z-index

A propriedade CSS **z-index** rege o empilhamento de boxes segundo o eixo z. O eixo z é perpendicular à tela do monitor e tudo se passa como em um sistema tridimensional com coordenadas horizontal, vertical e coordenada z ou de profundidade. A propriedade **z-index** determina qual box fica à frente quando dois ou mais boxes ocupam a mesma posição ou se sobrepõem parcialmente.

Quando retiramos um ou mais boxes do fluxo normal do documento com uso da propriedade **position**, pode ocorrer sobreposição de boxes, conforme vimos nos exemplos de posicionamento absoluto mostrados nas figuras 3.14, 3.15 e 3.16 anteriores, quando um parágrafo foi colocado sobre outro. Havendo sobreposição de boxes posicionados, entra em cena a propriedade **z-index**, que regula qual a ordem de empilhamento no eixo z, ou seja, que elemento está mais próximo do usuário e como eles se distribuem em profundidade.

Considere a marcação a seguir, na qual três elementos **div** vêm em sequência no fluxo do documento: ► **HTML**

```
<div class="um">DIV UM</div>
<div class="dois">DIV DOIS</div>
<div class="tres">DIV TRÊS</div>
```

E a folha de estilos posicionando os três divs:

► **CSS**

```
div {
border: 2px solid #000;
position: absolute;
```

```
}
```

```
div.um {
width: 200px;
height: 80px;
```

```
background: #cff;
```

```
}
```

```
div.dois {  
width: 110px;  
height: 120px;  
left: 120px;  
top: 20px;  
background: #6f9;
```

```
}
```

```
div.tres {  
width: 90px;  
height: 60px;  
left: 70px;  
top: 50px;  
background: #fcf;
```

```
}
```

Na figura 3.17, mostramos, à esquerda, uma vista tridimensional da renderização dos elementos `div`, posicionados e sobrepostos onde acrescentamos o eixo `z` de profundidade, e, à direita, uma visão bidimensional da renderização.

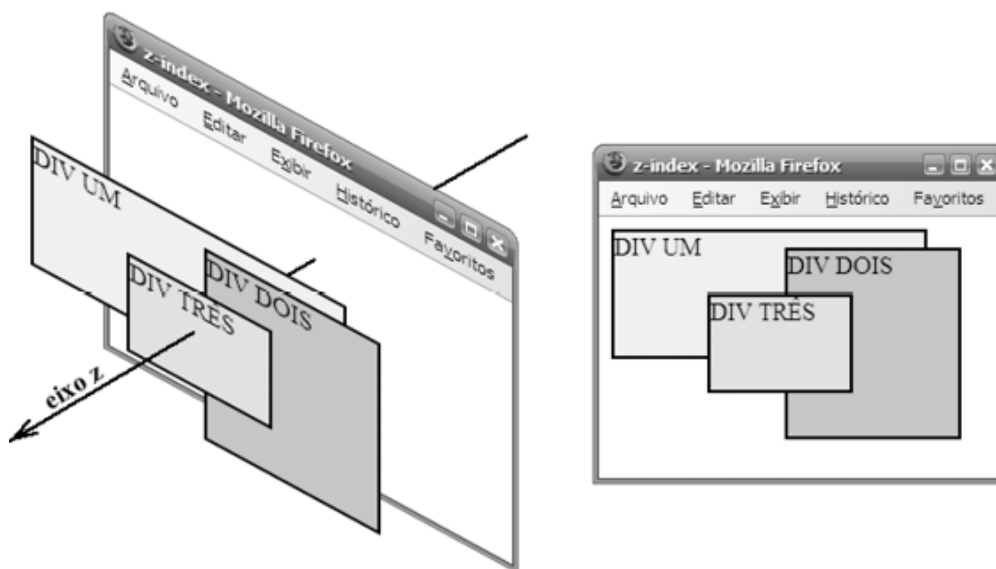


Figura 3.17 – Eixo `z` de profundidade.

Quando não há declaração explícita da propriedade `z-index`, cabe ao agente de usuário determinar a ordem de empilhamento. A maioria dos navegadores adota como critério posicionar mais à frente os elementos que aparecem por último na marcação. Assim, conforme mostrado na figura 3.17, o `div` mais próximo do usuário é o três, sendo em consequência posicionado mais à frente, seguindo-se o dois e por

último o um.

Considere os seguintes acréscimos na folha de estilos do exemplo anterior: ► **CSS**

```
div {  
border: 2px solid #000;  
position: absolute;
```

```
}
```

```
div.um {  
width: 200px;  
height: 80px;  
background: #cff;  
z-index: 3; }
```

```
div.dois {  
width: 110px;  
height: 120px;  
left: 120px;  
top: 20px;  
background: #6f9;  
z-index: 2; }
```

```
div.tres {  
width: 90px;  
height: 60px;  
left: 70px;  
top: 50px;  
background: #fcf;  
z-index: 1; }
```

Definimos coordenadas z para cada um dos elementos-filhos, estabelecendo a coordenada mais alta para o **div.um** e a mais baixa para o **div.tres**.

Na figura 3.18 é mostrado o efeito causado pela nova folha de estilos.

A propriedade **z-index** controla o posicionamento dos elementos sobrepostos no sentido da profundidade. Note que, com a declaração dessa propriedade, invertemos a ordem padrão de renderização mostrada no exemplo anterior. Podemos estabelecer qualquer ordem de posicionamento para atender às necessidades do layout.



Figura 3.18 – Declaração z-index.

Os valores possíveis para a propriedade **z-index** são: **auto**, **integer** e **inherit** (valor herdado). O valor inicial ou default é **auto**. O valor **integer** (inteiro) é um número inteiro positivo ou negativo.

Alguns agentes de usuário tratam valores negativos de **z-index** de forma não prevista nas Recomendações do W3C para as CSS, ou seja, em lugar de posicionar os elementos definidos com aqueles valores na ordem natural de crescimento do eixo z, os colocam atrás da tela, fazendo com que desapareçam da vista do usuário.

Não existe obrigatoriedade quanto à sequência de números a adotar para declarar os valores inteiros de **z-index**. A única regra é: quanto maior o valor, mais próximo do usuário o box é posicionado. Assim, uma sequência de valores 1, 2, 3, 4, 5 tem o mesmo efeito de 36, 120, 204, 999.

3.2.1 Contexto para empilhamento

Conforme vimos na seção anterior, o posicionamento em profundidade deu-se em relação à tela do monitor do usuário. Os elementos foram dispostos tendo como fundo, ou base, a viewport.

No entanto, o comportamento de renderização para posicionamento em profundidade não é tão simples assim, havendo outros fatores a considerar. As Recomendações do W3C para as CSS preveem um contexto de empilhamento segundo o eixo z. No exemplo anterior, os três elementos **div** foram posicionados de forma absoluta, tendo como origem das coordenadas horizontal e vertical o canto superior da área de renderização do navegador.

Você se lembra do conceito de contexto de posicionamento? Se sim, siga em frente. Se não, faça uma revisão desse conceito voltando à seção 3.1.3.1 deste capítulo.

Todo elemento posicionado com o valor **relative**, **absolute** ou **fixed** cria um contexto de posicionamento, estabelecendo uma origem para contagem das

coordenadas de posição horizontal e vertical. De modo similar, um elemento posicionado segundo uma das formas citadas anteriormente cria um contexto de empilhamento, estabelecendo uma origem para contagem da coordenada z de profundidade de seus elementos-filhos. Para exemplificar esse conceito vamos considerar a seguinte marcação: ► **HTML**

```
<div id="a">DIV A
<div class="a-um">DIV A-um</div>
<div class="a-dois">DIV A-dois</div>
<div class="a-tres">DIV A-tres</div>
</div>
<div id="b">DIV B
  <div class="b-um">DIV B-um</div>
  <div class="b-dois">DIV B-dois</div>
</div>
```

Na marcação, identificamos o `div#a`, que contém como elementos-filhos os `div.a-um`, `div.a-dois` e `div.a-tres` e, em seguida, no fluxo do documento, o `div#b`, contendo os `div.b-um` e `div.b-dois`.

E a folha de estilo com aplicação de bordas, cor de fundo e dimensões para fins de clareza do exemplo: ► **CSS**

```
div {
  border: 2px solid #000;
}
```

```
div#a, div#b {
  width: 300px;
  margin-bottom: 20px;
}
```

```
div#a {
  background: #ddd;
}
```

```
div.a-um {
  width: 200px;
  height: 80px;
  background: #cff;
}
```

```
div.a-dois {
  top: 70px;
```

```
width: 110px;  
height: 120px;  
background: #6f9;
```

```
}
```

```
div.a-tres {  
top: 90px;  
width: 90px;  
height: 60px;  
background: #fcf;
```

```
}
```

```
div.b-um {  
width: 200px;  
height: 120px;  
background: #ffd1d1;
```

```
}
```

```
div.b-dois {  
width: 50px;  
height: 60px;  
background: #ccc;
```

```
}
```

A renderização da marcação com aplicação da folha de estilos é mostrada na figura 3.19.

Vamos definir posicionamento absoluto para os cinco elementos-filhos dos **div#a** e **div#b**, fazendo com que eles se sobreponham. Os **div#a** e **div#b** não serão posicionados com a finalidade de não estabelecer um novo contexto de empilhamento para seus elementos-filhos. Assim, o contexto de empilhamento será o canto superior da área de renderização.

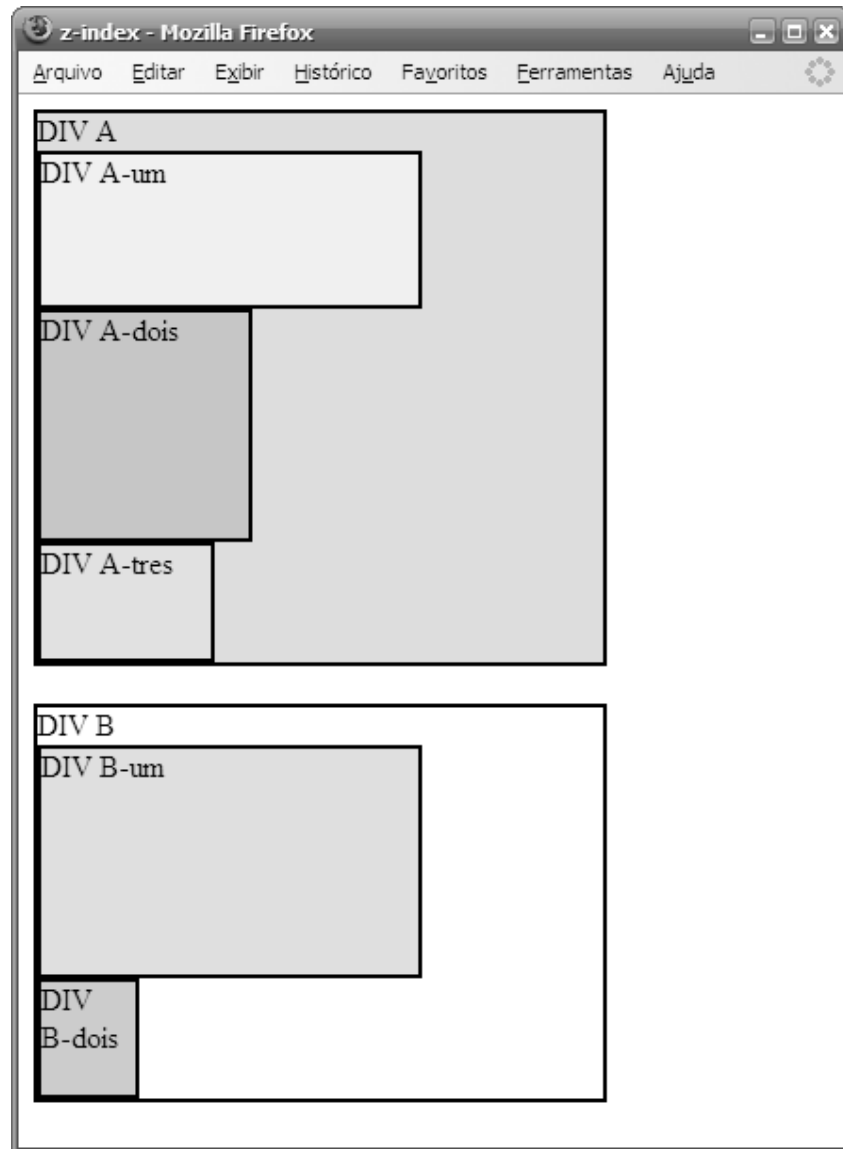


Figura 3.19 – Marcação básica.

Aplicaremos a folha de estilo anterior com as alterações definindo posicionamento, conforme mostradas as seguintes regras CSS: ► **CSS**

```
div { border: 2px solid #000; }  
div#a, div#b {  
width: 300px;  
margin-bottom: 20px;
```

```
}
```

```
div#a { background: #ffc; }  
div.a-um {  
position: absolute;  
left: 65px;  
top: 20px;  
width: 200px;  
height: 80px;
```

```
background: #cff;
```

```
}
```

```
div.a-dois {  
position: absolute;  
left: 120px;  
top: 70px;  
width: 110px;  
height: 120px;  
background: #6f9;
```

```
}
```

```
div.a-tres {  
position: absolute;  
left: 165px;  
top: 90px;  
width: 90px;  
height: 60px;  
background: #fcf;
```

```
}
```

```
div.b-um {  
position: absolute;  
left: 145px;  
top: 130px;  
width: 200px;  
height: 120px;  
background: #ffd1d1;
```

```
}
```

```
div.b-dois {  
position: absolute;  
left: 215px;  
top: 170px;  
width: 50px;  
height: 60px;  
background: #ccc;
```

```
}
```

Na figura 3.20, é mostrada a renderização da marcação com a folha de estilo aplicada.

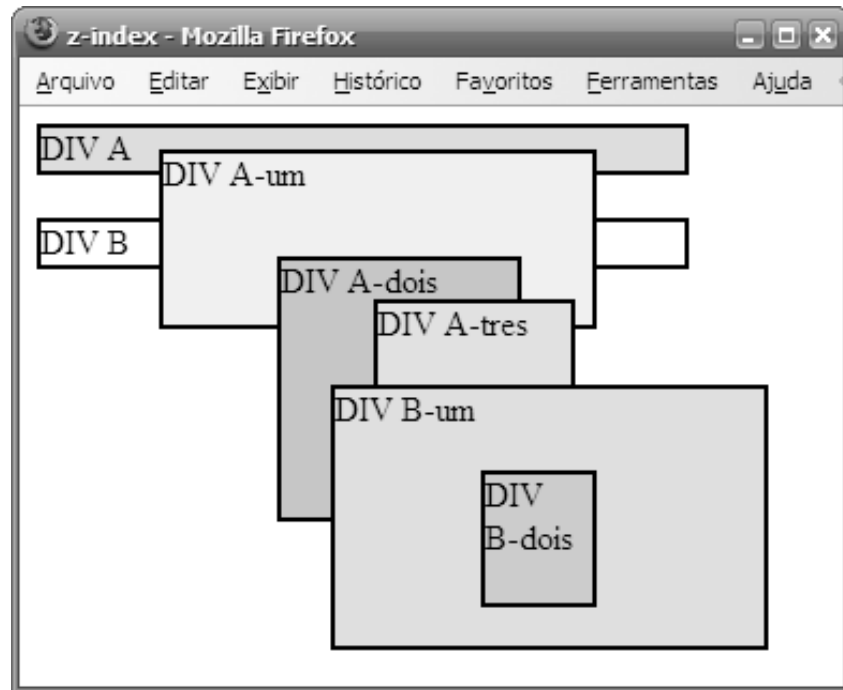


Figura 3.20 – Contexto de empilhamento padrão.

Convém ressaltar os seguintes comportamentos nesse contexto padrão de empilhamento:

- Os `div#a` e `div#b` encolheram até uma altura suficiente para conter apenas seus nomes; isso porque seus elementos-filhos foram posicionados de forma absoluta, liberando o espaço que ocupavam no fluxo do documento e esvaziando-o.

- A ordem de empilhamento obedece à ordem em que os elementos-filhos posicionados aparecem na marcação; isso porque nenhum dos elementos-filhos tem um elemento ancestral posicionado.

Vamos alterar o contexto de empilhamento no nosso exemplo. Observe os acréscimos que faremos na folha de estilo: ► **CSS**

```
div { border: 2px solid #000; }
```

```
div#a, div#b {
```

```
width: 300px;
```

```
margin-bottom: 20px;
```

```
}
```

```
div#a {
```

```
position: relative;
```

```
left: 10px;
```

```
top: 15px;
```

```
z-index: 25;
```

```
width: 350px;
```

```
height: 200px;
```

```
background: #ddd;
```

```
}
```

```
div#b {  
position: relative;  
top: -230px;  
z-index: 20;  
height: 200px;
```

```
}
```

...igual a anterior...

```
div.b-dois {  
position: absolute;  
left: 215px;  
top: 170px;  
width: 50px;  
height: 60px;  
background: #ccc;
```

```
}
```

Observe as seguintes alterações e sua finalidade:

- Os `div#a` e `div#b` foram posicionados de forma relativa e, em consequência, cada um deles estabeleceu um contexto de empilhamento. Para os `div.a-um`, `div.a-dois` e `div.a-tres`, a origem do sistema de coordenadas horizontal, vertical e de profundidade passa a ser o `div#a`, e não mais a viewport, como no exemplo anterior. O mesmo acontece para os `div.b-um` e `div.b-tres`, que agora têm como base do sistema o `div#b`. Dito de outra maneira: tanto o `div#a` como o `div#b` estabeleceram um novo contexto de empilhamento para seus elementos-filhos.
- Para os `div#a` e `div#b` definimos largura, altura e coordenadas horizontal e vertical com o propósito de tornar mais clara a renderização.
- Definimos `z-index` para os `div#a` e `div#b`. É aqui que o conceito de contexto de empilhamento fica claro. Ao definir um valor `z-index` para o `div#a` maior que para o `div#b`, invertemos o comportamento de sobreposição padrão. Forçamos um elemento anterior no código de marcação a ser colocado à frente de um posterior seu.
- Agora, o `div#a` foi colocado à frente do `div#b`, e todos os seus elementos-filhos ficarão sempre à frente dos elementos-filhos do `div#b`, independentemente do valor de `z-index`. Ou seja, `z-index` funciona independentemente para os elementos-filhos dos `div#a` e `div#b`.

Na figura 3.21, é mostrada a renderização da marcação com a nova folha de estilo a ela aplicada.

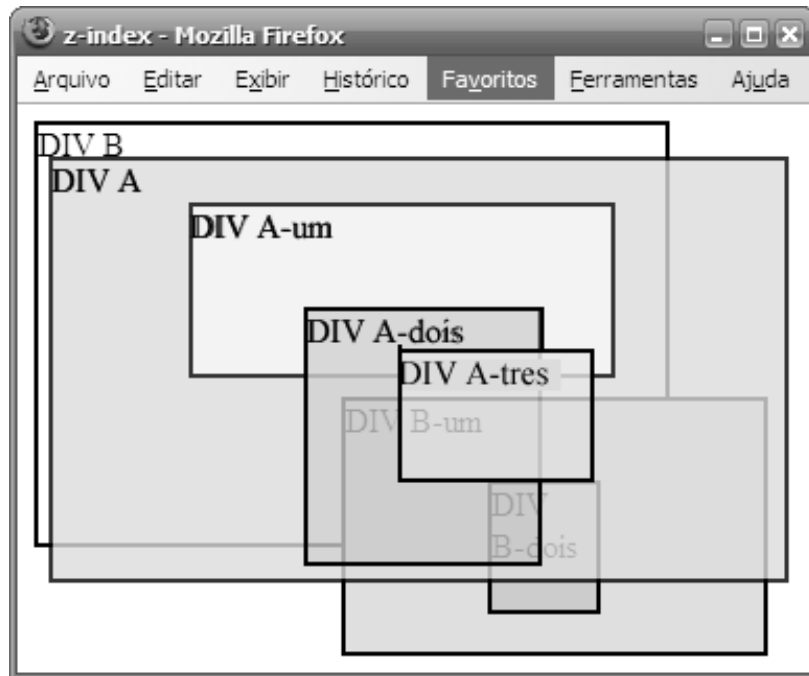


Figura 3.21 – Contexto de empilhamento z-index.

Vamos examinar a aplicação de **z-index** nos elementos-filhos dos **div#a** e **div#b**. Considere os seguintes acréscimos na folha de estilo anterior: ► **CSS**

```
div { border: 2px solid #000; }
div#a {
...
z-index: 25;
...
}

div#b {
...
z-index: 20;
...
}

div.a-um {
...
z-index: 10;
}

div.a-dois {
...
z-index: 5;
}
```

```
div.a-tres {
...
z-index: 2;

}

div.b-um {
...
z-index: 9500;

}

div.b-dois {
...
z-index: 1000;

}
```

Na figura 3.22, mostramos a renderização conforme os acréscimos na folha de estilos.

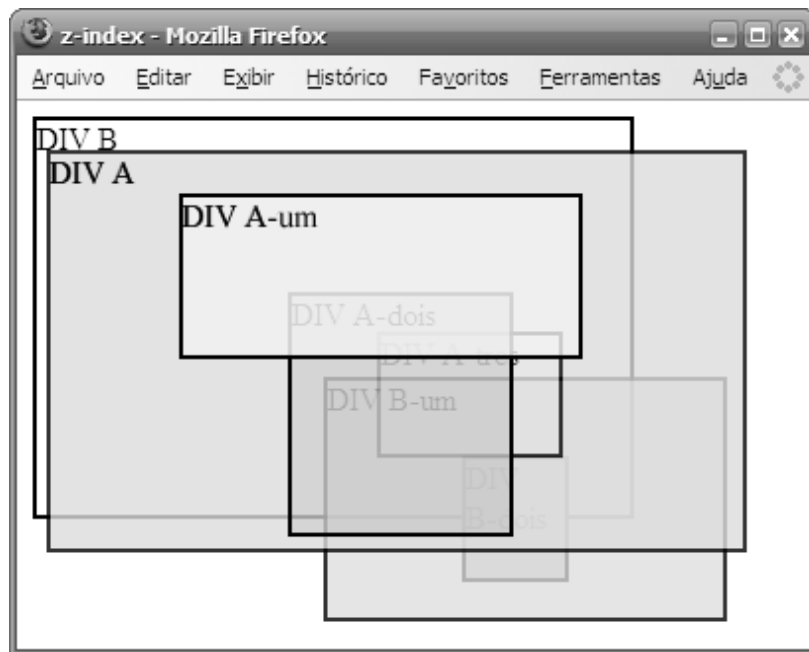


Figura 3.22 – Empilhamento z-index.

Note que, nesse novo cenário, a atribuição de **z-index** para os elementos-filhos contidos nos **div#a** e **div#b** alterou a ordem de empilhamento; contudo, o fato de se ter atribuído um valor **z-index** muito alto para aqueles contidos no **div#b** não fez com que eles se posicionassem à frente do **div#a**.

CAPÍTULO 4

Seletores, cascata e herança

Neste capítulo, faremos uma introdução aos seletores CSS, apresentando a definição e a nomenclatura dos termos usados para seletores CSS. Veremos as regras que regem a criação de um seletor desde sua forma mais simples até a composição de seletores complexos. Apresentaremos as diretrizes que regem o efeito cascata e a herança CSS e seus relacionamentos com os seletores.

4.1 Introdução

As especificações do W3C para as CSS vêm introduzindo novos seletores a cada versão. Assim, a versão inicial, CSS1, lançada em 1996, previu nove seletores; a versão CSS2, lançada em 1998, introduziu mais 11 seletores, perfazendo um total de 20; e a versão CSS3, que já atingiu o status de Recomendação do W3C, criou mais 22, resultando em um total de 42 seletores.

É importante notar que as CSS3 estão mais do que dobrando o número de seletores existentes até a versão anterior da especificação. Na tabela 1.1, mostramos os seletores das CSS3, o casamento de cada um deles, seu tipo e a versão CSS na qual foram criados.



O termo casar e casamento são usados neste livro com o mesmo sentido que eles têm quando se trata de expressões regulares. Assim, diz-se que um seletor casa com um elemento quando ele, seletor, tem como alvo o elemento ou os elementos.

Tabela 1.1 – Seletores das CSS3

Seletor	Casa com	Tipo	CSS
*	Qualquer elemento	Seletor universal	2
E	Qualquer elemento de nome E	Seletor tipo	1
E[foo]	Elemento E com atributo “foo”	Seletor de atributo	2
E[foo="bar"]	Elemento E cujo valor do atributo “foo” é “bar”	Seletor de atributo	2
E[foo~="bar"]	Elemento E cujo valor do atributo “foo” é exatamente “bar” ou contém o nome “bar” entre espaços em branco	Seletor de atributo	2
E[foo^="bar"]	Elemento E cujo valor do atributo “foo” começa com a string “bar”	Seletor de atributo	3

Seletor	Casa com	Tipo	CSS
E[foo\$="bar"]	Elemento E cujo valor do atributo "foo" termina com a string "bar"	Seletor de atributo	3
E[foo*="bar"]	Elemento E cujo valor do atributo "foo" contém a substring "bar"	Seletor de atributo	3
E[foo "val"]	Elemento E cujo valor do atributo "foo" é exatamente "bar" ou começa com "val"	Seletor de atributo	2
E:root	Elemento E raiz do documento	Pseudoclasse estrutural	3
E:nth-child(n)	Elemento E filho de ordem n do seu elemento-pai	Pseudoclasse estrutural	3
E:nth-last-child(n)	Elemento E filho de ordem n do seu elemento-pai, contando a ordem a partir do último filho	Pseudoclasse estrutural	3
E:nth-of-type(n)	Irmão do mesmo tipo e de ordem n do elemento E	Pseudoclasse estrutural	3
E:nth-last-of-type(n)	Irmão do mesmo tipo e de ordem n do elemento E, contando a ordem a partir do último irmão	Pseudoclasse estrutural	3
E:first-child	Elemento E primeiro-filho do seu elemento-pai	Pseudoclasse estrutural	2
E:last-child	Elemento E último-filho do seu elemento-pai	Pseudoclasse estrutural	3
E:first-of-type	Elemento E primeiro irmão do mesmo tipo de E	Pseudoclasse estrutural	3
E:last-of-type	Elemento E último irmão do mesmo tipo de E	Pseudoclasse estrutural	3
E:only-child	Elemento E filho único de seu elemento-pai	Pseudoclasse estrutural	3
E:only-of-type	Elemento E irmão único do mesmo tipo de E	Pseudoclasse estrutural	3
E:empty	Elemento E sem elementos-filho (inclusive nós de texto)	Pseudoclasse estrutural	3
E:link	Elemento E marcador de hyperlink em estado não visto	Pseudoclasse link	1
E:visited	Elemento E marcador de hyperlink em estado visitado	Pseudoclasse link	1
E:active	Elemento E marcador de hyperlink quando ativado	Pseudoclasse link	1
E:hover	Elemento E marcador de hyperlink com	Pseudoclasse link	2

Seletor	Casa com	Tipo	CSS
	ponteiro sobre ele		
E:focus	Elemento E sobre determinadas ações do usuário	Pseudoclassee ação do usuário	2
E:target	Elemento E alvo de um URI	Pseudoclassee alvo	3
E:lang(fr)	Elemento E no idioma especificado (por exemplo: “fr”)	Pseudoclassee :lang()	2
E:enabled	Elemento E habilitado		3
E:disabled	Elemento E desabilitado	Pseudoclassee estado do elemento E	3
E:checked	Elemento E “ticado” (por exemplo: radio-button ou checkbox)	Pseudoclassee estado do elemento E	3
E::first-line	Primeira linha de um elemento E	Pseudoelemento ::first-line	1
E::first-letter	Primeira letra de um elemento E	Pseudoelemento ::first-letter	1
E::before	Conteúdo gerado para ser posicionado antes do elemento E	Pseudoelemento ::before	3
E::after	Conteúdo gerado para ser posicionado depois do elemento E	Pseudoelemento ::after	3
E.warning	Elemento E ao qual foi atribuído o atributo classe com valor “warning”	Seletor classe	1
E#myid	Elemento E ao qual foi atribuído o atributo id com valor “myid”	Seletor ID	1
E:not(s)	Elemento E não casado por um seletor simples	Pseudoclassee negação	3
E F	Elemento F descendente do elemento E	Caractere de combinação espaço em branco	1
E > F	Elemento F filho do elemento E	Caractere de combinação filho	2
E + F	Elemento F imediatamente após o elemento E (E e F são irmãos)	Caractere de combinação irmão adjacente	2
E ~ F	Elementos F após o elemento E (E e F devem ser irmãos)	Caractere de combinação irmão geral	3

As especificações CSS3 para seletores introduziram as seguintes principais modificações nas especificações CSS2: • alteração nas definições básicas de seletores, tal como naquelas para seletores simples, grupos de seletores etc.; em particular, o que as CSS2 chamavam de seletor simples, agora se chama sequência de seletores, e o termo seletor simples passa a definir uma sequência de seletores; • criação de namespace opcional para os seletores universais, seletores tipos e seletores de atributos; • criação do caractere de combinação para o seletor-irmão geral subsequente;

- criação de seletor de atributo e de seletores pseudoclasses;
- criação de seletores pseudoelementos e nova sintaxe para eles (::);
- criação de um módulo independente para seletores;
- criação de uma suíte de testes exclusiva para seletores.

4.2 Definições

O entendimento dos mecanismos que regem o funcionamento e o comportamento dos seletores depende do exato conhecimento dos termos técnicos que se referem à árvore do documento e às definições próprias às estruturas dos seletores. Com a finalidade de equalizar e normalizar o uso de termos e, em consequência, facilitar o aprendizado dos conceitos contidos neste livro, passaremos a definir, nos itens que se seguem, os termos mais relevantes quando se trata da linguagem dos seletores.

4.2.1 Seletor

Tecnicamente, define-se seletor como sendo um padrão que casa com elementos da árvore de um documento. Ou, dito de outra forma; trata-se de uma tecnologia destinada a selecionar nós na árvore de um documento XML ou HTML.

A função que define um seletor pode ser expressa como mostrado a seguir:

*padrão * elemento --> booleano* **Cuja leitura é: comparando um padrão com um elemento da marcação, o resultado é falso ou verdadeiro. Se verdadeiro, diz-se que o padrão (seletor) casa com o elemento.**

Convém ressaltar que o casamento não é necessariamente único, ou seja, não é do tipo um para um, pois um padrão poderá casar não só com um elemento, mas também com um conjunto de elementos da árvore do documento.

4.2.2 Árvore do documento

Árvore do documento é uma estrutura que representa os nós do documento e a maneira como eles se relacionam, à semelhança de uma árvore genealógica que mostra em um diagrama o grau de parentesco entre membros de uma família. O

conceito de árvore do documento é mais bem assimilado com uso de um exemplo.

Considere um documento cuja marcação HTML é mostrada a seguir:

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="utf-8" />
<title>Seletores</title>
</head>
<body>
  <div id="tudo">
    <header>
      <h1>Cabeçalho 1</h1>
      <h2>Cabeçalho 2</h2>
    </header>
    <section>
      <ul>
        <li><a href="">Link-1</a></li>
        <li><a href="">Link-2</a></li>
        <li><a href="">Link-3</a></li>
      </ul>
    </section>
    <section>
      <h3>Cabeçalho 3</h3>
      <pre>
        ...
      </pre>
    </section>
    <footer>
      <p><em>Ver</em> <a href="">Link</a></p>
    </footer>
  </div>
</body>
</html>
```

O diagrama representativo da árvore do documento cuja marcação HTML foi mostrada é conforme a figura 4.1:

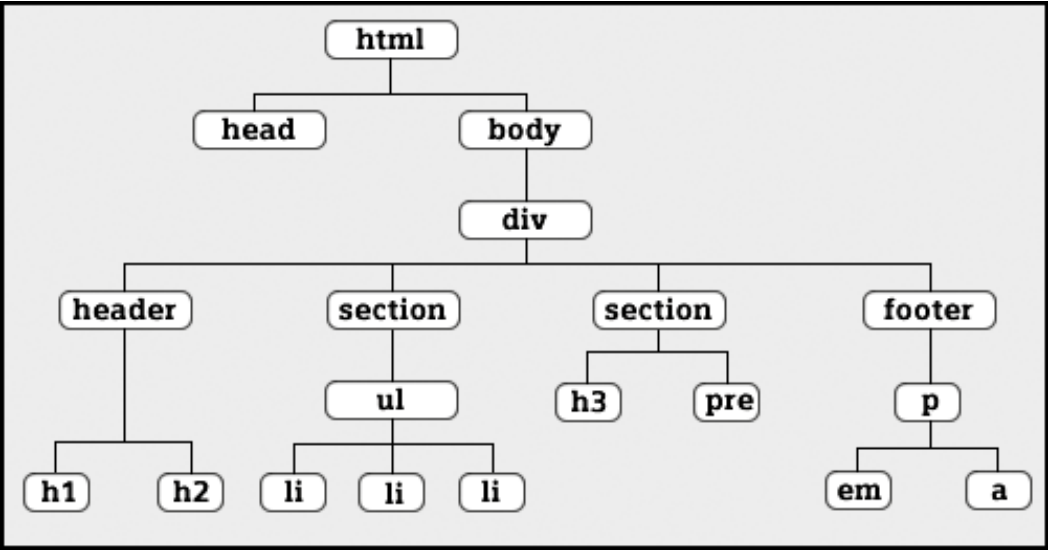


Figura 4.1 — Árvore do documento.

Nas definições que se seguem, quando for conveniente demonstrar o relacionamento entre os elementos, tomaremos como exemplo a árvore do documento mostrada na figura 4.1. Portanto, ao ler as definições, se preciso for, consulte a figura.

4.2.3 Elemento-filho e elemento-pai

Diz-se que um elemento A é filho de um elemento B quando A está contido diretamente em B, ou seja, quando A é descendente direto de B. Isso implica B ser elemento-pai de A.

Na árvore do documento mostrada na figura 4.1, identificamos, entre outros, os seguintes elementos-filhos e seus respectivos elementos-pais.

Elemento-pai	Elementos-filhos
html	head, body
body	div
div	header, section, section, footer
header	h1 e h2
ul	li, li, li
h3	Não tem elemento-filho

4.2.4 Elemento descendente e elemento ancestral

Diz-se que um elemento A é descendente de um elemento B quando A estiver contido em B em qualquer nível de aninhamento. Ou seja, quando A for elemento-filho de B ou elemento-filho de qualquer elemento-filho de B, ou elemento-filho de qualquer elemento-filho, de qualquer elemento-filho, e assim por diante.

Diz-se que B é elemento ancestral de A quando a relação entre eles se dá no sentido inverso da relação de descendência, como definida no parágrafo anterior.

Na árvore do documento mostrada na figura 4.1, identificamos, entre outros, os seguintes elementos descendentes e seus respectivos elementos ancestrais.

Elemento ancestral	Elementos descendentes
html	Todos os elementos da árvore
body	Todos os elementos da árvore, exceto html e head
div	header, section, section, footer
header	h1, h2
footer	p, em, a
pre	Não tem elemento descendente

4.2.5 Elemento-irmão

Diz-se que um elemento A é elemento-irmão de um elemento B quando A e B estiverem contidos no mesmo elemento-pai.

Elemento-pai	Elementos-irmãos
div	header, section, section, footer
section	h3 e pre
ul	li, li, li
p	em, a
footer	Não há elemento-irmão

4.2.6 Elemento precedente

Diz-se que um elemento A precede um elemento B quando A é ancestral de B ou elemento-irmão de B, posicionado antes dele na marcação HTML.

Referência	Elementos precedentes
div	body, head, html
section (primeiro)	header, body, head, html
section (segundo)	section (primeiro), header, body, head, html
p	Todos, exceto em e a
html	Não há elemento precedente

4.2.7 Elemento subsequente

Diz que um elemento A é subsequente, ou posterior, a um elemento B quando B precede A na marcação HTML.

Referência	Elementos subsequentes
div	Todos, exceto body, head e html
section (primeiro)	ul, li, li, li, section (segundo), h3, pre, footer, p, em, a
section (segundo)	h3, pre, footer, p, em, a
p	em, a
a	Não há elemento subsequente

4.2.8 Sintaxe para seletor

Regra CSS é a unidade básica de uma folha de estilo. Uma regra CSS é composta de duas partes: o *seletor* e a *declaração*. A declaração compreende uma *propriedade* e um *valor*. A sintaxe para se escrever uma regra CSS é mostrada na figura 4.2.



Figura 4.2 – Sintaxe da regra CSS.

4.2.9 Seletor universal

Seletor universal é aquele que representa todos os elementos da linguagem na qual o

documento foi escrito. A string para designar o seletor universal é o caractere asterisco (*), conforme sintaxe mostrada a seguir: ***** {

```
color: red; /* Todos os textos contidos no documento serão na cor
vermelha */
```

```
}
```

4.2.10 Seletor tipo

Seletor tipo é aquele que representa um determinado tipo de elemento da linguagem na qual o documento foi escrito. Por exemplo: **h1** e **p** são seletores tipos da linguagem HTML para cabeçalhos de nível um e parágrafos, respectivamente; **header** e **footer** são seletores tipos da linguagem HTML5 para blocos de cabeçalho e de rodapé; **book** e **catalog** são seletores tipos da linguagem XML que use esses elementos na marcação. A string para designar um seletor tipo é aquela que representa o elemento, conforme sintaxe mostrada a seguir: **h1** { color: blue; } /* Textos contidos nos elementos **h1** serão na cor azul */

```
footer { color: white; } /* Textos contidos nos elementos footer serão na
cor branca */
```

4.2.11 Seletor de atributo

Seletor de atributo é aquele cujo padrão é construído tomando como base um atributo e, opcionalmente, uma referência ao seu respectivo valor. Por exemplo: **h1[title]** ou **span[class="exemplo"]**, que usam o atributo **title** e **class**, com seu respectivo valor. Os seletores de atributo previstos nas especificações para as CSS3 são os seguintes.

E[att]

Casa com o elemento E da marcação ao qual foi definido o atributo de nome **att**, independentemente do seu valor.

Exemplo:

► CSS

```
p[title] { color: red; }
```

► HTML

```
<p title="Primeiro parágrafo">Texto do primeiro parágrafo</p>
<p class="dois">Texto do segundo parágrafo</p>
<p title = "Terceiro parágrafo">Texto do terceiro parágrafo</p>
<p>Texto do quarto parágrafo</p>
```

No exemplo mostrado, os textos do primeiro e terceiro parágrafos serão na cor vermelha (red), conforme definido na regra CSS. Ao segundo e quarto parágrafos

não se aplicará a regra de estilo, pois a eles não foi definido o atributo `title`.

E[att="val"]

Casa com o elemento E da marcação ao qual foi definido o valor `val` para o atributo `att`.

Exemplo:

► CSS

```
p[title="Terceiro parágrafo"] { color:blue; }
```

► HTML

```
<p title="Primeiro parágrafo">Texto do primeiro parágrafo</p>
<p class="dois">Texto do segundo parágrafo</p>
<p title = "Terceiro parágrafo">Texto do terceiro parágrafo</p>
<p>Texto do quarto parágrafo</p>
```

No exemplo mostrado, o texto do terceiro parágrafo será na cor azul (blue), conforme definido na regra CSS.

E[att|=“val”]

Casa com o elemento E da marcação ao qual foi definido o valor `val` ou um valor que comece com a string `val` para o atributo `att`.

Exemplo 1:

► CSS

```
p[lang|="pt"] { color:yellow; }
```

► HTML

```
<p lang="en">Texto do primeiro parágrafo</p>
<p lang="pt-br">Texto do segundo parágrafo</p>
<p lang="pt">Texto do terceiro parágrafo</p>
<p>Texto do quarto parágrafo</p>
```

No exemplo mostrado, os textos do segundo e terceiro parágrafos serão na cor amarela (yellow), conforme definido na regra CSS.

Exemplo 2:

► CSS

```
p[class|"dest"] { color: green; }
```

► HTML

```
<p class="dest-um">Texto do primeiro parágrafo</p>
<p class="dest_dois">Texto do segundo parágrafo</p>
<p class="dest-destaque">Texto do terceiro parágrafo</p>
```

```
<p class="destemido">Texto do quarto parágrafo</p>
<p class="dest">Texto do quinto parágrafo</p>
```

No exemplo mostrado, os textos do primeiro, terceiro e quinto parágrafos serão na cor verde (green), conforme definido na regra CSS.

E[att~="val"]

Casa com o elemento E da marcação ao qual foi definido um valor para o atributo att, que contenha a palavra val.

Exemplo:

► CSS

```
a[rel~="copyright"] { color: pink; }
```

► HTML

```
<a rel="copyright copyleft">Texto da primeira âncora</a>
<a rel="copyeditor">Texto da segunda âncora</a>
<a rel="copyright">Texto da terceira âncora</a>
<a rel="copyleft copyeditor">Texto da quarta âncora</a>
```

No exemplo mostrado, os textos da primeira e terceira âncoras serão na cor rosa (pink), conforme definido na regra CSS.

E[att^="val"]

Casa com o elemento E da marcação ao qual foi definido um valor para o atributo att, que comece com a string val.

Exemplo:

► CSS

```
a[rel^="copy"] { color: grey; }
```

► HTML

```
<a rel="copyright copyleft">Texto da primeira âncora</a>
<a rel="copyeditor">Texto da segunda âncora</a>
<a rel="copyright">Texto da terceira âncora</a>
<a rel="copyleft copyeditor">Texto da quarta âncora</a>
```

No exemplo mostrado, os textos de todas as âncoras serão na cor cinza (grey), conforme definido na regra CSS.

E[att\$="val"]

Casa com o elemento E da marcação ao qual foi definido um valor para o atributo att, que termine com a string val.

Exemplo:

► CSS

```
span[id$="imo"] { color: red; }
```

► HTML

```
<span id="maximo">Texto do primeiro span</span>
<span id="mimos">Texto do segundo span</span>
<span id="imoio">Texto do terceiro span</span>
<span id="primo">Texto do quarto span</span>
```

No exemplo mostrado, os textos do primeiro e quarto elementos `span` serão na cor vermelha (red), conforme definido na regra CSS.

E[att*="val"]

Casa com o elemento E da marcação ao qual foi definido um valor para o atributo `att`, que contenha a string `val`.

Exemplo:

► CSS

```
span[id*="imo"] { color: blue; }
```

► HTML

```
<span id="maximo">Texto do primeiro span</span>
<span id="mimos">Texto do segundo span</span>
<span id="imoio">Texto do terceiro span</span>
<span id="primo">Texto do quarto span</span>
```

No exemplo mostrado, os textos de todos os elementos `span` serão na cor azul (blue), conforme definido na regra CSS.

4.2.12 Seletor classe

Seletor classe é aquele cujo padrão é construído tomando como base o valor do atributo `class`. A string para designar um seletor classe é constituída de um ponto (.) seguido do valor do atributo `class` ou, no caso em que se deseja mais especificidade, esse seletor será constituído do nome do elemento seguido por um ponto (.), e este seguido do valor do atributo `class`, conforme sintaxe mostrada a seguir: **.exemplo** { color: blue; } /* Casa com elementos com valor do atributo class igual a exemplo */

```
h1.exemplo { color: white; } /* Casa com h1 que tenha valor do atributo
class igual a exemplo */
```

Convém ressaltar que se pode atribuir mais de um valor para o atributo `class`. Nesse caso, separam-se os valores por um espaço em branco, conforme mostrado a

seguir: `<h1 class="exemplo destaque diferente">Cabeçalho nível 1</h1>`
Quando se define vários valores para o atributo `class` de um elemento, ao elemento aplicam-se as regras CSS definidas para todos os valores do atributo. Havendo conflito de regras, aplica-se o efeito cascata.

É válido criar um seletor constituído de uma cadeia de valores de classe. Nesse caso, o seletor casa com o elemento que contenha todos os valores da cadeia no seu atributo classe. Observe o exemplo a seguir.

Exemplo:

► CSS

```
p.exemplo.diferente { color: red; }
```

► HTML

```
<p class="exemplo destaque diferente">Primeiro parágrafo</p>
<p class="exemplo destaque">Segundo parágrafo</p>
<p class="diferente">Terceiro parágrafo</p>
```

Nesse exemplo, o casamento do seletor será somente com o primeiro parágrafo.

4.2.13 Seletor ID

Seletor ID é aquele cujo padrão é construído tomando como base o valor do atributo `id`. A string para designar um seletor tipo `id` é constituída de um sinal de jogo da velha (`#`) seguido do valor do `id` ou, no caso em que se deseja mais especificidade, esse seletor será constituído do nome do elemento seguido de um sinal de jogo da velha (`#`), e este seguido do valor do `id`, conforme sintaxe mostrada a seguir: `#exemplo { color: blue; } /* O elemento do documento que tenha o valor do atributo id igual a exemplo será na cor azul */`

```
h1#exemplo { color: white; } /* O elemento h1 do documento que tenha o
valor do atributo id igual a exemplo será na cor branca */
```

4.2.14 Seletor pseudoclasse

As CSS criaram o conceito de seletores pseudoclasses estruturais. Trata-se de um conceito que possibilita selecionar um elemento baseado em informação que não consta da árvore do documento. A sintaxe para aplicar esse conceito é constituída de um sinal de dois-pontos (`:`) seguido do nome da pseudoclasse, conforme mostrado a seguir: `:link { color: blue; } /* Links não visitados serão na cor azul */`

```
:focus { color: white; } /* Elementos que ganham o foco serão na cor
branca */
```

```
a.externo:visited { color: red; } /* Links externos que já tenham sido
visitados serão na cor vermelha */
```

```
a:focus:visited { color: green; } /* É válido definir mais de uma
pseudoclasse */
```

Os seletores pseudoclasses são estudados a seguir.

:link

Esse seletor casa com o elemento `a` (para marcar links) não visitado. Por padrão, os navegadores renderizam o texto de um link não visitado na cor azul e sublinhado. Usamos esse seletor para alterar a estilização de links não visitados.

Exemplo:

► CSS

```
a:link { color: red; text-decoration: none; }
```

► HTML

```
<a href="http://maujor.com">Site do maujor</a>
```

Nesse exemplo, o texto do link quando ainda não visitado será na cor vermelha e sem o sublinhado padrão.

:visited

Esse seletor casa com o elemento `a` (para marcar links) que já tenha sido visitado. Por padrão, os navegadores renderizam o texto de um link visitado na cor roxa e sublinhado. Usamos esse seletor para alterar a estilização de links visitados.

Exemplo:

► CSS

```
a:visited { color: lime; }
```

► HTML

```
<a href="http://maujor.com">Site do maujor</a>
```

Nesse exemplo, o texto do link depois que for visitado será na cor verde (lime). O sublinhado segue o que foi definido para o estado não visitado.

E:hover

Esse seletor casa com o elemento `E` que tenha recebido sobre ele um dispositivo apontador (por exemplo: o ponteiro do mouse). Pode ser usado tanto em elementos `a` para links quanto em qualquer elemento da marcação.

Exemplo 1:

► CSS

```
a:hover { color: grey; }
```

► HTML


```
<a href="http://maujor.com">Site do maujor</a>
```

Nesse exemplo, o texto do link assumirá a cor cinza (grey) tão logo o usuário passe o mouse sobre ele. O sublinhado segue o que foi definido para o estado não visitado.

Exemplo 2:

► **CSS**

```
p:hover { background-color: green; }
```

► **HTML**

```
<p>Texto do parágrafo</p>
```

Nesse exemplo, a cor de fundo do parágrafo será verde (green) tão logo o usuário passe o mouse sobre ele.

E:active

Esse seletor casa com o elemento E que tenha sido ativado pelo usuário. Um elemento da marcação encontra-se no estado ativado enquanto o usuário permanece com o mouse clicado sobre ele, ou seja, o estado se inicia ao se pressionar o botão do mouse e termina ao se soltar o botão. Pode ser usado tanto em elementos **a** para links quanto em qualquer elemento da marcação.

Exemplo 1:

► **CSS**

```
a:active { color: red; }
```

► **HTML**

```
<a href="http://maujor.com">Site do maujor</a>
```

Nesse exemplo, o texto do link assumirá a cor vermelha (red) enquanto o usuário mantiver o botão do mouse pressionado sobre ele. O sublinhado segue o que foi definido para o estado não visitado.

Exemplo 2:

► **CSS**

```
p:active { background-color: red; }
```

► **HTML**

```
<p>Texto do parágrafo</p>
```

Nesse exemplo, a cor de fundo do parágrafo será vermelha (red) enquanto o usuário mantiver o botão do mouse pressionado sobre ele.

Notas sobre links

A ordem em que se declara a estilização para os diferentes estados de um link é importante para o funcionamento da estilização. As regras de estilo devem ser declaradas, rigorosamente, na seguinte ordem: `a:link { ... }`

```
a:visited { ... }  
a:hover { ... }  
a:active { ... }
```

Se você inverter essa ordem, entrará em cena o efeito cascata e o funcionamento dos diversos estados do link será prejudicado. Observe a ordem a seguir: `a:hover { ... }`

```
a:link { ... }  
a:visited { ... }  
a:active { ... }
```

Nesse exemplo, o estado over não funcionará, pois será sobrescrito pelo estado visitado ou estado inicial que foram declarados depois do estado over.

E:focus

Esse seletor casa com o elemento E que tenha sido posto em foco. Um elemento da marcação é posto em foco quando o usuário acessa o elemento via teclado. Pode ser usado tanto em elementos `a` para links quanto em qualquer elemento da marcação.

Exemplo 1:

► CSS

```
a:focus { outline: 2px solid red; }
```

► HTML

```
<a href="http://maujor.com">Site do maujor</a>
```

Nesse exemplo, o texto do link receberá uma borda sólida de **2px** na cor vermelha (red) quando o usuário o puser em foco. O sublinhado segue o que foi definido para o estado não visitado.

Exemplo 2:

► CSS

```
p:focus { background-color: yellow; }
```

► HTML

```
<p>Texto do parágrafo</p>
```

Nesse exemplo, a cor de fundo do parágrafo será amarela (yellow) quando o usuário o puser em foco.

Nota: por padrão, os navegadores inserem uma borda pontilhada de **1px** na cor preta em elementos que recebem o foco.



Os elementos **a** (links) e os controles de formulário (input, textarea, etc.) recebem o foco por padrão quando o usuário navega com uso do teclado. Para que outros elementos recebam o foco, defina na página uma ordem de tabulação usando o atributo **tabindex** para eles.

:root

Esse seletor representa o elemento raiz do documento. Em marcação HTML é o elemento `html`.

E F:nth-child(pos)

Esse seletor casa com o elemento-irmão do elemento F, filho do elemento E, que ocupa a posição de elemento-filho definida no parâmetro *pos*. O parâmetro *pos* pode assumir os seguintes valores:

- Um número inteiro positivo 1, 2, 3, 4 *etc.* representando a posição do elemento-irmão (primeiro irmão, segundo irmão, terceiro irmão, quarto irmão *etc.*) a casar com o seletor.

- As palavras-chave **odd** (ímpar) e **even** (par), representando os elementos-irmãos de ordem ímpar e par, respectivamente, a ser casados pelo seletor.
- A expressão matemática $an+b$, cujo resultado indica as posições dos elementos-irmãos a casar com o seletor. Nessa expressão, os valores de *a* e *b* devem ser um número inteiro positivo ou negativo ou zero, na qual *n* assume o valor zero e o valor de números inteiros positivos, sequencialmente. Por exemplo: a expressão $3n+2$ casa com $3 \times 0 + 2 = 2$ (segundo), $3 \times 1 + 2 = 5$ (quinto), $3 \times 2 + 2 = 8$ (oitavo)... Segundo, quinto, oitavo e assim por diante, elementos-irmãos do seletor.

Exemplos:

► CSS

```
.um tr:nth-child(3) {...} /* casa com a terceira linha da tabela cuja
classe é um */
#cinco td:nth-child(2) {...} /* casa com a segunda célula de cada linha
da tabela com
id="cinco", ou seja, a segunda coluna */
tr:nth-child(even) {...} ou tr:nth-child(2n) /* casa com as linhas pares
da tabela */
td:nth-child(odd) {...} /* casa com as colunas ímpares da tabela */
tr:nth-child(3n) {...} /* casa com as linhas nas posições 3, 6 e 9 */
tr:nth-child(2n-4) {...} /* casa com as linhas nas posições 2, 4, 6, 8 e
10 */
tr:nth-child(-n+6) {...} /* casa com as 6 primeiras linhas da tabela */
```

► HTML (tabela com dez linhas e três colunas) <table>

```
<tr>
<td>Célula 1-1</td>
```

```

<td>Célula 1-2</td>
<td>Célula 1-3</td>
</tr>
... linhas 2 até 9 ...
<tr>
<td>Célula 10-1</td>
<td>Célula 10-2</td>
<td>Célula 10-3</td>
</tr>
</table>

```

A sintaxe para a expressão $(an+b)$ é bastante rígida e deve estar de acordo com as seguintes diretrizes:

- espaços em branco são permitidos depois de “(” e antes de “)”;
- espaços em branco são permitidos antes e depois dos sinais + ou - que separam an de b .

Por exemplo:

```

( 3n+5 ), ( 3n + 5 ), (3n +5) e (3n+5 ) /* são válidos */
(+ 3n+5), (3 n + 5) e (+ 5) /* são inválidos */

```

E F:nth-last-child(pos)

Esse seletor casa com o elemento-irmão do elemento F, filho do elemento E, que ocupa a posição de elemento-filho definida no parâmetro *pos* contada de trás para frente. O parâmetro *pos* pode assumir os seguintes valores:

- Um número inteiro positivo 1, 2, 3 *etc.* representando a posição de trás para frente do elemento-irmão (último irmão, penúltimo irmão, antepenúltimo irmão *etc.*) a casar com o seletor.

- As palavras-chave **odd** (ímpar) e **even** (par), representando os elementos-irmãos de ordem ímpar e par, respectivamente e contados de trás para frente, a ser casados pelo seletor.
- A expressão matemática $an+b$, cujo resultado indica as posições de trás para frente dos elementos-irmãos a casar com o seletor. Nessa expressão, os valores de a e b devem ser um número inteiro positivo ou negativo ou zero, na qual n assume o valor zero e o valor de números inteiros positivos, sequencialmente. Por exemplo: a expressão $3n+2$ casa com $3 \times 0 + 2 = 2$ (penúltimo), $3 \times 1 + 2 = 5$ (quinto de trás para frente), $3 \times 2 + 2 = 8$ (oitavo de trás para frente)... Segundo, quinto, oitavo e assim por diante, contados de trás para frente, elementos-irmãos do seletor.

Note que esse seletor funciona de forma idêntica ao seletor **:nth-child(pos)**. A diferença é que, para esse seletor, o início da contagem da posição dos elementos-irmãos é do último para o primeiro.

E F:nth-of-type(pos)

Esse seletor casa com o elemento-irmão do elemento F, filho do elemento E, que ocupa a posição de elemento-filho definida no parâmetro *pos*. O parâmetro *pos* pode assumir os seguintes valores:

- Um número inteiro positivo 1, 2, 3, 4 *etc.* representando a posição do elemento-irmão (primeiro irmão, segundo irmão, terceiro irmão, quarto irmão *etc.*) a casar com o seletor.

- As palavras-chave **odd** (ímpar) e **even** (par), representando os elementos-irmãos de ordem ímpar e par, respectivamente, a ser casados pelo seletor.
- A expressão matemática $an+b$, cujo resultado indica as posições dos elementos-irmãos a casar com o seletor. Nessa expressão, os valores de *a* e *b* devem ser um número inteiro positivo ou negativo ou zero, na qual *n* assume o valor zero e o valor de números inteiros positivos, sequencialmente. Por exemplo: a expressão $3n+2$ casa com $3 \times 0 + 2 = 2$ (segundo), $3 \times 1 + 2 = 5$ (quinto), $3 \times 2 + 2 = 8$ (oitavo)... Segundo, quinto, oitavo e assim por diante, elementos-irmãos do seletor.

Note que esse seletor funciona de forma idêntica ao seletor **:nth-child(pos)**. A diferença é que, para esse seletor, os elementos-irmãos a ser considerados na contagem são do mesmo tipo. O exemplo a seguir esclarece a diferença.

Exemplo:

► CSS

```
<div> > :nth-child(3) { color: red; }  
div > p:nth-of-type(3) { color: blue; }
```

► HTML

```
<div>  
<h1>Cabeçalho 1</h1>  
<p>Parágrafo 1</p>  
<span>Texto em span</span> <!-- cor vermelha, aplica-se div p:nth-child(3) - terceiro filho geral --> <p>Parágrafo 2</p>  
<p>Parágrafo 3</p> <!-- cor azul, aplica-se div p:nth-of-type(3) - terceiro filho do tipo p --> </div>
```

E F:nth-last-of-type(pos)

Esse seletor funciona de forma idêntica ao seletor **:nth-of-type(pos)**. A diferença é que, para esse seletor, o início da contagem da posição dos elementos-irmãos do mesmo tipo é do último para o primeiro.

Exemplo:

► CSS

```
div > p:nth-last-of-type(3) { color: blue; }
```

► HTML

```
<div>
<h1>Cabeçalho 1</h1>
<p>Parágrafo 1</p> <!-- cor azul, aplica-se div p:nth-last-of-type(3) -
terceiro filho do tipo p de trás para frente -->
<span>Texto em span</span>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p>
</div>
```

E F:first-child

Esse seletor casa com o elemento F, primeiro filho do elemento E.

Exemplo 1:

► CSS

```
div > p:first-child { color: blue; }
```

► HTML

```
<div>
<p>Parágrafo 1</p> <!-- casa com este parágrafo -->
<span>Texto em span</span>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p>
</div>
```

Exemplo 2:

► CSS

```
div > p:first-child { color: blue; }
```

► HTML

```
<div>
<span>Texto em span</span>
<p>Parágrafo 1</p>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p>
</div>
```

Nesse exemplo não há casamento, pois o primeiro filho do `div` não é um parágrafo.

E F:last-child

Esse seletor casa com o elemento F, último filho do elemento E.

Exemplo 1:

► **CSS**

```
div > p:last-child { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p>
<span>Texto em span</span>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p> <!-- casa com este parágrafo -->
</div>
```

Exemplo 2:

► **CSS**

```
div > p:last-child { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p>
<p>Parágrafo 2</p>
<p>Parágrafo 3</p>
<span>Texto em span</span>
</div>
```

Nesse exemplo não há casamento, pois o último filho do `div` não é um parágrafo.

E F:first-of-type

Esse seletor casa com o elemento F, primeiro filho do elemento E que seja do tipo F.

Exemplo 1:

► **CSS**

```
div > span:first-of-type { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p>
<span>Texto em span</span> <!-- casa com este span -->
<p>Parágrafo 2</p>
<span>Texto em span</span>
<p>Parágrafo 3</p>
</div>
```

E F:last-of-type

Esse seletor casa com o elemento F, último filho do elemento E que seja do tipo F.

Exemplo:

► **CSS**

```
div > span:last-of-type { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p>
<span>Texto em span</span>
<p>Parágrafo 2</p>
<span>Texto em span</span> <!-- casa com este span -->
<p>Parágrafo 3</p>
</div>
```

E F:only-child

Esse seletor casa com o elemento F, desde que ele seja o único filho do elemento E.

Exemplo 1:

► **CSS**

```
div > p:only-child { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p> <!-- casa com este parágrafo -->
</div>
```

Exemplo 2:

► **CSS**

```
div > p:only-child { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p>
<span>Texto em span</span>
</div>
```

Nesse exemplo não há casamento, pois `div` tem mais de um filho.

Exemplo:

► **CSS**

```
div > p:only-child { color: blue; }
```

► **HTML**

```
<div>
<p>Parágrafo 1</p> <!-- casa com este parágrafo -->
</div>
```


E F:only-of-type

Esse seletor casa com o elemento F, desde que ele seja o único filho do elemento E e do tipo F.

Exemplo:

► CSS

```
div > p:only-of-type { color: red; }
```

► HTML

```
<div>
<span>Texto em span</span>
<p>Parágrafo 1</p> <!-- casa com este parágrafo -->
<a href="#">Link 1</a>
</div>
```

E:empty

Esse seletor casa com o elemento E, desde que ele esteja vazio, ou seja, não possua nenhum elemento-filho e nem mesmo nós de texto, nós CDATA ou referências a entidades de caracteres.

Exemplo:

► CSS

```
td:empty { background-color: green; }
```

► HTML

```
<table>
<tr>
<td>Célula</td>
<td>&nbsp;</td>
<td></td> <!-- casa somente com esta célula -->
<td><p>Texto</p></td>
</tr>
</table>
```

E:target

Esse seletor casa com o elemento E, desde que ele seja o destino de um link interno da página. Link interno em uma página é aquele que, ao ser clicado, leva o usuário para um determinado lugar dentro da própria página. Esse tipo de link permite construir um sistema de navegação na página, e um bom exemplo do seu uso é em páginas tipo FAQ – Frequently Asked Questions (perguntas frequentes ou mais comuns).

Páginas tipo FAQ são aquelas que logo no início apresentam uma lista de

perguntas. Os itens da lista (ou perguntas) são links que conduzem ao início do texto resposta. As respostas, por sua vez, são listadas em ordem dentro da própria página. O exemplo a seguir esclarece tanto o funcionamento desse seletor como o mecanismo de construção de links internos na página.

Exemplo:

► **CSS**

```
li:target { background-color: lime; }
```

► **HTML**

```
<h2>Índice</h2>
<ol>
<li><a href="#css">O que é CSS?</a></li>
<li><a href="#sel">O que é seletor CSS?</a></li>
...
<li><a href="#fl">O que é float?</a></li>
</ol>
<h2>Respostas</h2>
<ol>
<li id="css">CSS é a abreviatura para...</li>
<li id="sel">Seletor é um mecanismo...</li>
...
<li id="fl">Float é um esquema de posicionamento...</li>
</ol>
```

Note que na marcação HTML o atributo `href` dos links aponta para uma string precedida do sinal tralha, ou jogo da velha (por exemplo: `#css`). Essa sintaxe instrui o navegador a rolar a página até encontrar o elemento marcado com `id` igual à string (por exemplo: `id="css"`) quando o link é clicado.

O seletor `:target` casa com o destino de um link interno na página. Trata-se de um seletor de classe dinâmico, isto é, a regra CSS definida com uso desse seletor é aplicada no momento em que o usuário clica no link, e seu efeito permanece até que um novo link seja clicado.

Dessa forma, no exemplo mostrado, ao se clicar na primeira pergunta o texto da resposta à primeira pergunta ganhará um fundo verde (lime). Se em seguida clicarmos na terceira pergunta, o texto da primeira perderá o fundo na cor verde e a terceira resposta ganhará o fundo nessa cor.

E:lang(val)

Esse seletor casa com o elemento `E` para o qual tenha sido definido o atributo `lang` (atributo destinado a definir o idioma em que foi escrito o conteúdo do elemento) com o valor `val`.

Exemplo:

► CSS

```
span:lang(en) { color: grey; }
```

► HTML

<p>A tradução de beautiful é bonito.</p> Nesse exemplo, a palavra `beautiful` será estilizada na cor cinza (`grey`).

Há uma diferença fundamental entre esse seletor e o seletor de atributo `E[lang="val"]`, estudado anteriormente. Esse seletor é capaz de reconhecer conteúdos escritos em linguagem diferente daquela do documento como um todo, empregando um mecanismo nativo do agente de usuário que vai além de casar somente com elementos que contenham explicitamente o atributo `lang`. O exemplo mostrado a seguir esclarece a diferença.

Exemplo 1:

► CSS

```
em:lang(en) { font-style: normal; color: red; }
```

► HTML

<p>Eu amo o Rio é: I love Rio
</p> Nesse exemplo, a regra de estilo é aplicada e a palavra `love` será em fonte normal (não itálica) e na cor vermelha (`red`).

Exemplo 2:

► CSS

```
em:[lang!="en"] { font-style: normal; color: red; }
```

► HTML

<p>Eu amo o Rio é: I love Rio
</p> Nesse exemplo, a regra de estilo não será aplicada à palavra `love`. A fonte será itálica (padrão) e na cor preta (padrão).

E:not(sel)

Esse seletor casa com o elemento `E`, desde que ele não tenha sido definido no parâmetro `sel`, que por sua vez é um seletor simples (seletores compostos não são válidos).

Exemplo:

► CSS

```
em:not(:first-child) { color: red; }
```

► HTML

`<p>João gosta de <em>laranja</em>, Maria de <em>uva</em> e Paulo de <em>banana</em>.</p>` Nesse exemplo, as palavras uva e banana serão estilizadas na cor vermelha (red). A regra CSS estabelece que todos os elementos `em` serão na cor vermelha, exceto o elemento `em` primeiro filho.

E:enabled e E:disabled

Esse seletor casa com o elemento E da Interface de Usuário, cujo estado é habilitado (**E:enabled**) ou desabilitado (**E:disabled**), respectivamente.

Exemplo:

► CSS

```
input:enabled { border: 1px solid green; }
input:disabled { border: 1px solid red; }
```

► HTML

```
<form>
<input type="text" />
<input type="text" disabled="disabled" />
</form>
```

Nesse exemplo, o primeiro controle `input` terá uma borda na cor verde (green), e o segundo, na cor vermelha (red).

E:checked

Esse seletor casa com o elemento E da Interface de Usuário, cujo estado é selecionado. Aplica-se aos controles de formulário dos tipos **radio** e **checkbox** e a alguns itens de **menu** quando o usuário os seleciona.

Exemplo:

► CSS

```
input:checked { ... }
```

► HTML

```
<form>
<input type="radio" name="" value="" />
<input type="checkbox" name="" value="" />
</form>
```

Nesse exemplo, os dois controles `input` serão estilizados conforme a regra CSS. Convém ressaltar que a estilização desses tipos de controle é inconsistente entre navegadores e a maioria das propriedades CSS de estilização é por eles simplesmente ignorada.

4.2.15 Seletor pseudoelemento

Pseudoelemento é um conceito que possibilita selecionar uma parte do documento que não consta da árvore do documento, ou seja, criar abstrações não definidas na linguagem de marcação em que o documento foi escrito. A sintaxe para aplicar esse conceito é constituída de um sinal de dois-pontos dobrado (::) seguido do nome do pseudoelemento, conforme mostrado a seguir: **p::first-letter** { color: blue; }
/* A primeira letra dos parágrafos será na cor azul */

```
div::before { content: "Nota: "; } /* Após cada elemento div será escrito o texto "Nota: " */
```

Os pseudoelementos foram criados com as CSS1 e CSS2 e são :first-letter, :first-line, :before e :after. As especificações para as CSS3 alteraram a sintaxe dos pseudoelementos com a finalidade de diferenciá-los das pseudoclasses, acrescentando mais um sinal de dois pontos (:) no início, e a sintaxe atual passou a ser ::first-letter, ::first-line, ::before e ::after.

Os seletores pseudoelementos são estudados a seguir.

E::first-letter

Esse seletor casa com o primeiro caractere da primeira linha formatada do elemento E, se essa linha efetivamente começar com uma letra ou dígito. Havendo antes do primeiro caractere um conteúdo do tipo imagem ou tabela inline, por exemplo, não haverá casamento com o seletor.

Somente as seguintes propriedades CSS se aplicam ao seletor **::first-letter**:

- text-decoration

- text-transform
- letter-spacing
- word-spacing
- line-height
- float
- vertical-align (somente se float for none)
- margin
- padding
- border
- color
- background

Exemplo:

► CSS

```
p::first-letter { color: red; }
```

► HTML

```
<p>Parágrafo um</p>
<p>35 Parágrafo dois</p>
<p><br > Parágrafo três</p>
```

Nesse exemplo, a letra P do primeiro parágrafo e o número 3 do segundo parágrafo são estilizados na cor vermelha (**red**). Para o terceiro parágrafo não há estilização, pois a primeira linha formatada é uma quebra de linha e não tem letra ou número.

E::first-line

Esse seletor casa com a primeira linha de conteúdo formatado do elemento E. Note que a primeira linha de um texto varia de acordo com vários fatores, como a largura da janela do navegador e o tamanho da fonte.

Elementos-filhos que ocorram na primeira linha herdam as propriedades CSS definidas nesse seletor; contudo, se tais elementos não estiverem na primeira linha, herdarão as propriedades CSS do seu elemento-pai.

Somente as seguintes propriedades CSS se aplicam ao seletor **::first-line**:

- **text-decoration**

- **text-transform**
- **letter-spacing**
- **word-spacing**
- **line-height**
- **vertical-align**
- **font**
- **color**
- **background**

Exemplo:

► CSS

```
p::first-line { text-transform: uppercase; }
```

► HTML

```
<p>Parágrafo bla...bla...bla...</p>
```

Nesse exemplo, a primeira linha do parágrafo será em caixa-alta.

E::before e **E::after**

Esse seletor destina-se a inserir conteúdos antes (**E::before**) ou depois (**E::after**) do conteúdo constante da marcação do elemento E. O conteúdo é gerado pela propriedade CSS **content** e, uma vez que não consta da marcação, também não faz parte da árvore do documento.

Exemplo:

► **CSS**

```
p.nota::before { content: "Nota: "; }
```

► **HTML**

```
<p class="nota">Convém ressaltar que ...</p>
```

Nesse exemplo, o parágrafo será renderizado conforme mostrado a seguir:

Nota: Convém ressaltar que...

Note que o valor da propriedade **content** (**Nota:**) foi inserido no início do parágrafo.

Os conteúdos gerados e inseridos com uso desse seletor passam a integrar o conteúdo do box a que pertencem. Assim, as propriedades CSS aplicadas ao elemento que recebeu o conteúdo serão aplicadas ao conteúdo inserido normalmente. Se definirmos uma borda para o parágrafo mostrado no exemplo anterior, a borda abrangerá todo o parágrafo, inclusive a palavra *Nota:*.

Da mesma forma, os conteúdos gerados e inseridos com uso desse seletor herdam os valores das propriedades CSS definidas para o elemento ao qual foram inseridos; no nosso exemplo, herdam do parágrafo.

O valor inicial da propriedade CSS **display** para conteúdos gerados e inseridos é **inline**. Contudo, tanto a herança como esse valor inicial ou o valor de qualquer propriedade CSS para o conteúdo podem ser sobrescritos por regra CSS específica. O exemplo a seguir esclarece a aplicação dessas regras.

Exemplo:

► **CSS**

```
p.nota { color: black; }
p.nota::before {
  content: "Nota: ";
  display: block;
  color: red;
```

```
}
```

► **HTML**

```
<p class="nota">Convém ressaltar que ...</p>
```

Nesse exemplo, o parágrafo será renderizado conforme mostrado a seguir:

Nota: (renderizado na cor vermelha) *Convém ressaltar que...* (renderizado na cor preta)
Pela herança CSS, o conteúdo gerado e inserido (*Nota:*) deveria ser renderizado na cor preta (black) e, por padrão, inline com o texto constante da marcação do parágrafo. Contudo, regra CSS específica foi aplicada ao conteúdo gerado, fazendo-o nível de bloco e na cor vermelha (red).



O site css3.info oferece uma excelente ferramenta destinada a verificar o suporte aos seletores CSS3 pelos diferentes navegadores. Se você quer verificar se um navegador suporta determinado seletor, abra a página <http://kwz.me/rd> no navegador a testar e clique no botão para iniciar o teste.

4.2.16 Seletor simples

Os seguintes seletores são considerados seletores simples: o seletor universal, o seletor tipo, o seletor atributo, o seletor classe, o seletor id e as pseudoclasses.

4.2.17 Grupamento de seletores

É válido agrupar seletores que compartilham uma mesma regra CSS. Seletores devem ser agrupados com uso de vírgula como separador. A sintaxe para agrupar seletores é mostrada a seguir: **div, p, span** { color: red } /* Elementos div, p e span serão na cor vermelha */

4.2.18 Caracteres de combinação

Caracteres ou sinais de combinação são os caracteres destinados a escrever a sintaxe de um seletor composto por outros seletores. Os sinais de combinação de seletores são: espaço em branco, sinal maior que (>), sinal de adição (+) e sinal til (~). A sintaxe de uso dos sinais de combinação de seletores é mostrada a seguir: **div p** { ... } /* Elemento p descendente de div */

```
div > p { ... } /* Elemento p filho de div */
```

```
div + p { ... } /* Elemento p imediatamente após div */
```

```
div ~ p { ... } /* Elementos div e p são irmãos e p vem depois do div */
```

4.2.19 Sintaxe geral de um seletor

Seletor é uma cadeia de uma ou mais sequências de seletores simples, separados por um caractere de combinação. Um pseudoelemento pode ser adicionado somente ao último seletor simples da cadeia.

A grafia de seletores é insensível ao tamanho de caixa, mas deve ser respeitada a sensibilidade de caixa para nomes de elementos e nomes e valores de atributos que

fazem parte de um seletor. Por exemplo: em XML, nomes de elementos devem ser em minúsculas, portanto devem ser grafados também em minúsculas, quando presentes em um seletor.

4.2.20 Seletores por namespace

As especificações para as CSS3 criaram um mecanismo que permite estilizar elementos de marcação com base no namespace a que eles pertencem. Esse mecanismo oferece uma maneira simples e direta de estilizar elementos em documentos XML. Os seletores que admitem definição de namespace para estilização são os seletores universais, seletores tipos (seletor elemento) e seletores de atributo.

A sintaxe geral para esses seletores é constituída de uma string identificadora do namespace seguida por uma barra vertical (|) e pelo seletor, conforme mostrado a seguir: `ns|E { color: red; }` /* Todo elemento E que pertence ao namespace ns será na cor vermelha (red) */

Ao se declarar seletores com uso de namespace, é necessário especificar no início da folha de estilo o URI do(s) namespace(s) com uso da diretiva `@namespace`, como mostrado a seguir: `@namespace foo url(http://www.exemplo.com/foo);`

```
foo|E { color: red; } /* regra CSS para elemento E no namespace foo */
```

Observe, a seguir, as diferentes opções para a sintaxe de seletores em namespace:

```
@namespace foo url(http://www.exemplo.com/foo);
foo|h1 { color: blue } /* casa com h1 do namespace foo */
foo|* { color: yellow } /* casa com todos os elementos do namespace foo */
|h1 { color: red } /* casa com h1 sem namespace declarado */
*|h1 { color: green } /* casa com h1 em qualquer namespace */
h1 { color: green } /* equivalente à anterior, casa com h1 em qualquer namespace */
```

Exemplo:

► CSS

```
@namespace http://www.exemplo.com/Catalogo;
@namespace pessoa http://www.exemplo.com/Pessoa;
catalogo { background-color: yellow; }
pessoa|Nome { color: red; };
pessoa|telefone [ color: blue; }

}
```

► XML

```
<catalogo xmlns="http://www.exemplo.com/Catalogo"
xmlns:pessoa="http://www.exemplo.com/Pessoa"> <pessoa:Pessoa>
```

```
< Pessoa:Nome>Maurício Samy Silva< Pessoa:Nome>  
< Pessoa:Endereco>Rua: Paulo Silva, 34 apt 500< / Pessoa:Endereco>  
< Pessoa:Telefone>+55(xx)1234-5678< / Pessoa:Telefone>  
< / Pessoa:Pessoa>  
...  
< / catalogo>
```

4.2.21 Especificidade

Um dos fatores que influenciam na determinação da ordem do efeito cascata é a especificidade do seletor. Especificidade é uma característica que leva em conta a maneira como o autor escreveu o seletor.

Vamos mostrar um exemplo para esclarecer esse conceito. Considere um elemento `p`, filho de um elemento `div`, e observe as quatro regras de estilo a seguir, destinadas a aplicar uma cor ao elemento `p`.

```
p { color: red; }  
div p { color: blue; }  
body div p { color: pink; }  
div > p { color: yellow; }
```

Note que os quatro padrões casam com o elemento `p` e o conflito está criado. Pelos critérios de determinação da ordem do efeito cascata, há um empate no que diz respeito à origem e à importância das declarações, já que todas se encontram na mesma folha de estilo e nenhuma delas foi declarada com a diretiva `!important`.

Assim, entra a especificidade. Como calcular a especificidade? Leia o subitem a seguir e conclua sobre a cor a ser aplicada ao parágrafo mostrado no exemplo.

4.2.20.1 Cálculo da especificidade de um seletor

Para realizar o cálculo da especificidade de um seletor, examinamos a maneira como ele foi escrito. Olhando para as partes que formam o seletor, criaremos um número constituído de quatro dígitos no formato “abcd”, ou seja, primeiro dígito, `a`; segundo, `b`; terceiro, `c`; e quarto, `d`. No final, teremos formado um número para cada seletor conflitante. O maior número tem precedência e a regra de estilo a ele correspondente será aplicada. Lembro que, se houver empate, aplica-se o item 4 da ordem da cascata.

Os passos para formar o número que representa o seletor são:

- caso a regra de estilo seja inline, isto é, aplicada no elemento com uso do atributo `style`, o valor de `a` é um (`a=1`). Caso contrário (a regra está em uma folha de estilo), o valor de `a` é zero (`a=0`);
- o valor de `b` é igual ao número de vezes que o atributo `id` (um sinal `#`) aparece no seletor (`b=`);
- o valor de `c` é igual ao número de vezes que outros atributos (que não `id`) e pseudoclasses

aparecem no seletor (c=); • o valor de d é igual ao número de vezes que nomes de elementos e de pseudoelementos aparecem no seletor (d=).

Observe os exemplos a seguir, que demonstram o cálculo da especificidade de alguns seletores: * {} /* a=0 b=0 c=0 d=0 -> especificidade = 0000 */

```
li {} /* a=0 b=0 c=0 d=1 -> especificidade = 0001 */
li:first-line {} /* a=0 b=0 c=0 d=2 -> especificidade = 0002 */
ul li {} /* a=0 b=0 c=0 d=2 -> especificidade = 0002 */
ul ol+li {} /* a=0 b=0 c=0 d=3 -> especificidade = 0003 */
h1 + *[rel=up] {} /* a=0 b=0 c=1 d=1 -> especificidade = 0011 */
ul ol li.red {} /* a=0 b=0 c=1 d=3 -> especificidade = 0013 */
li.red.level {} /* a=0 b=0 c=2 d=1 -> especificidade = 0021 */
#x34y {} /* a=0 b=1 c=0 d=0 -> especificidade = 0100 */
style="" /* a=1 b=0 c=0 d=0 -> especificidade = 1000 */
```

4.2.22 Herança CSS

Herança é um mecanismo de propagação de valores de propriedades CSS de um elemento para seus elementos descendentes. Nem todas as propriedades CSS são herdadas, as especificações para as CSS definem as propriedades herdadas. Para as propriedades CSS que são herdadas, o valor especificado [1.1.6.2] para um determinado elemento é igual ao valor computado [1.1.6.3] para seu elemento-pai.

Todas as propriedades CSS admitem a palavra-chave `inherit` para explicitamente declarar que a propriedade deverá ser herdada, mesmo as propriedades que constam das especificações como sendo não herdadas. Nesses casos, o valor `inherit` poderá ser usado para anular um valor explicitamente declarado. Considere a marcação e a regra CSS conforme mostradas a seguir: ► **HTML**

```
<div>
<p>Parágrafo com <em>palavra</em> marcada com ênfase</p>
</div>
```

► CSS

```
div {
color: red;
border: 1px solid blue;
```

```
}
```

Nesse exemplo, a declaração em regra CSS de uma cor vermelha e borda na cor azul foi para o elemento `div`. O elemento `p` terá seu conteúdo estilizado na cor vermelha, pois, sendo filho do `div`, herda o valor da cor. Contudo, não herdará a borda na cor azul, pois não são todas as propriedades CSS que são passadas por herança. O mesmo acontece com o conteúdo do elemento `em`, que herdará de `p` a cor vermelha. Note que a herança vai se transmitindo para os descendentes, caso o

elemento-pai tenha herdado um valor.

Considere o acréscimo a seguir, em destaque, no exemplo mostrado anteriormente:

► **HTML**

```
<div>
<p>Parágrafo com <em>palavra</em> marcada com ênfase</p>
</div>
```

► **CSS**

```
div {
color: red;
border: 1px solid blue;

}
```

em { color: green; }

Nesse caso, o elemento **em** não mais herdará a cor vermelha do seu elemento-pai **p**, pois a ele foi explicitamente declarado o valor verde para a cor.

Felizmente, a herança foi criada somente para aquelas propriedades que visem a facilitar e simplificar a aplicação de regras de estilos. Imagine a situação em que se define uma borda ou uma imagem de fundo para o elemento **body** e os valores definidos para essas propriedades fossem herdados por todos os elementos-filhos e, conseqüentemente, por seus descendentes. Nessa hipótese, todos os elementos da página teriam uma borda e uma imagem de fundo, o que seria desastroso. Entre outras, as propriedades CSS **border** e **background** não são passadas por herança.

Por outro lado, os valores da propriedade CSS **font** passam por herança e isso é muito conveniente, pois uma declaração conforme a mostrada a seguir define as características da fonte para todo o site.

```
body { font: 12px/1.4 Arial, Sans-Serif; }
```

Todas as propriedades CSS admitem o valor **inherit**, que força a herança, ou seja, obriga o elemento a herdar do seu elemento-pai a propriedade, mesmo que ela não passe por herança naturalmente.

Considere o acréscimo a seguir, em destaque, no exemplo mostrado anteriormente:

► **HTML**

```
<div>
<p>Parágrafo com <em>palavra</em> marcada com ênfase</p>
</div>
```

► CSS

```
div {  
  color: red;  
  border: 1px solid blue;  
  
}
```

```
p { border: inherit; }  
em { color: green; border: inherit; }
```

Nesse caso, tanto o elemento `p` como o elemento `em` serão estilizados com uma borda na cor azul, herdada do elemento `div`.

4.2.23 Efeito cascata

Folhas de estilo podem ser de três origens diferentes:

- **Autor:** folha de estilo do autor é aquela criada pelo desenvolvedor do documento e se destina a controlar sua apresentação de acordo com o que foi projetado pelo designer.
- **Usuário:** para contemplar questões de acessibilidade. Os agentes de usuário devem prover um mecanismo capaz de proporcionar ao usuário a criação de uma folha de estilo própria. Por exemplo: os usuários com necessidades especiais de visão poderão se valer desse mecanismo com a finalidade de criar uma folha de estilo, definindo cores de fundo e de frente com alto contraste, bem como tamanho de fontes aumentado para serem empregadas nos sites que navega.
- **Agente de usuário:** os agentes de usuário possuem nativamente uma folha de estilos que define, por padrão, os valores das propriedades CSS dos elementos de uma linguagem de marcação. Por exemplo: na grande maioria dos navegadores, em marcações HTML, o elemento `em` é renderizado, visualmente, em itálico.

As regras de estilo contidas nas folhas de estilo provenientes dessas três origens e aplicáveis a um único documento podem se sobrepor, criando conflitos.



Diz-se que regras de estilo são conflitantes quando, definindo valores diferentes para uma mesma propriedade, elas se aplicam a um mesmo elemento da marcação.

Regras de estilo conflitantes podem estar contidas tanto em diferentes folhas de estilo como em uma mesma folha de estilo.

Exemplo de regras de estilo conflitantes:

```
p {  
  color: white;
```

```
background: blue;

}

...
h4, h5, p {
color: red;
background: green;

}
```

Destacamos, no código mostrado anteriormente, o conflito na declaração de estilo para a cor dos parágrafos. Qual será a cor aplicada aos parágrafos? Aqui entra em ação o efeito cascata para resolver o conflito.

4.2.24 Ordem de cascata

As especificações para as CSS3 estabelecem as seguintes diretrizes, que regem os cálculos para determinar a ordem do efeito cascata: 1. Encontrar, para cada tipo de mídia, todas as propriedades de estilização que se aplicam ao elemento, identificando suas origens.

2. Classificar, em ordem crescente de precedência, os tipos encontrados de acordo com a importância da declaração (normal e **!important**) e a origem (autor, usuário, agente de usuário).

- declarações do agente de usuário;
- declarações normais do usuário;
- declarações normais do autor;
- declarações importantes do autor;
- declarações importantes do usuário;

3. Classificar as regras de mesmo peso (ou seja, mesma importância) e origem pela sua especificidade. Seletores mais específicos têm precedência sobre os menos específicos. Pseudoelementos e pseudoclasses são considerados, para fins de especificidade, como elementos e classes normais, respectivamente.

4. Finalmente, classificar levando em consideração a posição que ocupam na folha de estilo. Para declarações de mesmo peso, mesma origem e mesma especificidade, prevalece aquela que aparece mais abaixo na folha de estilo.

Note que a palavra final quando se trata de estilização é do usuário, com sua folha de estilo personalizada contendo declarações que usam a regra **!important**.

4.2.25 Diretiva **!important**

As CSS tentam estabelecer um balanceamento entre as regras de estilo do autor e as do usuário. Por padrão, uma regra de estilo do autor prevalece sobre uma do usuário. A diretiva **!important** foi criada com a finalidade de dar precedência a uma declaração que entre em conflito com outra de mesmo peso, origem e especificidade. Ao mesmo tempo, essa diretiva inverte a precedência entre autor e usuário, ou seja, para regras de estilo que tenham sido declaradas com uso de **!important**, a regra criada pelo usuário prevalece sobre a do autor. Isso possibilita ao usuário visualizar um site estilizando-o da maneira mais conveniente para ele, independentemente das regras de estilo de outras fontes, como alto contraste de cores e tamanhos de fonte aumentados.

A sintaxe para uso da diretiva **!important** é mostrada a seguir: `p { color: red !important; }`

CAPÍTULO 5

Background

As CSS3 criaram novos mecanismos para estilização de cores e imagens de fundo dos boxes da marcação. Com as CSS3 é possível inserir múltiplas imagens de fundo. Além dessa inovação, não constante das versões anteriores das CSS, muitas outras funcionalidades foram introduzidas pelo Módulo CSS3 para Backgrounds e Borders, destacando-se aquelas destinadas a permitir um maior controle sobre posicionamento e segmentação das imagens de fundo. Neste capítulo, estudaremos as propriedades CSS para background, constantes desse módulo.

5.1 Introdução

Por padrão, cada box CSS é constituído de uma layer transparente, a qual pode ser preenchida, com uso de regra CSS, por uma cor de fundo ou por uma ou mais imagens. A propriedade CSS destinada a definir uma cor de fundo é **background-color** e para uma imagem de fundo é **background-image**.

Nenhuma das propriedades para definição de background é herdada, ou seja, cores ou imagens de fundo definidas para um elemento da marcação não são herdadas pelos seus elementos descendentes. Mas, uma vez que a layer de fundo é transparente por padrão, uma cor ou imagem definida para um container será vista através das layers dos elementos-filhos. Observe o exemplo a seguir.

Exemplo:

► CSS

```
div {  
padding: 20px;  
border: 2px solid black;  
  
}
```

```
#um { background-color: #ccc; }
```

► HTML

```
<div id="um">  
um  
<div id="dois">  
dois  
<div id="tres">  
tres  
</div>  
</div>
```


</div>

Nesse exemplo, foram aninhados três elementos `div`. Os `div#dois` e `div#tres` são descendentes do `div#um`. Definimos uma cor de fundo cinza-claro (`#ccc`) para o `div#um` e, uma vez que as propriedades de fundo não são herdadas, os `div#dois` e `div#tres` não deveriam ter a cor de fundo cinza-claro; contudo, eles a têm, não por herança, mas pelo fato de suas layers de fundo serem transparentes por padrão, mostrando a cor cinza aplicada ao `div#um` através deles.

As propriedades para definição de fundos admitem o valor `inherit`, que força um elemento descendente a herdar os valores de fundo definidos para seu elemento ancestral.

5.2 Propriedades background

As propriedades para estilizar fundos são listadas a seguir:

- `background-color`
- `background-image`
- `background-repeat`
- `background-attachment`
- `background-position` (criada nas CSS3) • `background-clip` (criada nas CSS3) • `background-origin` (criada nas CSS3) • `background-size` (criada nas CSS3) • `background`

Estudaremos, a seguir, cada uma dessas propriedades.

5.2.1 background-color

Essa propriedade define uma cor de fundo para o box. Os valores possíveis são: valores CSS para cores, conforme mostrado no capítulo 7, o valor `transparent` e o valor `inherit` (um valor herdado).

Vamos examinar os códigos que se seguem e comprovar o comportamento das cores de fundo definidas com uso dessa propriedade CSS, e também como elas se distribuem por elementos ascendentes e descendentes, com cores de fundo diferentes.

► HTML

```
<header>
<hgroup>
<h1>Cabeçalho nível um</h1>
<h2>Cabeçalho nível dois</h2>
</hgroup>
```

```
<p>Texto do parágrafo</p>
</header>
```

A marcação mostra a seção **header** de um documento HTML5. Essa seção tem dois elementos-filhos que são **hgroup** e **p**. O elemento **hgroup**, por sua vez, tem dois elementos-filhos, **h1** e **h2**. A relação entre elementos na árvore do documento é a seguinte:

- **header** é elemento-pai de **hgroup** e **p** e ancestral de **h1** e **h2**;
- **hgroup** é elemento-pai de **h1** e **h2**;
- **h1** e **h2** são elementos-filhos de **hgroup** e descendentes de **header**.

Considere a seguinte folha de estilo aplicada à marcação mostrada:

► **CSS**

```
hgroup, h1, h2, p {
border: 1px solid black;
padding: 10px;

}

header {
background-color: #999; /* cinza escuro */
border: 5px dashed black;
padding: 10px;

}

h1 { background-color: #ccc; } /* cinza claro */
p { background-color: #ccc; } /* cinza claro */
hgroup { background-color: #fff; } /* branco */
```

A folha de estilo aplicada à marcação define uma cor de fundo cinza-escura para o elemento **header** (container da marcação), uma cor de fundo cinza-clara para os elementos **h1** e **p** e a cor branca para fundo do elemento **hgroup**. Note que não foi definida uma cor de fundo para o elemento **h2**.

Para fins de clareza do exemplo, todos os elementos receberam uma borda na cor preta e um **padding** nos quatro lados. A borda do container é tracejada.

Na figura 5.1, observe o resultado no navegador.

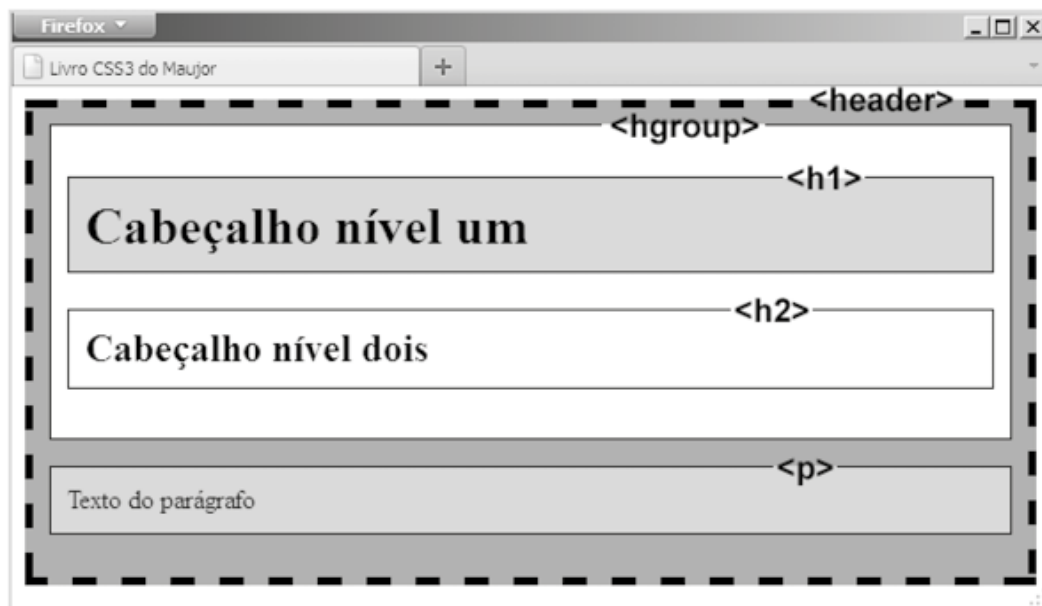


Figura 5.1 – Cores de fundo.

As especificações dizem que o valor inicial da propriedade **background-color** é **transparent**, isto é, elementos para os quais não foi definida explicitamente uma cor de fundo terão o fundo transparente. Isso foi o que aconteceu com o elemento **h2**. Seu fundo não foi definido e, sendo transparente, podemos ver o fundo que está atrás (o fundo na cor branca definido para seu elemento-pai **hgroup**).

Ainda segundo as especificações, o fundo de um elemento abrange a área de conteúdo, a área de **padding** e a área da borda. Pode-se observar esse comportamento examinando o fundo cinza-escuro definido para o elemento **header**. Note que a cor ocupa a área de conteúdo, a área de **padding** e os espaços deixados entre os traços da borda tracejada. Convém ressaltar que bordas são colocadas por padrão à frente dos fundos (cores ou imagens). Para que isso fosse mostrado, no exemplo, definimos borda tracejada; fosse a borda sólida, ela cobriria a cor de fundo. Note que para os demais elementos a borda sólida se sobrepõe à cor do fundo.

5.2.2 background-image

Essa propriedade define uma ou mais imagens a serem usadas como fundo de um box. Os valores possíveis são: um URI (endereço apontando para a imagem), **none** ou **inherit** (um valor herdado).

Recomenda-se aos autores que sempre que se defina uma imagem de fundo, defina também uma cor para o fundo. A cor será usada, caso a imagem esteja indisponível. As imagens serão colocadas sobre a cor do fundo e, havendo áreas transparentes nas imagens, será revelada a cor de fundo através delas. A sintaxe da regra CSS para essa propriedade é a seguinte: `div { background-image: url("imagensbg-div.png"); }`



Observe que usamos aspas duplas no endereço da imagem. Alternativamente, a sintaxe CSS admite usar aspas simples ou simplesmente não usar aspas.

Na maioria das aplicações práticas, essa propriedade é definida em conjunto com outras propriedades de fundo, como veremos adiante.

Quando se definem múltiplas imagens a serem aplicadas como fundo de um box, a ordem em que elas são definidas é importante, pois cada uma delas cria uma layer, e elas são empilhadas na ordem inversa em que são definidas as imagens na regra CSS, ou seja, a primeira imagem definida posiciona-se sobre a segunda, esta sobre a terceira e assim por diante.

Considere que a regra CSS mostrada a seguir é um fragmento de uma folha de estilo que define imagens de fundo para o elemento `section` de uma página com marcação HTML5.

► CSS

```
section {  
  background-image: url(surfista.jpg);  
  background-image: url(sol.jpg);  
  background-image: url(praia.jpg);  
}
```

Para essa ordem de declaração das imagens de fundo, a imagem mais à frente é *surfista.jpg*, depois *sol.jpg* e, por último, *praia.jpg*. Supondo que a imagem *praia.jpg* cubra toda a área do elemento `section` e as outras duas imagens ocupem uma pequena área, se declararmos *praia.jpg* em primeiro, ela cobrirá as outras duas que, em consequência, não serão mostradas.

5.2.3 background-repeat

Essa propriedade define como a imagem se repetirá no fundo do elemento. Os valores possíveis são: `repeat`, `repeat-x`, `repeaty`, `no-repeat`, `space`, `round` ou `inherit` (um valor herdado).

Para demonstrar o efeito de uma declaração com cada um desses valores, considere os códigos que se seguem e veja, na figura 5.2, o resultado no navegador.

► HTML

```
<div class="um"></div>  
<div class="dois"></div>  
<div class="tres"></div>  
<div class="quatro"></div>  
<div class="cinco"></div>
```

```
<div class="seis"></div>
```

Na marcação mostrada, temos seis elementos `div` que, para fins de clareza do exemplo, serão posicionados em duas linhas no navegador. Os elementos foram definidos com largura e altura fixa e com uma borda. Para cada `div` será atribuída a mesma imagem de fundo e a repetição da imagem será segundo as seguintes regras CSS: ► **CSS**

```
div { background-image: url(moeda.png); }  
div.um { background-repeat: repeat; }  
div.dois { background-repeat: repeat-x; }  
div.tres { background-repeat: repeat-y; }  
div.quatro { background-repeat: no-repeat; }  
div.cinco { background-repeat: space; }  
div.seis { background-repeat: round; }
```

Observe, na figura 5.2, o resultado da aplicação das regras CSS nos seis elementos `div` e a maneira como se repete a imagem de fundo.

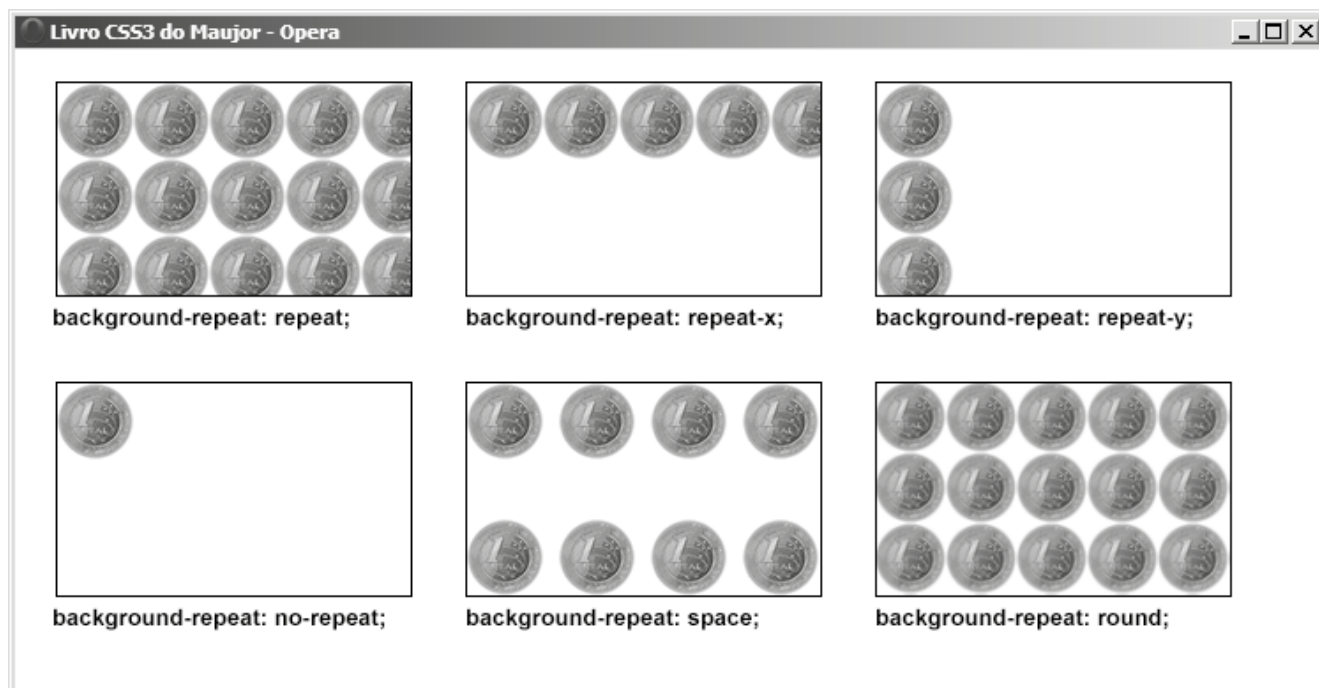


Figura 5.2 – Propriedade *background-repeat*.

Se não for definida uma posição inicial para a imagem (como veremos adiante), ela será posicionada inicialmente no canto superior esquerdo do elemento. O valor **repeat** faz com que a imagem se repita a partir de sua posição inicial, tanto na horizontal (eixo x) como na vertical (eixo y).

Os valores **repeat-x** e **repeat-y** causam a repetição da imagem na horizontal (eixo x) e na vertical (eixo y), respectivamente.

O valor **no-repeat** coloca a imagem na sua posição inicial e não há repetição. Adiante, iremos estudar como definir a posição inicial da imagem com uso da

propriedade **background-position**.

O valor **space** repete as imagens, de modo que elas encostem nas bordas internas do box, e distribui as imagens no box, espaçando-as o quanto for necessário para distribuí-las igualmente no fundo.

O valor **round** repete as imagens, de modo que elas encostem nas bordas internas do box, e distribui as imagens no box sem espaçá-las, redimensionando as imagens para acertá-las no interior do box.



A diferença entre **space** e **round** é que, para **space**, as imagens não são redimensionadas ajustando-se o espaçamento entre elas, e, para **round**, não há espaçamentos entre imagens, e elas são redimensionadas para ocupar todo o box.

5.2.4 background-attachment

Essa propriedade determina se a imagem ficará fixa ou se irá rolar junto com o conteúdo do elemento no qual ela foi declarada. Por exemplo: se a imagem for colocada como fundo do elemento **body**, a propriedade define se irá rolar com a tela quando o usuário acionar a barra de rolagem ou se permanecerá fixa na tela durante a rolagem. Esse é o emprego mais comum para essa propriedade, ou seja, a imagem permanece fixa na tela mesmo que o usuário acione a rolagem, em geral, vertical. Os valores possíveis são: **scroll**, **fixed**, **local** ou **inherit** (um valor herdado).

O valor inicial é **scroll**. O uso desse valor faz com que a imagem aplicada como fundo de um elemento seja posicionada em relação ao elemento. Se o elemento possui um mecanismo de rolagem, obtido com uso da declaração **overflow: auto**, a imagem não rola junto com a rolagem do conteúdo. Se o elemento de inserção for o elemento raiz do documento **html** ou o elemento **body**, a rolagem da tela causa a rolagem da imagem.

O uso do valor **fixed** faz com que a imagem aplicada como fundo seja posicionada em relação à viewport.

A posição de origem para uma imagem definida como fixa no fundo de qualquer elemento da marcação é o canto superior esquerdo da área onde foi inserida, independentemente da posição ocupada pelo elemento na tela.

Essa propriedade faz com que a imagem de fundo permaneça fixa como fundo do elemento, isto é, não rola junto com o conteúdo. Quando aplicada a um elemento que não esteja sobre a área onde a imagem foi posicionada na viewport, ela não será visível.

O valor **local** foi criado pelas CSS3 e não existia nas versões anteriores das CSS. O uso desse valor faz com que a imagem aplicada como fundo de um elemento seja posicionada em relação ao elemento. Se o elemento possui um mecanismo de

rolagem, obtido com uso da declaração **overflow: auto**, a imagem rola junto com a rolagem do conteúdo.

A propósito desse comportamento com imagens de fundo definidas com o valor **fixed** para a propriedade **background-attachment**, Eric Meyer publicou uma interessante demonstração, que pode ser visitada em <http://kwz.me/g5>. Visite a demonstração usando um navegador padrão para ver o efeito. Role a página e observe o efeito transparência no menu e na espiral à esquerda. Examine as imagens usadas e o código-fonte.

5.2.5 background-position

Essa propriedade permite que seja definida uma posição inicial para a imagem de fundo na área onde ela será aplicada. Lembre-se que se a posição inicial não for definida com essa propriedade, ela será tomada como sendo o canto superior esquerdo da área, incluindo a borda, se houver uma. Os valores possíveis são: uma medida CSS de comprimento, uma porcentagem, as palavras-chave **top**, **right**, **bottom**, **left**, **center** ou **inherit** (um valor herdado). Para medidas CSS e porcentagem, são válidos valores negativos.

A posição inicial será medida em coordenadas horizontal e vertical. O ponto de origem das coordenadas é o canto superior esquerdo do box (incluindo a margem), ao qual a imagem será aplicada.

Por padrão, a coordenada horizontal será contada a partir da borda esquerda do box, e a coordenada vertical, a partir da borda superior do box. Valores positivos são marcados em direção ao interior do box, e negativos, ao exterior.

A referência da borda para início da contagem das coordenadas pode ser alterada com uso das palavras-chave **right** e **bottom**, que definem contagem a partir da borda direita e inferior, respectivamente.

A sintaxe para essa propriedade admite um ou dois valores. Quando se especifica dois valores, as CSS3 criaram a possibilidade de se definir, antes de cada valor, a borda (**left**, **right**, **bottom** ou **top**) que servirá de origem da contagem do valor. Pode-se, como alternativa à opção de se especificar quatro valores, omitir um dos valores, nesse caso, ele será considerado zero. Observe as sintaxes possíveis para a declaração da posição da imagem de fundo: `body { background-position: valor; }`

```
body { background-position: valor1 valor2; }
```

```
body { background-position: borda valor1 borda valor2; }
```

```
body { background-position: borda valor1 borda; }
```

```
body { background-position: borda borda valor2; }
```

Se for definido um valor, ele será tomado como a coordenada horizontal, e o outro valor será considerado **center**. Se forem definidos dois valores, o primeiro será a

coordenada horizontal, e o segundo, a vertical. Se forem definidos valores e bordas, as bordas determinarão a coordenada, sendo **left** e **right** para coordenada horizontal e **top** e **bottom** para vertical.

O exemplo a seguir demonstra o mecanismo de posicionamento das imagens de fundo segundo cada uma das sintaxes descritas. São quatro imagens de um retângulo numerado inseridas segundo diferentes valores para a propriedade **background-position**.

As declarações CSS para cada uma das imagens e que interessam para demonstrar o posicionamento são mostradas a seguir: **background-image: url(1.png);**

background-position: 50px;

background-image: url(2.png);

background-position: 120px 30px;

background-image: url(3.png);

background-position: left 200px bottom;

background-image: url(4.png);

background-position: right 100px top 50px;

Na figura 5.3, mostramos o posicionamento das imagens no navegador Opera. Na época em que este capítulo foi escrito, somente o navegador Opera oferecia suporte para declarações com a sintaxe para as imagens *3.png* e *4.png*.

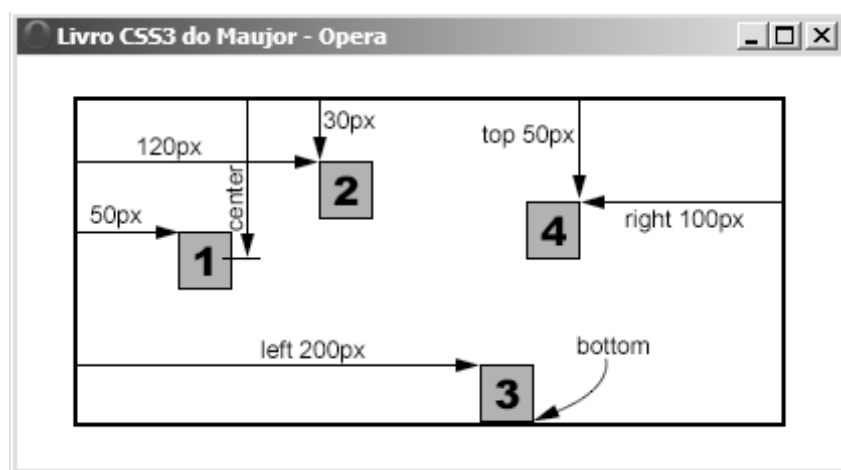


Figura 5.3 – Propriedade background-position.

Declarar valores da posição da imagem usando as palavras-chave **top**, **right**, **bottom**, **left** e **center** é intuitivo e não suscita nenhuma dúvida, sabendo-se que a tradução para as palavras-chave é *em cima*, *à direita*, *embaixo*, *à esquerda* e *no centro*. Convém ressaltar que a ordem em que se declaram os dois valores é indiferente, e podemos declarar um só valor, sendo que, nesse caso, o outro será tomado como **center**.

A sintaxe para declaração com uso de palavras-chave é mostrada a seguir:

background-position: left top;

`background-position: right; /* aqui a coordenada vertical é center */`

O quadro seguinte mostra as combinações possíveis para os valores declarados por palavras-chave:

Valor	Posição
left top	À esquerda, em cima
top left	Idem
right top	À direita, em cima
top right	Idem
left bottom	À esquerda, embaixo
bottom left	Idem
right bottom	À direita, embaixo
bottom right	Idem
center top	No centro, em cima
top center	Idem
top	Idem
center right	No centro, à direita
right center	Idem
right	Idem
center bottom	No centro, embaixo
bottom center	Idem
bottom	Idem
center left	No centro, à esquerda
left center	Idem
left	Idem
center center	No centro
center	Idem

Na figura 5.4, mostramos o posicionamento da imagem de fundo com uso de palavras-chave.

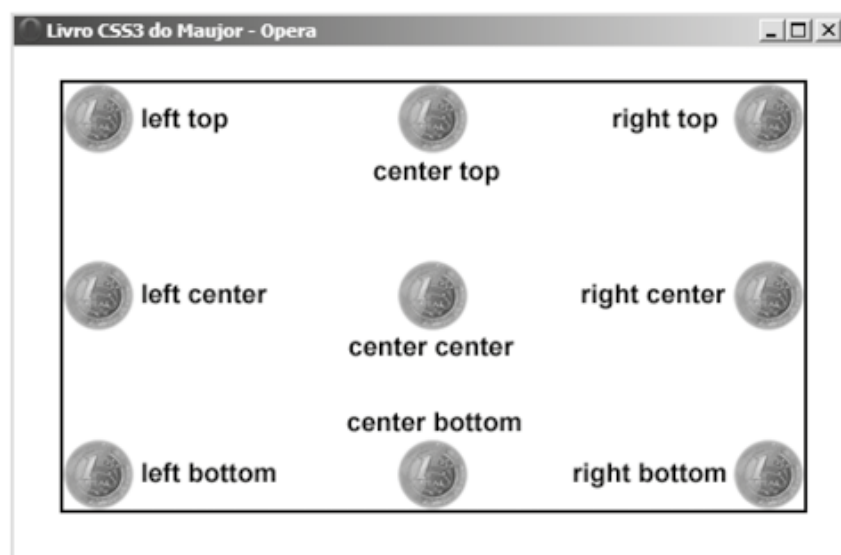


Figura 5.4 – Background-position – Palavra-chave.

Ao declarar valores usando palavras-chave, como mostrado na figura 5.4, ficamos restritos a nove posições para a imagem.

Valores outros que não palavras-chave permitem colocar a imagem em qualquer posição na tela. Porcentagem é outro valor válido, mas, ao contrário das palavras-chave, a ordem de declaração das porcentagens é importante e `25% 80%` é uma posição diferente de `80% 25%`. O primeiro valor declarado refere-se ao afastamento a partir do limite esquerdo do elemento onde a imagem está posicionada (coordenada horizontal – valores positivos resultam em afastamento para a direita, e valores negativos, para a esquerda), e o segundo valor, ao afastamento a partir do limite superior do elemento (coordenada vertical – valores positivos resultam em afastamento para baixo, e valores negativos, para cima).

A sintaxe para declaração com uso de porcentagem é mostrada a seguir:

```
background-position: 80% 20%;  
background-position: 100% 90%;
```

Na figura 5.5, podemos observar alguns exemplos ilustrativos desse tipo de posicionamento.

O exemplo mostrado na figura é ilustrativo. O ponto de referência tomado na imagem para posicionamento depende de como o valor é declarado. Veremos os detalhes, adiante.

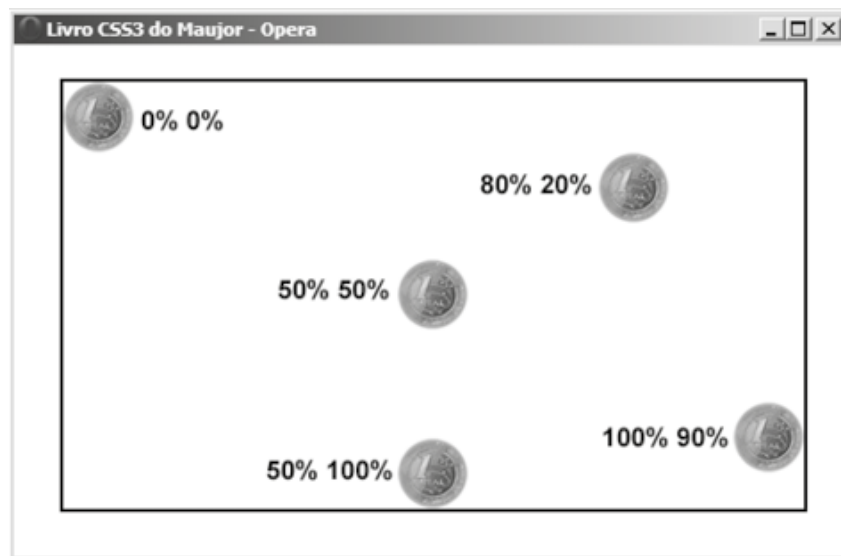


Figura 5.5 – Propriedade *background-position* – Porcentagem.



O valor 0% na figura tem por finalidade única uniformizar a apresentação do exemplo. Lembre-se que não há necessidade de especificar a unidade da medida quando seu valor é 0 (zero). A exceção a essa sintaxe é para declaração da posição do primeiro frame de uma animação, que deve ser 0%, com uso obrigatório do sinal % (ver capítulo 15).

Outra maneira de declarar valores para posicionamento de imagens de fundo é com uso de uma medida CSS de comprimento. Todas as regras CSS que se seguem usam uma medida CSS válida: `div { background-position: 1.2em 4.5em; }`

```
div { background-position: 20px 90px; }
div { background-position: 2cm 16cm; }
div { background-position: 3ex 6ex; }
div { background-position: 3pt 5pt; }
```

Tal como para valores em porcentagem, aqui a ordem de declaração dos valores também é importante. O primeiro valor indica o afastamento para a direita (coordenada horizontal), e o segundo, para baixo (coordenada vertical).



Você pode declarar um só valor em porcentagem ou em medida CSS. Esse valor será tomado para posicionar a imagem em relação ao limite direito do elemento (coordenada horizontal). O outro valor será considerado **center** e posiciona a imagem para baixo (coordenada vertical) e no centro.

Existem quatro grandezas diferentes para se definir valores de posicionamento para uma imagem de fundo (medida CSS, porcentagem, palavra-chave e o valor **inherit**). Para as grandezas dos tipos medida CSS e porcentagem, as especificações para as CSS3 criaram uma sintaxe que permite definir a borda de origem para contagem dos valores.

Caso não seja definida a borda de origem, a contagem é feita segundo um sistema cartesiano de coordenadas com origem no canto superior esquerdo do elemento, no qual a imagem é posicionada. O eixo *x* é horizontal e cresce para a direita (valores positivos), e o eixo *y* é vertical e cresce para baixo (valores positivos). Para valores

negativos, o sentido de crescimento é inverso.

Caso não seja definida a borda de origem, o ponto de referência na imagem, para posicionamento, depende da maneira como os valores foram declarados. Vamos considerar a imagem de um retângulo com um *xis* dentro para ilustrar essa questão. Observe a figura 5.6.

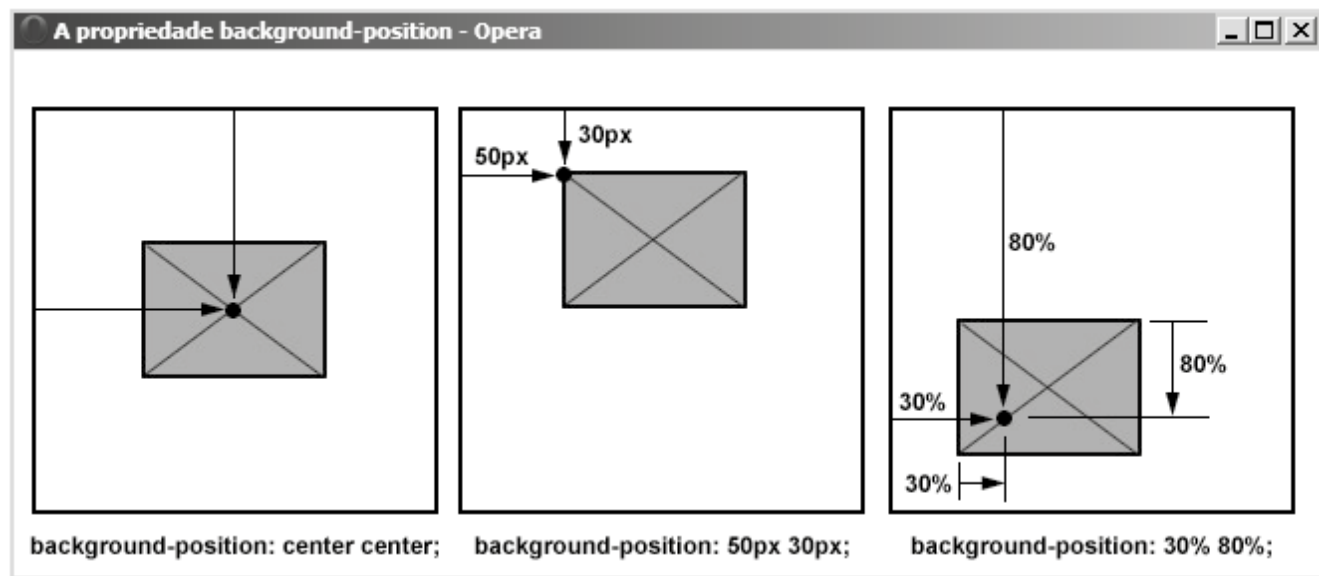


Figura 5.6 – Propriedade `background-position` – Referências 1.

A declaração da posição com uso de palavras-chave posiciona a imagem de fundo em nove locais distintos, conforme mostrado na figura 5.4 e no primeiro esquema da figura 5.6.

Quando se usa uma medida CSS de comprimento para posicionar a imagem de fundo, o ponto de referência na imagem é o canto superior esquerdo da imagem ou do retângulo que envolve a imagem, se a forma desta não for retangular, como mostrado no segundo esquema da figura 5.6, no qual as coordenadas foram `50px` e `30px`.

Quando se usa valores em porcentagem para posicionar a imagem de fundo, o ponto de referência não é fixo. Ele é um ponto situado no interior da imagem ou do retângulo que envolve a imagem, se a forma desta não for retangular, cujas coordenadas são iguais às declaradas no posicionamento, como mostrado no terceiro esquema da figura 5.6, no qual as coordenadas foram `30%` e `80%`.

O ponto de referência na imagem de fundo para posicionamento usando palavra-chave é calculado considerando a seguinte equivalência com porcentagens:

Palavra-chave	Porcentagem equivalente
top	0%
right	100%

bottom	100%
left	0%
center	50%

Usando a tabela de equivalência mostrada, a declaração `center center` produz o mesmo efeito que `50% 50%` e a declaração `right top` equivale a `100% 0%`.



Lembre-se que, com uso de palavras-chave, a ordem de declaração dos valores para **background-position** não importa, mas, com uso de valores em porcentagem, a ordem é importante. Assim, `top right` tem seu equivalente em `100% 0%`, e `bottom left` equivale a `0% 100%`.

É válido misturar valores em porcentagem com valores em medidas CSS numa declaração de posicionamento, conforme as seguintes regras: `h1 { background-position: 80px 60%; }`

```
h1 { background-position: 100% 4em; }
```

Nesse caso, a ordem de declaração de valores segue o que vimos para porcentagens e medidas CSS. O primeiro valor representa a coordenada *x*, e o segundo, a coordenada *y*.



Não há nenhuma recomendação do W3C proibindo a mistura de palavras-chave com porcentagem ou medida CSS. Desenvolvedores evitam essa mistura porque alguns navegadores antigos não a suportam.

Caso seja declarada a borda de referência para início da contagem dos valores definidos para posicionar a imagem (de acordo com a nova sintaxe criada pelas CSS3), tanto a origem do sistema cartesiano como o ponto de referência na imagem serão alterados de acordo com a borda declarada.

O exemplo a seguir esclarece o sistema de posicionamento com declaração de bordas: `background-image: url(1.png);`

```
background-position: 80px 40px;
```

```
background-image: url(2.png);
```

```
background-position: right 80px bottom 40px;
```

```
background-image: url(3.png);
```

```
background-position: right 40% top 20%;
```

Observe, na figura 5.7, que para as imagens de fundo *2.png* e *3.png* usou-se definição de borda para início de contagem dos valores de posicionamento. Para a imagem *2.png*, a coordenada horizontal foi contada a partir da margem direita (**right**) até a margem direita da imagem, e a coordenada vertical, a partir da margem inferior (**bottom**) até a margem inferior da imagem. O mesmo aconteceu com a imagem *3.png*, cujas coordenadas foram definidas em porcentagem. Note no detalhe a referência para posicionamento dentro da imagem.

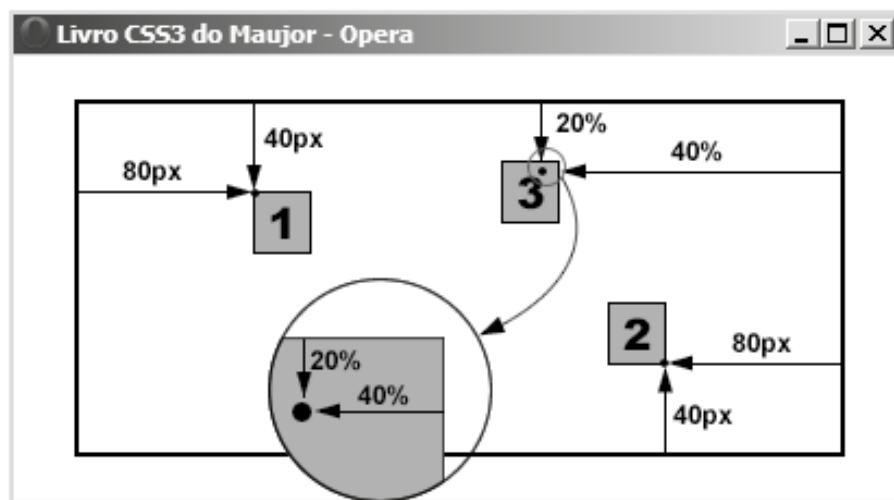


Figura 5.7 – Propriedade *background-position* – Referências 2.

Medidas CSS e porcentagem podem ser declaradas com valor negativo e causam o efeito de deslocar a imagem de fundo para fora dos limites do elemento onde forem posicionadas. Colocar a imagem parcialmente à vista em um dos cantos ou ao longo dos lados de um elemento pode causar efeitos bem interessantes. Parece não haver sentido em se construir uma imagem para apresentá-la parcialmente, cabendo perguntar se não seria melhor construir somente a porção da imagem a ser apresentada. Quando pretendemos usar a porção da imagem uma vez ou mesmo várias vezes, será melhor construí-la parcialmente, se isso resultar em arquivo consideravelmente menor. Mas, se quisermos usar a imagem inteira ou outras porções dela em outros locais do site, aí, sim, se justifica posicionar com coordenadas negativas.

Na figura 5.8, apresentamos um exemplo de renderização de imagens de fundo definidas com coordenadas de posicionamento negativas, conforme as seguintes regras CSS: `div { background-position: -45px -25px; }`

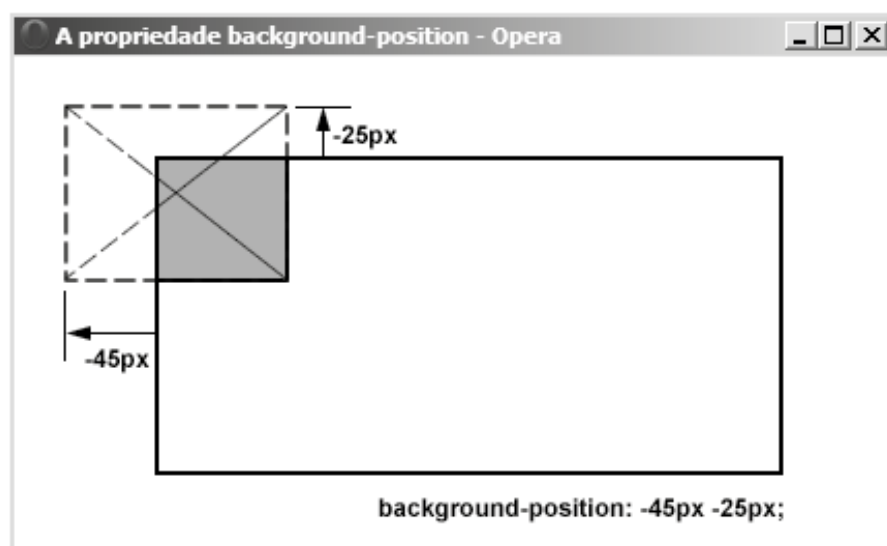


Figura 5.8 – Propriedade *background-position* – valores negativos.

5.2.6 background-clip

Essa propriedade permite que seja definida a área do box na qual será aplicada a imagem e a cor de fundo. Os valores possíveis são: **border-box**, **padding-box** e **content-box**. Essa propriedade foi criada pelas CSS3 e não existia nas versões anteriores das CSS.

Por padrão, a área de aplicação da cor ou da imagem de fundo de um box é aquela limitada pelo contorno externo da borda do box, isto é, a cor ou a imagem serão aplicadas inclusive na borda do box, se houver uma. Se a borda for sólida, ela se sobrepõe ao fundo, mas se for descontínua (tracejada ou pontilhada), o fundo será visível nos pontos de descontinuidade.

Esse comportamento padrão equivale ao valor **border-box** para a propriedade **background-clip**. Para alterar o comportamento padrão, podemos usar os valores **padding-box** e **content-box**. O primeiro exclui a área das bordas para aplicação do fundo, e o segundo exclui as áreas de bordas e de **padding**.

A figura 5.9 mostra três boxes com bordas tracejadas, aos quais foram definidos diferentes valores para a propriedade **background-clip**, que mostram as três áreas de aplicação do fundo.

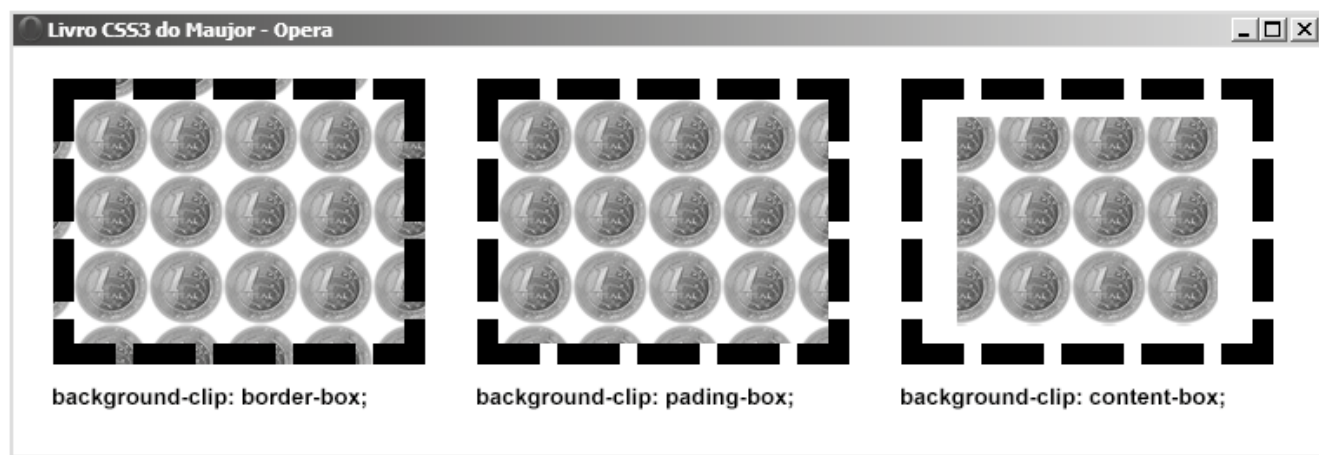


Figura 5.9 – Propriedade *background-clip*.

A figura mostra três boxes aos quais foi definida uma borda tracejada e uma área de **padding**. Para cada um dos boxes foi definido um valor para a propriedade **background-clip**, tudo conforme as regras CSS mostradas a seguir: `div {`

```
width: 150px;
height: 120px;
border: 12px dashed#000;
margin: 10px 15px;
padding: 10px 20px;
float: left;
```



```
background: url(moeda.png);

}
```

```
div.um { background-clip: border-box; }
div.dois { background-clip: padding-box; }
div.tres { background-clip: content-box; }
```

A imagem de fundo usada foi a de uma moeda que se repete na horizontal e na vertical. Note que, nos três casos, a posição inicial da imagem é a posição padrão, isto é, a face interior da borda, pois não definimos um valor para **background-position**. Pode-se observar com clareza o efeito da definição da área de fundo para cada um dos três boxes.

Essa propriedade não se aplica ao elemento raiz do documento. Em documentos HTML, não produz efeito sobre os elementos **html** e **body**.

5.2.7 background-origin

Essa propriedade define a origem do fundo, tomando como referência as áreas de fundo do box. Os valores possíveis são: **border-box**, **padding-box** e **content-box**. Essa propriedade foi criada pelas CSS3 e não existia nas versões anteriores das CSS.

A figura 5.10 esclarece a definição da origem do fundo com uso dessa propriedade.

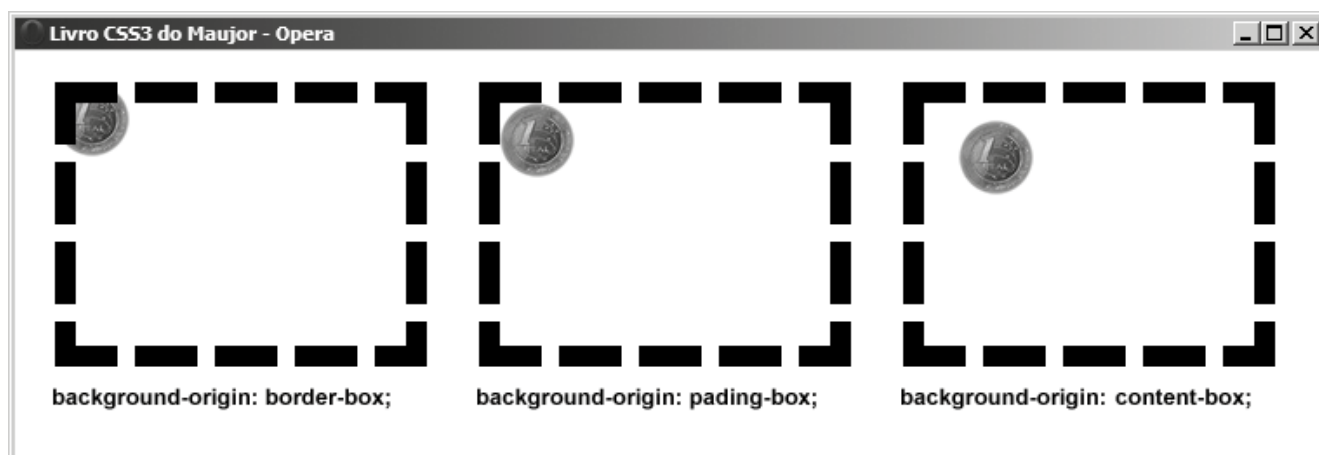


Figura 5.10 – Propriedade background-origin.

A figura mostra três boxes aos quais foi definida uma borda tracejada e uma área de **padding**. Para cada um dos boxes foi definido um valor para a propriedade **background-origin**, conforme as regras CSS mostradas a seguir: `div {`

```
width: 150px;
height: 120px;
border: 12px dashed #000;
margin: 10px 15px;
padding: 10px 20px;
float: left;
```



```
background: url(moeda.png) no-repeat;

}
```

```
div.um { background-origin: border-box; }
div.dois { background-origin: padding-box; }
div.tres { background-origin: content-box; }
```

A imagem de fundo usada foi a de uma moeda que não se repete. Pode-se observar com clareza o efeito da definição da origem da imagem de fundo para cada um dos três boxes.

Essa propriedade não produz qualquer efeito quando se define **background-attachment: fixed;** para a imagem de fundo.

5.2.8 background-size

Essa propriedade define as dimensões da imagem de fundo. Os valores possíveis são: uma medida CSS de comprimento, uma porcentagem, as palavras-chave **auto**, **contain** e **cover**. Essa propriedade foi criada pelas CSS3 e não existia nas versões anteriores das CSS.

Valores expressos em medidas CSS ou em porcentagem podem ser declarados com uso de um ou dois valores, como mostrado a seguir: `div { background-size: 60px 30px; }`

```
div { background-size: 100px; }
```

O primeiro valor redimensiona a largura da imagem, e o segundo, sua altura.

Usando-se apenas um valor na declaração, ele será usado para redimensionar a largura da imagem, e o segundo valor (para redimensionamento da altura) será considerado **auto**. Nesse caso, o valor **auto** tomado para redimensionar a altura é calculado de modo que seja preservada a *aspect ratio* (razão de aspecto) da imagem, isto é, a proporção entre suas dimensões. Por exemplo: se a imagem de fundo tem dimensões iguais a **200px x 50px**, ao se declarar **background-size: 100px;** ela será redimensionada para **100px x 25px**, mantendo a proporção entre largura e altura.

A referência para cálculo das dimensões de redimensionamento quando se usa valores em porcentagem é a altura ou a largura do box no qual a imagem será inserida.

O uso da palavra-chave **contain** redimensiona a imagem, mantendo suas proporções, de modo que sua maior dimensão ocupe toda a extensão interna do box, seja ela horizontal ou vertical.

O uso da palavra-chave **cover** redimensiona a imagem, mantendo suas proporções, de modo que sua menor dimensão ocupe toda a extensão interna do box, seja ela

horizontal ou vertical.

A figura 5.11 esclarece como a imagem de fundo é dimensionada com uso dessa propriedade.

A figura mostra seis boxes aos quais foi definida uma borda sólida e uma imagem de fundo posicionada no centro da borda superior do box. O primeiro box mostra a imagem em seu tamanho real.

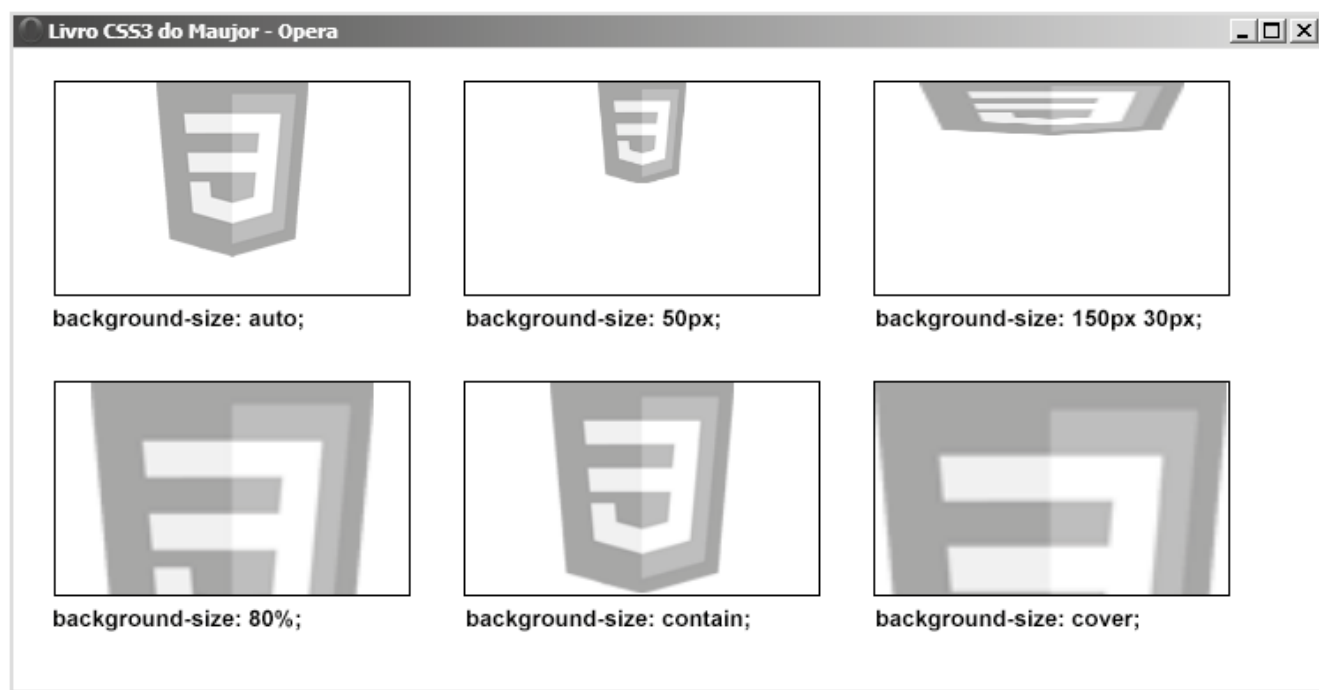


Figura 5.11 – Propriedade background-size.

Para cada um dos boxes foi definido um valor para a propriedade `background-size`, conforme as regras CSS mostradas a seguir: `div {`

```
width: 200px;
height: 120px;
border: 1px solid #000;
margin: 10px 15px;
float: left;
background: url(css3.png) top center no-repeat;

}
```

```
div.um { background-size: auto; }
div.dois { background-size: 50px; }
div.tres { background-size: 150px 30px; }
div.quatro { background-size: 80%; }
div.cinco { background-size: contain; }
div.seis { background-size: cover; }
```

A imagem de fundo usada foi a de um logotipo das CSS3 que não se repete. Pode-se observar com clareza o efeito da definição das dimensões da imagem de fundo

para cada um dos seis boxes.



Use essa técnica com cautela quando a imagem de fundo não for vetorial, pois alterações significativas nas dimensões originais de uma imagem bitmap causam degradação da imagem.

5.2.9 Propriedade background

A propriedade **background** é uma abreviatura para declarar todas as propriedades do fundo estudadas anteriormente. Você declara valores para todas as propriedades CSS do fundo usando a propriedade abreviada **background**, conforme sintaxe que se segue: seletor {

```
background: background-image background-position / background-size  
background-repeat background-attachment background-origin background-clip  
background-color;  
  
}
```

As seguintes diretrizes se aplicam à sintaxe abreviada para definição de fundo: • A ordem de declaração das propriedades não é relevante. Escolha uma ordem e siga com ela em seus projetos. A ordem de declaração das propriedades, mostrada anteriormente, é a constante das especificações.

- Propriedades não declaradas assumem o seu valor inicial.
- No caso de declaração de múltiplas imagens de fundo, a propriedade para definição da cor de fundo deve ser declarada na última layer de imagens.
- Se ambas as propriedades **background-position** e **background-size** estiverem presentes, elas devem ser separadas por uma barra (/).
- Para as propriedades **background-origin** e **background-clip**, podem ser declarados dois valores ou um valor somente. Se for declarado um só valor, ele será válido para as duas propriedades. Se forem declarados dois valores, o primeiro será tomado para a propriedade **background-origin**, e o segundo será tomado para a propriedade **background-clip**.

Suponha que na folha de estilo definimos o fundo para um elemento **div** com a seguinte regra CSS: **div** {

```
background-image: url(imagem.png);  
background-repeat: no-repeat;  
background-attachment: fixed;  
background-position: 50% 30%;  
background-color: red;  
background-origin: padding-box;  
background-clip: content-box;
```

```
background-size: contain;

}
```

Podemos abreviar a regra para:

```
div { background: url(imagem.jpg) 50% 30% / contain no-repeat fixed
padding-box content-box red }
```

Não é obrigatório declarar todos os valores. Valores não declarados são considerados como se fossem o valor inicial da propriedade. A declaração abreviada é válida, ainda que contenha *um só valor*. Nos exemplos seguintes, todas as declarações são válidas: `div { background: #ffe; }`

```
div { background: url(imagem.jpg); }
div { background: no-repeat; }
div { background: none; }
```

Note que a declaração `background: none` é abreviatura para `background-image`. Essa declaração normalmente é usada para anular uma declaração de imagem anterior, como mostrado no exemplo que se segue: `div { background: (imagem.jpg); }`

```
div.diferente { background: none; }
```

Nesse exemplo, todos os elementos `div` terão uma imagem de fundo, exceto aqueles da classe `diferente`.

5.3 Múltiplos backgrounds

A possibilidade de inserir mais de uma imagem de fundo em um mesmo box é uma funcionalidade que foi criada pelas especificações das CSS3. A restrição imposta pelas especificações CSS anteriores, que permitiam a inserção de uma só imagem por box, obrigou, durante anos, os desenvolvedores a criar marcação extra para conseguir, por exemplo, estilizar os cantos de um box com imagens decorativas. A técnica consistia em aninhar elementos em um container e definir uma imagem de fundo para cada elemento aninhado, posicionando quatro imagens, uma em cada canto, dos elementos aninhados.

Com CSS3 podemos definir quantas imagens de fundo quisermos, em um mesmo box, posicionando e dimensionando cada uma delas individualmente.

Cada uma das imagens de fundo definidas para um mesmo box cria uma layer com uma coordenada *z* (como se fosse um *z-index*) definindo sua visibilidade. As layers são posicionadas em uma pilha de layers colocadas umas sobre as outras, fazendo com que as layers com maior coordenada *z* se sobreponham àquelas com menor coordenada *z*.

Segundo as especificações para as CSS3, a ordem de empilhamento, ou seja, a coordenada z, é definida pela ordem em que a imagem é declarada dentro da regra CSS. A primeira imagem declarada recebe a maior coordenada z, a segunda recebe a segunda maior coordenada e assim por diante até a última imagem declarada, que recebe a menor coordenada z.

A sintaxe para declarar múltiplas imagens de fundo consiste em separar os valores das propriedades com uma vírgula. Observe o exemplo a seguir: `div {`

```
background-image: url(triângulo.png), url(circulo.png),  
url(retangulo.png);  
background-repeat: repeat-x, no-repeat, repeaty;  
background-position: right 120px, center, 0 0;  
background-color: #eee;  
}
```

Na figura 5.12, mostramos a renderização das imagens de fundo, esclarecendo seu posicionamento no eixo dos z.

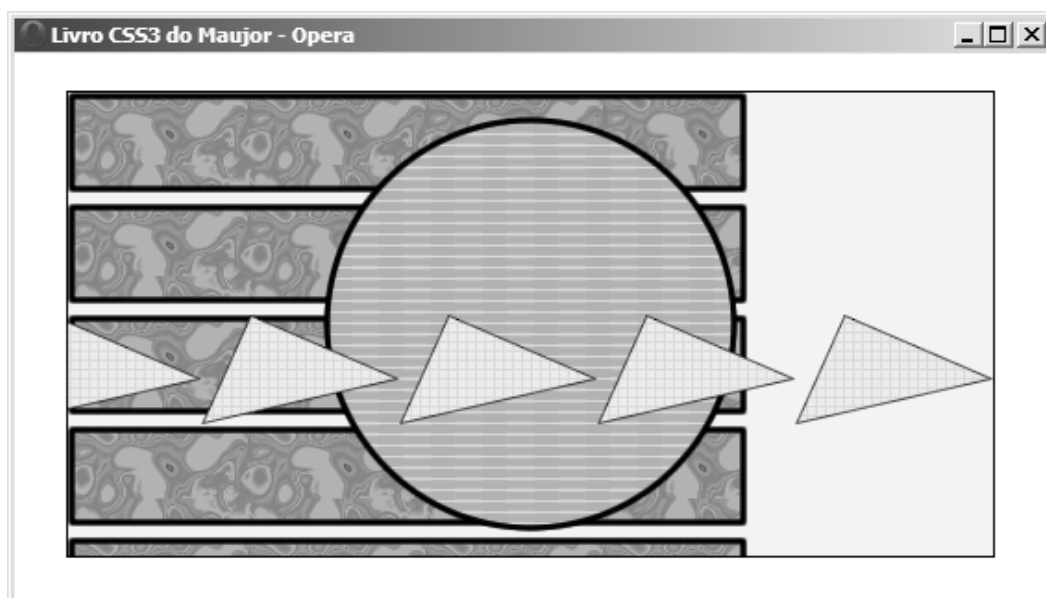


Figura 5.12 – Múltiplas imagens de fundo.

Para clareza do exemplo, definimos dimensões e borda (não constantes da folha de estilo mostrada) para o elemento `div` no qual as imagens de fundo foram inseridas.

Note que o triângulo, por ter sido a primeira imagem a ser definida, posicionou-se mais à frente. Tivéssemos declarado a imagem do triângulo depois da do retângulo, ele seria colocado por trás do retângulo.

A declaração para repetição das figuras é uma lista de três valores, **repeat-x**, **no-repeat** e **repeaty**, e se aplica nessa ordem às imagens declaradas. Genericamente, a ordem de declaração dos valores das demais propriedades determina sua aplicação de acordo com a ordem de declaração das imagens.

Se uma declaração contém uma lista de valores em menor quantidade do que a quantidade de imagens declaradas, a lista se repete a partir do início até completar o número de imagens. O exemplo a seguir esclarece esse comportamento: `div {`

```
background-image: url(um.png), url(dois.png), url(tres.png);
background-repeat: repeat-x, no-repeat;
background-position: center;

}
```

A repetição das imagens será assim:

- `um.png – repeat-x` • `dois.png – no-repeat` • `tres.png – repeat-x` Uma vez que foram declarados dois valores de repetição para três imagens, o terceiro valor será igual ao primeiro.

O posicionamento será **center** para as três imagens, pois como só houve um valor declarado, ele se repete duas vezes para completar os três valores.

A declaração de múltiplas imagens de fundo pode ser feita com uso da declaração abreviada para as propriedades de fundo, conforme mostrado a seguir: `div {`

```
background:
url(triangulo.png) repeat-x right 120px,
url(circulo.png) no-repeat center,
url(retangulo.png) repeaty 0 0 #eee;

}
```

Essa folha de estilo é idêntica àquela apresentada anteriormente, cuja renderização foi mostrada na figura 5.12.

Note que a cor de fundo foi declarada somente na última imagem. Essa é a sintaxe correta, pois declarar a cor de fundo junto com a declaração de uma imagem que não seja a última é inválido e faz com que o navegador ignore a regra CSS.

Navegadores que não suportam essa funcionalidade ignorarão a regra CSS e nenhuma imagem de fundo será mostrada. Contudo, em determinadas situações, poderá ser interessante apresentar uma imagem alternativa nesses navegadores. Isso é possível, declarando-se, antes das declarações para múltiplas imagens, uma imagem alternativa, conforme mostrado a seguir: `div {`

```
background: url(imagem-alternativa.png) no-repeat left; /* navegadores
sem suporte para múltiplas imagens */
background:
url(triangulo.png) repeat-x right 120px,
url(circulo.png) no-repeat center,
```

```
url(retangulo.png) repeaty 0 0 #eee;  
}
```

CAPÍTULO 6

Bordas e sombras

As CSS3 criaram novos mecanismos para estilização de bordas e aplicação de sombras em boxes da marcação. Com CSS3 é possível definir bordas arredondadas, bordas criadas com imagens e diferentes tipos de sombreadimento. A aplicação de bordas arredondadas e sombras com uso da versão anterior das CSS somente é possível com uso de imagens, inserção de elementos extras na marcação e consequente aumento do tempo de carregamento da página. As novas funcionalidades criadas vieram suprir velhos anseios e necessidades dos desenvolvedores. Neste capítulo, estudaremos as propriedades CSS para bordas e para aplicação de sombras, constantes das especificações para as CSS3.

6.1 Introdução

Em linhas gerais, podemos definir bordas como elementos decorativos a serem anexados aos quatro lados dos boxes. As CSS possibilitam especificar espessura, cor e estilo para as bordas, bem como definir cada uma das quatro bordas individualmente. De acordo com Box Model, a espessura das bordas não está incluída nas dimensões definidas para o box ao qual ela é aplicada.

Nenhuma das propriedades para definição de bordas é herdada, ou seja, as bordas não são herdadas pelos seus elementos descendentes.

6.2 Propriedade border

As propriedades para estilizar bordas são listadas a seguir:

- `border-width`
- `border-style`
- `border-color`
- `border-radius` (criada nas CSS3) • `border-image` (criada nas CSS3)

Estudaremos, a seguir, cada uma dessas propriedades.

6.2.1 border-width

Essa propriedade destina-se a definir a espessura da borda. Os valores possíveis são: uma medida CSS de comprimento, as palavras-chave `thin` (fina), `medium` (média), `thick` (grossa) ou `inherit` (um valor herdado). A espessura para declaração com uso de palavras-chave não é definida pela especificação, ficando por conta do agente de usuário. As especificações para as CSS3 sugerem que, para agentes de usuário cujo

tamanho de fonte **medium** seja igual a **17px** ou menor, se adote as espessuras de **1px**, **3px**, e **5px** para bordas fina, média e grossa, respectivamente. Embora não obrigatório, a maioria dos navegadores segue essas espessuras.

É possível definir a espessura das bordas individualmente, uma para cada lado do box, conforme sintaxe mostrada a seguir: `border-top-width: 2px; /* espessura da borda-superior */`

```
border-right-width: 3px; /* espessura da borda direita */
border-bottom-width: 4px; /* espessura da borda inferior */
border-left-width: 1px; /* espessura da borda esquerda */
```

Alternativamente, pode-se usar a sintaxe abreviada, lembrando que a declaração de valores para qualquer propriedade CSS que admita sintaxe abreviada pode ser feita de quatro maneiras, como mostrado a seguir, para definição da espessura das bordas: `border-width: 2px 3px 4px 1px; /* top right bottom left */`

```
border-width: 3px 2px 1px; /* top (right e left) bottom */
border-width: 5px 7px; /* (top e bottom) (right e left) */
border-width: 6px; /* (top right bottom left) */
```

A declaração de espessura de bordas com uso de palavras-chave se faz como mostrado a seguir: `border-width: thin;`

```
border-top-width: medium;
border-left-width: thick;
```

6.2.2 border-style

Essa propriedade destina-se a definir o estilo da borda. Os valores possíveis são mostrados a seguir:

Estilo	Descrição sumária
none	Define espessura 0 para a borda, ou seja, ausência de borda.
hidden	O mesmo efeito de none , mas com precedência na resolução de bordas conflitantes.
dotted	Borda pontilhada.
dashed	Borda tracejada.
solid	Borda contínua ou sólida.
double	Borda constituída de duas linhas contínuas. A soma das espessuras das linhas com a do espaço que as separa é igual ao valor de border-width .
groove	Borda com aparência entalhada.
ridge	Borda com aparência de resalto.

inset	Borda em baixo-relevo.
outset	Borda em alto-relevo.

É possível definir o estilo das bordas individualmente, uma para cada lado do box, conforme sintaxe mostrada a seguir: `border-top-style: solid;`

```
border-right-style: ridge;
border-bottom-style: double;
border-left-style: inset;
```

Alternativamente, pode-se usar a sintaxe abreviada, lembrando que a declaração de valores para qualquer propriedade CSS que admita sintaxe abreviada pode ser feita de quatro maneiras, como mostrado a seguir: `border-style: solid ridge double inset; /* top right bottom left */`

```
border-style: ridge double solid; /* top (right e left) bottom */
border-style: double inset; /* (top e bottom) (right e left) */
border-style: ridge; /* (top right bottom left) */
```

Você pode aplicar nove estilos para bordas ou declarar o valor **none** para definir ausência de bordas. Pode parecer estranha e inútil a declaração **none**, mas na prática é muito usada para retirar bordas colocadas por padrão ou declaradas anteriormente em elementos específicos da marcação ou ainda para retirar a borda padrão colocada em imagens que são links.

Quando usamos uma imagem para criar um link, por padrão, o navegador Internet Explorer versão 8 e anteriores e as versões mais antigas dos demais navegadores colocam uma borda na cor azul em volta da imagem. Para retirar a borda, declare em sua folha de estilo uma regra CSS, como mostrada a seguir: ► **CSS**

```
a img { border: none; }
```

► HTML

```
<a href="http://www.maujor.com"><a>
```

Na figura 6.1, mostramos a renderização, no Internet Explorer 8, de uma imagem transformada em link. Na imagem à esquerda, vemos a inserção da borda padrão na imagem, e à direita, o efeito de retirada da borda com uso da regra CSS, mostrada anteriormente.



Figura 6.1 – Bordas em imagem link.

Na figura 6.2, mostramos a renderização dos diferentes estilos de bordas nos navegadores Firefox, Opera e IE.

As principais diferenças entre os navegadores na apresentação das bordas são notadas nos estilos **dotted** e **dashed**. O navegador simula os efeitos para os quatro últimos estilos de borda mostrados, usando tons de cores. Se a cor definida for preta, os efeitos não são notados no IE. Sendo colorida, as tonalidades adotadas para o efeito diferem nos dois navegadores.

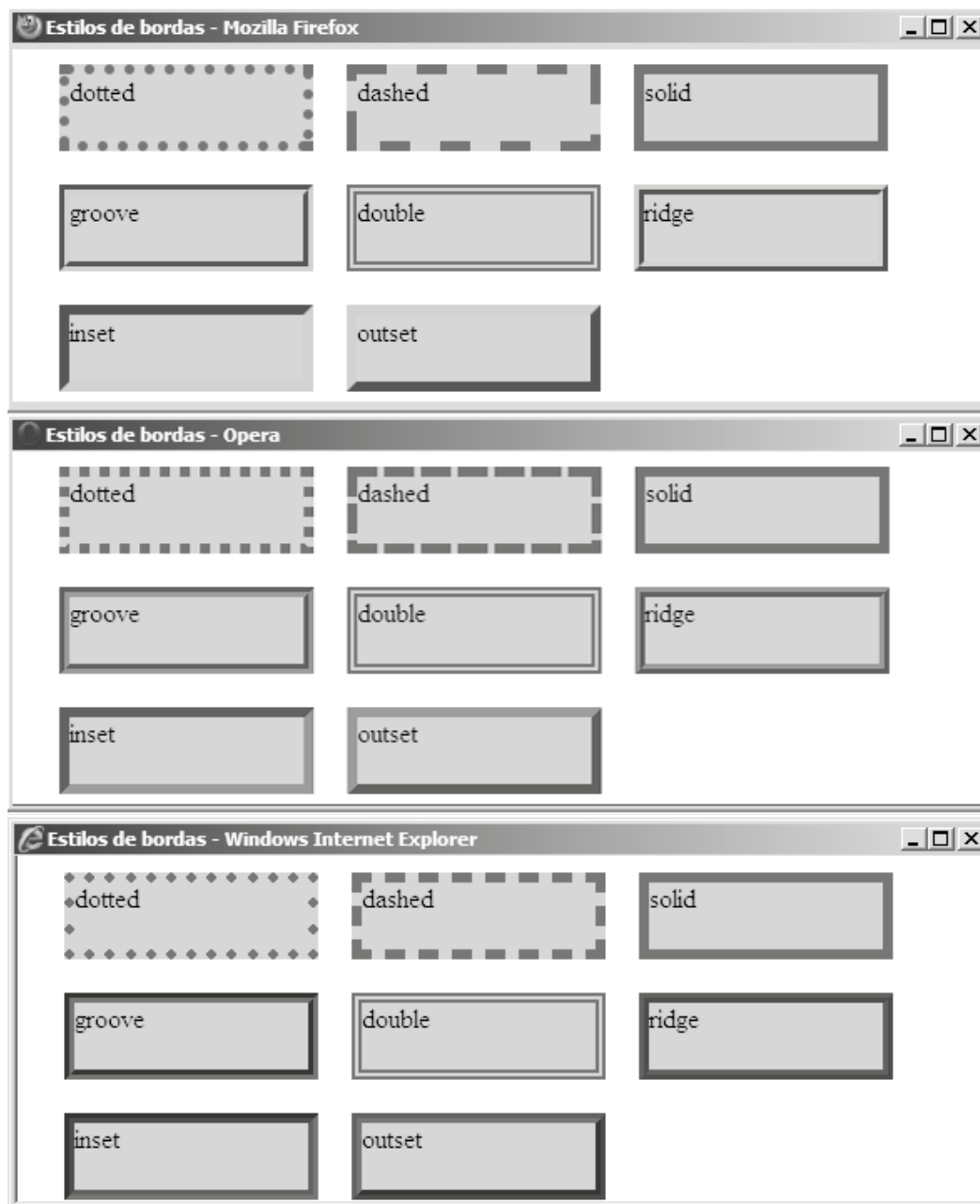


Figura 6.2 – Estilos de bordas.

Além disso, convém notar ainda que para os estilos **dotted** e **dashed** a renderização é diferente. Os traços da borda **dashed** têm comprimentos diferentes e o formato da borda **dotted** é também diferente.

6.2.3 border-color

Essa propriedade destina-se a definir a cor da borda. Os valores possíveis são: valores CSS para cores, conforme mostrado no capítulo 7 deste livro, o valor **transparent** e o valor **inherit** (um valor herdado).

É possível definir a cor das bordas individualmente, uma para cada lado do box, conforme sintaxe mostrada a seguir: `border-top-color: red; /* cor da borda-superior`

`*/`

```
border-right-color: #00f; /* cor da borda direita */
border-bottom-color: hsla(120, 100%, 75%, 0.6); /* cor da borda inferior
*/
border-left-color: rgb(0, 255, 0); /* cor da borda esquerda */
```

Alternativamente, pode-se usar a sintaxe abreviada, lembrando que a declaração de valores para qualquer propriedade CSS que admita sintaxe abreviada pode ser feita de quatro maneiras, como mostrado a seguir: `border-color: red black pink blue; /* top right bottom left */`

```
border-color: blue pink black; /* top (right e left) bottom */
border-color: black blue; /* (top e bottom) (right e left) */
border-color: pink; /* (top right bottom left) */
```

Por padrão, a cor da borda de um elemento é a mesma cor declarada para o conteúdo textual do elemento. Observe o exemplo a seguir: `div {`

```
border-width: 5px;
border-style: solid;
color: blue;
}
```

Nesse exemplo será aplicada uma borda sólida com 5px de espessura e na cor azul (blue). Como não foi declarada a cor da borda com uso da propriedade `border-color`, ela será da mesma cor que tenha sido declarada para o elemento com uso da propriedade `color`. Caso não tenha sido declarada cor para o elemento no qual a borda será aplicada, a cor adotada será a cor inicial preta.

6.2.4 border

A propriedade `border` é uma abreviatura para declarar as três propriedades para borda estudadas anteriormente. Você declara valores para as três propriedades CSS para bordas usando a propriedade abreviada `border`, conforme sintaxe que se segue: `border-top: 1px solid red;`

```
border-right: 2px ridge yellow;
border-bottom: 4px double black;
border-left: 6px inset cyan;
```

ou usando a sintaxe abreviada para bordas que sejam iguais nos quatros lados: `border: 2px dotted white;`

Ao usar a declaração abreviada `border`, não é obrigatório declarar os três valores, portanto as regras CSS mostradas a seguir são válidas: `border: 5px;`

```
border: dotted;
border: red;
border: 2px double;
border: solid red;
border: 4px blue;
```

Os valores não declarados são interpretados pelo navegador como sendo o valor inicial da propriedade. Os valores iniciais das três propriedades são: `border-width: medium;`

```
border-style: none;
```

```
border-color: /* o mesmo valor da propriedade color do elemento onde se aplica a borda. */
```

Como o valor inicial para o estilo da borda é `none` (nenhum), concluímos que declarações abreviadas que omitem o estilo da borda, como as mostradas a seguir, isoladamente não produzem efeito algum.

```
border: 5px;
```

```
border: red;
```

```
border: 4px blue;
```

Porém, se combinarmos aquelas declarações com outras, obteremos um método de estilização interessante. As regras CSS a seguir esclarecem o método proposto:

```
border: 4px blue;
```

ou ainda:

```
border: solid blue;
```

```
border-width: 5px 8px 10px 2px;
```

As combinações de espessuras de bordas podem resultar em criações gráficas bastante interessantes. No site de uma designer inglesa há um desafio aberto aos visitantes que consiste em criações gráficas com uso da propriedade CSS `border`. Há uma galeria com alguns trabalhos que pode ser visitada em: <http://kwz.me/g6>. Recomendo também visitar o endereço: <http://kwz.me/g7>. Lá você encontra uma página demonstrativa de algumas formas construídas com a propriedade `border`.

6.2.5 border-radius

Essa propriedade foi criada pelas especificações para as CSS3 e destina-se a definir um raio de curvatura a ser aplicado nos quatro cantos de um box. Convém ressaltar que o raio de curvatura será aplicado independentemente de o box ter ou não uma borda. Na figura 6.3, são mostrados dois boxes aos quais se aplicou um raio de curvatura de `30px` e uma cor de fundo cinza. Em ambos os boxes o raio de curvatura foi aplicado aos seus quatro cantos.

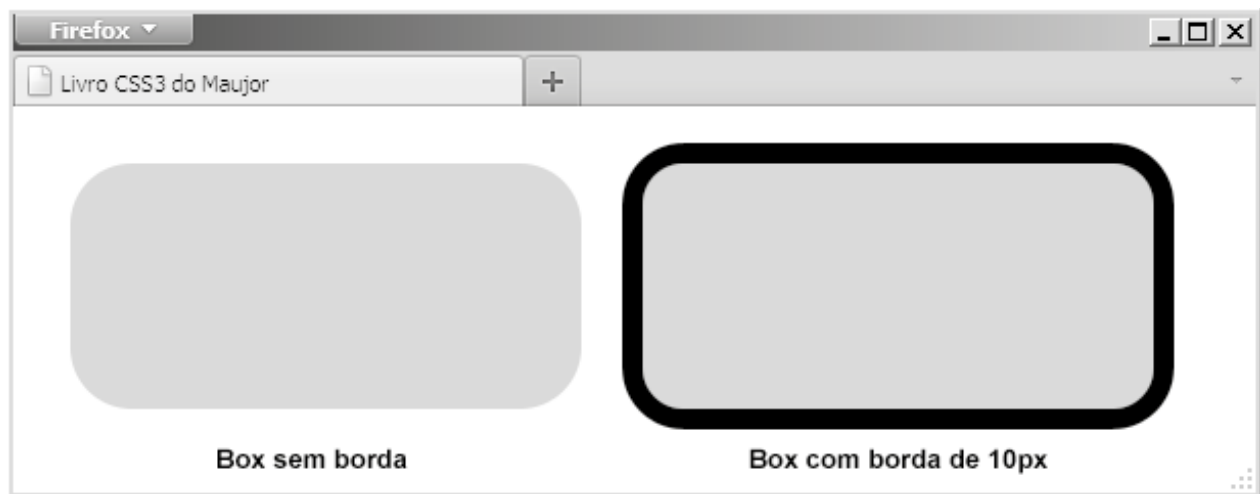


Figura 6.3 – Arredondamento sem e com bordas declaradas.

Os valores possíveis para definição dos raios de curvatura são: uma medida CSS de comprimento e porcentagem. Os valores declarados em porcentagem tomam como referência as dimensões do border-box.

Podem-se definir quatro, três, dois valores ou um valor para essa propriedade, como mostrado a seguir: `border-radius: 12pt 16pt 14px 18pt;`

`border-radius: 10% 8% 12%;`

`border-radius: 4em 2em;`

`border-radius: 20px;`

Quando se definem quatro valores, a ordem de declaração é:

Propriedade	Descrição
<code>border-top-left</code>	Borda superior esquerda
<code>border-top-right</code>	Borda superior direita
<code>border-bottom-right</code>	Borda inferior direita
<code>border-bottom-left</code>	Borda inferior esquerda

Quando se definem três valores, o quarto valor (não declarado) é tomado como sendo igual ao segundo valor declarado, ou seja, `border-bottom-left` é igual a `border-top-right`.

Quando se definem dois valores, o terceiro valor (não declarado) é tomado como sendo igual ao primeiro valor declarado, ou seja, `border-bottom-right` é igual a `border-top-left`, e o quarto valor (não declarado) é tomado como sendo igual ao segundo valor declarado, ou seja, `border-bottom-left` é igual a `border-top-right`.

Quando se define um valor, ele é tomado para os quatro cantos do box.

Considere as seguintes declarações:

```
border-radius: 20px;  
border-radius: 20px 50px;  
border-radius: 20px 50px 70px;  
border-radius: 20px 50px 0 70px;
```

A figura 6.4 esclarece a aplicação dos raios de curvatura com as quatro maneiras de se declarar seus valores.

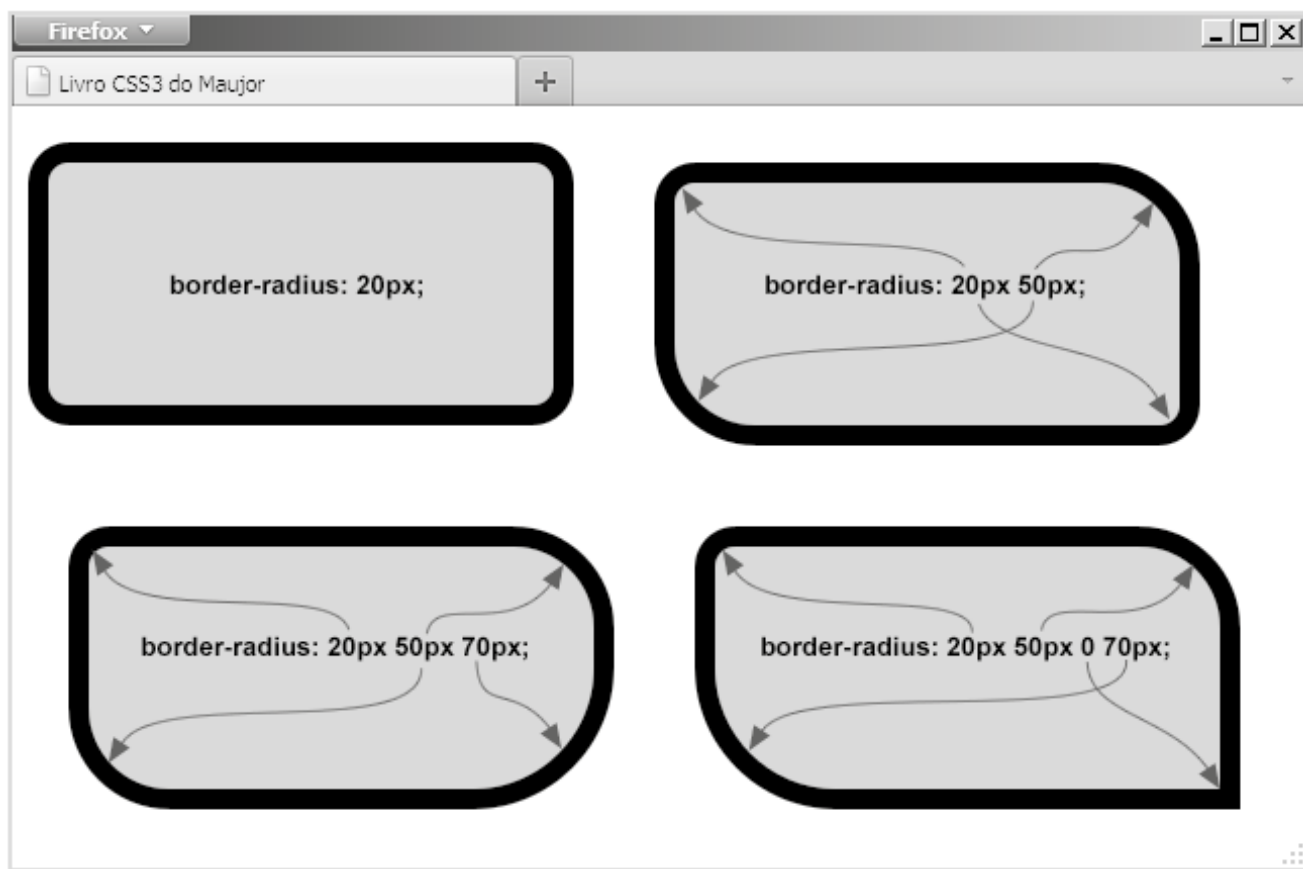


Figura 6.4 – Declaração de bordas arredondadas.

Os exemplos mostrados arredondam os cantos do box, construindo um quadrante circular com o raio definido como valor da propriedade. Contudo, a especificação prevê que a forma do arredondamento do canto do box é um quadrante elíptico obtido com definição de um raio de curvatura horizontal e um raio de curvatura vertical. Quando ambos os raios são iguais, obtém-se um quadrante circular, como mostrado nos exemplos anteriores.

A sintaxe para declarar o arredondamento dos cantos com uso de um quadrante elíptico consiste em se declarar os raios de arredondamento separados por uma barra (/), conforme mostrado a seguir. O uso de espaço antes e depois da barra é opcional.

```
border-radius: 90px/50px;  
border-radius: 50px 90px / 70px 50px;
```

Observe, na figura 6.5, o resultado da aplicação dessas duas regras para arredondamento.

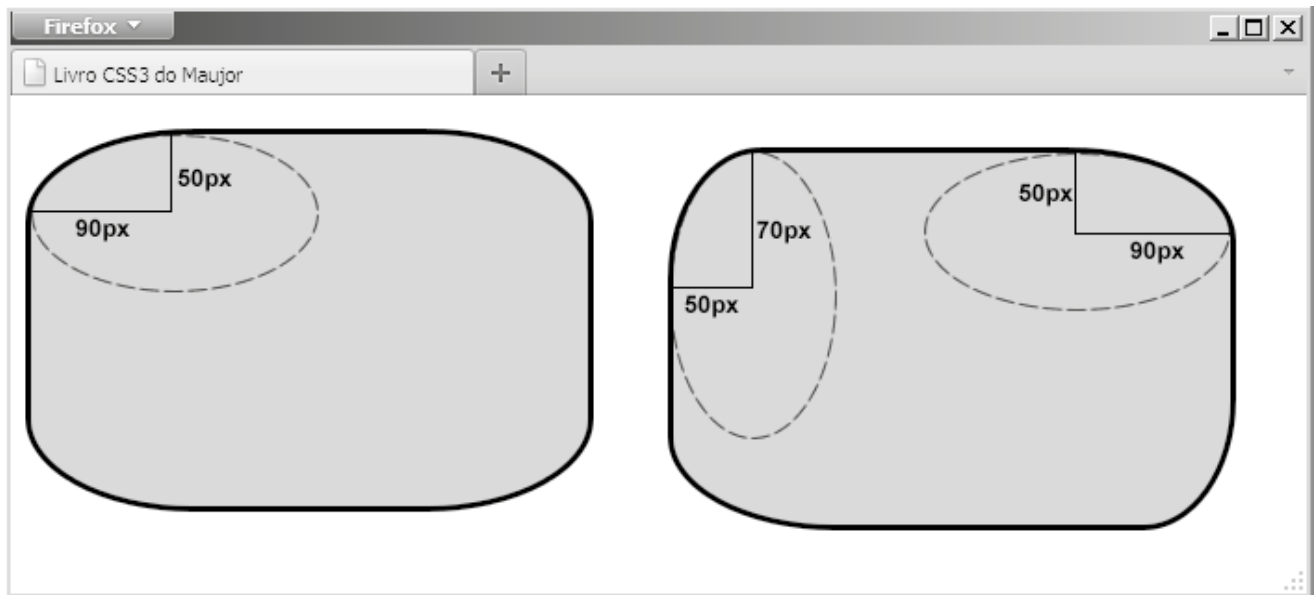


Figura 6.5 – Arredondamento elíptico.

A primeira regra CSS aplica, nos quatro cantos do box, um raio de curvatura horizontal igual a **90px** e vertical igual a **50px**.

A segunda regra CSS foi declarada com uso de dois valores para os raios de curvatura, o primeiro valor aplica um raio de curvatura horizontal igual a **50px**, e o segundo, **90px**; a seguir – depois da barra (/) –, o primeiro valor aplica um raio de curvatura vertical igual a **70px**, e o segundo, **50px**.

Note que a sintaxe geral declara valores para os raios de curvatura horizontais e, depois, separados por uma barra, para os raios de curvatura verticais. Observe as sintaxes a seguir: `border-radius: 50px/90px 70px/50px; /* ERRADO */`

`border-radius: 50px 90px 70px 50px; * CERTO */`

Podemos, então, estabelecer um padrão geral de sintaxe, como mostrado a seguir: `border-radius: { [valor do raio horizontal]{1,4} / [valor do raio vertical]{1,4} }`

Assim, as sintaxes a seguir são válidas:

`border-radius: 50px 30px 40px 90px / 40px 60px 20px 10px;`

`border-radius: 50px / 10px 30px 60px;`

`border-radius: 30px 80px 20px / 10px 50px;`

`border-radius: 20px 10px / 50px;`

Os valores não declarados em cada grupo de raios (horizontais e verticais) são tomados conforme estudado anteriormente e mostrado na figura 6.4.

Exemplo:

`border-radius: 50px 80px 40px / 30px 60px;`

Observe, na figura 6.6, o resultado da aplicação da regra CSS mostrada.

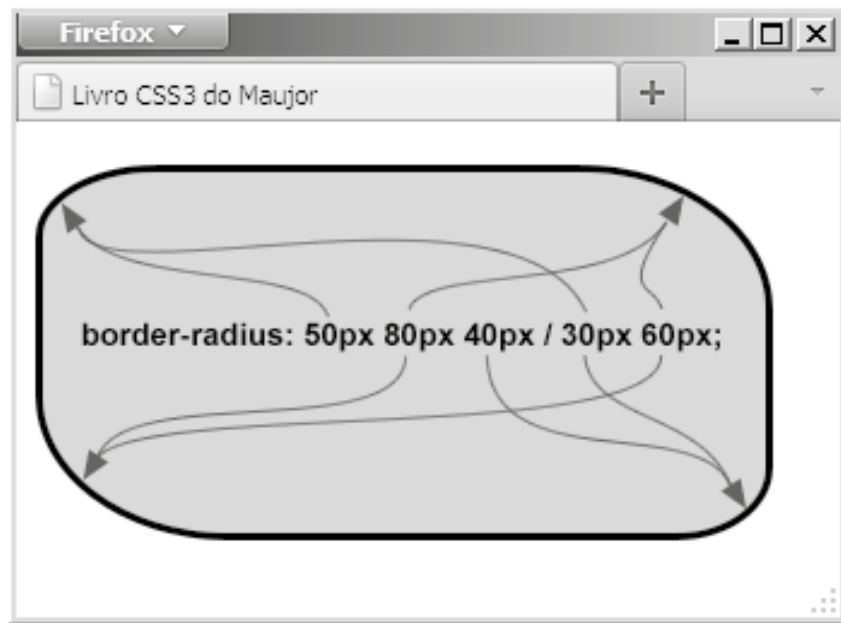


Figura 6.6 – Declarações para arredondamento elíptico.

Quando se declara valores para os raios de curvatura expressos em porcentagem, o comprimento tomado como base para cálculo da porcentagem é a largura do border-box para raios de curvatura horizontais e a altura do border-box para raios de curvatura verticais. Observe a declaração CSS a seguir: `border-radius: 50%;`

Essa declaração transforma um box retangular em uma elipse e um box quadrado em um círculo.

Convém destacar a obtenção de três efeitos interessantes em boxes com bordas arredondadas, quando se declara bordas adjacentes com espessuras diferentes. Observe as regras CSS a seguir: `div {`

```
width: 200px;
height: 120px;
background-color: #eee;
border: solid black;
```

```
}
```

```
div.um, div.dois { border-width: 20px 30px 40px 1px; }
div.um { border-radius: 40px; }
div.dois { border-radius: 40px 60px 60px 20px; }
div.tres { border-width: 40px border-radius: 40px; }
```

Observe, na figura 6.7, os efeitos da aplicação das regras CSS mostradas. Para o terceiro box, note que, quando se declara o raio de curvatura igual ou menor do que a espessura da borda, o canto interno da borda é em ângulo reto.

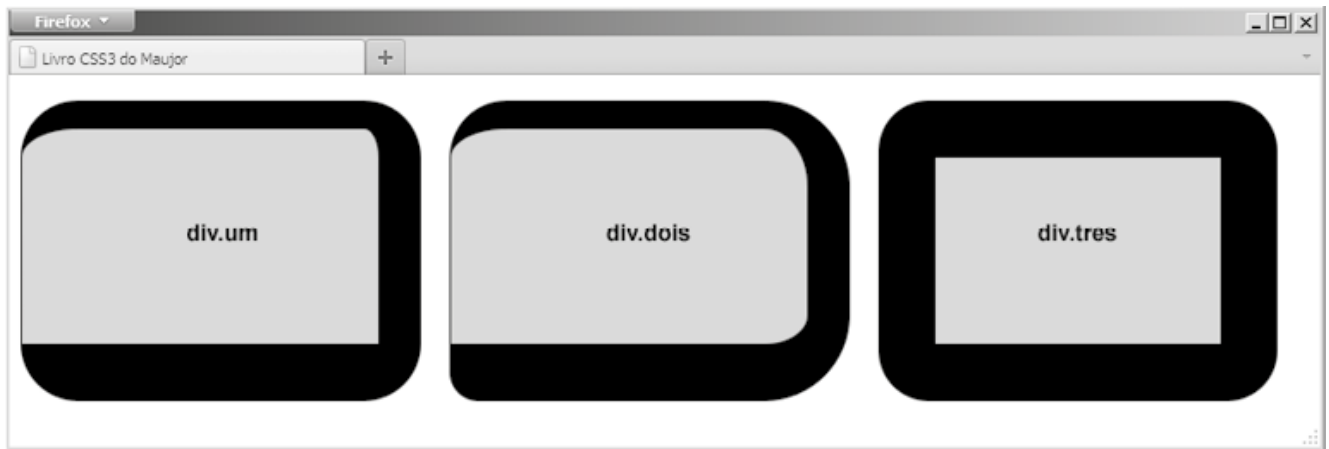


Figura 6.7 – Bordas com espessuras diferentes.

Vamos considerar o caso em que a soma dos valores dos raios de curvatura de cantos diferentes é menor, igual e maior do que a largura ou a altura do box. Considere a regra CSS mostrada a seguir. Observe que são três boxes com dimensões, bordas e cor de fundo iguais. Para cada um deles definiu-se as bordas superior esquerda e direita.

Para o box `div.um`, a soma dos valores dos raios de curvatura horizontal é menor do que a largura do box ($90\text{px} + 60\text{px} = 150\text{px} < 200\text{px}$).

Para o box `div.dois`, essa soma é igual, e para `div.tres`, a soma é maior.

```
div {  
width: 200px;  
height: 200px;  
background-color: #eee;  
border: 2px solid black;
```

```
}
```

```
div.um {  
border-left-radius: 90px;  
border-right-radius: 60px;
```

```
}
```

```
div.dois {  
border-left-radius: 120px;  
border-right-radius: 80px;
```

```
}
```

```
div.tres {  
border-left-radius: 180px;
```

```
border-right-radius: 60px;

}
```

Observe a aplicação das regras CSS nos três boxes.

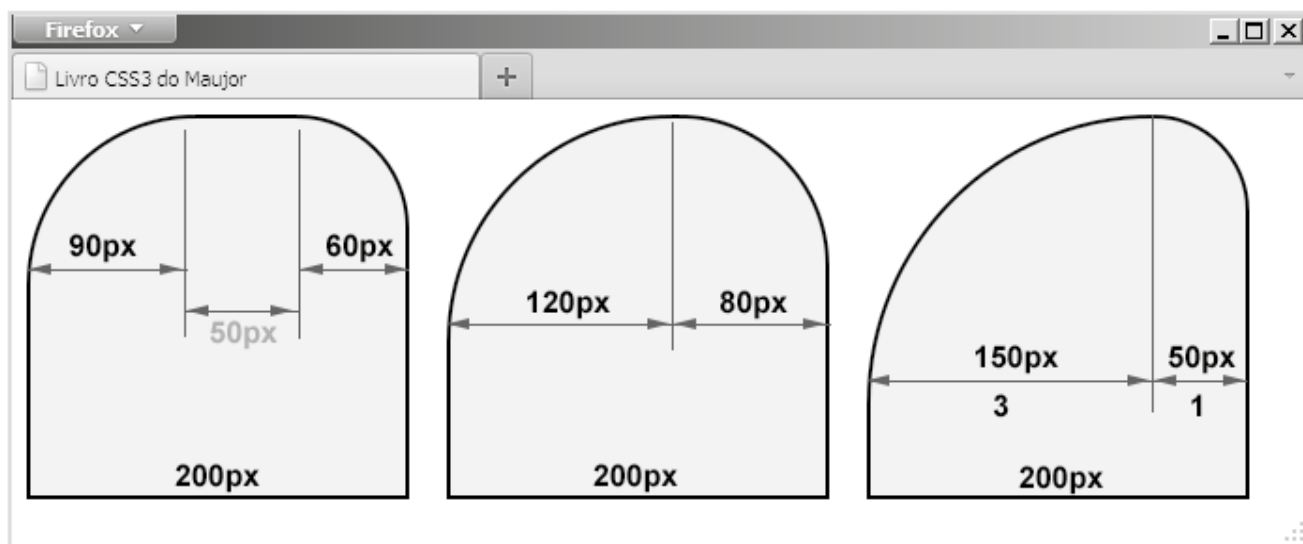


Figura 6.8 – Soma dos raios de curvatura.

Note que, para o primeiro box, a concordância horizontal das curvas se faz com um segmento de reta igual a $200\text{px} - (90\text{px} + 60\text{px}) = 50\text{px}$. Para o segundo box, a concordância horizontal das curvas se faz segundo um ponto.

Quando a soma dos valores dos raios de curvatura é maior do que o comprimento disponível no box, os valores usados são recalculados pelo agente de usuário de modo a se manter a proporção entre os valores declarados.

No nosso exemplo, para o terceiro bloco, a soma dos valores declarados $180\text{px} + 60\text{px} = 250\text{px}$ é maior do que a largura do box, 200px . A proporção entre os valores declarados é $180/60 = 3:1$.

Dividindo a largura disponível de 200px na proporção 3:1, obtemos $200\text{px} \times 3/4 = 150\text{px}$ e $200\text{px} \times 1/4 = 50\text{px}$, que são os valores a usar.

A propriedade `border-radius` aplica-se aos elementos `table`, `td` e `th`. Considere as seguintes regras de estilo: `table {`

```
border-radius: 5px;
border: 2px solid black;

}
```

```
td, th {
height: 40px;
border: 1px solid #666;
```

```
border-radius: 20px;
```

```
}
```

```
th { border-radius:10px; }
```

Observe, na figura 6.9, o resultado da aplicação dessas regras de estilo em uma tabela com três linhas, sendo a primeira uma linha de cabeçalhos e quatro colunas.

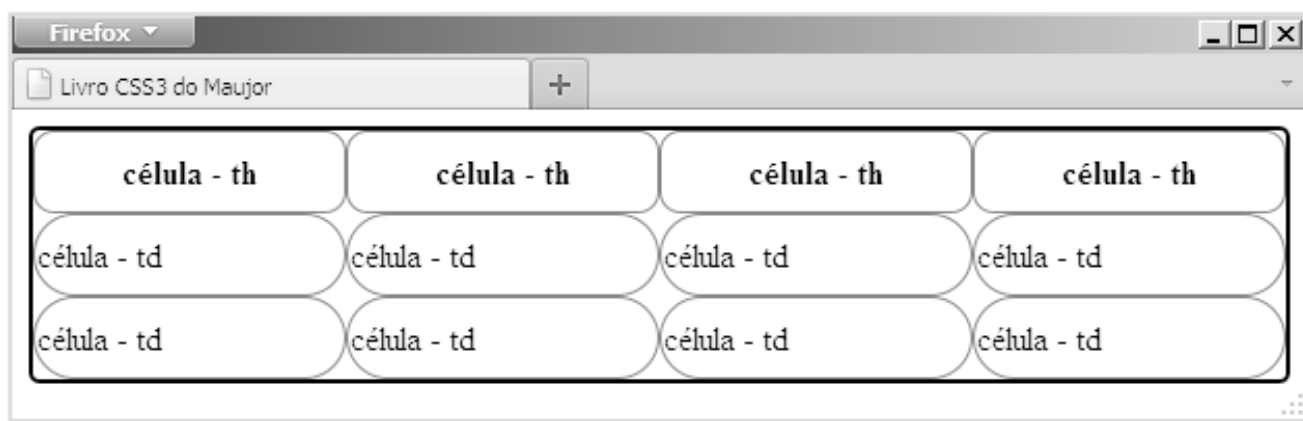


Figura 6.9 – Bordas arredondadas em tabelas.

Bordas arredondadas em tabelas não são aplicadas quando for definida uma declaração de estilo para a tabela, fazendo a propriedade `border-collapse` com o valor `collapse`.

Um box com conteúdo inserido e bordas arredondadas pode requerer declaração de `padding` para que o conteúdo não ultrapasse os limites da borda.

Na figura 6.10, mostramos o primeiro box sem declaração de `padding` horizontal e o segundo com a declaração. Observe que, para abrigar a mesma quantidade de conteúdo, o segundo box necessita de mais altura, uma vez que a largura permaneceu fixa.

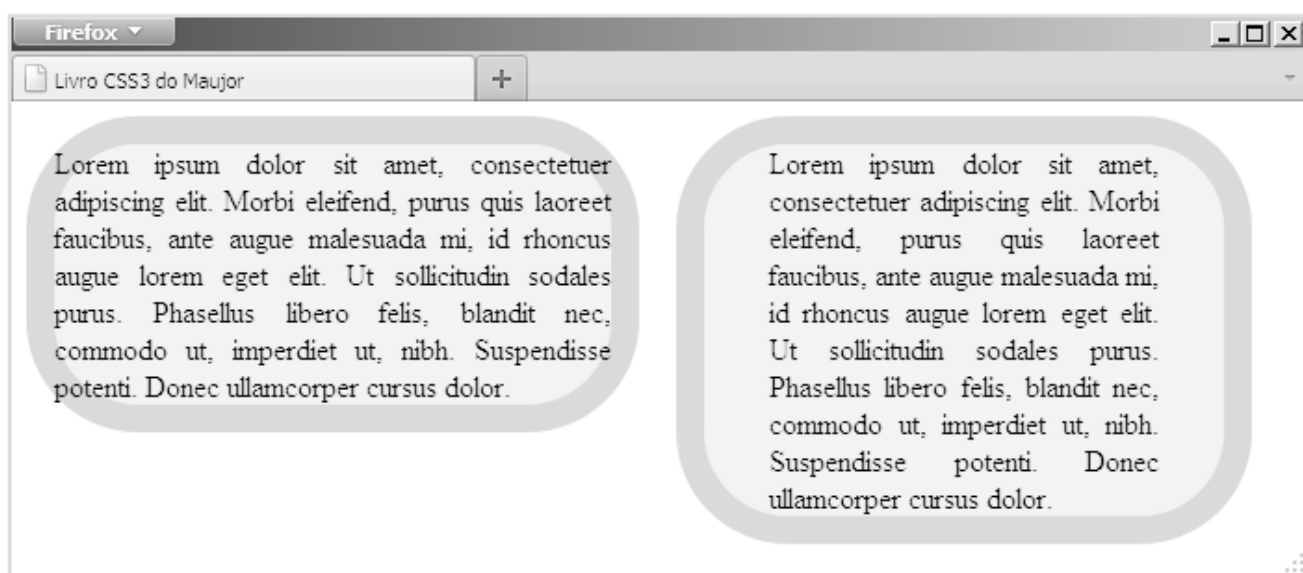


Figura 6.10 – Padding em boxes com bordas arredondadas.

6.2.6 border-image

Essa propriedade foi criada pelas especificações para as CSS3 e destina-se a definir uma imagem para ser usada como borda de um box. Em uma primeira leitura, podemos ser induzidos a concluir que basta definir uma imagem qualquer com determinada espessura e declarar uma borda para imagem que a imagem será inserida como borda. Contudo, não é assim que essa propriedade funciona. O mecanismo de inserção de imagens como borda é mais complexo e será explicado a seguir. A especificação do W3C no módulo Background e Borders diz o seguinte: Os autores podem criar uma imagem para ser usada como um estilo de borda. Nesse caso, a borda será construída com porções extraídas dos cantos e das laterais da imagem. Cada porção da imagem será devidamente recortada, aumentada, diminuída e/ou alongada de modo a preencher a área destinada à borda.

Extraír porções da imagem e tratá-las para que sejam inseridas como borda requer que se construam as imagens de forma a possibilitar uma consistente manipulação das imagens pelos mecanismos internos das CSS.

Observe, a seguir, uma típica imagem construída de forma modular (nove módulos de 17px x 17px) para ser aplicada como borda.

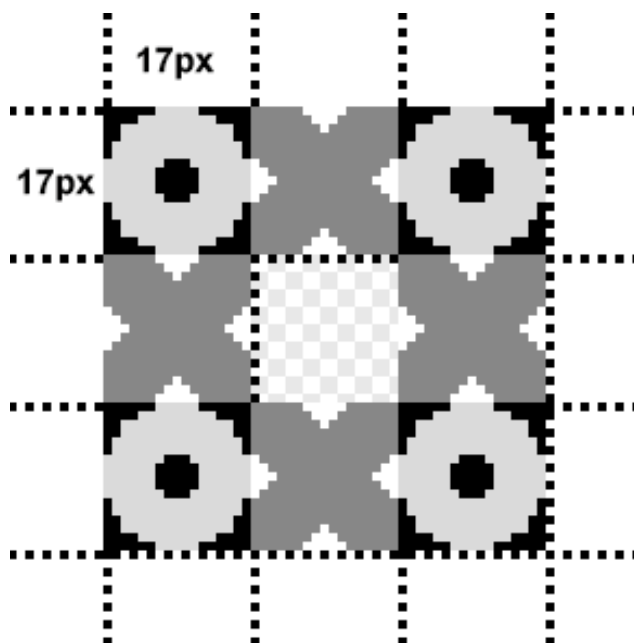


Figura 6.11 – Imagem modular para borda.

Imagens destinadas a servir como bordas de elementos HTML podem ser construídas em forma de uma matriz de 3 x 3, portanto com nove módulos, não necessariamente de dimensões iguais, ou, alternativamente, uma imagem qualquer poderá ser fatiada em nove módulos com uso da propriedade CSS `border-image-slice`, como veremos adiante.

Cada módulo será posicionado em um dos cantos e nas laterais do box,

destinando-se o módulo central à área de conteúdo do box.

Identificamos os seguintes módulos na imagem:

- **Cantos:** são os quatro círculos cinza-claro em fundo preto.
- **Borda superior:** é o módulo em forma de letra “x”, na parte superior da figura.
- **Borda lateral direita:** é o módulo em forma de letra “x”, na lateral direita da figura.
- **Borda inferior:** é o módulo em forma de letra “x”, na parte inferior da figura.
- **Borda lateral esquerda:** é o módulo em forma de letra “x”, na lateral esquerda da figura.
- **Conteúdo:** é o módulo central, no nosso caso foi feito transparente.

Você deve estar se perguntando o porquê de se construir uma imagem com nove módulos para ser inserida como borda de um box; ou ainda, como o conteúdo de um box de 400px x 300px pode ser inserido em um módulo de 17px x 17px?

As especificações dizem:

...cada porção da imagem será devidamente recortada, aumentada, diminuída e/ou alongada de modo a preencher a área destinada à borda.

Antes de prosseguirmos, observe, na figura 6.12, algumas possibilidades de inserção da nossa imagem como uma borda de 17px de espessura em um box de 200px x 150px.

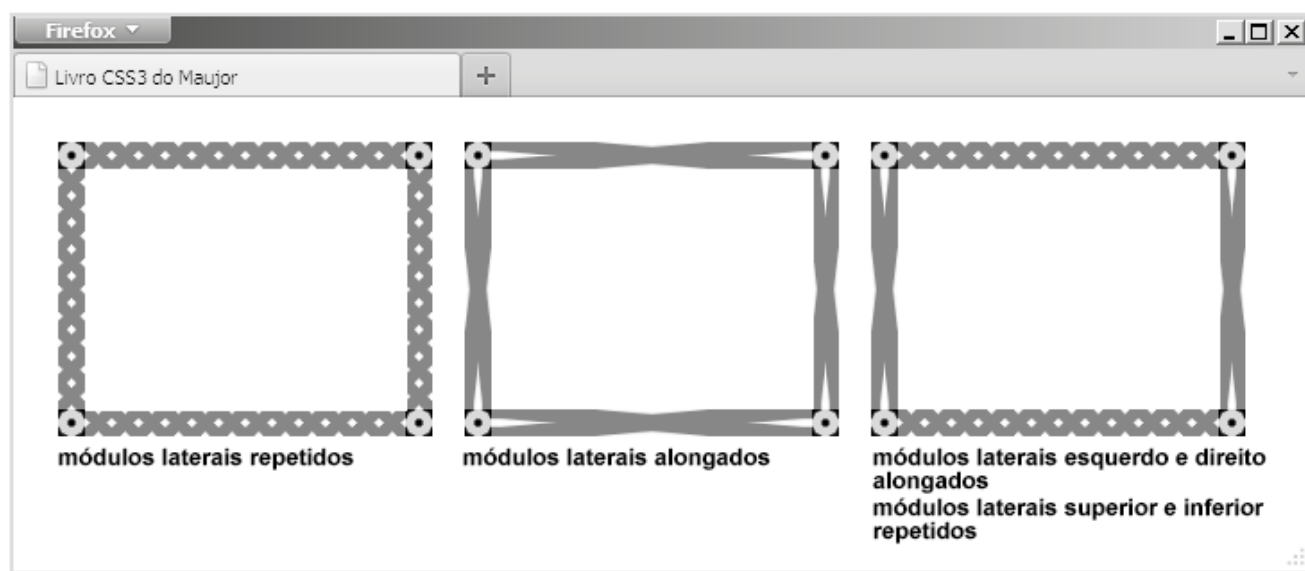


Figura 6.12 – Imagem para borda.

Note que os módulos para os cantos são inseridos tal como foram construídos, e os módulos laterais podem ser manipulados de forma a serem alongados ou se repetirem por toda a extensão da borda.

A manipulação dos módulos da imagem é feita com uso das propriedades CSS

border-image, que é uma abreviatura para declarar as seguintes propriedades: •
border-image-source

- **border-image-slice**
- **border-image-width**
- **border-image-outset**
- **border-image-repeat**

6.2.6.1 border-image-source

Essa propriedade destina-se a definir o URI no qual se encontra a imagem. A sintaxe para declarar essa propriedade é mostrada a seguir: **border-image-source: url(imagem-borda.png);**

6.2.6.2 border-image-slice

Essa propriedade destina-se a definir como está fatiada a imagem, de modo que sejam obtidos os nove módulos a ser manipulados para inserção da imagem como borda. Os valores possíveis para essa propriedade são: números, porcentagem e opcionalmente a palavra-chave **fill**.

A sintaxe geral para essa propriedade é mostrada nos exemplos a seguir:

```
border-image-slice: 17 23 12 34;  
border-image-slice: 20% 30% 40% 10%;  
border-image-slice: 15 17 fill;
```

Essa propriedade admite quatro, três, dois valores ou um valor e define a distância a ser considerada a partir de cada uma das bordas da imagem para seu fatiamento. A ordem de declaração dos valores define a distância em relação às bordas **top** (superior), **right** (direita), **bottom** (inferior) e **left** (esquerda), respectivamente.

Declarar quatro valores define as distâncias às quatro bordas, na ordem descrita anteriormente.

Declarar três valores define a distância à borda superior (primeiro valor), às bordas direita e esquerda (segundo valor) e à borda inferior (terceiro valor).

Declarar dois valores define a distância à borda superior e inferior (primeiro valor) e às bordas direita e esquerda (segundo valor).

Declarar um valor define uma distância igual às quatro bordas.

Para a imagem mostrada na figura 6.11, essa propriedade deverá ser declarada como mostrado a seguir: **border-image-slice: 17 17 17 17;**

```
border-image-slice: 17; /* igual ao anterior e mais apropriado */
```




Quando o valor dessa propriedade é em porcentagem, é necessário usar o sinal % depois do valor; contudo, por estranho que possa parecer, quando o valor é em pixels usa-se somente um número, sem especificar a unidade. Conclui-se que a única unidade de medida absoluta permitida para os valores dessa propriedade é o pixel.

Na figura 6.13, mostramos, à direita, como ocorre o fatiamento da imagem com uso dessa propriedade, e, à esquerda, a aplicação da imagem como borda em um elemento `div` da marcação.

A declaração dessa propriedade para a imagem mostrada é conforme a seguir: `border-image-slice: 10 20 15 fill;`

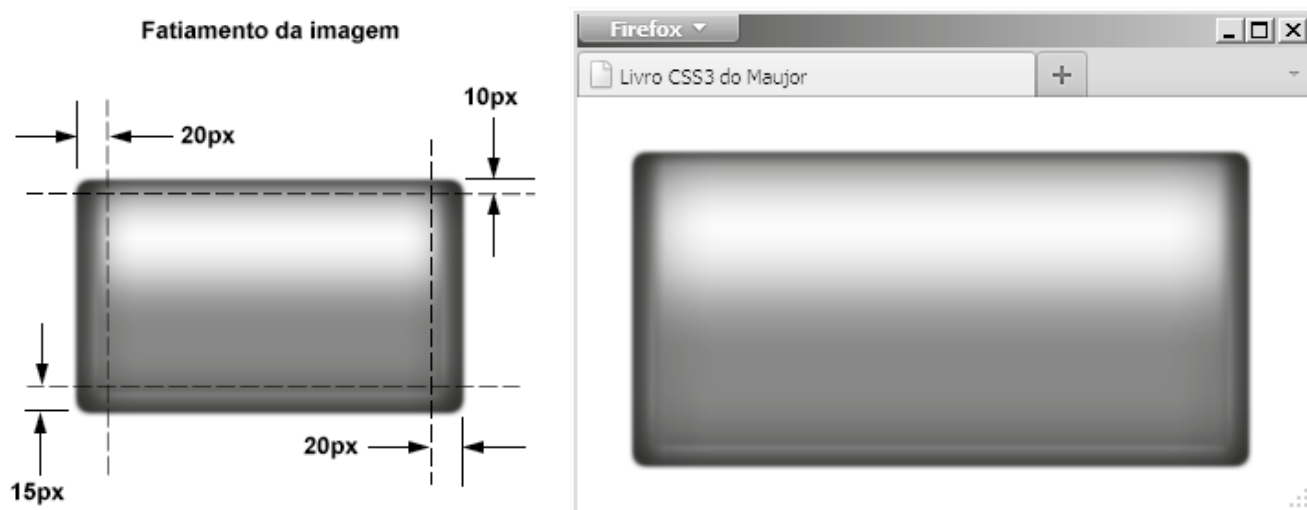


Figura 6.13 – Fatiamento de imagem para borda.

O fatiamento de uma imagem para borda pode também ser feito com uso de porcentagem.

A palavra-chave `fill`, conforme mostrada anteriormente, quando usada faz com que a parte central da imagem seja inserida na área de conteúdo do elemento ao qual se aplicou a borda. O não uso dessa palavra-chave torna a área de conteúdo transparente. O valor padrão é transparente, mas os navegadores atuais não suportam `fill` e por padrão mostram a área central.

6.2.6.3 border-image-width

Essa propriedade destina-se a definir a espessura da borda que receberá a imagem. Os valores possíveis para essa propriedade são: medidas CSS de comprimento, porcentagem, números e a palavra-chave `auto`.

Medidas CSS de comprimento, porcentagens e a palavra-chave `auto` dispensam explicações. Quando se usa números como valor para essa propriedade, eles representam o fator pelo qual se deve multiplicar a espessura da borda (quando definida com uso de `border-width`) para se obter a espessura da borda para inserção da imagem.

A sintaxe geral para essa propriedade é mostrada nos exemplos a seguir:

```
border-image-width: 10px 20px 30px 5px;  
border-image-width: 3em 5em 4em;  
border-image-width: 1 2 2;
```

Essa propriedade admite quatro, três, dois valores ou um valor que define a espessura a aplicar a cada uma das quatro bordas do box. A ordem de declaração dos valores é **top** (superior), **right** (direita), **bottom** (inferior) e **left** (esquerda), respectivamente.

6.2.6.4 border-image-outset

Essa propriedade destina-se a definir uma espessura fora da área normal de borda na qual a imagem se estenderá. Os valores possíveis para essa propriedade são: medidas CSS de comprimento e números.

A sintaxe geral para essa propriedade é mostrada nos exemplos a seguir:

```
border-image-outset: 20px 30px 35px 15px;  
border-image-outset: 6em 2em 1em;  
border-image-outset: 1 2 2;
```

Essa propriedade admite quatro, três, dois valores ou um valor que define a espessura a aplicar a cada uma das quatro bordas do box na qual a imagem se estenderá. A ordem de declaração dos valores é **top** (superior), **right** (direita), **bottom** (inferior) e **left** (esquerda), respectivamente.

6.2.6.5 border-image-repeat

Essa propriedade destina-se a definir como será dimensionada e distribuída a imagem pelas bordas e pela área de conteúdo. Os valores possíveis para essa propriedade são: **stretch**, **repeat**, **round** e **space**.

A sintaxe geral para essa propriedade é mostrada nos exemplos a seguir:

```
border-image-repeat: round;  
border-image-repeat: stretch round;  
border-image-repeat: repeat;
```

Essa propriedade admite dois valores ou um valor. O primeiro valor define como será a distribuição da imagem nas bordas superior e inferior. O segundo valor define como será a distribuição da imagem nas bordas esquerda e direita. Quando declarado apenas um valor, o segundo valor será igual ao primeiro.

O valor **stretch** faz com que o módulo da imagem seja alongado para preencher toda a extensão da borda.

O valor **repeat** faz com que o módulo da imagem se repita até preencher toda a extensão da borda.

O valor **round** tem o mesmo efeito de **repeat**, com a diferença de que se a repetição não resultar em número inteiro de módulos no preenchimento, o módulo é redimensionado, de modo que a repetição resulte em um número inteiro de módulos.

O valor **space** tem o mesmo efeito de **repeat**, com a diferença de que se a repetição não resultar em número inteiro de módulos no preenchimento, o espaçamento entre módulos é redimensionado, de modo que a repetição resulte em um número inteiro de módulos. Não redimensionamento de módulos, mas, sim, uma redistribuição deles.

6.2.6.6 border-image

Essa propriedade é a abreviatura para declaração de todas as propriedades para declarar bordas com imagens. Os valores para essa propriedade são: **border-image-source**, **border-image-slice**, **border-image-width**, **border-image-outset**, **border-image-repeat**.

A sintaxe geral para essa propriedade é rígida e segue o modelo conforme mostrado a seguir: **border-image: <'border-image-source'> || <'border-image-slice'> [<'border-image-width'>? [<'border-image-outset'>]?]? || <'border-image-repeat'>**
Exemplos:

```
border-image: url(borda.png) 30 20 30 20px 30px 40px 25px 25px round stretch;
border-image: url(borda.png) 10% 40% / 20px round;
border-image: url(borda.png) 30 stretch;
```

Para demonstrar o uso da declaração abreviada, considere a imagem mostrada e sua aplicação como borda em um box usando diferentes regras CSS. As regras CSS e a marcação HTML são mostradas a seguir: ► **HTML**

```
<div class="um"></div>
<div class="dois"></div>
<hr />
<div class="tres"></div>
<div class="quatro"></div>
```

► CSS

```
div {
border: 12px double #5ec874;
width:200px;
height:75px;
float:left;
```

```
}
```

```
hr {clear:both; height:20px; border:none; }
```

```
.um {
border-image: url('borda3.png') 62 / 62px round;
}

.dois {
margin: 0 40px;
border-image: url('borda3.png') 31 / 31px round;
}

.tres {
border-image: url('borda3.png') 1 / 1px round;
}

.quatro {margin-left:180px; }
```

Note que, para os quatro elementos **div** aos quais será aplicada a borda, definimos uma margem inicial de 12px de espessura no estilo **double** e na cor **#5ec874**. Se o navegador não suportar a inserção de bordas com uso de imagens, essa margem será renderizada. Para simular esse comportamento, não definimos borda com imagem para o elemento **div#quatro**.

Observe a renderização em diferentes navegadores.

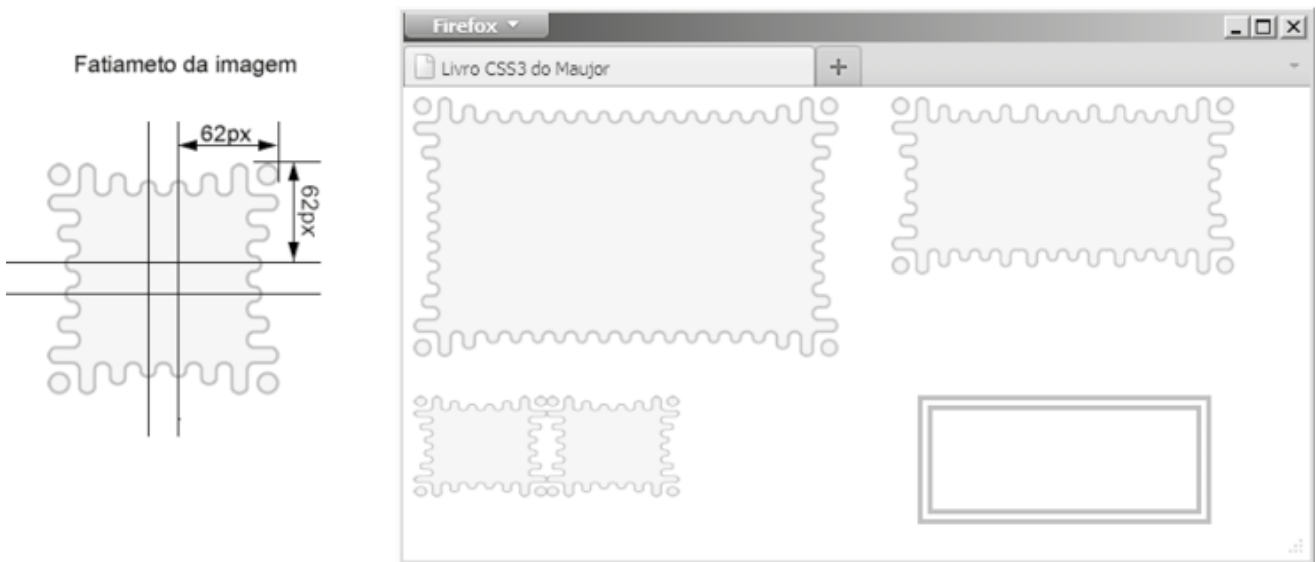


Figura 6.14 – Bordas com imagens no Firefox 4.

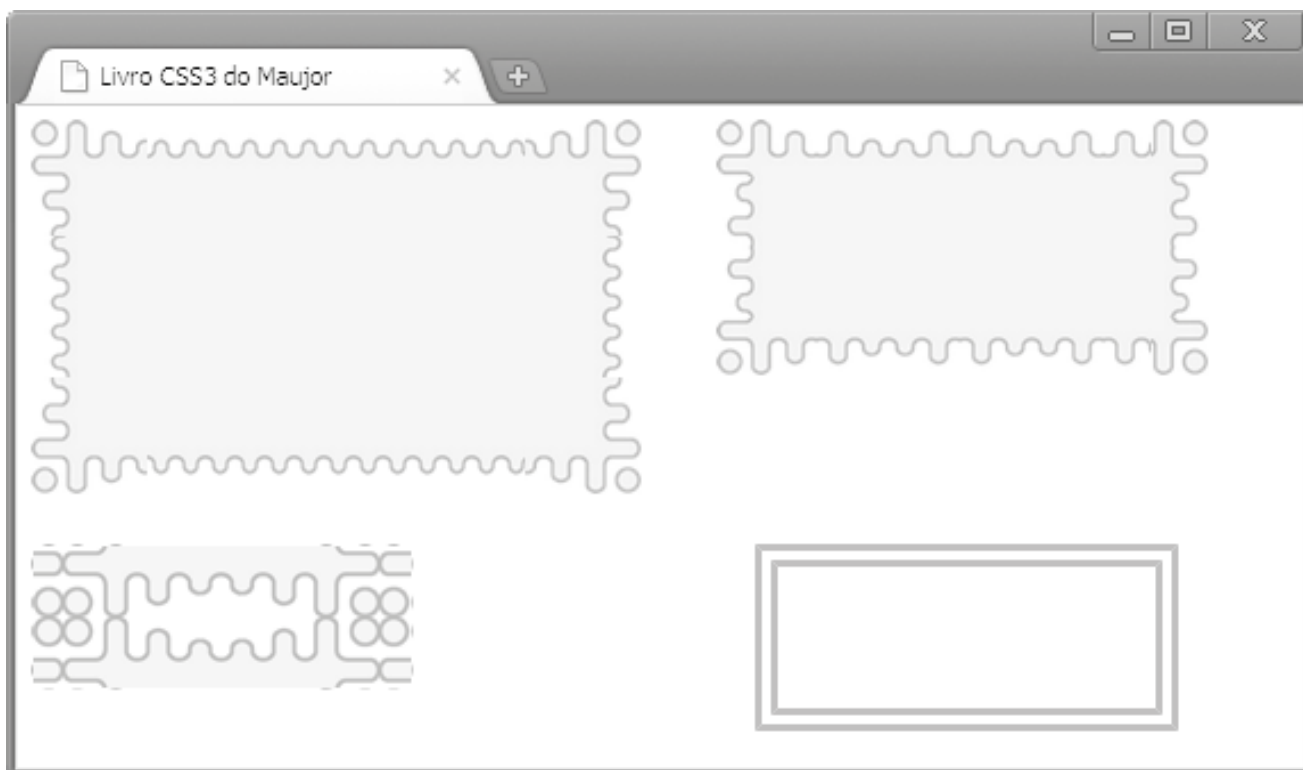


Figura 6.15 – Bordas com imagens no Chrome 11.0.

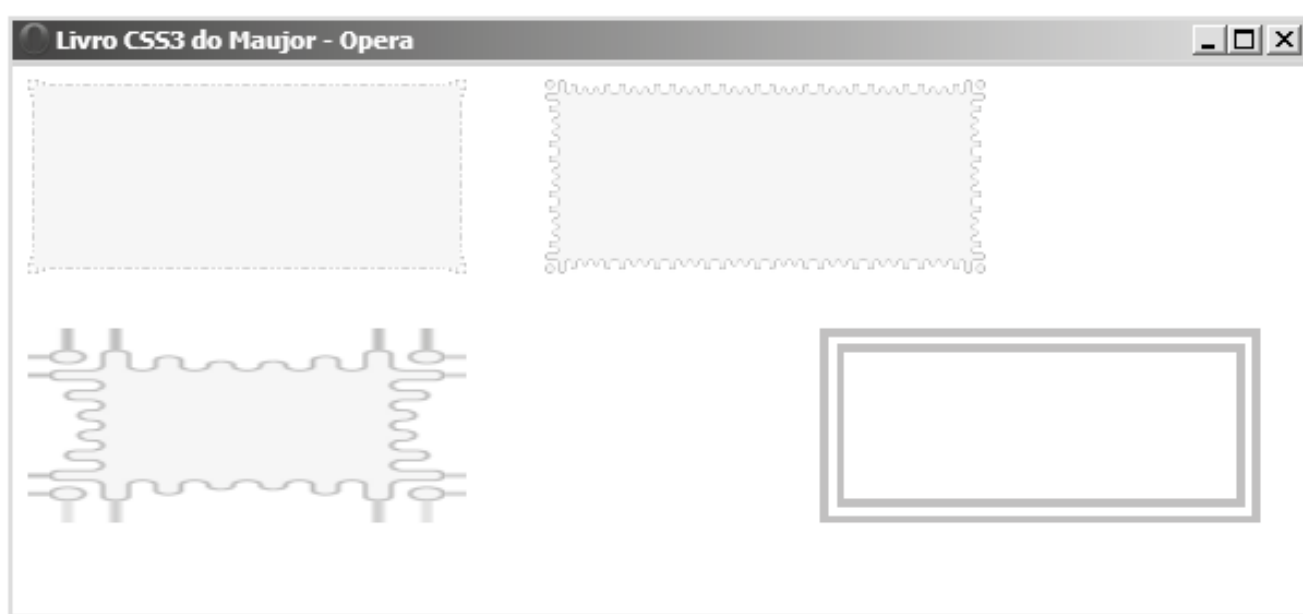


Figura 6.16 – Bordas com imagens no Opera 11.5.



Se você estiver interessado em aprofundar conhecimentos com relação a essa funcionalidade, indico duas ferramentas online nas quais você poderá fazer experimentos interativos. As ferramentas estão hospedadas em <http://www.norabrowndesign.com/css-experiments/border-image-anim.html> (<http://kwz.me/g8>) e <http://www.border-image.com/> (<http://kwz.me/g9>).

6.3 Box Shadow

Essa propriedade destina-se a criar sombras em boxes. A sintaxe geral para declarar os valores dessa propriedade é constituída de uma lista de valores para sombras.

Cada item da lista é formado pela palavra-chave **inset**, por duas, três ou quatro medidas CSS de comprimento e um valor para a cor da sombra.

A palavra-chave **inset** é opcional e, quando declarada, define sombra interna ao box. Omitindo-se essa palavra-chave, a sombra será externa ao box.

A primeira medida CSS define o deslocamento horizontal da sombra, a segunda, o deslocamento vertical, a terceira, o raio de esfumaçamento da sombra, e a quarta, o redimensionamento da sombra, ou seja, o quanto a sombra deverá crescer (valores positivos) ou encolher (valores negativos) em ambas as dimensões (largura e altura).

Valores positivos para o deslocamento horizontal movem a sombra para a direita, e negativos, para a esquerda. Valores positivos para o deslocamento vertical movem a sombra para baixo, e negativos, para cima.

Para melhor entender a aplicação de uma sombra em um box, considere um box retangular de dimensões **200px x 120px** na cor cinza-clara com um borda de **1px** de espessura na cor preta. O mecanismo CSS de aplicação de uma sombra na cor preta nesse box consiste na criação de um retângulo na cor preta com as mesmas dimensões do box (**200px x 120px**), posicionado atrás do box e deslocado na vertical e horizontal de modo que seus lados apareçam por trás do box.

Considere as seguintes regras CSS:

```
div.um { boxshadow: 10px 15px #000; }  
div.dois { boxshadow: 10px 15px 8px #000; }  
div.tres { boxshadow: -10px -15px #000; }
```

Para aplicação da sombra nos três elementos **div**, é criado um retângulo de **200px x 120px** na cor preta.

Para o **div.um**, o retângulo é deslocado **10px** para a direita e **15px** para baixo.

Para o **div.dois**, o comportamento é idêntico ao do **div.um**, mas aplicou-se um raio blur (espessura de esfumaçamento de **8px**).

Para o **div.três**, o retângulo é deslocado **10px** para a esquerda e **15px** para cima.

Note que, para os **div.um** e **div.tres**, foram declarados somente dois valores de medida CSS. Eles definem o deslocamento horizontal e vertical da sombra, respectivamente, e para o **div.dois**, a presença de um terceiro valor definiu um raio blur. A presença de um quarto valor definindo o redimensionamento da sombra será estudada adiante.

A aplicação de sombra nos três boxes conforme as regras CSS anteriores é mostrada na figura 6.17.

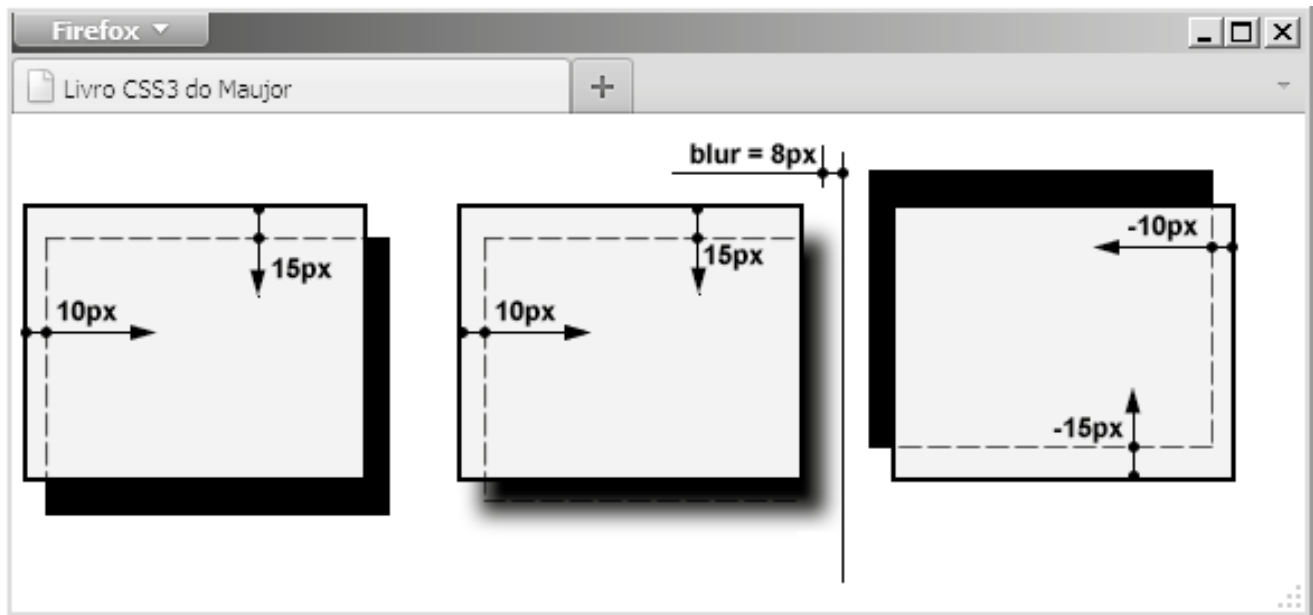


Figura 6.17 – Mecanismo de aplicação de sombras em boxes.



Quando se aplica um raio blur na sombra, as dimensões dela crescem em largura e altura, para cada lado, de um valor igual ao raio blur em relação às dimensões do box. Para o **div.dois** ao qual foi aplicado um raio blur de **8px**, as dimensões da sombra serão **216px x 136px**.

Observe, a seguir, mais algumas declarações para essa propriedade e a explicação de como cada uma delas funciona: `div.um { boxshadow: 35px 35px 0 -30px #000; }`

```
div.dois { boxshadow: 5px 5px 0 10px #000; }
div.tres { boxshadow: inset 5px 5px 8px #000; }
div.quatro { boxshadow: 5px 5px 3px #444, -5px -5px 3px #000; }
div.cinco { boxshadow: 5px 0 3px #000, -5px 0 3px #000; }
div.seis { boxshadow: 0 -10px #555, 30px 0 #999, 0 20px #666, -5px 0 #000; }
```

- Para o **div.um**, mostramos a declaração de um quarto valor de medida CSS igual a **30px**. Esse valor, por ser negativo, reduz as dimensões da sombra para **170px x 90px** (o box a aplicar a sombra é de **200px x 120px**). A sombra, para efeito de referência de deslocamento, se contrai e seu canto superior esquerdo permanece fixo e coincidente com o canto superior esquerdo do box.
- O **div.dois** demonstra o mesmo efeito mostrado para o **div.um** quando há um acréscimo nas dimensões da sombra por declaração de um quarto valor positivo. A sombra cresce **10px** e depois é deslocada **5px**. Note que, nesse caso, para efeito de referência de deslocamento, como a sombra se expandiu, o seu canto superior esquerdo não mais coincide com o canto superior do box.
- O **div.tres** demonstra a aplicação da palavra-chave **inset** com a finalidade de definir uma sombra interna ao box.

- O **div.quatro** demonstra a aplicação de sombras de forma independente para as laterais do box.
- O **div.cinco** demonstra a aplicação de sombra de forma independente para cantos contíguos do box.
- O **div.seis** demonstra a aplicação de sombra de forma independente para cada um dos cantos do box.

A aplicação de sombra nos seis boxes conforme as regras CSS anteriores é mostrada na figura 6.18.



Figura 6.18 – Variantes da aplicação de sombras em boxes.

Como pudemos constatar, a declaração de quatro medidas CSS para diferentes aspectos da sombra e a possibilidade de declarar uma lista de sombras para aplicação em cada um dos cantos do mesmo box nos fornecem uma infinidade de combinações para inserção de sombras. Mostramos os efeitos básicos e o mecanismo para aplicação das sombras. Entender esse mecanismo é indispensável para a criação de efeitos de alto impacto com uso de sombras.

É possível aplicar sombra no box do seletor **::first-letter**.



Se você estiver interessado em aprofundar conhecimentos com relação a essa funcionalidade, indico duas ferramentas online nas quais você poderá fazer experimentos interativos. As ferramentas estão hospedadas em <http://css3gen.com/boxshadow/> (<http://kwz.me/gf>) e <http://www.westciv.com/tools/boxshadows/index.html> (<http://kwz.me/gj>).

6.4 Text Shadow

Essa propriedade destina-se a criar sombras em textos. A sintaxe geral para declarar

os valores dessa propriedade é constituída por uma lista de valores para sombras. Cada item da lista é formado por duas, três ou quatro medidas CSS de comprimento e um valor para a cor da sombra.

A aplicação de sombras em textos se faz de forma idêntica à aplicação de sombras em boxes, a única diferença é que para textos não está previsto o valor `inset` para definição de sombra interna. Convém ressaltar que o quarto valor da medida CSS que define o redimensionamento da sombra, embora suportado para sombras em boxes, não é suportado pelos navegadores atuais, para sombra em textos.

O mecanismo de inserção de sombras em textos é idêntico ao de inserção de sombras em boxes. Assim, iremos apenas mostrar alguns exemplos, sem repetir as explicações que já constam do item 6.3.

Observe as seguintes regras CSS aplicadas de nove maneiras diferentes a nove instâncias de um mesmo texto: `.um { text-shadow: 3px 3px #a4a4a4; }`

```
.dois { text-shadow: 3px 3px 2px #a4a4a4; }  
.tres { text-shadow: -3px -3px #a4a4a4; }  
.quatro { text-shadow: 4px 0 #a4a4a4; }  
.cinco { text-shadow: 1px 1px #fff, 5px 5px 6px #000; }  
.seis { text-shadow: 2px -2px #a4a4a4, 4px -4px #eee, 6px -6px #000; }  
.sete { text-shadow: 0 0 2px #000; color:#eee; }  
.oito { text-shadow: 0 1px 0 #fff, 0 -1px 0 #000; background:#999; }  
.nove {  
  text-shadow:  
    1px -1px #000,  
    2px -2px #000,  
    3px -3px #000,  
    4px -4px #000,  
    5px -5px #fff,  
    6px -6px #000,  
    7px -7px #000,  
    8px -8px #000,  
    9px -9px #000;  
}
```

Na figura 6.19, mostramos o efeito de cada uma das regras de estilo no texto “CSS3”.



Figura 6.19 – Sombras em textos.



É possível aplicar sombras nos textos dos seletores `::first-letter` e `::first-line`.

Se você estiver interessado em aprofundar conhecimentos com relação a essa funcionalidade, indico uma ferramenta online na qual você poderá fazer experimentos interativos. A ferramenta está hospedada em <http://css3gen.com/text-shadow/> (<http://kwz.me/gq>).

CAPÍTULO 7

Cores e opacidade

O Módulo CSS3 para Cores descreve as propriedades CSS que permitem aos autores especificar a cor de frente e a opacidade para um elemento da marcação. Define ainda os valores CSS para cores e para opacidade. Neste capítulo, estudaremos esse módulo.

7.1 Introdução

Todas as funcionalidades para cores previstas nas versões 1 e 2 das CSS foram incluídas nesse módulo e, portanto, continuam válidas. Houve uma série de acréscimos de funcionalidades, principalmente no que diz respeito à criação de mecanismos para definir a opacidade.

As duas propriedades CSS definidas nesse módulo são:

- **color** • **opacity** (criada nas CSS3)

7.2 Propriedade color

Essa propriedade destina-se a definir a cor dos conteúdos textuais (ou cor de frente) de elementos da marcação. Os valores possíveis são: um valor hexadecimal, as palavras-chave **transparent**, **currentColor** e outras palavras-chave que são o nome de algumas cores (por exemplo: **red**, **blue**, **green** etc.), valores RGB, RGBA, HSL e HSLA.

Observe, a seguir, declarações CSS típicas, com uso de cada um desses valores, para definição da propriedade **color**: **color: #ff0000; /* hexadecimal minúsculas */**

color: #F9DA67; /* hexadecimal maiúsculas */

color: #ad8; /* hexadecimal abreviada */

color: pink; /* palavra-chave */

color: rgb(255, 200, 32); /* RGB inteiros */

color: rgb(100%, 26%, 47%); /* RGB porcentagem */

color: rgba(200, 100, 57, 0.4); /* RGB inteiros com opacidade */

color: rgba(90%, 86%, 37%, 0.6); /* RGB porcentagem com opacidade */

color: hsl(120, 75%, 50%) /* HSL */

color: hsla(0, 105%, 25%, 0.8) /* HSL com opacidade */

color: transparent; /* palavra-chave */

color: currentColor; /* criada nas CSS3 */

7.2.1 hexadecimal e RGB

A representação de uma cor em notação hexadecimal começa com um sinal de

tralha (#) seguido de seis números hexadecimais (os números hexadecimais são compostos de combinações de: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F). A grafia dos números hexadecimais é insensível ao tamanho da caixa, isto é, letras maiúsculas e minúsculas são igualmente válidas.

O valor de uma cor hexadecimal é determinado decompondo-se o número hexadecimal que representa a cor em três grupos de dois números hexadecimais. Essa estrutura para o número hexadecimal é denominada *triplete hexadecimal*.

Cada um dos três grupos representa as cores vermelha, verde e azul, respectivamente, que entram na composição final da cor. A abreviatura em inglês para vermelho, verde, azul é RGB (red, green, blue).

Cada grupo de dois números hexadecimais é representado por *1 byte*, que por sua vez é capaz de armazenar um número na faixa de 0 a 255 (em notação decimal) ou 00 a FF (em notação hexadecimal) ou ainda uma porcentagem na faixa de 0% a 100%.

As cores vermelha, verde e azul (RGB), que se combinam para formar a cor final, são também chamadas de *componentes* da cor. Cada um dos componentes da cor entra com uma *intensidade* que varia da cor preta, representada pelo decimal 0, hexadecimal 00 ou 0 em porcentagem, até a cor branca, representada pelo número decimal 255, hexadecimal FF ou 100 em porcentagem.

Note que na faixa de 0 a 255 existem 256 números. Assim, o total possível de combinações de números para formação de cores é igual a $256 \times 256 \times 256 = 16.777.216$, o que significa que o uso dessa representação de cores permite que se defina mais de 16 milhões de cores.

Por exemplo: a cor hexadecimal #2E8B57 é obtida pela mistura da cor vermelha em uma quantidade 2E de vermelho (R), 8B de verde (G) e 57 de azul (B).

Transformando os três números hexadecimais em decimais, temos:



Nos cálculos mostrados a seguir, o sinal * representa multiplicação, e **, potenciação.

$$\begin{aligned} 2E &= 2 \cdot 16^1 + 14 \cdot 16^0 = 32 + 14 = 46 \text{ (Nota: E hexadecimal é igual a 14 decimal)} \\ 8B &= 8 \cdot 16^1 + 11 \cdot 16^0 = 128 + 11 = 139 \text{ (Nota: B hexadecimal é igual a 11 decimal)} \\ 57 &= 5 \cdot 16^1 + 7 \cdot 16^0 = 80 + 7 = 87 \end{aligned}$$

Transformando os três números decimais em porcentagem, temos: $46 \cdot 100 / 255 = 18\%$

$$139 \cdot 100 / 255 = 54,5\%$$

$$87 \cdot 100 / 255 = 34\%$$

Assim, a cor **#2E8B57** é resultante de uma mistura cujo componente de cor vermelha entra com uma intensidade de 18%, de cor verde 54,5% e de cor azul 34%.

As seguintes sintaxes para declaração dessa cor são válidas (representam a mesma cor): `color: #2e8b57; /* hexadecimal minúsculas */`

`color: rgb(46, 139, 87); /* RGB inteiros */`

`color: rgb(18%, 54.5%, 34%); /* RGB porcentagem */`

As CSS admitem que a declaração de cores com uso de números hexadecimais seja abreviada para aquelas cores nas quais cada um dos três *componentes* seja representado por dígitos iguais, ou seja, números hexadecimais no formato **#XXYYZZ**. Nesses casos, a abreviatura se faz para **#XYZ**, ficando subentendido que cada um dos três dígitos da forma abreviada é dobrado. Observe, a seguir, alguns exemplos.

`color: #ff9900; = color: #f90;`

`color: #55ddaa; = color: #5DA;`

`color: #FFEECC; = color: #fec;`

`color: #88BB43; /* não é possível abreviar */`

`color: #333344; = color: #334;`

Você pode usar maiúsculas, minúsculas e até mesmo uma combinação delas para definir cor hexadecimal, bem como adotar ou não a sintaxe abreviada. A maioria dos autores usa minúscula e adota a sintaxe abreviada, mas isso é uma questão de gosto pessoal. Adote uma maneira e siga com ela em todos os seus trabalhos de desenvolvimento.

Computadores e monitores modernos suportam 24 bits de cores e são capazes de renderizar consistentemente as mais de 16 milhões de cores possíveis de se representar com a sintaxe mostrada. Equipamentos antigos suportam 8 bits de cores e estão restritos à renderização consistente de 256 cores, apenas. O conjunto dessas 256 cores recebeu o nome de *cores seguras para a web* e é resultante da combinação de R (red), G (green) e B (blue) com intensidades de 0%, 20%, 40%, 80% e 100%. Observe a transformação dessas porcentagens.

Em número decimal:

0% = 0

20% = 0,2 * 255 = 51

40% = 0,4 * 255 = 102

60% = 0,6 * 255 = 153

80% = 0,8 * 255 = 204

100% = 1 * 255 = 255

Em número hexadecimal:

$51 / 16 = 3$ restam $3 = 33$

$102 / 16 = 6$ restam $6 = 66$

$153 / 16 = 9$ restam $9 = 99$

$204 / 16 = 12$ restam $12 = CC$ (Nota: 12 é igual a C hexadecimal) $255 / 16 = 15$ restam $15 = FF$ (Nota: 15 é igual a F hexadecimal) Observe, a seguir, algumas declarações de *cores seguras para a web*: `color: #33cc66; /* tom de verde */`

`color: rgb(255, 204, 102); /* tom de amarelo */`

`color: rgb(40%, 20%, 100%); /* tom de azul */`

`color: #cc9966; /* marrom claro */`

`color: rgb(204, 153, 102); /* marrom claro */`

`color: rgb(80%, 60%, 40%); /* marrom claro */`

As três últimas cores declaradas resultam em um mesmo tom marrom-claro, pois as declarações com uso de hexadecimal e RGB são equivalentes.

A sintaxe hexadecimal abreviada foi definida na especificação para as CSS, mas não consta das especificações para a HTML. Alguns navegadores antigos e em particular o Internet Explorer, até pelo menos a versão 8, não reconhecem a sintaxe abreviada quando ela é definida em atributos HTML que não sejam o atributo `style`. O exemplo a seguir usa o atributo ultrapassado `font` para demonstrar esse comportamento.

Não use o atributo `font`. Ele foi empregado no exemplo apenas para demonstrar o suporte à sintaxe hexadecimal abreviada na linguagem HTML e nas CSS.

```
<p><font color="#f00">Texto do parágrafo um</font></p> <p><font  
style="color:#f00">Texto do parágrafo dois</font></p> O primeiro parágrafo  
será na cor vermelha, exceto em navegadores IE8 e anteriores, mas o  
segundo parágrafo será na cor vermelha tanto em navegadores antigos quanto  
nos IE.
```

7.2.2 transparent

O valor `transparent` para cores foi criado pelas especificações para as CSS1 e era válido apenas para a propriedade `background-color`. As CSS2 estenderam a validade desse valor para a propriedade `border-color`, e as CSS3 preveem validade do valor para qualquer propriedade CSS que admita declaração de cor. Como o próprio nome sugere, esse valor destina-se a tornar transparente a propriedade cuja cor for declarada com ele.

As CSS1 e CSS2 definiram o valor inicial da cor da borda de um elemento como sendo igual à cor do próprio elemento. Por exemplo: se definirmos uma espessura e um estilo para a borda de um elemento `h1` e omitirmos a cor da borda, ela será a cor

definida para o elemento `h1`. Observe o exemplo a seguir:

```
h1 {  
  color: red;  
  border: 2px solid;  
}
```

A borda do elemento `h1` será na cor vermelha.

7.2.3 Palavra-chave

Outra forma de declarar cores é com uso de valores expressos por palavras-chave. As especificações para as CSS3 preveem três grupos de palavras-chave.

- **Palavras-chave para definir cores básicas.** Esse grupo é constituído de 16 palavras-chave, a saber: `aqua`, `black`, `blue`, `fuchsia`, `gray`, `green`, `lime`, `maroon`, `navy`, `olive`, `purple`, `red`, `silver`, `teal`, `white` e `yellow`.



Consulte uma tabela mostrando as 16 cores básicas em <http://www.w3.org/TR/css3-color/#html4> (<http://kwz.me/gx>).

- **Palavras-chave para definir cores estendidas ou cores X11.** Esse grupo é constituído de 147 palavras-chave (incluindo as 16 para cores básicas), como: `brown`, `coral`, `cian`, `dimgray`, `indigo`, `khaki`, `mintcream`, `mistyrose`, `sienna`, `turquoise`, `tomato`, `whitesmoke` etc.



Consulte uma tabela mostrando as 147 cores X11 no apêndice D deste livro. Para mais detalhes sobre cores X11, consulte http://en.wikipedia.org/wiki/X11_color_names (<http://kwz.me/gv>).

- **Palavras-chave para definir cores de acordo com o sistema operacional.** Esse grupo é constituído de 28 palavras-chave que se referem a componentes da interface gráfica do sistema operacional do usuário. Esses valores são de uso prático bastante limitado e não faremos considerações detalhadas sobre eles. Para informações, consulte <http://www.w3.org/TR/css3-color/#css-system> (<http://kwz.me/gw>).

As recomendações de acessibilidade ao conteúdo web (WCAG2.0), em relação a cores, dizem o seguinte: 1.4.1 Utilização da cor: a cor não é utilizada como o único meio visual de transmitir informações, indicar uma ação, pedir uma resposta ou distinguir um elemento visual (nível A).

O item 1.4.1 das WCAG2.0 dispensa mais explicações, portanto é errado criar uma marcação como mostrada a seguir: `<p>Se você é a favor clique no box verde, mas se é contra clique no box vermelho</p>` O trecho a seguir, a respeito de cores, também foi retirado das WCAG2.0: 1.4.3 Contraste (mínimo): a apresentação visual de texto e imagens de texto tem uma relação de contraste de, no mínimo, 4.5:1, exceto para o seguinte: (nível AA).

Texto ampliado: texto ampliado e as imagens compostas por texto ampliado têm uma relação de contraste de, no mínimo, 3:1.

Texto em plano secundário: o texto ou as imagens de texto que fazem parte de um componente de interface de usuário inativo, que são meramente decorativos, que não estão visíveis para ninguém ou que são parte de uma imagem que inclui outro conteúdo visual significativo não têm requisito de contraste.

Logotipos: o texto que faz parte de um logotipo ou uma marca comercial não tem requisito de contraste.

Assim, ao definir cores para o plano de frente e de fundo, certifique-se de que o contraste entre as cores está dentro dos limites mínimos estabelecidos pela WCAG2.0.

Existem ferramentas para verificar o contraste de cores. Uma ferramenta online para verificar o contraste de cores em uma página web pode ser encontrada em <http://www.accesskeys.org/tools/color-contrast.html> (<http://kwz.me/gz>). Opcionalmente, você pode fazer download da ferramenta para análise de contraste de cores hospedada em <http://www.visionaustralia.org.au/info.aspx?page=628> (<http://kwz.me/gA>) e que se destina a instalação em sistema operacional Windows.

7.2.4 `currentColor`

Em SVG, esse mecanismo de aplicação de cor não funciona e, em função disso, a linguagem criou o valor `currentColor` para suprir uma omissão das CSS1 e CSS2. As CSS3 incorporaram a palavra-chave `currentColor`, já adotada pela SVG como um valor válido para definir cores.

Esse valor declara que a cor a ser aplicada a uma determinada propriedade CSS em um elemento é igual à cor declarada para a propriedade `color` de seu elemento ancestral mais próximo. O exemplo a seguir esclarece a aplicação desse valor, considerando elementos `p` como elementos-filhos de `div`.

```
div { color: red; }  
p { border: 1px solid currentColor; }
```

Todos os parágrafos contidos no `div` terão uma borda na cor vermelha (red). O uso desse valor para declarar cor não é muito comum, mas dependendo do contexto poderá tornar-se bastante útil. Note que, no exemplo mostrado, basta alterar a cor declarada para o `div` para que seja alterada a cor da borda de todos os parágrafos nele contidos.

Observe que, no exemplo mostrado, se alterarmos a declaração de borda para `p { border: 1px solid inherit; }`

nenhuma borda será adicionada aos parágrafos, pois não existe borda a ser herdada. Fica evidente que `currentColor` nem sempre é o mesmo que `inherit`.

Dizemos nem sempre, pois quando se trata de declarar a própria propriedade **color**, os valores se tornam equivalentes. O exemplo a seguir esclarece esse caso, considerando elementos **p** como elementos-filhos de **div**: `div { color: red; }`

```
p { color: currentColor; }
```

produz o mesmo efeito que

```
div { color: red; }
```

```
p { color: inherit; }
```

7.2.5 HSL

As CSS3 criaram o valor HSL para definir cores. A declaração de cores com uso de HSL (hue, saturation, lightness), não existia nas CSS2.1. Ela permite que você declare as cores com uso de três parâmetros: Hue = tom; saturation = saturação; e lightness = luminosidade.

A sintaxe geral para a declaração com uso desse valor é conforme mostrada no exemplo a seguir: `color: hsl(120, 75%, 50%);`

O primeiro valor é para o tom (hue) da cor. O seu valor é um número que representa a medida de um ângulo (varia de 0 grau a 360 graus) apontando para um tom de cor na roda de cores. Observe os valores do ângulo e os respectivos tons de cor:

Ângulo	Tons de cor
0	vermelho
60	amarelo
120	verde
180	ciano
240	azul
300	púrpura
360	vermelho

Convém ressaltar que o valor angular segue as regras da trigonometria para medida de ângulos. Por exemplo: 0 grau = 360 graus, -120 graus = 240 graus e assim por diante. Contudo, não vejo razão para se adotar um valor fora da faixa 0 grau – 360 graus para definir o parâmetro hue.

O segundo valor é para a saturação (saturation) da cor. O seu valor é expresso em porcentagem. Um valor igual a 100% representa saturação total da cor, e 0% é um

leve sombreado cinza de saturação.

O terceiro valor é para a luminosidade (lightness). O seu valor é expresso em porcentagem. Um valor igual a 100% resulta em cor branca, e 0%, em cor preta, sendo 50% o valor normal.

A figura 7.1 dá uma ideia aproximada de como os parâmetros HSL determinam a cor final. Para visualizar o diagrama em cores, visite http://en.wikipedia.org/wiki/HSL_and_HSV (<http://kwz.me/gD>).

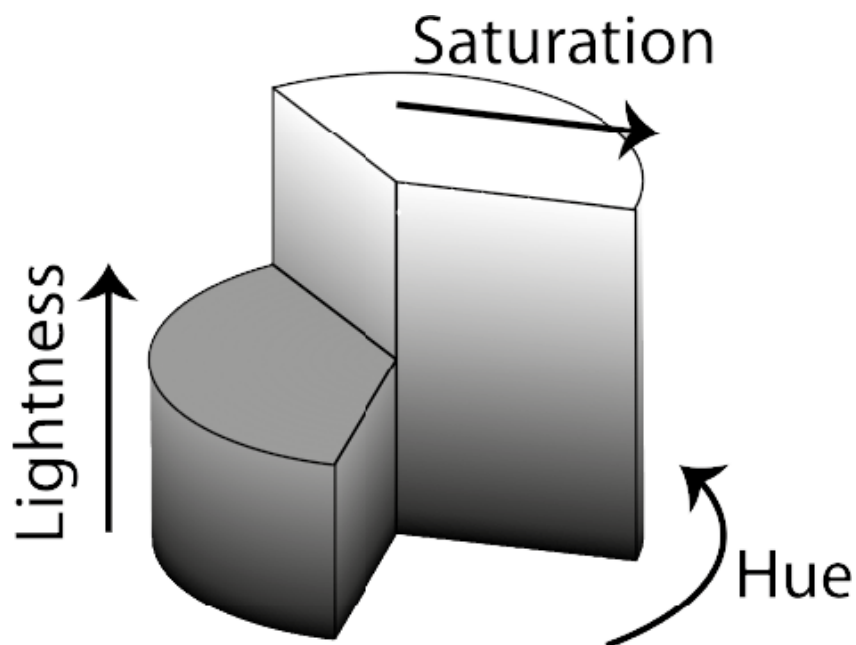


Figura 7.1 – Diagrama HSL.

Exemplos de declaração de cor com uso de valores HSL: `color: hsl(0, 100%, 50%)`
/* cor vermelha */

```
color: hsl(120, 100%, 50%) /* cor verde */  
color: hsl(120, 100%, 25%) /* cor verde escura */  
color: hsl(120, 100%, 75%) /* cor verde clara */  
color: hsl(120, 75%, 75%) /* cor verde pastel */
```

Como você já deve ter notado, o uso de HSL para declarar cores é semelhante ao uso de RGB. A vantagem do uso de HSL sobre RGB é que HSL proporciona uma maneira mais simples de se obter variações de uma mesma cor.



O segundo (saturation) e o terceiro (lightness) parâmetros dessa declaração são expressos em porcentagem, e a sintaxe para a declaração determina que o sinal de porcentagem é obrigatório mesmo se o valor for 0 (zero). Essa exigência pode parecer absurda, uma vez que zero é zero independentemente da unidade de medida, mas, nesse caso, é assim que a sintaxe exige.

7.2.6 HSLA

As CSS3 criaram uma extensão do valor HSL denominada HSLA, para definição de

cores. Esse valor acrescenta um quarto parâmetro à declaração (hue, saturation, lightness, alpha). O quarto parâmetro, denominado alpha, define a opacidade (ou transparência) da cor.

A sintaxe geral para a declaração com uso desse valor é conforme mostrada no exemplo a seguir: `color: hsla(120, 75%, 50%, 0.6);`

Os três primeiros parâmetros têm o mesmo significado que têm quando se usa a declaração HSL, e o quarto é um valor compreendido entre 0 e 1. O valor 0 representa transparência total, e o valor 1, opacidade total. Assim, um valor igual a 0.6 significa 60% opaco ou 40% transparente.

Para verificar o resultado da aplicação de cores com uso desse valor, visite a página web em <http://www.maujor.com/tutorial/css3-modulo-para-cores.php> (<http://kwz.me/gF>). Nela, sob o título “Declaração com HSLA”, você encontra um quadro interativo, no qual poderá inserir diferentes valores para HSLA e verificar o resultado.

7.2.7 RGBA

As CSS3 criaram uma extensão do valor RGB denominada RGBA, para definição de cores. Esse valor acrescenta um quarto parâmetro à declaração (red, green, blue, alpha). O quarto parâmetro, denominado alpha, define a opacidade (ou transparência) da cor.

A sintaxe geral para a declaração com uso desse valor é conforme mostrada no exemplo a seguir: `color: rgba(255, 204, 102, 0.4);`

`color: rgba(40%, 20%, 100%, 0.7);`

Os três primeiros parâmetros têm o mesmo significado que têm quando se usa a declaração RGB, e o quarto é um valor compreendido entre 0 e 1. O valor 0 representa transparência total, e o valor 1, opacidade total. Assim, um valor igual a 0.7 significa 70% opaco ou 30% transparente.

Para verificar o resultado da aplicação de cores com uso desse valor, visite a página web em <http://www.maujor.com/tutorial/css3-modulo-para-cores.php> (<http://kwz.me/gF>). Nela, sob o título “Declaração com RGBA”, você encontra um quadro interativo, no qual poderá inserir diferentes valores para RGBA e verificar o resultado.

7.2.8 Cores do S0

As CSS2 preveem um mecanismo capaz de definir cores com base no sistema operacional do usuário. Esse mecanismo é usado para estilizar elementos e controles utilizados em Interfaces de Usuário (UI – User Interface) de uma forma que assumam uma cor semelhante às cores usadas em elementos e controles de interface do sistema operacional do usuário. Por exemplo: os valores `ButtonFace` e `ButtonText`

se destinam a simular as cores usadas na face e no texto dos botões da interface do Windows ou do Mac, por exemplo, caso o visitante da página esteja usando Windows ou Mac.

As CSS2 previram 28 valores para simular as cores do sistema operacional. A título de informação, citamos alguns desses valores: `ActiveCaption`, `Highlight`, `InfoText`, `Scroll`, `ThreeDShadow` *etc.*

Por exemplo: para estilizar a cor de face de um botão com a mesma cor de face dos botões do sistema operacional do usuário, usamos a seguinte regra CSS: `button { background-color: ButtonFace; }`

Esse mecanismo foi colocado em desuso pelas CSS3. Por questões de retrocompatibilidade, os navegadores deverão continuar oferecendo suporte para ele; contudo, em novos projetos, use a alternativa prevista nas CSS3.

Para substituir a definição de cores com base no sistema operacional do usuário, que foi colocada em desuso, as CSS3 criaram uma propriedade nova denominada `appearance`, que se destina a definir a aparência de um elemento com base no sistema operacional utilizado pelo usuário.

Os valores possíveis para essa propriedade são: `icon`, `window`, `desktop`, `workspace`, `document`, `tooltip`, `status-bar`, `dialog`, `message-box`, `button`, `caption`, `small-caption`, `push-button`, `hyperlink`, `radio-button`, `checkbox`, `menu-item`, `tab`, `menu`, `menubar`, `pull-down-menu`, `pop-up-menu`, `list-menu`, `radio-group`, `checkbox-group`, `outline-tree`, `range`, `field`, `combo-box`, `signature`, `password`.

7.3 Propriedade `opacity`

Essa propriedade destina-se a definir a opacidade (ou transparência) de um elemento e seus elementos descendentes e bordas. O valor possível para essa propriedade é um número compreendido entre 0 e 1. O valor 0 representa transparência total, e o valor 1, opacidade total. Assim, um valor igual 0.9 significa 90% opaco ou 10% transparente.

Observe, a seguir, declarações CSS típicas para a propriedade `opacity`: `opacity: 0.7;`
`opacity: .4;`
`opacity: 0;`

Aplicar opacidade em um elemento faz com que todos os seus elementos-filhos herdem o valor da opacidade aplicada. Considere o texto de um parágrafo inserido em um `div`, como mostrado a seguir: `<div>`

```
<p class="um">Texto do parágrafo um</p>
<p class="dois">Texto do parágrafo dois</p> </div>
```

e as seguintes regras CSS:

```
div {  
  opacity: .5;  
  color: black;  
  
}
```

```
p.um { opacity: .5 }  
p.dois { opacity: 1 }
```

O texto dos dois parágrafos herdará a opacidade .5, definida para o elemento `div`. A regra de estilo que define a opacidade para o `p.um` faz com que o texto tenha opacidade $0.5 * 0.5 = 0.25$. A regra de estilo que define a opacidade para o `p.dois` não produz nenhum efeito, pois não é possível aumentar a opacidade herdada, somente diminuí-la, como foi feito com `p.um`.

Felizmente, o uso de RGBA ou HSLA para aplicar opacidade, ao contrário do uso de `opacity`, não faz com que os elementos descendentes herdem a opacidade. Observe o exemplo anterior, usando RGBA: `<div>`

```
<p class="um">Texto do parágrafo um</p>  
<p class="dois">Texto do parágrafo dois</p> </div>
```

e as regras CSS:

```
div {  
  color: rgba(0, 0, 0, .5);  
  
}
```

```
p.um { color: rgba(0, 0, 0, .5); }  
p.dois { color: rgba(0, 0, 0, 1); }
```

Nesse exemplo, diferentemente do anterior, `p.um` terá opacidade 0.5 e `p.dois` será totalmente opaco. Isso porque aplicar opacidade com uso RGBA (ou HSLA) não implica em herança CSS.

CAPÍTULO 8

Fontes

O Módulo CSS3 para Fontes descreve as propriedades CSS que permitem aos autores especificar as características das fontes a serem usadas em uma aplicação, bem como estabelece as diretrizes de carregamento e definição de fontes remotas carregadas dinamicamente. Neste capítulo, estudaremos esse módulo.

8.1 Introdução

Fontes definidas com CSS podem ter duas origens: fontes instaladas no sistema operacional do usuário e fontes instaladas em um servidor remoto que são baixadas para a máquina do usuário quando a página é carregada.

O mecanismo CSS de aplicação de fontes busca informações sobre fontes instaladas no sistema operacional diretamente na máquina do usuário, mas para fontes armazenadas em servidor remoto, as informações devem ser fornecidas com uso da declaração CSS `@font-face`, que estudaremos adiante. Fontes instaladas em servidor remoto são denominadas e conhecidas como *web fonts*.

8.1.1 Tipografia

A especificação para as CSS3 apresenta, a título de informação, uma rápida passada por conceitos tipográficos que considera relevantes para o entendimento do mecanismo CSS de aplicação de fontes. Traduzimos, a seguir, com pequenas adaptações, o que a especificação do W3C informa sobre tipografia.

Conceitos tipográficos variam de acordo com o lugar do mundo, assim, não existe uma maneira unificada de classificação de fontes que atenda a todos os idiomas e culturas. Até mesmo para letras latinas comuns, muitas variações são possíveis. Na figura 8.1, mostramos o caractere latino *a* escrito com diferentes glifos.



Figura 8.1 – Um caractere e vários glifos.

Fontes podem diferir no que diz respeito à sua anatomia. Fontes latinas podem conter adornos em suas extremidades, ser constituídas de hastes de diferentes espessuras ou ainda conter ou não serifas. Na figura 8.2, mostramos um mesmo caractere latino com e sem serifas.



Figura 8.2 – Caractere com e sem serifa.

Fontes de alfabeto não latino também podem apresentar diferenças na apresentação de suas hastes. Observe a figura 8.3 mostrando diferenças de apresentação de um mesmo caractere japonês.



Figura 8.3 – Um caractere japonês e duas variações.

Fontes que admitem sinal diacrítico podem exigir complexos algoritmos de formação. Na figura 8.4, mostramos um exemplo dessas fontes.



Figura 8.4 –Caracteres acentuados.

Na figura 8.5, podemos observar a união de dois caracteres latinos para formar um glifo. Essa união é denominada ligatura.

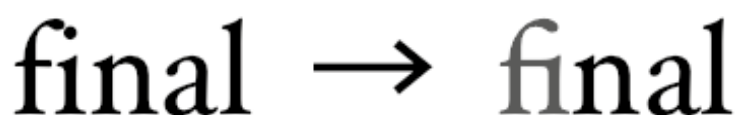


Figura 8.5 – Ligatura latina.

Na figura 8.6, mostramos a ligatura para um caractere árabe.

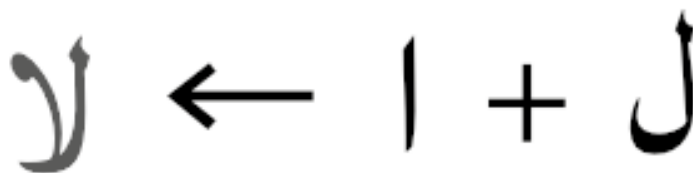


Figura 8.6 – Ligatura árabe.

Todas essas variações e composições de caracteres requerem algoritmos complexos de formação e muitas vezes a necessidade de inclusão de dados adicionais no conjunto da fonte.

Conjuntos de fontes e suas variações estilísticas normalmente são agrupados em famílias de fontes. Em sua

variação mais simples, um conjunto regular de fontes agrupa os caracteres normal, negrito e itálico. Contudo, muitas outras variações são possíveis, sendo as mais comuns a espessura das hastes que compõem as letras, os pesos da fonte, as proporções de suas medidas e a largura. Na figura 8.7, mostramos a variação de peso e de largura de uma letra da família de fontes denominada *Univers*. A largura cresce de cima para baixo, e o peso, da esquerda para a direita.

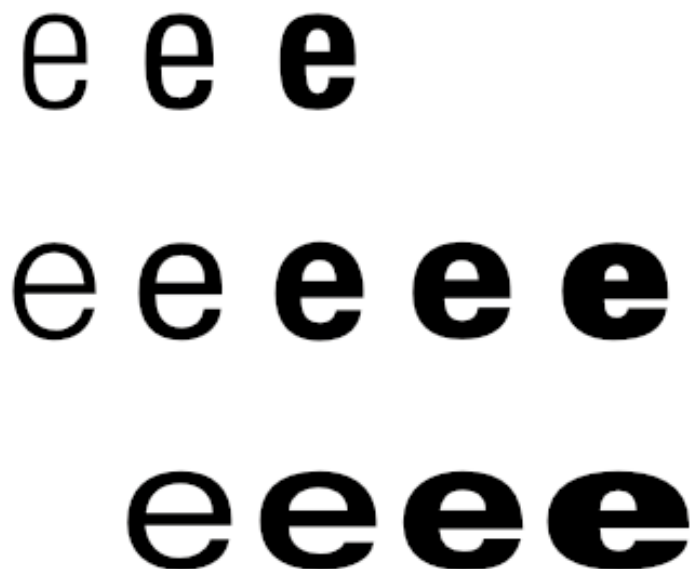


Figura 8.7 – Variação de peso e largura da fonte.

8.1.2 Fontes seguras

Apesar do caos, nem tudo está perdido, e você dispõe de vários mecanismos, entre eles as poderosas CSS, para garantir um mínimo de consistência para sua tão bem cuidada e escolhida tipografia.

O primeiro mecanismo consiste na escolha das chamadas fontes seguras para web. Cada sistema operacional vem com um pequeno conjunto de fontes pré-instaladas. O conjunto das fontes comuns aos sistemas operacionais mais usados é denominado fontes seguras. Na prática, a escolha de tais fontes assegura que a maioria dos usuários visualize a tipografia do nosso site tal como ela foi projetada. Ainda assim, embora raro, há a remota possibilidade de existirem usuários que optaram por desinstalar algumas fontes do seu sistema. A má notícia é que a lista de fontes seguras é ridiculamente pequena. Algumas fontes seguras são mostradas na figura 8.8.

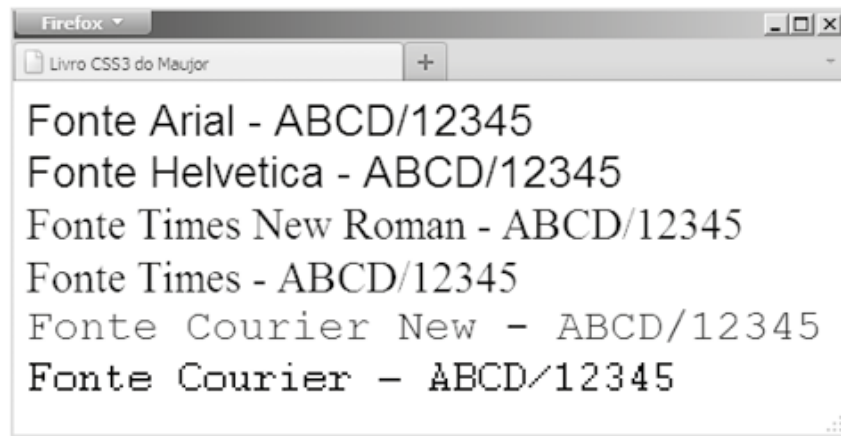


Figura 8.8 – Fontes seguras.

A boa notícia é que as CSS3 reabilitaram a funcionalidade **@font-face**, que tinha sido prevista nas CSS2 e retirada pelas CSS2.1. E os navegadores atuais, bem como as versões mais antigas do Internet Explorer, oferecem um bom suporte para essa funcionalidade. Estudaremos **@font-face** adiante, neste capítulo.

8.2 Propriedade **font-family**

A propriedade **font-family** se destina a definir uma família de fontes para os textos do documento. O valor para essa propriedade é uma string ou uma lista de strings separadas por vírgula. Cada string é o nome da fonte. Observe, a seguir, uma regra CSS típica para declaração de famílias de fontes: `p { font-family: Arial, Verdana, Sans-serif; }`

ou

```
p { font-family: "Times New Roman", Times, Georgia, serif; }
```

É indiferente usar maiúscula ou minúscula nos nomes das fontes. Para dar consistência ao código, recomenda-se escolher uma das duas formas e adotá-la sempre.

Declare uma lista de fontes pertencentes a famílias equivalentes e termine sua lista com uma fonte genérica (ver adiante). É importante a ordem de declaração na lista, pois a preferência será para a primeira fonte declarada; somente a falta dessa fonte no sistema operacional do usuário causará a escolha da segunda na lista, e assim até a fonte genérica, que deve ser declarada por último na lista. Separe o nome das fontes na lista com uma vírgula.

Nomes de fontes que contenham espaços em branco devem ser escritos entre aspas duplas (" ") ou simples (' ').

O autor escolhe sua fonte preferida e a declara nas folhas de estilo, mas em lugar de declarar somente a escolhida, declara uma lista de fontes similares relacionadas como valores da propriedade CSS **font-family**. A leitura, pelo navegador, da regra

CSS mostrada a seguir: `body { font-family: "Lucida Grande", Lucida, Verdana, Sans-serif; }`

é a seguinte: use a fonte Lucida Grande. Se Lucida Grande não estiver instalada no sistema operacional do usuário, use a fonte Lucida. Se ambas não estiverem instaladas, use a fonte Verdana. Finalmente, não encontrando nenhuma das fontes listadas, use a fonte Sans-serif, que é a fonte genérica para as famílias declaradas.

A última opção, aquela que ocorre quando o usuário não possui instalada em seu sistema nenhuma das fontes listadas pelo autor, é a fonte genérica, instalada nativamente no dispositivo do usuário.

O W3C define um conjunto de cinco fontes, as quais denominou de fontes genéricas, que se destinam a servir como uma última opção para o caso de falha nas escolhas do autor. Essas fontes possuem características gerais que procuram preservar tanto quanto possível as características das famílias de fontes escolhidas e estão disponíveis no dispositivo do usuário, independentemente de instalação prévia. Os fabricantes são encorajados a construir seus dispositivos de maneira capaz de preservar as características dos cinco tipos de fontes genéricas. Veremos adiante uma descrição das fontes genéricas.

8.2.1 Família serif

Fontes da família genérica serif, também chamadas de fontes com serifas, têm como característica letras proporcionalmente espaçadas e com pequenos traços de acabamento no terminal de suas hastes, conforme mostrado na figura 8.9. Existe uma grande variação dos caracteres pertencentes a essa família quanto à espessura das hastes, indo de fontes com hastes finas a grossas.



Figura 8.9 – Fonte com serifas.

De acordo com a forma dos traços de acabamento, as serifas podem ser de quatro tipos: Old-Style, Transitional, Modern e Slab, conforme mostrado na figura 8.10.

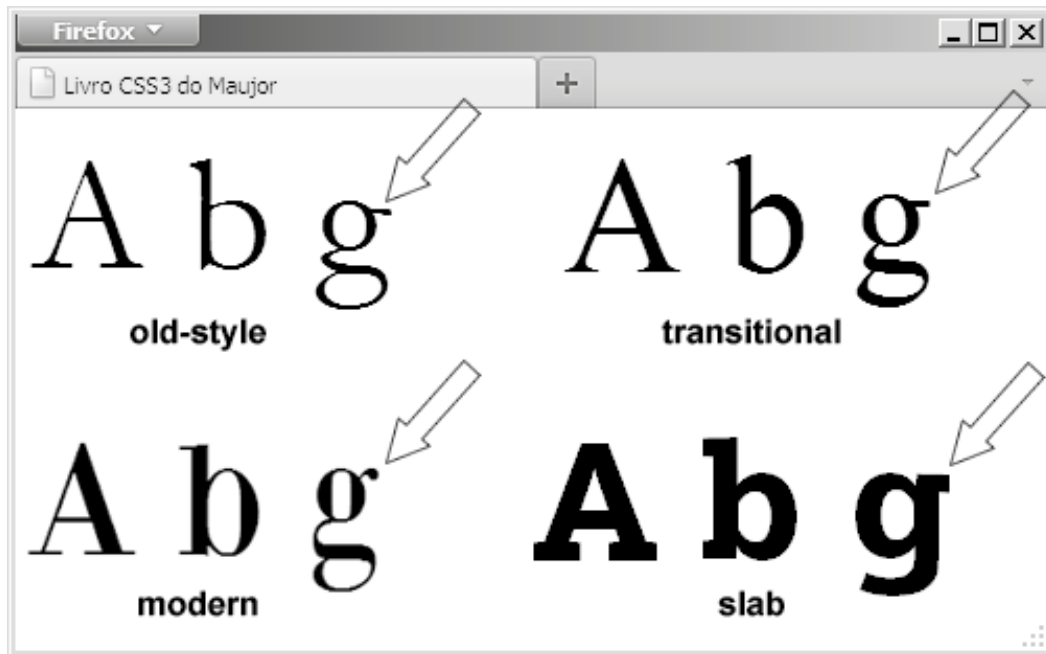


Figura 8.10 – Tipos de serifa.

Alguns exemplos de fontes da família serif são: Times New Roman, Times, Georgia, Garamond, Book Antiqua, Palatino, Bodoni e a fonte genérica Serif.

8.2.2 Família sans-serif

Fontes da família genérica sans-serif, também chamadas de fontes sem serifas, têm como característica letras proporcionalmente espaçadas e sem qualquer ornamento em suas hastes, conforme mostrado na figura 8.11. Ao contrário das fontes com serifas, não há grande variação das letras pertencentes a essa família quando se trata da espessura das hastes.



Figura 8.11 – Fonte sem serifas.

Alguns exemplos de fontes da família serif são: Arial, Verdana, Tahoma, Futura, Gill Sans, Helvetica, Geneva e a fonte genérica Sans-serif.

8.2.3 Família cursive

Fontes da família genérica cursive, também chamadas de fontes cursivas, são aquelas que se assemelham a escrita à mão livre, conforme mostrado na figura 8.12.



Figura 8.12 – Fonte cursiva.

Exemplos de fontes da família cursive: Lucida Handwriting, Monotype Corsiva, Vivaldi, Brush Script, Caflisch Script, Ex Ponto, Snell Roundhand e a fonte genérica Cursive.

8.2.4 Família fantasy

Fontes da família genérica fantasy, também chamadas de fontes fantasia, são aquelas com características decorativas, conforme mostrado na figura 8.13.



Figura 8.13 – Fonte fantasy.

Exemplos de fontes da família fantasy: FairydustyB, Jokerman, Chiller, Alpha Geometrique, Critter, Cottonwood, Fb Reactor, Studz e a fonte genérica Fantasy.

8.2.5 Família monospace

Fontes da família genérica monospace, também chamadas de fontes mono espaçadas, são aquelas em que a largura de todos os caracteres é a mesma. Essa fonte imita a escrita de uma máquina de escrever manual e é normalmente usada para apresentar exemplos de códigos de computador. Na figura 8.14, é mostrado esse tipo de fonte.



Figura 8.14 – Fonte monospace.

Exemplos de fontes da família monospace: Courier, Courier New, Prestige, Everson Mono e a fonte genérica Monospace.

8.3 Propriedade font-style

A propriedade `font-style` permite definir fontes em estilo normal, itálico ou oblíquo. A aplicação dos estilos itálico e oblíquo causa o efeito de inclinar o texto. O estilo normal é o que predomina nos textos deste livro, com as letras na posição vertical, e é também conhecido como estilo romano. Os valores possíveis para essa propriedade são: `normal`, `italic`, `oblique` e `inherit` (valor herdado).

Os estilos itálico e normal são diferentes instâncias de uma fonte, ou seja, cada um deles é um arquivo independente contendo os caracteres da fonte. Se você inspecionar o arquivo das fontes instaladas no sistema operacional do seu computador, vai encontrar os arquivos de fontes com nomes semelhantes aos mostrados a seguir.

- Arial (fonte arial normal)
- Arial Italic (fonte arial itálico)
- Arial Bold (font arial negrito)



O arquivo de fontes no sistema operacional Windows XP está em `C:\WINDOWS\Fonts`.

O efeito itálico é aquele que produz os caracteres no estilo cursivo e é diferente do estilo oblíquo. O efeito oblíquo resulta na simples inclinação para a direita do estilo normal.

Se o arquivo itálico da fonte não estiver presente no computador do usuário, uma regra CSS que declare o efeito itálico não será ignorada, pois o navegador, quando não encontra o estilo itálico, inclina a fonte, fazendo-a igual à fonte oblíqua.

Uma regra CSS típica para essa declaração é mostrada a seguir:

```
p { font-style: italic; }
```

A figura 8.15 esclarece o efeito dos três valores de estilo para a propriedade `font-style` aplicada à fonte *Droid Serif*. Observe, à esquerda, como é bem destacada a diferença de renderização entre itálico e oblíquo. À direita, temos a renderização do estilo itálico igual ao oblíquo causada pela ausência do arquivo de fonte itálica no computador do usuário.

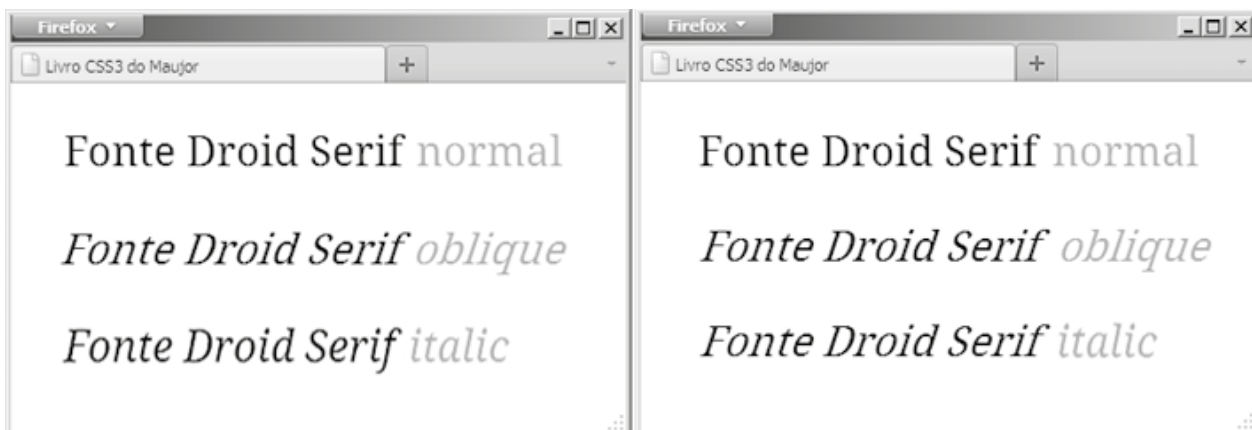


Figura 8.15 – Diferença dos estilos itálico e oblíquo.

8.4 Propriedade `font-variant`

A propriedade `font-variant` se destina a transformar as letras minúsculas de um texto em maiúsculas. Os valores possíveis para essa propriedade são: `normal`, `small-caps` e `inherit` (valor herdado).

É necessário que exista um arquivo específico de fontes do tipo small caps para a aplicação do efeito de forma real. Se não houver um arquivo específico para fontes small caps, o agente de usuário simula o efeito, conservando as letras escritas na marcação em maiúsculas exatamente como elas são e transformando as letras minúsculas na marcação em maiúsculas com altura reduzida (igual à altura das minúsculas).

Convém ressaltar que a simulação, como descrita no parágrafo anterior, resulta em dimensões dos caracteres small caps diferentes daquelas obtidas quando existe um arquivo específico de fonte small caps.



Nas inspeções que realizamos em fontes instaladas nativamente em diferentes sistemas operacionais, nunca encontramos o arquivo de fonte small caps instalado na máquina do usuário. Aparentemente, a estilização com essa propriedade sempre resulta em renderização de uma simulação de small caps. Contudo, encontram-se com facilidade fontes adquiridas de terceiros que vêm com o arquivo específico para small caps.

Uma regra CSS típica para essa declaração é mostrada a seguir:

```
p.small-caps { font-variant: small-caps; }
```

A figura 8.16 esclarece os efeitos da simulação dessa estilização, pelo agente de

usuário, em letras maiúsculas e minúsculas.

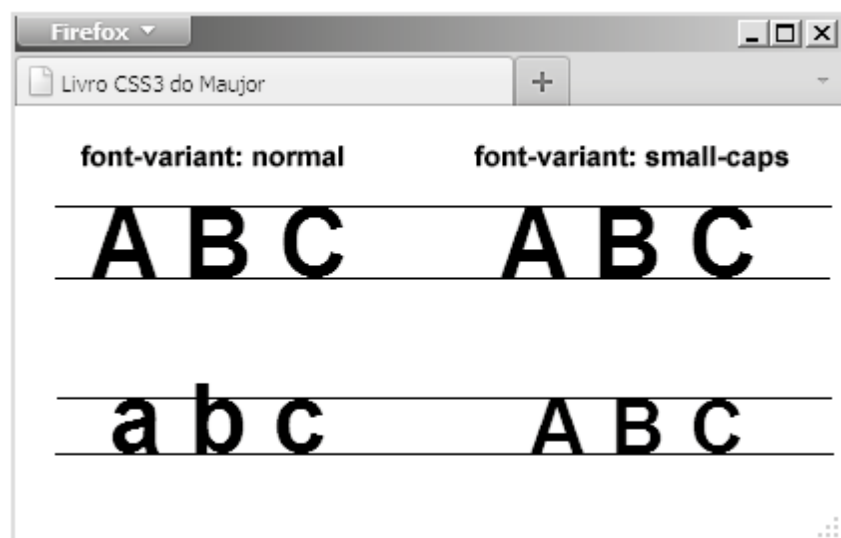


Figura 8.16 – Propriedade *font-variant*.

8.5 Propriedade *font-weight*

A propriedade *font-weight* se destina a definir o peso da fonte. Tecnicamente, peso de uma fonte é a espessura dos traços que formam um caractere em relação à sua altura. O peso da fonte define o menor ou maior grau de destaque (negritude) dos caracteres. Uma fonte em negrito tem sua característica peso maior do que uma fonte normal. Os valores possíveis para essa propriedade são: os números **100**, **200**, **300**, **400**, **500**, **600**, **700**, **800**, **900**, as palavras-chave *normal*, *bold*, *bolder*, *lighter* e *inherit* (valor herdado).

A renderização correta de cada um dos pesos das fontes está condicionada à existência de cada instância de peso ter seu respectivo arquivo instalado na máquina do usuário.

O uso de números se utiliza de uma escala que vai de 100 a 900 em intervalos de 100, resultando nove níveis de peso em valor crescente de destaque. Assim, cada valor na escala representa um peso e define uma tonalidade mais escura que a do valor anterior. O valor 100 é para a tonalidade mais clara, e o valor 900, para a mais escura. O valor *normal* equivale ao peso 400, e o valor *bold* define o que conhecemos como efeito negrito e equivale ao peso 700 na escala numérica de pesos.

Os valores *bolder* e *lighter* dão menos ou mais destaque, respectivamente, em relação ao valor herdado para o peso. O cálculo desses valores se faz segundo a tabela 8.1.

Tabela 8.1 – Valores *lighter* e *bolder*

Valor herdado	<i>lighter</i>	<i>bolder</i>
---------------	----------------	---------------

100	100	400
200	100	400
300	100	400
400	100	700
500	100	700
600	400	900
700	400	900
800	700	900
900	700	900

Observe o exemplo a seguir:

► HTML

```
<div>
  <p class="um">Texto do parágrafo um</p>
  <p class="dois">Texto do parágrafo dois</p>
  <p class="tres">Texto do parágrafo três</p>
</div>
```

► CSS

```
div { font-weight: 300; }
  p.um { font-weight: inherit; }
  p.dois { font-weight: lighter; }
  p.tres { font-weight: bolder; }
```

O parágrafo **p.dois** terá um peso de fonte igual a 100 (menos destaque que 300), e o parágrafo **p.tres**, 400 (mais destaque que 300). O parágrafo **p.um** herdará o peso 300 de seu elemento-pai **div**.

As fontes têm suas características intrínsecas e seus criadores adotam nomes descritivos para diferenciá-las. Uma dessas características é o peso da fonte. Ocorre que não existe uma padronização de nomes para identificar o peso da fonte. Para exemplificar a falta de padronização, tomemos como exemplo as seguintes fontes: *Rattlesnake Regular* e *Prawn Book*. Ambas são a versão de tonalidade mais clara da fonte, mas o criador da primeira fonte usou o sobrenome *Regular*, e o da segunda, o sobrenome *Book*, para nomear a versão mais clara da sua fonte.

Exemplificamos com duas fontes. Imagine as dificuldades quando pensamos nas centenas de fontes disponíveis. Para resolver essa inconsistência, as CSS

estabeleceram a escala numérica para valores da propriedade **font-weight**, conforme mostrada anteriormente, que se destina a definir as nove instâncias de diferentes nomes para cada fonte.

Observe, no exemplo que se segue, a aplicação desse conceito para uma fonte hipotética de nome *Panda*:

Peso	Descrição
100	Panda light
200	Panda light bold
300	Panda regular light
400	Panda regular
500	Panda normal light
600	Panda normal
700	Panda normal bold
800	Panda heavy
900	Panda extra heavy

As especificações para as CSS2.1 estabelecem critérios de correspondência com a escala de peso que dizem o seguinte: 1. Para fontes cujo nome contenha as palavras **Book**, **Regular**, **Roman**, **Normal** e, em alguns casos, **Medium**, é atribuído o peso 400.

2. Para fontes com uma instância do nome contendo uma das palavras **Book**, **Regular**, **Roman**, **Normal** e outra instância contendo a palavra **Medium**, é atribuído o peso 400 para a primeira e 500 para a segunda instância.

3. Para fontes cujo nome contenha as palavras **Bold**, é atribuído o peso 700.

4. Para fontes que não contenham nove instâncias, a atribuição de peso se faz:

- a) não havendo a instância de peso 500, ela será igual à de peso 400;
- b) não havendo as instâncias 600, 700, 800 ou 900, elas serão iguais à de peso maior. Não existindo a de peso maior, serão iguais à de peso menor;
- c) não havendo as instâncias 300, 200 ou 100, elas serão iguais à de peso menor. Não existindo a de peso menor, serão iguais à de peso maior.

O exemplo constante da tabela 8.2 mostra a distribuição dos pesos para uma fonte com quatro instâncias. Ao lado do peso foi anotado o critério para atribuí-lo, fazendo referência à numeração constante no parágrafo anterior.

Tabela 8.2 – Mapeamento de peso de fontes

Nome da fonte	Instância	Peso complementado
Rattlesnake Regular	400 (critério 2)	100, 200, 300 (critério 4.c)
Rattlesnake Medium	500 (critério 2)	
Rattlesnake Bold	700 (critério 2)	600 (critério 4.b)
Rattlesnake Heavy	800	900 (critério 4.b)

Adicionalmente, as especificações para as CSS3 sugerem uma tabela de mapeamento dos pesos com nomes de fontes, conforme mostrada a seguir:

Peso	Descrição
100	Thin
200	Extra Light (Ultra Light)
300	Light
400	Normal
500	Medium
600	Semi Bold (Demi Bold)
700	Bold
800	Extra Bold (Ultra Bold)
900	Black (Heavy)

Essa é a teoria explicando como funcionam os nove níveis de peso para efeitos de estilização. A má notícia é que os sistemas operacionais, em geral, só têm instalados os arquivos (instâncias) para o valor **normal** e o valor **bold**, permitindo estilizar os textos em tonalidade normal (que é o valor padrão) ou em negrito (uma tonalidade mais escura). Por esse motivo, os valores **bolder** e **lighter** que definem uma tonalidade mais escura ou mais clara, respectivamente, tomando como base o peso da fonte do elemento ancestral, simplesmente não funcionam.

As únicas regras CSS que funcionam para essa declaração são aquelas cujo valor declarado é **bold** ou **normal**. Você pode declarar o peso da fonte como previsto, mas pesos de 600 a 900 resultam em **bold** de 100 a 500 em **normal**. Exceto, é claro, se mais instâncias de peso de fontes, além de **normal** e **bold**, forem instaladas no computador do usuário: `p { font-weight: bold; } /* ou peso de 600 a 900 */`

e

```
p { font-weight: normal; } /* ou peso de 100 a 500 */
```

A figura 8.17 mostra um exemplo real da renderização de diferentes pesos de fontes em um sistema operacional no qual foram instaladas as instâncias dos pesos 100, 300, 400, 700 e 900 para a fonte Lato Sans.

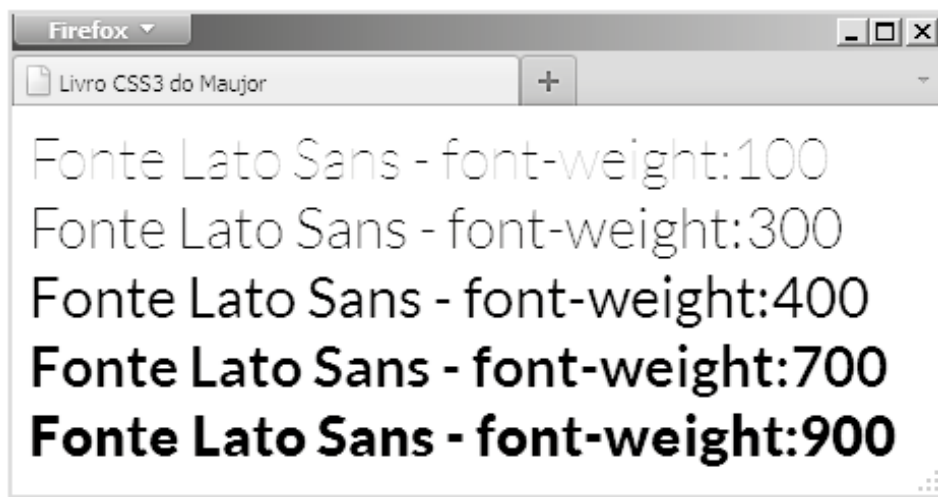


Figura 8.17 – Diferentes pesos de fontes.

8.6 Propriedade fontsize

A propriedade `fontsize` destina-se a definir o tamanho das fontes. Os valores possíveis são: um valor absoluto, um valor relativo, uma medida CSS de comprimento, um valor em porcentagem e `inherit` (valor herdado).

O tamanho de uma fonte é um valor calculado em função de um conceito tipográfico denominado *em square*, próprio para cada fonte. Não existe um valor fixo extraído das medidas físicas da fonte para determinar seu tamanho de fonte. O efeito de `fontsize` é o de fornecer uma medida que servirá de fator para cálculo do *em square* da fonte. Por isso não há sentido em afirmar, por exemplo, que uma fonte de tamanho igual a `10px` está contida em linhas paralelas distanciadas de `10px`, ou coisa semelhante. A questão é mais complexa e envolve características próprias das fontes, outras que não dimensões físicas, somente.

Os valores possíveis para declarar o tamanho de fonte com um valor absoluto são: `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large` e `xx-large`.

Os agentes de usuário devem tomar como base para renderizar os valores absolutos a tabela 8.3, na qual estão relacionados os valores absolutos e seu equivalente atributo `size` do elemento `font`.

Tabela 8.3 – Tamanhos absolutos para fontes CSS2.1

Valores CSS absolutos	xx-small	x-small	small	medium	large	x-large	xx-large	
-----------------------	----------	---------	-------	--------	-------	---------	----------	--

font size HTML	1		2	3	4	5	6	7
----------------	---	--	---	---	---	---	---	---

Lembrar que o uso do elemento `font` e seu atributo `size` definindo um tamanho de fonte tem a seguinte sintaxe: `<font size="2">texto</font>`, à maneira ultrapassada de marcar HTML.

A tabela mostra, por exemplo, que:

```
p {fontsize: x-large; }
```

deverá ser igual em tamanho de fonte a

```
<p><font size="5">texto</font></p>
```

As especificações CSS 2.1 sugerem a tabela anterior como uma orientação. Os agentes de usuário poderão criar suas tabelas próprias, considerando o valor `medium` como o preferido do usuário e, a partir dele, escalonar os sete tamanhos, sem atingir um tamanho menor do que 9px. As especificações CSS 2.1 sugerem ainda um fator de escalonamento entre 1.2 e 1.3.

As CSS3 acrescentaram outras referências de equivalência para tamanhos dos valores absolutos para fontes e sugerem a tabela 8.4.

Tabela 8.4 – Tamanhos absolutos para fontes CSS3

Valores CSS absolutos	xx-small	x-small	small	medium	large	x-large	xx-large	
fator de escala	3/5	3/4	8/9	1	6/5	3/2	2/1	3/1
cabeçalhos XHTML	h6		h5	h4	h3	h2	h1	
XHTML font size	1		2	3	4	5	6	7

Os valores possíveis para declarar o tamanho de fonte com um valor relativo são: `smaller` (menor) e `larger` (maior).

Esses valores definem o tamanho da fonte tomando como base o tamanho da fonte do elemento-pai. Por exemplo: se dentro de uma `div` para a qual foi definido um tamanho de fonte `x-large` tivermos um parágrafo com tamanho de fonte definido em `smaller`, o resultado será `large` (observe na tabela anterior que menor [`smaller`] que `x-large` é `large`). Ao contrário, se a definição for `larger`, o resultado será `xx-larger`.

Se o elemento-pai tiver sua fonte definida por um dos valores extremos da tabela (`xx-small` e `xx-large`) ou se a definição não for por um valor absoluto (por exemplo: uma medida CSS), o agente de usuário é livre para extrapolar no cálculo dos valores `smaller` (menor) e `larger` (maior).

Porcentagem e medidas CSS são as outras maneiras de definir o tamanho de

fontes, conforme sintaxe mostrada a seguir: `p { font-size: 80%; }`

```
p { font-size: 12px; }  
p { font-size: 1.8em; }  
p { font-size: 5mm; }  
p { font-size: 14pt; }
```

O valor inicial para a propriedade `font-size` é `medium`, que equivale a `16px`. Esse valor inicial é consistente entre diferentes navegadores para fontes inseridas diretamente nos elementos `body`, `p`, `li` e alguns outros. Elementos tais como `h1` até `h6`, `big`, `small` e outros têm um valor inicial para `font-size` que varia de acordo com o navegador.

O cálculo do tamanho de fonte declarada em porcentagem é feito tomando-se como referência o tamanho de fonte do elemento-pai. Observe os exemplos a seguir:

► HTML

```
<body>  
<ul>  
<li>Item um</li>  
<li>Item <span>dois</span></li>  
</ul>  
</body>
```

Exemplo 1:

► CSS

```
li { font-size: 150%; }  
span { font-size: 50%; }
```

O valor de `font-size` para o elemento `li` é 150% de `16px` = `24px`, e para o elemento `span` é 50% de `24px` = `12px`.

Exemplo 2:

► CSS

```
ul { font-size: 80%; }  
li { font-size: 70%; }  
span { font-size: 180%; }
```

O valor de `font-size` para o elemento `ul` é 80% de `16px` = `13px`, para o elemento `li` é 70% de `13px` = `9px`, e para o elemento `span` é 180% de `9px` = `16px`.

O cálculo do tamanho de fonte declarada em unidade `em` é feito tomando-se como referência o tamanho de fonte do elemento-pai. O valor `em` é um número decimal que representa o fator pelo qual deve ser multiplicado o tamanho de fonte do elemento-pai. Observe os exemplos a seguir: ► HTML

```
<body>  
<ul>
```

```
<li>Item um</li>
<li>Item <span>dois</span></li>
</ul>
</body>
```

Exemplo 1:

► CSS

```
li { font-size: 150%; }
span { font-size: 0.8em; }
```

O valor de `font-size` para o elemento `li` é 150% de 16px = 24px, e para o elemento `span` é $0,8 * 24px = 19px$.

Exemplo 2:

► CSS

```
ul { font-size: 20px; }
li { font-size: 1.5em; }
span { font-size: 0.5em; }
```

O valor de `font-size` para o elemento `li` é $1,5 * 20px = 30px$, e para o elemento `span` é $0,5 * 30px = 15px$.

Assim, podemos concluir que os mecanismos de aplicação do tamanho de fonte com uso de porcentagem e de unidade `em` funcionam exatamente da mesma maneira. As declarações mostradas a seguir se equivalem: `font-size: 150%;` = `font-size: 1.5em;`

`font-size: 70%;` = `font-size: 0.7;`
`font-size: 62.5%;` = `font-size: 0.625;`

Então, pode-se perguntar: se são iguais, qual a necessidade de termos esses dois valores? Na verdade, o mecanismo é igual para aplicação de tamanho de fonte. Para outras propriedades CSS, como as propriedades `width`, `height`, `line-height` etc., a referência para cálculo de porcentagem não é o tamanho de fonte do elemento-pai, mas, sim, o valor da propriedade para o elemento-pai.

Por exemplo: `width: 80%` não é igual a `width: 0.8em;`.

8.7 Propriedade `font-stretch`

A propriedade `font-stretch` se destina a declarar o grau de condensação ou expansão dos caracteres de uma fonte. Os valores possíveis para essa propriedade são palavras-chave que determinam quatro fatores de condensação e quatro fatores de expansão em relação ao caractere normal da fonte. Essas palavras-chave são: `normal`, `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded`, `ultra-expanded`, `inherit`. Observe, a seguir,

uma declaração CSS típica para essa propriedade: `p { font-stretch: extra-condensed; }`

A figura 8.18 foi retirada das especificações para as CSS3 no site do W3C e mostram o efeito dessa declaração.



Figura 8.18 – Propriedade *font-stretch*.



Essa propriedade foi criada nas CSS2, retirada pelas CSS2.1 e recriada nas CSS3. Atualmente, nenhum navegador suporta essa propriedade.

8.8 Propriedade *fontsize-adjust*

A propriedade *fontsize-adjust* foi criada pela especificação para as CSS3 e se destina a preservar a legibilidade da fonte quando a primeira família de fontes declarada com uso da propriedade *font-family* não está disponível. Os valores possíveis para essa propriedade são: um número decimal ou as palavras-chave *none* e *inherit* (valor herdado). Observe, a seguir, uma declaração CSS típica para essa propriedade: `body { fontsize-adjust: 0.47; }`

Para entender o funcionamento dessa propriedade é necessário conhecer uma propriedade tipográfica das fontes denominada *valor de aspecto* (aspect value). O valor de aspecto de uma fonte é uma grandeza que expressa a maior ou menor legibilidade da fonte. Calcula-se o valor de aspecto de uma fonte dividindo a altura da letra *x* minúscula (*x-height*) pelo tamanho da fonte. O resultado é um número decimal em geral próximo a 0,5 para a maioria das fontes.

A figura 8.19 apresenta três famílias de fontes diferentes. Na parte superior, a renderização de uma letra de cada família, e na parte inferior, um texto com tamanho de fonte igual a 12px para as três famílias.

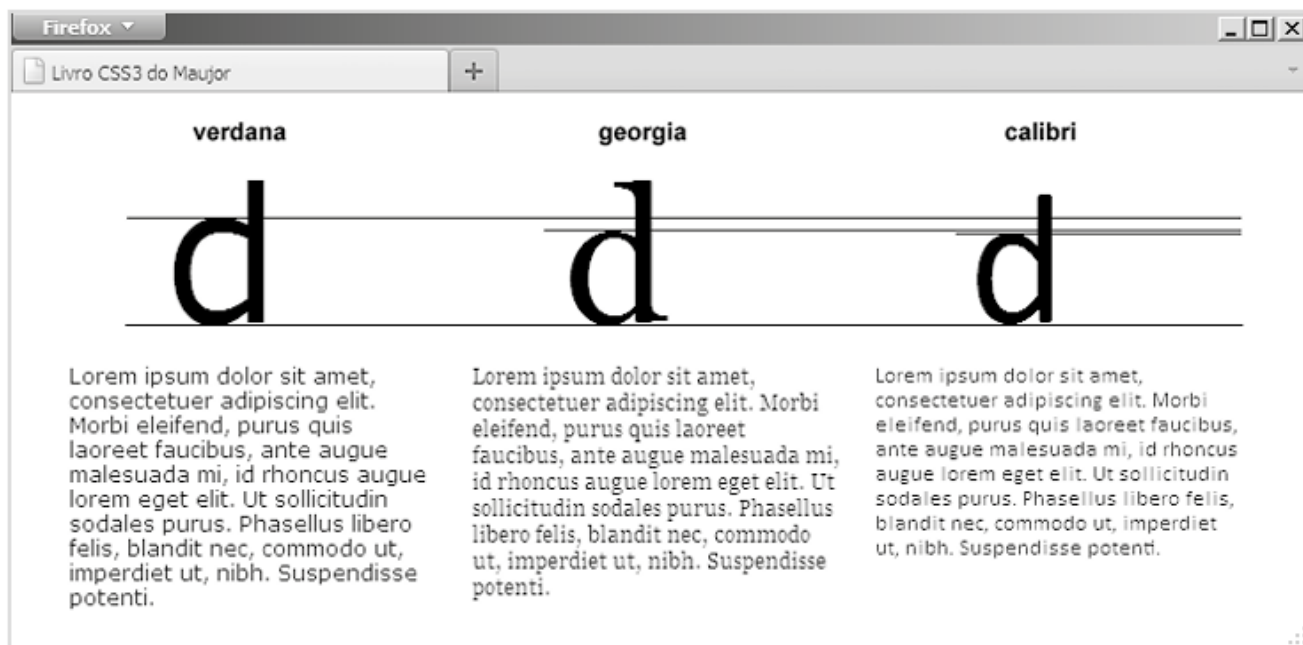


Figura 8.19 – Valor de aspecto das fontes.

Note a dependência da legibilidade das fontes com a família. Para os mesmos 12px de tamanho de fonte, a legibilidade diminui da *verdana* para a *georgia* e daí para a *calibri*.

As regras CSS para os três textos são:

```
p.um { font: 12px verdana; }
p.dois { font: 12px georgia; }
p.tres { font: 12px calibri; }
```

A propriedade `font-size-adjust` destina-se a padronizar a legibilidade entre as famílias de fontes definidas pela propriedade `font-family`. Observe, na figura 8.20, a renderização dos mesmos textos mostrados na figura 8.19, mas agora com aplicação da propriedade `font-size-adjust` para as famílias de fontes *georgia* e *calibri*, conforme as regras de estilos mostradas a seguir: `p.um { font: 12px verdana; }`

```
p.dois { font: 12px georgia; font-size-adjust: 0.545; }
p.tres { font: 12px calibri; font-size-adjust: 0.545; }
```

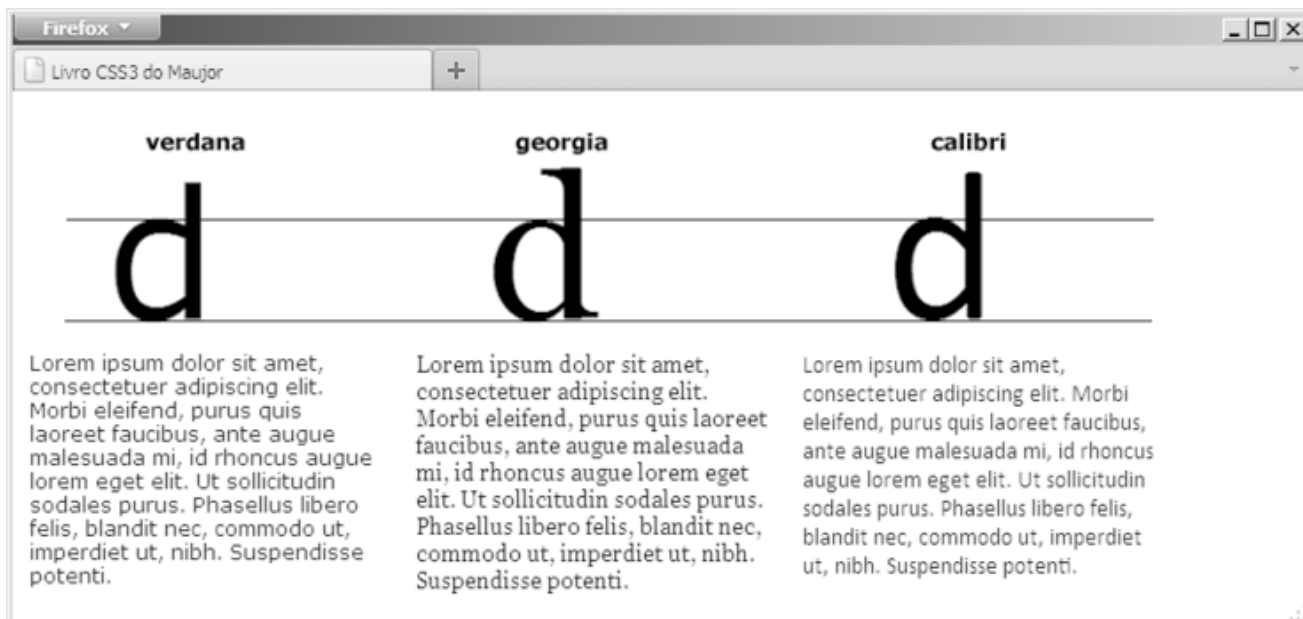



Figura 8.20 – Declaração *font-size-adjust*.

Mas de onde veio aquele número mágico *0.545*, declarado como valor para a propriedade **font-size-adjust**, nas regras CSS mostradas, que fez com que a legibilidade fosse a mesma para as três fontes?

O número mágico é o *valor de aspecto* da primeira fonte declarada (*Verdana*), cuja legibilidade se pretende ser preservada para as fontes *Georgia* e *Calibri*. Em uma regra CSS para a família de fontes de um documento web típico, declaramos uma lista de fontes opcionais, caso a fonte escolhida não esteja disponível, como no exemplo a seguir: `font { 12px Verdana, Georgia, Calibri, sans-serif; }`



Se você estranhou o fato de termos usado, no exemplo, uma declaração para a família de fontes com mistura de fontes sem serifas (*Verdana* e *Calibri*) com uma fonte com serifas (*Georgia*), alerto que a finalidade do exemplo foi apenas de ressaltar a legibilidade das fontes. Na prática, procure manter o mesmo tipo de fonte na lista de famílias.

O documento foi projetado para acomodar a fonte *Verdana* com tamanho **12px**. Caso ela não esteja disponível, a opção será a fonte *Georgia*. Note que essa fonte possui legibilidade e dimensões menores que aquela, isto é, a fonte *Georgia* de **12px** é menos legível e menor do que a fonte *Verdana* de **12px**. Antes das CSS3, esse era um problema insolúvel (ou de solução complicadíssima). Nas CSS3, a propriedade **font-size-adjust** resolveu o problema. Basta declarar, para as fontes alternativas àquela adotada como padrão para o projeto do documento web, o valor dessa propriedade como sendo igual ao *valor de aspecto* da fonte padrão. No nosso exemplo, o *valor de aspecto* da fonte *Verdana* é o número mágico *0.545*.

As especificações para as CSS3 sugerem um método prático para determinação do *valor de aspecto* de uma fonte que se baseia em tentativas, contudo existem ferramentas online que fornecem esse valor. Uma dessas ferramentas é a do site *FontDeck*, hospedada em <http://fontdeck.com/support/fontsizeadjust> (<http://kwz.me/gG>).

8.9 Propriedade font

A propriedade **font** é uma abreviatura para declarar todas as propriedades de fontes estudadas anteriormente. Você declara valores para todas as propriedades CSS para fontes usando a propriedade abreviada **font**, conforme a seguinte sintaxe: `seletor { font: font-style font-variant font-weight fontsize/line-height font-family; }`

Suponha que, na folha de estilo, definimos a fonte para uma área do nosso site com a seguinte regra CSS: `div.um {`

```
font-style: italic;
font-variant: small-caps;
font-weight: bold;
font-size: 12px;
line-height: 120%;
font-family: verdana sans-serif;
```

```
}
```

Podemos (e devemos) abreviar a regra para:

```
div.um { font: italic small-caps bold 12px/150% verdana sans-serif; }
```

Não é obrigatório declarar todos os valores. Valores não declarados são considerados como se fossem o valor inicial da propriedade.



Para que a declaração abreviada seja válida, ela deverá conter no mínimo os valores **font-size** e **font-family**.

Algumas propriedades CSS admitem uma forma abreviada, como a mostrada para a propriedade **font**. É de boa prática adotar a forma abreviada sempre que for permitido e possível. Isso facilita a manutenção, diminui o tamanho físico e o tempo de carga da folha de estilo.

A forma abreviada para fontes admite ainda o uso de seis palavras-chave para definir fontes. São elas: **caption**, **icon**, **menu**, **message-box**, **small-caption** e **status-bar**. Esses valores formatam as fontes com base no sistema operacional do usuário. Por exemplo: `p { font: menu; }`

estiliza os parágrafos com uma fonte igual à fonte usada nos menus das interfaces instaladas no sistema operacional do usuário. Observe que cada palavra-chave fornece uma referência à fonte usada em diferentes lugares das interfaces do sistema operacional.

8.10 @font-face

A regra **@font-face** permite aos autores CSS usar fontes hospedadas em um servidor remoto. Essa funcionalidade resolve o problema relacionado à utilização de fontes

dependentes de estarem instaladas no sistema operacional do usuário. Em resumo, hospeda-se um arquivo de fonte em um servidor remoto e declara-se em regra CSS o endereço e o nome da fonte hospedada. Ao ser renderizada a página, a fonte será buscada pelo navegador, não no sistema operacional do usuário, mas no endereço onde foi hospedada.

A sintaxe geral para declarar uma fonte remota e usá-la, na folha de estilo, é mostrada a seguir:

```
@font-face {  
  font-family: LeagueGothicRegular;  
  src: url('league_gothic-webfont.ttf');  
  
  }  
  
p { font-family: LeagueGothicRegular, serif; }
```

Nesse exemplo, hospedamos o arquivo da fonte **LeagueGothicRegular** denominado *league_gothic-webfont.ttf* no mesmo diretório em que está hospedada a folha de estilo, e com uso da regra **@font-face** informamos, na folha de estilo, o nome e o endereço de hospedagem da fonte.

A seguir, na folha de estilo, podemos usar a declaração da fonte em qualquer regra CSS. Não se esqueça de declarar uma fonte alternativa, para o caso de falha no carregamento da fonte pelo navegador do usuário.

Tanto o nome da fonte como o nome do arquivo são de livre escolha do autor, contudo é recomendável que se escolham os nomes iguais ao nome comercial da fonte, para facilitar futuras referências e manutenção.

Infelizmente (ou felizmente), existem vários formatos de arquivos de fontes, e os navegadores não os suportam de forma consistente. O quadro mostrado na figura 8.21 foi transcrito do site <http://fontspring.com>, e mostra o suporte pelos navegadores para os diferentes tipos de arquivos de fontes.

Browser	TrueType	WOFF	EOT	SVG
IE5-8			SIM	
IE9	PARCIAL	SIM	SIM	
Firefox 3.5	SIM			
Firefox 3.6+	SIM	SIM		
Safari 3.1+	SIM			SIM
Chrome 6+	SIM	SIM		SIM
Opera 10+	SIM	SIM		SIM
iOS < 4.2				SIM
iOS 4.2+	SIM			SIM

Figura 8.21 – Suporte para fontes.

São quatro os tipos de arquivos de fontes, a saber: `.ttf`, `.woff`, `.eot`, `.svg`. O formato `.eot` é proprietário da Microsoft, suportado apenas pelos navegadores Internet Explorer. O formato `.woff` (Web Open Font Format) é um formato aberto criado pela Mozilla, e o mais recente dos formatos. Esse formato tem todas as características para ser, no futuro, o formato único da web universal, tão sonhada por nós. Até lá, teremos que desenvolver nossos códigos para acomodar as inconsistências dos dispositivos de usuário. Assim, a regra mostrada anteriormente, contendo duas linhas de declaração CSS, deverá ser escrita conforme mostrado a seguir: `@font-face {`

```
font-family: LeagueGothicRegular;
src: local('League'), url('league_gothic-webfont.eot'); /* IE9 Modo de
compatibilidade */
src: url('league_gothic-webfont.eot?#iefix') format('embedded-
opentype'), /* IE6-IE8 */
url('league_gothic-webfont.woff') format('woff'), /* Navegadores
modernos */
url('league_gothic-webfont.ttf') format('truetype'), /* Safari, Android,
iOS */
url('league_gothic-webfont.svg#svgFontName') format('svg'); /* iOS
antigos */
font-weight: normal;
font-style: normal;

}
```

A declaração `local('...')` faz com que o navegador use uma fonte local, instalada na máquina do usuário, em lugar de importar a fonte definida pelo autor. Na prática, na maioria dos casos, é muito pouco provável que o usuário tenha instalada em sua máquina uma fonte que o autor tenha decidido hospedar e importar, contudo convém saber que a opção para uso de fonte local existe e pode ser usada.

O formato `.eot` destina-se ao Internet Explorer. As versões IE6 até IE8 precisam que seja passado um parâmetro falso no URL (`?#iefix`), caso contrário não reconhecem o endereço da fonte. Para o IE9 em modo de compatibilidade, o parâmetro falso deve ser omitido. Daí a necessidade de duas declarações `src`. Observe que a segunda declaração é uma lista de URL separados por vírgula. Os demais URL se destinam aos navegadores, conforme comentários no código.

A declaração do formato da fonte, com uso de `format('...')`, logo após o URL, faz com que o navegador não precise baixar todas as fontes para reconhecê-las e usar a apropriada. O navegador somente baixará a fonte cujo formato declarado ele suporte.

8.10.1 Obtendo fontes

Você obtém fontes em sites especializados baixando o arquivo de fontes para seu HD. As fontes disponíveis online podem ser gratuitas ou compradas, mas é importante que o autor se informe sobre direitos autorais de uso das fontes que pretende usar.

Para fins de ilustrar a obtenção de fontes na internet, vamos tomar como exemplo o site <http://www.fontsquirrel.com/>, cuja home page, em outubro de 2013, é mostrada, em parte, na figura 8.22.



Figura 8.22 – Site *fontsquirrel*.

Nos links da navegação HOME, POPULAR e RECENT, bem como nas páginas

resultantes de filtragem por tipos das inúmeras fontes disponíveis no site, encontramos um link para download de fontes nos formatos .ttf ou .otf.

Supondo que você fez o download de uma fonte e obteve o seu arquivo zipado em um dos formatos citados, descomprima o arquivo baixado e salve no seu HD.

O site oferece uma ferramenta online para gerar os demais tipos de arquivo a serem declarados na regra @font-face. A citada ferramenta encontra-se no link WEBFONT GENERATOR, e a parte da página contendo a interface da ferramenta é mostrada na figura 8.23.

Clicando no link +AddFont, abre-se uma caixa de diálogo para você escolher o arquivo da fonte que você baixou e gravou no HD. Escolhido o arquivo, a ferramenta gera os demais arquivos nos diversos formatos, mas não libera para download até que se marque o checkbox Agreement, declarando que as fontes são livres para uso na web. Marcado o checkbox, aparece um botão para download do kit da fonte.

The image shows the Webfont Generator website interface. At the top is a navigation bar with links: HOME, FIND FONTS, POPULAR, RECENT, WEBFONT GENERATOR (highlighted), and FAQ. There is also a FONT SEARCH input field. The main heading is 'WEBFONT GENERATOR'. Below it, a usage instruction says: 'Usage: Click the "Add Fonts" button, check the agreement and download your fonts. If you need more fine-grain control, choose the Expert option.' The main form has an 'Add Font' button. Below that, a table shows the font 'Henny Penny Regular' with details: OTF, 231 glyphs, 44 KB, 62%, and a close button. There are three radio button options: BASIC (Straight conversion with minimal processing), OPTIMAL (Recommended settings for performance and speed, which is selected), and EXPERT... (You decide how best to optimize your fonts). An 'Agreement' section has a checked checkbox with the text: 'Yes, the fonts I'm uploading are legally eligible for web embedding. Font Squirrel offers this service in good faith. Please honor the EULAs of your fonts.' Below the agreement is a 'DOWNLOAD YOUR KIT' button. At the bottom, there is a 'Donate' button and a message: 'Find this free service valuable and want to see it grow? Consider donating. Thank you!'. On the right side, there is a sidebar with a 'DESIGN your WEBSITE free online editor' advertisement and a 'FIND FONTS' section with 'CLASSIFICATIONS' (Blackletter, Calligraphic, Comic, Dingbat, Display, Grunge, Handdrawn, Initials) and 'TAGS' (Text, Cas, Har, Gru, Cal, Content, Ret).

Figura 8.23 – Ferramenta para gerar arquivo de fonte.

Feito o download do kit, obtemos um arquivo zipado contendo todos os formatos de fonte necessários para a regra @font-face, bem como a regra de estilo pronta para uso, além de outros arquivos para uso com o framework grid. Podemos usar a ferramenta para gerar os arquivos a partir de um arquivo de fonte obtido em outros sites ou mesmo arquivos enviados pelo designer.

8.10.2 API fontes do Google

O Google criou uma API com a finalidade de facilitar o uso de fontes remotas. A home page da API encontra-se em <http://code.google.com/apis/webfonts/>, e nela existem links para tutoriais explicando o uso da API e suas funcionalidades. As linhas gerais de funcionamento e uso da API serão mostradas a seguir, e caso você queira aprofundar-se no assunto, consulte a API no link indicado anteriormente.

Não há necessidade da regra `@font-face` inserida na folha de estilo criada pelo autor. Para usar uma fonte remota, basta criar um link na seção `head` do documento, conforme o exemplo mostrado a seguir: `<link rel="stylesheet" type="text/css" href="http://fonts.googleapis.com/css?family=Tangerine">` Trata-se de um link para uma folha de estilo externa hospedada nos servidores do Google apontando para o endereço <http://fonts.googleapis.com/css>, que recebe um parâmetro com o nome da fonte que se pretende usar. No nosso exemplo, escolhemos a fonte denominada *Tangerine*, e assim passamos no URL o parâmetro `?family=Tangerine`.

É somente isso. Podemos declarar na nossa folha de estilo do site ou da página a fonte *Tangerine*, sem necessidade de qualquer outra declaração ou regra. Quer que os cabeçalhos nível 1 do site usem a fonte *Tangerine* com tamanho **28px**? Basta declarar na folha de estilo que você desenvolveu para o site o seguinte: `h1 {`

```
font-family: 28px Tangerine, serif;
```

```
}
```

É óbvio que é necessário que a fonte escolhida pelo autor esteja hospedada nos servidores do Google. A relação das fontes disponíveis encontra-se no site da API das fontes no endereço <http://www.google.com/webfonts#ChoosePlace:select> (<http://kwz.me/gl>).

Se uma das fontes disponíveis estiver de acordo com nossas necessidades, podemos usar a API para declarar fontes externas no nosso site.

Na página das fontes disponíveis, é apresentada uma linha de texto mostrando o aspecto da fonte e seu respectivo nome. Poderemos simplesmente anotar o nome da fonte e usá-la como foi mostrado anteriormente ou, se preferirmos, podemos baixar a fonte, hospedá-la no servidor do site e usar como mostrado em [8.10], com aplicação da regra `@font-face`.

Há, ainda, na página das fontes, links para informações adicionais sobre a fonte e seu uso. Para mais detalhes, visite a página no link citado anteriormente.

Podemos requisitar mais de uma fonte da API conforme sintaxe para o URL do link na seção `head` do documento, como mostrado a seguir: [http://fonts.googleapis.com/css?](http://fonts.googleapis.com/css?family=Tangerine|Inconsolata|Droid+Sans)

family=Tangerine|Inconsolata|Droid+Sans Nesse exemplo, pretendemos usar em nosso projeto as fontes *Tangerine*, *Inconsolata* e *Droid Sans*. Note que fontes cujo

nome é composto de duas palavras como *Droid Sans* devem ser escritas no parâmetro do URL com o sinal + (mais) no lugar do espaço que separa as palavras. A separação entre os nomes das fontes deve ser feita com o sinal | (barra vertical).

Exemplo de regras de estilo usando as três fontes mostradas:

```
p.um { font-family: Tangerine, serif; }  
p.dois { font-family: Inconsolata, sans-serif; }  
p.tres { font-family: 'Droid Sans', sans-serif; }
```

Podemos acrescentar vários outros parâmetros no URL que requisita as fontes. A título de ilustração, mostrarei o parâmetro que permite requisitar a coleção de fontes em suas gradações de negrito.

Sabemos que a propriedade `font-weight` das CSS admite, além dos valores `normal`, `bold`, `bolder`, `lighter` e `inherit`, os valores numéricos 100, 200, ..., 800, 900, sendo que o valor 400 corresponde à fonte `normal`, e o valor 700, à fonte `bold`, como estamos acostumados a usar em nossas regras CSS.

Quando declaramos um valor numérico diferente de 400 ou 700 para a propriedade `font-weight`, ele será considerado pelo navegador como `normal` ou `bold`, pois os arquivos de fontes normalmente têm um peso de fonte igual a `normal` (ou 400), sendo, em geral, o `bold` (ou 700) criado por mecanismos internos do agente de usuário. Mas não existe um mecanismo interno para criação de pesos de fonte diferentes do `bold` (ou 700).

Arquivos de fontes completos compreendem diversas variações da fonte, não somente de peso, mas também para itálico. Cada uma dessas variações é um arquivo separado.

A API de fontes do Google permite que você utilize, por exemplo, todos os nove arquivos com os diferentes pesos de fonte, bastando acrescentar mais um parâmetro no URL da requisição.

Suponha que você pretenda usar a fonte denominada Lato e declarar regras CSS que usem peso 100, 400, 600, 700 e 900 em seu projeto. O URL do link de requisição tem a sintaxe conforme mostrado a seguir: `http://fonts.googleapis.com/css?family=Lato:100,400,600,700,900`

Basta colocar : (dois-pontos) após o nome da fonte e, a seguir, listar os pesos da fonte separados por vírgula. Na folha de estilo, podemos usar as seguintes regras CSS para estilizar parágrafos com peso de fonte diferente: `p.um { font-weight: 100; }`

```
p.dois { font-weight: 400; }  
p.tres { font-weight: 600; }  
p.quatro { font-weight: 700; }  
p.cinco { font-weight: 900; }
```


Para conhecer outros parâmetros e mais funcionalidades da API fontes do Google, consulte <http://goo.gl/7J8HZ>.

CAPÍTULO 9

Layout multicolunas

O Módulo CSS3 para Layout Multicolunas descreve as propriedades CSS que permitem aos autores especificar regras CSS para construção de layouts com várias colunas. Neste capítulo, estudaremos esse módulo.

9.1 Introdução

A construção de layouts com uso das funcionalidades desse módulo resulta em vantagens, principalmente em relação à flexibilidade do layout e à acomodação do fluxo de conteúdos pelas colunas, de modo que eles possam pertencer a colunas diferentes, dependendo da resolução ou das dimensões da viewport, bem como do ajuste automático do número de colunas no layout, em função do espaço de renderização disponível.

À época em que esta edição do livro foi escrita, somente o navegador Internet Explorer 10 suporta nativamente as funcionalidades desse módulo. O navegador Firefox as suporta com o uso do prefixo `-moz-` e os navegadores Opera, Chrome e Safari as suportam com o uso do prefixo `-webkit-`.

9.2 Modelo multicolunas

No modelo tradicional dos boxes CSS, os conteúdos inseridos em um elemento container fluem da esquerda para a direita (em escrita ocidental) e de cima para baixo, ocupando o box container.

O modelo de boxes CSS multicolunas, definido nas especificações para as CSS3, criou funcionalidades capazes de fazer com que os conteúdos inseridos em um elemento container fluam por colunas dentro do elemento container. Observe, na figura 9.1, a diferença entre os dois modelos.

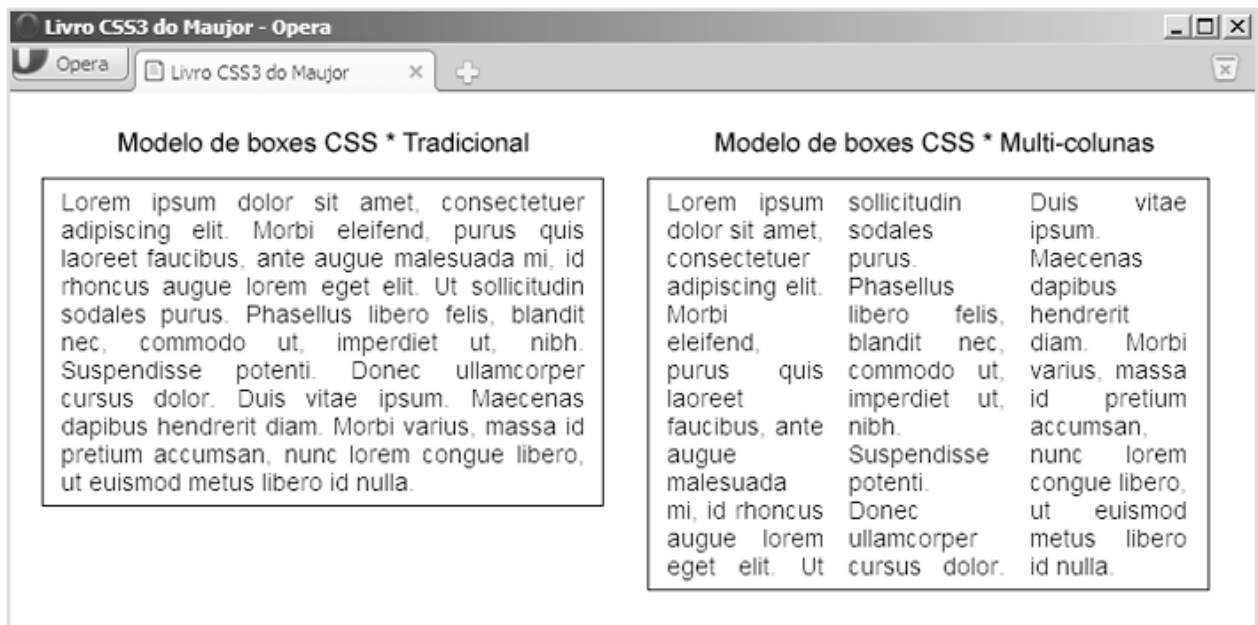


Figura 9.1 – Modelos CSS.

Para o exemplo mostrado na figura 9.1, usamos dois elementos `div` com **300px** de largura, borda na cor preta e padding. Em ambos, inserimos o conteúdo de um parágrafo “Lorem ipsum”. No primeiro `div`, deixamos o fluxo de texto fluir sem aplicar regras CSS. No segundo, aplicamos ao `div` uma classe denominada `.multicolunas` e uma regra CSS definindo três colunas, conforme mostrado a seguir:

```
div.multicolunas { column-count: 3 }
```

A propriedade `column-count`, prevista no módulo, define o número de colunas pelas quais o conteúdo do elemento deve fluir. Assim, com uso dessa propriedade, alteramos o modelo de boxes de um elemento container para multicolunas. A especificação reconhece ainda o termo *box coluna* para designar uma coluna.

O modelo multicolunas é regido pelas seguintes diretrizes:

- as colunas ou boxes colunas se distribuem, pelo elemento container, em linhas;
- tal como ocorre com as células de uma tabela, os boxes colunas de uma linha são ordenados em sequência inline dentro do elemento container;
- a largura de uma coluna é o comprimento do box coluna na direção inline;
- a altura de uma coluna é o comprimento do box coluna na direção do bloco container;
- todos os boxes colunas em uma linha têm a mesma largura e a mesma altura;
- os boxes colunas em uma linha são separados por um espaço denominado *column gap*;
- o espaço *column gap*, entre boxes colunas, poderá conter um separador denominado *column rule*;
- todos os espaços *column gap* e separadores *column rule* (se existirem) têm a mesma largura;
- não é possível declarar regras CSS para boxes colunas individualmente. Por exemplo: o fundo, as cores, as margens e as bordas de uma determinada coluna não são passíveis de estilização.



O W3C admite que futuras especificações para as CSS poderão criar funcionalidades para estilização de boxes colunas individualmente.

A figura 9.2 mostra os locais que são estilizados pelas propriedades `column-width`, `column-gap` e `column-rule` em um elemento multicolumnas.

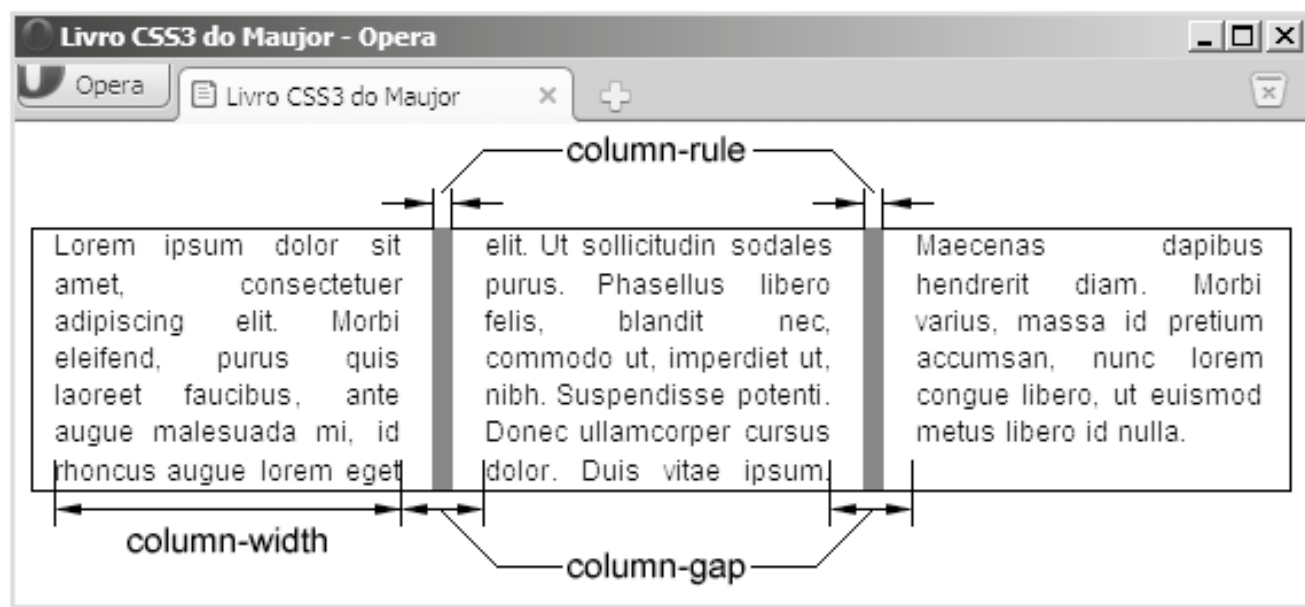


Figura 9.2 – Propriedades para layout multicolumnas.

9.3 Propriedades CSS

As propriedades CSS para criar e estilizar elementos multicolumnas são listadas a seguir: • `column-count` • `column-width` • `columns`

- `column-gap`
- `column-rule`
- `column-span`
- `column-fill`
- `break-before` • `break-after`
- `break-inside` Estudaremos, a seguir, cada uma dessas propriedades.

9.3.1 `column-count`

Essa propriedade se destina a definir o número de colunas a ser criado em um elemento. Os valores possíveis para essa propriedade são: **auto** e **inteiro**. O valor padrão é **auto** e significa que não há divisão em colunas ou que a divisão é definida por outra propriedade (por exemplo: por `column-width`). O valor **inteiro** é um número inteiro maior do que zero, definindo quantidade de boxes colunas a serem inseridos no elemento multicolumnas.

Não sendo definidas explicitamente as dimensões do box coluna, ele terá sua

largura igual a 100% (valor padrão para a propriedade `width`) e o número colunas definido por essa propriedade, independentemente da resolução, isto é, o conteúdo será acomodado nas colunas definidas variando-se a altura das colunas conforme o espaço horizontal disponível. Observe as regras CSS aplicadas à marcação HTML, mostradas a seguir: ► **CSS**

```
div.multicolunas {  
-moz-column-count: 4;  
-webkit-column-count: 4;  
column-count: 4;  
border: 1px solid #000;  
padding: 0 10px;  
}
```

► **HTML**

```
<div class="multicolunas">  
<p>Lorem ipsum dolor...</p>  
</div>
```

Nesse exemplo, definimos quatro colunas para um `div` contendo um texto. Mostramos, na figura 9.3, a distribuição do conteúdo pelos quatro boxes colunas em três resoluções diferentes.

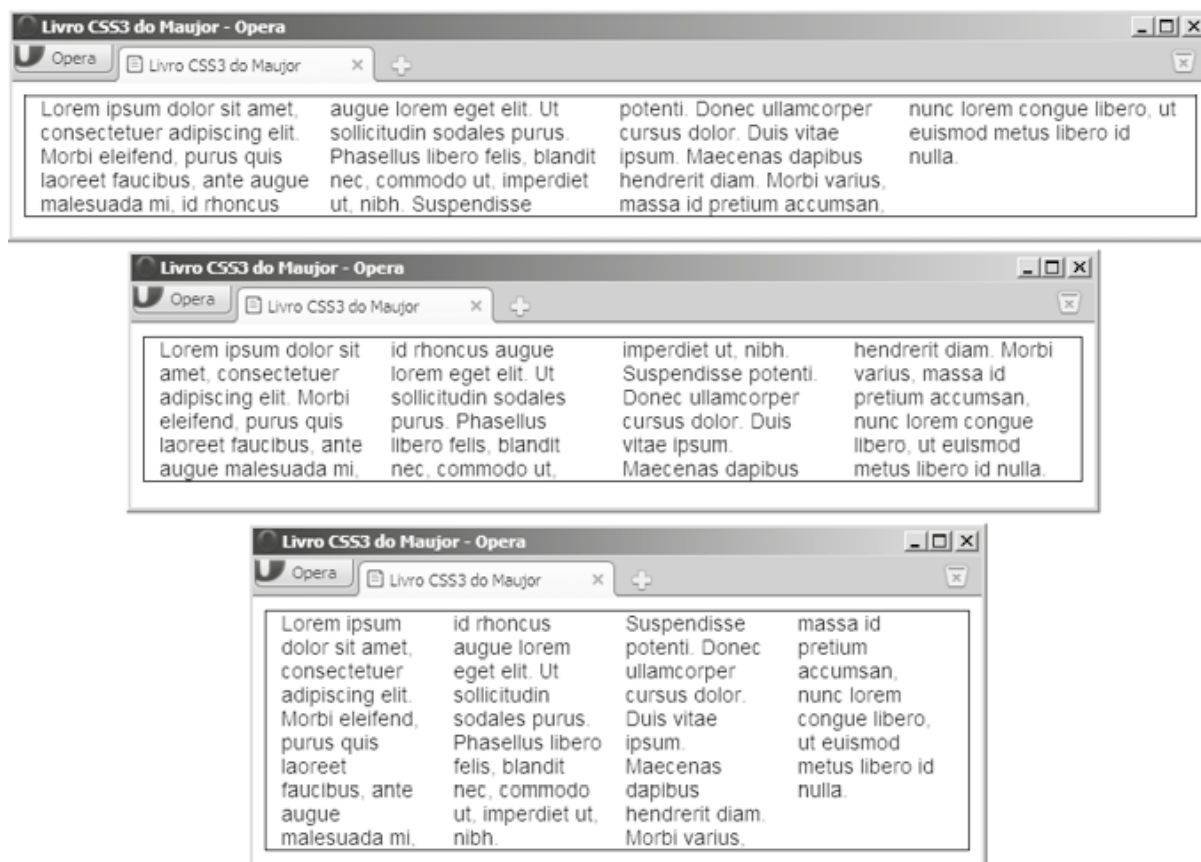


Figura 9.3 – Propriedade `column-count`.

Note que, para qualquer que seja a resolução, o conteúdo acomoda-se nas quatro colunas, variando-se a altura das colunas.

Podemos definir uma altura fixa para as colunas e, em resoluções para as quais a altura definida seja insuficiente para acomodar o conteúdo, fazer aparecer uma barra de rolagem vertical. Observe o acréscimo na regra CSS aplicada à marcação mostrada anteriormente: `.multicolunas {`

```
-moz-column-count: 4;  
-webkit-column-count: 4;  
column-count: 4;  
max-height: 100px; overflow: auto; }
```

Nesse exemplo, definimos uma altura máxima para o elemento multicolunas. Em resoluções para as quais a altura máxima definida para os quatro boxes colunas não seja suficiente para acomodar todo o conteúdo, teremos uma barra de rolagem vertical, como mostrado, para duas resoluções, na figura 9.4.

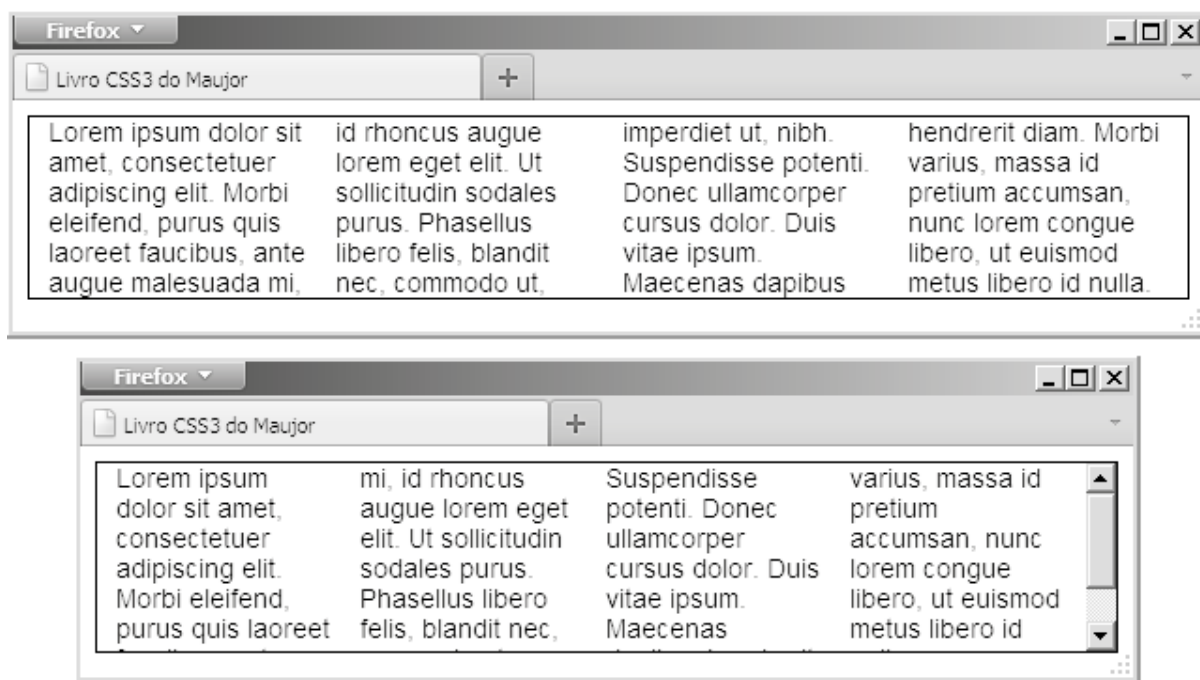


Figura 9.4 – Altura máxima para colunas.

9.3.2 column-width

Essa propriedade se destina a definir a largura das colunas a serem criadas em um elemento. Os valores possíveis para essa propriedade são: **auto** ou uma medida CSS. O valor padrão é **auto** e significa que não há divisão em colunas ou que a divisão é definida por outra propriedade (por exemplo: por `column-count`). O valor medida CSS deve ser maior do que zero, definindo a largura dos boxes colunas a serem inseridos no elemento multicolunas.

Ao contrário da propriedade `column-count`, para a qual o número de colunas é

fixo, essa propriedade faz com que o número de colunas seja dinâmico, ou seja, a quantidade de colunas necessárias para acomodar o conteúdo depende da resolução. Observe a regra CSS aplicada à marcação mostrada anteriormente:

```
.multicolunas {  
  -moz-column-width: 150px;  
  -webkit-column-width: 150px;  
  column-width: 150px;  
}
```

Nesse exemplo, definimos que as colunas de um **div** deverão conter colunas com 150px de largura. Mostramos, na figura 9.5, a distribuição do conteúdo em três resoluções diferentes.

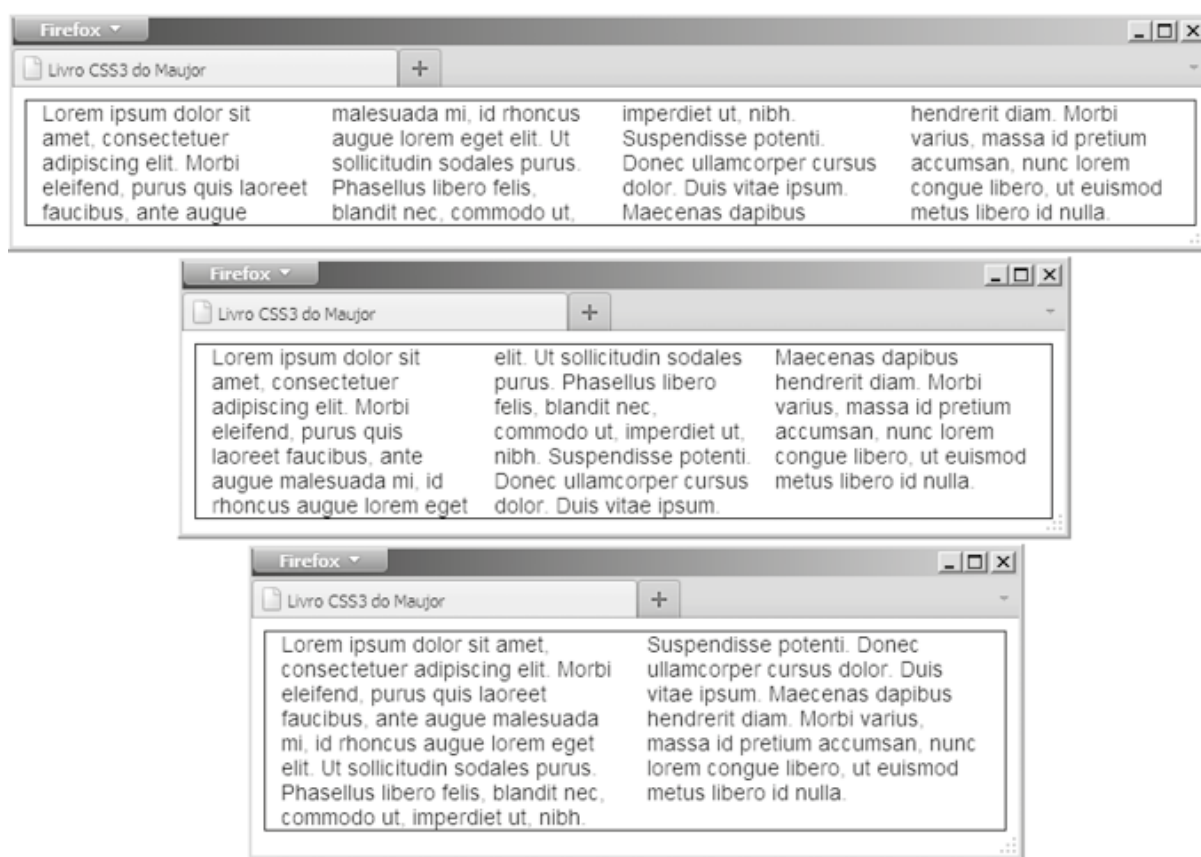


Figura 9.5 – Propriedade column-width.

Note que o número de colunas varia com a resolução.

A especificação esclarece que o valor definido para essa propriedade é uma referência a qual denomina de valor de largura ótimo. O algoritmo de cálculo do valor atual (renderizado) a ser tomado efetivamente como largura das colunas pode resultar valores maiores ou menores do que o valor especificado na regra CSS ou valor ótimo. Note que o cálculo é feito de modo a sempre acomodar um número inteiro de colunas na largura disponível, o que inevitavelmente conduz a pequenos ajustes em torno da largura ótima.

O valor atual será menor do que o valor definido na regra CSS, somente se a largura do elemento for menor do que a largura definida para a coluna.

9.3.3 columns

Essa propriedade é a forma abreviada de se definir `column-count` e `column-width`. Observe o exemplo a seguir: `.multicolunas {`

```
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;
-moz-column-width: 150px;
-webkit-column-width: 150px;
column-width: 150px;

}
```

Para escrever de forma abreviada a regra CSS mostrada, usamos a propriedade `columns`, como mostrado a seguir: `.multicolunas {`

```
-moz-columns: 4 150px;
-webkit-columns: 4 150px;
columns: 4 150px;

}
```

A forma abreviada admite a declaração de dois valores ou de um valor, e, nesse caso, o valor omitido é considerado como **auto**. Nenhum navegador atual suporta essa propriedade abreviada. A especificação prevê vários algoritmos de cálculo do número e da largura de colunas quando se declara essa propriedade. A título de exemplo, comentaremos, a seguir, um caso previsto no algoritmo, e se você estiver interessado em mais detalhes, consulte a especificação em <http://kwz.me/pS>.

Se forem declarados para a propriedade `columns` dois valores diferentes de **auto**, os cálculos serão feitos segundo o seguinte algoritmo: if (column-width != auto) and (column-count != auto) then $N := \min(\text{column-count}, \text{floor}((\text{available-width} + \text{column-gap}) / (\text{column-width} + \text{column-gap})))$ $W := ((\text{available-width} + \text{column-gap}) / N) - \text{column-gap}$;

No algoritmo, o significado de cada termo é explicado a seguir:

Termo	Descrição
<code>column-width</code>	Medida CSS da largura da coluna.
<code>column-count</code>	Número CSS da quantidade de colunas.
<code>column-gap</code>	Medida CSS do espaçamento entre colunas.

<code>min(a,b)</code>	Valor mínimo entre <code>a</code> e <code>b</code> .
<code>floor(n)</code>	Arredondamento para baixo do número <code>n</code> .
<code>N:</code>	Número de colunas calculado.
<code>W:</code>	Largura calculada para a coluna.

Interpretando o algoritmo, temos:

- **Linha 1:** se o valor de `column-width` for diferente de `auto` e o valor de `column-count` for diferente de `auto`, calcule conforme seguir.
- **Linha 2:** o número de colunas será igual ao menor número entre o definido em `column-count` e o número resultante do arredondamento para menor da soma entre largura disponível com a largura do espaçamento entre colunas.
- **Linha 3:** a largura de cada coluna será igual à diferença entre a soma da largura disponível com a largura do espaçamento entre colunas dividido pelo número de colunas e a largura do espaçamento entre colunas.

9.3.4 column-gap

Essa propriedade se destina a definir a largura das colunas a serem criadas em um elemento. Os valores possíveis para essa propriedade são: `normal` ou uma medida CSS. O valor padrão é `normal` e significa que o espaçamento entre colunas será calculado pelo navegador. O valor recomendado pela especificação para a largura do espaçamento entre colunas é de `1em`. O valor medida CSS deve ser maior do que zero.

Na figura 9.6, mostramos dois `div` com quatro boxes colunas. No primeiro, o valor dessa propriedade é `normal`, e no segundo, `60px`, conforme o código mostrado a seguir: `.multicolunas {`

```
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;
```

```
    -moz-column-gap: normal; -webkit-column-gap: normal; column-gap:
normal; }
```

```
.multi-colunas2 {
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;
```

```
-moz-column-gap: 60px; -webkit-column-gap: 60px; column-gap: 60px; }
```

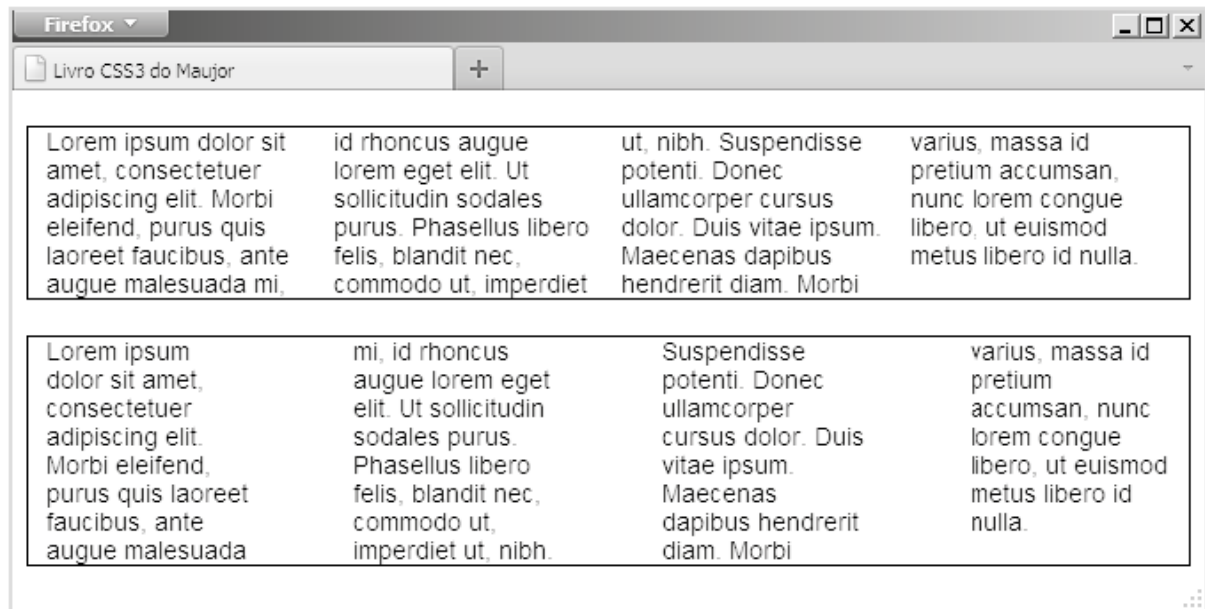


Figura 9.6 – Propriedade column-gap.

9.3.5 column-rule

Essa propriedade se destina a definir uma barra vertical entre as colunas. A barra será posicionada no meio do espaçamento entre colunas e a propriedade permite que se defina espessura, estilo e cor da barra, de modo semelhante à definição de bordas CSS. Os valores possíveis para essa propriedade são: uma medida CSS para espessura, um estilo de borda, uma cor e **transparent**.

Na figura 9.7, mostramos dois **div** com quatro boxes colunas. No primeiro, inserimos uma barra vertical pontilhada com **2px** de espessura na cor preta, e no segundo, uma barra vertical sólida com **8px** de espessura na cor cinza-clara, conforme o código mostrado a seguir: **.multicolunas {**

```
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;

-moz-column-rule: 2px dotted #000; -webkit-column-rule: 2px dotted
#000; column-rule: 2px dotted #000; }

.multi-colunas2 {
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;
-moz-column-gap: 60px;
-webkit-column-gap: 60px;
column-gap: 60px;
-moz-column-rule: 8px solid #ccc; -webkit-column-rule: 8px solid #ccc;
column-rule: 8px solid #ccc; }
```

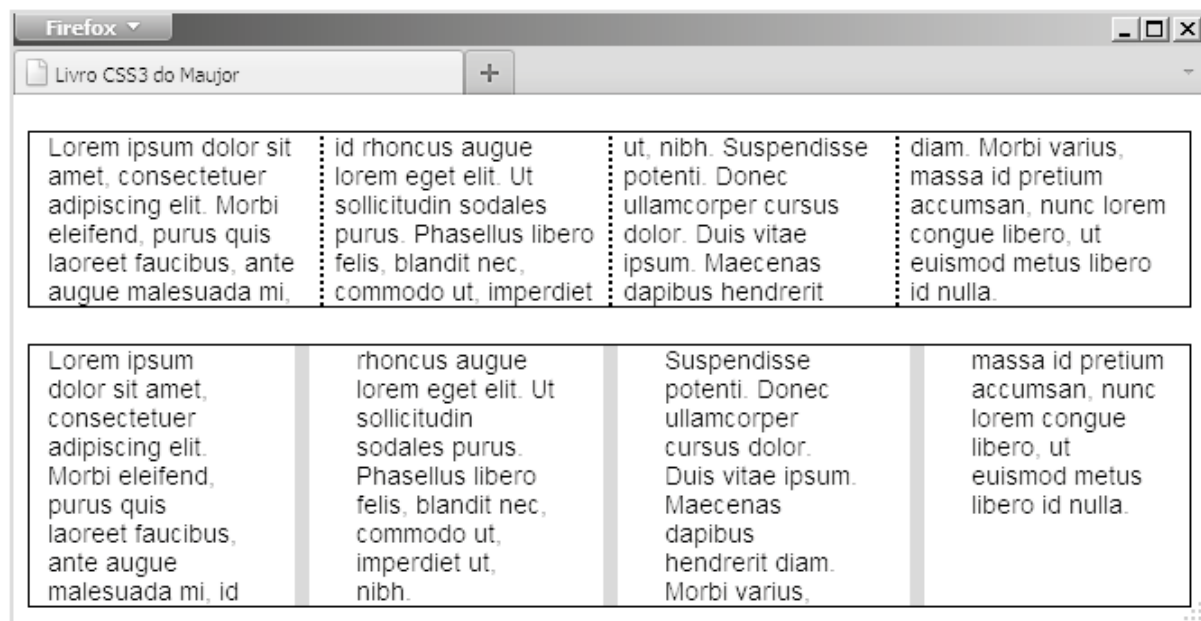


Figura 9.7 – Propriedade *column-rule*.

Podemos preencher toda a largura que separa as colunas com uma barra vertical, bastando, para isso, declarar a espessura da barra igual ao valor de **column-gap**.

Tal como ocorre com a propriedade **border**, essa propriedade é na verdade uma abreviatura para as propriedades **column-rule-width**, **column-rule-style** e **column-rule-color**. Assim, a regra CSS que definiu a barra vertical para o segundo **div** do exemplo anterior poderia ser escrita como mostrado a seguir: **.multi-colunas2 {**

column-rule-width: 8px; column-rule-style: solid; column-rule-color: #ccc; }

Os valores possíveis para a propriedade **column-rule-style** são os mesmos valores válidos para a propriedade **border-style**, ou seja: **none**, **hidden**, **dotted**, **dashed**, **solid**, **double**, **groove**, **ridge**, **inset** e **outset**.

9.3.6 column-span

Essa propriedade se destina a fazer com que um elemento contido no box multicolumna se estenda por toda a largura do box, como se não houvesse colunas no sentido horizontal do elemento. Os valores possíveis para essa propriedade são: **none** e **all**. O valor padrão é **none** e faz com que o elemento permaneça confinado na sua coluna. O valor **all** estende o elemento por todas as colunas.

Essa propriedade é suportada nativamente pelo navegador Opera e pelos navegadores Safari e Chrome, com uso do prefixo **-webkit-**.

Na figura 9.8, mostramos dois **div** com quatro boxes colunas. No conteúdo dos **div** foi inserido um elemento **h4**. No primeiro **div**, não foi declarado valor para a propriedade **column-span** do elemento **h4**, e no segundo, o valor declarado para aquela propriedade foi **all**, conforme o código mostrado a seguir: **.multicolunas {**

```

-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;

}

.multi-colunas2 {
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;

}

h4 {
text-align: center;
background: #ccc;
margin: 0;

}

.multi-colunas2 h4 {
-moz-column-span: all; -webkit-column-span: all; column-span: all; }

```

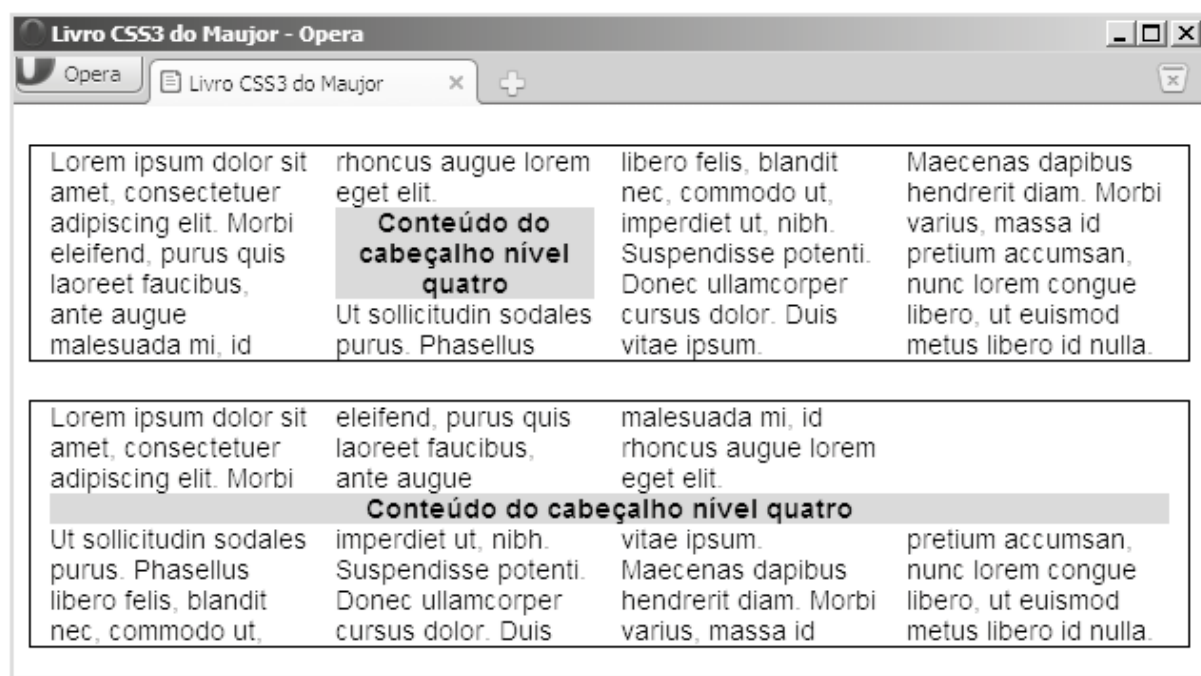


Figura 9.8 – Propriedade `column-span`.

Observe que, para o primeiro `div`, o elemento `h4` ficou confinado em uma coluna, e para o segundo `div`, o elemento `h4` estendeu-se por toda a largura ocupada pelas colunas.

9.3.7 column-fill

Essa propriedade se destina a definir como o conteúdo será distribuído pelas colunas. Os valores possíveis para essa propriedade são: **auto** e **balance**. O valor padrão é **auto** e faz com que o conteúdo preencha as colunas sequencialmente. O valor **balance** faz com que o agente de usuário tente, se for possível, distribuir o conteúdo igualmente pelas colunas.

Essa propriedade atualmente não é suportada por nenhum navegador, e sua sintaxe é conforme mostrado a seguir: `.multicolunas {`

```
column-fill: balance; }
```

9.3.8 break-*

As propriedades **break-before**, **break-after** e **break-inside** se destinam a definir em que ponto do conteúdo as colunas/páginas devem ou não devem começar e/ou terminar, ou seja, onde serão inseridas e/ou não poderá haver quebras de colunas/páginas. O comportamento padrão do navegador é quebrar as colunas/páginas de modo a distribuir o conteúdo de forma corrida ou balanceada por elas. Essas propriedades se destinam a alterar esse comportamento, forçando ou impedindo a quebra de colunas/páginas.

Suponha que o conteúdo a ser distribuído pelas colunas/páginas contenha cabeçalhos, listas, imagens e demais elementos típicos da marcação HTML. As propriedades **break-*** permitem, por exemplo, fazer com que cabeçalhos sejam posicionados sempre no topo de uma coluna/página ou ainda que itens de listas estejam contidos totalmente em uma coluna/página, ou ainda que imagens não sejam posicionadas no topo da coluna/página.



Essas propriedades se aplicam não só à renderização das colunas em um navegador, mas também à impressão em papel ou outra mídia qualquer.

Estudaremos, a seguir, as três propriedades para controle de quebras de colunas/páginas.

9.3.8.1 break-before

Essa propriedade se destina a controlar a quebra de colunas/páginas, tendo como referência o ponto imediatamente antes do elemento para a qual foi definida. Os valores possíveis e seus significados são:

Valor	Descrição
auto	Não há controle de quebra. Esse é o valor padrão da propriedade.
always	Força quebra de página antes do elemento.
avoid	Evita quebra de coluna/página antes do elemento.

left	Força a quebra de uma ou duas páginas antes do elemento e faz com que a próxima página seja formatada como página esquerda.
right	Força a quebra de uma ou duas páginas antes do elemento e faz com que a próxima página seja formatada como página direita.
page	Força quebra de página antes do elemento.
column	Força quebra de coluna antes do elemento.
avoid-page	Evita quebra de página antes do elemento.
avoid-column	Evita quebra de coluna antes do elemento.
 Note que algumas propriedades se aplicam a colunas e páginas, outras somente a colunas e outras ainda somente a páginas.	

9.3.8.2 break-after

Essa propriedade se destina a controlar a quebra de colunas/páginas, tendo como referência o ponto imediatamente depois do elemento para a qual foi definida. Os valores possíveis e seus significados são:

Valor	Descrição
auto	Não há controle de quebra. Esse é o valor padrão da propriedade.
always	Força quebra de página depois do elemento.
avoid	Evita quebra de coluna/página depois do elemento.
left	Força a quebra de uma ou duas páginas depois do elemento e faz com que a próxima página seja formatada como página esquerda.
right	Força a quebra de uma ou duas páginas depois do elemento e faz com que a próxima página seja formatada como página direita.
page	Força quebra de página depois do elemento.
column	Força quebra de coluna depois do elemento.
avoid-page	Evita quebra de página depois do elemento.
avoid-column	Evita quebra de coluna depois do elemento.
 Note que algumas propriedades se aplicam a colunas e páginas, outras somente a colunas e outras ainda somente a páginas.	

outras ainda somente a páginas.

9.3.8.3 break-inside

Essa propriedade se destina a controlar a quebra de colunas/páginas, tendo como referência o conteúdo do elemento para a qual foi definida. Os valores possíveis e seus significados são:

Valor	Descrição
auto	Não há controle de quebra. Esse é o valor padrão da propriedade.
avoid	Evita quebra de coluna/página dentro do conteúdo do elemento.
avoid-page	Evita quebra de página dentro do conteúdo do elemento.
avoid-column	Evita quebra de coluna dentro do conteúdo do elemento.



Note que algumas propriedades se aplicam a colunas e páginas, outras somente a colunas e outras ainda somente a páginas.

9.3.8.4 Exemplo

Essas propriedades atualmente são suportadas somente pelo navegador Opera. Considere os códigos a seguir: ► **CSS**

```
.multicolunas {  
  -moz-column-count: 4;  
  -webkit-column-count: 4;  
  column-count: 4;  
  
  -moz-column-gap: 18px;  
  -webkit-column-gap: 18px;  
  column-gap: 18px;  
  
  -moz-column-rule: 3px double #000;  
  -webkit-column-rule: 3px double #000;  
  column-rule: 3px double #000;  
  
  text-align: justify;  
}
```

► HTML

```
<div class="multicolunas">  
<h4>Cabeçalho 1</h4>
```

```

<p>Lorem ipsum dolor...</p>
<h4>Cabeçalho 2</h4>
<p>Ut sollicitudin...</p>
<h4>Cabeçalho 3</h4>
<p>Suspendisse potenti...</p>
<h4>Cabeçalho 4</h4>
<p>Lorem ipsum dolor...</p>
</div>

```

Trata-se de marcação HTML para um conteúdo composto por quatro cabeçalhos nível 4, cada um deles com um parágrafo. As regras CSS definem que o `div` elemento `multicolunas` deverá conter quatro colunas. Outras estilizações, como margens, espaçamentos e barra vertical entre colunas têm efeito somente decorativo, com a finalidade de clareza do exemplo. A figura 9.9 mostra a renderização do código apresentado, no navegador Opera.

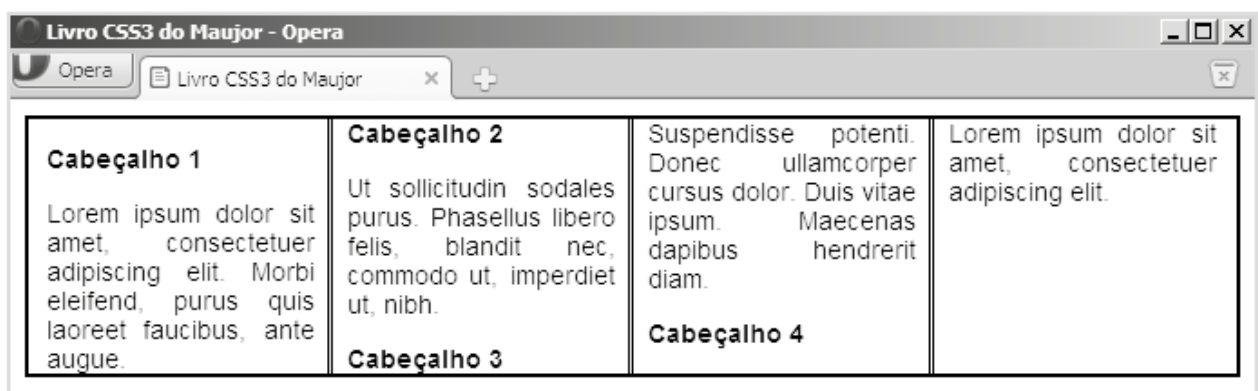


Figura 9.9 – Propriedade `break-*` (não aplicada).

Note que, para a resolução mostrada na figura 9.9, os quatro cabeçalhos se distribuíram pelas colunas de acordo com a posição que ocupam no fluxo do documento.

Vamos aplicar a propriedade `break-*` na marcação HTML do exemplo mostrado na figura 9.9, acrescentando as regras CSS em destaque, no código a seguir: ► **CSS**

```

.multicolunas {
-moz-column-count: 4;
-webkit-column-count: 4;
column-count: 4;

-moz-column-gap: 18px;
-webkit-column-gap: 18px;
column-gap: 18px;

-moz-column-rule: 3px double #000;
-webkit-column-rule: 3px double #000;

```



```
column-rule: 3px double #000;
```

```
text-align: justify;
```

```
}
```

```
h4 {
```

```
    -moz-break-before: column; -webkit-break-before: column; break-before:  
column; text-align: center; }
```

```
p {break-inside: avoid;}
```

► HTML

```
<div class="multicolunas">  
<p>Lorem ipsum dolor...</p>  
</div>
```

A figura 9.10 mostra o efeito da aplicação da propriedade `break-before` e `break-inside` nos cabeçalhos e nos parágrafos, respectivamente, no navegador Opera.

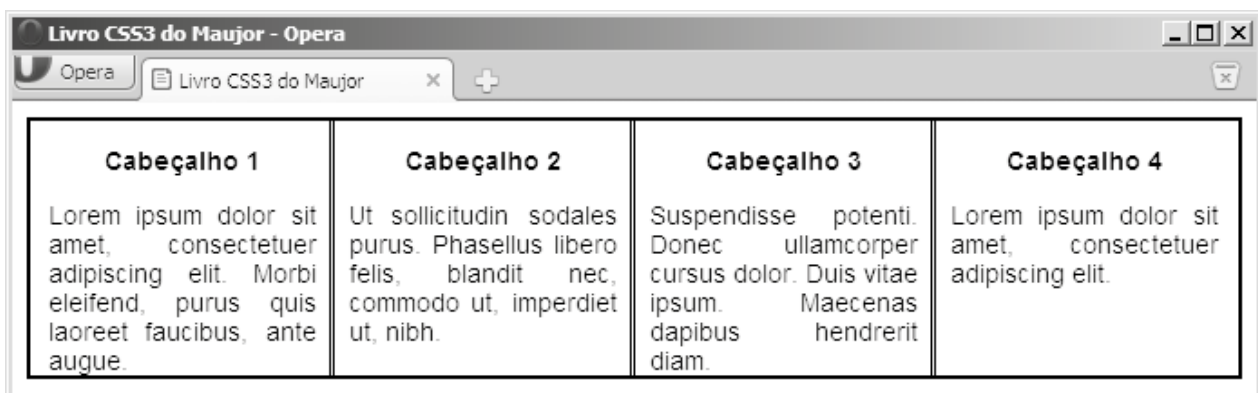


Figura 9.10 – Propriedade `break-*` (aplicada).

9.3.9 Overflow

9.3.9.1 Overflow de elemento

Se um elemento conteúdo de uma coluna tiver largura maior do que a largura da coluna, ele será interrompido quando seu limite direito alcançar o meio do espaçamento entre as colunas (`column-gap`), fazendo com que o trecho em **overflow** à coluna se torne invisível. Observe o exemplo a seguir, que mostra esse comportamento: ► **CSS**

```
.multicolunas {  
    -moz-column-count: 3;  
    -webkit-column-count: 3;  
    column-count: 3;  
    -moz-column-gap: 18px;  
    -webkit-column-gap: 18px;
```

```
column-gap: 18px;
-moz-column-rule: 3px double #000;
-webkit-column-rule: 3px double #000;
column-rule: 3px double #000;
text-align: justify;
```

```
}
```

```
p {
-moz-break-inside: avoid;
break-inside: avoid;
-webkit-break-inside: avoid;
```

```
}
```

```
p.a {
-moz-break-before: column;
-webkit-break-inside: avoid;
-webkit-break-inside: avoid;
```

```
}
```

```
.overflow {
width: 220px;
height: 40px;
padding: 0;
margin: 10px 0;
background: #ccc;
-moz-break-after: avoid;
-webkit-break-after: avoid;
break-after: avoid;
```

```
}
```

► HTML

```
<div class="multicolunas">
<div class="overflow">div.overflow</div>
<p>Lorem ipsum...</p>
<p class="a">Ut sollicitudin...</p>
<p class="a">Lorem ipsum...</p>
</div>
```

A figura 9.11 mostra a renderização do HTML com aplicação das regras de estilo, mostradas no código, no navegador Opera, em duas resoluções diferentes. Na primeira, a largura da coluna é menor do que largura do `div.overflow`, e na segunda, aquela largura é maior. Note que, para a resolução menor, o `div.overflow`

foi interrompido no meio do espaçamento entre colunas.

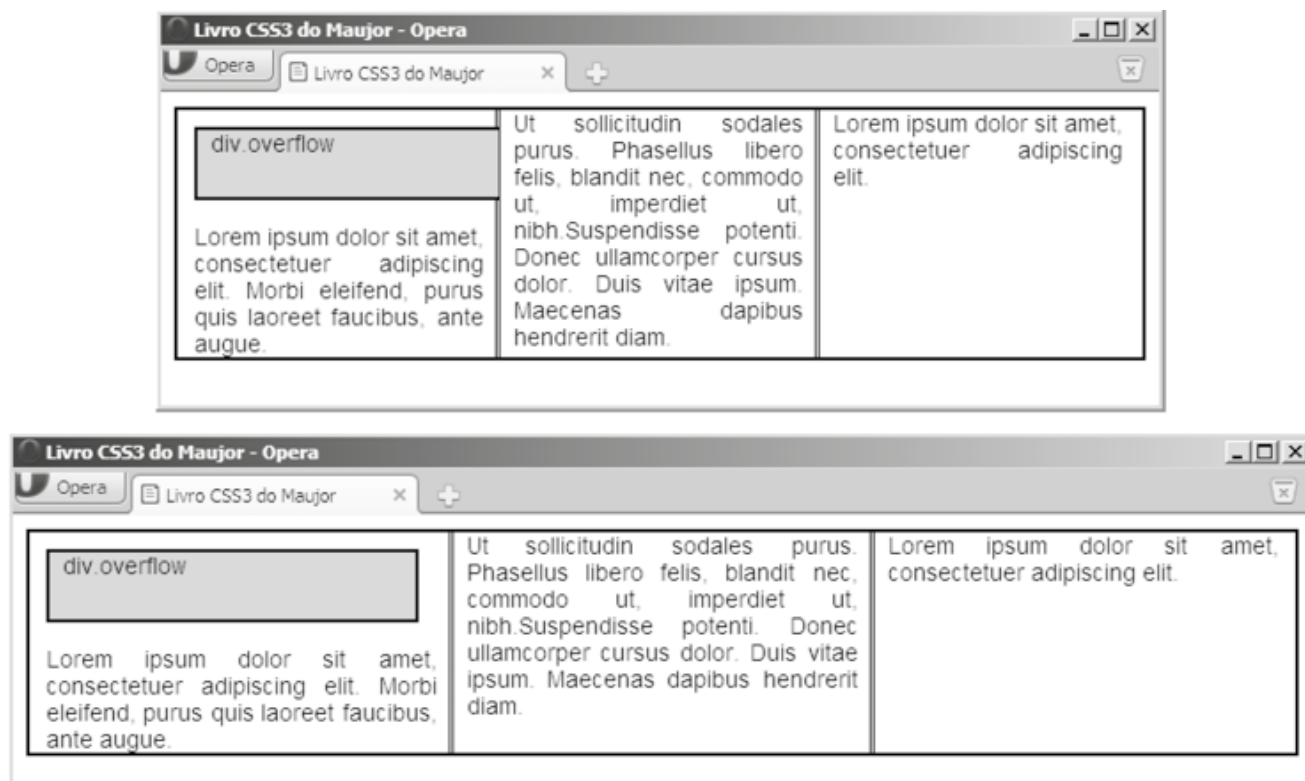


Figura 9.11 – Overflow de elementos.

9.3.9.2 Overflow de coluna

É perfeitamente possível ocorrer o caso em que um elemento multicolumns tenha uma quantidade de colunas cuja soma das larguras seja maior do que a largura do elemento. Nesse caso, as colunas excedentes e seus respectivos conteúdos ultrapassarão o limite direito do elemento. As colunas além daquele limite são chamadas de colunas em overflow.

O aparecimento de colunas em overflow poderá ocorrer em um dos seguintes casos: • quando se declara uma altura fixa para o elemento coluna (uso das declarações `height` ou `max-height`); • em determinadas situações, quando se declara `break_*` para elementos nas colunas; • em determinadas situações, quando a largura da página tem dimensões limitadas.

Observe o exemplo a seguir, que mostra um caso de ocorrência de colunas em overflow: ► **CSS**

```
.multicolunas {  
  max-height: 160px; -moz-column-count: 3;  
  -webkit-column-count: 3;  
  column-count: 3;
```

```
-moz-column-gap: 18px;  
-webkit-column-gap: 18px;
```

```
column-gap: 18px;

-moz-column-rule: 3px double #000;
-webkit-column-rule: 3px double #000;
column-rule: 3px double #000;

text-align: justify;

}
```

► HTML

```
<div class="multicolunas">
<p>Lorem ipsum...</p>
<p>Suspendisse potenti...</p>
<p>Lorem ipsum...</p>
<p>Ut sollicitudin...<p>
</div>
```

Observe que forçamos um overflow de coluna ao declarar para o elemento `div.multicolunas` uma altura máxima de **160px**.

A figura 9.12 mostra a renderização do HTML com aplicação das regras de estilo, mostradas no código, no navegador Opera. Note a barra de rolagem horizontal no navegador. Na parte superior, a barra de rolagem está à esquerda, e na parte inferior, rolamos a tela para a esquerda na horizontal, revelando as duas colunas em overflow. Atualmente, somente o navegador Opera apresenta esse comportamento para colunas em overflow.

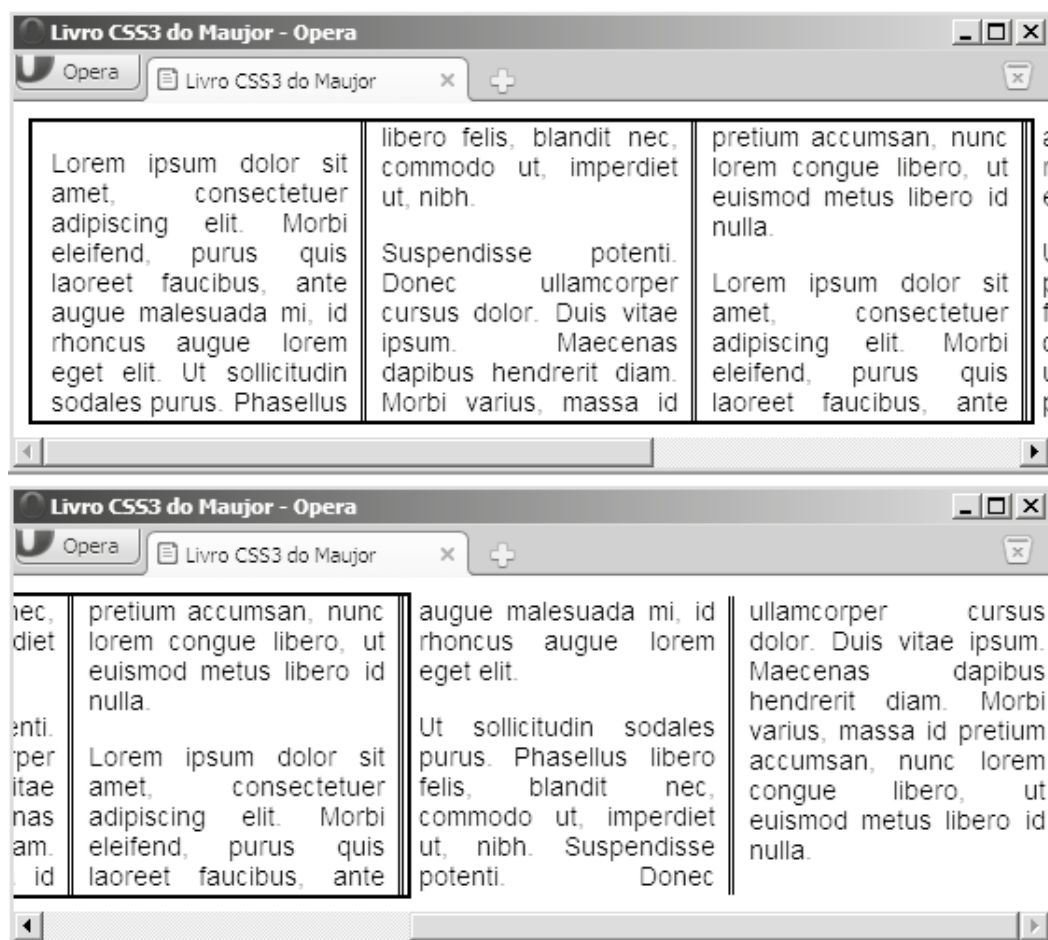


Figura 9.12 – Overflow de coluna.

CAPÍTULO 10

Media Queries

A especificação CSS3 para Media Queries é uma Recomendação do W3C. Essa especificação descreve as propriedades CSS que permitem aos autores criar regras CSS a serem servidas aos agentes de usuário, cujas características e tipo de mídia satisfaçam às condições estabelecidas pelo autor. Neste capítulo, estudaremos as funcionalidades previstas naquela especificação.

10.1 Introdução

O conceito “Uma marcação HTML para todos os tipos de mídia” resume não só um objetivo que o W3C vem tentando alcançar há muito tempo, mas também uma das maiores expectativas que nós, autores HTML e CSS, temos.

Escrever uma marcação HTML que se adapte de forma consistente e acessível tanto às diferentes resoluções em desktop como a impressoras, dispositivos aurais e braille ou às telas dos inúmeros dispositivos móveis é um problema que o W3C vem tentando resolver desde longa data.

A primeira vez que o W3C introduziu uma funcionalidade com vistas a contemplar a possibilidade de se especificar folhas de estilo de acordo com a mídia para a qual o documento fosse servido foi quando lançou a versão da Proposta de Recomendação para as CSS2, datada de 24 de março de 1998, quando estabeleceu o seguinte (traduzido daquela especificação): As CSS2 suportam folhas de estilo específicas para diferentes mídias, possibilitando aos autores criar a apresentação de seus documentos para servi-los de maneira diferenciada para navegadores visuais, dispositivos aurais, impressoras, dispositivos braille, dispositivos móveis *etc.*

Os tipos de mídia previstos naquela versão e mantidos até a versão atual das CSS2.1 foram: **all**, **aural**, **braille**, **embossed**, **handheld**, **print**, **projection**, **screen**, **tty** e **tv**. A especificação para as CSS2.1 preveem duas maneiras de se vincular folhas de estilos, dependentes de mídia, em um documento. Importá-las para uma folha de estilos principal ou linká-las diretamente ao documento com uso do elemento **link** e seu atributo **media**, na seção **head** do documento, conforme as sintaxes mostradas a seguir: • **Importação da folha de estilo (duas opções):** /* Folha de estilo principal */

```
body {  
...
```

```
}
```

```
/* regras de estilos para todas as mídias */  
...
```

```

/* regras de estilos para mídia impressa a seguir */
@import url(print.css) print; /* opção 1 em folhas de estilo principal*/
@media print { /* opção 2 em folhas de estilo principal */
/* regras de estilo para impressão */

}

```

- **Link no documento:** ...

```

<head>
...
<link rel="stylesheet" type="text/css" media="print" href="print.css">
</head>

```

Navegadores implementaram suporte, de forma aceitável, para leitura e aplicação de regras de estilos para a mídia **print**, mas, infelizmente, para as demais mídias a implementação foi parcial ou mesmo nenhuma. Com a chegada dos dispositivos móveis e a incrível diversidade deles, o suporte precário do valor **handheld** tornava-o, na prática, inútil.

Assim, as CSS3 estenderam o mecanismo de vinculação de folhas de estilos dependentes de mídia e criaram um conceito totalmente novo para essa funcionalidade, denominado Media Queries.

10.2 Anatomia da media query

Media query, em tradução livre, significa “consulta ao tipo de mídia”. O objetivo é servir a folha de estilo específica para determinado tipo de mídia mediante uma consulta ao tipo de mídia ao qual a folha de estilo será servida.

As maneiras de se vincular folhas de estilo dependentes de mídia continuam iguais àquelas preconizadas pelas especificações para CSS2.1. O que mudou foi a sintaxe para definir a mídia, que agora assumiu o formato de uma consulta. Observe os exemplos mostrados a seguir, destinados a vincular uma folha de estilo para determinado tipo de mídia.

- **Importação da folha de estilo (duas opções):** `@import url(color.css) screen and (color)`; `@media screen and (color) {`
`/* regras de estilo */`

```

}
```

- **Link no documento:** `<link rel="stylesheet" type="text/css" media="screen and (color)" href="color.css">` Note que a sintaxe é igual à prevista nas versões anteriores da HTML e das CSS, como mostrado no item [10.1]. As únicas diferenças são o valor do atributo **media**, que agora se transformou em uma

consulta ao tipo de mídia (media query) para folhas de estilo linkadas, e o acréscimo da media query para as folhas de estilo importadas.

A especificação para as CSS3 define media query conforme tradução a seguir: Media query é uma sentença formada por um tipo de mídia e zero ou mais expressões destinadas a verificar as condições de ocorrência de uma determinada funcionalidade naquele tipo de mídia.

Uma media query é uma expressão lógica que retorna os valores **true** ou **false**. Se o tipo de mídia definido em uma media query casa com o tipo de mídia do dispositivo no qual o agente de usuário está rodando e se todas as expressões definidas na media query são verdadeiras, então a media query retorna **true** e a folha de estilo especificada deve ser aplicada ao documento.

Na figura 10.1, mostramos a sintaxe de uma media query e suas partes componentes.



Figura 10.1 – Sintaxe de media query.

Os operadores e seus significados, previstos nas especificações, para uso em media query, são:

Operador	Descrição
and	Operador de concatenação, retorna true se as condições à esquerda e à direita do operador são ambas verdadeiras.
only	Palavra-chave usada para esconder folhas de estilo dos navegadores antigos que não suportam media query.
not	Operador de negação, retorna true se a funcionalidade não existir.

Na sintaxe para media query, é obrigatório definir o tipo de mídia, enquanto o operador e a expressão são facultativos e, se usados, podem ser declarados em cadeia (dois ou mais), conforme exemplos mostrados a seguir: @media all {...}

```
@media screen and (min-width: 400px) and (max-width: 700px) {...}
```

```
@media only print and (min-width: 25cm) and (color) {...}
```

Podemos concatenar múltiplas media queries, bastando, para isso, separá-las com vírgula, conforme os exemplos mostrados a seguir: @media screen and (max-width: 400px), handheld and (color) {...}


```
@media screen and (min-width: 400px) and (max-width: 700px), print and (min-width: 25cm) and (color) {...}
```

É preciso atenção com o operador **not**. Ele tem a precedência mais baixa entre os demais operadores. O exemplo mostrado a seguir esclarece a precedência: `@media not screen and (color) {...}`

Existem duas opções para interpretar a media query mostrada, que são as seguintes: `@media not (screen and (color)) {...}`

```
@media (not screen) and (color) {...}
```

Como a precedência do operador **not** é mais baixa que a do operador **and**, este é avaliado antes daquele, portanto a opção correta é a primeira.

10.3 Funcionalidades de mídia

Sintaticamente, funcionalidades de mídia se assemelham às propriedades CSS, ou seja, possuem um nome e aceitam determinados valores. As diferenças entre funcionalidades de mídia e propriedades CSS são descritas a seguir:

- Propriedades CSS são usadas em declarações CSS que fornecem informações sobre como apresentar o documento. Funcionalidades de mídia são usadas em expressões destinadas a descrever requisitos do dispositivo de saída do documento.

- A maioria das funcionalidades de mídia aceita os prefixos **min-** e **max-** para descrever valores “menores ou igual a” e “maiores ou igual a”, respectivamente.
- Propriedades CSS requerem a declaração de um valor. Funcionalidades de mídia podem ser usadas com ou sem declaração de um valor, exceto aquelas com os prefixos **min-** e **max-**, cujo valor deve ser declarado.
- As unidades de medida para os valores das funcionalidades de mídia são as mesmas unidades de medida adotadas para valores das propriedades CSS.
- Propriedades CSS podem aceitar valores complexos, tais como aqueles que exigem cálculos envolvendo outros valores. Funcionalidades de mídia aceitam somente valores simples, tais como uma palavra-chave, um número, um número com definição da unidade de medida. A exceção é para valores das funcionalidades de mídia **aspect-ratio** e **device-aspect-ratio**, cujos valores são números fracionários da forma *a/b*.

As funcionalidades de mídia previstas nas especificações para as CSS3 e destinadas a criar expressões para consultas de mídia dos tipos visual, tátil e aural são:

- **width**

- **height**
- **device-width**
- **device-height**

- orientation
- aspect-ratio
- device-aspect-ratio • color
- color-index
- monochrome
- resolution
- scan
- grid

Estudaremos, a seguir, cada uma dessas funcionalidades.

10.3.1 width

Essa funcionalidade descreve a largura da área de saída do dispositivo do usuário. Para mídia contínua, é a largura da viewport, incluindo barra de rolagem, se existente, e para mídia paginada, é a largura do box da página. O valor possível para essa funcionalidade é uma medida de comprimento CSS não negativa. Essa funcionalidade aceita os prefixos `min-` e `max-`.

Exemplos:

```
<link rel="stylesheet" media="print and (width: 25cm)" href="http://..." />
@media screen and (min-width: 400px) and (max-width: 700px) {...}
@media handheld and (max-width: 20em), screen and (width: 20em) {...}
```

10.3.2 height

Essa funcionalidade descreve a altura da área de saída do dispositivo do usuário. Para mídia contínua, é a altura da viewport, incluindo barra de rolagem, se existente, e para mídia paginada, é a altura do box da página. O valor possível para essa funcionalidade é uma medida de comprimento CSS não negativa. Essa funcionalidade aceita os prefixos `min-` e `max-`.

Exemplos:

```
<link rel="stylesheet" media="print and (height: 29cm)" href="http://..." />
@media screen and (min-height: 600px) and (max-height: 768px) {...}
@media handheld and (max-height: 14em), screen and (height: 14em) {...}
```

10.3.3 device-width

Essa funcionalidade descreve a largura da área de renderização do dispositivo do usuário. Para mídia contínua, é a largura da tela, e para mídia paginada, é a largura da página. O valor possível para essa funcionalidade é uma medida de comprimento

CSS não negativa. Essa funcionalidade aceita os prefixos **min-** e **max-**.

Exemplos:

```
<link rel="stylesheet" media="print and (max-device-width: 23cm)"
href="http://..." /> @media screen and (min-device-width: 400px) and (max-
device-width: 700px) {...}
@media handheld and (max-device-width: 20em), screen and (device-width:
20em) {...}
```



A diferença entre **width** e **device-width** é que aquela funcionalidade é dinâmica, e esta, fixa. Por exemplo: se o usuário redimensiona a largura da janela do navegador, ele está alterando dinamicamente o valor da funcionalidade **width** do seu dispositivo de usuário, mas, para qualquer dimensão da janela, a propriedade **device-width** é sempre a mesma, pois trata-se de uma característica física do dispositivo.

10.3.4 device-height

Essa funcionalidade descreve a altura da área de renderização do dispositivo do usuário. Para mídia contínua, é a altura da tela, e para mídia paginada, é a altura da página. O valor possível para essa funcionalidade é uma medida de comprimento CSS não negativa. Essa funcionalidade aceita os prefixos **min-** e **max-**.

Exemplos:

```
<link rel="stylesheet" media="print and (device-height: 29cm)"
href="http://..." /> @media screen and (min-device-height: 600px) and (max-
device-height: 768px) {...}
@media handheld and (max-device-height: 14em), screen and (device-height:
14em) {...}
```



A diferença entre **height** e **device-height** é que aquela funcionalidade é dinâmica, e esta, fixa. Por exemplo: se o usuário redimensiona a altura da janela do navegador, ele está alterando dinamicamente o valor da funcionalidade **height** do seu dispositivo de usuário, mas, para qualquer dimensão da janela, a propriedade **device-height** é sempre a mesma, pois trata-se de uma característica física do dispositivo.

10.3.5 orientation

Essa funcionalidade se baseia em um mecanismo de comparação entre a largura (funcionalidade **width**) e a altura (funcionalidade **height**) da área de saída do dispositivo do usuário. Os valores possíveis para essa funcionalidade são **landscape** e **portrait**. O valor **landscape** casa com mídias quando a funcionalidade **width** é maior do que a funcionalidade **height**; caso contrário, o valor é **portrait**. Essa funcionalidade não aceita os prefixos **min-** e **max-**.

Exemplos:

```
<link rel="stylesheet" media="screen and (orientation: portrait)"
href="http://..." /> @media screen and (orientation: landscape) {...}
@media screen and (orientation:portrait) {...}
```

10.3.6 aspect-ratio

Essa funcionalidade se baseia na razão entre a largura (funcionalidade **width**) e a altura (funcionalidade **height**) da área de saída do dispositivo do usuário. Os valores possíveis para essa funcionalidade são frações do tipo a/b . Essa funcionalidade aceita os prefixos **min-** e **max-**.

Exemplos:

```
<link rel="stylesheet" media="screen and (aspect-ratio: 4/3)"
href="http://..." /> @media screen and (aspect-ratio: 16/9) {...}
@media screen and (min-aspect-ratio: 5/2) {...}
```

10.3.7 device-aspect-ratio

Essa funcionalidade se baseia na razão entre a largura (funcionalidade **device-width**) e a altura (funcionalidade **device-height**) da área de renderização do dispositivo do usuário. Os valores possíveis para essa funcionalidade são frações do tipo a/b . Essa funcionalidade aceita os prefixos **min-** e **max-**.

Exemplos:

```
<link rel="stylesheet" media="screen and (device-aspect-ratio: 4/3)"
href="http://..." /> @media screen and (device-aspect-ratio: 16/9) {...}
@media screen and (min-device-aspect-ratio: 800/600) {...}
```



A diferença entre **aspect-ratio** e **device-aspect-ratio** é que aquela funcionalidade é dinâmica, e esta, fixa. Por exemplo: se o usuário redimensiona a altura da janela do navegador, ele está alterando dinamicamente o valor da funcionalidade **aspect-ratio** do seu dispositivo de usuário, mas, para qualquer dimensão da janela, a propriedade **device-aspect-ratio** é sempre a mesma, pois trata-se de uma característica física do dispositivo.

10.3.8 color

Essa funcionalidade descreve o número de bits por componente de cor no dispositivo do usuário. Essa funcionalidade aceita os prefixos **min-** e **max-**, e seu valor deve ser um número inteiro não negativo.

Por exemplo: em sistemas de cores diretas de 8 bits que permitem 256 cores, os componentes R (red) e G (green) são descritos por 3 bits cada um, e o componente B (blue), por 2 bits. Em sistemas de cores diretas 12 bits que permitem 4.096 cores, os componentes R (red), G (green) e B (blue) são descritos por 4 bits cada um.

Em sistemas nos quais o número de bits que representa cada componente não é o mesmo para cada componente, o valor da funcionalidade é o menor número.

Omitindo-se o valor da funcionalidade ou atribuindo-se a ela o valor 1, o alvo da media query são todos os dispositivos coloridos, independentemente do número de bits usado para representar cada componente.

Atribuindo-se o valor 0 para a funcionalidade, o alvo da media query são os dispositivos monocromáticos.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
<link rel="stylesheet" media="screen and (color)" href="http://..." > *
todos os dispositivos coloridos */
@media screen and (color: 2) {...} /* dispositivos coloridos de 8 bits */
@media screen and (max-color: 4) {...} /* dispositivos coloridos de 12 ou 8
bits */
@media screen and (color: 0) {...} /* dispositivos monocromáticos */
```

10.3.9 color-index

Essa funcionalidade descreve o número de entradas na paleta de cores disponíveis no dispositivo do usuário. Essa funcionalidade aceita os prefixos `min-` e `max-`, e seu valor deve ser um número inteiro não negativo.

Por exemplo: em sistemas de cores diretas de 8 bits, o número máximo de cores que o sistema pode representar é 256 cores. Assim, 256 é o valor de `color-index` desses sistemas ou dispositivos.

Omitindo-se o valor da funcionalidade ou atribuindo-se a ela o valor 1, o alvo da media query são todos os dispositivos coloridos que usam paleta de cores.

Atribuindo-se o valor 0 para a funcionalidade, o alvo da media query são os dispositivos que não usam paleta de cores.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
<link rel="stylesheet" media="screen and (color-index)" href="http://..." >
* todos os dispositivos de paleta de cores */
@media screen and (color-index: 0) {...} /* dispositivos que não usam paleta
de cores */
@media screen and (min-color-index: 4096) {...} /* dispositivos coloridos de
4096 ou mais entradas */
```

10.3.10 monochrome

Essa funcionalidade descreve o número de bits por pixel no dispositivo monocromático do usuário. Essa funcionalidade aceita os prefixos `min-` e `max-`, e seu valor deve ser um número inteiro não negativo.

Omitindo-se o valor da funcionalidade ou atribuindo-se a ela o valor 1, o alvo da media query são todos os dispositivos monocromáticos, independentemente do número de bits por pixel.

Atribuindo-se o valor 0 para a funcionalidade, o alvo da media query são os dispositivos não monocromáticos.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
<link rel="stylesheet" media="screen and (monochrome)" href="http://..." > *  
todos os dispositivos monocromáticos */  
@media screen and (monochrome: 2) {...} /* dispositivos monocromáticos de 2  
bits por pixel */
```

10.3.11 resolution

Essa funcionalidade descreve a resolução, ou seja, a densidade de pixels do dispositivo do usuário. Essa funcionalidade aceita os prefixos `min-` e `max-`, e seu valor deve ser um número inteiro não negativo representando a densidade de pixels por unidade de medida.

As duas unidades de medida previstas nas especificações para os valores dessa funcionalidade são: `dpi` e `dpcm`.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
@media screen and (min-resolution: 300dpi) {...} /* dispositivos com  
resolução maior que 300  
pontos por polegada */  
@media screen and (resolution: 118dpcm) {...} /* dispositivos de 118 pontos  
por centímetro */
```

10.3.12 scan

Essa funcionalidade descreve o processo de escaneamento de imagens em mídias do tipo TV. Essa funcionalidade não aceita os prefixos `min-` e `max-`, e seu valor deve ser a palavra-chave `progressive` ou `interlace`.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
@media tv and (scan: progressive) {...}  
@media tv and (scan: interlace) {...}
```

10.3.13 grid

Essa funcionalidade descreve se a saída do dispositivo é em grid ou bitmap. Essa funcionalidade não aceita os prefixos `min-` e `max-`, e seu valor deve ser 0 ou 1. O valor 1 ou a omissão de valor define um dispositivo com saída em grid, por exemplo: um terminal `tty` ou um dispositivo móvel com um só tamanho fixo de fonte. O

valor 0 define saída bitmap.

O exemplo a seguir esclarece a sintaxe dessa funcionalidade.

Exemplos:

```
@media handheld and (grid) and (max-width: 15em) {...}
@media handheld and (grid) and (device-max-height: 7em) {...}
```

10.4 Exemplo

Para esclarecer com um exemplo prático o uso da funcionalidade media query, vamos considerar o seguinte: • **Problema:** construir um layout elástico com largura máxima de **1000px**, constituído de topo, coluna de navegação, coluna principal, coluna auxiliar e rodapé, que se adapte a diferentes resoluções de tela, tanto em desktop como em dispositivos móveis.

- **Solução:** optamos por um layout de três colunas em resoluções que vão desde **760px** até **1000px**. Entre **400px** e **760px**, o layout será com duas colunas, e abaixo de **400px**, teremos um layout com uma coluna.

O exemplo completo, funcionando, encontra-se no site do livro.

10.4.1 Primeiro passo

Criar a marcação HTML e a folha de estilo para construção do layout que atenda à resolução mais alta. No nosso exemplo, um layout elástico com três colunas a ser apresentado em resoluções acima de **760px**, e que a partir de **1000px** se torne fixo, conforme mostrado a seguir: ► **HTML**

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="utf-8" />
<title>Livro CSS3 do Maujor - Media Queries</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1" /> </head>
<body>
<div id="tudo">
<div id="topo"><p>TOP0</p></div>
<div id="nav">
<p>NAV</p>
<ul>
<li>Link 1</li>
<li>Link 2</li>
<li>Link 3</li>
<li>Link 4</li>
</ul>
```

```

</div> <!-- Fim div#nav -->
<div id="principal">
<p>PRINCIPAL</p>
<p>Esta página é interativa ... acontece no Firefox.</p> </div> <!-- Fim
div#principal -->
<div id="aux">
<p>AUX</p>
<p>Lorem ipsum dolor sit amet.</p>
</div> <!-- Fim div#aux -->
<div id="rodape">
<p>RODAPÉ</p>
</div> <!-- Fim div#rodape -->
</div> <!-- Fim div#tudo -->
<!--[if lt IE 9]>
<script type="text/javascript" src="css3-mediaqueries.js"></script> <![endif]-->
</body>
</html>

```

Note, na seção `head` do documento, a presença da metatag `viewport`. Ela é necessária para servir o layout para dispositivos móveis. Consulte <http://kwz.me/II> para mais informações sobre essa metatag.

► CSS

```

<style>
body {
font: 16px/1.2 sans-serif;
margin: 0;
padding: 0;
background: #000;

}

p { margin: 5px; }
#topo p, #nav p, #principal p:first-child, #aux p:first-child, #rodape p
{
color: #fff;
font-weight: bold;
text-shadow: 2px 2px 2px #000;

}

ul {
margin: 0 0 0 5px;
padding: 0;

```



```
list-style: none;
```

```
}
```

```
#tudo {  
max-width: 1000px;  
margin: 0 auto;  
background: url(nav-3col.gif) 18% repeaty, url(aux-3col.gif) 82%  
repeaty; background-color: #ccc;
```

```
}
```

```
#topo {  
height: 60px;  
padding-top: 1px;  
background: #c60;
```

```
}
```

```
#nav {  
width: 18%;  
float: left;  
background: #9cf;
```

```
}
```

```
#principal {  
width: 64%;  
float: left;  
background: #ffe;
```

```
}
```

```
#aux {  
width: 18%;  
float: right;  
background: #fc9;
```

```
}
```

```
#rodape {  
height: 30px;  
clear: both;  
background: #9c3;
```

```
}
```

Na figura 10.2, mostramos a renderização do layout com três colunas.

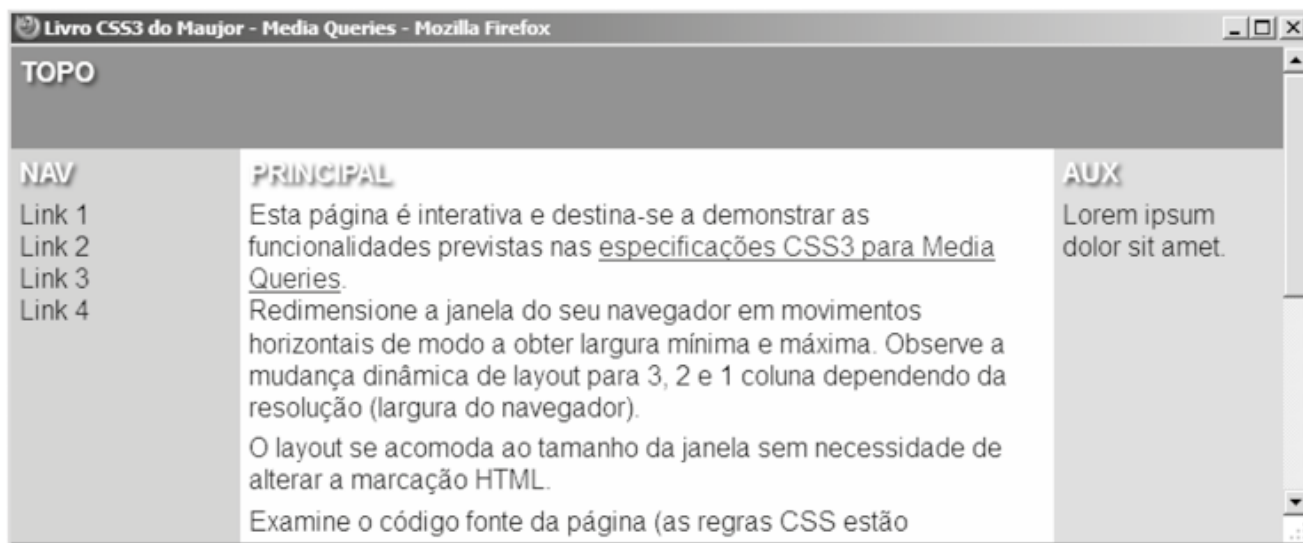


Figura 10.2 – Media query: três colunas.

10.4.2 Segundo passo

Para a mesma marcação HTML, criar regras de estilo adicionais a serem inseridas na folha de estilo já criada para construção do layout que atenda a outras resoluções. No nosso caso, um layout elástico de duas colunas para resoluções entre 760px e 1000px.

É importante notar que, para o layout que atenda à nova resolução, nenhuma modificação poderá ser feita tanto na marcação HTML como na folha de estilo desenvolvida no passo anterior. Havendo necessidade, neste passo, de se eliminar regras de estilo já criadas, não devemos simplesmente retirá-las da folha de estilo (o que iria quebrar o layout já construído), mas escrever outra regra de estilo que anule (sobrescreva) a anterior.

Uma das soluções para transformar o layout de três colunas em um layout de duas colunas consiste em deslocar a coluna da direita (**div#aux**) para baixo da coluna da esquerda (**div#nav**). Para isso, precisaríamos fazer as seguintes “modificações” na folha de estilo:

- A coluna **div#aux**, que fora fluída à direita para o layout de três colunas, deverá, agora, ser fluída à esquerda.

- Como a coluna esquerda agora compreende **div#nav** e **div#aux**, ambas com largura (**width**) igual a 18%, a coluna principal (**div#principal**) deverá ser redimensionada para uma largura de 82% (18% + 82% = 100%). Note que poderíamos também ter redimensionado as duas colunas laterais para outra largura (por exemplo, 20%) e adotado, para a coluna principal, a largura para completar os 100% (20% + 80% = 100%).
- A coluna principal, que fora fluída à esquerda para o layout de três colunas, deverá, agora, ser fluída à direita.

- Como as duas colunas laterais têm cores diferentes (**div#nav** é em azul e **div#aux** é em laranja), optamos por mudar a cor de **div#nav** de azul para laranja, de modo que toda a coluna tenha a mesma cor.
- O **div#tudo**, container para todo o documento, para o layout de três colunas, recebeu duas imagens de fundo (múltiplos backgrounds) para estender a cor das colunas laterais até o rodapé, quando elas não tiverem conteúdos que exijam alturas iguais para as colunas. Ver técnica faux-columns em <http://kwz.me/p1>. Para o layout de duas colunas, a imagem de fundo é única, com a parte laranja no lado esquerdo.

Essas são as regras de estilo a serem acrescentadas na folha de estilo já construída, para atender ao layout de duas colunas. Aqui entra media query, como estudamos neste capítulo, e o acréscimo na folha de estilo é o seguinte: ► **CSS**

```
/* Folha de estilos anterior para 3 colunas */
@media screen and (max-width: 760px) and (min-width: 400px) {
#aux { float:left; }
#principal { width: 82%; float:right; }
#nav { background: #fc9; }
#tudo { background: url(aux-2col.gif) 18% repeaty; }

}
```



É importante notar que esses acréscimos devem ser colocados no final da folha de estilo já existente para que o efeito cascata sobrescreva as regras anteriores. Atenção, também, para a especificidade dos seletores, por exemplo: se na folha de estilo anterior existir um seletor do tipo **#tudo #nav { ... }**, e nos acréscimos foi escrito o seletor **#nav { ... }**, essas regras não sobrescreverão aquelas, pois o primeiro seletor é mais específico que o do acréscimo.

Na figura 10.3, mostramos a renderização do layout com duas colunas, depois dos acréscimos na folha de estilo com uso da regra **@media**.

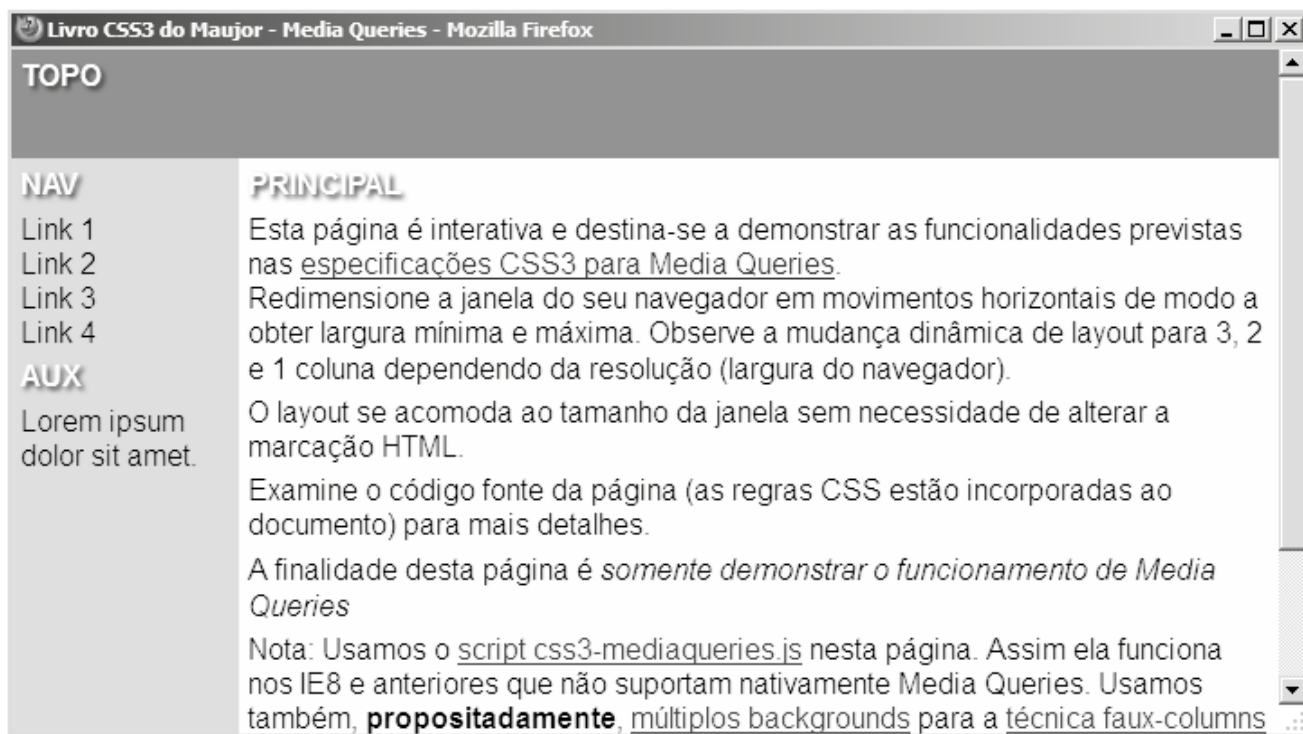


Figura 10.3 – Media query: duas colunas.

10.4.3 Terceiro passo

Continuando com a mesma marcação HTML, criar regras de estilo adicionais a serem inseridas na folha de estilo já criada no segundo passo, para construção dos layouts de duas e três colunas. No nosso caso, um terceiro layout elástico e linear para resoluções abaixo de 400px.

É importante notar que, para o layout que atenda à nova resolução, nenhuma modificação poderá ser feita tanto na marcação HTML como na folha de estilos desenvolvida no passo anterior. Havendo necessidade, neste passo, de se eliminar regras de estilo já criadas, não devemos simplesmente retirá-las da folha de estilo (o que iria quebrar o layout já construído), mas escrever outra regra de estilo que anule (sobrescreva) a anterior.

A solução para transformar o layout de duas colunas em um layout linear consiste simplesmente em se deixar fluir os boxes de conteúdos normalmente, na ordem em que eles aparecem na marcação HTML, e a eles atribuir uma largura igual a 100%. Além disso, vamos colocar a navegação em linha. Para isso, precisaríamos fazer as seguintes “modificações” na folha de estilo: • nenhuma coluna precisa ser flutuada e todas terão declaração de largura igual 100%; • não há necessidade de imagem de fundo na `div#tudo` para `faux-column`; • a navegação deverá ser tornada inline.

Essas são as regras de estilo a serem acrescentadas na folha de estilo já construída, para atender ao layout linear. Aqui entra media query, como estudamos neste capítulo, e o acréscimo na folha de estilo é o seguinte: ► **CSS**

```

/* Folha de estilos anterior para 3 e 2 colunas */
@media screen and (max-width: 400px) { #aux { float:left; }
#principal, #nav, #aux { width: 100%; float:none; }
#tudo { background-image: none }
ul li { display:inline; }

}

```



É importante notar que esses acréscimos devem ser colocados no final da folha de estilo já existente para que o efeito cascata sobrescreva as regras anteriores. Atenção, também, para a especificidade dos seletores, por exemplo: se na folha de estilo anterior existir um seletor do tipo `#tudo #nav { ... }`, e nos acréscimos foi escrito o seletor `#nav { ... }`, essas regras não sobrescreverão aquelas, pois o primeiro seletor é mais específico que o do acréscimo.

Na figura 10.4, mostramos a renderização do layout linear, depois dos acréscimos na folha de estilo com uso da regra `@media`.



Figura 10.4 – Media Query: Mobile.

10.4.4 Breakpoint

Breakpoint é uma palavra inglesa que foi incorporada ao jargão dos layouts

adaptáveis e que em tradução livre significa "pontos de quebra".

Breakpoint são os pontos em que há quebra do layout em consequência da mudança de resolução e essa definição é de mais fácil entendimento com o uso de um exemplo prático conforme descrito a seguir.

Considere o exemplo de layout que acabamos de estudar nesse item [10.4]. Ele foi construído para ser em três colunas nas resoluções acima de **1000px**, duas colunas entre **768px** e **1000px** e uma coluna para resoluções abaixo de **400px**. Estes três pontos (**400px**, **768px** e **1000px**) são os breakpoints do projeto desenvolvido no exemplo.

Mas, de onde vieram esses três valores de resolução? Como determiná-los?

É importante saber que esses valores não dependem dos dispositivos. Há uma errônea interpretação que considera as resoluções que adotamos no exemplo destinadas a servir layouts para desktop (**1000px**), para tablets (**768px**) e para dispositivos móveis (**400px**). Esse conceito apareceu logo após o lançamento de dispositivos móveis, quando os primeiros tablets e smartphones vinham com resoluções de **768px** e **400px**.

Hoje, com a diversidade daqueles dispositivos e o constante lançamento de novos modelos e resoluções, não há mais sentido naquele conceito. Projetar considerando todas as resoluções existentes é impraticável.

Assim, a maneira moderna de se determinar breakpoints é experimentalmente. Construa seu layout, visualize-o e lentamente vá redimensionando a janela do navegador, observando em que pontos o layout quebra e precisa ser adaptado. Anote as resoluções dos pontos de quebra e estes serão os breakpoints do seu projeto.

Encontrou **1180px**, **974px**, **653px** e **390px**? Sem problemas, vá em frente, pois estes são seus breakpoints.

10.5 Suporte

O suporte, nos navegadores, para as funcionalidades de Media Query, é consistente, exceto no Internet Explorer 8 e nos anteriores, que simplesmente não oferecem suporte. Contudo, o desenvolvedor *Wouter van der Graaf* criou um script em JavaScript destinado a fazer com que aqueles navegadores suportem Media Queries.

O projeto de desenvolvimento do script está no GitHub. Você pode baixá-lo em <http://kwz.me/p2> e usar nas suas aplicações com Media Query. O nome do script é *css3-mediaqueries.js*, e basta você linká-lo para a página com uso da marcação HTML mostrada a seguir que deverá ser inserida imediatamente antes da tag de fechamento do elemento **body**.

```
<!--[if lt IE 9]>  
<script type="text/javascript" src="css3-mediaqueries.js"></script> <![endif]-->
```



Para suporte nativo das funcionalidades mostradas neste capítulo, consulte o site <http://caniuse.com>.

CAPÍTULO 11

Gradientes

A especificação para o módulo denominado “CSS Image Values and Replaced Content Module Level 3”, que em tradução livre significa Módulo CSS3 para valores de imagens e conteúdos substituídos, encontra-se, à época em que esta segunda edição do livro foi escrita, em fase de Candidata a Recomendação do W3C. Essa especificação compreende duas partes: uma parte define a sintaxe para os valores CSS para imagens, e a outra parte descreve as propriedades usadas para controlar a interação de conteúdos substituídos com os algoritmos das CSS.

Valores CSS para imagens podem ser: um URI simples, uma lista de URIs, uma referência a um elemento da página ou gradientes.

As propriedades usadas para controlar a interação de conteúdos substituídos com os algoritmos das CSS podem afetar a resolução de uma imagem, a orientação de objetos substituídos ou a preservação do *aspect ratio* de um objeto.

A especificação para as CSS define elemento substituído como aquele que está fora do escopo do Modelo de formatação CSS, tal como uma imagem, um conteúdo incorporado ao documento ou um applet. Por exemplo: o conteúdo de um elemento `img` é substituído pela imagem designada pelo seu atributo `src`. Em geral, elementos substituídos têm dimensões intrínsecas.

Neste capítulo, estudaremos as funcionalidades para gradientes previstas naquele módulo.

11.1 Introdução

Gradiente é uma palavra que pode ter inúmeros significados. No contexto das CSS, gradiente é uma imagem constituída de no mínimo duas cores, na qual a transição de uma cor para outra se faz de forma gradual e suave, com uso de um efeito de esfumaçamento.

Observe, na figura 11.1, alguns exemplos de gradiente aplicados em um retângulo e construídos em um editor de imagem. Conforme a forma do gradiente, ele recebe um nome. Alguns nomes de gradiente são: linear, radial, barras, cone, retângulo, ondas *etc.*

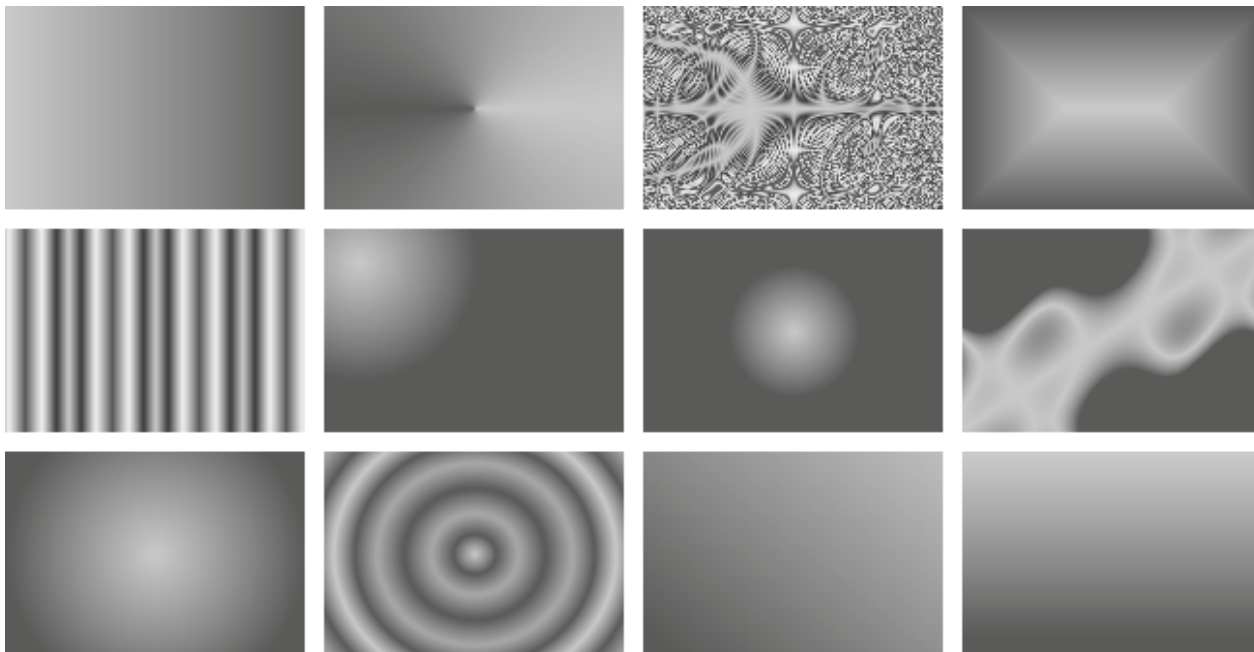


Figura 11.1 – Gradientes.

Aplicar a imagem de um gradiente como fundo de um elemento HTML é uma técnica que produz um efeito visual bem agradável e tem sido amplamente usada, principalmente como fundo da página, para apresentar uma transição suave de cor a partir do topo da página.

Com CSS2, para obter esse efeito, construímos, em um editor gráfico capaz de criar gradientes, uma imagem do gradiente. Em um retângulo com aproximadamente 10px de largura e altura capaz de acomodar a transição de cor, aplicamos o gradiente e salvamos a imagem, por exemplo, com o nome *gradiente.png*, e aplicamos a imagem como fundo do elemento `body`, criando a regra CSS mostrada a seguir:

```
body {
    background: #fff url(gradiente.png) left top repeat-x;
}
```

A figura do gradiente construída para esse exemplo foi com transição da cor cinza-clara para branca. Na figura 11.2, à esquerda, vemos a imagem criada em um editor gráfico e, à direita, o resultado da aplicação da imagem como fundo da página.

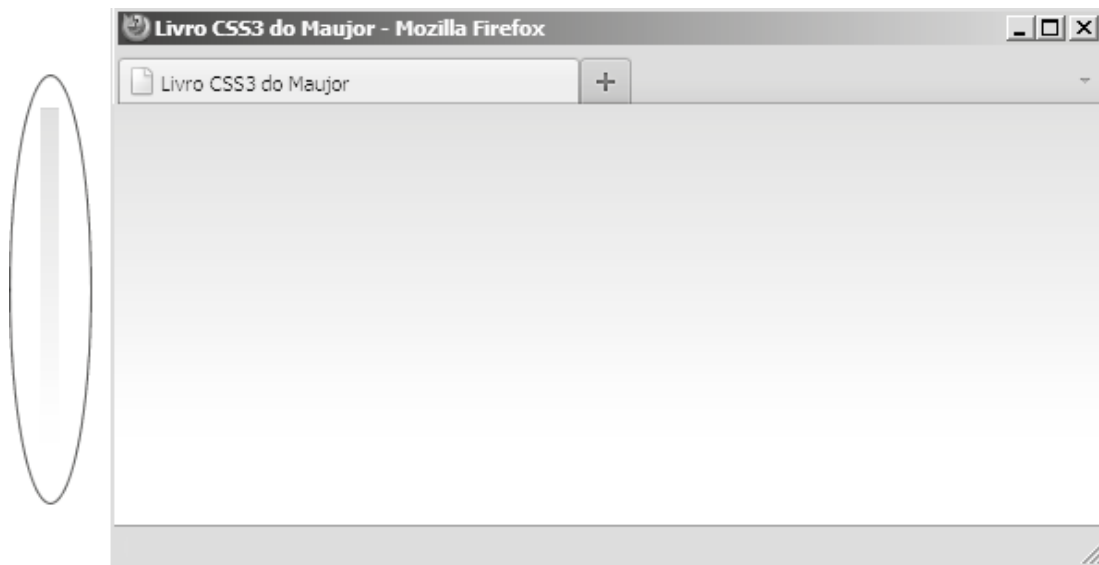


Figura 11.2 – Gradiente com imagem de fundo.

As funcionalidades previstas no módulo para valores de cores, em sua seção “Gradientes”, permitem aplicar gradientes sem necessidade de criação de uma imagem, como mostrado anteriormente. Os tipos de gradiente previstos são quatro: o linear e o radial, simples ou com repetição. Não estão previstas funcionalidades para criação de gradientes cônicos, retangulares, em ondas ou outro qualquer. Se você precisar de um gradiente não previsto, use uma imagem e aplique como previsto nas CSS2 ou, se for o caso, tente combinar gradientes lineares com radiais e obter o que você pretende. Quem sabe em futuras versões das CSS sejam criados novos tipos de gradiente.

Os navegadores atuais suportam gradientes nativamente, exceto o navegador Safari, que o suporta com uso do prefixo `-webkit-`.

Os exemplos mostrados nesse capítulo não se aplicam ao navegador Safari na versão atual, pois este navegador, além de não suportar gradientes nativamente, ainda adota sintaxe CSS diferente daquela constante das Recomendações do W3C. Se você pretende servir aquele navegador, consulte a sintaxe para gradientes no site da Apple em <http://kwz.me/5i>. Faço votos que quando você estiver lendo este livro o Safari já tenha aderido às recomendações do W3C.

11.2 Gradiente linear

Com CSS, cria-se um gradiente linear especificando uma linha ou direção e cores que serão colocadas ao longo e perpendicularmente a essa linha. A transição entre cores ao longo da linha faz-se de maneira suave, como mostrado nas imagens anteriores.

A sintaxe para a função `linear-gradient()` é mostrada a seguir: `<linear-gradient> = linear-gradient(`

```
[ [ <ângulo> | [ to <lado ou canto> ] ],,]? <color-stop>[, <color-stop>]+
)
```

sendo

```
<lado ou canto> = [left | right] || [top | bottom]
<color-stop> = <color> [ <percentage> | <length> ]?
```

Criado um gradiente com uso da função mostrada, para todos os efeitos da sintaxe CSS, ele (gradiente) é uma imagem. Assim, todas as propriedades CSS que aceitam como valor uma imagem podem ser declaradas com uso do gradiente criado. Por exemplo: `body { background-image: <linear-gradiente>; }`

```
ul { list-style-image: <linear-gradiente> }
```

11.2.1 Parâmetros da função gradiente

Estudaremos, a seguir, cada um dos parâmetros da função para criar gradientes lineares.

11.2.1.1 [[<ângulo> | [to <lado ou canto>]],]?

Esse parâmetro indica a direção do gradiente ou a *linha do gradiente*. É um parâmetro facultativo que, se for omitido, define, por padrão, a linha do gradiente na direção de cima para baixo.

Para se definir outra direção que não a padrão, podemos usar duas opções: uma medida CSS para ângulo ou a palavra-chave `to` (para) seguida do lado ou canto para o qual o gradiente vai se desenvolver.

Medidas CSS para ângulos são: `deg`, `grad`, `rad` e `turn`. Ver [11.5.4].

Os lados são `top` (superior), `right` (direito), `bottom` (inferior) e `left` (esquerdo).

Os cantos são: `top left` (superior esquerdo), `top right` (superior direito), `bottom right` (inferior direito), `bottom left` (inferior esquerdo).

Exemplos:

```
background-image: linear-gradient(45deg, black, white) background-image:
linear-gradient(to left, black, white) background-image: linear-
gradient(to top right, black, white) background-image: linear-
gradient(-50grad, black, white) background-image: linear-gradient(black,
white) /* direção padrão de cima para baixo */
```

```
background-image: linear-gradient(to bottom left, black, white) A
convenção para desenvolvimento do gradiente linear sobre a linha de
gradiente e segundo a direção especificada é a seguinte: • Quando a
direção for declarada com um valor angular, considera-se a origem para
contagem dos ângulos o ponto superior de um círculo trigonométrico, no
qual valores positivos representam ângulos no sentido horário, e
negativos, no sentido anti-horário. A direção para onde o ângulo aponta é
a direção da segunda cor declarada, e a direção oposta é a da primeira
```

cor. A figura 11.3 esclarece o desenvolvimento do gradiente para declaração angular.

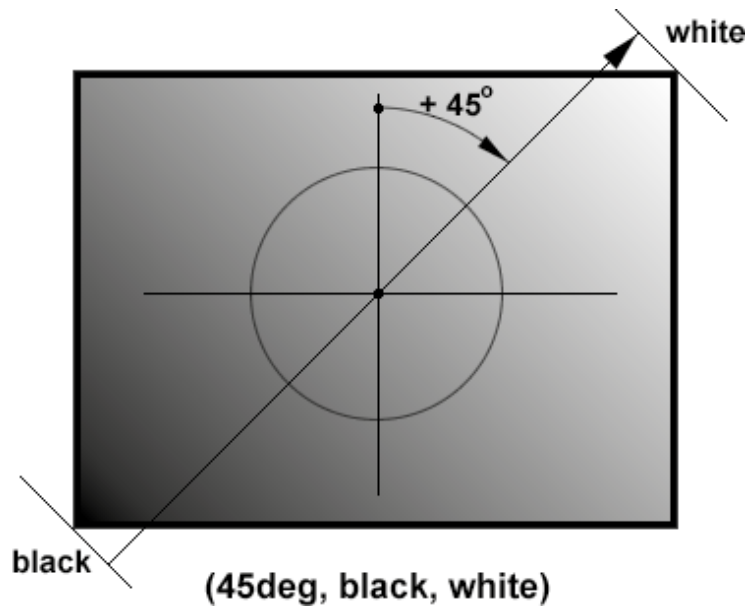


Figura 11.3 – Gradiente com direção angular.

- Quando a direção for declarada com o valor de um lado, o desenvolvimento do gradiente faz-se do lado oposto para o lado declarado. O lado declarado é a direção da segunda cor, e o lado oposto é a da primeira cor. A figura 11.4 esclarece o desenvolvimento do gradiente para declaração de lado.

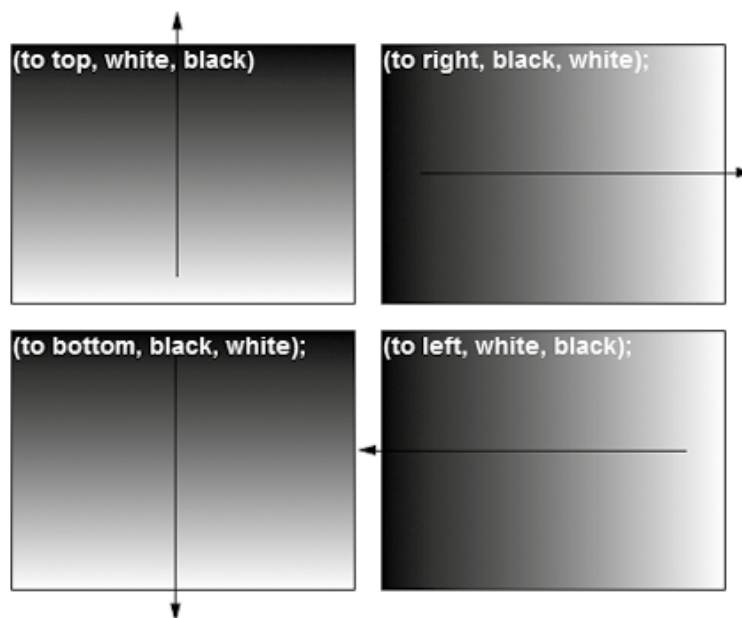


Figura 11.4 – Gradiente com direção de lado.

- Quando a direção for declarada com o valor de um canto, o desenvolvimento do gradiente faz-se do canto oposto para o canto declarado. O canto declarado é a direção da segunda cor, e o canto oposto é a da primeira cor. A figura 11.5 esclarece o desenvolvimento do gradiente para declaração de lado.

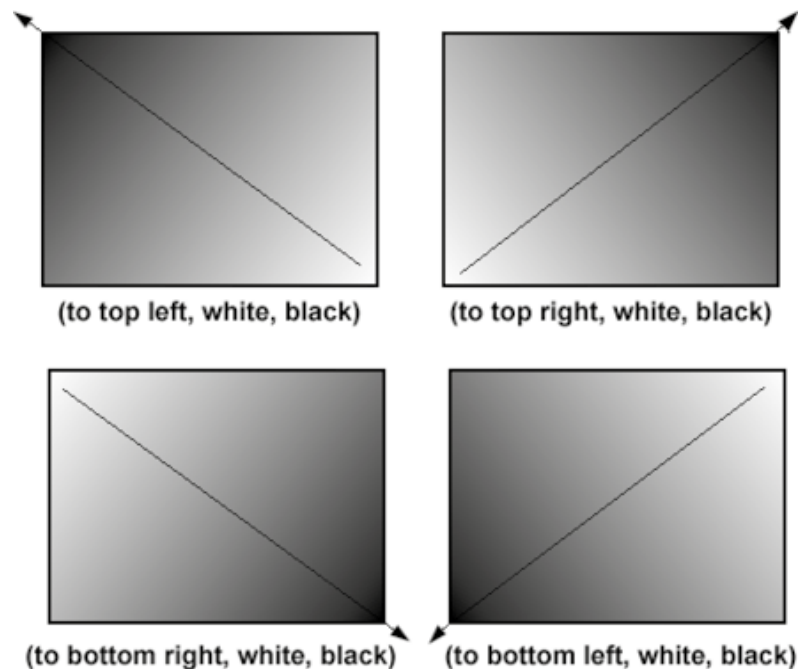


Figura 11.5 – Gradiente com direção de canto.

11.2.1.2 <color-stop>[, <color-stop>]+

Esse parâmetro define as cores do gradiente. Devem ser declaradas no mínimo duas cores.

O parâmetro <color-stop> admite a seguinte sintaxe: <color-stop> = <cor> [<percentage> | <length>]?

Segundo a sintaxe mostrada, cada uma das cores definidas para o gradiente deve ser declarada com uso de um valor CSS para cores – ver [7.2] –, seguido opcionalmente por um valor em porcentagem ou uma medida CSS que define o posicionamento da cor no eixo do gradiente.

Por padrão, as cores do gradiente se distribuem igualmente pelo comprimento disponível para sua criação, por exemplo: um gradiente com duas cores aplicado segundo um eixo de gradiente horizontal será distribuído de modo que a primeira cor comece em uma das laterais do box na qual o gradiente é aplicado e a segunda termine na lateral oposta. Podemos alterar esse comportamento declarando, opcionalmente, uma medida CSS logo após a declaração da cor e que se destina a posicionar a cor no eixo do gradiente conforme os exemplos dois, três e cinco, a seguir: Exemplos:

```
background-image: linear-gradient(to left, black, white)
background-image: linear-gradient(to right, white, black 20%)
background-image: linear-gradient(to right, white, black 180px)
background-image: linear-gradient(to left, black, white, black, white)
background-image: linear-gradient(to right, black 33%, pink 33%, pink 67%, gray 67%)
```

As regras gerais para a distribuição das cores do gradiente conforme definidas nos parâmetros <color-stop> são descritas a seguir.

- As cores se desenvolvem do início para o final do eixo do gradiente e na ordem em que são especificadas na declaração CSS.
- Valores em porcentagem são calculados tomando como referência o comprimento do eixo do gradiente. Assim 0% situa-se no ponto inicial do eixo e 100% no ponto final. Os demais valores em unidades de medida CSS, que não sejam porcentagem, seguem este mesmo critério geral.
- Normalmente as cores do gradiente são definidas para se desenvolver nos limites do box em que o gradiente será aplicado. Contudo, o eixo do gradiente se estende infinitamente para ambos os lados e assim é possível definir cores fora do box.
- Se não for definida uma medida CSS para posicionar a primeira cor declarada, ela (cor) será considerada posicionada no início do eixo do gradiente, ou seja, um valor de posição igual a 0%, e se não for definida uma medida CSS para posicionar a última cor declarada, ela (cor) será considerada posicionada no final do eixo do gradiente, ou seja, um valor de posição igual a 100%. No caso de haver mais de duas cores, haverá interpolação das cores ao longo do eixo e o posicionamento das cores será feito de acordo com número de cores declaradas. Por exemplo: para quatro cores as posições são 0%, 25%, 75% e 100%.
- Se uma determinada cor for posicionada em um ponto com coordenada menor do que aquela de uma cor qualquer declarada anteriormente, o seu posicionamento será considerado igual à maior das coordenadas anteriormente declaradas.
- Se duas cores adjacentes forem posicionadas no mesmo ponto, a transição de uma cor para outra será abrupta, ou seja, não haverá o efeito gradiente entre elas. Se mais de duas cores forem posicionadas no mesmo ponto, a transição abrupta será da primeira cor declarada para a última cor declarada.

Observe, na figura 11.6, a renderização dos cinco gradientes mostrados nos exemplos anteriores.

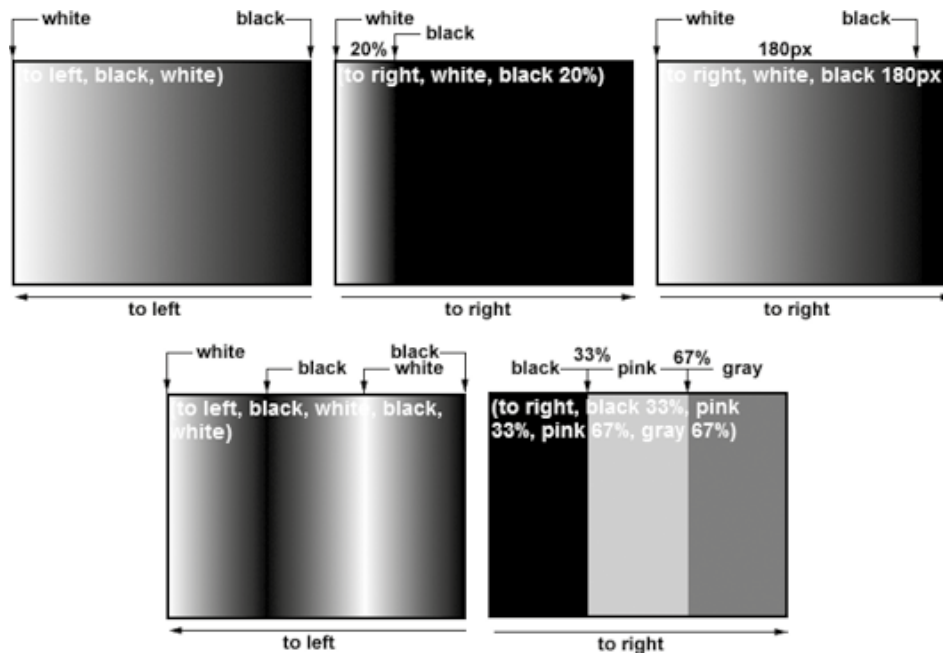


Figura 11.6 – `<color-stop>` para gradientes.

11.3 Gradiente radial

Com CSS cria-se um gradiente radial, especificando um ponto e uma forma. O ponto é o centro do gradiente, e a forma pode ser circular ou elíptica. A transição entre cores faz-se ao longo de linhas radiais a partir do centro especificado, de maneira suave, como mostrado na figura 11.7.

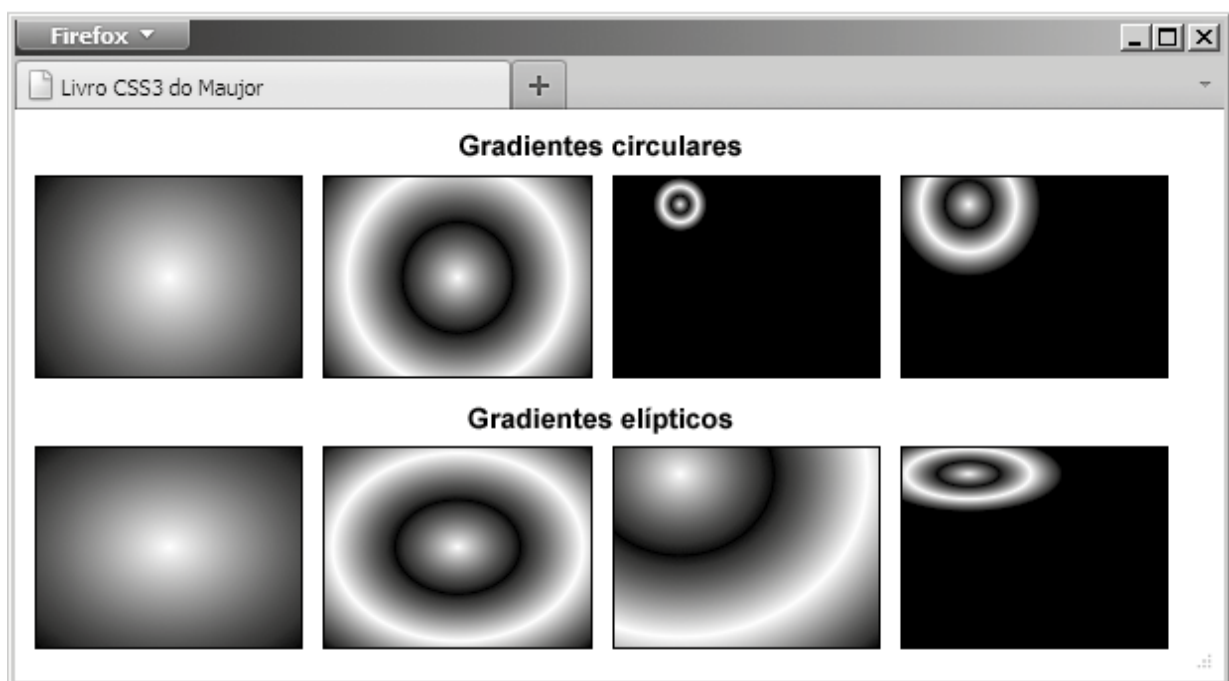


Figura 11.7 – Gradientes radiais.

A sintaxe para a função `radial-gradient()` é mostrada a seguir: `<radial-gradient> = radial-gradient([ [ <shape> || <size> ] [ at <position> ]? , | at <position>, ]?`

`<color-stop> [ , <color-stop> ]+`

sendo:

- `<position>` – define as coordenadas do centro do gradiente radial. Os valores para definir essas coordenadas são os mesmos valores CSS válidos para a propriedade `background-position`. O valor padrão é `center`.
- `<shape>` – define a forma do gradiente radial. Os valores possíveis são: `circle` e `ellipse`, para definir gradientes circulares ou radiais, respectivamente. Caso esse parâmetro seja omitido e o valor de `<size>` seja uma só medida CSS, o gradiente será circular. E se o valor de `<size>` for duas medidas CSS, o gradiente será elíptico.
- `<size>` – esse parâmetro define a forma como o gradiente se desenvolve em relação aos limites do seu elemento container. Os valores possíveis são: `closest-side`, `farthest-side`, `closest-corner` e `farthest-corner`. Alternativamente, podemos definir um valor explícito com fornecimento de coordenadas. O valor padrão é `farthest-corner`. Vejamos, a seguir, o significado de cada um desses valores:
 - `closest-side` – se a forma do gradiente for `circle`, ele deverá estender-se até o lado do container mais próximo do centro. Se a forma do gradiente for `ellipse`, ele deverá estender-se até os lados horizontal e vertical do container, mais próximos do centro;
 - `farthest-side` – se a forma do gradiente for `circle`, ele deverá estender-se até o lado do container mais afastado do centro. Se a forma do gradiente for `ellipse`, ele deverá estender-se até os lados horizontal e vertical do container, mais afastados do centro;
 - `closest-corner` – se a forma do gradiente for `circle`, ele deverá estender-se até o canto do container mais próximo do centro. Se a forma do gradiente for `ellipse`, ele deverá estender-se até o canto do container mais próximo do centro, mas mantendo o mesmo aspect-ratio;
 - `farthest-corner` – se a forma do gradiente for `circle`, ele deverá estender-se até o canto do container mais afastado do centro. Se a forma do gradiente for `ellipse`, ele deverá estender-se até o canto do container mais afastado do centro, mas mantendo o mesmo aspect-ratio;Se o parâmetro `<shape>` for omitido ou for definido como `circle` o parâmetro `<size>` deverá ser explicitamente expresso por uma medida CSS `<length>` (menos porcentagem que não é válida), que representa o raio do círculo do gradiente.

Se o parâmetro `<shape>` for definido como `ellipse` o parâmetro `<size>` deverá ser explicitamente expresso por duas medidas CSS `<length>` (porcentagem é válida), que representam os raios horizontal e vertical da ellipse do gradiente, respectivamente. No caso de porcentagem o valor é calculado em relação às dimensões do box em que o gradiente é aplicado.

- `<color-stop>` – define a cor do gradiente radial e, opcionalmente, uma medida

CSS estabelecendo os limites da cor. A sintaxe geral é idêntica à dos gradientes lineares: <color-stop> = <color> [<percentage> | <length>]?

Criado um gradiente com uso da função mostrada, para todos os efeitos da sintaxe CSS, ele (gradiente) é uma imagem. Assim, todas as propriedades CSS que aceitam como valor uma imagem podem ser declaradas com uso do gradiente criado. Por exemplo: `body { background-image: <radial-gradiente>; }`

```
ul { list-style-image: <radial-gradiente> }
```

O exemplo a seguir mostra a aplicação de gradiente radial circular como fundo de um `div`. Cada um dos gradientes adota um valor para o parâmetro `size`, como descrito anteriormente.

```
div {
  color: #black;
  width: 200px;
  height: 150px;
  border: 2px solid #000;
  float: left;
  margin: 30px 15px 15px 0;
  display: table-cell;
  vertical-align: middle; }
#um { /* closest-side */
  background-image: radial-gradient(circle closest-side at 40px 60px, white,
  black, white); }
#dois { /* farthest-side */
  color: white;
  background-image: radial-gradient(circle farthest-side at 40px 60px,
  white, black, white); }
#tres { /* closest-corner */
  background-image: radial-gradient(circle closest-corner at 40px 60px,
  white, black, white); }
#quatro { /* farthest-corner */
  color: white;
  background-image: radial-gradient(circle farthest-corner at 40px 60px,
  white, black, white); }
#cinco { /* sem declaração de shape */
  background-image: radial-gradient(40px, white, black, white);

}

#seis { /* default, farthest-corner */
  color: white;
  background-image: radial-gradient(circle at 40px 60px, white, black,
  white); }
```

Observe, na figura 11.8, os gradientes radiais circulares do exemplo. O navegador Safari não suporta gradientes radiais definidos com a sintaxe prevista no W3C,

conforme explicado quando estudamos gradientes lineares.

O exemplo a seguir mostra a aplicação de gradiente elíptico como fundo de um div. Cada um dos gradientes adota um valor para o parâmetro size, como descrito anteriormente.

```
div {
  color: black;
  width: 200px;
  height: 150px;
  border: 2px solid #000;
  float: left;
  margin: 30px 15px 15px 0;
  display: table-cell;
  vertical-align: middle; }
#um { /* closest-side */
  background-image: radial-gradient(ellipse closest-side at 40px 60px,
  white, black, white); }
#dois { /* farthest-side */
  color: white;
  background-image: radial-gradient(ellipse farthest-side at 40px 60px,
  white, black, white); }
#tres { /* closest-corner */
  background-image: radial-gradient(ellipse closest-corner at 40px 60px,
  white, black, white); }
#quatro { /* farthest-corner */
  color: white;
  background-image: radial-gradient(ellipse farthest-corner at 40px 60px,
  white, black, white); }
#cinco { /* sem declaração de shape */
  background-image: radial-gradient(40px 60px, white, black, white);

}

#seis { /* default, farthest-corner */
  color: white;
  background-image: radial-gradient(ellipse at 40px 60px, white, black,
  white); }
```

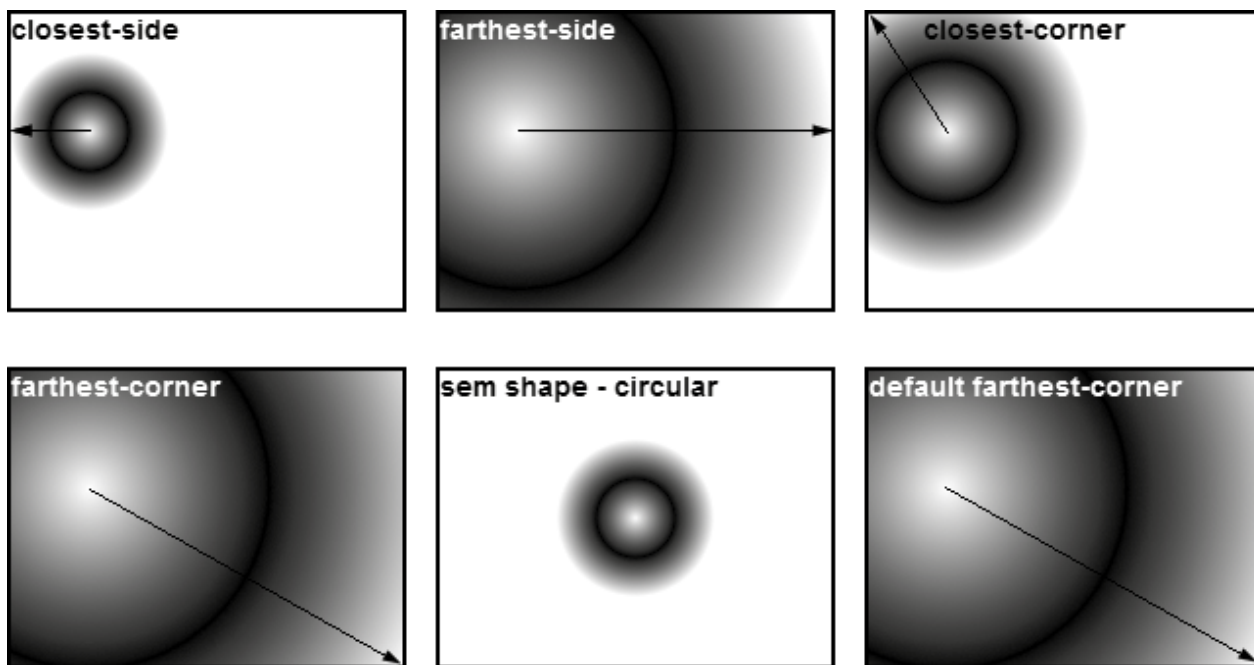


Figura 11.8 – Size para gradientes radiais circulares.

Observe, na figura 11.9, os gradientes radiais elípticos do exemplo. O navegador Safari não suporta gradientes radiais definidos com a sintaxe prevista no W3C, conforme explicado quando estudamos gradientes lineares.

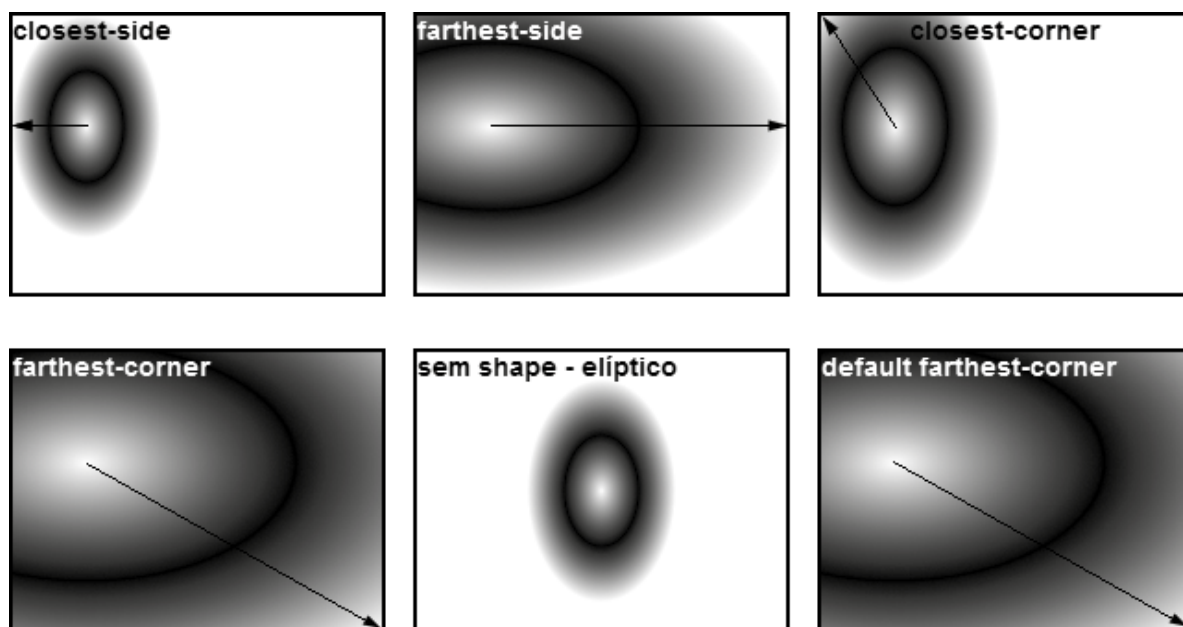


Figura 11.9 – Size para gradientes radiais elípticos.

11.5 Repetição de gradientes

As especificações criaram uma funcionalidade que permite repetir um gradiente, quer seja linear, quer radial. Essa funcionalidade aplica-se a gradientes aos quais tenham sido definidos pontos de interrupção de cores do gradiente. Ela faz com que o gradiente se repita indefinidamente a partir do ponto no qual foi interrompido.

A sintaxe para aplicação de gradiente repetido é idêntica à sintaxe já estudada neste capítulo para gradientes circulares e radiais simples, com acréscimo da palavra `repeating` no nome da função. Observe a sintaxe geral mostrada a seguir: `repeating-linear-gradient()`

`repeating-radial-gradient()`

O exemplo a seguir mostra a aplicação de gradiente repetido como fundo de um `div`. Nos `div#um` e `div#tres`, aplicou-se um gradiente simples, e nos `div#dois` e `div#quatro`, o mesmo gradiente com repetição. Nos `div#cinco` e `div#seis`, aplicaram-se gradientes com repetição linear e radial, respectivamente.

```
div {
  color: #fff;
  width: 200px;
  height: 150px;
  border: 2px solid #000;
  float: left;
  margin: 30px 15px 15px 0;

}

#um {
  background: linear-gradient(to right, black, white 10px, black 20px); }
#dois {
  background: repeating-linear-gradient(to right, black, white 10px, black
20px); }
#tres {
  background: radial-gradient(ellipse at 50% 50%, black, white 10px, black
20px); }
#quatro {
  background: repeating-radial-gradient(ellipse at 50% 50%, black, white
10px, black 20px); }
#cinco {
  background: repeating-linear-gradient(-45deg, black, white 2px, black 5px,
white 20px); }
#seis {
  background: repeating-radial-gradient(ellipse at 100% 50%, black, white
10px, black 20px); }
```

Observe, na figura 11.10, o efeito da aplicação dos gradientes com repetição, conforme mostrado no código anterior. O navegador Safari não suporta repetição de gradientes definidos com a sintaxe prevista no W3C.

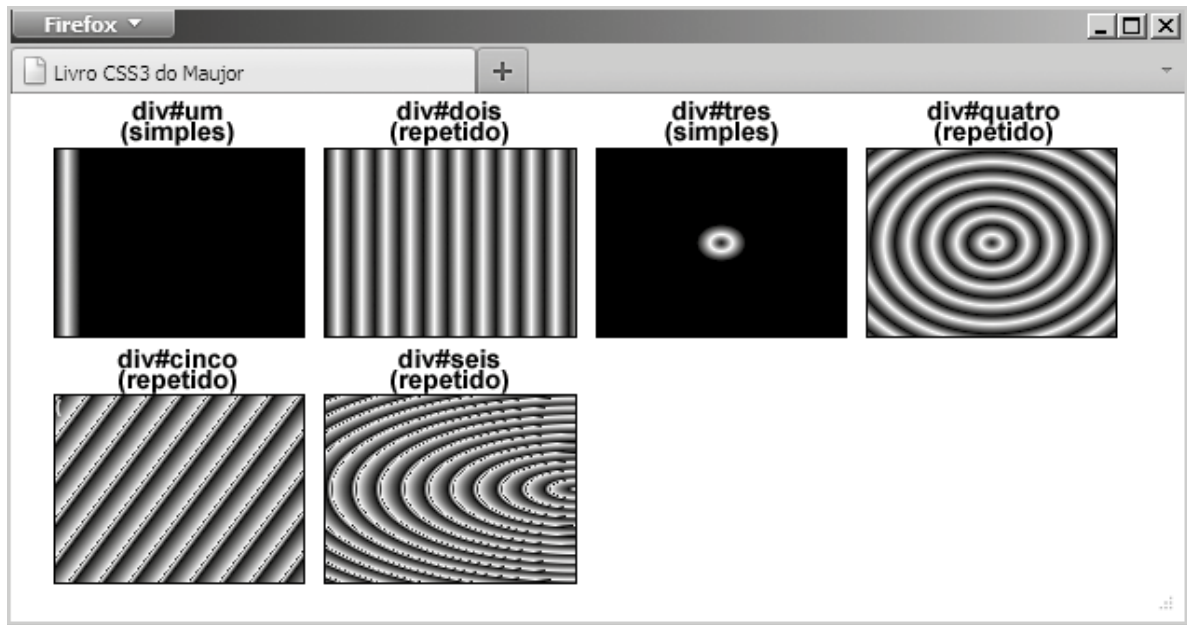


Figura 11.10 – Gradientes repetidos.

CAPÍTULO 12

Transformações 2D

As especificações para transformações CSS estavam divididas em dois módulos independentes até 28 de fevereiro de 2012. Um para transformações 2D e outro para transformações 3D. A partir daquela data foram reunidas em especificação única denominada CSS Transforms, quando esta edição do livro foi escrita estava no estágio de Rascunho de Trabalho..

Neste capítulo, estudaremos as funcionalidades para transformações 2D. Os navegadores modernos já as suportam com uso de prefixos proprietários, e assim podemos começar a realizar nossos estudos e nossas experiências com essas transformações. No capítulo 13, estudaremos as transformações 3D.

12.1 Introdução

O Box Model tradicional das CSS estabelece que cada elemento da marcação cria um espaço retangular que se acomoda no fluxo do documento verticalmente, se elementos nível de bloco, ou na horizontal, se elementos inline. As funcionalidades de posicionamento das CSS2 permitem que o autor interfira nesse comportamento padrão, retirando um elemento da sua posição e posicionando-o em lugar diferente na página. Tais funcionalidades são capazes de definir deslocamentos dos boxes CSS segundo os eixos do sistema de coordenadas, ou seja, na vertical, na horizontal ou simultaneamente nas duas direções.

Assim, com CSS2 é possível, por exemplo, retirar um box do canto inferior direito da tela e posicioná-lo no centro da parte superior, ou mesmo logo após o topo do site. Esse é um procedimento comum e tem sido amplamente empregado quando o autor inclui a navegação no final da marcação e posiciona-a no início da página.

As CSS3, neste módulo, estenderam as funcionalidades de posicionamento, possibilitando aos autores movimentar os boxes em direções diferentes de horizontal e vertical. Com uso delas é possível não só movimentar conforme previsto nas CSS2, com translações verticais e horizontais, mas também rotacionar, mudar escala, inclinar horizontal e verticalmente e criar transformações compostas.

À época em que foi escrita a primeira edição deste livro os navegadores, incluindo-se o IE9, suportavam transformações com uso dos prefixos proprietários `-moz-`, `-webkit-`, `-o-` e `-ms-` para Firefox, Chrome/Safari, Opera e IE9, respectivamente.

Já quando da presente edição os navegadores Firefox 23 e Internet Explorer 10 suportavam as transformações 2D nativamente e os navegadores Chrome 29, Opera 16 e Safari 6 as suportavam com o uso do prefixo `-webkit-`.

Uma vez que as especificações encontram-se em fase de Rascunho de Trabalho, vamos manter no livro a sintaxe usada à época do lançamento da primeira edição. Para saber mais sobre suporte consulte <http://caniuse.com/#feat=transforms2d>.

12.2 Sistema de coordenadas

Sabemos que o sistema de coordenadas CSS (e HTML) adota como origem o canto superior esquerdo do box. A coordenada x ou coordenada horizontal cresce (valores positivos) da origem para a direita, e decresce (valores negativos) da origem para a esquerda. O mesmo se passa com a coordenada vertical y, que cresce para baixo e decresce para cima.

As funções de transformação, quando aplicadas a um box, disparam um mecanismo que altera seu sistema de coordenadas, posicionando e redimensionando o box de acordo com os parâmetros da função.

Vejamos um exemplo prático. Considere a função de transformação para rotacionar um box de um ângulo igual a 20 graus. Lembre-se que medidas de ângulos CSS começam sua contagem (valor 0) na vertical, crescem (valores positivos) no sentido horário e decrescem (valores negativos) no sentido anti-horário.

Observe, na figura 12.1, a ilustração do mecanismo de aplicação de uma transformação rotacional de 20 graus.

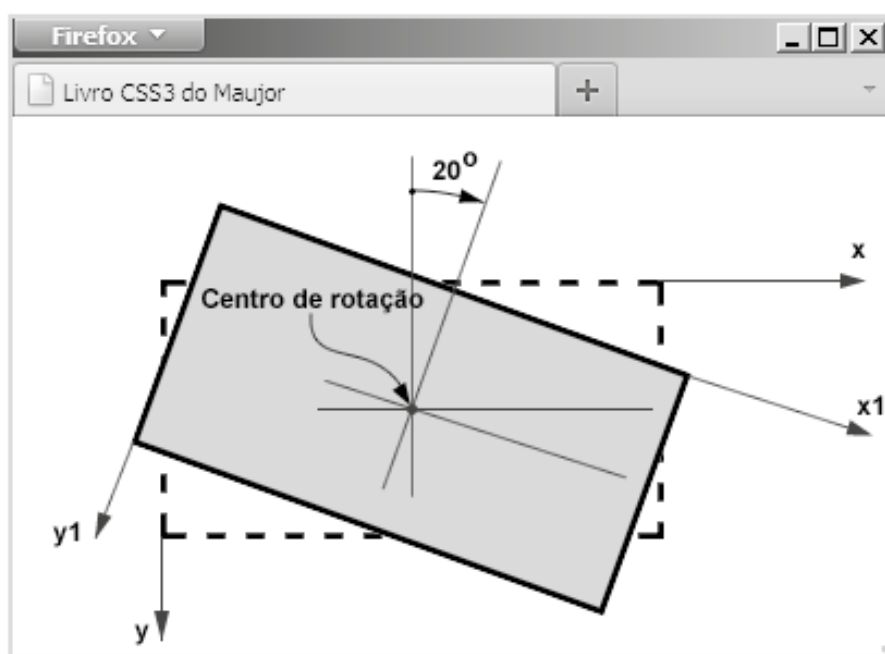


Figura 12.1 – Mecanismo das transformações.

Na figura, mostramos, em linha tracejada, a posição original do box, e nele o sistema de coordenadas tradicional xy. Com fundo cinza, o mesmo box, após ter sido aplicada uma rotação positiva de 20 graus (sentido horário) e seu novo sistema de coordenadas x1y1. Note que destacamos ainda o centro de rotação que por

padrão é o centro do box.

A finalidade da figura 12.1 é apenas ilustrar o mecanismo de aplicação de transformações CSS3. A seguir, iremos estudar com detalhes as funcionalidades do módulo.



Ressalto desde já e recomendo que não se esqueçam de que uma transformação qualquer altera o posicionamento de um box, mas não interfere com o posicionamento de outros boxes no fluxo do documento. Esse comportamento é idêntico àquele que conhecemos das CSS2, quando retiramos um box de sua posição com uso da propriedade **position** com valor **relative**.

12.3 Transformações

Estão previstas as seguintes funções para transformações 2D: • **rotate()**

- **translateX()**
- **translateY()**
- **translate ()**
- **scaleX()**
- **scaleY()**
- **scale()**
- **skewX()**
- **skewY()**
- **skew()**
- **matrix()**

Vejamos cada uma delas detalhadamente.

12.3.1 rotate()

Essa função destina-se a rotacionar um box, e sua sintaxe é conforme mostrado a seguir: `seletor { transform: rotate(a); }`

O parâmetro **a** para essa função é uma medida CSS para ângulos. Medidas CSS para ângulos são: **deg**, **grad**, **rad** e **turn**. Ver [1.1.5.4].

Na figura 12.2, mostramos o efeito de rotacionar um box cabeçalho seguido de um box parágrafo, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
```



```
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
margin: 80px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: rotate(-30deg); /* Firefox */
-webkit-transform: rotate(-30deg); /* Chrome e Safari */
-o-transform: rotate(-30deg); /* Opera */
-ms-transform: rotate(-30deg); /* IE9+ */
transform: rotate(-30deg); /* Suporte nativo */
background: #ccc;
height: 40px;
line-height: 40px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
</style>
```



Figura 12.2 – Rotacionar um box.

Na figura, mostramos a renderização do código antes e depois da rotação. Note que a rotação do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo

elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.2 translateX()

Essa função destina-se a deslocar um box na horizontal, e sua sintaxe é conforme mostrado a seguir: seletor { transform: translateX(x); }

O parâmetro *x* para essa função é uma medida CSS de comprimento. Para medidas CSS de comprimento, ver [1.1.5.4].

Não há um ponto de referência único para essa transformação, pois todos os pontos do box se deslocam igualmente na direção da transformação.

Na figura 12.3, mostramos o efeito de deslocar, **80px** para a direita, um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ►

HTML

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
margin: 80px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: translateX(80px); /* Firefox */
-webkit-transform: translateX(80px); /* Chrome e Safari */
-o-transform: translateX(80px); /* Opera */
-ms-transform: translateX(80px); /* IE9+ */
transform: translateX(80px); /* Suporte nativo */
background: #ccc;
height: 40px;
line-height: 40px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
```

</style>

Na figura, mostramos a renderização do código antes e depois do deslocamento horizontal. Note que o deslocamento do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.



Figura 12.3 – Translação horizontal de um box.

12.3.3 translateY()

Essa função destina-se a deslocar um box na vertical, e sua sintaxe é conforme mostrado a seguir: seletor { transform: translateY(y); }

O parâmetro *y* para essa função é uma medida CSS de comprimento. Para medidas CSS de comprimento, ver [1.1.5.4].

Não há um ponto de referência único para essa transformação, pois todos os pontos do box se deslocam igualmente na direção da transformação.

Na figura 12.4, mostramos o efeito de deslocar, **100px** para baixo, um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ►

HTML

```
<div>
<h2>Sem dúvidas, eu ♥ CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► **CSS**

```
<style>
body {
font: 14px sans-serif;
```

```
margin: 80px 0 0 20px;
```

```
}
```

```
div { width: 300px; }
```

```
h2 {
```

```
-moz-transform: translateY(100px); /* Firefox */
```

```
-webkit-transform: translateY(100px); /* Chrome e Safari */
```

```
-o-transform: translateY(100px); /* Opera */
```

```
-ms-transform: translateY(100px); /* IE9+ */
```

```
transform: translateY(100px); /* Suporte nativo */
```

```
background: #ccc;
```

```
height: 40px;
```

```
line-height: 40px;
```

```
text-align: center;
```

```
opacity: .8;
```

```
}
```

```
p { margin: 0; text-align: justify; }
```

```
</style>
```



Figura 12.4 – Translação vertical de um box.

Na figura, mostramos a renderização do código antes e depois do deslocamento vertical. Note que o deslocamento do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.4 translate()

Essa função destina-se a deslocar um box na horizontal e na vertical ao mesmo tempo, e sua sintaxe é conforme mostrado a seguir: seletor { transform: translate(x,

```
y); }
```

Os parâmetros *x* e *y* para essa função são medidas CSS de comprimento. Para medidas CSS de comprimento, ver [1.1.5.4].

Caso um dos parâmetros seja omitido, o parâmetro presente representa uma translação horizontal, e o outro parâmetro será considerado 0 (zero), ou seja, não haverá translação vertical, ou ainda, essa função com um parâmetro é equivalente à função `translateX()`.

Não há um ponto de referência único para essa transformação, pois todos os pontos do box se deslocam igualmente na direção da transformação.

Na figura 12.5, mostramos o efeito de deslocar, **80px** para a direita e **100px** para baixo, um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► **CSS**

```
<style>
body {
font: 14px sans-serif;
margin: 80px 0 0 20px;
```

```
}
```

```
div { width: 300px; }
h2 {
-moz-transform: translate(80px, 100px); /* Firefox */
-webkit-transform: translate(80px, 100px); /* Chrome e Safari */
-o-transform: translate(80px, 100px); /* Opera */
-ms-transform: translate(80px, 100px); /* IE9+ */
transform: translate(80px, 100px); /* Suporte nativo */
background: #ccc;
height: 40px;
line-height: 40px;
text-align: center;
opacity: .8;
```

```
}
```

```
p { margin: 0; text-align: justify; }
</style>
```

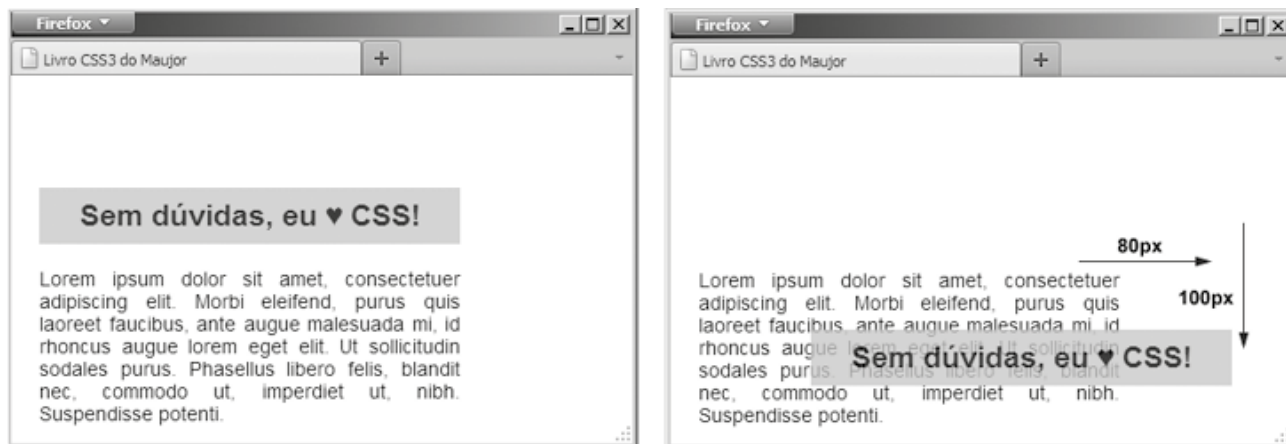


Figura 12.5 – Translação horizontal e vertical de um box.

Na figura, mostramos a renderização do código antes e depois dos deslocamentos. Note que o deslocamento do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.5 scaleX()

Essa função destina-se a expandir ou contrair a dimensão horizontal de um box, e sua sintaxe é conforme mostrado a seguir: seletor { transform: scaleX(n); }

O parâmetro *n* para essa função é um número positivo decimal maior que 1 (expansão) ou menor que 1 (contração), que define o número de vezes que a medida horizontal será expandida ou contraída.

A referência padrão para essa transformação é o ponto central do box, que permanece o mesmo (fixo) antes e depois da transformação.

Na figura 12.6, mostramos os efeitos de contração horizontal de um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu ♥ CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
```

```
margin: 20px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: scaleY(.6); /* Firefox */
-webkit-transform: scaleY(.6); /* Chrome e Safari */
-o-transform: scaleY(.6); /* Opera */
-ms-transform: scaleY(.6); /* IE9+ */
transform: scaleY(.6); /* Suporte nativo */
background: #ccc;
height: 40px;
line-height: 40px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
</style>
```



Figura 12.6 – Contração horizontal de um box.

Na figura, mostramos a renderização do código antes e depois da contração horizontal. Note que a contração do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.6 scaleY()

Essa função destina-se a expandir ou contrair a dimensão vertical de um box, e sua

sintaxe é conforme mostrado a seguir: seletor { transform: scaleY(n); }

O parâmetro *n* para essa função é um número positivo decimal maior que 1 (expansão) ou menor que 1 (contração), que define o número de vezes que a medida vertical será expandida ou contraída.

A referência padrão para essa transformação é o ponto central do box, que permanece o mesmo (fixo) antes e depois da transformação.

Na figura 12.7, mostramos os efeitos da expansão vertical de um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► **CSS**

```
<style>
body {
font: 14px sans-serif;
margin: 20px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: scaleY(3); /* Firefox */
-webkit-transform: scaleY(3); /* Chrome e Safari */
-o-transform: scaleY(3); /* Opera */
-ms-transform: scaleY(3); /* IE9+ */
transform: scaleY(3); /* Suporte nativo */
background: #ccc;
height: 40px;
line-height: 40px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
</style>
```




Figura 12.7 – Expansão vertical de um box.

Na figura, mostramos a renderização do código antes e depois da expansão vertical. Note que a expansão do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.7 scale()

Essa função destina-se a expandir ou contrair as dimensões horizontal e vertical de um box, e sua sintaxe é conforme mostrado a seguir: seletor { transform: scale(n, m); }

O parâmetro *n* define a transformação horizontal, e o parâmetro *m*, a transformação vertical. Ambos os parâmetros dessa função são um número positivo decimal maior que 1 (expansão) ou menor que 1 (contração), que define o número de vezes que as medidas serão expandidas ou contraídas.

Caso um dos parâmetros seja omitido, os dois parâmetros serão considerados iguais ao parâmetro declarado, e a contração ou expansão será a mesma nas direções horizontal e vertical.

A referência padrão para essa transformação é o ponto central do box, que permanece o mesmo (fixo) antes e depois da transformação.

Na figura 12.8, mostramos os efeitos da transformação, em ambas as direções, de um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu ♥ CSS!</h2>
<p>Lorem ipsum ...</p>
```

```
</div>
```

► CSS

```
<style>
```

```
body {
```

```
font: 14px sans-serif;
```

```
margin: 20px 0 0 20px;
```

```
}
```

```
div { width: 300px; }
```

```
h2 {
```

```
-moz-transform: scale(.6, 2); /* Firefox */
```

```
-webkit-transform: scale(.6, 2); /* Chrome e Safari */
```

```
-o-transform: scale(.6, 2); /* Opera */
```

```
-ms-transform: scale(.6, 2); /* IE9+ */
```

```
transform: scale(.6, 2); /* Suporte nativo */
```

```
background: #ccc;
```

```
height: 40px;
```

```
line-height: 40px;
```

```
text-align: center;
```

```
opacity: .8;
```

```
}
```

```
p { margin: 0; text-align: justify; }
```

```
</style>
```



Figura 12.8 – Contração horizontal e expansão vertical de um box.

Na figura, mostramos a renderização do código antes e depois da transformação. Note que a transformação do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do

documento.

12.3.8 skewX()

Essa função destina-se a inclinar, segundo o eixo x, um box, e sua sintaxe é conforme mostrado a seguir: seletor { transform: skewX(a); }

O parâmetro *a* para essa função é uma medida CSS para ângulos. Medidas CSS para ângulos são: **deg**, **grad**, **rad** e **turn**. Ver [1.1.5.4].

A linha de referência para essa transformação é o eixo horizontal que passa pelo centro do box. Valores positivos de ângulos inclinam para a esquerda, e negativos, para a direita.

Na figura 12.9, mostramos o efeito de inclinar em 30 graus, segundo o eixo x, um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir:

► HTML

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
margin: 20px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: skewX(30deg); /* Firefox */
-webkit-transform: skewX(30deg); /* Chrome e Safari */
-o-transform: skewX(30deg); /* Opera */
-ms-transform: skewX(30deg); /* IE9+ */
transform: skewX(30deg); /* Suporte nativo */
background: #ccc;
height: 80px;
line-height: 80px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
</style>
```

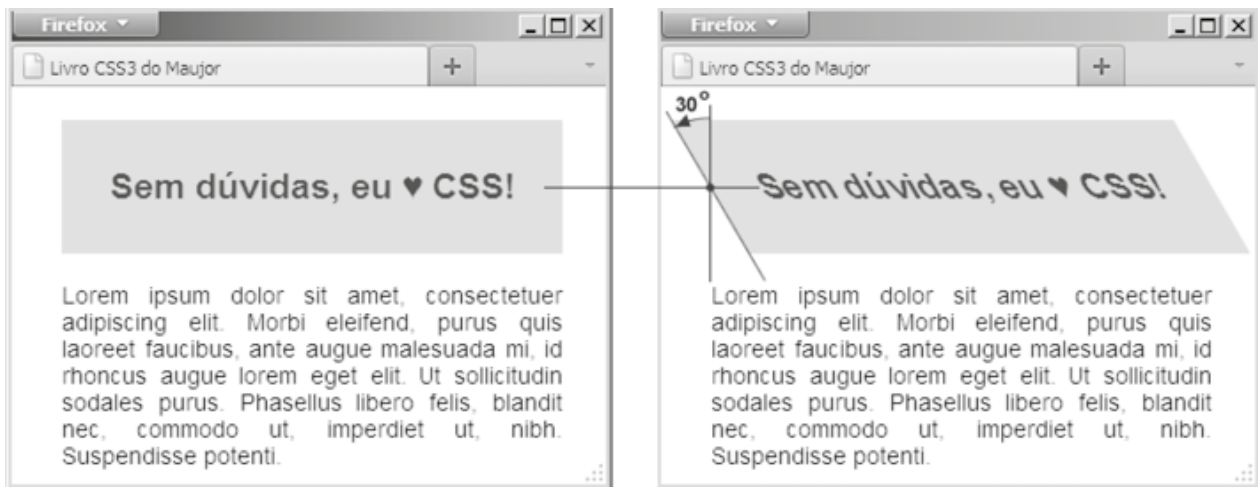


Figura 12.9 – Inclinação de um box no eixo x.

Na figura, mostramos a renderização do código antes e depois da inclinação segundo o eixo x. Note que a inclinação do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.9 skewY()

Essa função destina-se a inclinar, segundo o eixo y, um box, e sua sintaxe é conforme mostrado a seguir: seletor { transform: skewY(a); }

O parâmetro **a** para essa função é uma medida CSS para ângulos. Medidas CSS para ângulos são: **deg**, **grad**, **rad** e **turn**. Ver [1.1.5.4].

A linha de referência para essa transformação é o eixo vertical que passa pelo centro do box. Valores negativos de ângulos inclinam para a esquerda, e positivos, para a direita.

Na figura 12.10, mostramos o efeito de inclinar em -20 graus, segundo o eixo y, um box cabeçalho, seguido de um box parágrafo, conforme códigos mostrados a seguir:

► HTML

```
<div>
<h2>Sem dúvidas, eu ♥ CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
```

```
margin: 20px 0 0 20px;
```

```
}
```

```
div { width: 300px; }
```

```
h2 {
```

```
-moz-transform: skewY(-20deg); /* Firefox */
```

```
-webkit-transform: skewY(-20deg); /* Chrome e Safari */
```

```
-o-transform: skewY(-20deg); /* Opera */
```

```
-ms-transform: skewY(-20deg); /* IE9+ */
```

```
transform: skewY(-20deg); /* Suporte nativo */
```

```
background: #ccc;
```

```
height: 80px;
```

```
line-height: 80px;
```

```
text-align: center;
```

```
opacity: .8;
```

```
}
```

```
p { margin: 0; text-align: justify; }
```

```
</style>
```

Na figura, mostramos a renderização do código antes e depois da inclinação vertical. Note que a inclinação do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.



Figura 12.10 – Inclinação de um box no eixo y.

12.3.10 skew()

Essa função destina-se a deslocar um box na horizontal e na vertical ao mesmo

tempo, e sua sintaxe é conforme mostrado a seguir: seletor { transform: skew(a, b); }

Os parâmetros **a** e **b** para essa função são medidas CSS para ângulos. Medidas CSS para ângulos são: **deg**, **grad**, **rad** e **turn**. Ver [1.1.5.4].

Caso um dos parâmetros seja omitido, o parâmetro presente representa uma translação horizontal, e o outro parâmetro será considerado 0 (zero), ou seja, não haverá translação vertical, ou ainda, essa função com um parâmetro é equivalente à função **skewX()**.

As linhas de referência para a inclinação são como mostradas anteriormente, para inclinação segundo os eixos **x** e **y**.

Na figura 12.11, mostramos o efeito de inclinar um box cabeçalho, seguido de um box parágrafo, 30 graus segundo o eixo horizontal e -20 graus segundo o eixo vertical, conforme códigos mostrados a seguir: ► **HTML**

```
<div>
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► **CSS**

```
<style>
body {
font: 14px sans-serif;
margin: 20px 0 0 20px;

}

div { width: 300px; }
h2 {
-moz-transform: skew(30deg, -20deg); /* Firefox */
-webkit-transform: skew(30deg, -20deg); /* Chrome e Safari */
-o-transform: skew(30deg, -20deg); /* Opera */
-ms-transform: skew(30deg, -20deg); /* IE9+ */
transform: skew(30deg, -20deg); /* Suporte nativo */
background: #ccc;
height: 80px;
line-height: 80px;
text-align: center;
opacity: .8;

}

p { margin: 0; text-align: justify; }
</style>
```



Figura 12.11 – Inclinação de um box nos eixos x e y.

Na figura, mostramos a renderização do código antes e depois das inclinações. Note que a inclinação do cabeçalho, tal como acontece com qualquer função de transformação, não afetou o posicionamento do parágrafo. Em uma transformação, para efeito de layout, tudo se passa como se o lugar inicialmente ocupado pelo elemento transformado fosse preservado no fluxo do documento, e ele, transformado, fosse colocado em uma nova camada sobreposta aos conteúdos do documento.

12.3.11 matrix()

Essa função destina-se a aplicar qualquer uma, ou mais de uma simultaneamente, das funções estudadas anteriormente neste capítulo. O algoritmo de aplicação das transformações baseia-se em cálculo matricial, e sua explicação teórica é bastante complexa e extensa para constar deste livro. Por essa razão, mostraremos a sintaxe para aplicação da função sem entrar em detalhes matemáticos. Se você estiver interessado em conhecer esses detalhes, recomendo começar seus estudos com a leitura dos seguintes documentos, disponíveis na web: <http://kwz.me/pQ> (Wikipédia) e <http://kwz.me/pR> (W3C).

A sintaxe para essa função é mostrada a seguir:

```
seletor { transform: matrix(a, b, c, d, e, f); }
```

Os seis parâmetros são números que definem o tipo e as características da transformação. O valor inicial de cada parâmetro é 0 (zero), e a transformação é obtida conforme a combinação dos parâmetros que se definem.

A seguir, listamos o tipo de valor, a finalidade de cada um dos parâmetros e seu significado para definição de uma transformação.

Valor	Tipo	Descrição
-------	------	-----------

a	número	Fator de escala no eixo x. Equivale a scaleX(a) .
b	número	Valor da tangente do ângulo a inclinar segundo o eixo y. Equivale a skewY(b) .
c	número	Valor da tangente do ângulo a inclinar segundo o eixo x. Equivale a skewX(c) .
d	número	Fator de escala no eixo y. Equivale a scaleY(d) .
e	comprimento CSS	Define o deslocamento no eixo x. Equivale a translateX(e) .
f	comprimento CSS	Define o deslocamento no eixo y. Equivale a translateY(f) .

O navegador Firefox implementou a sintaxe para os parâmetros **e** e **f**, que definem uma translação, com uso da unidade de medida. Para os demais navegadores, a unidade de medida, que não deve ser declarada, é o pixel. Dessa forma, translações em medidas que não sejam em pixel só são possíveis no navegador Firefox. A Mozilla já solicitou ao W3C que adote a sintaxe com definição da unidade. Restamos aguardar, para saber como essa questão vai evoluir nas especificações.

Observe, a seguir, alguns exemplos comentados que esclarecem a obtenção de transformações com uso da função **matrix()**: `transform: matrix(2,0,0,1,0,0);` /* equivale a `scaleX(2)` */

```
transform: matrix(1,0,0,.8,0,0); /* equivale a scaleY(.8) */
transform: matrix(1.5,0,0,3,0,0); /* equivale a scale (1.5, 3) */
transform: matrix(1,0,0,1,90,0); /* equivale a translateX(90px) */
transform: matrix(1,0,0,1,0,120); /* equivale a translateY(120px) */
transform: matrix(1,0,0,1,45,30); /* equivale a translate(45px, 30px) */
transform: matrix(1,.57,0,1,0,0); /* equivale a skewY(30deg) - tan30 = 0.57 */
transform: matrix(1,0,1,1,0,0); /* equivale a skewX(45deg) - tan45 = 1 */
transform: matrix(1,.36,.84,1,0,0); /* equivale a skew(40deg, 20deg) - tan20 = 0.36 e tan40 = 0.84 */
```

Note que os valores dos parâmetros **a** e **d** não podem ser 0 (zero). Se definirmos zero para um deles ou ambos, teremos um fator nulo para multiplicação das dimensões do box, o que o fará simplesmente desaparecer. Note o valor 1 para esses parâmetros, nas transformações que não sejam **scale()**.

A sintaxe para obter o efeito de rotação com uso da função **matrix()** é mostrada a seguir: `transform: matrix(cos(a), sen(a), -sen(a), cos(a), e, f);` Os quatro primeiros parâmetros são os valores do seno e cosseno do ângulo de rotação. No exemplo a seguir, é definida uma rotação de 30 graus para um elemento **div**.

Sabemos que $\cos 30 = 0.87$ e $\sin 30 = 0.5$: div {
transform: matrix(.87, .5, -.5, .87, 0, 0);
}

A regra CSS mostrada faz o mesmo efeito que:

```
div {  
transform: rotate(30deg);  
}
```

12.4 Múltiplas transformações

A sintaxe para transformações prevê uma forma de se aplicar mais de uma transformação a um elemento. Para isso, devemos declarar as funções de transformação separadas por espaço em branco. Por exemplo: para expandir e deslocar um box, declaramos o seguinte: div { transform: scale(1.5) translate(240px, -6px); }

A ordem em que declaramos as funções de transformação é importante, pois as transformações são processadas na ordem em que são declaradas. Dessa forma, a regra CSS mostrada a seguir produz efeito diferente daquele produzido pela regra CSS mostrada anteriormente: div { transform: translate(240px, -6px) scale(1.5); }

Observe que apenas invertemos a ordem das funções.

Considere que as duas regras CSS mostradas foram aplicadas em dois boxes diferentes, com dimensões iguais a **190px x 70px**.

Note que a aplicação de **scale(1.5)** aumenta as dimensões do box para: $1,5 \times 190\text{px} = 285\text{px}$ por $1,5 \times 70\text{px} = 105\text{px}$. Ou seja, **285px x 105px**. No primeiro caso, o box é expandido 1,5 vez e depois deslocado. No segundo caso, o box é deslocado e depois expandido. A figura 12.12, esclarece a diferença entre as duas regras.

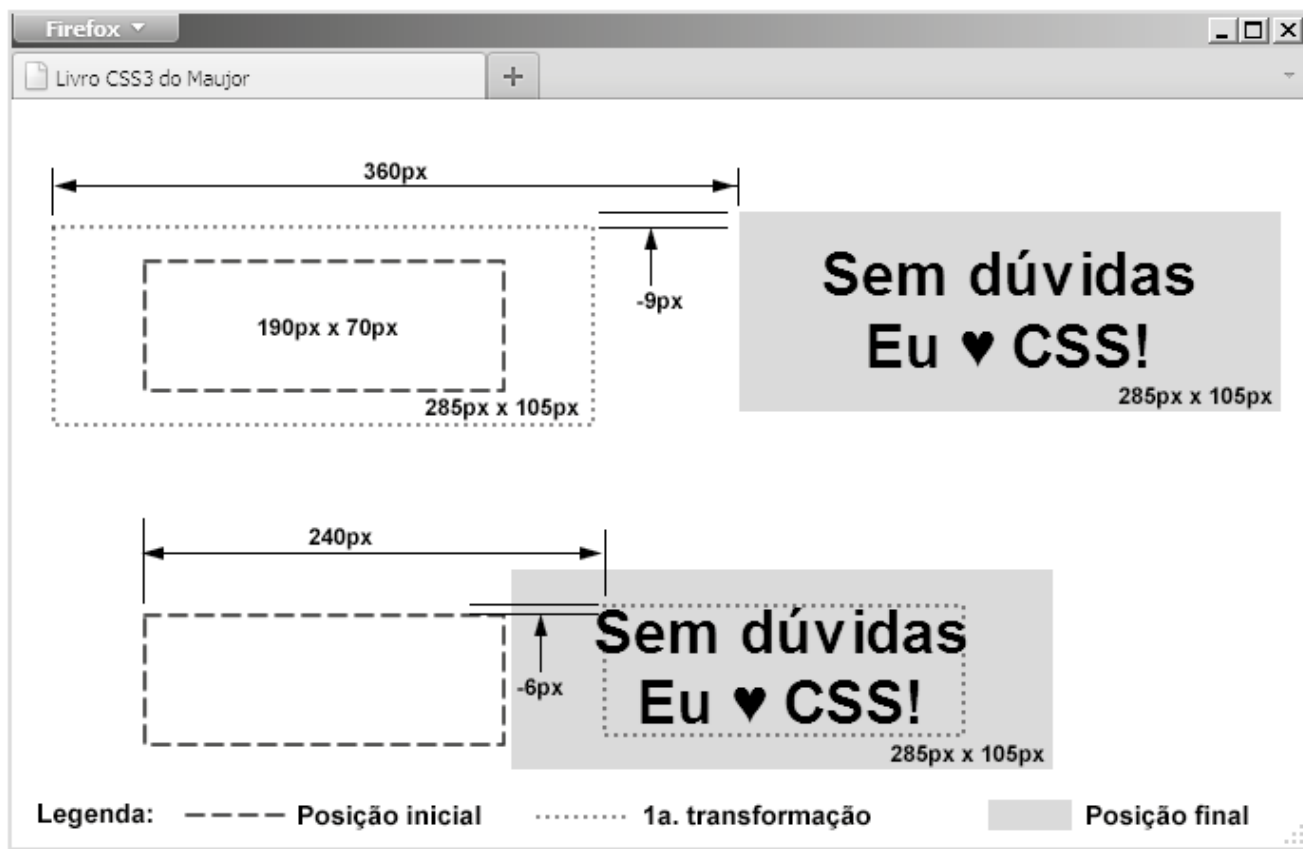


Figura 12.12 – Ordem de declaração das funções de transformação.

Note que, no primeiro caso, ao aplicar a função `scale()`, primeiro mudamos o sistema de coordenadas do box, passando a unidade a ter valor igual ao fator de multiplicação (1,5). Assim, o deslocamento horizontal passou de 240px para $1,5 \times 240px = 360px$, e o deslocamento vertical passou de -6px para $1,5 \times 6px = -9px$. Isso não aconteceu no segundo caso, pois a função `scale()` foi aplicada depois do deslocamento.

Em uma transformação múltipla, não estamos limitados à aplicação de duas funções. Podemos aplicar quantas transformações quisermos, bastando seguir a sintaxe de separar as funções por espaço em branco. Por exemplo, ao final da transformação mostrada anteriormente, poderíamos rotacionar o box usando a seguinte regra CSS: `div { transform: scale(1.5) translate(240px, -6px) rotate(30deg); }`

Seguindo a sintaxe mostrada, é possível aplicar transformações múltiplas, também, com uso da função `matrix()`. Por exemplo: `div { transform: matrix(1.5, 0, 0, 1.5, 0, 0) matrix(1, 0, 0, 1, 240, -6); }`

Essa transformação causa o mesmo efeito da primeira transformação mostrada na figura 12.12, ou seja, primeiro é aplicada a escala ao box e, a seguir, feita a translação.

Finalmente, a sintaxe geral para aplicação de transformações múltiplas com uso da função `matrix()` admite uma variante que consiste em declarar todas as variáveis das transformações em uma só função, conforme mostrado a seguir: `div {`

```
transform: matrix(1.5, 0, 0, 1.5, 240, -6); }
```

Essa transformação causa o mesmo efeito da segunda transformação mostrada na figura 12.12, ou seja, primeiro é feita a translação, depois é aplicada a escala ao box.

Exemplo com três transformações:

```
div { transform: matrix(1.5, 0, .57, 1.5, 240, -6); }
```

Nesse exemplo, a ordem das transformações é: translação, escala e inclinação.

12.5 Mudança da origem da transformação

Vimos que, por padrão, as transformações ocorrem em relação a uma referência. Por exemplo: a referência padrão para as transformações rotação e escala é o ponto central do box; para uma inclinação segundo o eixo x, a referência é a dimensão horizontal superior do box e assim por diante.

As especificações oferecem ao autor a possibilidade de alterar a referência da transformação, definindo uma referência personalizada. Para tanto, criou-se a propriedade **transform-origin**. Essa propriedade admite um ou dois valores, que definem as coordenadas personalizadas da origem da transformação. Os valores das coordenadas podem ser definidos com as seguintes medidas CSS: [[<percentage> | <length> | left | center | right] [<percentage> | <length> | top | center | bottom]?] | [[left | center | right] || [top | center | bottom]]

O valor padrão é, como já vimos, 50% 50%, ou seja, o ponto central do box.

O exemplo a seguir esclarece a personalização da origem de três transformações do tipo rotação: div#um {

```
transform-origin: top left;
transform: rotate(30deg);
```

```
}
```

```
div#dois {
transform-origin: 100% 100%;
transform: rotate(30deg);
```

```
}
```

```
div#tres {
transform-origin: bottom
transform: rotate(30deg);
```

```
}
```

Observe, na figura 12.13, as três rotações, conforme os códigos mostrados.

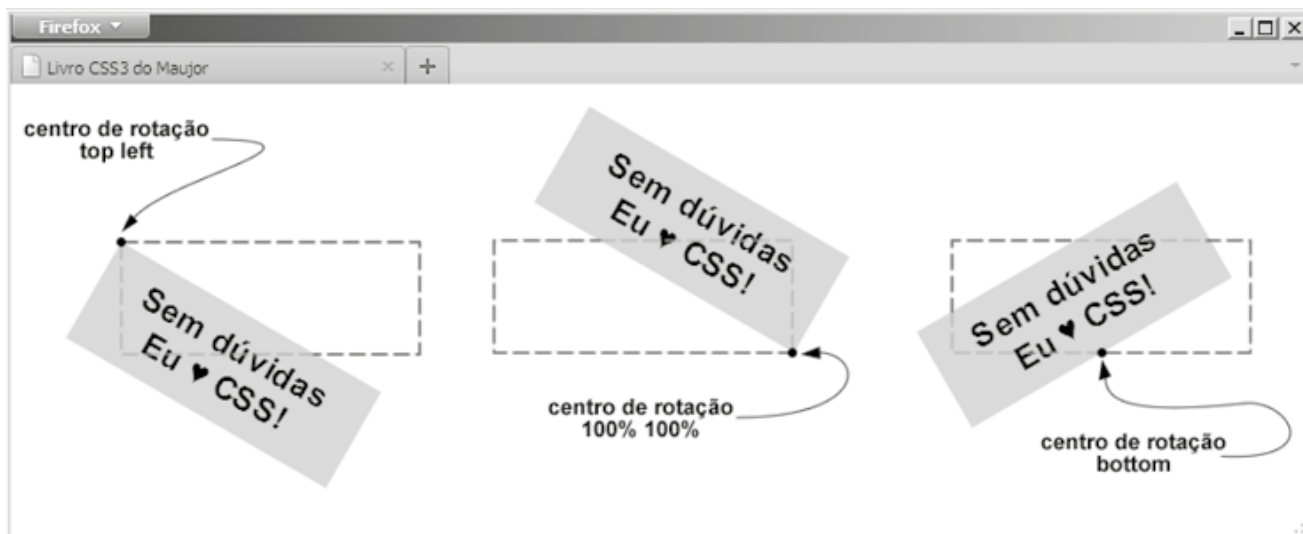


Figura 12.13 – Propriedade *transform-origin*.

Note que as coordenadas da origem da transformação são estabelecidas em relação ao box ao qual a transformação está sendo aplicada, isto é, o ponto de coordenadas (0,0) é o canto superior esquerdo do box.

A convenção para as coordenadas da origem da transformação é a seguinte: se forem definidos dois valores e pelo menos um deles não for uma palavra-chave (**top**, **right**, **bottom** ou **left**), o primeiro é a coordenada horizontal, e o segundo, a vertical. Se for definido apenas um valor, ele será a coordenada horizontal, e a coordenada vertical será **center**. São válidos valores negativos.

12.6 Exemplo

Para ilustrar uma das incontáveis aplicações das funcionalidades de transformações, vejamos um exemplo prático para criar um efeito muito usado em sites. Trata-se da inserção de um destaque na página constando de uma pequena faixa, normalmente com um texto, colocada inclinada no canto superior da janela. As páginas da documentação do site do framework jQuery Mobile usam esse recurso no canto superior direito, como mostrado na figura 12.14, extraída do site em outubro de 2011.



Figura 12.14 – Faixa destaque com imagem.

A faixa ali colocada é uma imagem, contudo usando-se os recursos das CSS3 transformações, é possível obter-se o mesmo efeito sem uso de imagem, como mostrado a seguir: ► **CSS**

```
body {overflow-x: hidden; }
h2 {
font: bold 15px sans-serif;
margin: 0;
height: 25px;
line-height: 28px;
text-align: center
}

h2 span { color: #c1272d; }
div {
width: 150px;
position: absolute;
top: 0;
right: 0;
background: #fcd249;
border: 1px solid white;
box-shadow: 4px 0 5px black;

-moz-transform-origin: top left;
-webkit-transform-origin: top left;
-o-transform-origin: top left;
-ms-transform-origin: top left;
```

```
-transform-origin: top left;
```

```
-moz-transform: translate(77px, -30px) rotate(45deg);  
-webkit-transform: translate(77px, -30px) rotate(45deg);  
-o-transform: translate(77px, -30px) rotate(45deg);  
-ms-transform: translate(77px, -30px) rotate(45deg);  
transform: translate(77px, -30px) rotate(45deg);  
  
}
```

► HTML

```
<div>  
<h2>1.0 <span>RC1</span></h2>  
</div>
```

Observe, na figura 12.15, a renderização do código mostrado.

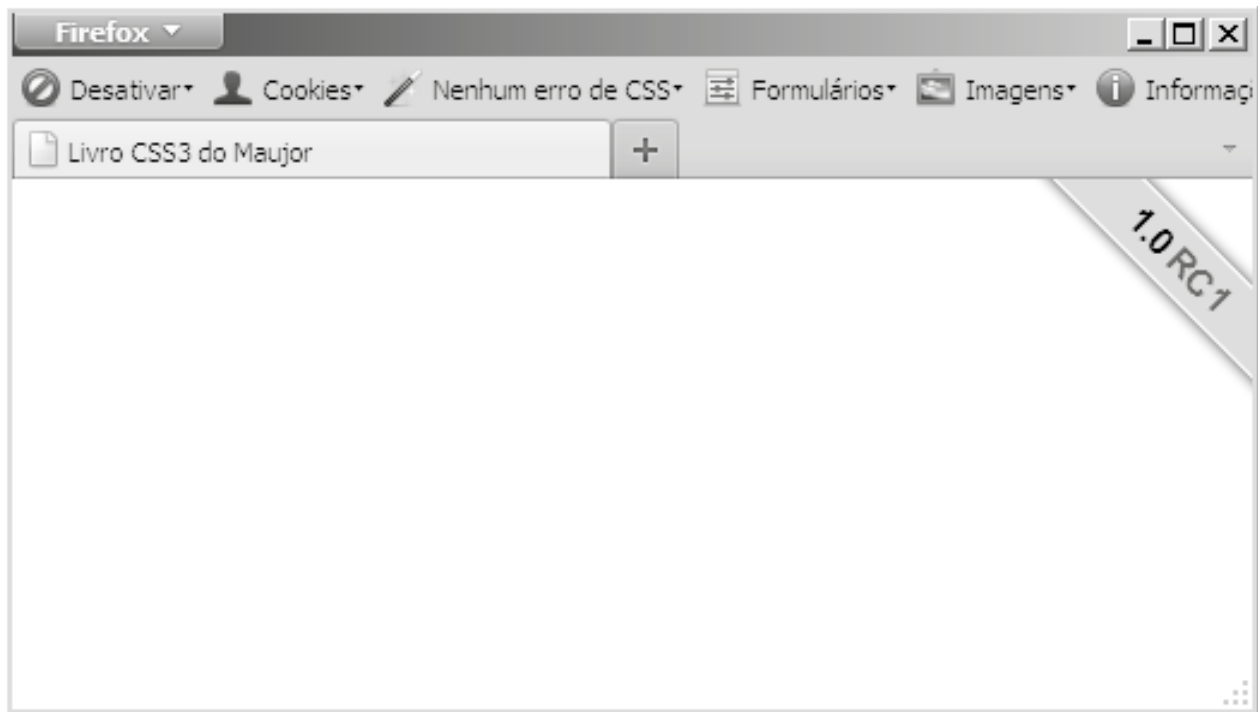


Figura 12.15 – Faixa destaque com transformação.

CAPÍTULO 13

Transformações 3D

As especificações para transformações CSS estavam divididas em dois módulos independentes até 28 de fevereiro de 2012: um para transformações 2D e outro para transformações 3D. A partir daquela data foram reunidas em especificação única denominada CSS Transforms e na data que esta edição do livro foi escrita estava no estágio de Rascunho de Trabalho do W3C. Neste capítulo, estudaremos as funcionalidades para transformações 3D. Os navegadores que suportam essas transformações com uso do prefixo proprietário `-webkit-` são o Chrome, o Opera e o Safari. O navegador Firefox já oferece suporte nativo. No capítulo 12, estudamos as transformações 2D.

13.1 Introdução

Segundo o W3C, as transformações 3D estendem as funcionalidades das transformações 2D, de modo a permitir que regras CSS determinem a renderização dos elementos em espaço tridimensional.

As propriedades `transform` e `transform-origin` são previstas com as mesmas finalidades que têm para transformações 2D, conforme estudamos no capítulo anterior.

Todos os exemplos deste capítulo serão desenvolvidos com uso do prefixo proprietário `-webkit-`. Para servir o navegador Firefox acrescente a declaração de estilo sem o prefixo proprietário.

13.2 Propriedades

As propriedades específicas para transformações 3D são:

- `transform-style` • `perspective`
- `perspective-origin` • `backface-visibility` A seguir, veremos a finalidade e os valores possíveis para cada uma delas.

13.2.1 `transform-style`

Essa propriedade define como elementos-filhos são renderizados no espaço 3D. Antes de estudar o efeito dessa propriedade quando aplicada para elementos-filhos (ou elementos aninhados), vejamos um exemplo mais simples, mostrando o efeito dessa propriedade sobre elementos não aninhados, ou seja, elementos que se seguem no fluxo normal do documento.

Segundo o modelo CSS de posicionamento, elementos que seguem outros elementos na marcação HTML têm **z-index** maior, ou seja, quanto mais abaixo na marcação HTML, tanto maior o **z-index** do elemento.

No capítulo 12, quando estudamos transformações 2D, mostramos o exemplo de um elemento **h2** que, ao ser rotacionado, se sobrepunha a um elemento parágrafo que o seguia no fluxo do documento (ver [12.3.1]). Isso contraria o modelo de posicionamento CSS, segundo o qual o elemento parágrafo, por vir depois do elemento **h2**, tem um **z-index** maior e, em consequência, deveria sobrepor-se ao elemento **h2**, e não ser sobreposto por ele.

Segundo o modelo de transformações CSS, um elemento transformado, ao mesmo tempo em que é retirado de sua posição, também é colocado em uma camada que se sobrepõe aos demais elementos da página, ou seja, o **z-index** de um elemento transformado passa a ser maior que o **z-index** dos elementos da página.

Podemos alterar esse comportamento definindo explicitamente um **z-index** para o elemento não transformado. Essa solução traz um inconveniente, pois ao alterarmos explicitamente o **z-index** de um elemento, nós o posicionaremos à frente não só do elemento transformado, mas também dos demais elementos da página, cujo **z-index** não tenha sido declarado.

Assim, em lugar de declarar **z-index**, declaramos a propriedade **transform-style**, que faz com que o **z-index** padrão do elemento seja preservado em uma transformação. Os valores possíveis para essa propriedade são: **flat** e **preserve-3d**, sendo o primeiro o valor padrão, e o segundo, o valor que força a preservação do posicionamento padrão segundo o eixo z.

Confuso? Um exemplo prático esclarece o que dissemos:

► HTML

```
<div id="um">
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

```
<div id="dois">
<h2>Sem dúvidas, eu &#9829; CSS!</h2>
<p>Lorem ipsum ...</p>
</div>
```

► CSS

```
<style>
body {
font: 14px sans-serif;
```



```
margin: 0 0 0 20px;
```

```
}
```

```
div {  
float: left;  
width: 300px;  
margin-right: 60px;
```

```
}
```

```
h2 {  
border: 1px solid #000;  
background: #f69;  
height: 80px;  
line-height: 80px;  
text-align: center;
```

```
}
```

```
p {  
border: 1px solid #000;  
background: #ffc;  
margin: 0;  
text-align: justify;
```

```
}
```

```
#um h2, #dois h2 {  
-webkit-transform-origin: bottom right; /* Definição da referência para  
rotação dos dois h2 */  
-webkit-transform: rotate(-30deg); /* Rotação para os dois elemento h2  
nos dois divs */
```

```
}
```

```
#dois p { -webkit-transform-style: preserve-3d; }  
</style>
```

O exemplo mostra dois `div` iguais contendo, cada um deles, um elemento `h2` seguido de um elemento parágrafo. Em ambos, para o elemento `h2`, foi definida uma rotação de -30 graus.

Observe, na figura 13.1, a renderização dos dois `div` de acordo com a folha de estilo mostrada. Para o primeiro `div`, temos a renderização segundo as regras das transformações 2D, como estudamos no capítulo 12. Para o segundo `div`, temos a renderização segundo as regras das transformações 3D, com uso da declaração `transform-style` para preservar o `z-index` padrão do parágrafo.

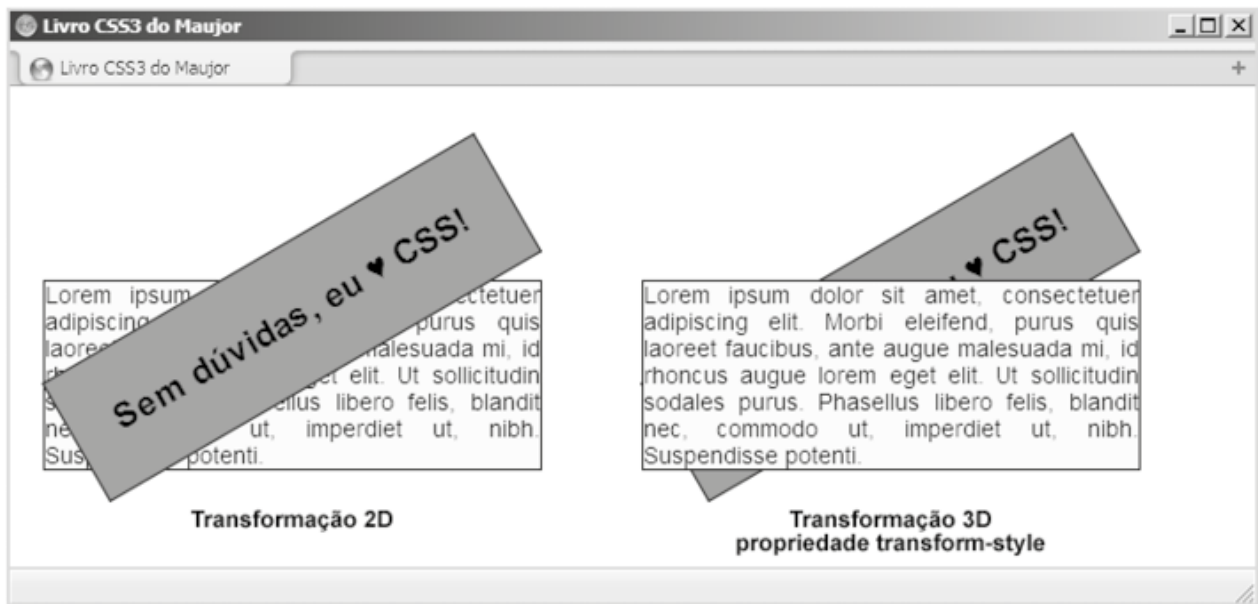


Figura 13.1 – Propriedade transform-style para elementos irmãos.

Mostramos o efeito da propriedade `transform-style` para elementos não aninhados. Vejamos, a seguir, como essa propriedade interfere nas transformações 3D para elementos-filhos. O exemplo que mostraremos consiste em dois `div` aninhados. Tanto para o `div` container como para o `div` filho foi definida uma rotação.

Para melhor visualizar o efeito da propriedade, construímos quatro conjuntos de dois `div`. Os conjuntos um e dois representam uma rotação de 60 graus em torno do eixo `y` para o `div` container e de 30 graus para o `div` filho. Ao primeiro conjunto, não foi definida a propriedade `transform-style`, e ao segundo, a propriedade foi definida.

O terceiro e o quarto conjuntos de dois `div` são idênticos, exceto pela definição da rotação para o `div` filho, que foi de 60 graus.

Assim, podemos visualizar o efeito da aplicação dessa propriedade em dois casos semelhantes.

Antes de apresentar os códigos para a página exemplo, observe, na figura 13.2, como ela é renderizada.

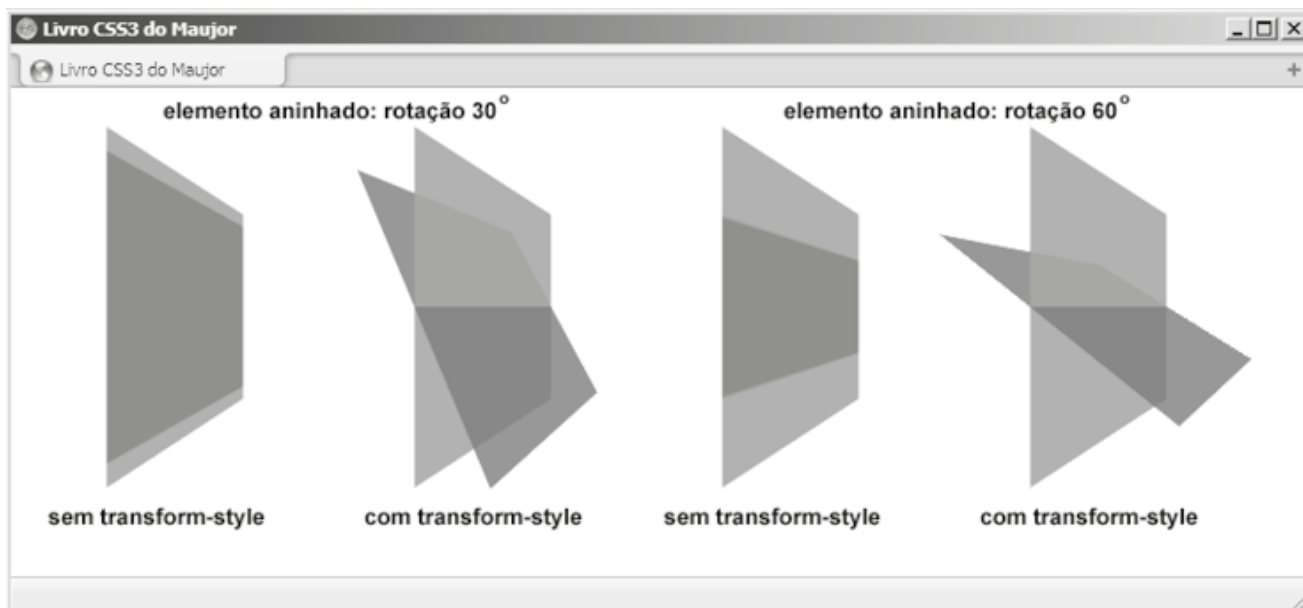


Figura 13.2 – Propriedade `transform-style` para elementos aninhados.

Marcação e estilização para a página exemplo mostrada na figura 13.2: ► **HTML**

```
<div style="-webkit-perspective: 200;">
  <div style="background: magenta; -webkit-transform: rotateY(60deg)">
    <div style="background: lime; -webkit-transform: rotateX(30deg)"></div>
  </div>
</div>
```

```
<div style="-webkit-perspective: 200; ">
  <div style=" background: magenta; -webkit-transform-style: preserve-3d;
    -webkit-transform: rotateY(60deg)">
    <div style="background: lime; -webkit-transform: rotateX(30deg)"></div>
  </div>
```

```
<div style="-webkit-perspective: 200; ">
  <div style=" background: magenta; -webkit-transform: rotateY(60deg)">
    <div style="background: lime; -webkit-transform: rotateX(60deg)"></div>
  </div>
```

```
<div style="-webkit-perspective: 200; ">
  <div style=" background: magenta; -webkit-transform-style: preserve-3d;
    -webkit-transform: rotateY(60deg)">
    <div style="background: lime; -webkit-transform: rotateX(60deg)"></div>
  </div>
```

Note que os quatro conjuntos de dois `div` estão dentro de um container, ao qual foi definida a propriedade `perspective` com valor 200. Esse container cria um contexto de renderização 3D, como veremos adiante. Note também que definimos

rotações com as funções `rotateX()` e `rotateY()`, não existentes em transformações 2D. Veremos as funções 3D adiante.

13.2.2 perspective

Essa propriedade destina-se a criar a ilusão de profundidade. Seu valor é um número positivo que determina o quanto (em pixels) os elementos transformados são redimensionados com base na distância ao longo do eixo z até o plano de renderização.

Ou seja, o valor dessa propriedade é a simulação da distância do observador até a superfície de renderização. Valores altos para a propriedade resultam em pequeno redimensionamento das dimensões em perspectiva, e valores baixos, grandes redimensionamentos. O valor 0 (zero) e os valores negativos não causam redimensionamento, sendo 0 o valor padrão (ou inicial). Na prática, valores entre 500 e 1.000 resultam, na maioria dos casos, em uma boa perspectiva.

No exemplo a seguir, rotacionamos três `div`, e a cada um deles definimos um valor para a propriedade `perspective`. Para o primeiro `div`, foi definido o valor igual a 100, para o segundo, 300, e para o terceiro, 1.000, conforme o código mostrado a seguir: ►

CSS

```
body { margin: 90px 0 0 90px; }
div {
  float: left;
  width: 150px;
  height: 150px;
  margin-right: 40px;
  opacity: .8;

  }
```

► HTML

```
<div style="-webkit-perspective: 100; border: 1px solid black;"> <div
style="background: tomato; -webkit-transform: rotateY(45deg)"></div>
</div>
```

```
<div style="-webkit-perspective: 300; border: 1px solid black; "> <div
style="background: tomato; -webkit-transform: rotateY(45deg)"></div>
</div>
```

```
<div style="-webkit-perspective: 1000; border: 1px solid black; "> <div
style="background: tomato; -webkit-transform: rotateY(45deg)"></div>
</div>
```

Observe, na figura 13.3, a renderização dos três `div` e o efeito da propriedade

perspective.

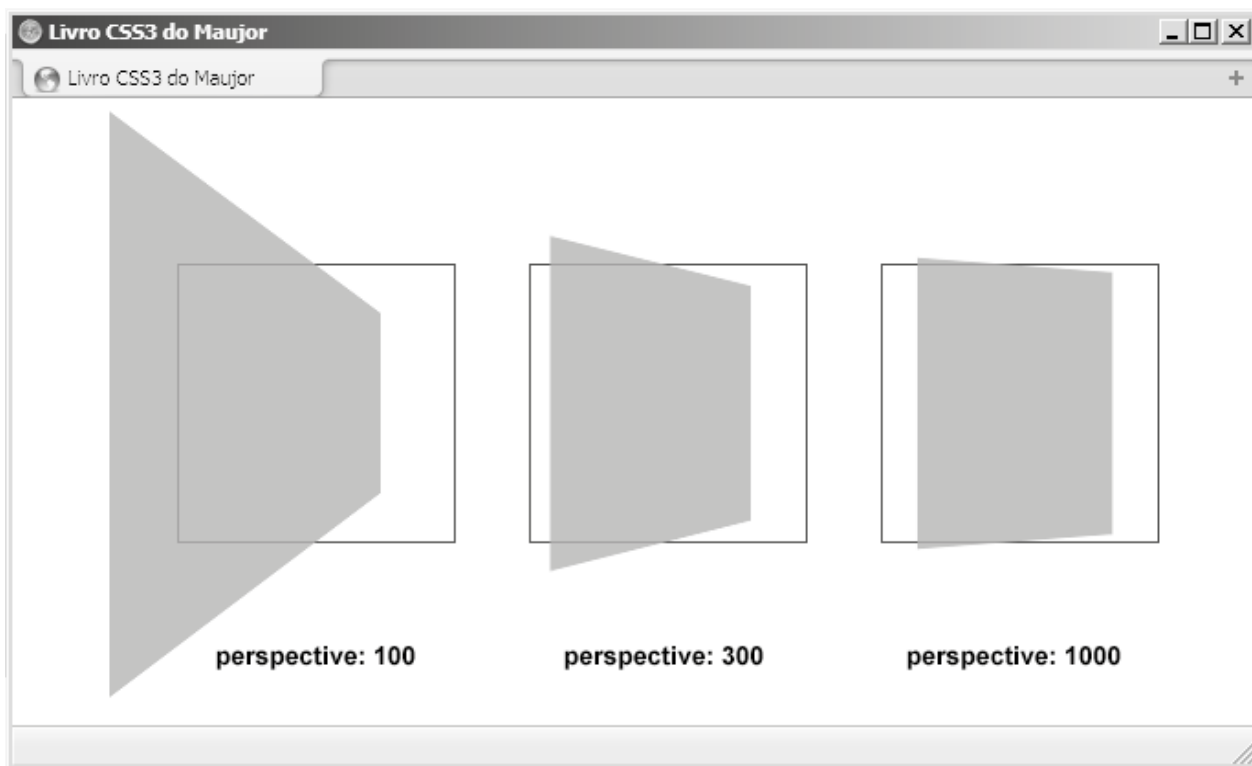


Figura 13.3 – Propriedade perspective.

13.2.3 perspective-origin

Essa propriedade destina-se a definir o ponto de origem para a propriedade **perspective**. Declaramos como valor para essa propriedade duas medidas CSS de comprimento que estabelecem as coordenadas x e y, respectivamente, do ponto de origem. O valor inicial para essa propriedade é 50% 50%.

No exemplo mostrado a seguir, três **div** foram rotacionados em torno do eixo y. Para o segundo **div**, não foi declarado um ponto de origem para a perspectiva, ou seja, ele será o ponto inicial, **x = 50%** e **y = 50%**. Para o primeiro **div**, declaramos um ponto de origem com coordenadas iguais a **x = right** e **y = 300px**. Para o terceiro **div**, as coordenadas do ponto de origem foram **x = -500px** e **y = -200px**. Observe os códigos a seguir: ► **CSS**

```
body { margin: 90px 0 0 90px; }
div {
float: left;
width: 150px;
height: 150px;
line-height: 120px;
font: bold 120px sans-serif;
text-align: center;
margin-right: 40px;
opacity: .8;
```

```
-webkit-perspective: 300px; }
```

► HTML

```
<div style="-webkit-perspective-origin: right 300px; border: 1px solid black;"> <div style="background: tomato; -webkit-transform: rotateY(45deg)">1</div> </div>
```

```
<div style="border: 1px solid black;">  
<div style="background: tomato; -webkit-transform: rotateY(45deg)">2</div> </div>
```

```
<div style="-webkit-perspective-origin: -500px -200px; border: 1px solid black; "> <div style="background: tomato; -webkit-transform: rotateY(45deg)">3</div> </div>
```

Observe, na figura 13.4, a renderização dos três div e o efeito da propriedade `perspective-origin`.

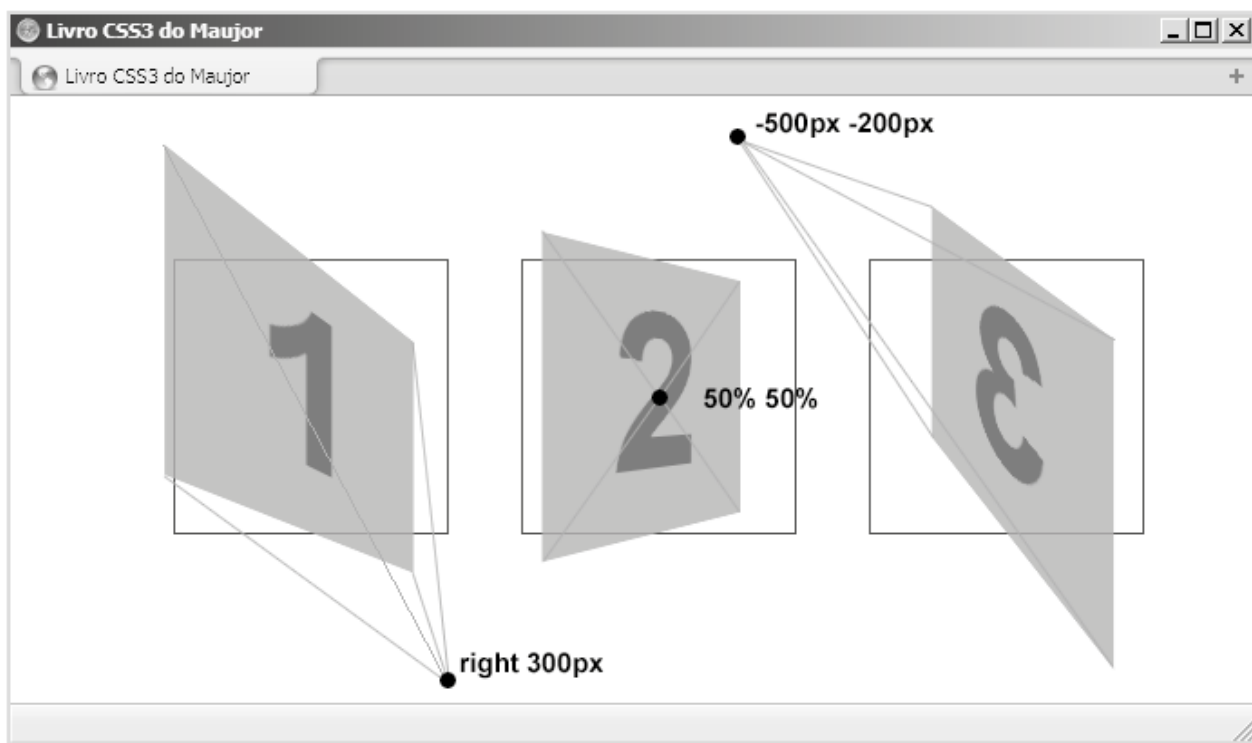


Figura 13.4 – Propriedade `perspective-origin`.

A posição do ponto de origem na figura, para o primeiro e o terceiro `div`, é apenas ilustrativa. As coordenadas são contadas a partir do canto superior esquerdo do container dos `div`, assim, `300px`, `-200px` e `-500px` não estão em escala real. Representamos apenas, para efeito de entendimento do exemplo, a posição relativa de cada ponto.

13.2.4 backface-visibility

Essa propriedade destina-se a definir a visibilidade de um elemento colocado atrás de outro com uso de uma rotação 3D em torno do eixo y igual a 180 graus. Os valores possíveis são: **visible** e **hidden**, sendo o primeiro o valor inicial.

Quando um elemento é rotacionado e colocado atrás de outro elemento, o conteúdo nele inserido é visto em ordem reversa. Nem sempre a visibilidade em ordem reversa é desejada. Nesses casos, o uso dessa propriedade resolve o problema.

No exemplo mostrado a seguir, apresentamos dois conjuntos de dois **div**, colocados um atrás do outro. No primeiro conjunto, é apresentada a renderização padrão. Note que, nesse caso, uma vez que o **div** contendo o número 2 foi declarado na marcação HTML depois do **div** com o número 1, ele tem **z-index** maior e, em consequência, é colocado à vista. O segundo conjunto de dois **div** é exatamente igual ao primeiro, mas foi declarada a propriedade **backface-visibility** com valor **hidden**. Observe os códigos a seguir: ► **CSS**

```
body { margin: 90px 0 0 90px; }
div {
width: 200px;
height: 300px;
margin-right: 40px;
```

```
}
```

```
#um div, #dois div {
position: absolute;
font: bold 200px sans-serif;
color: white;
line-height: 300px;
text-align: center;
border-radius: 10px;
```

```
}
```

```
#um {
float: left;
-webkit-perspective: 600;
position: relative;
```

```
}
```

```
#dois {
float: left;
-webkit-backface-visibility: hidden; position: relative;
```

```
}
```

► HTML

```
<div id="um">  
<div style="background: green;">1</div>  
<div style="background: magenta; -webkit-transform:  
rotateY(180deg)">2</div> </div>
```

```
<div id="dois">  
<div style="background: green;">1</div>  
<div style="background: magenta; -webkit-transform:  
rotateY(180deg)">2</div> </div>
```

Observe, na figura 13.5, a renderização dos dois conjuntos de `div` e o efeito da propriedade `backface-visibility` no segundo conjunto.

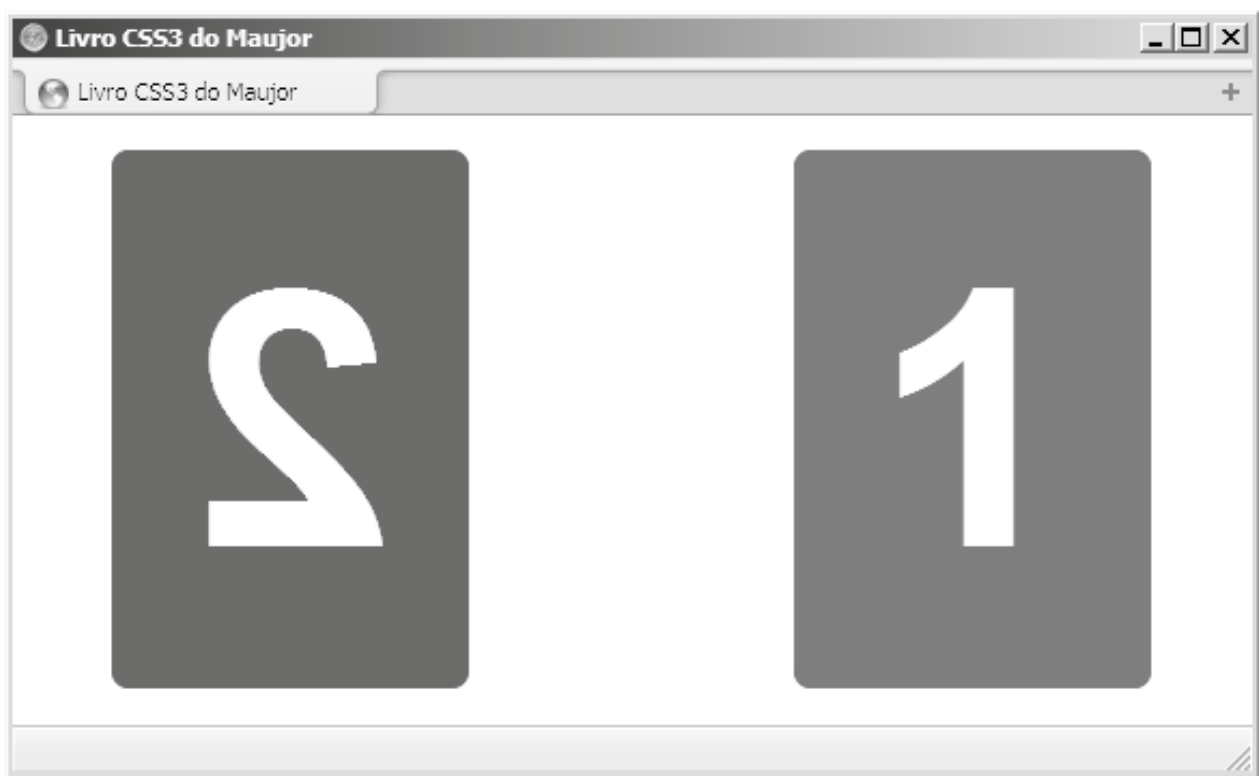


Figura 13.5 – Propriedade backface-visibility.

13.3 Funções

Estão previstas as seguintes funções para transformações 3D: • `perspective()`

- `rotate()`
- `rotateX()`
- `rotateY()`
- `rotateZ()`
- `rotate3d()`

- `translate()`
- `translateX()`
- `translateY()`
- `translateZ()`
- `translate3d()`
- `scale()`
- `scaleX()`
- `scaleY()`
- `scaleZ()`
- `scale3d()`
- `skew()`
- `skewX()`
- `skewY()`
- `matrix()`
- `matrix3d()`

Vejamos cada uma dessas funções detalhadamente.

13.3.1 perspective(n)

Essa função tem o mesmo efeito da propriedade `perspective`, estudada anteriormente. Os valores possíveis são números positivos representando a distância, em pixels, do observador em relação à área de renderização. Tal como acontece para a propriedade `perspective`, essa função destina-se a criar a ilusão de profundidade. Seu valor é um número positivo que determina (em pixels) o quanto os elementos transformados são redimensionados com base na distância ao longo do eixo z até o plano de renderização. Ver [13.2.2].

A sintaxe para emprego dessa função de transformação é mostrada a seguir: `seletor { transform: perspective(600); }`

A seguir, lembro a sintaxe para emprego da propriedade `perspective`, como estudada em [13.2.2]: `seletor { perspective: 600; }`

Existe uma diferença relacionada ao emprego dessa função quando comparada ao emprego da propriedade de mesmo nome. A função é empregada para definir o espaço 3D para cada elemento, isoladamente. Se para um conjunto de elementos quisermos definir um espaço 3D comum, devemos definir a propriedade para o

elemento container.

O exemplo a seguir esclarece o uso e mostra a diferença de comportamento entre as duas funcionalidades. Temos dois containers, cada um com quatro `div`, rotacionados 60 graus em torno do eixo y. Para o primeiro container, definimos a função `perspective()` em cada um dos quatro `div`, e para o segundo container, definimos a propriedade `perspective` para o container dos quatro `div`.

► CSS

```
body { margin: 20px 0 0 20px; }

#um, #dois {
float: left;
width: 210px;
height: 260px;
margin-right: 40px;
border: 1px solid black;

}

#um div, #dois div {
float: left;
width: 80px;
height: 100px;
margin: 15px 10px 10px 10px;
border-radius: 10px;
background: turquoise;

}

#um div {
-webkit-transform: perspective(300) rotateY(60deg); }
#dois {
-webkit-perspective: 300; }
#dois div {
-webkit-transform: rotateY(60deg);

}
```

► HTML

```
<div id="um">
<div></div>
<div></div>
<div></div>
<div></div>
</div>
<div id="dois">
```

```
<div></div>
<div></div>
<div></div>
<div></div>
</div>
```

Observe, na figura 13.6, a renderização dos dois containers de quatro `div` e o efeito da definição da função `perspective()` nos `div` do primeiro container e da propriedade `perspective` no segundo container.

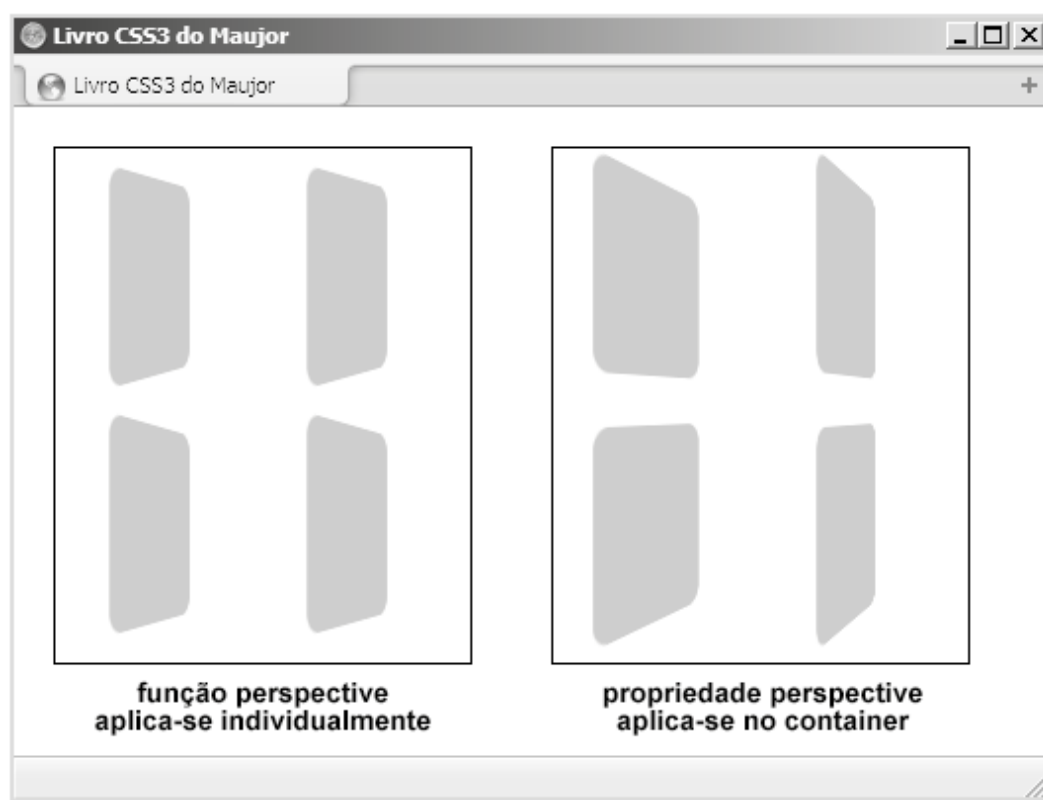


Figura 13.6 – Função e propriedade `perspective`.

13.3.2 `rotate()`

Essa função destina-se a rotacionar um box. No capítulo 12, estudamos essa função para transformações 2D e vimos que ela rotaciona um box no plano xy, ou seja, rotações são possíveis apenas em um espaço bidimensional, realizando-se um movimento no sentido horário ou anti-horário, e sua sintaxe é conforme mostrado a seguir: seletor { transform: rotate(a); }

O parâmetro `a` para essa função é uma medida CSS para ângulos. Medidas CSS para ângulos são: `deg`, `grad`, `rad` e `turn`. Ver [1.1.5.4].

Em um espaço tridimensional, as opções de rotação aumentam consideravelmente. Assim, esse módulo, além da função de rotação, como estudada nas transformações 2D, criou funcionalidades de rotação em torno dos eixos x, y e z e também em torno de um eixo personalizado definido pelas coordenadas

tridimensionais x, y e z. A sintaxe para cada uma dessas novas funcionalidades é mostrada a seguir: • **rotate(a)** – igual à do módulo 2D; • **rotateX(a)** – rotação em torno do eixo x; • **rotateY(a)** – rotação em torno do eixo y; • **rotateZ(a)** – rotação em torno do eixo z; • **rotate3d(x,y,z,a)** – rotação de um ângulo a em torno do eixo definido pelas coordenadas x, y e z.

Para demonstrar o efeito de cada um desses tipos de rotação 3D, construímos a página com os códigos mostrados a seguir: ► **CSS**

```
body {margin: 20px 0 0 50px; }
div {
float: left;
width: 100px;
height: 130px;

}
```

```
body>div:nth-of-type(n) {
-webkit-perspective: 300;
margin: 0 80px 40px 0;
border: 1px solid black;

}
```

```
div div {
background: lightblue;

}
```

```
div:nth-child(1) div {
-webkit-transform: rotate(45deg); }
div:nth-child(2) div {
-webkit-transform: rotateX(45deg); }
div:nth-child(3) div {
-webkit-transform: rotateY(45deg); }
div:nth-child(4) div {
-webkit-transform: rotateZ(45deg); }
div:nth-child(5) div {
-webkit-transform: rotate3d(1,1,1,45deg); }
div:nth-child(6) div {
-webkit-transform: rotate3d(-1,-1,-1,45deg); }
```

► **HTML**

```
<div>
<div></div>
</div>
```

```

<div>
<div></div>
</div>
<div>
<div></div>
</div>
<div>
<div></div>
</div>
<div>
<div></div>
</div>
<div>
<div></div>
</div>
</div>

```

Na figura 12.7, apresentamos a renderização da página, mostrando o efeito de rotacionar os seis boxes, conforme os códigos mostrados anteriormente.



Figura 13.7 – Função *rotate()*.

13.3.3 translate()

Essa função destina-se a deslocar um box. No capítulo 12, estudamos essa função para transformações 2D e vimos que ela desloca um box no plano xy, ou seja, translações são possíveis apenas em um espaço bidimensional, realizando-se um movimento no sentido do eixo x, do eixo y ou simultaneamente no sentido dos dois eixos. Para cada um desses três tipos de deslocamento, existe uma função de translação, e suas sintaxes são conforme mostrado a seguir: seletor { transform: translate(x, y); }

```
seletor { transform: translateX(x); }  
seletor { transform: translateY(y); }
```

Os parâmetros *x* e *y* para essa função são uma medida CSS de comprimento.

Em um espaço tridimensional, é possível realizar mais dois tipos de translação. Assim, esse módulo criou as funções de translação ao longo do eixo z para translação ao longo de um eixo personalizado definido pelas coordenadas tridimensionais x, y e z. A sintaxe para cada uma dessas novas funcionalidades é mostrada a seguir: • **translate(x, y)** – igual à do módulo 2D; • **translateX(x)** – translação ao longo do eixo x, igual à do módulo 2D; • **translateY(y)** – translação ao longo do eixo y, igual à do módulo 2D; • **translateZ(z)** – translação ao longo do eixo z; • **translate3d(x,y,z)** – translação ao longo do eixo definido pelas coordenadas x, y e z.

Para demonstrar o efeito de cada um desses tipos de translação 3D, construímos a página com os códigos mostrados a seguir: ► **CSS**

```
body { margin: 20px 0 0 50px; }  
div {  
  float: left;  
  width: 50px;  
  height: 70px;  
}
```

```
body>div:nth-of-type(n){  
  -webkit-perspective: 300;  
  margin: 0 80px 140px 0;  
  border: 1px solid black;  
}
```

```
div div {  
  font: bold 60px sans-serif;  
  text-align: center;  
  line-height: 70px;
```

```
background: lightblue;
```

```
}
```

```
div:nth-child(1) div {  
-webkit-transform: translate(20px, 100px) rotateY(45deg); }  
div:nth-child(2) div {  
-webkit-transform: translateX(-60px) rotateY(45deg); }  
div:nth-child(3) div {  
-webkit-transform: translateY(40px) rotateY(45deg); }  
div:nth-child(4) div {  
-webkit-transform: translateZ(100px) rotateY(45deg); }  
div:nth-child(5) div {  
-webkit-transform: translate3d(20px,30px,120px) rotateY(45deg); }  
div:nth-child(6) div {  
-webkit-transform: translate3d(-20px,-10px,-120px) rotateY(45deg); }
```

► HTML

```
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>  
  
<div>  
<div></div>  
</div>
```

Na figura 12.8, apresentamos a renderização da página, mostrando o efeito de translação dos seis boxes, conforme os códigos mostrados anteriormente. Para melhor visualizar o efeito em 3D, definimos um rotação de 45 graus em torno do eixo y para cada box, antes de fazer a translação.

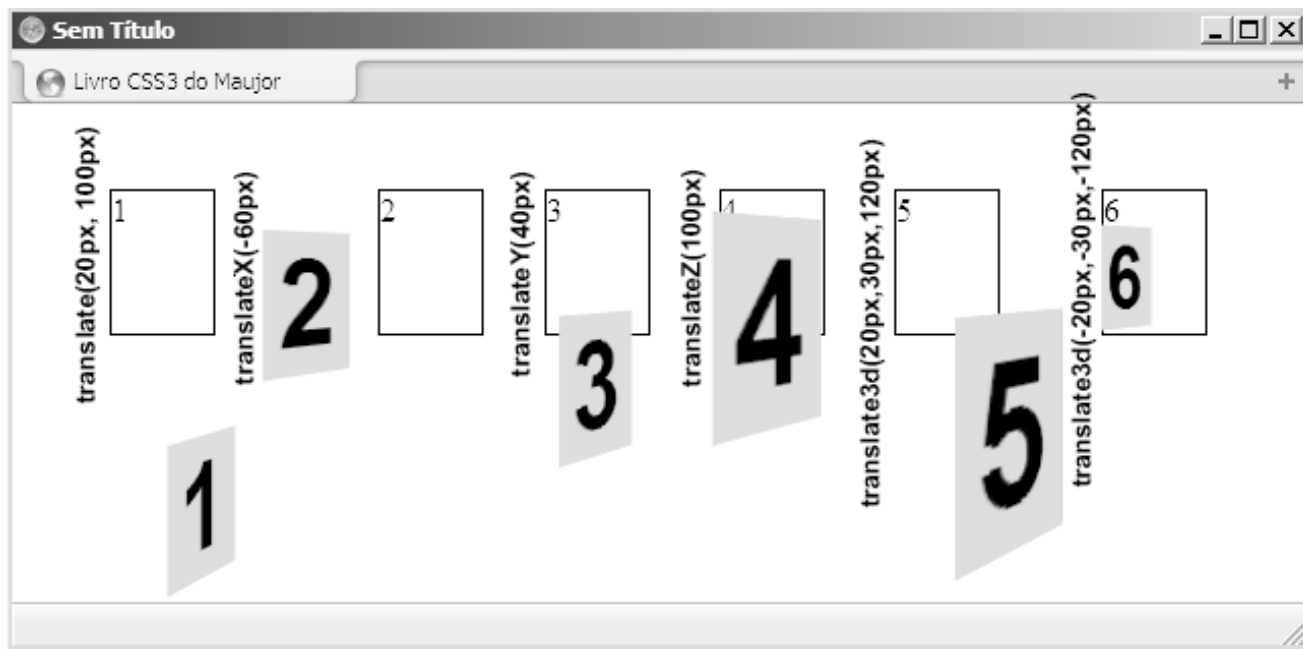


Figura 13.8 – Função *translate()*.

13.3.4 scale()

Essa função destina-se a expandir ou contrair um box. No capítulo 12, estudamos essa função para transformações 2D e vimos que ela altera dimensões de um box no plano xy, ou seja, alterações de dimensões são possíveis apenas em um espaço bidimensional, realizando-se a alteração no sentido do eixo x, do eixo y ou simultaneamente no sentido dos dois eixos. Para cada um desses três tipos de alteração de dimensões existe uma função de escala, e suas sintaxes são conforme mostrado a seguir: seletor { transform: scale(n); }

```
seletor { transform: scaleX(m); }
seletor { transform: scaleY(p); }
```

Os parâmetros *n*, *m* e *p* para essa função são números que expressam o fator de multiplicação para a nova dimensão.

Em um espaço tridimensional, é possível realizar mais dois tipos de mudança de dimensões. Assim, esse módulo criou as funções de escala ao longo do eixo z, sendo possível aplicar escala ao longo dos três eixos, x, y e z, simultaneamente. A sintaxe para cada uma dessas novas funcionalidades é mostrada a seguir: • **scale(*n*, *m*)** – igual à do módulo 2D; • **scaleX(*n*)** – redimensionamento ao longo do eixo x, igual ao do módulo 2D; • **scaleY(*m*)** – redimensionamento ao longo do eixo y, igual ao do módulo 2D; • **scaleZ(*p*)** – redimensionamento ao longo do eixo z; • **scale3d(*n*,*m*,*p*)** – redimensionamento ao longo dos três eixos, simultaneamente.

Para demonstrar o efeito de cada um desses tipos de redimensionamento 3D, construímos a página com os códigos mostrados a seguir: ► **CSS**

```
body {margin: 80px 0 0 80px; -webkit-perspective:300; }
```



```
div {
float:left;
width: 50px;
height: 70px;
font: bold 60px sans-serif;
text-align: center;
line-height: 70px;
background: lightblue;
margin-right: 80px;
```

```
}
```

```
div:nth-child(1) {
-webkit-transform: rotateY(45deg);
```

```
}
```

```
div:nth-child(2) {
-webkit-transform: scale(1.2, 2) rotateY(45deg); }
div:nth-child(3) {
-webkit-transform: scaleX(3) rotateY(45deg); }
div:nth-child(4) {
-webkit-transform: scaleY(2) rotateY(45deg); }
div:nth-child(5) {
-webkit-transform: scaleZ(.5) rotateY(45deg); }
div:nth-child(6) {
-webkit-transform: scale3d(3, .5, 3) rotateY(45deg); }
```

► HTML

```
<div></div>
<div>1</div>
<div>2</div>
<div>3</div>
<div>4</div>
<div>5</div>
```

Na figura 12.9, apresentamos a renderização da página, mostrando o efeito de aplicação da função `scale()` em cinco boxes, conforme os códigos mostrados anteriormente. Para melhor visualizar o efeito em 3D, definimos um rotação de 45 graus em torno do eixo y para cada box, antes de fazer o redimensionamento.

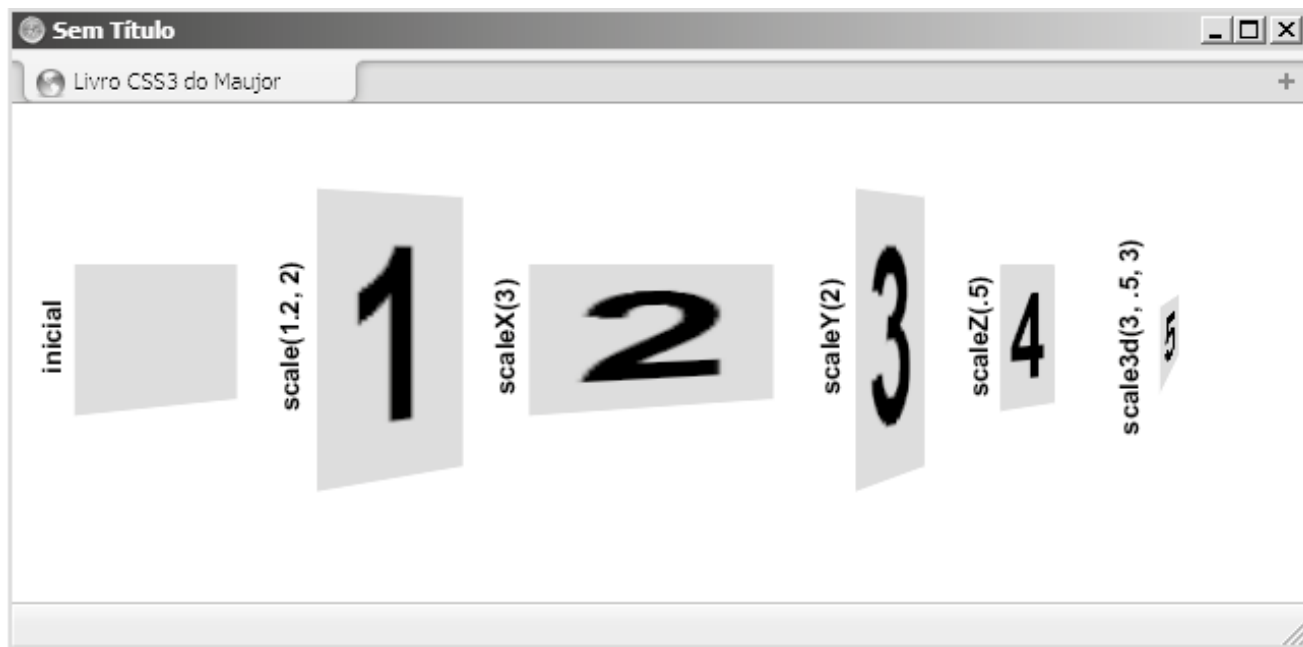


Figura 13.9 – Função *scale()*.

13.3.5 skew()

Essa função destina-se a inclinar um box. No capítulo 12, estudamos essa função para transformações 2D e vimos que ela inclina os lados de um box no plano xy, ou seja, inclinações são possíveis apenas em um espaço bidimensional, realizando-se a alteração no sentido do eixo x, do eixo y ou simultaneamente no sentido dos dois eixos. Para cada um desses três tipos de inclinação existe uma função, e suas sintaxes são conforme mostrado a seguir: `seletor { transform: skew(a); }`

```
seletor { transform: skewX(a); }  
seletor { transform: skewY(a); }
```

Os parâmetro *a* para essa função é um ângulo que expressa o quanto deve ser inclinado o box.

Para o espaço tridimensional, as inclinações são as mesmas do espaço bidimensional, como mostradas anteriormente. Para mais detalhes, ver [12.3.8, 12.3.9 e 12.3.10].

13.3.6 matrix()

Essa função destina-se a aplicar qualquer uma, ou mais de uma simultaneamente, das funções estudadas anteriormente neste capítulo. No capítulo 12, estudamos essa função para transformações 2D e vimos que o algoritmo de aplicação das transformações baseia-se em cálculo matricial, e sua explicação teórica é bastante complexa e extensa para constar deste livro. Por essa razão, mostraremos a sintaxe para aplicação da função sem entrar em detalhes matemáticos. Se você estiver interessado em conhecer tais detalhes, recomendo começar seus estudos com a

leitura dos seguintes documentos, disponíveis na web: <http://kwz.me/pQ> (Wikipédia) e <http://kwz.me/pR> (W3C).

A sintaxe para essa função é mostrada a seguir:

```
seletor { transform: matrix(a, b, c, d, e, f); }
```

Os seis parâmetros são números que definem o tipo e as características da transformação. O valor inicial de cada parâmetro é 0 (zero) e a transformação é obtida conforme a combinação dos parâmetros que se definem.

No espaço bidimensional as transformações são possíveis no sentido do eixo x, do eixo y ou simultaneamente no sentido dos dois eixos.

Em um espaço tridimensional, é possível realizar mais um tipo de transformação com uso dessa função. Assim, esse módulo criou uma função capaz de aplicar o algoritmo de transformação ao longo dos três eixos, x, y e z, simultaneamente. A sintaxe para essa nova funcionalidade é mostrada a seguir: `seletor { transform: matrix3d(a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p); }`

Para essa funcionalidade, definimos 16 parâmetros que determinam como serão as transformações. Por tratar-se de uma funcionalidade que aplica diferentes transformações simultaneamente, que podem ser obtidas com uso isolado das funções mostradas anteriormente, e tendo em conta que a explicação teórica é complexa, deixo de entrar em detalhes. Se você estiver interessado em mais detalhes, aprofunde seus conhecimentos em cálculo matricial, começando com a leitura dos seguintes documentos, disponíveis na web: <http://kwz.me/pQ> (Wikipédia) e <http://kwz.me/pR> (W3C).

CAPÍTULO 14

Transições CSS

As especificações para o Módulo Transições CSS estão no estágio de Rascunho de Trabalho do W3C e previstas com prioridade média para desenvolvimento.

Neste capítulo, estudaremos as funcionalidades deste módulo. Os navegadores modernos já as suportam com uso de prefixos proprietários, e assim podemos começar a realizar nossos estudos e nossas experiências com transições CSS.

14.1 Introdução

Segundo o W3C, a finalidade do módulo para transições é criar funcionalidades que permitam ao autor definir uma mudança de valor de uma propriedade CSS, de modo que ela (a mudança de valor) ocorra de forma suave, em um espaço de tempo especificado.

Normalmente, quando o valor de uma propriedade CSS é alterado, ocorre uma mudança instantânea na renderização. Um exemplo clássico desse comportamento é a mudança de apresentação de um link quando o usuário coloca o mouse sobre ele. Efeito que se consegue com uso do seletor `a:hover`.

As funcionalidades desse módulo visam a criar mecanismos que permitam definir uma mudança de apresentação de forma suave ao longo de um determinado tempo.

Suponha que o autor tenha definido uma transição da cor de fundo de um link, de azul para vermelha, para ocorrer em dois segundos. Se a cor de fundo do link for recuperada por um script, o resultado será uma cor intermediária entre as duas, ou seja, aquela que ocorre transcorrido um segundo.

Nem todas as propriedades CSS são elegíveis para transições. As especificações para as CSS3 estabelecem que somente as propriedades animáveis (passíveis de ser animadas) podem ser declaradas com transição. A tabela 14.1, constante das especificações, apresenta as propriedades CSS animáveis.

Tabela 14.1 – Propriedades CSS animáveis

Propriedade	Tipo
<code>background-color</code>	cor
<code>background-image</code>	somente gradientes
<code>background-position</code>	medida CSS e porcentagem
<code>border-bottom-color</code>	cor

Propriedade	Tipo
border-bottom-width	medida CSS
border-color	cor
border-left-color	cor
border-left-width	medida CSS
border-right-color	cor
border-right-width	medida CSS
border-spacing	medida CSS
border-top-color	cor
border-top-width	medida CSS
border-width	medida CSS
bottom	medida CSS e porcentagem
color	cor
crop	retângulo
font-size	medida CSS e porcentagem
font-weight	número
grid-*	vários
height	medida CSS e porcentagem
left	medida CSS e porcentagem
letter-spacing	medida CSS
line-height	medida CSS, porcentagem e número
margin-bottom	medida CSS
margin-left	medida CSS
margin-right	medida CSS
margin-top	medida CSS
max-height	medida CSS e porcentagem

Propriedade	Tipo
max-width	medida CSS e porcentagem
min-height	medida CSS e porcentagem
min-width	medida CSS e porcentagem
opacity	número
outline-color	cor
outline-offset	número inteiro
outline-width	medida CSS
padding-bottom	medida CSS
padding-left	medida CSS
padding-right	medida CSS
padding-top	medida CSS
right	medida CSS e porcentagem
text-indent	medida CSS e porcentagem
text-shadow	sombra
top	medida CSS e porcentagem
vertical-align	palavra-chave, medida CSS e porcentagem
visibility	visibilidade
width	medida CSS e porcentagem
word-spacing	medida CSS e porcentagem
z-index	integer
zoom	número

Os navegadores atuais suportam as funcionalidades desse módulo nativamente, exceto o navegador Safari, que as suporta com uso do prefixo proprietário `-webkit-`.

Todos os exemplos deste capítulo serão desenvolvidos com uso de prefixos proprietários.

14.2 Propriedades

As propriedades para transições são:

- `transition-property` • `transition-duration` • `transition-timing-function` • `transition-delay` • `transition`
- A seguir, veremos a finalidade e os valores possíveis para cada uma delas.

14.2.1 `transition-property`

Essa propriedade destina-se a definir a qual(is) propriedade(s) CSS será(ão) aplicada(s) a(s) transição(ões). Os valores possíveis são: `none`, `all` ou uma lista dos nomes das propriedades às quais serão aplicadas a(s) transição(ões). Essa lista deverá ter os nomes das propriedades separados por vírgula. O valor inicial é `all` (todas as propriedades elegíveis para transição), e essa propriedade não é herdável. Aplica-se a todos os elementos da marcação, bem como aos pseudo-elementos `:before` e `:after`.

A sintaxe para essa propriedade é conforme exemplos mostrados a seguir: seletor { `transition-property: background-color;` } /* transição na cor de fundo */

```
seletor { transition-property: background-image, color; } /* transição na
imagem de fundo e na cor do texto */
```

```
seletor { transition-property: all; } /* transição nas propriedades
elegíveis para transição */
```

```
seletor { transition-property: margin-left, font-size, color; } /*
transição na margem esquerda, no tamanho de
fonte e na cor do texto */
```

14.2.2 `transition-duration`

Essa propriedade define a duração da transição. O valor dessa propriedade é o número de segundos, expressando a duração da transição. No caso de uma lista de nomes das propriedades às quais serão aplicadas transições, o valor dessa propriedade poderá ser também uma lista de números de segundos, separados por vírgula, ou um só número de segundos válido para todas as propriedades. Caso a lista dos números de segundos não seja em quantidade igual ao número de propriedades, ela (lista de segundos) se repetirá até completar o total do número de propriedades.

O valor inicial para a duração da transição é `0` (a transição é instantânea), e números negativos serão considerados zero. Essa propriedade não é herdável e se aplica a todos os elementos da marcação, bem como aos pseudo-elementos `:before` e `:after`.

A sintaxe para essa propriedade é conforme exemplos mostrados a seguir: seletor { `transition-property: background-color;`

```
transition-duration: 2s; /* tempo da transição igual a 2s */
```

```
}
```

```
seletor {
```

```
transition-property: background-image, color;
```

```
  transition-duration: 2s, 3s; /* transição na imagem de fundo em 2s e na  
  cor do texto em 3s */
```

```
}
```

```
seletor {
```

```
transition-property: margin-left, font-size, color;
```

```
transition-duration: 1s, 3s; /* transição na margem esquerda em 1s, no  
tamanho de fonte em 3s e na cor do texto em 1s */
```

```
}
```

14.2.3 transition-timing-function

Essa propriedade define uma função que descreve como são calculados os valores intermediários de uma transição. Uma transição CSS nada mais é do que a passagem de um valor de uma propriedade para outro valor em um determinado intervalo de tempo. Para entendermos como funciona a propriedade **transition-timing-function**, vamos tomar como exemplo a transição da propriedade **width** de um box de um valor inicial igual a **100px** para um valor final igual a **200px**, a realizar-se em um intervalo de tempo igual a dez segundos.

Uma transição como a do exemplo proposto está indefinida, pois apesar de sabermos as larguras inicial (no tempo 0) e final (no tempo 10 segundos) do box, nada sabemos sobre a largura em determinado tempo dentro do intervalo da transição, por exemplo: qual é a largura do box depois de decorridos cinco segundos do início da transição? **150px**? **120px**? **180px**? Não há como saber, portanto falta definir uma variável que determine os valores intermediários da transição. Essa variável é definida pela propriedade **transition-timing-function**, que descreve como são calculados esses valores intermediários.

O algoritmo de cálculo do mecanismo CSS para determinação dos valores intermediários de uma transição usa a função que descreve a *curva cúbica Bézier*.

A citação a seguir foi extraída da Wikipédia, em: <http://kwz.me/gi>.

A curva de Bézier é uma curva polinomial expressa como a interpolação linear entre alguns pontos representativos, chamados de pontos de controle. É uma curva utilizada em diversas aplicações gráficas como o Illustrator, Freehand, Fireworks, Gimp, Photoshop, Inkscape e CorelDRAW, e formatos de imagem vetorial como o SVG. Esse tipo de curva também é bastante utilizado em modelagem tridimensional e animações.

A curva de Bézier foi desenvolvida em 1962, e seu nome é devido ao criador da curva, o francês Pierre Bézier, funcionário da Renault, que a usou para o design de automóveis. Foi desenvolvida como resultado do Algoritmo de De Casteljau, em 1957 (P. De Casteljau, Citroën), e formalizada na década de 1960.

Se você trabalha com um programa gráfico vetorial, com certeza conhece a ferramenta “caneta”, destinada a desenhar curvas cujo formato pode-se controlar por “alças” em determinados pontos. Pois a ferramenta citada gera curvas Bézier.

O gráfico mostrado na figura 14.1 foi extraído das especificações do W3C para transições e mostra a curva Bézier e o sistema de eixos cartesianos que foram utilizados para descrever transições.

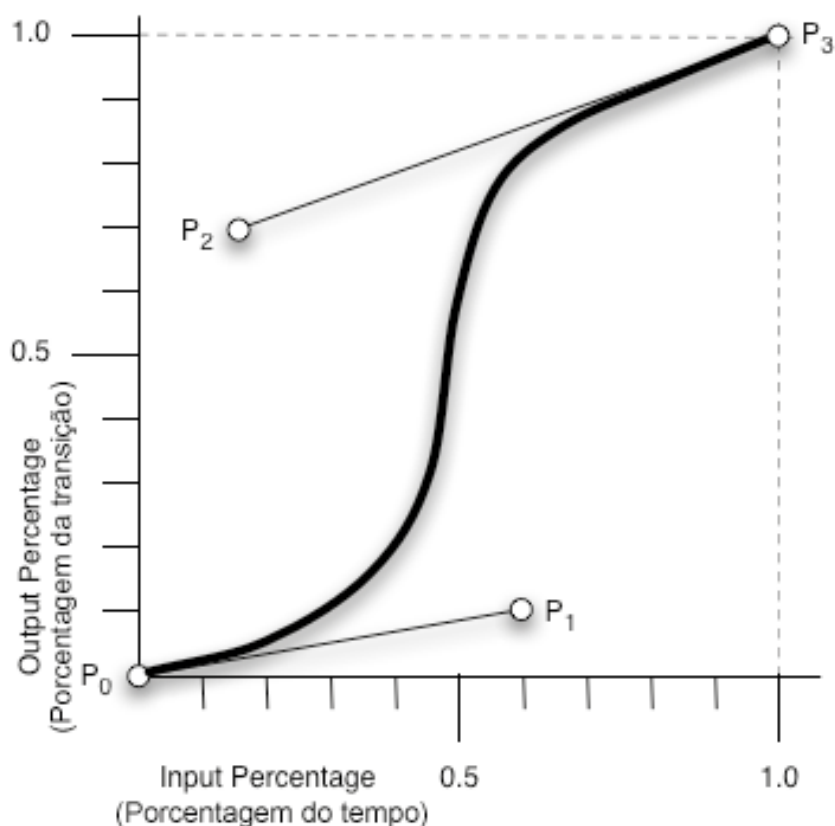


Figura 14.1 – Pontos de controle da função de transição.

No eixo x está representada a porcentagem de entrada do valor da transição, ou seja, a porcentagem do tempo transcorrido, e no eixo y, a porcentagem de saída do valor da transição, ou seja, a porcentagem de transição executada.

O início da transição é a origem do sistema de eixos $P_0(0,0)$, e o final da transição, o ponto $P_3(1,0)$.

Os pontos P_1 e P_2 determinam as “alças” de formatação da curva Bézier e podem ser colocados em qualquer coordenada do sistema de eixos. Note que variando-se a posição das “alças”, estamos variando a tangente à curva nos pontos P_0 e P_3 . Assim, se $P_1(1,0)$ e $P_2(0,0)$, teremos uma linha reta unindo P_0 a P_3 , e nesse caso, a uma

porcentagem de tempo corresponde a mesma porcentagem de transição.

Em nosso exemplo, se a propriedade `transition-timing-function` for definida com uso da função Bézier, que define a reta citada, teremos uma transição chamada *linear*, e decorridos cinco segundos, a largura do box será **150px**, assim como decorridos oito segundos, a largura será **180px**.

Antes de prosseguirmos com considerações teóricas, observe as sintaxes para declarar essa propriedade: `seletor { transition-timing-function: ease; }`

```
seletor { transition-timing-function: linear; }
seletor { transition-timing-function: ease-in; }
seletor { transition-timing-function: ease-out; }
seletor { transition-timing-function: ease-in-out; }
seletor { transition-timing-function: cubic-bezier(x1, y1, x2, y2); }
```

Os valores possíveis para essa propriedade são as palavras-chave: `ease`, `linear`, `ease-in`, `ease-out`, `ease-in-out` e a função `cubic-bezier()`. As palavras-chave definem um tipo predefinido de curva Bézier, e a função `cubic-bezier` permite definir uma curva Bézier personalizada.

O valor inicial para essa propriedade é `ease`. Essa propriedade não é herdável e se aplica a todos os elementos da marcação, bem como aos pseudoelementos `:before` e `:after`.

As palavras-chave mostradas definem curvas padrão para transições, conforme mostrado na figura 14.2.

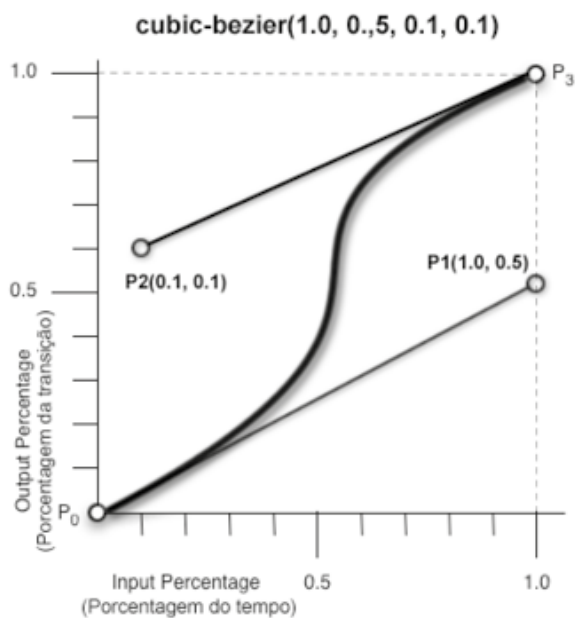
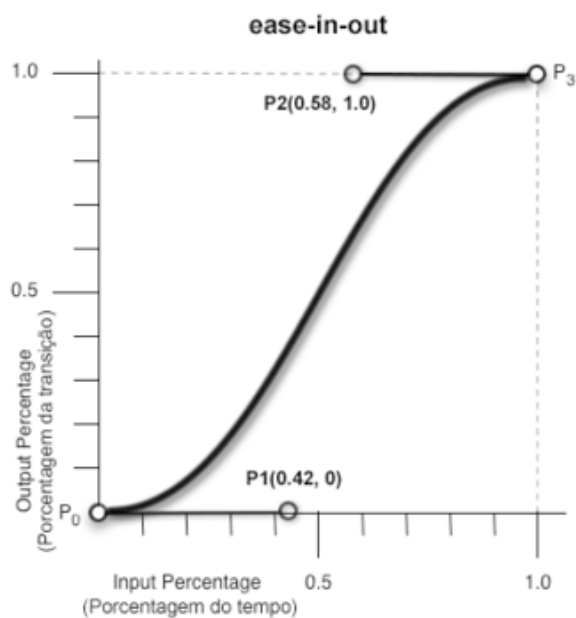
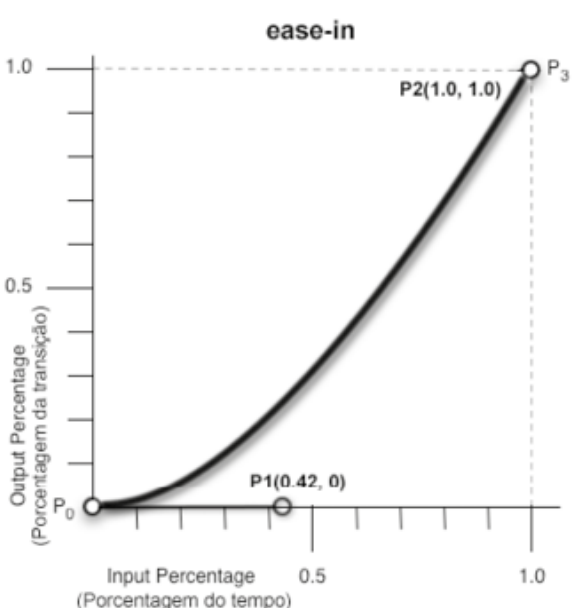
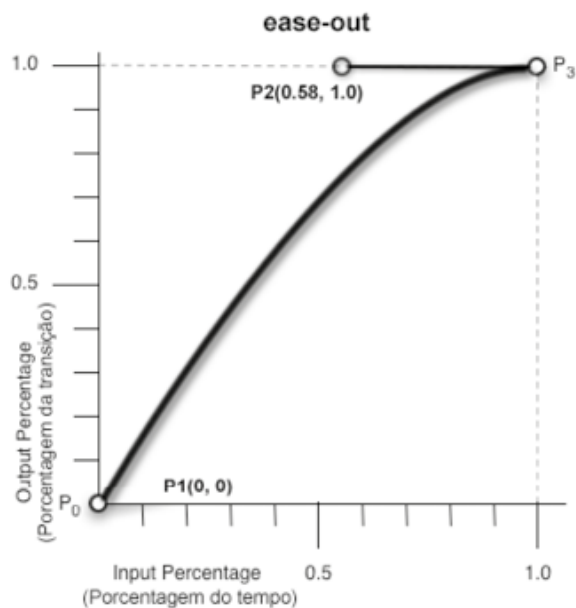
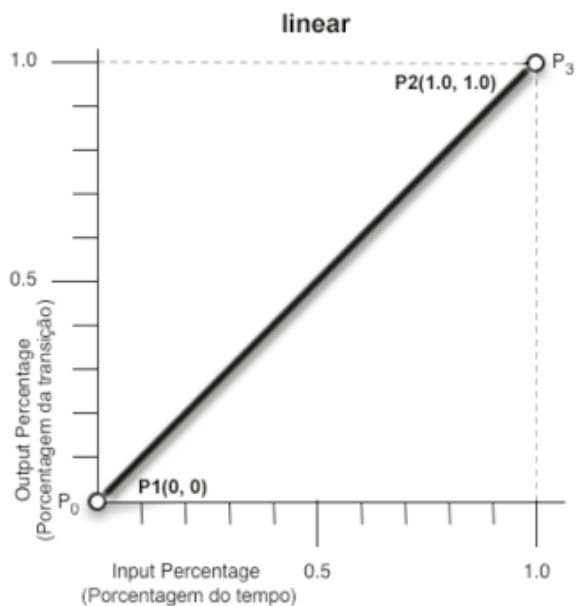
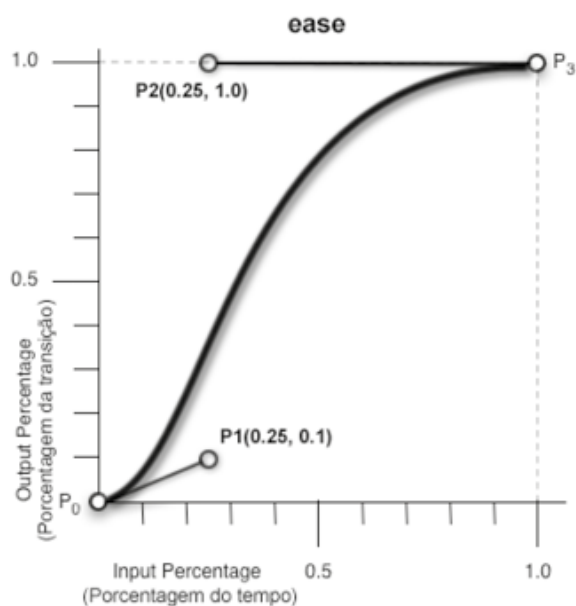


Figura 14.2 – Funções padrão de transição.

Para definir uma curva Bézier de transição personalizada, usamos a função `cubic-bezier(x1, x2, y2, y2)`, na qual os parâmetros têm o seguinte significado: • `x1` – coordenada horizontal do ponto P1; • `y1` – coordenada vertical do ponto P1; • `y1` – coordenada horizontal do ponto P2; • `y2` – coordenada vertical do ponto P2.

As palavras-chave para valor dessa propriedade definem curvas padrão, conforme mostrado na figura 14.2. Assim, as seguintes declarações são equivalentes: `transition-timing-function: ease = transition-timing-function: cubic-bezier(0.25, 0.1, 0.25, 1.0)` `transition-timing-function: linear = transition-timing-function: cubic-bezier(0, 0, 1.0, 1.0)` `transition-timing-function: ease-in = transition-timing-function: cubic-bezier(0.42, 0.0, 1.0, 1.0)` `transition-timing-function: ease-out = transition-timing-function: cubic-bezier(0, 0, 0.58, 1.0)` `transition-timing-function: ease-in-out = transition-timing-function: cubic-bezier(0.42, 0, 0.58, 1.0)`

14.2.4 transition-delay

Essa propriedade define um tempo de espera para o começo da transição a ser contado a partir do momento em que uma ação inicia a transição. O valor dessa propriedade é o número de segundos expressando o tempo de espera.

O valor inicial para essa propriedade é `0` (a transição começa imediatamente), e números negativos são permitidos. Definir um número negativo para essa propriedade equivale a definir a transição com seu início no ponto em que estaria se já tivesse decorrido o tempo declarado no valor negativo.

Para exemplificar e esclarecer esse comportamento, considere uma transição linear da propriedade `width` de um `box` (de `100px` para `500px`), a se realizar em quatro segundos e com um valor para essa propriedade igual a `-2` segundos, conforme regra CSS a seguir: `.box {`

```
width: 800px;
transition-property: width;
transition-duration: 4s;
transition-timing-function: linear;
transition-delay: -2s;

}
```

Na figura 14.3, mostramos o início da transição, comparando-a com uma transição idêntica sem definição de retardo negativo.

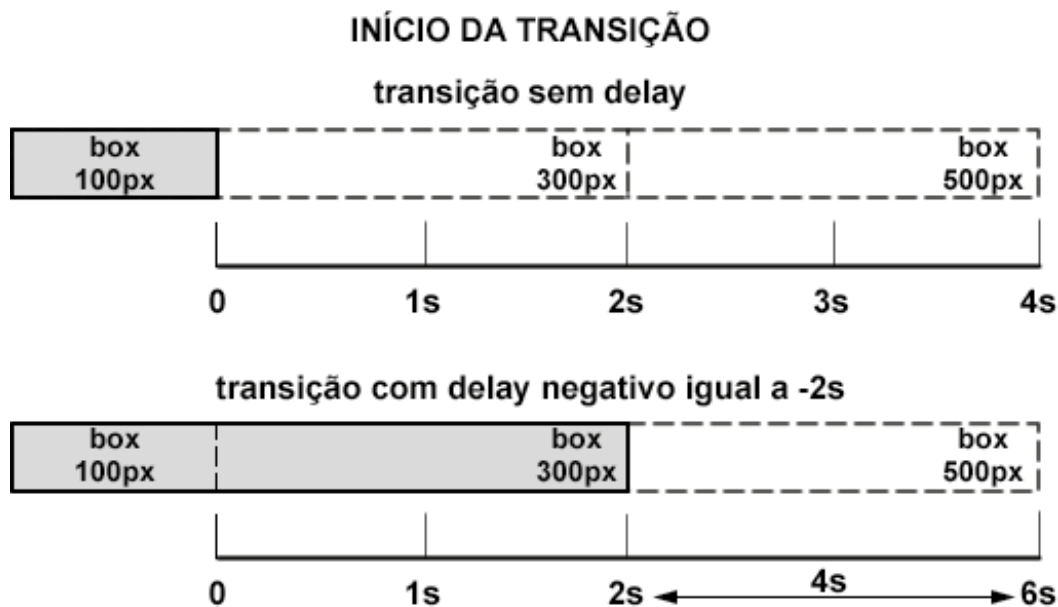


Figura 14.3 – Propriedade *transition-delay*.

Note que o retardo negativo de dois segundos força o início da transição, como se a largura do box fosse aquela que ele teria quando transcorridos dois segundos (**300px**). O box “pula” instantaneamente para a largura de **300px** e continua a transição linear por quatro segundos.

Essa propriedade não é herdável e se aplica a todos os elementos da marcação, bem como aos pseudo-elementos **:before** e **:after**.

14.2.5 transition

Essa propriedade é a forma abreviada de se declarar todas as propriedades para transição. Os valores para essa propriedade são: **transition-property**, **transition-duration**, **transition-timing-function** e **transition-delay**, que devem ser declarados nessa ordem, separados por espaço em branco. Com uso dessa sintaxe é possível, também, definir transições diferentes para diferentes propriedades. Para isso, separe a lista de valores com vírgula. Os exemplos a seguir esclarecem a sintaxe para essa propriedade: `seletor { color: blue; transition: color 2s ease-in; }`

```
seletor { background-color: red; transition: background-color 4s linear 1s
}
seletor { border-width: 8px; transition: border-width 3s ease }
seletor { color: blue; border-width: 8px; transition: color 2s ease-in 1s,
border-width 3s ease }
```

14.3 Exemplo

O exemplo a seguir, disponível no site do livro, demonstra uma transição da propriedade **width** de sete boxes, simultaneamente. Cada um deles usa uma função de transformação, sendo que, para o último box, repetimos a função linear, mas

definimos um retardo na execução da transição. Nesse caso, o retardo foi por um tempo igual ao da transição dos seis boxes anteriores. Um script JavaScript controla o início da transição por clique em um botão.

► CSS

```
div {
width: 200px;
padding: 5px 0;
text-align: center;
border: 3px solid;
border-color: red;
border-radius: 10px;
font: bold 14px Arial,sans-serif;
display: block;
background: gray;
color: white;
margin: 5px 0;

}

.tr1 {
width: 800px;
-moz-transition: width 2s linear;
-webkit-transition: width 2s linear;
-o-transition: width 2s linear;
transition: width 2s linear; }
.tr2 {
width: 800px;
-moz-transition: width 2s ease;
-webkit-transition: width 2s ease;
-o-transition: width 2s ease;
transition: width 2s ease; }
.tr3 {
width: 800px;
-moz-transition: width 2s ease-in;
-webkit-transition: width 2s ease-in;
-o-transition: width 2s ease-in;
transition: width 2s ease-in; }
.tr4 {
width: 800px;
-moz-transition: width 2s ease-out;
-webkit-transition: width 2s ease-out;
-o-transition: width 2s ease-out;;
transition: width 2s ease-out; }
.tr5 {
width: 800px;
```

```

-moz-transition: width 2s ease-in-out;
-webkit-transition: width 2s ease-in-out;
-o-transition: width 2s ease-in-out;
transition: width 2s ease-in-out; }

.tr6 {
width:800px;
-moz-transition: width 2s cubic-bezier(1.0, 0.6, 0, 1.0); -webkit-
transition: width 2s cubic-bezier(1.0, 0.6, 0, 1.0); -o-transition:
width 2s cubic-bezier(1.0, 0.6, 0, 1.0); transition: width 2s cubic-
bezier(1.0, 0.6, 0, 1.0); }

.tr7 {
width: 800px;
-moz-transition: width 2s linear 2s;
-webkit-transition: width 2s linear 2s;
-o-transition: width 2s linear 2s;
transition: width 2s linear 2s; }

h2 { margin:0; }

```

► HTML

```

<div><h2>linear</h2></div> <div><h2>ease</h2></div>
<div><h2>ease-in</h2></div>
<div><h2>ease-out</h2></div>
<div><h2>ease-in-out</h2></div> <div><h2>cubic-bezier</h2></div> <div>
<h2>DELAY</h2></div>
<button type="button">IR</button>
<button type="button">RESET</button>

```

```

► JavaScript var boxes = document.getElementsByTagName('div');
var botao = document.getElementsByTagName('button');
botao[0].onclick = function() {
for (var i=0; i<boxes.length; i++) {
boxes[i].className = 'tr'+(i+1);

}

}

botao[1].onclick = function() {
for (var j=0; j<boxes.length; j++) {
boxes[j].removeAttribute('class');
boxes[j].removeAttribute('style');

}

}

```

Para visualizar mais exemplos de transição, visite <http://kwz.me/gm>.

CAPÍTULO 15

Animações CSS

As especificações para o Módulo Animações CSS estão no estágio de Rascunho de Trabalho do W3C.

Neste capítulo, estudaremos as funcionalidades desse módulo. Os navegadores atuais Internet Explorer 10 e Firefox 23 já as suportam nativamente e os navegadores Chrome 29, Safari 6 e Opera 16 as suportam com uso do prefixo proprietário -**webkit-**, e assim podemos começar a realizar nossos estudos e nossas experiências com animações CSS.

15.1 Introdução

Segundo o W3C, a finalidade do Módulo para Animações é criar funcionalidades que permitam ao autor definir uma mudança de valor de uma propriedade CSS, de modo que ela (a mudança de valor) ocorra de forma suave, em um espaço de tempo especificado.

Normalmente, quando o valor de uma propriedade CSS é alterado, ocorre uma mudança instantânea na renderização. Um exemplo clássico desse comportamento é a mudança de apresentação de um link quando o usuário coloca o mouse sobre ele. Efeito que se consegue com uso do seletor **a:hover**.

As funcionalidades desse módulo visam a criar mecanismos que permitam definir uma mudança de apresentação de forma suave ao longo de um determinado tempo.

Esse módulo estende as funcionalidades do Módulo para Transições, estudado no capítulo 14. A diferença desse módulo para o módulo das transições é que as funcionalidades desse módulo fornecem mecanismos adicionais, capazes de exercer um controle refinado sobre os valores intermediários da animação.

Tais mecanismos foram criados com uma regra denominada **@keyframe**. Uma animação é uma sequência de **frames**, na qual cada um deles representa o estado da animação em um determinado instante. De modo análogo, a regra **@keyframe** foi criada para definir os valores da propriedade CSS do elemento animado em um determinado instante da animação.

Embora seja possível criar uma animação definindo todos seus **frames**, a finalidade principal das funcionalidades desse módulo não é essa, mas, sim, estender as funcionalidades do módulo para transições, ou seja, usar **frames** na animação em quantidade suficiente para obter efeitos que não são possíveis com uso de transições que seguem seu curso ao longo de uma curva predefinida e não permitem interferência.

A sintaxe CSS para criação e inserção de um **frame**, com regras de estilização em uma animação, é muito simples. Observe um exemplo de regra CSS para criar um frame: `@keyframes nome {`

```
0% { width: 1px; }  
50% { height: 400px; }  
100% { width: 900px; }
```

```
}
```



A regra CSS mostrada, sozinha, não faz efeito algum, ela deve ser combinada com regras CSS que definem os valores para as propriedades da animação, como veremos adiante.

A sintaxe mostrada, destinada a criar **frames**, deve estar em conformidade com algumas diretrizes básicas que enumeramos a seguir:

- A regra CSS deve começar com a declaração da diretiva `@keyframe`, imediatamente seguida por uma string livremente escolhida pelo autor e que designará o nome da animação. Como toda regra CSS, essa também começa e termina com sinais abre chaves (`{`) e fecha chaves (`}`).

- O ponto de inserção do **frame** na animação deve ser definido com uso de porcentagem. A porcentagem é o seletor que casa com a quantidade de animação já transcorrida, por exemplo, em uma animação com duração total igual a seis segundos, uma porcentagem de 50% insere um frame depois de transcorridos três segundos do início da animação. A cada porcentagem, são definidas uma ou mais declarações CSS que estilizam propriedade(s) naquele frame.
- Não é válida a declaração de um só frame em uma animação. Para declarar um ou mais frames, devemos obrigatoriamente declarar os frames 0% e 100%. Assim, a quantidade mínima de frames em uma animação (se houver declaração de frame intermediário) será três.
- É obrigatório constar o sinal de porcentagem (%) quando se usa o seletor 0%, contrariando a regra geral de que zero é zero e não depende da unidade.
- Existe uma sintaxe alternativa para os **frames** inicial e final. Em lugar de 0%, podemos usar a palavra-chave **from**, e em lugar de 100%, podemos usar a palavra-chave **to**.

Nem todas as propriedades CSS são elegíveis para animações. As especificações para as CSS3 estabelecem que somente as propriedades animáveis (passíveis de ser animadas) podem ser declaradas com transição. Reproduzimos, a seguir, a tabela constante do capítulo 14, que apresenta as propriedades CSS animáveis.

Tabela 15.1 – Propriedades CSS animáveis

Propriedade	Tipo
background-color	cor
background-image	somente gradientes
background-position	medida CSS e porcentagem
border-bottom-color	cor
border-bottom-width	medida CSS
border-color	cor
border-left-color	cor
border-left-width	medida CSS
border-right-color	cor
border-right-width	medida CSS
border-spacing	medida CSS
border-top-color	cor
border-top-width	medida CSS
border-width	medida CSS
bottom	medida CSS e porcentagem
color	cor
crop	retângulo
font-size	medida CSS e porcentagem
font-weight	número
grid-*	vários
height	medida CSS e porcentagem
left	medida CSS e porcentagem
letter-spacing	medida CSS
line-height	número, medida CSS e porcentagem
margin-bottom	medida CSS

Propriedade	Tipo
margin-left	medida CSS
margin-right	medida CSS
margin-top	medida CSS
max-height	medida CSS e porcentagem
max-width	medida CSS e porcentagem
min-height	medida CSS e porcentagem
min-width	medida CSS e porcentagem
opacity	número
outline-color	cor
outline-offset	número inteiro
outline-width	medida CSS
padding-bottom	medida CSS
padding-left	medida CSS
padding-right	medida CSS
padding-top	medida CSS
right	medida CSS e porcentagem
text-indent	medida CSS e porcentagem
text-shadow	sombra
top	medida CSS e porcentagem
vertical-align	palavra-chave, medida CSS e porcentagem
visibility	visibilidade
width	medida CSS e porcentagem
word-spacing	medida CSS e porcentagem
z-index	integer

Propriedade	Tipo
zoom	número

Os navegadores atuais Firefox 7, Safari 5 e Chrome 14 suportam as funcionalidades desse módulo com uso dos prefixos proprietários `-moz-` e `-webkit-`, para Firefox e Chrome/Safari, respectivamente. O Opera 11 e os Internet Explorer anteriores ao IE10 não as suportam.

Todos os exemplos deste capítulo serão desenvolvidos com uso do prefixo proprietário.

15.2 Propriedades

As propriedades para animações são:

- `animation-name`
- `animation-duration` • `animation-timing-function` • `animation-iteration-count`
- `animation-direction` • `animation-play-state` • `animation-delay`
- `animation`

A seguir, veremos a finalidade e os valores possíveis para cada uma delas.

15.2.1 `animation-name`

Essa propriedade destina-se a definir o nome ou uma lista de nomes de animação, separados por vírgula. Tais nomes são aqueles que foram livremente escolhidos pelo autor para constar na regra CSS `@keyword`, como mostrado anteriormente. O valor inicial é `none` (nenhum animação será aplicada), essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

A sintaxe para essa propriedade é conforme exemplos mostrados a seguir:

```
@keyframes alongar {
  0% { width: 100px; }
  50% { height: 200px; }
  100% { width: 900px; }
}
```

```
@keyframes aumentar {
  0% { font-size:14px; }
  25% { font-size: 18px; }
  50% { font-size: 22px; }
  75% { font-size: 26px; }
```

```
100% { font-size: 30px; }
```

```
}
```

```
seletor { animation-name: alongar; } /* aplica a animação alongar */  
seletor { animation-name: aumentar; } /* aplica a animação aumentar */  
seletor { animation-name: alongar, aumentar; } /* aplica a animação  
alongar e aumentar */
```

Para as regras CSS que definem os **frames**, a sintaxe alternativa com uso de palavras-chave em lugar de 0% e 100% é mostrada a seguir: **@keyframes** alongar {

```
from { width: 100px; }  
50% { height: 200px; }  
to { width: 900px; }
```

```
}
```

```
@keyframes aumentar {  
from { font-size:14px; }  
25% { font-size: 18px; }  
50% { font-size: 22px; }  
75% { font-size: 26px; }  
to { font-size: 30px; }
```

```
}
```

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo **-webkit-**.

```
@-moz-keyframes alongar {  
...
```

```
}
```

```
@-moz-keyframes aumentar {  
...
```

```
}
```

```
seletor { -moz-animation-name: alongar; }  
seletor { -moz-animation-name: aumentar; }  
seletor { -moz-animation-name: alongar, aumentar; }
```

15.2.2 animation-duration

Essa propriedade define o tempo para a animação completar um ciclo. O valor dessa propriedade é o número de segundos, expressando a duração de um ciclo completo

da animação. No caso de uma lista de nomes de animação a aplicar definidos com a propriedade **animation-name**, o valor dessa propriedade poderá ser também uma lista dos números de segundos para cada ciclo, separados por vírgula, ou um só número de segundos válido para o ciclo de todas as propriedades animadas. Caso a lista dos números de segundos não seja em quantidade igual ao número de nomes de animação, ela (lista de segundos) se repetirá até completar o total do número de nomes de animação.

O valor inicial para a duração do ciclo da animação é 0 (não há animação), e números negativos serão considerados zero. Essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir:

```
seletor {
```

```
  animation-name: pular;
  animation-duration: 8s; /* tempo da animação igual a 8s */

}
```

```
seletor {
  animation-name: saltar, quicar;
  animation-duration: 12s, 10s; /* animação saltar em 12s e quicar em 10s
  */
}
```

```
seletor {
  animation-name: subir, aumentar, cair;
  animation-duration: 10s, 5s; /* animação subir em 10s, aumentar em 5s e
  cair em 10s */
}
```

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo **-webkit-**.

```
seletor {
  ...
  -moz-animation-duration: 8s; }
seletor {
  ...
  -moz-animation-duration: 12s, 10s; }
seletor {

...;
```

```
-moz-animation-duration: 10s, 5s; }
```

15.2.3 animation-timing-function

Essa propriedade define uma função que descreve como são calculados os valores intermediários de um ciclo de uma animação. Toda a teoria de funcionamento dessa propriedade foi descrita quando estudamos transições, no capítulo 14. Caso você não tenha lido aquele capítulo ou não esteja seguro de ter entendido o mecanismo de funcionamento dessa propriedade, recomendo a leitura do item [14.2.3] antes de prosseguir.

Os valores possíveis para essa propriedade são as palavras-chave: **ease**, **linear**, **ease-in**, **ease-out**, **ease-in-out** e a função **cubic-bezier()**. As palavras-chave definem um tipo predefinido de curva Bézier, e a função **cubic-bezier** permite definir uma curva Bézier personalizada.

O valor inicial para essa propriedade é **ease**. Essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir: seletor { **animation-timing-function**: ease; }

```
seletor { animation-timing-function: linear; }  
seletor { animation-timing-function: ease-in; }  
seletor { animation-timing-function: ease-out; }  
seletor { animation-timing-function: ease-in-out; }  
seletor { animation-timing-function: cubic-bezier(x1, y1, x2, y2); }
```

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo **-webkit-**.

```
seletor { -moz-animation-timing-function: ease; }  
seletor { -moz-animation-timing-function: linear; }  
seletor { -moz-animation-timing-function: ease-in; }  
seletor { -moz-animation-timing-function: ease-out; }  
seletor { -moz-animation-timing-function: ease-in-out; }  
seletor { -moz-animation-timing-function: cubic-bezier(x1, y1, x2, y2); }
```

15.2.4 animation-iteration-count

Essa propriedade define o número de ciclos da animação. O valor dessa propriedade é um número real positivo.

O valor inicial é **1** (a animação completa um ciclo e para), um valor igual a infinito determina a repetição contínua da animação (a animação nunca para). Números negativos serão considerados zero (a animação não acontece). São válidos números decimais, e nesses casos, a parte decimal da animação define a fração de um ciclo em

que a animação acontecerá, por exemplo: um valor igual a 3,4 faz com que a animação complete três ciclos e continue por mais 40% (0,4) de um ciclo, perfazendo os 3,4 ciclos definidos. Essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir:

```
seletor { animation-iteration-count: 4; }  
seletor { animation-iteration-count: 6,8; }  
seletor { animation-iteration-count: infinite; }
```

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo `-webkit-`.

```
seletor { -moz-animation-iteration-count: 4; }  
seletor { -moz-animation-iteration-count: 6,8; }  
seletor { -moz-animation-iteration-count: infinite; }
```

15.2.5 animation-direction

Essa propriedade define a ocorrência da animação em ordem inversa, ou seja, há alternância da animação conforme o ciclo seja de ordem par ou ímpar. Nos ciclos de ordem ímpar, a animação é na direção normal, e nos ciclos de ordem par, a animação é na ordem inversa. Conclui-se com facilidade que essa propriedade só funciona em animações com o valor declarado para a propriedade **animation-iteration-count** igual ou maior do que dois. Os valores possíveis para essa propriedade são **normal** e **alternate**.

O valor inicial é **normal** (a animação não reverte). Essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

Convém notar que em animações reversas a função de animação também é revertida em ciclos pares. Por exemplo: uma animação alternada, cujo valor da propriedade **animation-timing-function** é **ease-in**, quando reverter o efeito será **ease-out**.

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir:

```
seletor { animation-direction: normal; }  
seletor { animation-direction: alternate; }
```

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo `-webkit-`.

```
seletor { -moz-animation-direction: normal; }  
seletor { -moz-animation-direction: alternate; }
```


15.2.6 animation-play-state

Essa propriedade define o estado do curso da animação, ou seja, se a animação está em andamento ou em pausa. Os valores possíveis para essa propriedade são **running** e **paused**.

O valor inicial é **running** (a animação está em andamento). Essa propriedade não é herdável e se aplica a todos os elementos nível de bloco e inline.

Quando uma animação é colocada em estado de pausa, ela para no estado em que se encontra, e se mais tarde for colocada no estado animado, ela inicia no ponto em que parou.

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir:
seletor { **animation-play-state**: paused; }

seletor { **animation-play-state**: running; }

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo **-webkit-**.

seletor { **-moz-animation-play-state**: paused; }

seletor { **-moz-animation-play-state**: running; }



O W3C está estudando a opção de retirar essa propriedade da especificação, tendo em vista que seu efeito pode ser obtido com uso de outras técnicas CSS.

15.2.7 animation-delay

Essa propriedade define um tempo de espera para o começo da animação a ser contado a partir do momento em que uma ação inicia a animação. O valor dessa propriedade é o número de segundos expressando o tempo de espera.

O valor inicial para essa propriedade é **0** (a animação começa imediatamente), e números negativos são permitidos. Definir um número negativo para essa propriedade equivale a definir a transição com seu início no ponto em que estaria se já tivesse decorrido o tempo declarado no valor negativo.

Para mais detalhes sobre o funcionamento dessa propriedade, consulte o item [14.2.4].

A sintaxe para essa propriedade é conforme os exemplos mostrados a seguir:
seletor { **animation-delay**: 4s; }

seletor { **animation-delay**: -3s; }

A sintaxe para aplicação dessas regras com uso dos prefixos proprietários é conforme mostrado a seguir para navegadores Mozilla. Para a engine WebKit, use o prefixo **-webkit-**.

```
seletor { -moz-animation-delay: 4s; }  
seletor { -moz-animation-delay: 3s; }
```

15.2.8 animation

Essa propriedade é a forma abreviada de se declarar todas as propriedades para animação. Os valores para essa propriedade são: `animation-name`, `animation-duration`, `animation-timing-function`, `animation-delay`, `animation-iteration-count` e `animation-direction`, que devem ser declarados nessa ordem, separados por espaço em branco. Com uso dessa sintaxe é possível, também, definir transições diferentes para diferentes propriedades. Para isso, separe a lista de valores com vírgula. Os exemplos a seguir esclarecem a sintaxe para essa propriedade: `seletor { animation: rodar 4s ease-out 1s 8 alternate; }`

```
seletor { transition: subir 8s linear 1s infinite normal; }  
seletor { transition: virar 12s ease-in-out; }  
seletor { transition: colorir 6s ease-in infinite, aumentar 14s ease 2s 4  
alternate; }
```

15.3 Exemplo

Mostramos, a seguir, um exemplo muito simples que visa a fornecer uma visão geral do princípio de funcionamento das animações. A partir dele, você poderá seguir com o desenvolvimento de animações mais elaboradas e sofisticadas. Um script jQuery controla a animação por clique em um botão.

► CSS

```
body {  
margin:20px;  
background: aliceblue;  
font: 100% sans-serif;  
  
}  
  
div {  
font-size: 80%;  
width: 100px;  
height: 100px;  
line-height: 100px;  
margin: 5px 30px 0 0;  
border: 4px solid lime;  
border-radius: 10px;  
background: black;  
  
}
```

```

button {
display: block;
width: 100px;
margin-top: 2px;
font-size: 60%;
font-weight: bold;
height: 25px;
border-radius: 6px;
color: white;
background: green;

}

.animar {
-moz-animation: alongar 2s linear 6 alternate;
-webkit-animation: alongar 2s linear 6 alternate;

}

h2 { margin: 0;color: lime; text-align: center; }
@-moz-keyframes alongar {
0% { width: 100px; }
50% { height: 200px; }
100% { width: 920px; }

}

@-webkit-keyframes alongar {
0% { width: 100px; }
50% { height: 200px; }
100% { width: 920px; }

}

@keyframes alongar {
0% { width: 100px; }
50% { height: 200px; }
100% { width: 920px; }

}

```

► HTML

```

<button type="button">ANIMAR</button>
<div><h2>Eu &#9829; CSS!</h2></div>

```

```

► jQuery $(document).ready(function() {
  $('button').eq(0).click(function() {
    if ($('#div').hasClass('animar')) {

```

```
$('#div').removeClass();
$(this).html('<span>ANIMAR</span>').css('background','green'); } else {
$('#div').addClass('animar');
$(this).html('<span>RESET</span>').css('background','red'); }
})
})
```

CAPÍTULO 16

Template Layout

As especificações CSS para o Módulo (Grid) Template Layout estão no estágio de Rascunho de Trabalho do W3C.

Neste capítulo, estudaremos as funcionalidades desse módulo. Em outubro de 2013, nenhum navegador suportava as funcionalidades desse módulo exceto o Internet Explorer 10 com uso do prefixo proprietário `-ms-`; contudo, como veremos neste capítulo, existe um plugin jQuery que habilita os navegadores Internet Explorer 7+, Firefox 2+, Opera 9.6+, Safari 3.1+ e Chrome 1+ para suporte, e assim podemos começar a realizar nossos estudos e nossas experiências com a construção de layouts usando as funcionalidades desse módulo.

16.1 Introdução

O W3C descreve a finalidade do Módulo Template Layout conforme tradução adaptada, a seguir: CSS é uma linguagem simples para criar folhas de estilo que especificam a renderização de documentos HTML e outros documentos estruturados. Essa especificação faz parte das CSS3 e cria funcionalidades capazes de construir layouts de alto nível destinados a tarefas complexas, tais como posicionamento e alinhamento de “widgets” em interfaces gráficas de usuários ou construção de um grid para layout de uma página ou janela. Em particular, tais tarefas se referem ao desenvolvimento de projetos nos quais a ordem de apresentação visual é diferente da ordem dos elementos no código do documento. As propriedades para especificar fontes, cores, alinhamentos de textos, listas numeradas, tabelas *etc.* são objeto de outros módulos das especificações para CSS3.

A estilização de uma página web, um formulário ou uma interface gráfica pode ser dividida em duas partes: definição do grid e especificação de fontes, cores, margens *etc.* para os elementos da página. Essas duas partes não são completamente independentes, pois uma vez que o modelo CSS1 de layout e posicionamento estabelece regras de layout (ou posicionamento) dos elementos-filhos de um container dependentes do próprio container, fica implícita a dependência. Tal dependência pode ser classificada como política de *posicionamento baseado em container*.

Esse módulo altera a política de layout estabelecida pelas CSS1 e cria uma política chamada *posicionamento baseado em template*. Segundo essa política, cada elemento container é capaz de conter seu próprio grid invisível para alinhamento e posicionamento.

16.2 Visão geral

Antes de estudarmos as funcionalidades desse módulo, vamos apresentar uma visão geral do mecanismo de construção de layout nele previsto. Essa visão geral tem por objetivo fornecer uma base inicial para melhor entendimento do funcionamento de cada propriedade do módulo.

16.2.1 Plugin CSS Template Layout

Como dissemos anteriormente, nenhum navegador atual suporta esse módulo, contudo o desenvolvedor alemão Alexis Richard Pieter Devéria, nascido na Inglaterra e residente nos EUA, lançou, no final de abril de 2009, um plugin jQuery denominado *css-template-layout*, que torna possível aos navegadores Internet Explorer 7+, Firefox 2+, Opera 9.6+, Safari 3.1+ e Chrome 1+ suportar as funcionalidades do rascunho de trabalho do W3C para esse módulo.

O plugin é de distribuição e uso livres e gratuitos e está disponível para download no repositório de códigos do Google, no endereço <http://kwz.me/ge>. Se você pretende fazer suas experiências ou desenvolver com esse módulo, a primeira providência é baixar o plugin e linkar para a página não só o plugin, mas também uma versão da biblioteca jQuery, já que se trata de um plugin jQuery.

Além disso, você precisa inicializar o plugin. Tudo como mostrado no código a seguir, a ser inserido na seção **head** do documento: `<head>`

```
...
<script
src="http://ajax.googleapis.com/ajax/libs/jquery/1.6.4/jquery.min.js">
</script> <!-- Biblioteca jQuery --> <script
src="jquery.tpl_layout1.1.6.min.js"></script> <!-- Plugin para suporte
para o Módulo --> <script>
$(document).ready(function() {
$.setTemplateLayout(); // inicialização do plugin

});

</script>
</head>>
```

A função que inicializa o plugin pode, opcionalmente, receber vários parâmetros. Não é do escopo deste livro analisar o plugin, assim, sugiro que você leia a documentação que se encontra na página para download do plugin, indicada anteriormente.

16.2.2 Regra CSS para descrição do layout

Considere um layout de três colunas com topo e rodapé, conforme a marcação

mostrada a seguir: ► **HTML**

```
<body>
<div id="tudo">
<header id="topo">
<h1>Topo</h1>
</header>
<nav id="nav">
<ul>
<li><a href="#">Link A</a></li>
<li><a href="#">Link B</a></li>
<li><a href="#">Link C</a></li>
</ul>
</nav>
<div id="principal">
<p id="p1">Lorem ipsum ...</p>
<p id="p2">Morbi eleifend ...</p>
<p id="p3">Phasellus libero ...</p>
</div>
<aside id="aux">
<p>Pellentesque sapien...</p>
</aside>
<footer id="rodape">
<p>Rodapé</p>
</footer>

</div> <!-- fim da div#tudo -->
</body>
```

Considere a renderização do layout conforme mostrado na figura 16.1. Os círculos com letras e pontos não fazem parte da marcação e foram acrescentados para facilitar a explicação que daremos adiante.

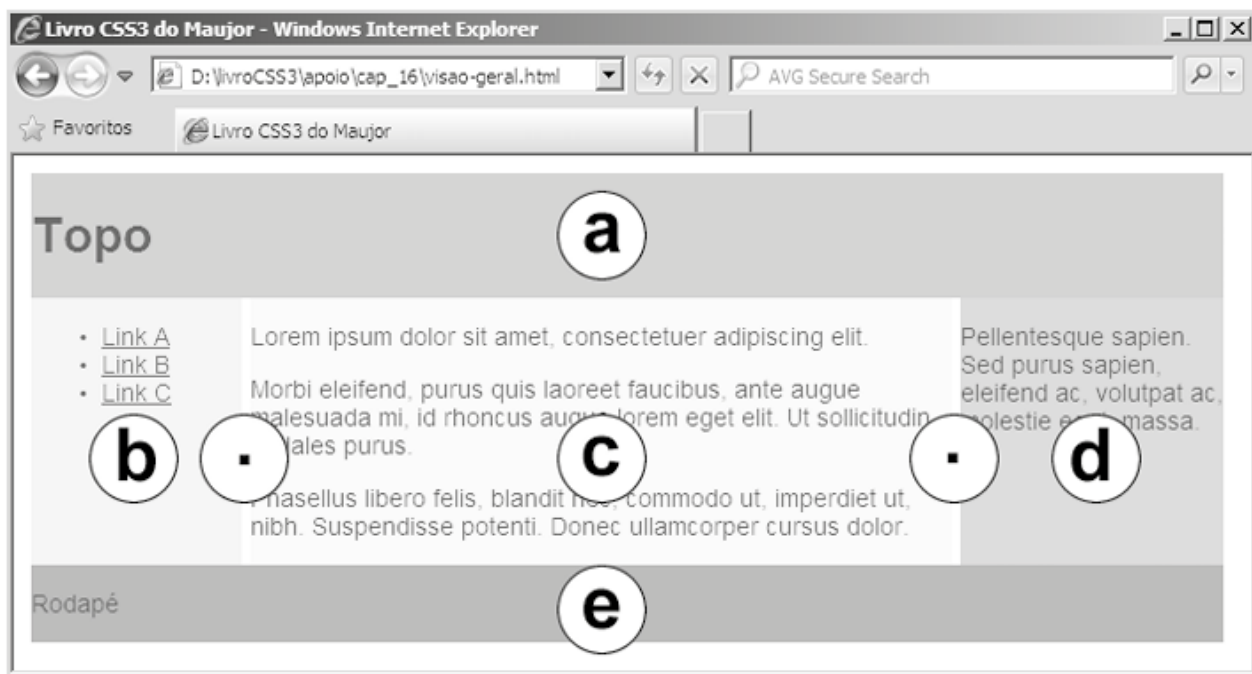


Figura 16.1 – Layout três colunas.

O mecanismo de construção de layout previsto por esse módulo segue duas diretrizes gerais, conforme relacionadas a seguir:

- Cada um dos boxes do layout deve ser designado por uma letra do alfabeto (sem acentuações), minúscula ou maiúscula, de livre escolha do autor. No nosso layout, escolhemos letras minúsculas, como listadas a seguir:

Letra	Box
a	Box topo
b	Box navegação
c	Box principal
d	Box auxiliar
e	Box rodapé

- Espaçamentos horizontais (entre colunas) são representados por um ponto, espaçamentos verticais serão mostrados adiante.

Para fixar a ideia de nomeação dos boxes do layout com uso de letras e pontos, observe outro exemplo, conforme marcação mostrada a seguir: ► **HTML**

```
<header id="topo">
<h1>Topo1</h1>
<h2>Topo2</h2>
</header>
<div id="principal">
<p>Lorem ipsum ...</p>
```



```

<p>Morbi eleifend... </p>
<p>Phasellus libero ...</p>
</div>
<aside id="aux">
<p>Pellentesque sapien. ...</p>
</aside>
<nav id="nav-principal">
<ul>
<li><a href="#">Link A</a></li>
<li><a href="#">Link B</a></li>
<li><a href="#">Link C</a></li>
</ul>
</nav>
<nav id="nav-secundaria">
<ul>
<li><a href="#">Link A</a></li>
<li><a href="#">Link B</a></li>
<li><a href="#">Link C</a></li>
</ul>
</nav>
<footer id="rodape">
<p>Rodapé</p>
</footer>

```

Considere a renderização do layout, conforme mostrado na figura 16.2. Os círculos com letras e pontos não fazem parte da marcação e foram acrescentados para facilitar a explicação que daremos adiante.

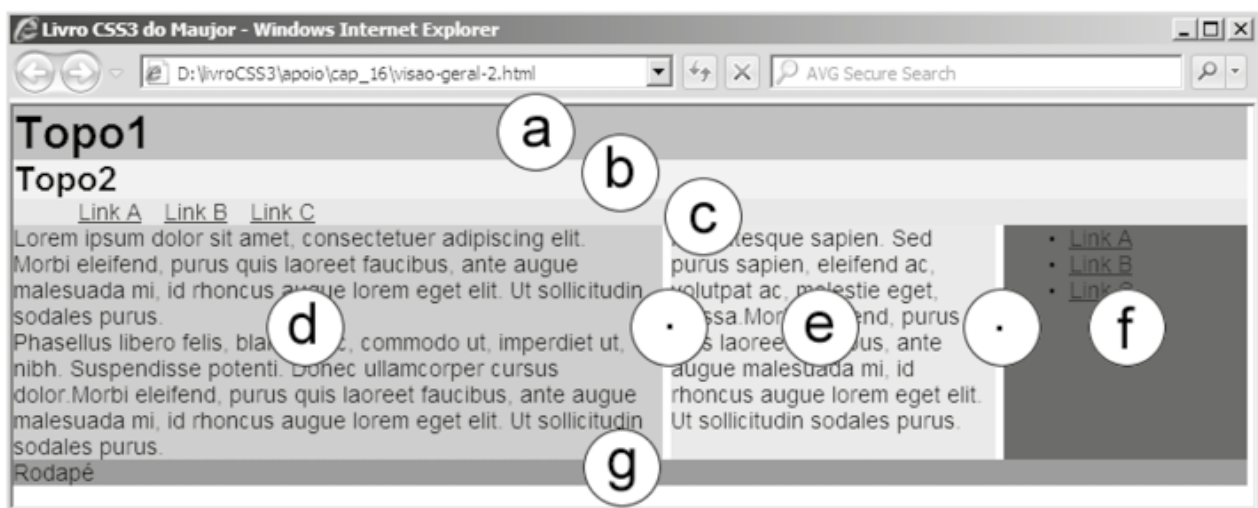


Figura 16.2 – Layout três colunas.

Tendo entendido a designação dos boxes e espaçamentos horizontais com letras e pontos, vamos passar ao estudo das propriedades desse módulo.

16.3 Propriedades

Existem apenas duas propriedades e um pseudoelemento para construção de layouts com uso das funcionalidades do Módulo Template Layout, conforme listado a seguir: • **display**

- **position**
- **::slot()**

A seguir, veremos a finalidade e os valores possíveis para cada uma das propriedades e do pseudoelemento.

16.3.1 display

Essa propriedade destina-se a descrever o layout com uma regra CSS. Para descrever o layout, a regra CSS usa as letras e os pontos que o autor escolheu para designar os boxes do layout, conforme mostrado na figura 16.1.

Para o layout daquela figura, a regra CSS que o descreve é mostrada a seguir: ► **CSS**

```
#tudo {  
  display: "aaaaa"  
  "b.c.d"  
  "eeee"  
}
```

Para escrever a sintaxe da regra CSS que descreve o layout, seguimos as seguintes diretrizes: • O seletor da regra CSS é o box container do layout. No nosso exemplo, englobamos o layout em um elemento **div** com **id="tudo"**, assim, nosso seletor é **#tudo**. Poderíamos não ter usado esse **div**, e nesse caso, o seletor que englobaria o layout e, em consequência, o seletor para a regra CSS descritiva do layout seria o elemento **body**.

- Declaramos a propriedade **display** com o valor igual a um conjunto de strings formado pelas letras e pelos pontos designativos dos boxes dos layouts e dispostos em um formato idêntico àquele que têm visualmente no layout. Começamos pela linha que contenha o maior número de letras e pontos. No nosso layout, a linha dois, *b.c.d*, que contém cinco caracteres. Esse número de caracteres deverá estar contido em todas as demais linhas. Nas linhas 1 e 3 (topo e rodapé) não há espaçamentos, assim, as letras *a* e *e* se repetem cinco vezes.



Olhe para o layout com as letras designativas dos boxes como se fosse uma tabela com suas linhas, células e colunas. Cada letra ou ponto é uma célula. Comece pela linha que contém mais células. Lembre-se: em uma tabela, se uma coluna engloba duas (ou mais) células da coluna vizinha (rowspan), o número de células nas duas (ou mais) linhas é dois.

Vamos examinar o layout da figura 16.2. Sugestão: antes de continuar a leitura, pegue lápis e papel e, olhando para a figura, escreva a regra CSS que o descreve.

Olhando para o layout como se fosse uma tabela, identificamos cinco linhas e cinco colunas. As linhas 1, 2, 3 e 5 englobam as três colunas, e cada uma tem uma célula. A linha 4 tem três colunas e três células, sendo ela a que contém o maior número de letras e pontos, ou seja, cinco células, cujas letras e cujos pontos designativos são: *d.e.f*. Assim, as outras quatro strings de letras são: *aaaaa*, *bbbbb*, *cccc*, e *gggggg*. E, finalmente, a regra CSS: ► **CSS**

```
#tudo {  
  display: "aaaaa"  
  "bbbbb"  
  "cccc"  
  "d.e.f"  
  "ggggg"  
}
```

Nota: a forma que adotamos para a grafia da regra CSS tem por finalidade facilitar a visualização, o entendimento e a leitura da regra. Não é obrigatório separar o valor das strings em diferentes linhas e nem endentá-las. A regra CSS do exemplo anterior poderia ser escrita como mostrado a seguir: ► **CSS**

```
body { display: "aaaaa" "bbbbb" "cccc" "d.e.f" "ggggg" }
```

Para os layouts mostrados nas figuras 16.1 e 16.2, definimos as larguras das colunas com uso de regra CSS não mostrada na folha de estilo. A seguir, veremos como declarar larguras e espaçamentos na regra CSS descritiva do layout.

A definição da largura das colunas, da altura das linhas e dos espaçamentos entre elas faz-se com uso de declaração CSS inserida na regra que descreve o layout.

Para definir a largura das colunas e o espaçamento entre elas, a declaração é feita logo após as strings que definem o layout, e delas separada por pelo menos um espaço em branco. Observe a regra CSS mostrada anteriormente, contendo a definição de largura e espaçamento: ► **CSS**

```
body {  
  display: "aaaaa"  
  "bbbbb"  
  "cccc"  
  "d.e.f"  
  "ggggg"  
  400px 5px 200px 5px 150px; }
```

Para definir largura, observe a string que contém o maior número de caracteres diferentes. No nosso caso, é a string que descreve a quarta linha, *d.e.f*. Defina a

largura tomando como base essa linha, e na mesma sequência que as letras e os pontos nela aparecem. No nosso caso, definimos: coluna $d = 150\text{px}$, coluna $e = 200\text{px}$, coluna $f = 150\text{px}$, separação entre colunas d e $e = 5\text{px}$, e entre colunas e e f , também 5px .

Temos um layout posicionado à esquerda na tela com largura fixa igual a $400\text{px} + 5\text{px} + 200\text{px} + 5\text{px} + 150\text{px} = 760\text{px}$.

A definição da largura das colunas não é obrigatória. Você pode omitir a declaração, mas, nesse caso, todas as colunas terão a mesma largura.

Podemos definir larguras elásticas com uso de porcentagem. Observe os exemplos a seguir: • **Layout totalmente fluido:** body {

```
display: "aaaaa"
"bbbbbb"
"ccccc"
"d.e.f"
"ggggg"
53% 1% 25.5% 1% 19.5%; }
```

Nosso layout, anteriormente com largura fixa de 760px , agora é um layout totalmente fluido, preenchendo toda a largura da tela, qualquer que seja a resolução ou o tamanho da janela do usuário.

• **Layout fluido com uma coluna elástica:** body {

```
display: "aaaaa"
"bbbbbb"
"ccccc"
"d.e.f"
"ggggg"
* 5px 200px 5px 150px; }
```

Essa declaração de largura de colunas define um layout com espaçamento entre colunas igual a 5px , com as duas colunas auxiliares e navegação com largura fixa e com a coluna principal fluida. O caractere `*` (curinga) é usado nessa regra CSS para declarações de largura e altura, indicando “todo o comprimento que falta para se estender até os limites do container”.

Um layout diz-se fluido quando ocupa toda a tela, independentemente do tamanho da janela. A analogia com líquido é válida, pois tudo se passa como se a janela do navegador fosse um recipiente com líquido dentro. Para qualquer largura de janela, o líquido (layout) ocupa toda sua extensão. Layout elástico é um caso particular do layout líquido, no qual a largura total do layout depende de fatores outros, que não a largura da janela.

• **Layout fluido com duas (ou mais) colunas elásticas:** body {

```
display: "aaaaa"
```

```
"bbbbbb"
"cccccc"
"d.e.f"
"gggggg"
* 5px 22.5% 5px 150px; }
```

Essa declaração de largura define um layout com espaçamento entre colunas igual a 5px, com a coluna auxiliar elástica (22.5%), a de navegação com largura fixa (150px) e com a coluna principal fluida. O caractere * (curinga) é usado nessa regra CSS para declarações de largura e altura, indicando “todo o comprimento que falta para se estender até os limites do container”.

- **Layout fixo centrado na tela:** #tudo {
display: ".aaaaa."
".bbbbbb."
".cccccc."
".d.e.f."
".gggggg."
* 400px 5px 200px 5px 150px *; }

Para centramos, na tela, um layout fixo, usamos o caractere curinga para larguras. Primeiro, temos que informar, na string que descreve o layout, que queremos um espaçamento nas duas laterais. Para isso, acrescentamos um ponto (.) no início e no fim de cada string. Na declaração de larguras, definimos esses espaçamentos com um caractere curinga, ou seja, igual àquela largura necessária para completar a largura total do container #tudo. Teremos nosso layout centrado sem necessidade de declarar margens automáticas, como fazemos com CSS2.

E quanto à altura das linhas/colunas e seus respectivos espaçamentos verticais? Como declarar? Assim como não precisamos usar a declaração **width** para declarar largura, também não precisamos usar a declaração **height** para declarar altura.



Na verdade, tanto largura como altura podem ser declaradas com uso das propriedades **width** e **height** diretamente nos boxes (colunas); contudo, isso só é possível em layouts nos quais não existam espaçamentos, pois, nesses casos, ao contrário dos boxes, não há um seletor para espaçamentos.

Para inserir um espaçamento vertical entre as linhas que descrevem o layout, basta inserirmos uma string de pontos (.) entre as linhas a separar e, depois dela, declarar a altura do espaçamento, precedida de sinal de barra normal (/).

Observe o exemplo a seguir, que mostra o layout fixo centrado, no qual inserimos espaçamento de 5px entre o topo 1 e topo 2, e de 10px entre as três colunas e o rodapé: #tudo {

```
display: ".aaaaa."
"....." / 5px ".bbbbbb."
".cccccc."
```

```

".d.e.f."
"....." / 10px ".ggggg."
400px 5px 200px 5px 150px ;

```

```

}
```

Por padrão, a altura dos boxes (colunas) do layout expande-se automaticamente para acomodar o conteúdo nelas inserido. Mas podemos declarar explicitamente a altura das colunas (linhas) do layout usando a mesma sintaxe da declaração de altura para espaçamentos.

Observe o exemplo a seguir, no qual fixamos as alturas dos dois topos e do rodapé: #tudo {

```

display: ".aaaaa." / 90px "....." / 5px
".bbbbbb." / 40px ".cccccc."
".d.e.f."
"....." / 10px
".ggggg." / 25px 400px 5px 200px 5px 150px ;

```

```

}
```

A expansão de uma coluna, quer seja automática ou por declaração explícita de sua altura, faz com que as demais colunas vizinhas também se expandam sem perder a estilização da cor do fundo. Isso significa que, ao contrário do que acontece com as CSS2, as funcionalidades desse módulo dispensam as técnicas das colunas falsas (faux-columns – ver <http://kwz.me/p1>), que exigem a declaração de uma imagem de fundo para preservar a cor de colunas expandidas por força de inserção de conteúdos em colunas vizinhas.

As especificações preveem ainda três palavras-chave e uma função para definir largura de colunas. Esses valores, **min-content**, **max-content**, **fit-content** e a função **minmax()**, permitem que o autor defina larguras máxima e mínima.

Valor	Descrição
min-content	A largura da coluna se ajusta à largura do seu box com conteúdo mínimo.
max-content	A largura da coluna se ajusta à largura do seu box com conteúdo máximo.
fit-content	A largura da coluna se ajusta às larguras mínima e máxima, de acordo com o tamanho da janela.
min-max(m,n)	A largura da coluna é personalizada para ajuste a uma largura mínima especificada no parâmetro m e máxima especificada no parâmetro n, de acordo com o tamanho da janela.

Para exemplificar o uso dessa funcionalidade, considere o exemplo conforme marcação mostrada a seguir: ► **HTML**

```
<section>
<article>
<h1>Título 1</h1>
<p>Texto da seção 1 - Lorem ipsum ...</p>
</article>

<article>
<h1>Título 2</h1>
<p>Texto da seção 2 - Cum sociis natoque ...</p> </article>

<article>
<h1>Título 3</h1>
<p>Texto da seção 3 - Aliquam ligula nulla...</p> </article>
</section>
```

Conforme pode ser notado, trata-se de um elemento `section` contendo três elementos `article`. Por padrão, os elementos `article` renderizam-se um abaixo do outro. Vamos criar um layout que posicione os dois elementos `article` um após outro e à esquerda, e deslocaremos o terceiro elemento `article` para a direita.

Em CSS2, seria como englobar os dois primeiros elementos `article` em um container, flutuá-lo à esquerda e flutuar o terceiro elemento `article` à direita, mantendo iguais as alturas dos dois elementos flutuados.

A regra CSS que descreve o layout pretendido é mostrada a seguir: ► **HTML**

```
section {
display: ".ac."
".bc."
* minmax(460px, 700px) 300px *; }
```

Para o layout pretendido, podemos ler a regra CSS mostrada da seguinte maneira: A letra *a* representa o primeiro elemento `article`, a letra *b*, o segundo, e a letra *c*, o terceiro.

O box *a* posiciona-se acima do box *b*, e ao lado de ambos posiciona-se o box *c*.

Na declaração que define larguras, concluímos que se trata de um layout centrado na tela, pois notamos os sinais de asteriscos (*) em ambos os lados das strings e da definição das larguras.

A largura da coluna *c* é fixa e igual a **300px**.

Para a largura da coluna que contém *a* e *b*, usamos a função `minmax()`, que define valor mínimo de **460px** e máximo de **700px** para aquela coluna. Isso significa que se

trata de uma coluna elástica, adaptando sua largura entre aqueles dois valores, de acordo com o tamanho da janela.

Observe que o valor total da largura do layout vai de um mínimo igual a $460\text{px} + 300\text{px} = 760\text{px}$ até um valor máximo de $700\text{px} + 300\text{px} = 1000\text{px}$.

É o caso clássico de um layout elástico para o qual, em resoluções abaixo de 800px , há uma barra de rolagem horizontal, e em resoluções iguais ou acima de 1024px , o layout é fixo, com 1000px de largura.

Na figura 16.3, mostramos o layout proposto nesse exemplo.

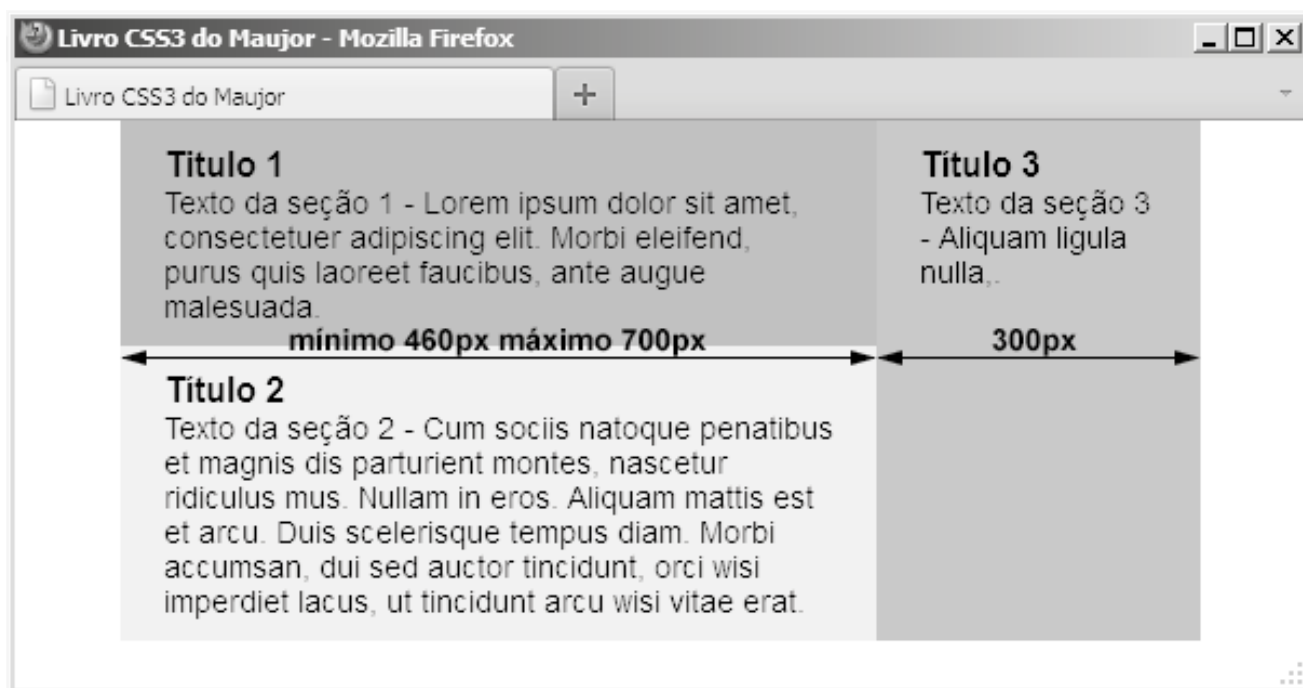


Figura 16.3 – Função *minmax()*.

16.3.2 position

Essa propriedade destina-se a definir a posição de cada box no layout, tomando como referência a letra que o autor atribuiu ao box na regra CSS descritiva do layout.

Para o layout da figura 16.3, mostrada anteriormente, a regra CSS que posiciona os boxes com uso dessa propriedade é mostrada a seguir: ► **CSS**

```
section article:first-child { position: a; }
section article:nth-child(2) { position: b; }
section article:last-child { position: c; }
```

Para o layout da figura 16.2, a regra CSS que posiciona os boxes com uso dessa propriedade é mostrada a seguir: ► **CSS**

```
#topo h1 { position: a; }
#topo h2 { position: b; }
```



```
#nav-principal { position: c; }
#principal { position: d; }
#aux { position: e; }
#nav-secundaria { position: f; }
#rodape { position: g; }
```

Essa propriedade admite ainda o valor **same**, que equivale ao posicionamento idêntico ao do último box posicionado com uso de uma letra. Observe o exemplo a seguir, que esclarece o uso desse valor: ► **HTML**

```
<dl>
<dt>Canvas</dt>
<dd>O elemento canvas é usado para renderização de gráficos, jogos e
imagens com uso de scripts.</dd> <dd>A área de canvas pode ser definida
tanto por atributos como por regra CSS.</dd> <dt>Audio</dt>
<dd>O elemento audio destina-se a incorporar um som ou stream de áudio
em uma página web</dd> <dt>Vídeo</dt>
<dd>Este elemento destina-se a incorporar um vídeo ou um filme em uma
página web</dd> </dl>
```

► CSS

```
dl {
display: ".a.b.c."
250px 5px 250px 5px 250px ;

}

dt:nth-of-type(1) { position: a; }
dt:nth-of-type(2) { position: b; }
dt:nth-of-type(3) { position: c; }
dd { position: same; }
```

A figura 16.4 mostra a renderização do layout proposto para a lista de definição, com uso do valor **same** para a propriedade **position**.

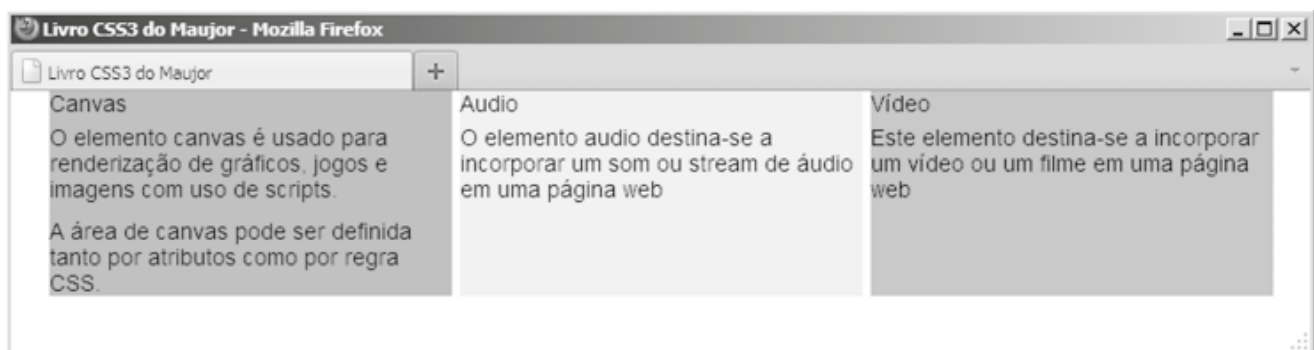


Figura 16.4 – Valor same para propriedade position.

Note que os elementos **dd** foram posicionados nos mesmos boxes em que tinham sido posicionados os elementos **dt** anteriores. Para Canvas, existem duas definições (elementos **dd**), e para Áudio e Vídeo, uma só. Se para todos os elementos colocados

na marcação após o primeiro **dt** e antes do segundo **dt** definirmos **position: same;**, eles serão posicionados no mesmo box que o primeiro **dt**, e esse comportamento é válido para os demais **dt** do exemplo.

16.3.3 ::slot()

Esse pseudoelemento casa com os boxes criados pela regra CSS que define o layout, ou seja, cada box designado por uma letra na regra CSS declarativa do layout pode ser casado por esse pseudoelemento. Observe, a seguir, a sintaxe para uma regra CSS destinada a definir a cor de fundo do box designado pela letra na regra CSS que descreve o layout cujo container é o **div#tudo**.

```
#tudo::slot(a) { background: red; }
```

As únicas propriedades CSS estilizáveis com uso desse pseudoelemento são: **background-*** (todas as propriedades para fundo), **vertical-align** e **overflow**. O W3C estuda a inclusão, nessa lista, das propriedades **block-flow**, **direction** e **box-shadow**.

Para a propriedade **vertical-align**, os valores válidos são: **top**, **middle** e **baseline**.

Convém notar que o mecanismo CSS3 de definição do layout de um container com uso das funcionalidades desse módulo implementou alguns conceitos aplicados na construção de layouts com tabelas. Assim, o layout criado se comporta como uma tabela, com suas linhas, colunas e células, guardadas algumas diferenças. Essas diferenças dizem respeito à acomodação de conteúdos nas células, do mecanismo de justaposição de bordas e da dependência da ordem de apresentação à marcação HTML.

No modelo com tabelas, a expansão de células de acordo com os conteúdos é automática; no modelo template layout, isso não é possível. No modelo com tabelas, é possível justapor bordas de células adjacentes; no modelo template layout, não é. No modelo com tabelas, a sequência de linhas e células é fixa e depende da marcação HTML; no modelo template layout, é possível alterar a sequência com regras CSS.

Uma consequência imediata dessa implementação baseada no modelo de tabelas é que a propriedade **vertical-align** para o pseudosseletor **::slot()** funciona com seus três valores possíveis nesse módulo. Observe o exemplo a seguir, que ilustra esse comportamento: ► **HTML**

```
<section>
<article>
<h1>Titulo 1</h1>
<p>Texto da seção 1 - Lorem ipsum dolor sit amet.</p> </article>
```

```
<article>
<h1>Título 2</h1>
<p>Texto da seção 2 - Cum sociis natoque penatibus et magnis.</p>
</article>
```

```
<article>
<h1>Título 3</h1>
<p>Texto da seção 3 - Aliquam ligula nulla Cum sociis natoque penatibus
et magnis.</p> </article>
</section>
```

► CSS

```
section {
display: ".a.b.c." / 200px
200px 5px 200px 5px 200px ;
```

```
}
```

```
section article:first-child { position: a; }
section article:nth-child(2) { position: b; }
section article:last-child { position: c; }
section::slot(a) { background: orange; vertical-align: top; }
section::slot(b) { background: yellow; vertical-align: middle; }
section::slot(c) { background: cyan; vertical-align: baseline; }
```

A figura 16.5 mostra a renderização dos três boxes e o efeito da declaração `vertical-align`, conforme mostrado no código anterior.

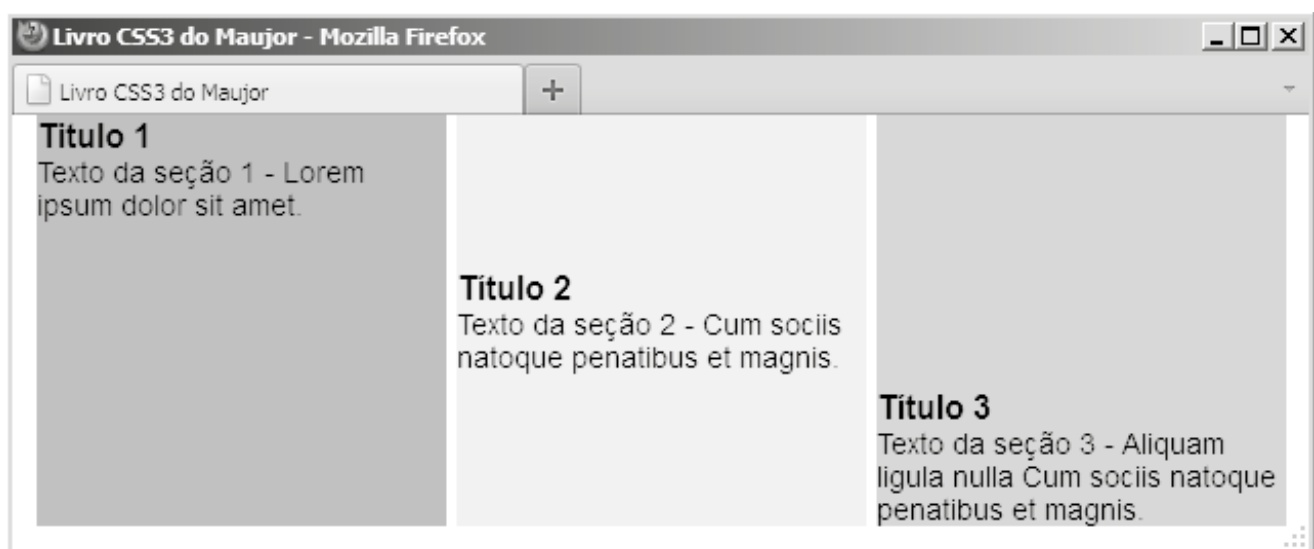


Figura 16.5 – Propriedade `vertical-align`.

16.4 Exemplos

As funcionalidades desse módulo não se aplicam somente à construção do layout

da página como um todo. Vão muito além disso, pois possibilitam ao autor construir o layout dos elementos-filhos de qualquer container da página e até mesmo criar layouts aninhados. A título de ilustração, mostraremos alguns exemplos práticos simples, mas que ilustram o uso das ferramentas criadas por esse módulo.

16.4.1 Exemplo 1 – Formulários

Criar layout de formulários, principalmente aqueles mais complexos, com CSS2, exige um trabalho extra do desenvolvedor, e muitas vezes inserção de marcação adicional, principalmente se não pudermos contar com os poderosos seletores CSS3. Mesmo com eles, a tarefa de alinhar controles e rótulos de formulários não é trivial. Nesse exemplo, mostraremos um formulário que, embora simples, bem ilustra a técnica de layout de um formulário com uso de template layout.

Na figura 16.6, mostramos nosso formulário de exemplo e convidamos o leitor a colocar mãos à obra, interromper a leitura (ou pular o exemplo) para codificar e estilizar, agora ou mais tarde, o formulário mostrado, usando as funcionalidades estudadas neste capítulo, de modo a obter uma renderização semelhante. O convite é um desafio ao seu aprendizado.

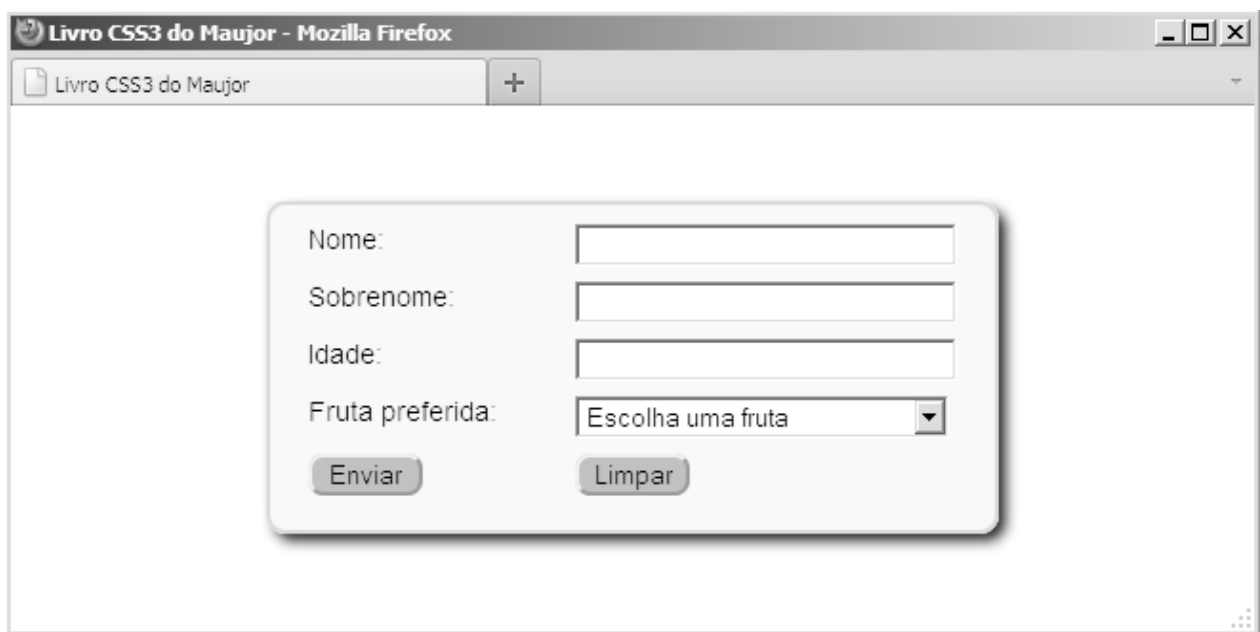


Figura 16.6 – Layout de formulário.

O uso de seletores CSS3 não suportados pelo navegador IE8 e anteriores quebra o layout nesses navegadores. Use seletores ID no lugar dos seletores pseudoclasses `:nth-of-type()` que usamos.

► HTML

```
form action="#" method="get">
<fieldset>
<label for="nome">Nome:</label>
```

```

<input type="text" name="nome" id="nome" />
<label for=sbnome>Sobrenome:</label>
<input type="text" name="sbnome" id="sbnome" />
<label for="idade">Idade:</label>
<input type="text" name="idade" id="idade" />
<label for="fruta">Fruta preferida:</label>
<select name="fruta" id="fruta">
<option value=" ">Escolha uma fruta</option>
<option value="abacate">Abacate</option>
<option value="abacaxi">Abacaxi</option>
<option value="framboesa">Framboesa</option>
<option value="goiaba">Goiaba</option>
<option value="outra">Outra</option>
</select>
<input type=submit value="Enviar" id="enviar" />
<input type=reset value="Limpar" id="limpar" />
</fieldset>
</form>

```

► CSS

```

body {
font: 14px sans-serif;
margin: 0;

```

```

}

```

```

fieldset {
width: 340px;
border: 2px solid #ccc;
border-radius: 10px;
box-shadow: 4px 4px 6px black;
padding:10px 20px;
background: aliceblue;
margin: 50px auto;
display: ".ab." / 30px
".cd." / 30px
".ef." / 30px
".gh." / 30px
".ij." / 30px
140px 200px ;

```

```

}

```

```

label:nth-of-type(1) { position: a; }
input:nth-of-type(1) { position: b; }
label:nth-of-type(2) { position: c; }
input:nth-of-type(2) { position: d; }

```

```

label:nth-of-type(3) { position: e; }
input:nth-of-type(3) { position: f; }
label:nth-of-type(4) { position: g; }
form select { position: h; }
input:nth-of-type(4) { position: i; width: 60px; }
input:nth-of-type(5) { position: j; width: 60px; }
input:nth-of-type(4), input:nth-of-type(5) {
background: orange;
border-radius: 8px;

}

```

16.4.2 Exemplo 2 – Templates aninhados

Podemos aninhar templates em quantos níveis quisermos. Para mostrar a sintaxe geral de aninhamento de templates, considere um layout com três colunas descrito pelas letras *abc* (*a.b.c*), no qual a coluna *a* contém três elementos dispostos um à esquerda e os outros dois à direita. O alinhamento dos três elementos dentro da coluna *a* é um template nela aninhado. Observe a regra CSS que declara os templates: ► **CSS**

```

#tudo {
display: "a.b.c" /* descreve layout de 3 colunas */
400px 5px 200px 5px 150px ;

}

#principal {
position: a;
display: "d.e" /* descreve layout aninhado na coluna a */
"d.f"
39% 1% 60%;

}

```

Na figura 16.7, mostramos o aninhamento do layout, conforme descrito.

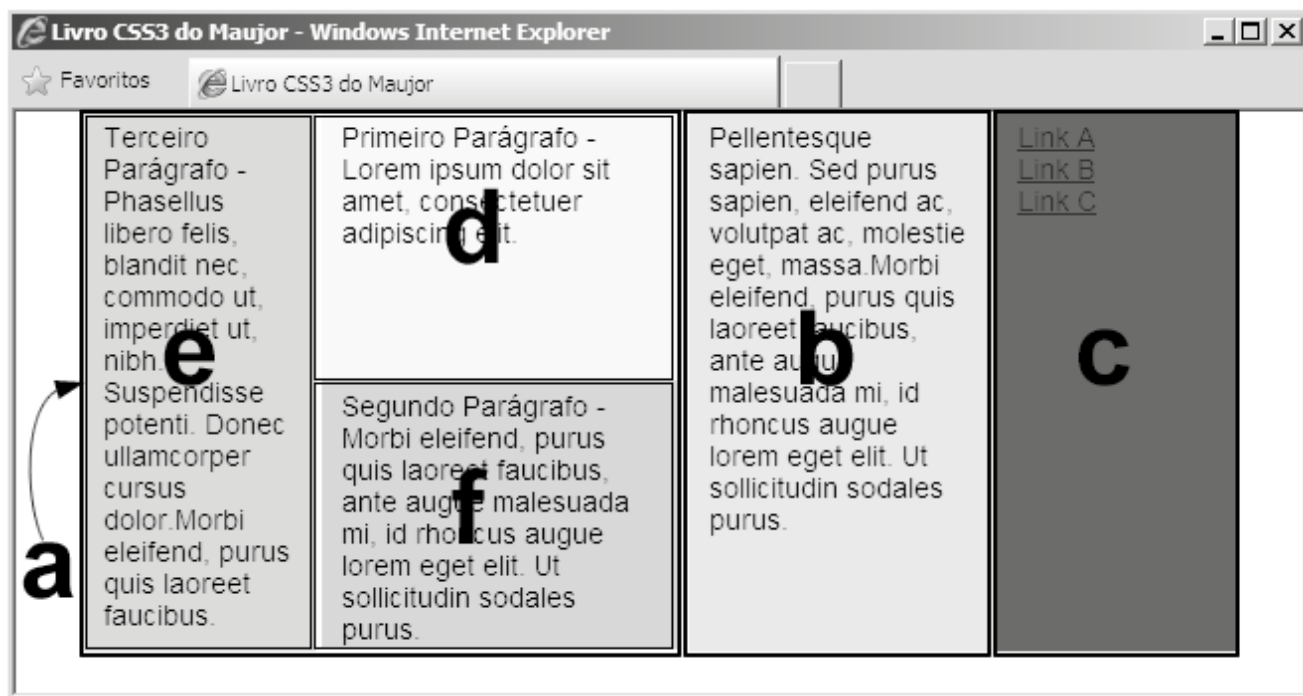


Figura 16.7 – Layout aninhado.

Os códigos para obtenção do efeito de aninhamento são conforme mostrado a seguir: ► **HTML**

```
<div id="tudo">
<section id="principal">
<p id="um">Primeiro Parágrafo - Lorem ...</p>
<p id="dois">Segundo Parágrafo - Morbi eleifend...</p> <p
id="tres">Terceiro Parágrafo - Phasellus libero ...</p> </section>
```

```
<aside id="aux">
<p>Pellentesque sapien...</p>
</aside>
```

```
<nav id="nav">
<ul>
<li><a href="#">Link A</a></li>
<li><a href="#">Link B</a></li>
<li><a href="#">Link C</a></li>
</ul>
</nav>
</div>
```

► **CSS**

```
body {
font: 14px sans-serif;
margin: 0;
```

```
}
```

```

section, aside, nav {display: block; }
ul, p { margin:0; padding:0; }
#nav-principal li { display:inline;margin-right:10px; }
ul li { list-style: none; }
#tudo {
    display: ".a.b.c."
    * 300px 5px 150px 5px 120px *; }
#principal {
    position: a; display: "e.d"
    "e.f"
    39% 1% 60%; }

#aux { position: b; padding: 5px 10px; }
#nav { position: c; padding: 5px 10px; }
#tudo::slot(b) { background: khaki; }
#tudo::slot(c) { background: magenta; }
#um { position: d; padding: 5px 10px; }
#tres { position: e; padding: 5px 10px; }
#dois { position: f; padding: 5px 10px; }
#principal::slot(d) { background: aliceblue; }
#principal::slot(e) { background: pink; }
#principal::slot(f) { background: cyan; }

```

É importante ressaltar que boxes posicionados com template layout se comportam como se fossem posicionados segundo as regras das CSS2 para efeito da ordem de empilhamento, ou seja, pode-se fazer uso da propriedade `z-index` para alterar aquela ordem.

16.4.3 Exemplo 3 – flutuando boxes

As propriedades `float` e `clear` funcionam normalmente, segundo os mecanismos das CSS2.

Na figura 16.8, mostramos o efeito de se flutuar um box, contido em um box criado com template layout, à direita, com e sem uso da propriedade `clear`.



Figura 16.8 – Float.

Os códigos para flutuar o box são mostrados a seguir:

► **HTML**

```
body {  
  <div id="a">  
    <div id="flt">FLOAT</div>  
    <p>Suspendisse potenti...</p>  
    <p><p>Lorem ipsum dolor sit amet...</p>  
  </div>  
</body>
```

► **CSS**

```
body {  
  font: 14px sans-serif;  
  display: "a";  
  
  }  
  
body::slot(a) { background: yellow; vertical-align: middle; }  
#a { position: a; width: 280px; padding: 10px; }  
#flt {  
  width: 80px;  
  height: 100px;  
  background: cyan;  
  float: right;  
  
  }  
  
#a p + p { clear: right; }
```

Sem dúvida, as funcionalidades desse módulo oferecem mecanismos para criação de layouts complexos que facilitam a tarefa do desenvolvedor, colocando à sua disposição ferramentas para criação que, com uso das CSS2, só seriam possíveis com adição de marcação extra e declarações CSS intrincadas, que não raro acrescentam trabalho extra para resolução de bugs decorrentes de inconsistências dos navegadores.

CAPÍTULO 17

Box Layout flexível

As especificações CSS para o Módulo Box Layout Flexível estão no estágio de Candidata a Recomendação do W3C.

Neste capítulo, estudaremos as funcionalidades desse módulo. Em outubro de 2011, nenhum navegador suportava as funcionalidades desse módulo. Contudo, os navegadores Firefox 2+, Safari 3.1+, Chrome 4+ e IE 10+ proveem suporte com uso dos prefixos proprietários `-moz-`, `-webkit-` e `-ms-`.

Os prefixos proprietários para as funcionalidades desse módulo foram criados antes de 2010, com base em uma versão anterior à atual do Rascunho de Trabalho para as especificações.

Em 22 de março de 2011, 22 de março de 2012 e 12 de junho de 2012, foram lançadas novas versões daquele Rascunho de Trabalho, alterando a sintaxe de propriedades e valores, bem como acrescentando e criando novas funcionalidades.

Em 18 de setembro de 2012, a especificação atingiu o status de Candidata a Recomendação do W3C.

Como você pode notar, esse Módulo das CSS3 vem sofrendo constantes alterações e acréscimos de funcionalidades. A grande maioria dos tutoriais que se encontra na Internet, pelo menos até o segundo semestre de 2013, a respeito desse assunto, baseia-se em sintaxe que já foi descartada pelo W3C e uso de prefixos proprietários.

À época em que esta segunda edição do livro foi escrita, por ter a especificação já atingido a fase de Candidata a Recomendação, acreditamos e torcemos para que não haja mais mudanças radicais como foram as anteriores.

Os exemplos serão desenvolvidos com uso da sintaxe atual e conforme o suporte oferecido pelos navegadores, ou seja, uso do prefixo proprietário `-webkit-` para Safari 6 e Opera 16. Os navegadores Chrome 29 e Firefox 23 (suporte parcial) suportam as funcionalidades desse módulo nativamente, sem uso de prefixos.



Aconselho a visualizar os exemplos constantes deste capítulo e disponíveis no site do livro no navegador Chrome 29+ ou Opera 16+, pois os demais navegadores atuais apresentam bugs em alguns exemplos.

17.1 Introdução

O W3C descreve a finalidade do Módulo Box Layout Flexível conforme tradução adaptada a seguir: Essa especificação descreve um Box Model CSS otimizado para desenho de interfaces. Nesse modelo de layout de boxes flexíveis os boxes filhos de

um container flexível podem ser flexíveis em ambas as dimensões, dispostos na vertical ou na horizontal e podem preencher eventuais espaços vagos no container, seja por expansão ou contração. Para criar um layout, é possível manipular facilmente o alinhamento dos boxes bem como aninhar boxes horizontais em boxes verticais e vice-versa.

Para definir o modelo layout de boxes flexíveis, adotaremos o termo usado pelas especificações: *flex layout*.

O modelo CSS2 para construção de layout define algoritmos de cálculo das dimensões e do posicionamento dos boxes com base em seus boxes vizinhos e boxes ancestrais. Assim, temos boxes nível de bloco, inline e tabulares, que não oferecem muitas opções de controle. Esse módulo das CSS3 cria condições de desenvolvimento de aplicações e páginas web bem mais complexas que aquelas criadas com CSS2. Flex layout é um modelo parecido com block level layout. Ele não possui as funcionalidades de flutuar e criar colunas, mas, em contrapartida, é muito mais poderoso para alinhamento de conteúdos, característica básica de aplicações web complexas. Os conteúdos de um flexbox podem ser dispostos tanto horizontal como verticalmente, ter sua ordem trocada e expandir e contrair para se ajustar ao container. Além disso, é possível declarar flexboxes inline.

Os elementos descendentes de um flex container são denominados *flexbox items*.

As principais características do modelo flex layout com relação ao layout dos boxes são:

- Podem ser dispostos em qualquer direção (*flow direction*) – para a esquerda, para a direita, para cima ou para baixo.

- Podem ter sua ordem de posicionamento invertida ou rearranjada sem alteração da ordem em que aparecem na marcação HTML.
- Podem ser dispostos em linha ao longo de um eixo principal (*main axis*) ou em múltiplas linhas ao longo de um eixo secundário (*cross axis*).
- Podem ser flexíveis em ambas as direções, ou seja, adaptáveis aos espaços disponíveis.
- Podem ser alinhados em relação ao seu container ou em relação a outros boxes.
- Podem ser dinamicamente unidos ou separados ao longo do eixo principal sem alterar o eixo secundário do container.

Ao contrário do modelo layout nível de bloco e inline previstos nas CSS2, nos quais os cálculos são baseados em fluxos cujas direções são determinadas segundo blocos ou inline, o modelo flex layout baseia seus cálculos em flow-directions que determinam a direção (esquerda, direita, para cima e para baixo), o eixo (horizontal – *main* – e vertical – *cross* –) e as dimensões (*width* e *height*).

A figura 17.1 extraída das especificações do W3C esclarecem a terminologia para essas direções e dimensões.

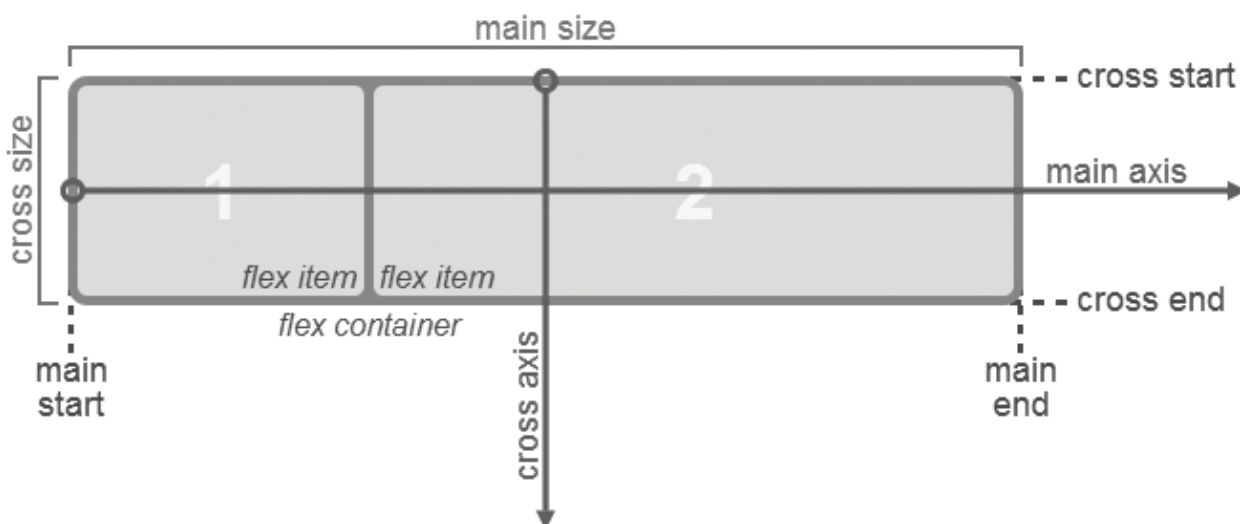


Figura 17.1 – Terminologia para direções e dimensões no flex layout.

Os termos constantes da terminologia mostrados na figura 17.1 têm o seguinte significado.

- *main-axis* – eixo primário ou principal ao longo do qual os *flex-items* são dispostos horizontalmente. Esse eixo determina a direção da *main dimension* (dimensão horizontal).
- *main-start* e *main-end* – os *flex-items* são dispostos horizontalmente a partir do ponto *main-start* em direção ao ponto *main-end*.
- *cross-axis* – eixo secundário ao longo do qual os *flex-items* são dispostos verticalmente. Esse eixo determina a direção da *cross dimension* (dimensão vertical).
- *cross-start* e *cross-end* – os *flex-items* são dispostos verticalmente a partir do ponto *cross-start* em direção ao ponto *cross-end*.
- *main-size* e *cross-size* – os *flex-items* têm suas dimensões segundo essas direções.

17.2 Propriedades

Nesse módulo são previstas as propriedades, conforme listadas a seguir: **display**, **flex-direction**, **flex-wrap**, **flex-flow**, **order**, **flex**, **justify-content**, **align-items**, **align-self** e **align-content**.

A seguir, veremos a finalidade e os valores possíveis para cada uma das propriedades.

17.2.1 display

Essa propriedade destina-se a definir o contexto para layout flexbox. Os valores possíveis são: **flex** e **inline-flex**. Sabemos que o valor inicial da propriedade **display** de um elemento depende do elemento, por exemplo: elementos **div** têm sua propriedade **display** com valor inicial igual a **block**, enquanto elementos **span** a têm com valor inicial igual a **inline**.

Os valores dessa propriedade, quando aplicados a um elemento, o define como um container flexbox, ou seja, todos os elementos descendentes do container se comportam seguindo o modelo flex layout. Se pretendemos que os elementos em uma página web sigam esse modelo, declaramos um desses valores para a propriedade **display** do elemento **body**. Observe a seguir os exemplos que esclarecem a sintaxe dessa propriedade: `body { display: flex; }`

```
div { display: flex; }
footer { display: inline-flex; }
div { display: inline-flex; }
span { display: flex; }
section { display: inline-flex; }
```

Ao declararmos os valores **flex** ou **inline-flex** para a propriedade **display** de um container, estaremos forçando o container a usar um novo algoritmo de layout, o algoritmo para flexboxes, e, em consequência, algumas regras que regem o layout de elementos nível de bloco, como no modelo CSS2, não fazem sentido, por exemplo: as funcionalidades de **float** e **clear** não se aplicam, a propriedade **vertical-align** não funciona e nenhuma das propriedades **column-*** previstas no Módulo Multi Colunas (Capítulo 9) se aplicam.

A diferença entre os valores **flex** e **inline-flex**, como o próprio nome sugere, é que o primeiro cria um container para flexboxes nível de bloco e o segundo um container para flexboxes inline.

Considere o exemplo mostrado a seguir:

► CSS

```
<style>
html, body {
font: 100%/1.4 sans-serif;
margin: 0;
padding: 0;

}

* { box-sizing: border-box; }
.tudo { max-width: 75rem; margin: 0 auto; }
h1, h2, p { margin: .8rem 0 .6rem 0 }
h1, h2 { font-size: 1.4rem; }
header, section, footer { padding: 1px 0.8rem; }
```

```

header { background: #555; color: #eee; }
section {
  display: -webkit-flex; /* Para Safari e Opera */
  display: flex; /* Para suporte nativo */
  background: #eee;

  }

```

```

article {
border: 1px solid black;
margin: 0.4rem;
padding: 0.3rem 0.6rem;

}

```

```

footer { background: #444; color: #eee; }
</style>

```

► HTML

```

<div class="tudo">
<header>
<h1>Topo</h1>
</header>
<section>
<article>
<h2>Título 1</h2>
<p>Texto da seção 1 - Lorem ipsum dolor sit amet...</p>
</article>
<article>
<h2>Título 2</h2>
<p>Texto da seção 2 - Curabitur hendrerit erat ut augue...</p> </article>
<article>
<h2>Título 3</h2>
<p>Texto da seção 3 - Pellentesque sapien...</p>
</article>
</section>
<footer>
<h2>Rodapé</h2>
</footer>
</div> <!-- /.tudo -->

```

Trata-se de um container geral (elemento `div.tudo`) no qual foram inseridos três elementos-filhos (elementos `header`, `section` e `footer`). Declararamos o valor `flex` para a propriedade `display` do elemento `section`. Em consequência aos seus três elementos-filhos `article` poderão ser aplicadas as funcionalidades do flex layout.

Na figura 17.2 mostramos a renderização dos três elementos `article`, que

demonstra como são dispostos por padrão elementos que seguem o modelo flex layout.

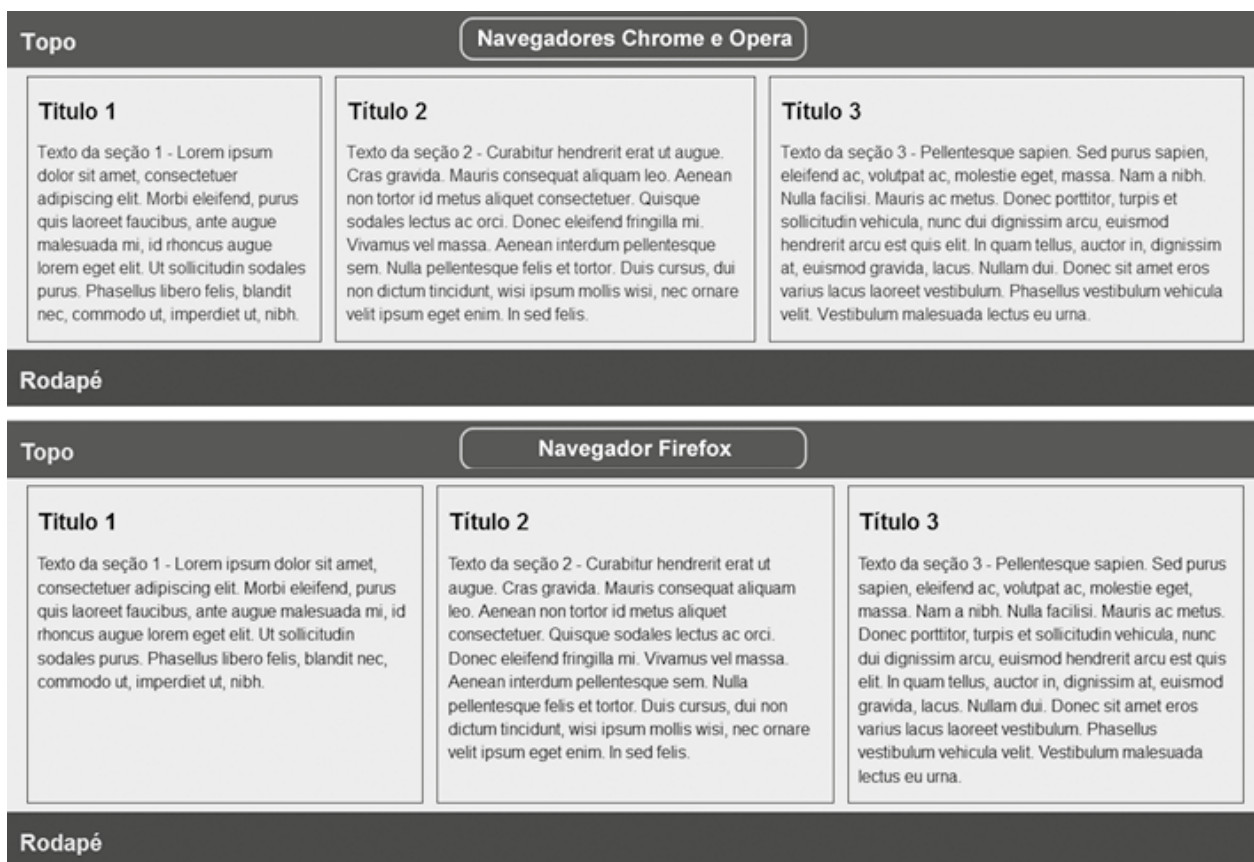


Figura 17.2 – Propriedade `display` cria contexto para flexboxes.

A figura 17.2 mostra que, ao se declarar o contexto de layout flexbox para um container, seus elementos-filhos expandem seus boxes até uma largura tal que todos os boxes ocupem o container com alturas iguais.

Observe que, quanto à largura dos boxes, existe uma inconsistência de renderização entre navegadores. No Firefox a largura dos três boxes é igual e as alturas são iguais à altura do box com mais conteúdo. Nos navegadores Opera e Chrome a altura dos três boxes é igual e tal que todo o conteúdo que se estenda por essa altura e as larguras são diferentes.

Notar que não houve necessidade de declarar explicitamente as propriedades `width` e `height`.

17.2.2 flex-direction

Essa propriedade destina-se a definir a direção em que os elementos-filhos serão dispostos no elemento container. Os valores possíveis são: `row`, `column`, `row-reverse` e `column-reverse`. O valor inicial é `row`. Aplica-se a essa propriedade a flexboxes e não é herdada.

O efeito de cada um dos valores é descrito a seguir.

Efeito	Descrição
row	Os boxes são dispostos em linha, segundo o <i>main axis</i> (na horizontal).
column	Os boxes são dispostos como boxes nível de bloco, segundo o <i>cross axis</i> (na vertical).
row-reverse	Os boxes em linha são dispostos na ordem inversa do código.
column-reverse	Os boxes nível de bloco são dispostos na ordem inversa do código.

Os exemplos a seguir esclarecem o efeito desses valores. Em todos os exemplos, para essa e para as propriedades que se seguem, usaremos a marcação HTML e regras de estilos comuns, mostradas a seguir: ► **CSS**

```
<style>
html, body {
font: 100%/1.4 sans-serif;
margin: 0;
padding: 0;

}

* { box-sizing: border-box; }
.tudo { max-width: 75rem; margin: 0 auto; }
h1, h2, p { margin: .8rem 0 .6rem 0 }
h1, h2 { font-size: 1.4rem; }
header, section, footer { padding: 1px 0.8rem; }
header { background: #555; color: #eee; }
section {
display: -webkit-flex; /* Para Safari e Opera */
display: flex; /* Para suporte nativo */
background: #eee;

}

article {
border: 1px solid black;
margin: 0.4rem;
padding: 0.3rem 0.6rem;

}

footer { background: #444; color: #eee; }
</style>
```

► **CSS**


```

<div class="tudo">
  <header>
    <h1>Topo</h1>
  </header>
  <section>
    <article>
      <h2>Titulo 1</h2>
      <p>Texto da seção 1 - Lorem ipsum...</p>
    </article>
    <article>
      <h2>Titulo 2</h2>
      <p>Texto da seção 2 - Curabitur...</p>
    </article>
    <article>
      <h2>Titulo 3</h2>
      <p>Texto da seção 3 - Pellentesque sapien...</p>
    </article>
  </section>
  <footer>
    <h2>Rodapé</h2>
  </footer>
</div> <!-- /.tudo -->

```

Trata-se de um container geral (elemento `div.tudo`) no qual foram inseridos três elementos-filhos (elementos `header`, `section`, e `footer`). Declararamos o valor `flex` para a propriedade `display` do elemento `section`. Em consequência, aos seus três elementos-filhos `article` poderão ser aplicadas as funcionalidades do flex layout.

flex-direction: row

Define a direção horizontal para disposição dos boxes, ou seja, segundo o *main axis* e da esquerda para a direita. Essa é a direção padrão para disposição de flexboxes conforme mostrado na figura 17.2.

flex-direction: column

Define a direção vertical para disposição dos boxes, ou seja, segundo o *cross axis* e de cima para baixo. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```

section {
  display: -webkit-flex; /* Para Safari e Opera */
  display: flex; /* Para suporte nativo */
  flex-direction: column;
  -webkit-flex-direction: column;

```

background: #eee;

}

Na figura 173 mostramos o efeito da declaração desta propriedade com o valor `column`.

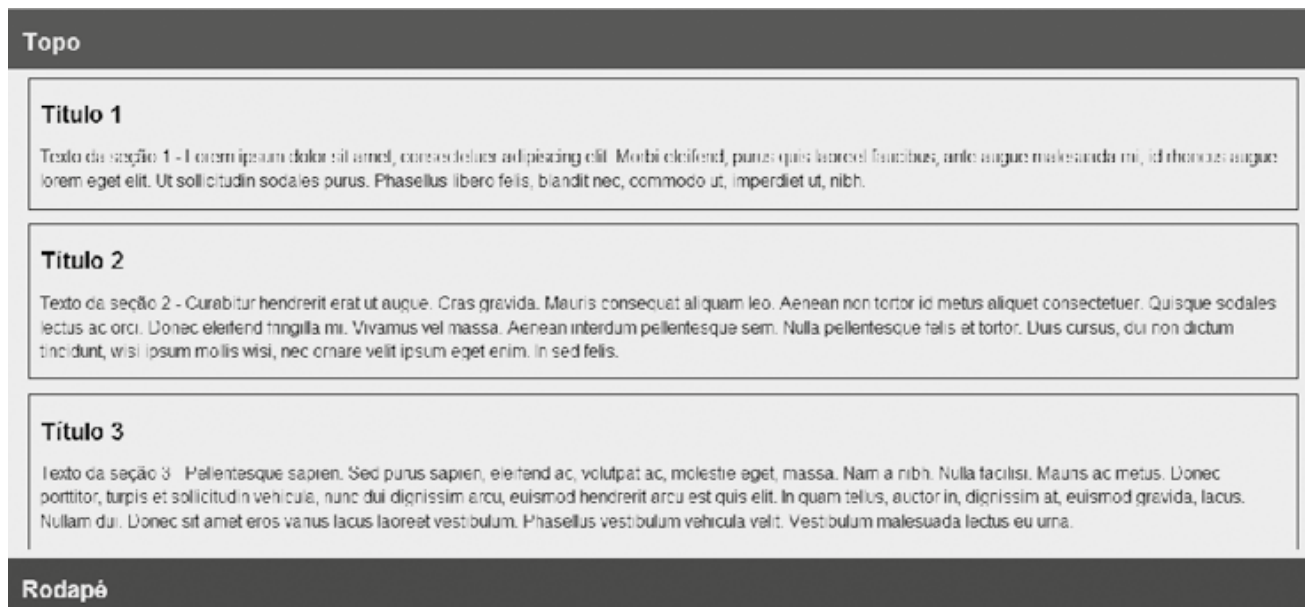


Figura 17.3 – Propriedade `flex-direction` com valor `column`.

flex-direction: row-reverse

Define a direção horizontal e o sentido inverso para disposição dos boxes, ou seja, segundo o *main axis* e da direita para a esquerda. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```
section {  
display: -webkit-flex; /* Para Safari e Opera */  
display: flex; /* Para suporte nativo */  
flex-direction: row-reverse;  
-webkit-flex-direction: row-reverse;  
background: #eee;  
}
```

Na figura 17.4 mostramos o efeito da declaração desta propriedade com o valor `row-reverse`.

Topo		
Título 3 Texto da seção 3 - Pellentesque sapien. Sed purus sapien, eleifend ac, volutpat ac, molestie eget, massa. Nam a nibh. Nulla facilisi. Mauris ac metus. Donec porttitor, turpis et sollicitudin vehicula, nunc dui dignissim arcu, euismod hendrerit arcu est quis elit. In quam tellus, auctor in, dignissim at, euismod gravida, lacus. Nullam dui. Donec sit amet eros varius lacus laoreet vestibulum. Phasellus vestibulum vehicula velit. Vestibulum malesuada lectus eu urna.	Título 2 Texto da seção 2 - Curabitur hendrerit erat ut augue. Cras gravida. Mauris consequat aliquam leo. Aenean non tortor id metus aliquet consectetur. Quisque sodales lectus ac orci. Donec eleifend fringilla mi. Vivamus vel massa. Aenean interdum pellentesque sem. Nulla pellentesque felis et tortor. Duis cursus, dui non dictum tincidunt, wisi ipsum mollis wisi, nec ornare velit ipsum eget enim. In sed felis.	Título 1 Texto da seção 1 - Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi eleifend, purus quis laoreet faucibus, ante augue malesuada mi, id rhoncus augue lorem eget elit. Ut sollicitudin sodales purus. Phasellus libero felis, blandit nec, commodo ut, imperdiet ut, nibh.
Rodapé		

Figura 174 – Propriedade *flex-direction* com valor *row-reverse*.

flex-direction: column-reverse

Define a direção vertical e o sentido inverso para disposição dos boxes, ou seja, segundo o *cross axis* e de baixo para cima. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```
section {
display: -webkit-flex; /* Para Safari e Opera */
display: flex; /* Para suporte nativo */
flex-direction: column-reverse;
-webkit-flex-direction: column-reverse;
background: #eee;

}
```

Na figura 17.5, mostramos o efeito da declaração desta propriedade com o valor *column-reverse*.

17.2.3 flex-wrap

Essa propriedade destina-se a definir como será a disposição de linhas no flex container. Os valores possíveis para essa propriedade são: *nowrap*, *wrap* e *wrap-reverse*. O valor inicial é *nowrap*. Essa propriedade aplica-se ao container de flexboxes e não é herdada.

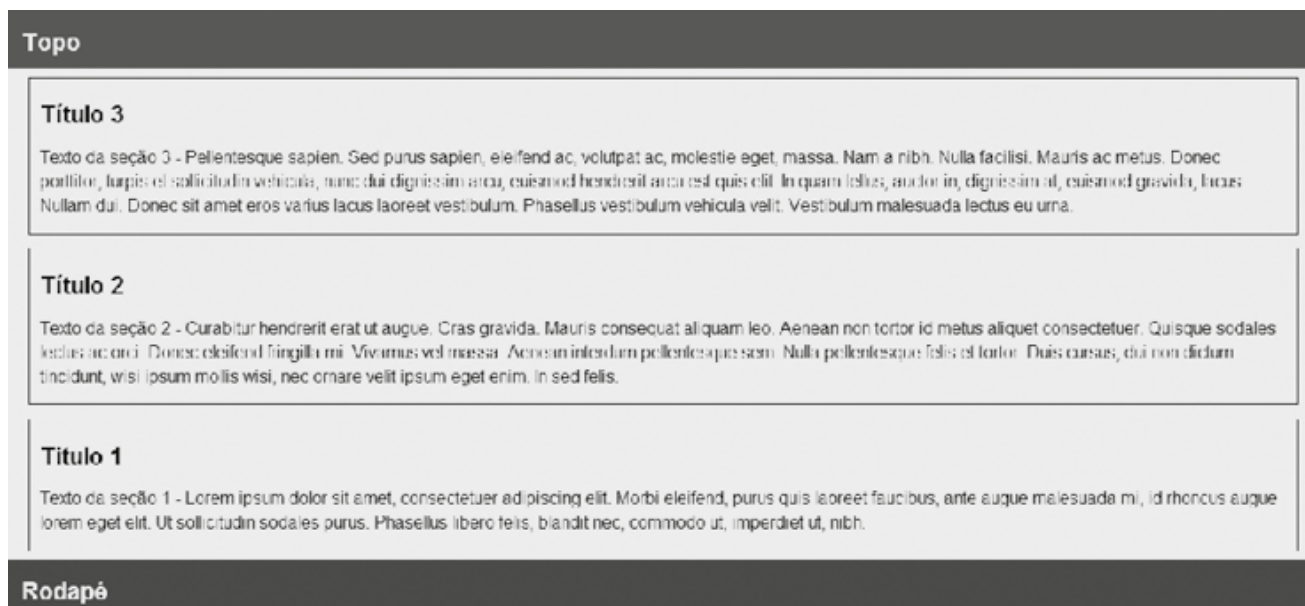


Figura 17.5 – Propriedade *flex-direction* com valor *column-reverse*.

No exemplo a seguir definimos uma largura para os três boxes de forma que, quando somadas, ultrapassem a largura total do container. Nesses casos, por padrão os boxes se ajustam automaticamente à largura do container, mas se declararmos essa propriedade com o valor *wrap* alteramos o comportamento padrão fazendo com que a largura de cada box seja respeitada e eles sejam dispostos em múltiplas linhas, ou melhor, em tantas linhas quanto necessárias. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```
section {
display: -webkit-flex; /* Para Safari e Opera */
display: flex; /* Para suporte nativo */
flex-wrap: wrap;
-webkit-flex-wrap: wrap;
background: #eee;

}

article {
width: 35.9rem;
border: 1px solid black;
margin: 0.4rem;
padding: 0.3rem 0.6rem;

}
```

Na figura 17.6 mostramos o efeito da declaração desta propriedade com o valor *wrap*.

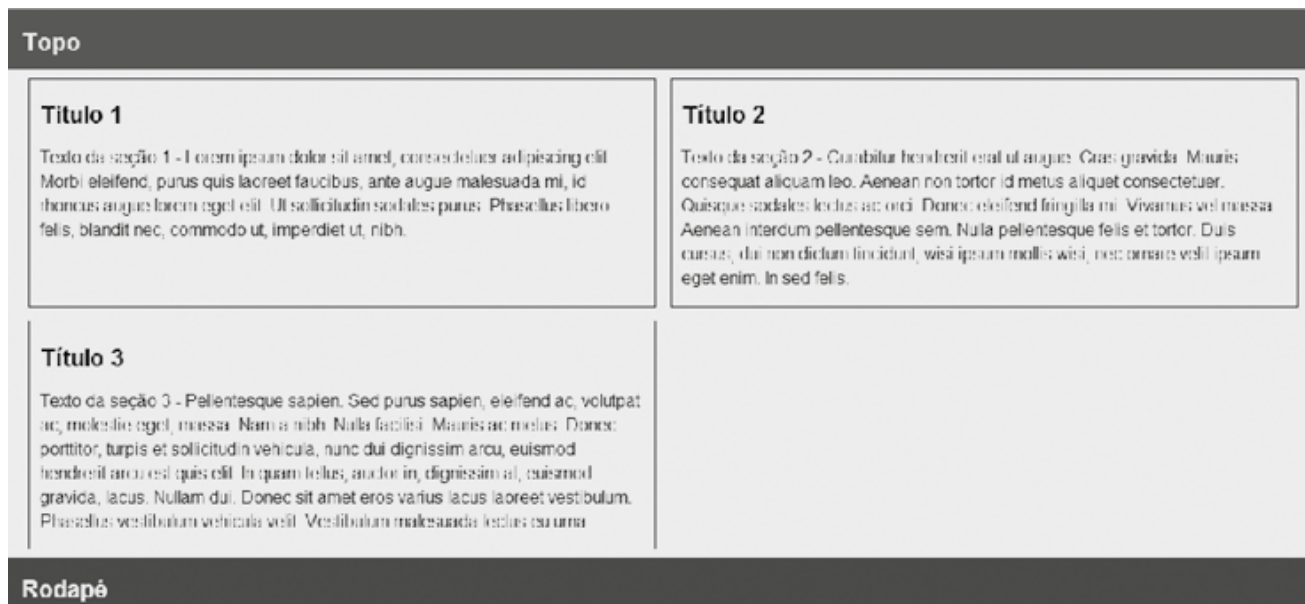


Figura 17.6 – Propriedade *flex-wrap* com valor *wrap*.

O outro valor possível para essa propriedade é **wrap-reverse**, que faz com que a disposição dos boxes seja em múltiplas linhas e com sentido inverso. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```
section {
display: -webkit-flex; /* Para Safari e Opera */
display: flex; /* Para suporte nativo */
flex-wrap: wrap-reverse;
-webkit-flex-wrap: wrap-reverse;
background: #eee;

}

article {
width: 35.9rem;
border: 1px solid black;
margin: 0.4rem;
padding: 0.3rem 0.6rem;

}
```

Na figura 17.7 mostramos o efeito da declaração desta propriedade com o valor **wrap-reverse**.

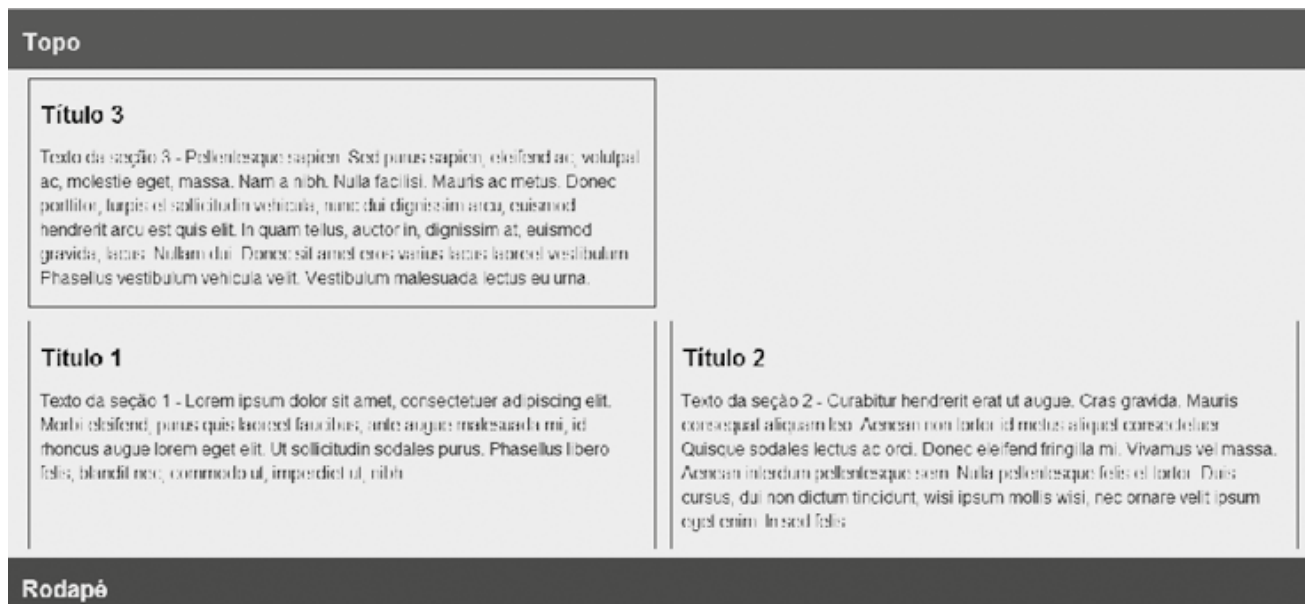


Figura 17.7 – Propriedade flex-wrap com valor wrap-reverse.

17.2.4 flex-flow

Essa propriedade é a forma abreviada de se declarar as propriedades **flex-direction** e **flex-wrap**. O exemplo a seguir esclarece a sintaxe dessa propriedade.

```
section {
display: -webkit-flex; /* Para Safari e Opera */
display: flex; /* Para suporte nativo */
flex-flow: row-reverse wrap-reverse;
-webkit-flex-flow: row-reverse wrap-reverse;
background: #eee;

}
```

Nesse exemplo usamos a forma abreviada **flex-flow** para declarar que a direção dos boxes no container será da direita para a esquerda e em múltiplas linhas.

Na figura 17.8, mostramos o efeito da declaração desta propriedade com os valores **row-reverse** e **wrap-reverse**.

17.2.5 order

Essa propriedade permite que se altere a ordem em que os flex boxes são apresentados quando renderizados. Por padrão a ordem de apresentação segue o modelo geram da HTML, ou seja, a mesma ordem em que eles, boxes, aparecem na marcação da página. Essa propriedade permite alterar a ordem sem alterar a marcação.



Figura 17.8 – Propriedade flex-flow com valor row-reverse e wrap-reverse.

Inicialmente todos os boxes, em qualquer quantidade, recebem o número de ordem 0 (zero). A partir daí o autor pode controlar a ordem de apresentação dos boxes definindo para o valor da propriedade **order** um número inteiro (valores negativos são válidos) para cada um deles. Os boxes serão ordenados segundo a sequência crescente dos números definidos pelo autor. Boxes cuja ordem não for definida serão considerados como de ordem inicial 0 (zero) e para boxes de mesma ordem prevalece a ordem em que aparecem na marcação.

O exemplo a seguir, que usa a mesma marcação HTML dos exemplos anteriores, esclarece a sintaxe e o efeito dessa propriedade, colocando os boxes **article** em ordem diferente da ordem padrão.

► CSS

```
article:first-child { order: 2; }
article + article { order: 3; }
article:last-child { order: 1; }
```

Na figura 17.9 mostramos o efeito da declaração da propriedade **order** para alterar a ordem de renderização dos boxes **article**.

17.2.6 flex

Essa propriedade é a forma abreviada de declarar os valores **flex-grow**, **flex-shrink** e **flex-basis** e destina-se a determinar como as larguras dos flex boxes se expandem ou se contraem proporcionalmente para preencher o container.

Topo		
Título 3 Texto da seção 3 - Pellentesque sapien. Sed purus sapien, eleifend ac, volutpat ac, molestie eget, massa. Nam a nibh. Nulla facilisi. Mauris ac metus. Donec porttitor, turpis et sollicitudin vehicula, nunc dui dignissim arcu, euismod hendrerit arcu est quis elit. In quam tellus, auctor in, dignissim at, euismod gravida, lacus. Nullam dui. Donec sit amet eros varius lacus laoreet vestibulum. Phasellus vestibulum vehicula velit. Vestibulum malesuada lectus eu urna.	Título 1 Texto da seção 1 - Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi eleifend, purus quis laoreet faucibus, ante augue malesuada mi, id rhoncus augue lorem eget elit. Ut sollicitudin sodales purus. Phasellus libero felis, blandit nec, commodo ut, imperdiet ut, nibh.	Título 2 Texto da seção 2 - Curabitur hendrerit erat ut augue. Cras gravida. Mauris consequat aliquam leo. Aenean non tortor id metus aliquet consectetur. Quisque sodales lectus ac orci. Donec eleifend fringilla mi. Vivamus vel massa. Aenean interdum pellentesque sem. Nulla pellentesque felis et tortor. Duis cursus, dui non dictum tincidunt, wisi ipsum mollis wisi, nec ornare velit ipsum eget enim. In sed felis.
Rodapé		

Figura 17.9 – Propriedade *order*.

Os valores de **flex-grow** (expansão) e **flex-shrink** (contração) são expressos por um número inteiro positivo e definem o fator de crescimento ou expansão.

O valor de **flex-basis** é um comprimento a ser tomado como base para expansão ou contração.

A sintaxe geral para declarar essa propriedade é mostrada a seguir.

```
flex: none | [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ]
```

O exemplo a seguir, que usa a mesma marcação HTML dos exemplos anteriores, esclarece a sintaxe e o efeito da declaração do valor **flex-grow**. As modificações a serem feitas na folha de estilo para definir essa declaração são mostradas a seguir.

► CSS

```
article:first-child { flex: 3; }
article + article { flex: 2; }
article:last-child { flex: 5; }
```

Na figura 17.10 mostramos o efeito da declaração dos valores para **flex-grow** com a finalidade distribuir as larguras dos boxes proporcionalmente aos números 3, 2 e 5, respectivamente. Notar que essa propriedade permite que o autor declare as larguras dos boxes de forma relativa umas às outras, larguras.

O valor **flex-basis** define uma largura específica para os boxes e, quando combinado com o valor **flex-grow** ou **flex-shrink**, distribui a sobra ou excesso de espaçamento no container de modo proporcional pelos flex boxes.

Topo		
3 Título 1 Texto da seção 1 - Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi eleifend, purus quis laoreet faucibus, ante augue malesuada mi, id rhoncus augue lorem eget elit. Ut sollicitudin sodales purus. Phasellus libero felis, blandit nec, commodo ut, imperdiet ut, nibh.	2 Título 2 Texto da seção 2 - Curabitur hendrerit erat ut augue. Cras gravida. Mauris congue quam aliquam leo. Aenean non tortor id mollis aliquet congue. Quisque sodales lectus ac orci. Donec eleifend fringilla mi. Vivamus vel massa. Aenean interdum pellentesque sem. Nulla pellentesque felis et tortor. Duis cursus, dui non dictum tincidunt, wisi ipsum mollis wisi, nec ornare velit ipsum eget enim. In sed felis.	5 Título 3 Texto da seção 3 - Pellentesque sapien. Sed purus sapien, eleifend ac, volutpat ac, molestie eget, massa. Nam a nibh. Nulla facilisi. Mauris ac metus. Donec porttitor, lacinia et sollicitudin vehicula, nunc dui dignissim arcu, euismod hendrerit arcu est quis elit. In quam tellus, auctor in, dignissim at, euismod gravida, lacinia. Nullam dui. Donec sit amet eros varius lacinia laoreet vestibulum. Phasellus vestibulum vehicula velit. Vestibulum malesuada lectus eu urna.
Rodapé		

Figura 17.10 – Propriedade flex-grow.

O exemplo a seguir usa a mesma marcação HTML dos exemplos anteriores, mas foram definidas larguras fixas para o container e para os boxes, bem como retirados padding e margin laterais. Nele são esclarecidas a sintaxe e o efeito da declaração do valor **flex-basis**. As modificações a serem feitas na folha de estilo para esse exemplo e para definir essa declaração são mostradas a seguir.

► CSS

```
.tudo {
width: 1000px; /* largura total do container */
margin: 0 auto;

}

h1, h2, p { margin: 12px 0 5px 0 } /* margens laterais zeradas */
header, section, footer { padding: 1px 0; } /* paddings laterais zerados */
article {
border: 1px solid black;
padding: 3px 0; /* paddings laterais zerados */

}

article:first-child { flex: 3 200px; }
article + article { flex: 2 300px; }
article:last-child { flex: 1 380px; }
```

Notar que a largura total do container (1000px) é maior do que a soma das larguras dos boxes (200px + 300px + 380px = 880px). A diferença 1000px - 880px = 120px deverá ser dividida em 3+2+1 = 6 partes, portanto cada uma de 120px/6 = 20px.

A distribuição deste espaço de 120px pelos boxes será conforme mostrado a seguir:

Primeiro box - $200\text{px} + 3 \times 20\text{px} = 260\text{px}$;

Segundo box - $300\text{px} + 2 \times 20\text{px} = 340\text{px}$;

Terceiro box - $380\text{px} + 1 \times 20\text{px} = 400\text{px}$;

Na figura 17.11 mostramos o efeito da declaração dos valores para **flex-basis**, conforme o exemplo descrito, com a finalidade distribuir o excesso de largura do container pelos boxes proporcionalmente aos números 3, 2 e 1, respectivamente.

Topo		
Título 1 260px	Título 2 340px	Título 3 400px
Texto da seção 1 - Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi eleifend, purus quis laoreet faucibus, ante augue malesuada mi, id rhoncus augue lorem eget elit. Ut sollicitudin sodales purus. Phasellus libero felis, blandit nec, commodo ut, imperdiet ut, nibh.	Texto da seção 2 - Curabitur hendrerit erat ut augue. Cras gravida. Mauris consequat aliquam leo. Aenean non tortor id metus aliquet consectetur. Quisque sodales lectus ac orci. Donec eleifend fringilla mi. Vivamus vel massa. Aenean interdum pellentesque sem. Nulla pellentesque felis et tortor. Duis cursus, dui non dictum tincidunt, wisi ipsum mollis wisi, nec ornare velit ipsum eget enim. In sed felis.	Texto da seção 3 - Pellentesque sapien. Sed purus sapien, eleifend ac, volutpat ac, molestie eget, massa. Nam a nibh. Nulla facilisi. Mauris ac metus. Donec porttitor, turpis et sollicitudin vehicula, nunc dui dignissim arcu, euismod hendrerit arcu est quis elit. In quam tellus, auctor in, dignissim at, euismod gravida, lacus. Nullam dui. Donec sit amet eros varius lacus laoreet vestibulum. Phasellus vestibulum vehicula velit. Vestibulum malesuada lectus eu urna.
Rodapé		

Figura 17.11 – Propriedade *flex-basis*.

17.2.7 justify-content

Essa propriedade define como serão alinhados os flexboxes com relação a eventuais espaços em branco criados pelo fato de que a soma de suas larguras seja menor do que a largura do container.

Os valores possíveis para essa propriedade são **flex-start**, **flex-end**, **center**, **space-between**, **space-around** e **stretch**, sendo **flex-start** o valor padrão. Essa propriedade deve ser aplicada ao container dos flexboxes e seus valores causam os efeitos conforme descritos a seguir.

- **flex-start** – Esse é o valor padrão. Os boxes são alinhados à esquerda ou em cima conforme a direção seja segundo o *main axis* ou o *cross axis*, respectivamente.
- **flex-end** – Os boxes são alinhados à direita ou embaixo conforme a direção seja segundo o *main axis* ou o *cross axis*, respectivamente.
- **center** – Os boxes são alinhados no centro.
- **space-between** – Os boxes são alinhados de modo que o espaço remanescente seja dividido igualmente entre eles.
- **space-around** – Os boxes são alinhados de modo que o espaço remanescente seja dividido igualmente entre eles e as margens internas do container.

No site do livro existem dois arquivos interativos (*propriedade-justify-content-row* e *propriedade-justify-content-column*) para você verificar na prática os efeitos da aplicação desta propriedade e seus diferentes valores.

Na figura 17.12, mostramos o efeito da declaração dos valores **flex-start**, **center**, **space-around** e **space-between** quando aplicados em layout disposto segundo o main axis.

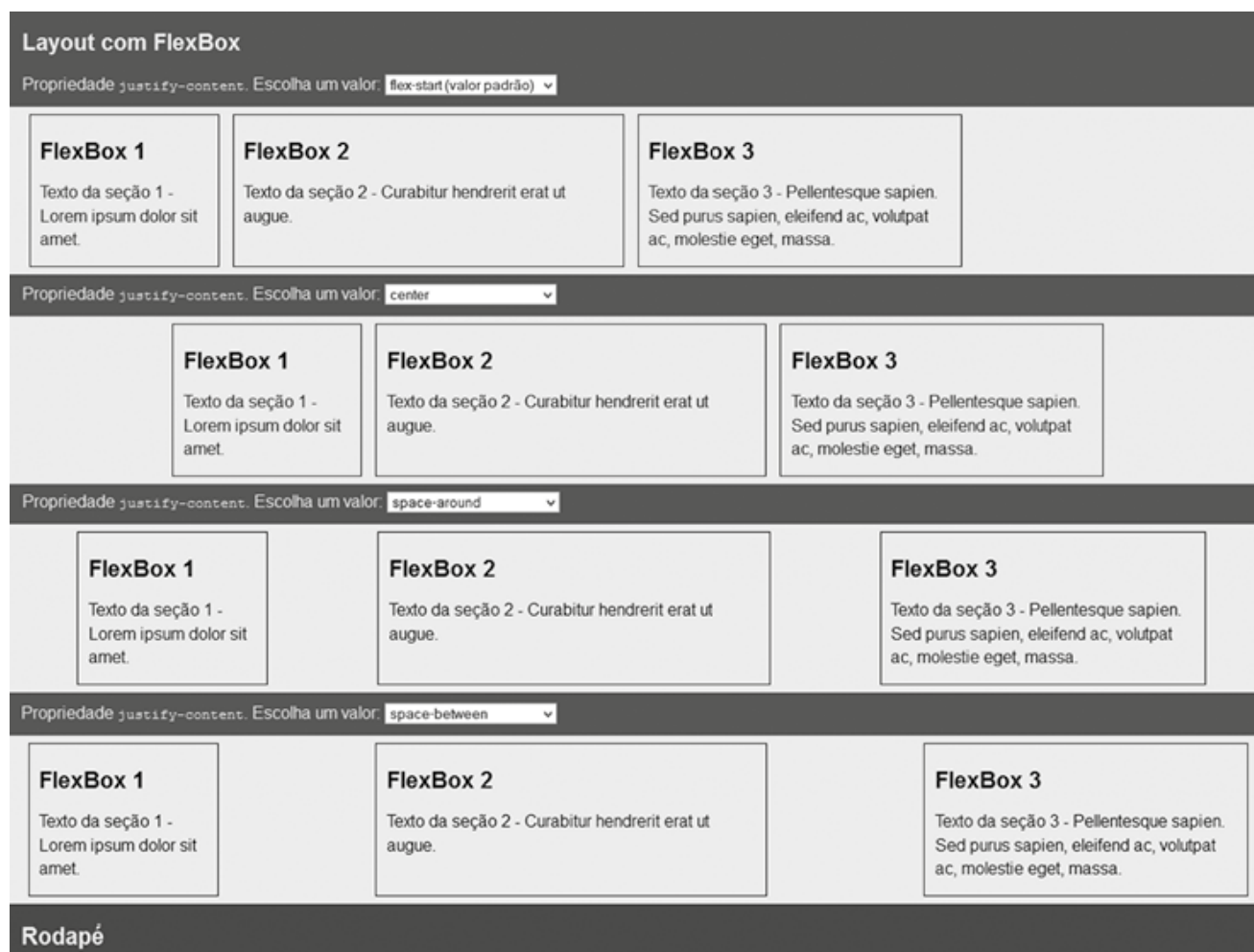


Figura 17.12 – Propriedade *justify-content*.

17.2.8 align-items

Essa propriedade ajusta as dimensões do flexbox aos seus respectivos conteúdos e define como serão alinhados entre si.

Os valores possíveis para essa propriedade são **flex-start**, **flex-end**, **center**, **stretch** e **baseline**, sendo **stretch** o valor padrão. Essa propriedade deve ser aplicada ao container dos flexboxes e seus valores causam os efeitos conforme descritos a seguir.

- **flex-start** – Os boxes são alinhados à esquerda ou em cima conforme a direção seja segundo o cross axis ou o main axis, respectivamente.

- **flex-end** – Os boxes são alinhados à direita ou embaixo conforme a direção seja segundo o *cross axis* ou o *main axis*, respectivamente.
- **center** – Os boxes são alinhados no centro.
- **stretch** – Esse é o valor padrão. Os boxes são alinhados de modo que todo o espaço seja ocupado pelos flexboxes.
- **baseline** – Os boxes são alinhados de modo que a linha de base do conteúdo de cada bloco esteja alinhada.

No site do livro existem dois arquivos interativos (*propriedade-align-items-row* e *propriedade-align-items-column*) para você verificar na prática os efeitos da aplicação desta propriedade e seus diferentes valores.

Na figura 17.13 mostramos o efeito da declaração do valor **center** quando aplicados em layout disposto segundo o *main axis*.



Figura 17.13 – Propriedade *align-items*.

17.2.9 align-self

Enquanto a propriedade **align-items**, estudada anteriormente, alinha todos os flexboxes, essa propriedade permite que se alinhe o flexbox individualmente.

Os valores possíveis para essa propriedade são **auto**, **flex-start**, **flex-end**, **center**, **stretch** e **baseline**, sendo **auto** o valor padrão. Essa propriedade deve ser aplicada

ao flexbox que se deseja alinhar e seus valores causam os efeitos conforme descritos em [17.2.8].

17.2.10 align-content

Essa propriedade define como serão alinhados os flexboxes com relação a eventuais espaços em branco criados pelo fato de que a soma de suas alturas seja menor do que a altura do container.

Os valores possíveis para essa propriedade são **flex-start**, **flex-end**, **center**, **space-between**, **space-around** e **stretch**, sendo **stretch** o valor padrão. Essa propriedade causa efeito semelhante ao da propriedade **justify-content** e deve ser aplicada ao container multilinhas (**flex-wrap: wrap;**) dos flexboxes e seus valores causam os efeitos conforme descritos em [17.2.7].

APÊNDICE A

Propriedades CSS

Neste anexo, mostraremos as propriedades CSS3, que já podem ser testadas em navegadores atuais, descrevendo sumariamente suas finalidades, relacionando os valores válidos para cada uma delas, seu valor inicial, o comportamento para herança CSS, a base para cálculo de valores relativos, a mídia à qual se aplica e a versão CSS na qual foi criada.

Não se trata de uma lista completa de todas as propriedades CSS. Optamos por listar somente aquelas propriedades que com ou sem prefixos proprietários já podem ser usadas e testadas.

A.1 Propriedades das CSS3

Como vimos no capítulo 1, cada nova versão das CSS não somente amplia o conjunto de funcionalidades já existentes, mas também mantém intactas as características e a forma de emprego das funcionalidades constantes das versões anteriores. Veremos, a seguir, uma descrição das propriedades das CSS3 que presumivelmente já atingiram um estágio estável e deixaremos de fora várias propriedades que ainda encontram-se em fase de estudos nas especificações.

Contudo, não há garantias de que algumas das propriedades mostradas permanecerão como estão até o lançamento da especificação final de cada um dos módulos das CSS3.

Mostraremos não somente as novas propriedades, mas também as constantes das versões anteriores que continuam válidas e não mudarão. Na descrição de cada propriedade, assinalamos a versão na qual ela foi criada.

Muitas das novas propriedades criadas pelas CSS3 já estão implementadas em navegadores modernos e podem ser usadas com toda segurança. Algumas delas, quando não suportadas por esse ou aquele navegador, não causam perda de conteúdo ou danos ao layout. A prática e, sobretudo, a experimentação dirão ao desenvolvedor quais propriedades CSS3 usar e quais não usar.



Para facilitar o trabalho de investigação dos leitores deste livro, iremos criar vários exemplos de uso das novas propriedades e disponibilizá-los no site do livro. Tais exemplos não somente servirão para ilustrar a teoria, mas também para servir de fonte de consulta para propriedades já suportadas.

A1.1 Convenções

As convenções adotadas para apresentar as propriedades CSS são conforme descrito

a seguir:

Valores:

- palavras-chave: (por exemplo, `auto`, `disc` etc.);
- tipo de dado: entre “<” e “>” (por exemplo, `<length>`, `<percentage>` etc.);
- valores seguidos devem ocorrer na ordem em que aparecem;
- barra vertical (`|`): separa duas ou mais alternativas na ordem em que devem ocorrer;
- barra vertical dupla (`||`): separa duas ou mais alternativas – uma ou mais devem ocorrer em qualquer ordem;
- duplo e comercial (`&&`): separa dois ou mais componentes – todos devem ocorrer em qualquer ordem;
- colchetes (`[ ]`): indicam agrupamento.

Modificadores:

- asterisco (`*`): valor deve ocorrer zero ou mais vezes;
- sinal de adição (`+`): valor deve ocorrer uma ou mais vezes;
- interrogação (`?`): a ocorrência do valor é opcional;
- par de números entre chaves (`{A,B}`): valor deve ocorrer no mínimo A vezes e no máximo B vezes.

alignment-adjust

Essa propriedade se destina a alinhar com precisão elementos inline.

Valor inicial	auto
Aplica-se	a elementos inline
Herdada	não
Referência para porcentagem	propriedade <code>line-height</code> do elemento
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `alignment-adjust` são: `baseline` | `before-edge` | `text-before-edge` | `middle` | `central` | `after-edge` | `text-after-edge` | `ideographic` | `alphabetic` | `hanging` | `mathematical` | `<percentage>` | `<length>` `alignment-baseline`

Essa propriedade se destina a definir o alinhamento de um elemento inline com

relação ao seu elemento-pai. Ou seja, qual ponto das linhas de base do elemento-pai servirá de referência para o elemento inline.

Valor inicial	auto
Aplica-se	a elementos inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `alignment-baseline` são: `baseline` | `use-script` | `before-edge` | `text-before-edge` | `middle` | `central` | `after-edge` | `text-after-edge` | `ideographic` | `alphabetic` | `hanging` | `mathematical` `animation-name`

Essa propriedade se destina a definir uma lista de nomes ou o nome da animação a ser aplicada ao elemento.

Valor inicial	nenhum
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-name` são: `none` | `IDENT` [, `none` | `IDENT`]*

`IDENT` = nome escolhido para a animação `animation-duration`

Essa propriedade se destina a definir o tempo de duração de um ciclo da animação.

Valor inicial	0
----------------------	---

Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-duration` são: `<time> [, <time>]*`

animation-delay

Essa propriedade se destina a definir o tempo de espera para início da animação.

Valor inicial	0
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-delay` são: `<time> [, <time>]*`

Valores negativos são admitidos e significam que a animação inicia imediatamente no ponto em que ela estaria se já tivesse sido transcorrida a quantidade de tempo especificada como valor negativo.

animation-direction

Essa propriedade se destina a definir o sentido da animação.

Valor inicial	normal
Aplica-se	a elementos nível de bloco e inline

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-direction` são: `normal` | `alternate` | `[, normal | alternate]*`

animation-timing-function

Essa propriedade se destina a definir a maneira como a animação se processa ao longo de um ciclo.

Valor inicial	<code>ease</code>
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-timing-function` são: `ease` | `linear` | `ease-in` | `ease-out` | `ease-in-out` | `cubic-bezier(<number>, <number>, <number>, <number>)` | `[, ease | linear | ease-in | ease-out | ease-in-out | cubic-bezier(<number>, <number>, <number>, <number>)]*`

animation-iteration-count

Essa propriedade se destina a definir o número de vezes que o ciclo da animação se repete.

Valor inicial	1
Aplica-se	a elementos nível de bloco e inline

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-iteration-count` são: `infinite` | `<number>` [, `infinite` | `<number>`]*

animation-play-state

Essa propriedade se destina a definir o estado da animação, ou seja, se em pausa ou em movimento.

Valor inicial	<code>running</code>
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `animation-play-state` são: `running` | `paused` [, `running` | `paused`]*

Essa funcionalidade pode ser obtida com uso de outros métodos, e, por essa razão, poderá ser removida das especificações.

animation

Essa propriedade é a forma abreviada de se declarar seis propriedades da animação, conforme relacionadas a seguir, no subtítulo Valores.

Valor inicial	ver propriedades individuais
Aplica-se	a elementos nível de bloco e inline

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para animation são: [`<animation-name>` || `<animation-duration>` || `<animation-timing-function>` || `<animation-delay>` || `<animation-iteration-count>` || `<animation-direction>`] [, [`<animation-name>` || `<animation-duration>` || `<animation-timing-function>` || `<animation-delay>` || `<animation-iteration-count>` || `<animation-direction>`]]*

appearance

Usa-se essa propriedade para fazer com que um elemento tenha aparência semelhante à de um elemento de interface standard na plataforma usada.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual, interativa
Criada na versão	CSS3

Valores

normal | `<appearance>` | inherit Alguns valores possíveis para appearance são: button, push-button, hyperlink, radio-button, checkbox, pop-up-menu, list-menu, radio-group, checkbox-group, field, password.

azimuth

Essa propriedade se destina à mídia falada e é empregada para um azimuth para o som.

Valor inicial	center
----------------------	--------

Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	falada
Criada na versão	CSS2

Valores

Os valores possíveis para azimuth são:

<angle> | [[left-side | far-left | left | center-left | center | center-right | right | far-right | right-side] || behind] | leftwards | rightwards | inherit
backface-visibility

Essa propriedade se destina a definir se a parte posterior de um elemento será ou não visível quando a ele se aplica uma transformação 3D.

Valor inicial	visible
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para backface-visibility são: visible | hidden background-image

Essa propriedade se destina a definir uma ou mais imagens de fundo para um elemento.

Valor inicial	none
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<bg-image> [<bg-image>]*`

Os valores possíveis para **bg-image** são: `URI` | `none` `background-position`

Essa propriedade se destina a definir a posição inicial da imagem de fundo para um elemento.

Valor inicial	0 0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	dimensão da área de posicionamento do fundo menos dimensão da imagem
Mídia	visual
Criada na versão	CSS1

Valores

`<bg-position> [<bg-position>]*`

Os valores possíveis para **bg-position** são: `[ [ top | bottom ] | [ <percentage> | <length> | left | center | right ] [ <percentage> | <length> | top | center | bottom ]? | [ center | [ left | right ] [ <percentage> | <length> ]? ] && [ center | [ top | bottom ] [ <percentage> | <length> ]? ] ]`

background-size

Essa propriedade se destina a definir as dimensões da imagem de fundo para um elemento.

Valor inicial	auto
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	dimensão da área de posicionamento do fundo
Mídia	visual
Criada na versão	CSS3

Valores

<bg-size> [,<bg-size>]*

Os valores possíveis para **bg-size** são: [<length> | <percentage> | auto]{1,2} | cover | contain background-origin

Essa propriedade se destina a definir as dimensões da imagem de fundo para um elemento.

Valor inicial	padding-box
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

<box> [,<box>]*

Os valores possíveis para **box** são: padding-box | border-box | content-box background-clip

Essa propriedade se destina a definir a área, no elemento, na qual a imagem de fundo será inserida.

Valor inicial	border-box
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

`<box> [,<box>]*`

Os valores possíveis para `box` são: `padding-box` | `border-box` | `content-box`
`background-attachment`

Essa propriedade se destina a definir se a imagem de fundo será fixa em relação à viewport, se rolará com o elemento ou rolará com seu conteúdo.

Valor inicial	<code>scroll</code>
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<attachment> [, <attachment>]*`

Os valores possíveis para `attachment` são: `scroll` | `fixed` | `local` `background-repeat`

Essa propriedade se destina a definir como será a repetição da imagem de fundo após ela ter sido dimensionada (`background-size`) e posicionada (`background-position`).

Valor inicial	<code>repeat</code>
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<repeat-style> [,<repeat-style>]*`

Os valores possíveis para `repeat-style` são: `repeat-x` | `repeat-y` | `repeat` | `space` | `round` | `no-repeat` | `repeat background-color`

Essa propriedade se destina a definir a cor de fundo para um elemento. A imagem de fundo, se houver uma, será posicionada sobre a cor de fundo.

Valor inicial	transparent
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<color>`

Os valores possíveis para `color` são:

`<keyword>` | `<numeric-values>` `background`

Essa propriedade é a forma abreviada de se declarar as propriedades para `background`.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não

Referência para porcentagem	ver propriedades individuais
Mídia	visual
Criada na versão	CSS1

Valores

[<bg-layer>,]* <final-bg-layer> <bg-layer> = <bg-image> || <bg-position> [/ <bg-size>]? || <repeat-style> || <attachment> || <box>{1,2}

<final-bg-layer> = <bg-image> || <bg-position> [/ <bg-size>]? || <repeat-style> || <attachment> || <box>{1,2} || <'background-color'> border

Essa propriedade é a forma abreviada de se declarar as propriedades para as quatro bordas de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	ver propriedades individuais
Mídia	visual
Criada na versão	CSS1

Valores

<border-width> || <border-style> || <color> <border-width> = <lenght> | thin | medium | thick <border-style> = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset <color> = <keyword> | <numeric-values> border-top

Essa propriedade é a forma abreviada de se declarar as propriedades para a borda superior de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS1

Valores

`<border-width> || <border-style> || <color> <border-width> = <lenght> | thin | medium | thick <border-style> = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset <color> = <keyword> | <numeric-values> border-right`

Essa propriedade é a forma abreviada de se declarar as propriedades para a borda direita de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<border-width> || <border-style> || <color> <border-width> = <lenght> | thin | medium | thick <border-style> = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset <color> = <keyword> | <numeric-values> border-bottom`

Essa propriedade é a forma abreviada de se declarar as propriedades para a borda inferior de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual

Criada na versão	CSS1
-------------------------	------

Valores

`<border-width> || <border-style> || <color> <border-width> = <lenght> | thin | medium | thick <border-style> = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset <color> = <keyword> | <numeric-values> border-left`

Essa propriedade é a forma abreviada de se declarar as propriedades para a borda esquerda de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

`<border-width> || <border-style> || <color>` Os valores possíveis para `border-left` são: `<border-width> = <lenght> | thin | medium | thick <border-style> = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset <color> = <keyword> | <numeric-values> border-width`

Essa propriedade se destina a definir a espessura para as quatro bordas de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do elemento-pai
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `<border-width>` são: `<border-width>{1,4} = <lenght> | thin | medium | thick` `border-top-width`

Essa propriedade se destina a definir a espessura para a borda superior de um elemento.

Valor inicial	medium
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do elemento-pai
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `border-top-width` são: `<lenght> | thin | medium | thick` `border-right-width`

Essa propriedade se destina a definir a espessura para a borda direita de um elemento.

Valor inicial	medium
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do elemento-pai
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `border-right-width` são: `<lenght> | thin | medium | thick` `border-bottom-width`

Essa propriedade se destina a definir a espessura para a borda inferior de um elemento.

Valor inicial	medium
----------------------	--------

Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do elemento-pai
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **border-bottom-width** são: `<lenght>` | `thin` | `medium` | `thick`

Essa propriedade se destina a definir a espessura para a borda esquerda de um elemento.

Valor inicial	medium
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do elemento-pai
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **border-left-width** são: `<lenght>` | `thin` | `medium` | `thick`

Essa propriedade se destina a definir o estilo para as quatro bordas de um elemento, exceto quando há uma imagem para o estilo da borda.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual

Criada na versão	CSS1
-------------------------	------

Valores

Os valores possíveis para `<border-style>` são: `<border-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset border-top-style`

Essa propriedade se destina a definir o estilo para a borda superior de um elemento.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `<border-top-style>` são: `<border-top-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset border-right-style`

Essa propriedade se destina a definir o estilo para a borda direita de um elemento.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `<border-right-style>` são: `<border-right-style>{1,4}`

= none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset
border-bottom-style

Essa propriedade se destina a definir o estilo para a borda inferior de um elemento.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para <border-bottom-style> são: <border-bottom-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset border-left-style

Essa propriedade se destina a definir o estilo para a borda esquerda de um elemento.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para <border-left-style> são: <border-left-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset border-color

Essa propriedade se destina a definir a cor para as quatro bordas de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `border-color` são: `<color> {1,4} = <keyword> | <numeric-values> border-top-color`

Essa propriedade se destina a definir a cor para a borda superior de um elemento.

Valor inicial	cor definida para o elemento
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `border-top-color` são: `<color>{1,4} = <keyword> | <numeric-values> border-right-color`

Essa propriedade se destina a definir a cor para a borda direita de um elemento.

Valor inicial	cor definida para o elemento
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **border-right-color** são: `<color>{1,4} = <keyword> | <numeric-values>` **border-bottom-color**

Essa propriedade se destina a definir a cor para a borda inferior de um elemento.

Valor inicial	cor definida para o elemento
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **border-bottom-color** são: `<color>{1,4} = <keyword> | <numeric-values>` **border-left-color**

Essa propriedade se destina a definir a cor para a borda esquerda de um elemento.

Valor inicial	cor definida para o elemento
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **border-left-color** são:

`<keyword> | <numeric-values>` **border-image**

Essa propriedade se destina a definir como uma imagem será usada para criar as bordas de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	ver propriedades individuais
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `border-image` são: `<'border-image-source'> || <'border-image-slice'> [ / <'border-image-width'>? [ / <'border-image-outset'> ]? ]? || <'border-image-repeat'> border-image-source`

Essa propriedade se destina a definir a imagem que será usada para criar as bordas de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	ver propriedades individuais
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `border-image-source` são: `none` | `URI` `border-image-slice`

Essa propriedade se destina a definir como a imagem será recortada em nove regiões para ser usada na criação das bordas de um elemento.

Valor inicial	100%
Aplica-se	a todos os elementos, exceto elementos internos de tabelas, quando as bordas

	se sobrepõem (collapse)
Herdada	não
Referência para porcentagem	dimensões da imagem
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **border-image-slice** são: [<number> | <percentage>] {1,4} && fill?

border-image-width

Essa propriedade se destina a definir como a área da borda será recortada em nove regiões para ser usada na criação das bordas de um elemento.

Valor inicial	1
Aplica-se	a todos os elementos, exceto elementos internos de tabelas, quando as bordas se sobrepõem (collapse)
Herdada	não
Referência para porcentagem	dimensões da área da imagem
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **border-image-width** são: [<length> | <percentage> | <number> | auto] {1,4}

border-image-outset

Essa propriedade se destina a definir quanto a área da borda se estenderá pelos quatro lados.

Valor inicial	0
Aplica-se	a todos os elementos, exceto elementos internos de tabelas, quando as bordas se sobrepõem (collapse)

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `border-image-outset` são: [<length> | <number>]{1,4}

border-image-repeat

Essa propriedade se destina a definir quanto a área da borda se estenderá pelos quatro lados.

Valor inicial	stretch
Aplica-se	a todos os elementos, exceto elementos internos de tabelas, quando as bordas se sobrepõem (<code>collapse</code>)
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `border-image-repeat` são: [stretch | repeat | round]{1,2}

border-collapse

Essa propriedade se destina a definir a apresentação das bordas das células de uma tabela como justapostas.

Valor inicial	separate
Aplica-se	a elementos table e inline-table

Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `border-collapse` são: `collapse` | `separate` | `inherit`
`border-spacing`

Essa propriedade se destina a definir a distância que separa as bordas das células de uma tabela.

Valor inicial	0
Aplica-se	a elementos table e inline-table
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `border-spacing` são: `<length>` `<length>?` | `inherit` `box-shadow`

Essa propriedade define os efeitos de sombreamento para um elemento.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual

Criada na versão	CSS3
-------------------------	------

Valores

Os valores possíveis para **box-shadow** são: `none` | `<shadow>` [, `<shadow>`]*

`<shadow>` = `inset?` && [`<length>`{2,4} && `<color>?`]

box-sizing

Essa propriedade define como serão computadas as dimensões do box.

Valor inicial	<code>content-box</code>
Aplica-se	a todos os elementos que aceitam <code>width</code> e <code>height</code>
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **box-sizing** são: `content-box` | `border-box` | `inherit` `top`

Essa propriedade se destina a definir a distância que separa a borda superior de um bloco de conteúdo da linha de margem superior de um box (bloco de conteúdo) nele posicionado de forma absoluta. Para boxes posicionados de forma absoluta, a distância é medida em relação à borda superior do próprio box, e não do seu bloco de conteúdo.

Valor inicial	<code>auto</code>
Aplica-se	a elementos posicionados com <code>position</code> outra que não <code>static</code>
Herdada	não
Referência para porcentagem	altura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **top** são: `<length>` | `<percentage>` | `auto` | `inherit` `right`

Essa propriedade se destina a definir a distância que separa a borda direita de um bloco de conteúdo da linha de margem direita de um box (bloco de conteúdo) nele posicionado de forma absoluta. Para boxes posicionados de forma absoluta, a distância é medida em relação à borda direita do próprio box, e não do seu bloco de conteúdo.

Valor inicial	auto
Aplica-se	a elementos posicionados com position outra que não static
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **right** são: `<length>` | `<percentage>` | `auto` | `inherit` `bottom`

Essa propriedade se destina a definir a distância que separa a borda inferior de um bloco de conteúdo da linha de margem inferior de um box (bloco de conteúdo) nele posicionado de forma absoluta. Para boxes posicionados de forma absoluta, a distância é medida em relação à borda inferior do próprio box, e não do seu bloco de conteúdo.

Valor inicial	auto
Aplica-se	a elementos posicionados com position outra que não static
Herdada	não
Referência para porcentagem	altura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **bottom** são: `<length>` | `<percentage>` | `auto` | `inherit` `left`

Essa propriedade se destina a definir a distância que separa a borda esquerda de

um bloco de conteúdo da linha de margem esquerda de um box (bloco de conteúdo) nele posicionado de forma absoluta. Para boxes posicionados de forma absoluta, a distância é medida em relação à borda esquerda do próprio box, e não do seu bloco de conteúdo.

Valor inicial	auto
Aplica-se	a elementos posicionados com position outra que não static
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **left** são: `<length>` | `<percentage>` | `auto` | `inherit` | `caption-side`

Essa propriedade se destina a posicionar o bloco de conteúdo da legenda descritiva de uma tabela.

Valor inicial	top
Aplica-se	a elementos “table-caption”
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **caption-side** são: `top` | `bottom` | `inherit` | `clear`

Essa propriedade se destina a definir qual dos lados de um elemento não será adjacente a um bloco anterior que tenha sido flutuado (com uso da propriedade `float`).

Valor inicial	none
----------------------	------

Aplica-se	a elementos nível de bloco
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **clear** são: **none** | **left** | **right** | **both** | **inherit** clip

Essa propriedade se aplica somente a elementos posicionados de forma absoluta e se destina a definir uma área de recorte do elemento.

Valor inicial	auto
Aplica-se	a elementos posicionados absolutamente
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **clip** são: **<shape>** | **auto** | **inherit** O valor possível para **<shape>** é: **<shape>** = **rect(<top>, <right>, <bottom>, <left> color**

Essa propriedade se destina a definir uma cor para o conteúdo do elemento.

Valor inicial	depende do agente de usuário
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual

Criada na versão	CSS1
-------------------------	------

Valores

Os valores possíveis para **color** são: <keyword> | <numeric-values> column-width

Essa propriedade se destina a definir a largura de colunas e de elementos multicolumnares.

Valor inicial	auto
Aplica-se	a elementos nível de bloco (exceto tabelas), células de tabelas e elementos inline-block
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **column-width** são: <length> | auto column-count

Essa propriedade se destina a definir o número de colunas de um elemento multicolumnar.

Valor inicial	auto
Aplica-se	a elementos nível de bloco (exceto tabelas), células de tabelas e elementos inline-block
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **column-count** são: <integer> | auto columns

Essa propriedade é a forma abreviada de se declarar as propriedades **column-width**

e `column-count`.

Valor inicial	ver propriedades individuais
Aplica-se	a elementos nível de bloco (exceto tabelas), células de tabelas e elementos inline-block
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `columns` são: `<column-width> || <column-count>` `column-gap`

Essa propriedade se destina a definir o espaçamento entre colunas.

Valor inicial	<code>normal</code>
Aplica-se	a elementos multicolunares
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-gap` são: `<length> | normal` `column-rule-width`

Essa propriedade se destina a definir a espessura de uma linha posicionada entre colunas.

Valor inicial	<code>medium</code>
Aplica-se	a elementos multicolunares

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-rule-width` são: `<border-width>{1,4} = <lenght> | thin | medium | thick` `column-rule-style`

Essa propriedade se destina a definir o estilo de uma linha posicionada entre colunas.

Valor inicial	none
Aplica-se	a elementos multicolunares
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-rule-style` são: `<border-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset` `column-rule-color`

Essa propriedade se destina a definir a cor de uma linha posicionada entre colunas.

Valor inicial	depende do agente de usuário
Aplica-se	a elementos multicolunares
Herdada	não
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-rule-color` são: `<border-style>{1,4} = none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset` `column-rule`

Essa propriedade é a forma abreviada de se declarar as propriedades `column-rule-width`, `column-rule-style`, `column-rule-color`.

Valor inicial	ver propriedades individuais
Aplica-se	a elementos multicolumnares
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-rule` são: `<column-rule-width> || <column-rule-style> || [ <column-rule-color> | transparent ]`

column-span

Essa propriedade se destina a definir por quantas colunas um elemento se estende.

Valor inicial	none
Aplica-se	a elementos nível de bloco, exceto floats e elementos posicionados de forma absoluta
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `column-span` são: `none` | `all` content

Essa propriedade se destina a definir o conteúdo a ser renderizado dentro de um elemento ou pseudoelemento.

Valor inicial	normal
Aplica-se	a todos os elementos, <code>::before</code> , <code>::after</code> , <code>::alternate</code> , <code>::marker</code> , <code>::line-marker</code> , área de margens, área de <code>@footnotes</code>
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `content` são: [`<uri>` ',']* [`normal` | `none` | `inhibit` | `<content-list>`]

`<content-list>` = [`pending(<identifier>)` | `<string>` | `contents` | `footnote` | `endnote` | `section-note` | `list-item` | `<counters>` | `<strings>` | `open-quote` | `close-quote` | `no-open-quote` | `no-close-quote` | `<glyph>` | `<uri>` | `<date>` | `document-url` | `<target>`]+

CURSOR

Essa propriedade se destina a definir o aspecto do cursor do mouse.

Valor inicial	auto
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `cursor` são: `[[<uri> [<x> <y>]?,* [auto | default | none | context-menu | help | pointer | progress | wait | cell | crosshair | text | vertical-text | alias | copy | move | no-drop | not-allowed | e-resize | n-resize | ne-resize | nw-resize | s-resize | se-resize | sw-resize | w-resize | ew-resize | ns-resize | nesw-resize | nwse-resize | col-resize | row-resize | all-scroll]] | inherit direction`

Essa propriedade se destina a definir a direção da escrita.

Valor inicial	ltr
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `direction` são: `ltr | rtl | inherit display`

Essa propriedade se destina a definir o tipo de box gerado para um elemento.

Valor inicial	inline
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `display` são: `inline | block | inline-block | list-item | run-in | compact | table | inline-table | table-row-group | table-header-group | table-footer-group | table-row | table-column-group | table-column | table-cell | table-caption | ruby | ruby-base | ruby-text | ruby-base-group | ruby-text-`

`group` | `<template>` | `none` O valor `<template>` se destina a definir valores, de acordo com o módulo `template layout`. Ver capítulo 16.

empty-cells

Essa propriedade se destina a definir o modo de renderização das bordas e do fundo de células vazias de tabela.

Valor inicial	show
Aplica-se	a células de tabelas
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `empty-cells` são: `show` | `hide` | `inherit` float

Essa propriedade se destina a definir a posição de flutuação de um bloco.

Valor inicial	none
Aplica-se	a todos os elementos, exceto blocos posicionados de forma absoluta
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `float` são: `left` | `right` | `none` | `<page-floats>` O valor `<page-float>` se destina a definir valores, de acordo com as regras de geração de conteúdos para Paged Media.

font-style

Essa propriedade se destina a definir se a fonte será em itálico ou oblíqua.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font-style` são: `normal` | `italic` | `oblique` | `inherit` `font-variant`

Essa propriedade se destina a definir uma fonte renderizada como se fosse em caixa-alta, mas com tamanho reduzido e diferentes proporções em relação àquelas em caixa-alta.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font-variant` nas CSS2.1 são: `normal` | `small-caps` | `inherit` As CSS3 criaram novos valores para essa propriedade. Tratamos desses valores no capítulo 8.

font-weight

Essa propriedade se destina a definir o peso das fontes, ou seja, a intensidade do negrito ou a espessura dos traços da fonte.

Valor inicial	normal
----------------------	--------

Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font-weight` são: `normal` | `bold` | `bolder` | `lighter` | `100` | `200` | `300` | `400` | `500` | `600` | `700` | `800` | `900`

font-size

Essa propriedade se destina a definir o tamanho da fonte.

Valor inicial	medium
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font-size` são: `<absolute-size>` | `<relative-size>` | `<length>` | `<percentage>` | `inherit` `line-height`

Essa propriedade se destina a definir a distância entre linhas.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `line-height` são: `normal` | `<number>` | `<length>` | `<percentage>` | `none` `font-family`

Essa propriedade se destina a definir as famílias de fontes.

Valor inicial	depende do agente de usuário
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font-family` são: `[ [ <family-name> | <generic-family> ] [ , <family-name> | <generic-family> ]* ]` | `inherit` `font`

Essa propriedade é a forma abreviada de se declarar as propriedades para fontes.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `font` são: `[ [ <'font-style'> | <'font-variant-css21'> | <'font-weight'> ]? <'font-size'> [ / <'line-height'> ]? <'font-family'> ]` |

`caption` | `icon` | `menu` | `message-box` | `small-caption` | `status-bar` | `inherit` Os valores de `font-variant`, quando declarados com uso da forma abreviada, devem ser os previstos nas especificações para as CSS2.1, ou seja, os valores criados pelas CSS3 não podem ser declarados. Para isso, use a declaração individual, e não a abreviada.

font-size-adjust

Essa propriedade se destina a preservar a legibilidade da fonte alternativa definida com uso da propriedade `font-family`. Vimos essa propriedade com detalhes no capítulo 8.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2/CSS3 (não existe na versão 2.1)

Valores

Os valores possíveis para `font-size-adjust` são: `<number>` | `none` | `inherit` `font-stretch`

Essa propriedade se destina a definir o quanto uma fonte será comprimida ou expandida.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `font-stretch` são: `normal` | `ultra-condensed` | `extra-condensed` | `condensed` | `semi-condensed` | `semi-expanded` | `expanded` | `extra-expanded` | `ultra-expanded` | `inherit` `height`

Essa propriedade se destina a definir a altura de uma área do box, de acordo com o Box Model.

Valor inicial	auto
Aplica-se	a todos os elementos, exceto elementos inline “non-replaced”, colunas e grupos de colunas e tabelas
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS3

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `height` são: `<length>` | `<percentage>` | `auto` `letter-spacing`

Essa propriedade se destina a definir as distâncias mínima, máxima e ótima entre os caracteres de uma fonte.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	distância padrão entre os caracteres
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `letter-spacing` são: `<spacing-limit>` {1,3}

`<spacing-limit> = normal | <length> | <percentage> list-style-type`

Essa propriedade se destina a definir o estilo do marcador dos itens de uma lista.

Valor inicial	disc
Aplica-se	a todos os elementos com <code>display: list-item</code>
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `list-style-type` são: `<string> | <counter-style> | inline | none list-style-position`

Essa propriedade se destina a definir uma imagem para servir como marcador dos itens de uma lista.

Valor inicial	outside
Aplica-se	a todos os elementos com <code>display: list-item</code>
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `list-style-position` são: `inside | hanging | outside list-style-image`

Essa propriedade se destina a definir uma imagem para servir como marcador dos itens de uma lista.

Valor inicial	none
Aplica-se	a todos os elementos com <code>display: list-item</code>

Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **list-style-image** são: `<image> | none <image> = URL` endereço de uma imagem list-style

Essa propriedade é a forma abreviada de se declarar as propriedades **list-style-type**, **list-style-position** ou **list-style-image** para marcador dos itens de uma lista.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos com display: list-item
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **list-style** são: `<list-style-type> || <list-style-position> || <list-style-image> margin-top`

Essa propriedade se destina a definir a dimensão da margem superior de um elemento.

Valor inicial	0
Aplica-se	a todos os boxes, exceto alguns boxes table-* e alguns blocos inline
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical

Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **margin-top** são: `<length>` | `<percentage>` | **auto** **margin-right**

Essa propriedade se destina a definir a dimensão da margem direita de um elemento.

Valor inicial	0
Aplica-se	a todos os boxes, exceto alguns boxes table-* e alguns blocos inline
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **margin-right** são: `<length>` | `<percentage>` | **auto** **margin-bottom**

Essa propriedade se destina a definir a dimensão da margem inferior de um elemento.

Valor inicial	0
Aplica-se	a todos os boxes, exceto alguns boxes table-* e alguns blocos inline
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **margin-bottom** são: `<length>` | `<percentage>` | `auto`
margin-left

Essa propriedade se destina a definir a dimensão da margem esquerda de um elemento.

Valor inicial	0
Aplica-se	a todos os boxes, exceto alguns boxes <code>table-*</code> e alguns blocos inline
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **margin-left** são: `<length>` | `<percentage>` | `auto` `margin`

Essa propriedade é a forma abreviada de se definir as dimensões das quatro margens de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os boxes, exceto alguns boxes <code>table-*</code> e alguns blocos inline
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **margin** são: [`<length>` | `<percentage>` | `auto`] {1,4}

max-width

Essa propriedade se destina a definir a largura máxima de um box.

Valor inicial	none
----------------------	------

Aplica-se	a todos os boxes, exceto elementos inline “non-replaced”, linhas e grupos de linhas de tabelas
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `max-width` são: `<length>` | `<percentage>` | `none` `min-width`

Essa propriedade se destina a definir a largura mínima de um box.

Valor inicial	none
Aplica-se	a todos os boxes, exceto elementos inline “non-replaced”, linhas e grupos de linhas de tabelas
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `min-width` são: `<length>` | `<percentage>` | `none` `max-height`

Essa propriedade se destina a definir a altura máxima de um box.

Valor inicial	none
Aplica-se	a todos os boxes, exceto elementos inline “non-replaced”, linhas e grupos de linhas de tabelas
Herdada	não
Referência para porcentagem	altura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `max-height` são: `<length>` | `<percentage>` | `none` `min-height`

Essa propriedade se destina a definir a altura mínima de um box.

Valor inicial	none
Aplica-se	a todos os boxes, exceto elementos inline “non-replaced”, linhas e grupos de linhas de tabelas
Herdada	não
Referência para porcentagem	altura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS2

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `min-height` são: `<length>` | `<percentage>` | `none` `opacity`

Essa propriedade se destina a definir o grau de opacidade de um elemento.

Valor inicial	1
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `opacity` são: `<alphavalue>` | `inherit <alphavalue>` = varia de 0 (transparente) até 1 (opaco)

Essa propriedade se destina a definir a cor de uma linha de contorno para dar destaque a um elemento.

Valor inicial	<code>invert</code>
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `outline-color` são: `<color>` | `invert` | `inherit` A palavra-chave `invert` garante a inversão de cor em relação à cor de fundo, garantindo a visibilidade de outline, qualquer que seja a cor do fundo.

outline-style

Essa propriedade se destina a definir o estilo de uma linha de contorno para dar destaque a um elemento.

Valor inicial	<code>none</code>
----------------------	-------------------

Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `outline-style` são: `<border-style>` | `inherit <border-style> = none` | `hidden` | `dotted` | `dashed` | `solid` | `double` | `groove` | `ridge` | `inset` | `outset` `outline-width`

Essa propriedade se destina a definir a espessura de uma linha de contorno para dar destaque a um elemento.

Valor inicial	<code>medium</code>
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `outline-width` são: `<border-width>` | `inherit <border-width> = <lenght>` | `thin` | `medium` | `thick` `outline`

Essa propriedade é a forma abreviada de se declarar as propriedades **outline-color**, **outline-style**, **outline-width**.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para **outline** são: [`<outline-color>` || `<outline-style>` || `<outline-width>`] | `inherit` `overflow-x`

Essa propriedade se destina a definir como será apresentado o conteúdo que ultrapassa a largura da área de conteúdo do elemento.

Valor inicial	<code>visible</code>
Aplica-se	a elementos nível de bloco e inline do tipo “non-replaced”
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Elementos “non-replaced” são os elementos que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para **overflow-x** são: `visible` | `hidden` | `scroll` | `auto` | `no-display` | `no-content` `overflow-y`

Essa propriedade se destina a definir como será apresentado o conteúdo que ultrapassa a altura da área de conteúdo do elemento.

Valor inicial	<code>visible</code>
Aplica-se	a elementos nível de bloco e inline do tipo “non-replaced”
Herdada	não

Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Elementos “non-replaced” são os elementos que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `overflow-y` são: `visible` | `hidden` | `scroll` | `auto` | `no-display` | `no-content overflow`

Essa propriedade é a forma abreviada de se definir como será apresentado o conteúdo que ultrapassa a largura e a altura da área de conteúdo do elemento.

Valor inicial	<code>visible</code>
Aplica-se	a elementos nível de bloco e inline do tipo “non-replaced”
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Elementos “non-replaced” são os elementos que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para marcar controles de formulários.

Valores

Os valores possíveis para `overflow` são: [`visible` | `hidden` | `scroll` | `auto` | `no-display` | `no-content`] {1,2}

padding-top

Essa propriedade se destina a definir a dimensão da área de enchimento superior de um elemento.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **padding-top** são: `<length> | <percentage> padding-right`
Essa propriedade se destina a definir a dimensão da área de enchimento direita de um elemento.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **padding-right** são: `<length> | <percentage> padding-bottom`

Essa propriedade se destina a definir a dimensão da área de enchimento inferior de um elemento.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo

porcentagem	vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **padding-bottom** são: `<length>` | `<percentage>` `padding-left`

Essa propriedade se destina a definir a dimensão da área de enchimento esquerda de um elemento.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **padding-left** são: `<length>` | `<percentage>` `padding`

Essa propriedade é a forma abreviada de se definir as dimensões dos quatro enchimentos de um elemento.

Valor inicial	ver propriedades individuais
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	largura do bloco de conteúdo horizontal ou altura do bloco de conteúdo vertical
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **padding** são: [**<length>** | **<percentage>**]{1,4}

perspective() - função de transformação

Essa propriedade define uma matriz para projeção da perspectiva 3D.

Valor inicial	1
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para **perspective** é: **<número>**

perspective-origin

Essa propriedade define o ponto de origem para a perspectiva dos efeitos de transformações 3D.

Valor inicial	50% 50%
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	dimensões do box
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **perspective-origin** são: [[**<percentage>** | **<length>** | **left** | **center** | **right**] [**<percentage>** | **<length>** | **top** | **center** | **bottom**]?] | [[**left** | **center** | **right**] || [**top** | **center** | **bottom**]]

position

Essa propriedade define como um elemento será posicionado.

Valor inicial	<code>static</code>
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `position` são: `static` | `relative` | `absolute` | `fixed` | `inherit`

`rotate(a)` – função de transformação

Função para aplicar uma rotação 2D em torno de um ponto.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `rotate` é:

<medida de ângulo CSS>

`rotateX(x)` – função de transformação

Função para aplicar rotação 3D em torno do eixo x.

Valor inicial	0
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `rotateX` é:

<medida de ângulo CSS>

rotateY(a) – função de transformação

Função para aplicar rotação 3D em torno do eixo y.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `rotateY` é:

<medida de ângulo CSS>

rotateZ(a) – função de transformação

Função para aplicar rotação 3D em torno do eixo z.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `rotateZ` é:

<medida de ângulo CSS>

scale(x, y) – função de transformação

Função para aplicar escala 2D.

Valor inicial	1, 1
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `scale` são: <número> [, <número>]

scaleX(x) – função de transformação

Função para aplicar escala na dimensão horizontal 2D.

Valor inicial	1, 1
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possíveis para `scaleX` é:

<número>

`scaleY(y)` – função de transformação

Função para aplicar escala na dimensão vertical 2D.

Valor inicial	1, 1
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `scaleY` é:

<número>

`skew(a, b)` – função de transformação

Função para aplicar transformação ao longo dos eixos x e y.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `skew` são:

<ângulo> [, <ângulo>]

skewX(a) – função de transformação

Função para aplicar transformação ao longo do eixo x.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para skewX é:

<ângulo>

skewY(b) – função de transformação

Função para aplicar transformação ao longo do eixo y.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para skewY é:

<ângulo>

table-layout

Essa propriedade define como será o layout de células, linhas e colunas de tabelas.

Valor inicial	auto
Aplica-se	a elementos table e inline table
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS2

Valores

Os valores possíveis para `table-layout` são: `auto` | `fixed` | `inherit` `text-align`

Essa propriedade define como será o alinhamento horizontal de conteúdos inline.

Valor inicial	start
Aplica-se	a blocos de conteúdos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `text-align` são: [`start` | `<string>`]? [`start` | `end` | `left` | `right` | `center` | `justify` | `match-parent`]

text-decoration

Essa propriedade define como será o elemento decorativo a ser anexado ao texto.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—

Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **text-decoration** são: `none` | `[ underline || overline || line-through || blink ]` | `inherit` `text-indent`

Essa propriedade define a endentação das linhas de um texto.

Valor inicial	0
Aplica-se	a blocos de conteúdos
Herdada	sim
Referência para porcentagem	largura do bloco de conteúdo
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **text-indent** são: `[ <length> | <percentage> ]` && `[ hanging || each-line ]`?

text-shadow

Essa propriedade define os efeitos de sombreamento para um texto.

Valor inicial	0
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **text-shadow** são: `none` | `[ <shadow> , ] * <shadow> <shadow>`

= inset? && [<length>{2,4} && <color>?]

text-transform

Essa propriedade define os efeitos de capitalização para um texto.

Valor inicial	none
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para `text-transform` são: `capitalize` | `uppercase` | `lowercase` | `none` | `inherit` transform

Essa propriedade define efeitos de transformações 2D e 3D para um box.

Valor inicial	none
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	dimensões do box
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `transform` são: `none` | <função de transformação> [<função de transformação>]*

<função de transformação> = `matrix()` | `matrix3d()` | `translate()` | `translate3d()` | `translateX()` | `translateY()` | `translateZ()` | `scale()` | `scale3d()` | `scaleX()` | `scaleY()` | `scaleZ()` | `rotate()` | `rotate3d()` | `rotateX()` | `rotateY()` | `rotateZ()` | `skew()` | `skewX()` | `skewY()` | `perspective` transform-origin

Essa propriedade define o ponto de origem para aplicação de efeitos de

transformações 2D e 3D.

Valor inicial	50% 50% 0
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	dimensões do box
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `transform-origin` são: [[[<percentage> | <length> | left | center | right] [<percentage> | <length> | top | center | bottom]?] <length>] | [[[left | center | right] || [top | center | bottom]] <length>]

`transform-style`

Essa propriedade define o estilo das transformações 2D e 3D.

Valor inicial	flat
Aplica-se	a elementos nível de bloco e inline
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para `transform-style` são: `flat` | `preserve 3d` `transition-property`

Essa propriedade define a propriedade CSS à qual será aplicada uma transição.

Valor inicial	todas
Aplica-se	a todos os elementos e aos pseudo-elementos <code>:before</code> e <code>:after</code>

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

Os valores possíveis para **transition-property** são: `none` | `all` | [`<IDENT>`] [`'`, `'` `<IDENT>`]*

`<IDENT>` = uma propriedade CSS

transition-duration

Essa propriedade define o tempo de duração de uma transição.

Valor inicial	0
Aplica-se	a todos os elementos e aos pseudoelementos <code>:before</code> e <code>:after</code>
Herdada	não
Referência para porcentagem	—
Mídia	interativa
Criada na versão	CSS3

Valores

O valor possível para **transition-duration** é: `<time>` [, `<time>`]*

`<time>` = número de segundos

transition-timing-function

Essa propriedade define como serão calculados os valores intermediários de uma transição.

Valor inicial	<code>ease</code>
Aplica-se	a todos os elementos e aos pseudoelementos <code>:before</code> e <code>:after</code>

Herdada	não
Referência para porcentagem	—
Mídia	interativa
Criada na versão	CSS3

Valores

Os valores possíveis para **transition-timing-function** são: `ease` | `linear` | `ease-in` | `ease-out` | `ease-in-out` | `cubic-bezier(<number>, <number>, <number>, <number>)` [, `ease` | `linear` | `ease-in` | `ease-out` | `ease-in-out` | `cubic-bezier(<number>, <number>, <number>, <number>)`]*

transition-delay

Essa propriedade define o tempo de retardo para início de uma transição.

Valor inicial	0
Aplica-se	a todos os elementos e aos pseudoelementos <code>:before</code> e <code>:after</code>
Herdada	não
Referência para porcentagem	—
Mídia	interativa
Criada na versão	CSS3

Valores

O valor possível para **transition-delay** é: `<time>` [, `<time>`]*

`<time>` = número de segundos

transition

Essa propriedade é a forma abreviada de definição de uma transição.

Valor inicial	0
Aplica-se	a todos os elementos e aos pseudoelementos <code>:before</code> e <code>:after</code>

Herdada	não
Referência para porcentagem	—
Mídia	interativa
Criada na versão	CSS3

Valores

Os valores possíveis para `transition` são: [`<'transition-property'>` || `<'transition-duration'>` || `<'transition-timing-function'>` || `<'transition-delay'>`] [, [`<'transition-property'>` || `<'transition-duration'>` || `<'transition-timing-function'>` || `<'transition-delay'>`]]*

`<time>` = número de segundos

translate(x, y) – função de transformação

Função para aplicar uma translação 2D.

Valor inicial	0, 0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `translate` é: `<medida CSS>` [, `<medida CSS>`]

translateX(x) – função de transformação

Função para aplicar uma translação horizontal 2D.

Valor inicial	0, 0
Aplica-se	a todos os elementos

Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `translateX` é: <medida CSS>

translateY(y) – função de transformação

Função para aplicar uma translação vertical 2D.

Valor inicial	0, 0
Aplica-se	a todos os elementos
Herdada	não
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS3

Valores

O valor possível para `translateY` é: <medida CSS>

vertical-align

Essa propriedade define o alinhamento vertical para elementos inline inseridos em blocos de conteúdos inline.

Valor inicial	não definido
Aplica-se	a elementos inline e table-cell
Herdada	não
Referência para porcentagem	line-height do elemento
Mídia	visual

Criada na versão	CSS1
-------------------------	------

Valores

Os valores possíveis para **vertical-align** são: `auto` | `use-script` | `baseline` | `sub` | `super` | `top` | `text-top` | `central` | `middle` | `bottom` | `text-bottom` | `<percentage>` | `<length>` | `white-space`

Essa propriedade define o comportamento para espaços em branco nos textos.

Valor inicial	não definido
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **white-space** são: `normal` | `pre` | `nowrap` | `pre-wrap` | `pre-line` | `width`

Essa propriedade se destina a definir a largura de uma área do box, de acordo com o Box Model.

Valor inicial	auto
Aplica-se	a todos os elementos, exceto elementos inline “non-replaced”, colunas e grupos de colunas e tabelas
Herdada	largura do bloco de conteúdo
Referência para porcentagem	—
Mídia	visual
Criada na versão	CSS1

Elementos inline “non-replaced” são os elementos inline que não possuem dimensões intrínsecas. Ao contrário, elementos inline “replaced” são aqueles que possuem dimensões intrínsecas, tais como os elementos `img` e elementos para

marcar controles de formulários.

Valores

Os valores possíveis para **width** são: `<length>` | `<percentage>` | **auto** **word-spacing**

Essa propriedade se destina a definir as distâncias mínima, máxima e ótima entre as palavras de um texto.

Valor inicial	normal
Aplica-se	a todos os elementos
Herdada	sim
Referência para porcentagem	distância padrão entre as palavras
Mídia	visual
Criada na versão	CSS1

Valores

Os valores possíveis para **letter-spacing** são: `<spacing-limit>{1,3}`

`<spacing-limit>` = **normal** | `<length>` | `<percentage>`

APÊNDICE B

Elementos da HTML5

Essa tabela foi extraída do site do W3C e relaciona os elementos da HTML5. Fornece ainda uma breve descrição do elemento, além de relacionar os atributos a eles aplicáveis.

Elemento	Descrição	Atributos
a	Hyperlink	globais; href; target; rel; media; hreflang; type
abbr	Abreviação	globais
address	Informações de contato para uma página ou seção	globais
area	Área em um mapa de imagem, com ou sem hyperlink	globais; alt; coords; shape; href; target; rel; media; hreflang; type
article	Conteúdo passível de sindicalização (distribuição independente) ou conteúdo reusável	globais
aside	Conteúdo tangencialmente relacionado a outro conteúdo	globais
audio	Player de áudio	globais; src; preload; autoplay; loop; controls
b	Marca palavras-chave	globais
base	URL base para hyperlinks	globais; href; target
bdi	Marca de direção de texto	globais
bdo	Formata direção de texto	globais
blockquote	Citação longa	globais; cite
body	Corpo do documento	globais; onafterprint; onbeforeprint; onbeforeunload; onblur; onerror; onfocus; onhashchange; onload; onmessage; onoffline; ononline; onpagehide; onpageshow; onpopstate; onredo; onresize; onscroll; onstorage; onundo; onunload
br	Quebra de linha	globais

Elemento	Descrição	Atributos
button	Controle tipo botão	globais; autofocus; disabled; form; formaction; formenctype; formmethod; formnovalidate; formtarget; name; type; value
canvas	Área de bitmap para scripts	globais; width; height
caption	Legenda de tabela	globais
cite	Título de um trabalho	globais
code	Código máquina	globais
col	Coluna de tabela	globais; span
colgroup	Grupamento de colunas de uma tabela	globais; span
command	Comando de menu	globais; type; label; icon; disabled; checked; radiogroup
datalist	Container para opções de um controle tipo combo box	globais
dd	Conteúdo para correspondente(s) elemento(s) dt	globais
del	Conteúdo removido do documento	globais; cite; datetime
details	Controle para apresentar detalhes	globais; open
dfn	Instância de definição	globais
div	Container genérico	globais
dl	Container para listas de definição	globais
dt	Legenda de conteúdos em elemento(s) dd	globais
em	Forte ênfase	globais
embed	Plugin	globais; src; type; width; height; any*
fieldset	Grupamento de controles	globais; disabled; form; name

Elemento	Descrição	Atributos
	de formulário	
figcaption	Legenda de figura	globais
figure	Figura com legenda opcional	globais
footer	Rodapé para uma página ou seção	globais
form	Formulário	globais; accept-charset; action; autocomplete; enctype; method; name; novalidate; target
h1, h2, h3, h4, h5, h6	Cabeçalho de uma seção	globais
head	Container para metadados do documento	globais
header	Conteúdo introdutório ou de auxílio de navegação para uma página ou seção	globais
hgroup	Grupamento de cabeçalhos	globais
hr	Quebra temática de linha	globais
html	Elemento raiz	globais; manifest
i	Alteração de entonação	globais
iframe	Contexto de navegador aninhado	globais; src; srcdoc; name; sandbox; seamless; width; height
img	Imagem	globais; alt; src; usemap; ismap; width; height
input	Controle de formulário	globais; accept; alt; autocomplete; autofocus; checked; dirname; disabled; form; formaction; formenctype; formmethod; formnovalidate; formtarget; height; list; max; maxlength; min; multiple; name; pattern; placeholder; readonly; required; size; src; step; type; value; width
ins	Acréscimo no documento	globais; cite; datetime
kbd	Entrada de usuário	globais
keygen	Controle de formulário gerador de chave	globais; autofocus; challenge; disabled; form; keytype; name

Elemento	Descrição	Atributos
	criptográfica	
label	Legenda para controle de formulário	globais; form; for
legend	Legenda para fieldset	globais
li	Item de uma lista	globais; value*
link	Metadado link	globais; href; rel; media; hreflang; type; sizes
map	Mapa de imagem	globais; name
mark	Destaque	globais
menu	Menu de comandos	globais; type; label
meta	Metadado texto	globais; name; http-equiv; content; charset
meter	Medidor	globais; value; min; max; low; high; optimum; form
nav	Seção contendo links de navegação	globais
noscript	Conteúdo alternativo para script	globais
object	Imagem, contexto de navegador aninhado ou plugin	globais; data; type; name; usemap; form; width; height
ol	Lista ordenada	globais; reversed; start
optgroup	Grupamento de opções em um controle list box	globais; disabled; label
option	Opção em um controle list box ou combo box	globais; disabled; label; selected; value
output	Container para um valor calculado	globais; for; form; name
p	Parágrafo	globais
param	Parâmetro para o elemento object	globais; name; value
pre	Bloco de texto pré-formatado	globais

Elemento	Descrição	Atributos
progress	Barra de progresso	globais; value; max; form
q	Citação curta	globais; cite
rp	Parênteses para anotação ruby em texto	globais
rt	Anotação ruby em texto	globais
ruby	Anotação ruby	globais
s	Texto incorreto	globais
samp	Saída genérica de computador	globais
script	Script incorporado no documento	globais; src; async; defer; type; charset
section	Documento ou seção genérica de uma aplicação.	globais
select	Controle do tipo list box	globais; autofocus; disabled; form; multiple; name; required; size
small	Comentário à parte	globais
source	Mídia source para vídeo ou áudio	globais; src; type; media
span	Container genérico	globais
strong	Importância	globais
style	Informe sobre estilização incorporada no documento	globais; media; type; scoped
sub	Subescrito	globais
summary	Legenda para o elemento details	globais
sup	Superescrito	globais
table	Tabela	globais; summary
tbody	Grupo de linhas em uma tabela	globais

Elemento	Descrição	Atributos
td	Célula de tabela	globais; colspan; rowspan; headers
textarea	Campo de texto multilinhas	globais; autofocus; cols; disabled; form; maxlength; name; placeholder; readonly; required; rows; wrap
tfoot	Grupo de linhas de rodapé de uma tabela	globais
th	Célula de cabeçalho de uma tabela	globais; colspan; rowspan; headers; scope
thead	Grupo de células de cabeçalho de uma tabela	globais
time	Data e/ou data-hora	globais; datetime; pubdate
title	Título do documento	globais
tr	Linha de tabela	globais
track	Trilha de texto	globais; default; kind; label; src; srclang
u	Texto sublinhado	globais
ul	Container para lista	globais
var	Variável	globais
video	Player de vídeo	globais; src; poster; preload; autoplay; loop; controls; width; height
wbr	Quebra de linha	globais

APÊNDICE C

Atributos da HTML5

Essa tabela foi extraída do site do W3C e relaciona os atributos da HTML5. Fornece ainda uma relação dos elementos a que se aplicam, além de uma breve descrição do atributo.

Atributo	Elemento(s)	Descrição
accept	input	Dica para o tipo de arquivo esperado por um controle tipo upload.
accept-charset	form	Codificação de caracteres usados nos dados enviados por um formulário.
accesskey	elementos HTML	Atalho de teclado para focar um elemento.
action	form	URL para envio e processamento dos dados de um formulário.
alt	area; img; input	Teste alternativo para descrição de imagens, quando não disponíveis.
async	script	Execução assíncrona de script.
autocomplete	form; input	Controla o autocompletamento em controles de formulário.
autofocus	button; input; keygen; select; textarea	Põe em foco automaticamente, quando a página carrega.
autoplay	audio; video	Controla o início da execução de mídia, quando a página é carregada.
challenge	keygen	String a ser anexada ao pacote gerado para uma chave pública.
charset	meta	Declaração de codificação de caracteres.
charset	script	Declaração de codificação de caracteres de um script.
checked	command; input	Marcar um comando ou controle.
cite	blockquote; del; ins; q	Link para a fonte da citação ou as informações adicionais sobre a citação.
class	elementos HTML	Classe à qual o elemento pertence.

Atributo	Elemento(s)	Descrição
cols	textarea	Número máximo de caracteres por linha.
colspan	td; th	Número de colunas em uma célula.
content	meta	Valor do elemento.
contenteditable	elementos HTML	Define conteúdo editável de um elemento.
contextmenu	elementos HTML	Menu de contexto.
controls	audio; video	Controle de mídia nativo do agente de usuário.
coords	area	Coordenadas de uma forma de um mapa de imagens.
data	object	Endereço do recurso.
datetime	del; ins	Data e, opcionalmente, hora de uma modificação.
datetime	time	Valor do elemento.
default	track	Usa track default se não houver um disponível.
defer	script	Adia a execução do script.
dir	elementos HTML	Direção do texto de um elemento.
dirname	input; textarea	Define a direção do texto de um controle de formulário para ser enviada quando o formulário é submetido ao processamento.
disabled	button; command; fieldset; input; keygen; optgroup; option; select; textarea	Define um controle de formulário desabilitado.
draggable	elementos HTML	Define a condição de arraste de um elemento.
dropzone	elementos HTML	Tipo de itens aceitos para drag-and-drop.
enctype	form	Tipo de codificação de dados a ser usada no envio de um formulário para processamento.
for	label	Associa um rótulo a um controle de formulário.
for	output	Especifica o controle usado para o cálculo do

Atributo	Elemento(s)	Descrição
		valor de saída.
form	button; fieldset; input; keygen; label; meter; object; output; progress; select; textarea	Associa um controle a um formulário.
formaction	button; input	URL para envio do formulário para processamento.
formenctype	button; input	Tipo de codificação de dados a ser usada no envio de um formulário para processamento.
formmethod	button; input	Método HTTP usado no envio do formulário para processamento.
formnovalidate	button; input	Pula a etapa de validação do formulário.
formtarget	button; input	Contexto do navegador para envio do formulário.
headers	td; th	Células de cabeçalho para uma célula.
height	canvas; embed; iframe; img; input; object; video	Dimensão vertical – altura.
hidden	elementos HTML	Define a relevância do elemento.
high	meter	Valor alto em uma faixa de valores.
href	a; area	Endereço de um hyperlink.
href	link	Endereço de um hyperlink.
href	base	URL base do documento.
hreflang	a; area; link	Idioma do destino do link.
http-equiv	meta	Diretiva pragma.
icon	command	Ícone para um comando.
id	elementos HTML	ID de um elemento.
ismap	img	É um mapa de imagem no lado do servidor.
keytype	keygen	Tipo de chave criptográfica a gerar.
kind	track	Tipo da trilha.

Atributo	Elemento(s)	Descrição
label	command; menu; optgroup; option; track	Rótulo.
lang	elementos HTML	Idioma do conteúdo do elemento.
list	input	Lista de autocompletamento.
loop	audio; video	Define mídia executada continuamente.
low	meter	Valor baixo em uma faixa de valores.
manifest	html	Manifesto para cache local.
max	input	Valor máximo.
max	meter; progress	Limite superior de uma faixa de valores.
maxlength	input; textarea	Maior comprimento de um valor.
media	a; area; link; source; style	Define a mídia aplicável.
method	form	Método HTTP para envio de formulário.
min	input	Valor mínimo.
min	meter	Limite inferior de uma faixa de valores.
multiple	input; select	Define aceitação de múltiplos valores.
name	button; fieldset; input; keygen; output; select; textarea	Nome de controles de formulário a usar em form.elements API.
name	form	Nome de formulário a usar em document.forms API.
name	iframe; object	Nome de contextos de navegador aninhados.
name	map	Nome de um mapa de imagem referenciado com o atributo usemap.
name	meta	Nome de metadado.
name	param	Nome de parâmetro.
novalidate	form	Pula a etapa de validação de um controle de formulário.
open	details	Define a visibilidade do elemento details.

Atributo	Elemento(s)	Descrição
optimum	meter	Valor ótimo em um widget de medição.
pattern	input	Expressão regular a casar com um valor de um controle de formulário.
placeholder	input; textarea	Rótulo a ser inserido em um controle de formulário.
poster	video	Frame inicial mostrado no início de um vídeo.
preload	audio; video	Indicação da quantidade de buffer necessária para uma mídia.
pubdate	time	Data da publicação de um artigo ou de uma página.
radiogroup	command	Nome do grupo de comandos relacionado a um grupo de botões radio.
readonly	input; textarea	Define que o valor é somente para leitura e não pode ser editado pelo usuário.
rel	a; area; link	Relacionamento existente entre o documento que contém o link e o alvo do link.
required	input; select; textarea	Define um controle de formulário de preenchimento obrigatório.
reversed	ol	Numeração dos itens de listas em ordem inversa.
rows	textarea	Número de linhas visíveis.
rowspan	td; th	Número de linhas contidas em uma célula.
sandbox	iframe	Regras de segurança para conteúdos aninhados.
spellcheck	elementos HTML	Define correção gramatical para o conteúdo de um elemento.
scope	th	Define a quais células uma célula cabeçalho se refere.
scoped	style	Define o escopo de aplicação de estilos.
seamless	iframe	Define aplicação de estilo em conteúdo aninhado no documento.

Atributo	Elemento(s)	Descrição
selected	option	Define seleção padrão para uma opção.
shape	area	O tipo de forma de um mapa de imagem.
size	input; select	Tamanho do controle de formulário.
sizes	link	Tamanho dos ícones (para rel = “icon”).
span	col; colgroup	Número de colunas no elemento.
src	audio; embed; iframe; img; input; script; source; track; video	Endereço do alvo.
srcdoc	iframe	Documento a ser renderizado em um iframe.
srclang	track	Idioma da trilha da mídia.
start	ol	Valor ordinal do primeiro item da lista.
step	input	Intervalo granular da medida de um controle.
style	elementos HTML	Instruções de apresentação e formatação.
summary	table	Texto explicativo da estrutura de uma tabela complexa.
tabindex	elementos HTML	Ordem sequencial de foco no elemento.
target	a; area	Contexto de navegador para destino de hyperlink.
target	base	Contexto de navegador padrão para destino de hyperlinks e envio de formulário.
target	form	Contexto de navegador padrão para destino de envio de formulário.
title	elementos HTML	Informação complementar sobre o elemento.
title	abbr; dfn	Texto completo de uma abreviatura.
title	command	Dica descritiva de um comando.
title	link	Título de um link.
title	link; style	Nome de uma folha de estilo alternativa.
type	a; area; link	Dica para o tipo do alvo referenciado.

Atributo	Elemento(s)	Descrição
type	button	Controle tipo botão.
type	button; input	Tipo do controle de formulário.
type	command	Tipo de comando.
type	embed; object; script; source; style	Tipo do recurso incorporado.
type	menu	Tipo de menu.
usemap	img; object	Nome do mapa de imagem a usar.
value	button; option	Valor a ser usado no envio do formulário.
value	input	Valor do controle de formulário.
value	li	Valor ordinal de um item de lista.
value	meter; progress	Valor corrente do controle.
value	param	Valor do parâmetro.
width	canvas; embed; iframe; img; input; object; video	Dimensão horizontal – largura.
wrap	textarea	Modo de quebra de linhas para o valor do controle.

APÊNDICE D

Cores CSS

Neste apêndice, apresentamos uma tabela das cores CSS3 que podem ser definidas com uso de palavras-chave e seus respectivos códigos hexadecimal e RGB.

Cor – Palavra-chave	HEX	RGB
aliceblue	#f0f8ff	240, 248, 255
antiquewhite	#faebd7	250, 235, 215
aqua	#00ffff	0, 255, 255
aquamarine	#7fffd4	127, 255, 212
azure	#f0ffff	240, 255, 255
beige	#f5f5dc	245, 245, 220
bisque	#ffe4c4	255, 228, 196
black	#000000	0, 0, 0
blanchedalmond	#ffeacd	255, 235, 205
blue	#0000ff	0, 0, 255
blueviolet	#8a2be2	138, 43, 226
brown	#a52a2a	165, 42, 42
burlywood	#deb887	222, 184, 135
cadetblue	#5f9ea0	95, 158, 160
chartreuse	#7fff00	127, 255, 0
chocolate	#d2691e	210, 105, 30
coral	#ff7f50	255, 127, 80
cornflowerblue	#6495ed	100, 149, 237
cornsilk	#fff8dc	255, 248, 220
crimson	#dc143c	220, 20, 60
cyan	#00ffff	0, 255, 255

Cor – Palavra-chave	HEX	RGB
darkblue	#00008b	0, 0, 139
darkcyan	#008b8b	0, 139, 139
darkgoldenrod	#b8860b	184, 134, 11
darkgray	#a9a9a9	169, 169, 169
darkgreen	#006400	0, 100, 0
darkgrey	#a9a9a9	169, 169, 169
darkkhaki	#bdb76b	189, 183, 107
darkmagenta	#8b008b	139, 0, 139
darkolivegreen	#556b2f	85, 107, 47
darkorange	#ff8c00	255, 140, 0
darkorchid	#9932cc	153, 50, 204
darkred	#8b0000	139, 0, 0
darksalmon	#e9967a	233, 150, 122
darkseagreen	#8fbc8f	143, 188, 143
darkslateblue	#483d8b	72, 61, 139
darkslategray	#2f4f4f	47, 79, 79
darkslategrey	#2f4f4f	47, 79, 79
darkturquoise	#00ced1	0, 206, 209
darkviolet	#9400d3	148, 0, 211
deeppink	#ff1493	255, 20, 147
deepskyblue	#00bfff	0, 191, 255
dimgray	#696969	105, 105, 105
dimgrey	#696969	105, 105, 105
dodgerblue	#1e90ff	30, 144, 255
firebrick	#b22222	178, 34, 34

Cor – Palavra-chave	HEX	RGB
floralwhite	#fffaf0	255,250,240
forestgreen	#228b22	34,139,34
fuchsia	#ff00ff	255,0,255
gainsboro	#dcdcdc	220,220,220
ghostwhite	#f8f8ff	248,248,255
gold	#ffd700	255,215,0
goldenrod	#daa520	218,165,32
gray	#808080	128,128,128
green	#008000	0,128,0
greenyellow	#adff2f	173,255,47
grey	#808080	128,128,128
honeydew	#f0fff0	240,255,240
hotpink	#ff69b4	255,105,180
indianred	#cd5c5c	205,92,92
indigo	#4b0082	75,0,130
ivory	#ffffff	255,255,240
khaki	#f0e68c	240,230,140
lavender	#e6e6fa	230,230,250
lavenderblush	#fff0f5	255,240,245
lawngreen	#7cfc00	124,252,0
lemonchiffon	#fffacd	255,250,205
lightblue	#add8e6	173,216,230
lightcoral	#f08080	240,128,128
lightcyan	#e0ffff	224,255,255

Cor – Palavra-chave	HEX	RGB
lightgoldenrodyellow	#fafad2	250,250,210
lightgray	#d3d3d3	211,211,211
lightgreen	#90ee90	144,238,144
lightgrey	#d3d3d3	211,211,211
lightpink	#ffb6c1	255,182,193
lightsalmon	#ffa07a	255,160,122
lightseagreen	#20b2aa	32,178,170
lightskyblue	#87cefa	135,206,250
lightslategray	#778899	119,136,153
lightslategrey	#778899	119,136,153
lightsteelblue	#b0c4de	176,196,222
lightyellow	#ffffe0	255,255,224
lime	#00ff00	0,255,0
limegreen	#32cd32	50,205,50
linen	#faf0e6	250,240,230
magenta	#ff00ff	255,0,255
maroon	#800000	128,0,0
mediumaquamarine	#66cdaa	102,205,170
mediumblue	#0000cd	0,0,205
mediumorchid	#ba55d3	186,85,211
mediumpurple	#9370db	147,112,219
mediumseagreen	#3cb371	60,179,113
mediumslateblue	#7b68ee	123,104,238
mediumspringgreen	#00fa9a	0,250,154
mediumturquoise	#48d1cc	72,209,204

Cor – Palavra-chave	HEX	RGB
mediumvioletred	#c71585	199,21,133
midnightblue	#191970	25,25,112
mintcream	#f5fffa	245,255,250
mistyrose	#ffe4e1	255,228,225
moccasin	#ffe4b5	255,228,181
navajowhite	#ffdead	255,222,173
navy	#000080	0,0,128
oldlace	#fdf5e6	253,245,230
olive	#808000	128,128,0
olivedrab	#6b8e23	107,142,35
orange	#ffa500	255,165,0
orangered	#ff4500	255,69,0
orchid	#da70d6	218,112,214
palegoldenrod	#eee8aa	238,232,170
palegreen	#98fb98	152,251,152
paleturquoise	#afeeee	175,238,238
palevioletred	#db7093	219,112,147
papayawhip	#ffefd5	255,239,213
peachpuff	#ffdab9	255,218,185
peru	#cd853f	205,133,63
pink	#ffc0cb	255,192,203
plum	#dda0dd	221,160,221
powderblue	#b0e0e6	176,224,230
purple	#800080	128,0,128

Cor – Palavra-chave	HEX	RGB
red	#ff0000	255,0,0
rosybrown	#bc8f8f	188,143,143
royalblue	#4169e1	65,105,225
saddlebrown	#8b4513	139,69,19
salmon	#fa8072	250,128,114
sandybrown	#f4a460	244,164,96
seagreen	#2e8b57	46,139,87
seashell	#fff5ee	255,245,238
sienna	#a0522d	160,82,45
silver	#c0c0c0	192,192,192
skyblue	#87ceeb	135,206,235
slateblue	#6a5acd	106,90,205
slategray	#708090	112,128,144
slategrey	#708090	112,128,144
snow	#fffafa	255,250,250
springgreen	#00ff7f	0,255,127
steelblue	#4682b4	70,130,180
tan	#d2b48c	210,180,140
teal	#008080	0,128,128
thistle	#d8bfd8	216,191,216
tomato	#ff6347	255,99,71
turquoise	#40e0d0	64,224,208
violet	#ee82ee	238,130,238
wheat	#f5deb3	245,222,179
white	#ffffff	255,255,255

Cor – Palavra-chave	HEX	RGB
whitesmoke	#f5f5f5	245, 245, 245
yellow	#ffff00	255, 255, 0
yellowgreen	#9acd32	154, 205, 50

Referências bibliográficas

GASTON, Peter. *The book of CSS3: A developer guide to the future of web design*. 1 ed. No Starch Press, 2011.

GOLDSTEIN, Alexis; LAZARIS Louis; WEYL Estelle. *HTML5 & CSS3 for the real world*. Sitepoint, 2011.

KEITH, Jeremy. *DOM Scripting: Web Design with JavaScript and the Document Object Model*. Friends of Ed, 2005.

RESIG, John. *Pro Javascript Techniques*. Apress, 2006.

W3C. Disponível em <http://www.w3.org>.

Mozilla Developer Network. Disponível em <https://developer.mozilla.org/en-US/>.